

Credit rating Final Project (Kamila Makhambetova)

April 10, 2025

1 Final Project

1. Data preparation + Data Discovery

```
[217]: import pandas as pd

# Read the CSV file into a DataFrame
credit_rating = pd.read_csv('/Users/kamila/Desktop/Fidelity final project/
→corporate credit rating/corporate_rating.csv')

credit_rating
```

```
[217]:
```

	Rating	Name	Symbol	\
0	A	Whirlpool Corporation	WHR	
1	BBB	Whirlpool Corporation	WHR	
2	BBB	Whirlpool Corporation	WHR	
3	BBB	Whirlpool Corporation	WHR	
4	BBB	Whirlpool Corporation	WHR	
...	...	...	...	
2024	BBB	NVR, Inc.	NVR	
2025	BB	Kaiser Aluminum Corporation	KALU	
2026	B	Cresud S.A.C.I.F. y A.	CRESY	
2027	B	Cresud S.A.C.I.F. y A.	CRESY	
2028	CCC	Cresud S.A.C.I.F. y A.	CRESY	

	Rating Agency	Name	Date	Sector	\
0	Egan-Jones Ratings Company		11/27/2015	Consumer Durables	
1	Egan-Jones Ratings Company		2/13/2014	Consumer Durables	
2	Fitch Ratings		3/6/2015	Consumer Durables	
3	Fitch Ratings		6/15/2012	Consumer Durables	
4	Standard & Poor's Ratings Services		10/24/2016	Consumer Durables	
...	...	...	...	...	
2024	Moody's Investors Service		9/5/2012	Capital Goods	
2025	Standard & Poor's Ratings Services		4/28/2016	Capital Goods	
2026	Fitch Ratings		11/30/2012	Finance	
2027	Fitch Ratings		6/15/2012	Finance	

2028 Fitch Ratings 8/1/2014 Finance

	currentRatio	quickRatio	cashRatio	daysOfSalesOutstanding	...	\
0	0.945894	0.426395	0.099690	44.203245	...	
1	1.033559	0.498234	0.203120	38.991156	...	
2	0.963703	0.451505	0.122099	50.841385	...	
3	1.019851	0.510402	0.176116	41.161738	...	
4	0.957844	0.495432	0.141608	47.761126	...	
...	...	...	...	...	...	
2024	11.757767	7.115059	7.057420	1.079762	...	
2025	2.962788	1.294743	0.428234	30.602414	...	
2026	0.883875	0.842553	0.233830	147.599371	...	
2027	0.911713	0.748356	0.310640	131.644566	...	
2028	1.085007	1.026375	0.203490	151.660513	...	

	effectiveTaxRate	freeCashFlowOperatingCashFlowRatio	\
0	0.202716	0.437551	
1	0.074155	0.541997	
2	0.214529	0.513185	
3	1.816667	-0.147170	
4	0.166966	0.451372	
...	...	...	
2024	0.343500	0.953231	
2025	0.363636	0.602645	
2026	35.017544	0.926665	
2027	0.331525	0.692804	
2028	0.266987	1.101462	

	freeCashFlowPerShare	cashPerShare	companyEquityMultiplier	\
0	6.810673	9.809403	4.008012	
1	8.625473	17.402270	3.156783	
2	9.693487	13.103448	4.094575	
3	-1.015625	14.440104	3.630950	
4	7.135348	14.257556	4.012780	
...	...	...	...	
2024	50.293155	230.194971	1.759461	
2025	5.563630	5.958956	1.614282	
2026	1.459801	1.056480	4.034952	
2027	0.744377	1.470201	3.825856	
2028	2.487817	6.109814	3.939161	

	ebitPerRevenue	enterpriseValueMultiple	operatingCashFlowPerShare	\
0	0.049351	7.057088	15.565438	
1	0.048857	6.460618	15.914250	
2	0.044334	10.491970	18.888889	
3	-0.012858	4.080741	6.901042	
4	0.053770	8.293505	15.808147	

...	...	...	...
2024	0.086376	15.867701	52.760726
2025	-0.267117	-4.729967	9.232021
2026	0.265092	9.358311	1.575328
2027	0.130692	22.440102	1.074441
2028	0.302997	9.604061	2.258650

	operatingCashFlowSalesRatio	payablesTurnover
0	0.058638	3.906655
1	0.067239	4.002846
2	0.074426	3.483510
3	0.028394	4.581150
4	0.058065	3.857790
...	...	...
2024	0.083018	15.758348
2025	0.114089	14.542373
2026	0.283634	2.300168
2027	0.217783	1.997608
2028	0.252606	1.865682

[2029 rows x 31 columns]

```
[218]: credit_rating[credit_rating['Date']=='8/16/2005']
```

```
[218]:
```

	Rating	Name	Symbol	Rating Agency	Name	Date	Sector	\
178	AA	Total S.A.	TOT		DBRS	8/16/2005	Energy	

	currentRatio	quickRatio	cashRatio	daysOfSalesOutstanding	...	\
178	0.882592	0.556147	0.144622		36.783384	...

	effectiveTaxRate	freeCashFlowOperatingCashFlowRatio	\
178	0.0		0.545713

	freeCashFlowPerShare	cashPerShare	companyEquityMultiplier	\
178	17.009489	10.417039		1.094559

	ebitPerRevenue	enterpriseValueMultiple	operatingCashFlowPerShare	\
178	0.18998	1.264522		31.169316

	operatingCashFlowSalesRatio	payablesTurnover
178	0.15227	0.0

[1 rows x 31 columns]

```
[219]: #Non Null entries in each column

credit_rating.count()
```

```
[219]: Rating                2029
      Name                  2029
      Symbol                2029
      Rating Agency Name    2029
      Date                  2029
      Sector                2029
      currentRatio          2029
      quickRatio            2029
      cashRatio             2029
      daysOfSalesOutstanding 2029
      netProfitMargin        2029
      pretaxProfitMargin     2029
      grossProfitMargin      2029
      operatingProfitMargin   2029
      returnOnAssets          2029
      returnOnCapitalEmployed 2029
      returnOnEquity          2029
      assetTurnover           2029
      fixedAssetTurnover      2029
      debtEquityRatio         2029
      debtRatio              2029
      effectiveTaxRate        2029
      freeCashFlowOperatingCashFlowRatio 2029
      freeCashFlowPerShare    2029
      cashPerShare            2029
      companyEquityMultiplier 2029
      ebitPerRevenue          2029
      enterpriseValueMultiple 2029
      operatingCashFlowPerShare 2029
      operatingCashFlowSalesRatio 2029
      payablesTurnover        2029
      dtype: int64
```

```
[220]: #Null entries in each column
```

```
credit_rating.isnull().sum()
```

```
[220]: Rating                0
      Name                  0
      Symbol                0
      Rating Agency Name    0
      Date                  0
      Sector                0
      currentRatio          0
      quickRatio            0
      cashRatio             0
      daysOfSalesOutstanding 0
```

```

netProfitMargin          0
pretaxProfitMargin       0
grossProfitMargin        0
operatingProfitMargin    0
returnOnAssets           0
returnOnCapitalEmployed  0
returnOnEquity           0
assetTurnover            0
fixedAssetTurnover       0
debtEquityRatio          0
debtRatio                0
effectiveTaxRate         0
freeCashFlowOperatingCashFlowRatio  0
freeCashFlowPerShare     0
cashPerShare             0
companyEquityMultiplier  0
ebitPerRevenue           0
enterpriseValueMultiple  0
operatingCashFlowPerShare 0
operatingCashFlowSalesRatio 0
payablesTurnover         0
dtype: int64

```

```
[221]: credit_rating.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 2029 entries, 0 to 2028
```

```
Data columns (total 31 columns):
```

#	Column	Non-Null Count	Dtype
0	Rating	2029 non-null	object
1	Name	2029 non-null	object
2	Symbol	2029 non-null	object
3	Rating Agency Name	2029 non-null	object
4	Date	2029 non-null	object
5	Sector	2029 non-null	object
6	currentRatio	2029 non-null	float64
7	quickRatio	2029 non-null	float64
8	cashRatio	2029 non-null	float64
9	daysOfSalesOutstanding	2029 non-null	float64
10	netProfitMargin	2029 non-null	float64
11	pretaxProfitMargin	2029 non-null	float64
12	grossProfitMargin	2029 non-null	float64
13	operatingProfitMargin	2029 non-null	float64
14	returnOnAssets	2029 non-null	float64
15	returnOnCapitalEmployed	2029 non-null	float64
16	returnOnEquity	2029 non-null	float64

```

17  assetTurnover          2029 non-null    float64
18  fixedAssetTurnover     2029 non-null    float64
19  debtEquityRatio        2029 non-null    float64
20  debtRatio              2029 non-null    float64
21  effectiveTaxRate       2029 non-null    float64
22  freeCashFlowOperatingCashFlowRatio  2029 non-null    float64
23  freeCashFlowPerShare   2029 non-null    float64
24  cashPerShare           2029 non-null    float64
25  companyEquityMultiplier 2029 non-null    float64
26  ebitPerRevenue         2029 non-null    float64
27  enterpriseValueMultiple 2029 non-null    float64
28  operatingCashFlowPerShare 2029 non-null    float64
29  operatingCashFlowSalesRatio 2029 non-null    float64
30  payablesTurnover       2029 non-null    float64
dtypes: float64(25), object(6)
memory usage: 491.5+ KB

```

```

[222]: #convert date column to datetime format

# Convert 'date' column to datetime format
credit_rating['Date'] = pd.to_datetime(credit_rating['Date'], format='%m/%d/%Y')

# Sort the dataset by date
credit_rating = credit_rating.sort_values(by='Date')

```

```

[223]: credit_rating.info()

```

```

<class 'pandas.core.frame.DataFrame'>
Int64Index: 2029 entries, 178 to 1968
Data columns (total 31 columns):
 #   Column                                Non-Null Count  Dtype
---  -
 0   Rating                               2029 non-null   object
 1   Name                                 2029 non-null   object
 2   Symbol                              2029 non-null   object
 3   Rating Agency Name                  2029 non-null   object
 4   Date                                2029 non-null   datetime64[ns]
 5   Sector                              2029 non-null   object
 6   currentRatio                        2029 non-null   float64
 7   quickRatio                          2029 non-null   float64
 8   cashRatio                           2029 non-null   float64
 9   daysOfSalesOutstanding              2029 non-null   float64
10  netProfitMargin                     2029 non-null   float64
11  pretaxProfitMargin                  2029 non-null   float64
12  grossProfitMargin                   2029 non-null   float64
13  operatingProfitMargin                2029 non-null   float64
14  returnOnAssets                      2029 non-null   float64
15  returnOnCapitalEmployed             2029 non-null   float64

```

```

16 returnOnEquity          2029 non-null float64
17 assetTurnover           2029 non-null float64
18 fixedAssetTurnover      2029 non-null float64
19 debtEquityRatio         2029 non-null float64
20 debtRatio               2029 non-null float64
21 effectiveTaxRate        2029 non-null float64
22 freeCashFlowOperatingCashFlowRatio 2029 non-null float64
23 freeCashFlowPerShare    2029 non-null float64
24 cashPerShare            2029 non-null float64
25 companyEquityMultiplier 2029 non-null float64
26 ebitPerRevenue          2029 non-null float64
27 enterpriseValueMultiple  2029 non-null float64
28 operatingCashFlowPerShare 2029 non-null float64
29 operatingCashFlowSalesRatio 2029 non-null float64
30 payablesTurnover        2029 non-null float64

```

dtypes: datetime64[ns](1), float64(25), object(5)

memory usage: 507.2+ KB

[224]: credit_rating.head(5)

#should we delete 2005/2009? Ask Stepan

[224]:

	Rating	Name	Symbol	\
178	AA	Total S.A.	TOT	
576	BB	United States Steel Corporation	X	
496	CCC	Tenet Healthcare Corporation	THC	
1282	BBB	Magellan Midstream Partners L.P.	MMP	
229	BBB	Pitney Bowes Inc.	PBI	

	Rating	Agency Name	Date	Sector	\
178		DBRS	2005-08-16	Energy	
576	Standard & Poor's	Ratings Services	2009-04-28	Basic Industries	
496	Standard & Poor's	Ratings Services	2010-08-03	Health Care	
1282	Standard & Poor's	Ratings Services	2010-08-31	Energy	
229	Standard & Poor's	Ratings Services	2010-09-30	Miscellaneous	

	currentRatio	quickRatio	cashRatio	daysOfSalesOutstanding	...	\
178	0.882592	0.556147	0.144622	36.783384	...	
576	2.063355	0.510799	0.260619	0.000000	...	
496	1.339710	1.082319	0.234783	45.322651	...	
1282	1.398245	0.443477	0.027206	21.605923	...	
229	1.178049	0.511873	0.189680	53.293224	...	

	effectiveTaxRate	freeCashFlowOperatingCashFlowRatio	\
178	0.000000	0.545713	
576	0.283671	1.000000	
496	-6.183544	1.000000	

1282	0.004400	-0.521408
229	0.384921	0.874208

	freeCashFlowPerShare	cashPerShare	companyEquityMultiplier \
178	17.009489	10.417039	1.094559
576	6.293486	12.084515	3.286415
496	0.974560	1.494876	4.813137
1282	-2.022368	0.272046	2.529922
229	4.041128	2.500252	-87.429443

	ebitPerRevenue	enterpriseValueMultiple	operatingCashFlowPerShare \
178	0.189980	1.264522	31.169316
576	0.133847	0.965856	6.293486
496	0.017165	15.656926	0.974560
1282	0.200058	11.799222	3.878668
229	0.098535	10.455327	4.622616

	operatingCashFlowSalesRatio	payablesTurnover
178	0.152270	0.000000
576	0.032894	13.696528
496	0.051276	0.000000
1282	0.272662	0.000000
229	0.175496	0.077504

[5 rows x 31 columns]

```
[225]: earliest_date = credit_rating['Date'].min()
latest_date = credit_rating['Date'].max()

print("Earliest Date:", earliest_date)
print("Latest Date:", latest_date)

#Period from 2005-08-16 to 2016-12-23
```

Earliest Date: 2005-08-16 00:00:00

Latest Date: 2016-12-23 00:00:00

```
[226]: credit_rating.groupby('Date').size().reset_index(name='number of observations').
↳sort_values(by= 'number of observations', ascending=False)
```

```
[226]:      Date  number of observations
81  2012-06-15                223
677 2015-10-28                 8
699 2015-11-30                 8
870 2016-10-24                 8
661 2015-09-30                 7
..      ...
```


210	2013-05-20	1
619	2015-07-22	1
211	2013-05-21	1
446	2014-08-14	1
0	2005-08-16	1

[904 rows x 2 columns]

```
[227]: credit_rating[credit_rating["Date"]=="2012-06-15"]
```

```
[227]:
```

	Rating	Name	Symbol	Rating Agency	Name \
1830	BB	Semtech Corporation	SMTC	Moody's	Investors Service
954	A	V.F. Corporation	VFC	Moody's	Investors Service
1271	BBB	Crane Co.	CR	Moody's	Investors Service
1733	B	Iron Mountain Incorporated	IRM	Moody's	Investors Service
254	BB	LyondellBasell Industries NV	LYB	Moody's	Investors Service
...	...	...	...	...	...
1969	B	New York Times Company (The)	NYT	Moody's	Investors Service
1214	BBB	Black Hills Corporation	BKH	Moody's	Investors Service
1264	BB	LKQ Corporation	LKQ	Moody's	Investors Service
222	A	Pfizer, Inc.	PFE	Moody's	Investors Service
3	BBB	Whirlpool Corporation	WHR		Fitch Ratings

	Date	Sector	currentRatio	quickRatio	cashRatio \
1830	2012-06-15	Technology	3.769763	2.464759	1.268521
954	2012-06-15	Consumer Non-Durables	1.913495	0.900673	0.204815
1271	2012-06-15	Capital Goods	1.936300	1.233079	0.459747
1733	2012-06-15	Consumer Services	1.077053	3.035078	0.211824
254	2012-06-15	Basic Industries	2.278796	1.008171	0.212236
...	...	...	...	...	...
1969	2012-06-15	Consumer Services	1.458362	1.107234	0.341220
1214	2012-06-15	Public Utilities	0.864309	0.224339	0.024631
1264	2012-06-15	Consumer Services	2.891011	0.121317	0.121317
222	2012-06-15	Health Care	2.056646	1.438099	0.126082
3	2012-06-15	Consumer Durables	1.019851	0.510402	0.176116

	daysOfSalesOutstanding	...	effectiveTaxRate \
1830	0.000000	...	0.085212
954	43.226532	...	0.235546
1271	50.071842	...	-0.296372
1733	65.799336	...	0.301751
254	25.618301	...	0.328733
...	...	...	...
1969	45.299789	...	-9.818720
1214	44.979602	...	-1.960202
1264	0.000000	...	0.373787
222	73.665851	...	0.315233

3 41.161738 ... 1.816667

	freeCashFlowOperatingCashFlowRatio	freeCashFlowPerShare	cashPerShare \
1830	1.000000	1.504692	3.709652
954	1.000000	9.873782	3.472480
1271	0.768129	1.979955	5.301067
1733	0.624888	1.974463	10.439657
254	0.634019	3.202465	2.600352
...	...	...	...
1969	0.392820	0.197296	1.902283
1214	0.994926	5.583208	1.008705
1264	0.591939	0.857862	0.330174
222	0.917984	2.376871	3.423052
3	-0.147170	-1.015625	14.440104

	companyEquityMultiplier	ebitPerRevenue	enterpriseValueMultiple \
1830	1.248438	0.174547	14.025900
954	2.057699	0.123133	3.374291
1271	3.495201	0.008034	28.199479
1733	4.849736	0.117060	9.680102
254	2.156046	0.062467	5.447381
...	...	...	...
1969	5.694466	-0.001600	10.211015
1214	3.412685	-0.007308	8.078449
1264	1.946191	0.102687	7.877027
222	2.287407	0.189277	9.183443
3	3.630950	-0.012858	4.080741

	operatingCashFlowPerShare	operatingCashFlowSalesRatio	payablesTurnover
1830	1.504692	0.206382	6.284249
954	9.873782	0.114319	8.049715
1271	2.577632	0.058845	8.668677
1733	3.159706	0.204145	7.962604
254	5.051056	0.056216	17.920765
...	...	...	...
1969	0.502256	0.031818	9.215806
1214	5.611680	0.175842	5.489260
1264	1.449242	0.064765	8.905129
222	2.589229	0.300185	3.932482
3	6.901042	0.028394	4.581150

[223 rows x 31 columns]

```
[228]: credit_rating.groupby(['Name', 'Date']).size().reset_index(name='number of
↳rating agencies').sort_values(by= 'number of rating agencies',
↳ascending=False)
```

#Every company in the dataset was rated by one rating agency per specific date

```
[228]:
```

	Name	Date	number of rating agencies
0	3M Company	2012-06-15	1
1348	Oasis Petroleum Inc.	2014-02-27	1
1361	Olin Corporation	2013-12-26	1
1360	Olin Corporation	2012-08-09	1
1359	Olin Corporation	2012-06-15	1
...	...	...	...
672	Eldorado Gold Corporation	2016-03-31	1
671	Eldorado Gold Corporation	2014-09-09	1
670	Eldorado Gold Corporation	2013-08-16	1
669	El Paso Electric Company	2015-11-30	1
2028	eBay Inc.	2015-07-21	1

[2029 rows x 3 columns]

```
[229]: credit_rating['year-month'] = credit_rating['Date'].dt.strftime('%Y-%m')
```

```
[230]: credit_rating
```

```
[230]:
```

	Rating	Name	Symbol	\
178	AA	Total S.A.	TOT	
576	BB	United States Steel Corporation	X	
496	CCC	Tenet Healthcare Corporation	THC	
1282	BBB	Magellan Midstream Partners L.P.	MMP	
229	BBB	Pitney Bowes Inc.	PBI	
...	...	...	...	
1129	BBB	PLDT Inc.	PHI	
1478	BBB	CSG Systems International, Inc.	CSGS	
741	B	HD Supply Holdings, Inc.	HDS	
1622	BBB	Zoetis Inc.	ZTS	
1968	BB	Royal Caribbean Cruises Ltd.	RCL	

	Rating Agency Name	Date	Sector	\
178	DBRS	2005-08-16	Energy	
576	Standard & Poor's Ratings Services	2009-04-28	Basic Industries	
496	Standard & Poor's Ratings Services	2010-08-03	Health Care	
1282	Standard & Poor's Ratings Services	2010-08-31	Energy	
229	Standard & Poor's Ratings Services	2010-09-30	Miscellaneous	
...	...	...	...	
1129	Standard & Poor's Ratings Services	2016-12-19	Public Utilities	
1478	Standard & Poor's Ratings Services	2016-12-20	Technology	
741	Standard & Poor's Ratings Services	2016-12-21	Consumer Services	
1622	Standard & Poor's Ratings Services	2016-12-23	Health Care	
1968	Standard & Poor's Ratings Services	2016-12-23	Consumer Services	

	currentRatio	quickRatio	cashRatio	daysOfSalesOutstanding	...	\
178	0.882592	0.556147	0.144622	36.783384	...	
576	2.063355	0.510799	0.260619	0.000000	...	
496	1.339710	1.082319	0.234783	45.322651	...	
1282	1.398245	0.443477	0.027206	21.605923	...	
229	1.178049	0.511873	0.189680	53.293224	...	
...	...	...	...	...	...	
1129	0.472296	0.363831	0.212743	53.969697	...	
1478	2.221721	1.926080	0.501335	100.215058	...	
741	2.276693	1.345580	0.308840	40.264618	...	
1622	3.034915	1.468218	0.650850	68.176146	...	
1968	0.168476	0.030660	0.029855	0.000000	...	

	freeCashFlowOperatingCashFlowRatio	freeCashFlowPerShare	cashPerShare	\
178	0.545713	17.009489	10.417039	
576	1.000000	6.293486	12.084515	
496	1.000000	0.974560	1.494876	
1282	-0.521408	-2.022368	0.272046	
229	0.874208	4.041128	2.500252	
...	...	...	...	
1129	0.278240	72.041583	193.403763	
1478	0.740456	2.012917	8.928507	
741	0.796209	0.001705	0.001812	
1622	0.697055	1.002592	1.466568	
1968	0.008872	0.103164	0.629226	

	companyEquityMultiplier	ebitPerRevenue	enterpriseValueMultiple	\
178	1.094559	0.189980	1.264522	
576	3.286415	0.133847	0.965856	
496	4.813137	0.017165	15.656926	
1282	2.529922	0.200058	11.799222	
229	-87.429443	0.098535	10.455327	
...	...	...	...	
1129	4.392133	0.132608	2.652202	
1478	3.548214	0.131412	9.797003	
741	8.086022	0.032620	6438.074149	
1622	5.143914	0.251227	20.874287	
1968	2.445929	0.151051	12.241731	

	operatingCashFlowPerShare	operatingCashFlowSalesRatio	\
178	31.169316	0.152270	
576	6.293486	0.032894	
496	0.974560	0.051276	
1282	3.878668	0.272662	
229	4.622616	0.175496	
...	...	...	
1129	258.919254	0.338499	

1478	2.718484	0.110632
741	0.002142	0.057120
1622	1.438326	0.145867
1968	11.628574	0.296207

	payablesTurnover	year-month
178	0.000000	2005-08
576	13.696528	2009-04
496	0.000000	2010-08
1282	0.000000	2010-08
229	0.077504	2010-09
...	...	...
1129	0.526270	2016-12
1478	10.682471	2016-12
741	9.708661	2016-12
1622	6.286792	2016-12
1968	16.427532	2016-12

[2029 rows x 32 columns]

```
[231]: credit_rating.groupby(['Name', 'year-month']).size().reset_index(name='number_
↳of rating agencies').sort_values(by= 'number of rating agencies',
↳ascending=False)
```

#Every company in the dataset was rated by one rating agency per specific date

```
[231]:
```

	Name	year-month	number of rating agencies
0	3M Company	2012-06	1
1348	Oasis Petroleum Inc.	2014-02	1
1361	Olin Corporation	2013-12	1
1360	Olin Corporation	2012-08	1
1359	Olin Corporation	2012-06	1
...	...	...	...
672	Eldorado Gold Corporation	2016-03	1
671	Eldorado Gold Corporation	2014-09	1
670	Eldorado Gold Corporation	2013-08	1
669	El Paso Electric Company	2015-11	1
2028	eBay Inc.	2015-07	1

[2029 rows x 3 columns]

```
[232]: credit_rating[['Rating', 'Name', 'Symbol', 'Rating Agency Name', 'Sector',
↳'Date']].nunique()
```

```
[232]: Rating          10
Name          593
```

```

Symbol          593
Rating Agency Name    5
Sector           12
Date             904
dtype: int64

```

```

[233]: credit_rating_per_company = credit_rating.groupby('Name').size().
        ↪reset_index(name='rating occurrence')
credit_rating_per_company = credit_rating_per_company.sort_values(by='rating_
        ↪occurrence', ascending=False)
credit_rating_per_company

```

```

[233]:
           Name  rating occurrence
91      Bunge Limited              7
512  Tenet Healthcare Corporation      7
215      Exelon Corporation          7
422  Parker-Hannifin Corporation      7
569  Walt Disney Company (The)       7
..      ...
102  Cadence Design Systems, Inc.      1
231      Fortive Corporation          1
108  Canadian Pacific Railway Limited  1
417      PRA Group, Inc.              1
463  SBA Communications Corporation    1

```

[593 rows x 2 columns]

```

[234]: credit_rating_per_company[credit_rating_per_company['rating occurrence']==4]

```

```

[234]:
           Name  rating occurrence
265      Hess Corporation          4
264      Hershey Company (The)      4
44  Arcos Dorados Holdings Inc.      4
252  Group 1 Automotive, Inc.       4
258      Halliburton Company        4
..      ...
416      PPL Corporation            4
207      Entergy Corporation         4
220      FedEx Corporation          4
523  Titan International, Inc.       4
75      BorgWarner Inc.             4

```

[100 rows x 2 columns]

```

[235]: credit_rating_per_company[credit_rating_per_company['rating occurrence']==3]

```

```
[235]:
```

	Name	rating	occurrence
349	Markel Corporation		3
348	Marathon Petroleum Corporation		3
516	Teva Pharmaceutical Industries Limited		3
406	Orange		3
351	Martin Marietta Materials, Inc.		3
..	...		
8	AMC Networks Inc.		3
300	JetBlue Airways Corporation		3
317	Kronos Worldwide Inc		3
297	J.B. Hunt Transport Services, Inc.		3
318	L Brands, Inc.		3

[110 rows x 2 columns]

```
[236]: print("Average Rating Occurrence during [2005-08-16, 2016-12-23] period :",
          credit_rating_per_company['rating occurrence'].mean())

#No time series analysis can be made. We can't investigate time dependency, as
↳ not enough data.
#Also raings per company are very inconsistent.
```

Average Rating Occurrence during [2005-08-16, 2016-12-23] period :
3.421585160202361

```
[237]: credit_rating2 = credit_rating.copy()

# Label encode 'rating'

rating_mapping = {'AAA': 9, 'AA': 8, 'A': 7, 'BBB': 6, 'BB': 5, 'B': 4,
                  'CCC': 3, 'CC': 2, 'C': 1, 'D': 0}

# Apply encoding
credit_rating2['Rating_encoded'] = credit_rating2['Rating'].map(rating_mapping)

# Print encoding
print("Rating Encoding:", rating_mapping)
```

Rating Encoding: {'AAA': 9, 'AA': 8, 'A': 7, 'BBB': 6, 'BB': 5, 'B': 4, 'CCC': 3, 'CC': 2, 'C': 1, 'D': 0}

```
[238]: credit_rating2[['Name', 'Rating_encoded', 'Rating']].head(10)
```

```
[238]:
```

	Name	Rating_encoded	Rating
178	Total S.A.	8	AA
576	United States Steel Corporation	5	BB
496	Tenet Healthcare Corporation	3	CCC

1282	Magellan Midstream Partners L.P.	6	BBB
229	Pitney Bowes Inc.	6	BBB
398	Jabil Inc.	5	BB
1691	Tutor Perini Corporation	5	BB
2019	IRSA Inversiones Y Representaciones S.A.	4	B
536	Carpenter Technology Corporation	6	BBB
820	Biogen Inc.	6	BBB

```
[239]: import matplotlib.pyplot as plt
import seaborn as sns

def plot_company_rating(data, company, ycol):

    # Filter for 'Bunge Limited'
    data2 = data[data['Name'] == company]

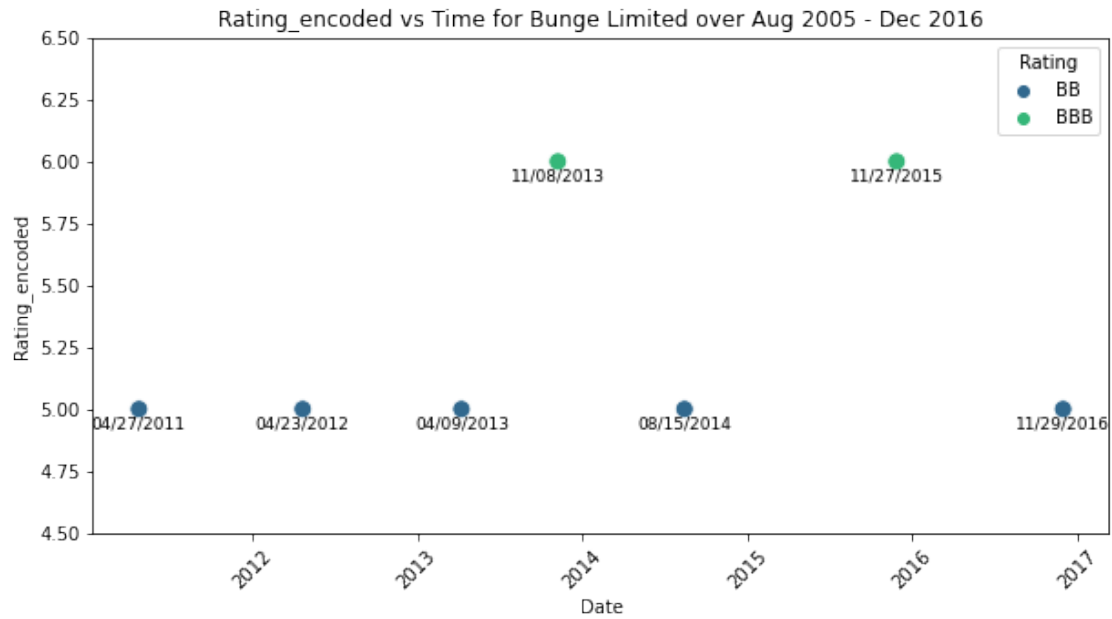
    # Plot
    plt.figure(figsize=(10, 5))
    sns.scatterplot(data=data2, x='Date', y=ycol, hue='Rating',
    ↪palette='viridis', s=100)

    # Annotate each point with the date
    for i in range(len(data2)):
        plt.annotate(data2['Date'].iloc[i].strftime('%m/%d/%Y'),
                     (data2['Date'].iloc[i], data2[ycol].iloc[i]),
                     textcoords="offset points", xytext=(0,-10), ha='center',
    ↪fontsize=9)

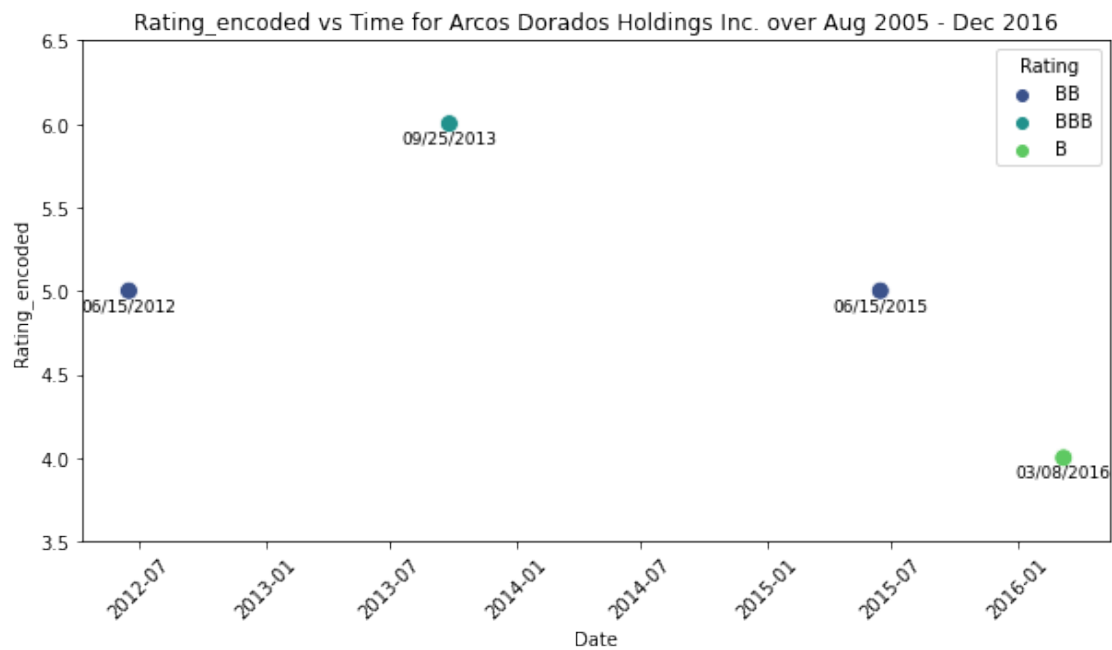
    if(data2['Rating_encoded'].min() != 0):
        plt.ylim(data2['Rating_encoded'].min() - 0.5, data2['Rating_encoded'].
    ↪max() + 0.5)
    else:
        plt.ylim(data2['Rating_encoded'].min(), data2['Rating_encoded'].max() +
    ↪0.5)

    plt.xlabel("Date")
    plt.ylabel(ycol)
    plt.title(ycol+ " vs Time for " + company + " over Aug 2005 - Dec 2016")
    plt.xticks(rotation=45)
    plt.legend(title="Rating")
    plt.show()
```

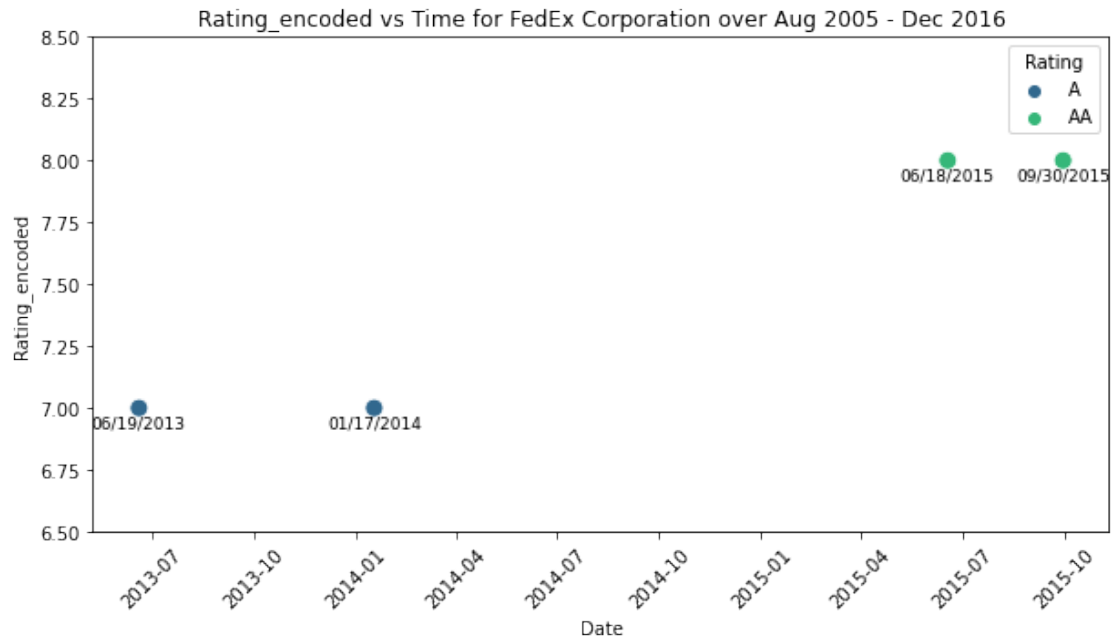
```
[240]: plot_company_rating(credit_rating2, 'Bunge Limited', 'Rating_encoded')
```

```
[241]: plot_company_rating(credit_rating2, 'Arcos Dorados Holdings Inc.',
↪ 'Rating_encoded')
```

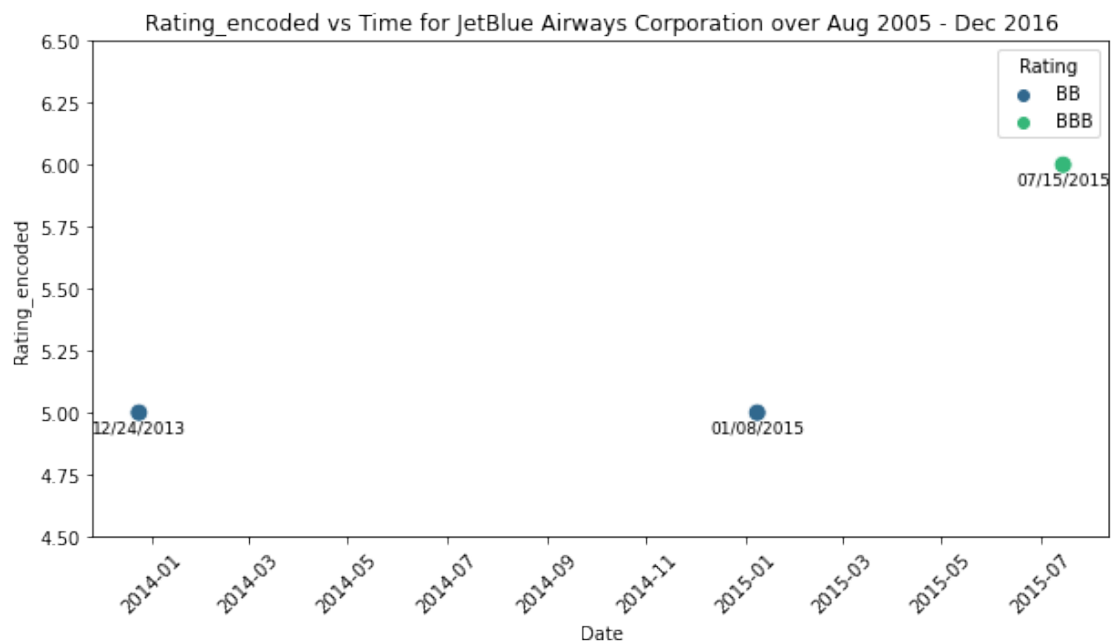


```
[242]: plot_company_rating(credit_rating2, 'FedEx Corporation', 'Rating_encoded')
```

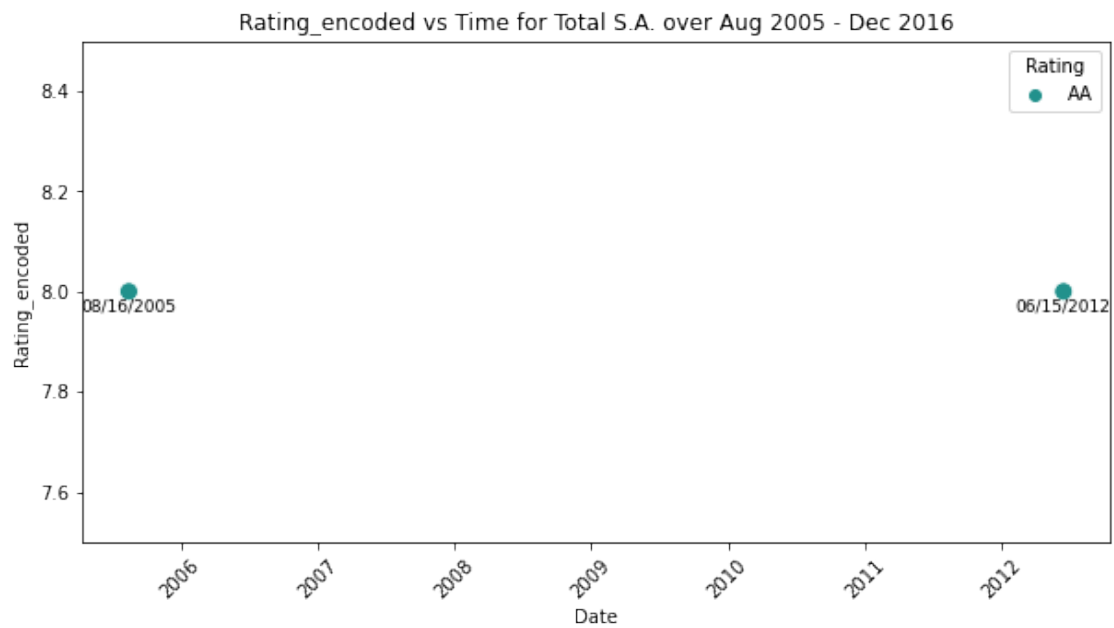


[243]: *#No time series analysis can be made. We can't investigate time dependency, as ↪ not enough data.*
#Also raings per company are very inconsistent across btime period

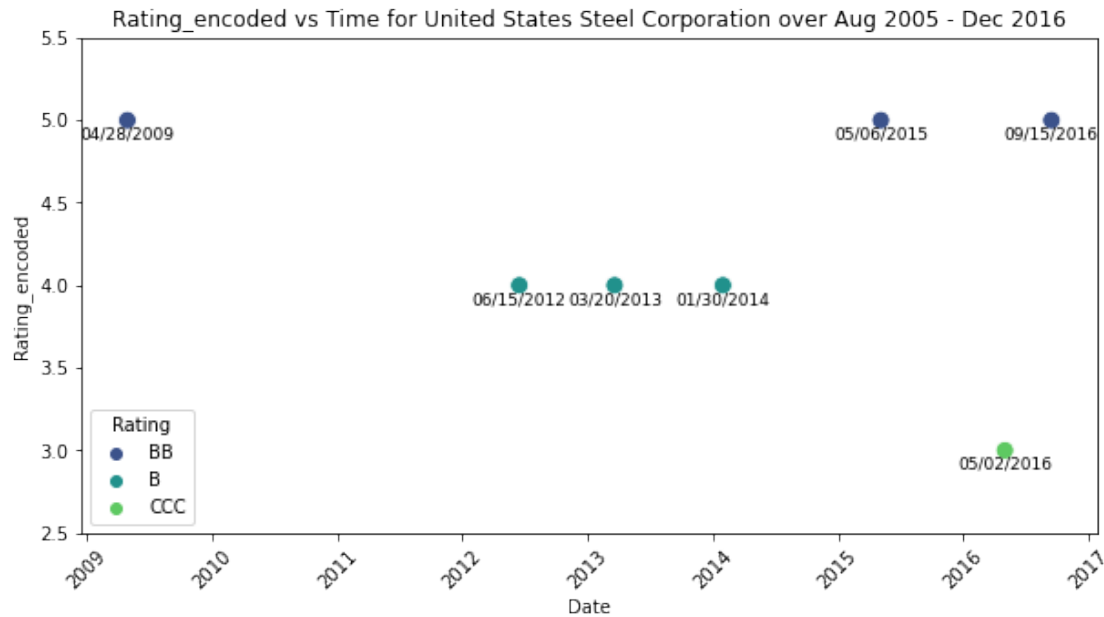
[244]: `plot_company_rating(credit_rating2, 'JetBlue Airways Corporation', ↪
 ↪ 'Rating_encoded')`



```
[245]: plot_company_rating(credit_rating2, 'Total S.A.', 'Rating_encoded')
```



```
[246]: plot_company_rating(credit_rating2, 'United States Steel Corporation',  
    ↪ 'Rating_encoded')  
  
#Credit rating are issued queaterly (max: yearly).
```



```
[247]: credit_rating2['Year'] = credit_rating2['Date'].dt.year
```

```
[248]: credit_rating2.head(5)
```

```
[248]:
```

	Rating	Name	Symbol	\
178	AA	Total S.A.	TOT	
576	BB	United States Steel Corporation	X	
496	CCC	Tenet Healthcare Corporation	THC	
1282	BBB	Magellan Midstream Partners L.P.	MMP	
229	BBB	Pitney Bowes Inc.	PBI	

	Rating Agency	Name	Date	Sector	\
178		DBRS	2005-08-16	Energy	
576	Standard & Poor's Ratings Services		2009-04-28	Basic Industries	
496	Standard & Poor's Ratings Services		2010-08-03	Health Care	
1282	Standard & Poor's Ratings Services		2010-08-31	Energy	
229	Standard & Poor's Ratings Services		2010-09-30	Miscellaneous	

	currentRatio	quickRatio	cashRatio	daysOfSalesOutstanding	...	\
178	0.882592	0.556147	0.144622	36.783384	...	
576	2.063355	0.510799	0.260619	0.000000	...	
496	1.339710	1.082319	0.234783	45.322651	...	
1282	1.398245	0.443477	0.027206	21.605923	...	
229	1.178049	0.511873	0.189680	53.293224	...	

	cashPerShare	companyEquityMultiplier	ebitPerRevenue	\
178				
576				
496				
1282				
229				

178	10.417039	1.094559	0.189980
576	12.084515	3.286415	0.133847
496	1.494876	4.813137	0.017165
1282	0.272046	2.529922	0.200058
229	2.500252	-87.429443	0.098535

	enterpriseValueMultiple	operatingCashFlowPerShare	\
178	1.264522	31.169316	
576	0.965856	6.293486	
496	15.656926	0.974560	
1282	11.799222	3.878668	
229	10.455327	4.622616	

	operatingCashFlowSalesRatio	payablesTurnover	year-month	\
178	0.152270	0.000000	2005-08	
576	0.032894	13.696528	2009-04	
496	0.051276	0.000000	2010-08	
1282	0.272662	0.000000	2010-08	
229	0.175496	0.077504	2010-09	

	Rating_encoded	Year
178	8	2005
576	5	2009
496	3	2010
1282	6	2010
229	6	2010

[5 rows x 34 columns]

2 Descriptive graphs + tables (Data discovery)

```
[249]: import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

def freq_graph(data, col, rotation):

    if data[col].dtype != 'object':
        data[col] = data[col].astype(str)

    category_counts = data[col].value_counts()
```

```

sns.barplot(x=category_counts.index, y=category_counts.values,
palette="pastel")

# Add labels and title
plt.title('Frequency of ' + col, fontsize=14)
plt.xlabel(col, fontsize=12)
plt.ylabel('Frequency', fontsize=12)

if(rotation==1):

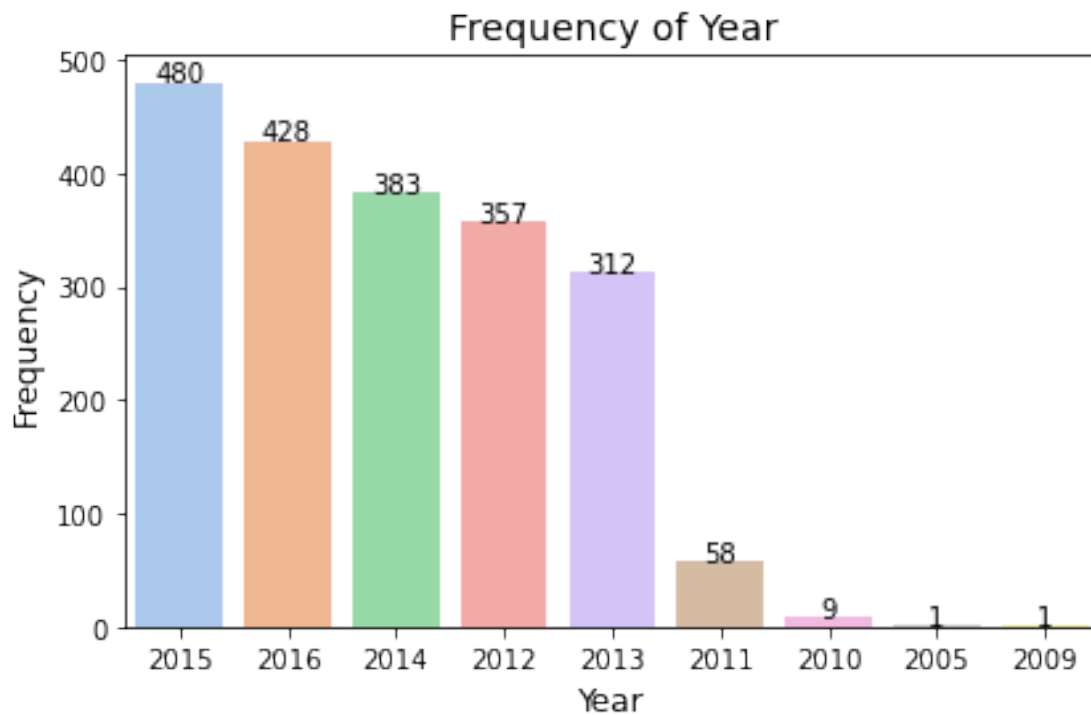
    # Rotate the x-axis labels to avoid overlap
    plt.xticks(rotation=45, ha='right')

# Annotate the bars with the frequency count
for i, count in enumerate(category_counts.values):
    plt.text(i, count + 0.5, str(count), ha='center', fontsize=10)

# Show the plot
plt.tight_layout()
plt.show()

```

```
[250]: freq_graph(credit_rating2.copy(), 'Year', 0)
```



```
[251]: credit_rating2.groupby('Year').size().reset_index(name='rating occurrence')
```

```
[251]:   Year  rating occurrence
0  2005                1
1  2009                1
2  2010                9
3  2011               58
4  2012             357
5  2013             312
6  2014             383
7  2015             480
8  2016             428
```

```
[252]: #remove all rows that before 2012
```

```
credit_rating_2012_2016 = credit_rating2[credit_rating2['Year']>2011]
```

```
[253]: credit_rating_2012_2016.head(10)
```

```
[253]:   Rating                                     Name Symbol \
466      A                                     ENI S.p.A.      E
704     BB      Williams Companies, Inc. (The)      WMB
1648    AA      United Parcel Service, Inc.      UPS
1762    BB      Griffon Corporation      GFF
1720     B      Cogent Communications Holdings, Inc.  CCOI
1051    BB      Continental Resources, Inc.      CLR
798     BB  Fidelity National Information Services, Inc.  FIS
329     BB      Edison International      EIX
1029     A      Pepsico, Inc.      PEP
392     BB      Jabil Inc.      JBL
```



```
   Rating Agency Name      Date      Sector \
466  Standard & Poor's Ratings Services 2012-01-17      Energy
704  Standard & Poor's Ratings Services 2012-02-16  Public Utilities
1648  Standard & Poor's Ratings Services 2012-02-17      Transportation
1762  Standard & Poor's Ratings Services 2012-02-29      Capital Goods
1720  Standard & Poor's Ratings Services 2012-02-29  Consumer Services
1051  Standard & Poor's Ratings Services 2012-03-05      Energy
798  Standard & Poor's Ratings Services 2012-03-05      Miscellaneous
329  Standard & Poor's Ratings Services 2012-03-06  Public Utilities
1029  Standard & Poor's Ratings Services 2012-03-27  Consumer Non-Durables
392  Standard & Poor's Ratings Services 2012-04-05      Technology
```



```
   currentRatio  quickRatio  cashRatio  daysOfSalesOutstanding ... \
466      1.071930    0.858273    0.042097      96.848531 ...
704      1.130746    0.934925    0.530746      29.319672 ...
```

1648	1.885784	1.615137	0.465766	42.929856	...
1762	2.857637	1.623020	0.771817	53.435320	...
1720	5.712163	5.502080	4.978931	29.903715	...
1051	0.842213	0.383750	0.048160	81.071558	...
798	1.241937	0.940291	0.306665	54.536871	...
329	1.031279	0.571067	0.337856	25.973354	...
1029	0.960725	0.624490	0.224028	37.935763	...
392	1.320182	0.511464	0.228441	24.326061	...

	cashPerShare	companyEquityMultiplier	ebitPerRevenue	\
466	0.439911	2.576886	0.222555	
704	1.578447	12.733025	0.151576	
1648	4.357798	4.932623	0.108766	
1762	4.124798	2.861223	-0.007838	
1720	5.272343	3.644191	0.018272	
1051	0.339056	2.446178	0.416687	
798	1.382236	2.129525	0.129540	
329	4.831288	4.333303	0.001881	
1029	2.807741	3.520189	0.132834	
392	4.142670	3.780121	0.027581	

	enterpriseValueMultiple	operatingCashFlowPerShare	\
466	6.205352	3.590689	
704	7.104383	5.843144	
1648	10.531309	7.209990	
1762	8.590875	0.600570	
1720	5.764237	1.678025	
1051	7.552498	6.013374	
798	7.450026	3.897206	
329	10.849987	11.981595	
1029	10.169132	5.675127	
392	5.140315	3.860146	

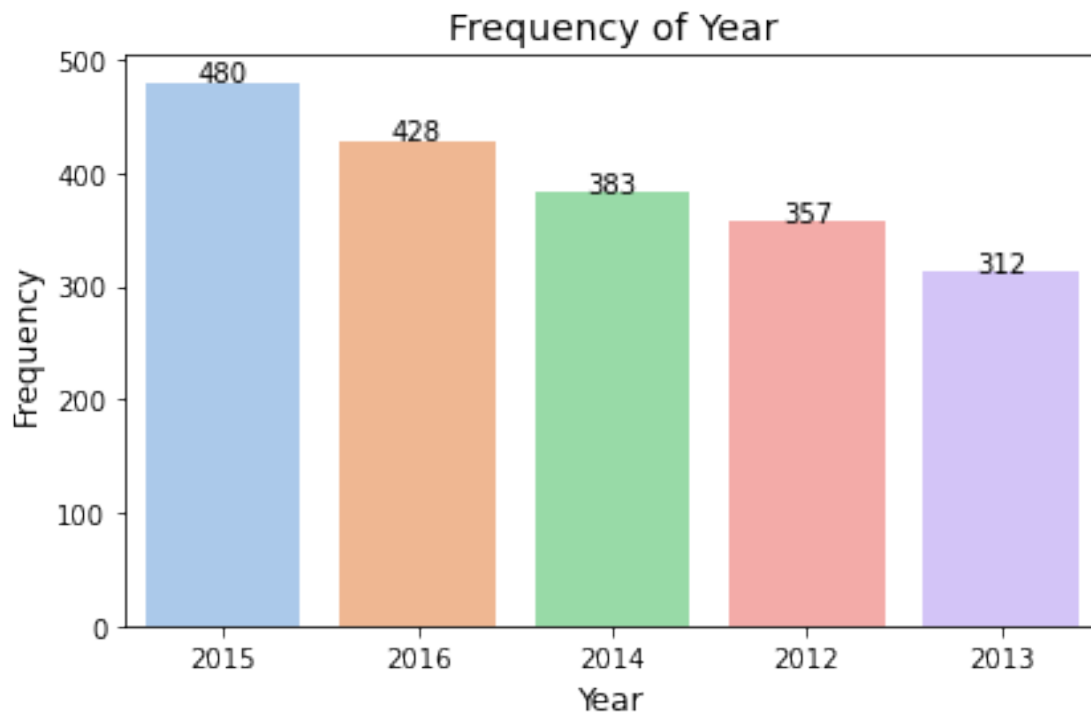
	operatingCashFlowSalesRatio	payablesTurnover	year-month	\
466	0.132411	3.439028	2012-01	
704	0.433670	0.000000	2012-02	
1648	0.133189	1.759130	2012-02	
1762	0.019328	7.848490	2012-02	
1720	0.248164	9.307698	2012-02	
1051	0.647304	0.000000	2012-03	
798	0.203892	6.232268	2012-03	
329	0.306113	0.821705	2012-03	
1029	0.134488	2.687165	2012-03	
392	0.050125	5.290596	2012-04	

Rating_encoded	Year
466	7 2012

704	5	2012
1648	8	2012
1762	5	2012
1720	4	2012
1051	5	2012
798	5	2012
329	5	2012
1029	7	2012
392	5	2012

[10 rows x 34 columns]

```
[254]: freq_graph(credit_rating_2012_2016.copy(), 'Year', 0)
```



```
[255]: credit_rating_per_company_2012_2016 = credit_rating_2012_2016.groupby('Name').
        ↳size().reset_index(name='rating occurrence')
credit_rating_per_company_2012_2016 = credit_rating_per_company_2012_2016.
        ↳sort_values(by='rating occurrence', ascending=False)
credit_rating_per_company_2012_2016
```

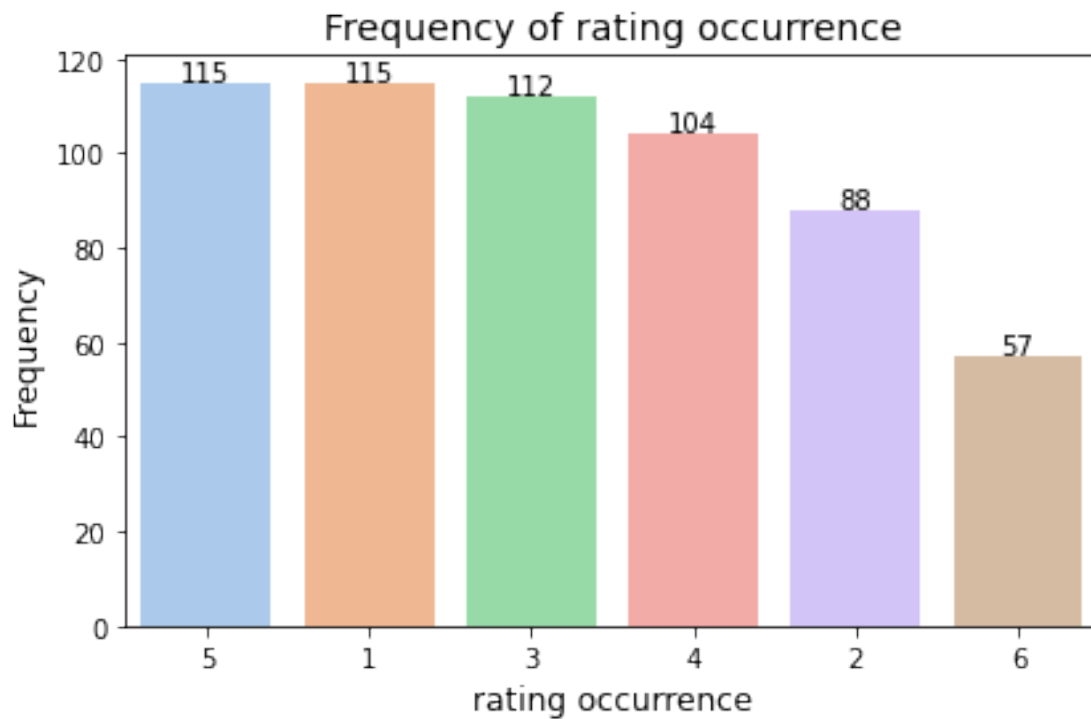
```
[255]:
```

	Name	rating occurrence
96	CMS Energy Corporation	6
561	W&T Offshore, Inc.	6

34	Amkor Technology, Inc.	6
182	ENI S.p.A.	6
553	Verizon Communications Inc.	6
..	...	...
468	Sabre Corporation	1
137	Coca-Cola European Partners plc	1
133	Clean Harbors, Inc.	1
353	Mastercard Incorporated	1
329	Loews Corporation	1

[591 rows x 2 columns]

```
[256]: freq_graph(credit_rating_per_company_2012_2016.copy(), 'rating occurrence', 0)
```



```
[257]: earliest_date = credit_rating_2012_2016['Date'].min()
latest_date = credit_rating_2012_2016['Date'].max()

print("Earliest Date:", earliest_date)
print("Latest Date:", latest_date)

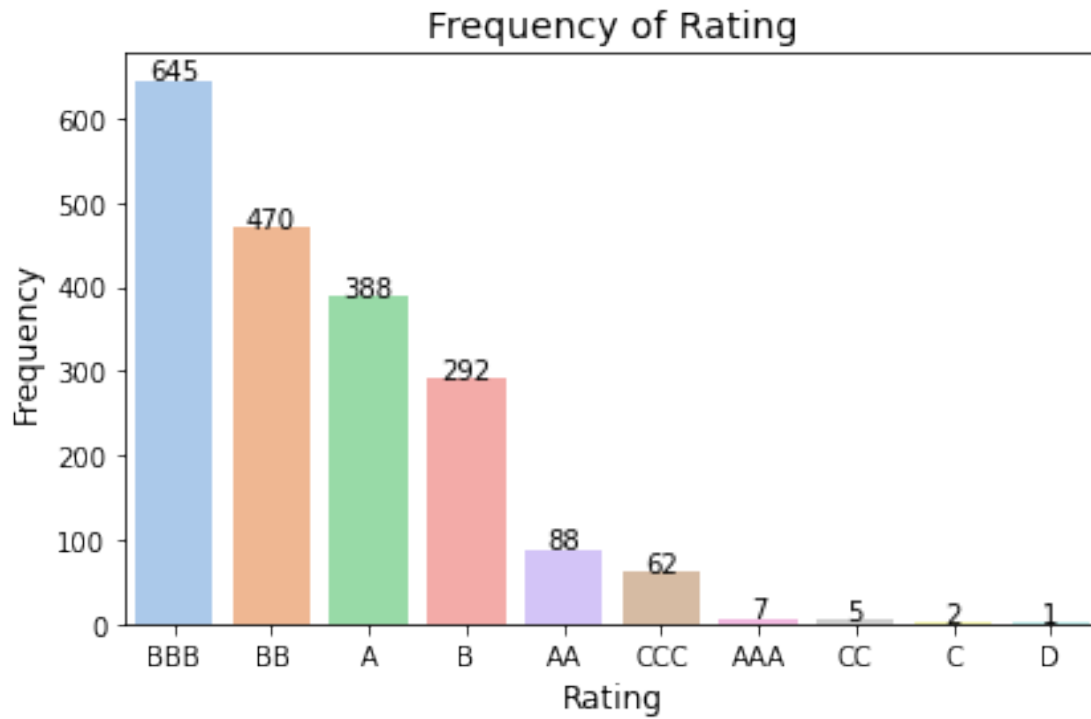
print("Average Rating Occurrence during [2012-01-17, 2016-12-23] period :",
      credit_rating_per_company_2012_2016['rating occurrence'].mean())
```

Earliest Date: 2012-01-17 00:00:00

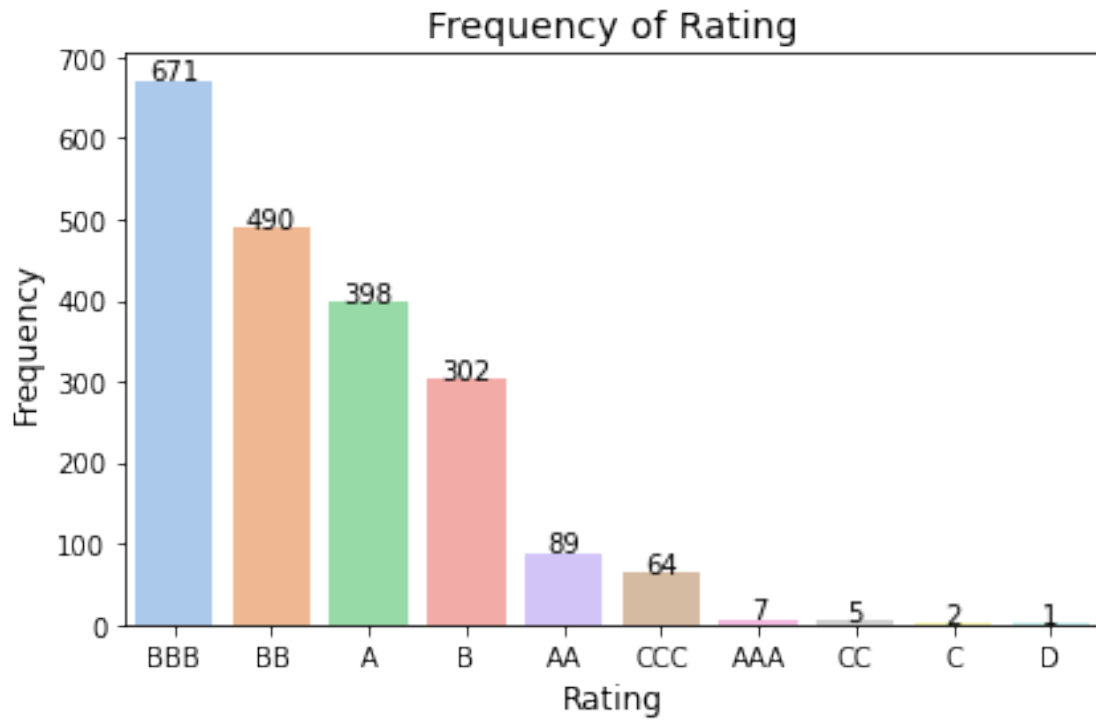
Latest Date: 2016-12-23 00:00:00

Average Rating Occurrence during [2012-01-17, 2016-12-23] period :
3.3164128595600677

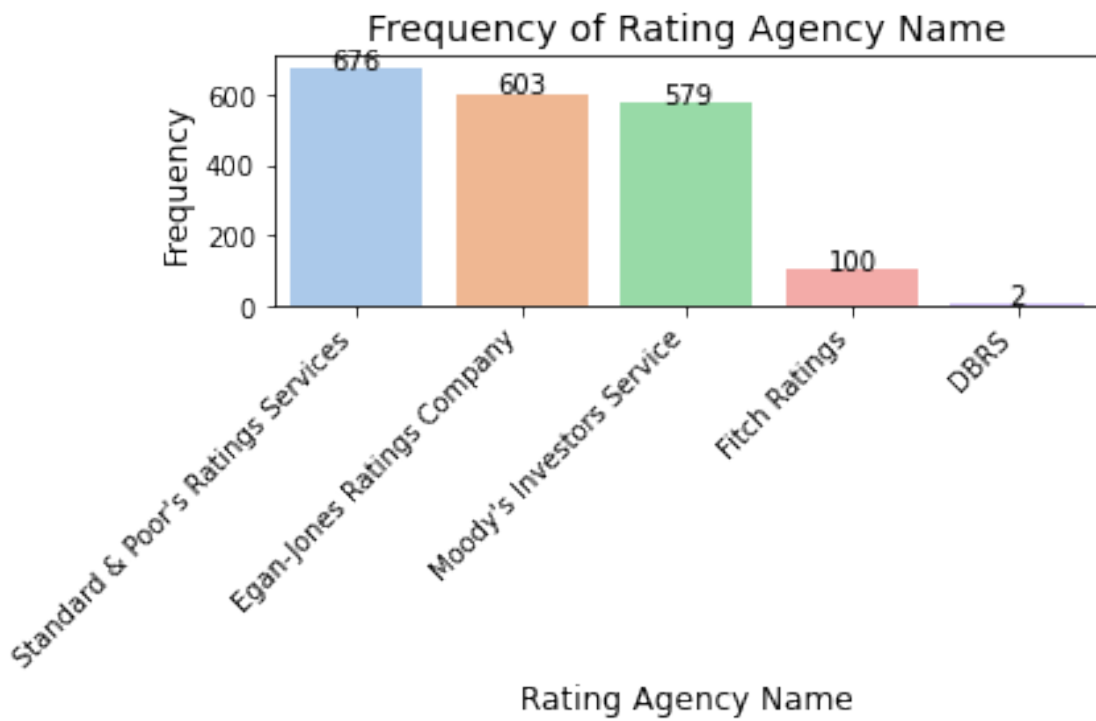
```
[258]: freq_graph(credit_rating_2012_2016.copy(), 'Rating', 0)
```



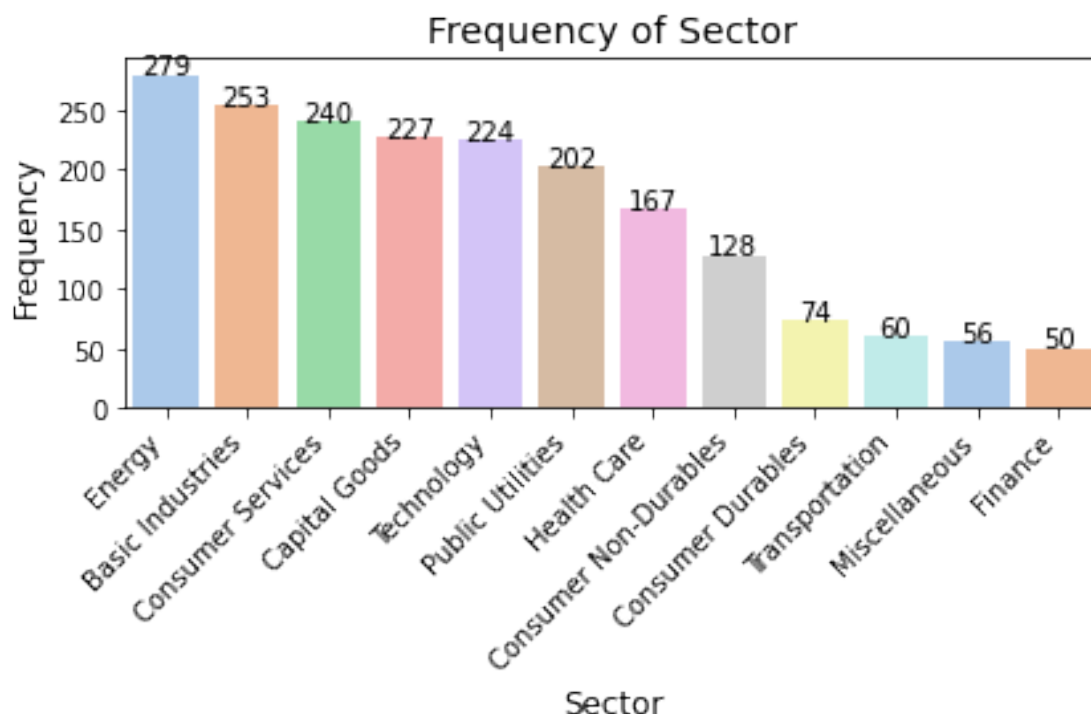
```
[259]: freq_graph(credit_rating2.copy(), 'Rating', 0)
```



```
[260]: freq_graph(credit_rating_2012_2016, 'Rating Agency Name', 1)
```



```
[261]: freq_graph(credit_rating_2012_2016, 'Sector', 1)
```



```
[262]: credit_rating_2012_2016[['Rating', 'Name', 'Symbol', 'Rating Agency Name', 'Sector']].nunique()
```

```
#Use sector as independent variable
```

```
[262]: Rating          10
      Name          591
      Symbol        591
      Rating Agency Name    5
      Sector         12
      dtype: int64
```

```
[263]: #Create risk_level
```

```
rating_dict = {'AAA': 'Lowest Risk',
               'AA': 'Low Risk',
               'A': 'Low Risk',
               'BBB': 'Medium Risk',
               'BB': 'High Risk',
               'B': 'High Risk',
```

```

        'CCC': 'Highest Risk',
        'CC': 'Highest Risk',
        'C': 'Highest Risk',
        'D': 'In Default'}

# Create the 'Risk_level' column using the dictionary
credit_rating_2012_2016['Risk_level'] = credit_rating_2012_2016['Rating'].
    ↪map(rating_dict)

credit_rating_2012_2016

```

/var/folders/10/54kf6c4x4ggbwjtfk_yk4lkh0000gn/T/ipykernel_13489/2302167403.py:1

6: SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame.

Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```

    credit_rating_2012_2016['Risk_level'] =
    credit_rating_2012_2016['Rating'].map(rating_dict)

```

```

[263]:
      Rating                                     Name Symbol \
466      A                               ENI S.p.A.      E
704     BB      Williams Companies, Inc. (The)      WMB
1648    AA      United Parcel Service, Inc.      UPS
1762    BB      Griffon Corporation      GFF
1720     B  Cogent Communications Holdings, Inc.  CCOI
...     ...
1129    BBB                               PLDT Inc.  PHI
1478    BBB      CSG Systems International, Inc.  CSGS
741     B      HD Supply Holdings, Inc.      HDS
1622    BBB                               Zoetis Inc.  ZTS
1968    BB      Royal Caribbean Cruises Ltd.  RCL

      Rating Agency Name      Date      Sector \
466  Standard & Poor's Ratings Services 2012-01-17      Energy
704  Standard & Poor's Ratings Services 2012-02-16  Public Utilities
1648  Standard & Poor's Ratings Services 2012-02-17      Transportation
1762  Standard & Poor's Ratings Services 2012-02-29      Capital Goods
1720  Standard & Poor's Ratings Services 2012-02-29  Consumer Services
...     ...
1129  Standard & Poor's Ratings Services 2016-12-19  Public Utilities
1478  Standard & Poor's Ratings Services 2016-12-20      Technology
741   Standard & Poor's Ratings Services 2016-12-21  Consumer Services
1622  Standard & Poor's Ratings Services 2016-12-23      Health Care
1968  Standard & Poor's Ratings Services 2016-12-23  Consumer Services

```

	currentRatio	quickRatio	cashRatio	daysOfSalesOutstanding	...	\
466	1.071930	0.858273	0.042097	96.848531	...	
704	1.130746	0.934925	0.530746	29.319672	...	
1648	1.885784	1.615137	0.465766	42.929856	...	
1762	2.857637	1.623020	0.771817	53.435320	...	
1720	5.712163	5.502080	4.978931	29.903715	...	
...	...	...	...	...	...	
1129	0.472296	0.363831	0.212743	53.969697	...	
1478	2.221721	1.926080	0.501335	100.215058	...	
741	2.276693	1.345580	0.308840	40.264618	...	
1622	3.034915	1.468218	0.650850	68.176146	...	
1968	0.168476	0.030660	0.029855	0.000000	...	

	companyEquityMultiplier	ebitPerRevenue	enterpriseValueMultiple	\
466	2.576886	0.222555	6.205352	
704	12.733025	0.151576	7.104383	
1648	4.932623	0.108766	10.531309	
1762	2.861223	-0.007838	8.590875	
1720	3.644191	0.018272	5.764237	
...	...	...	...	
1129	4.392133	0.132608	2.652202	
1478	3.548214	0.131412	9.797003	
741	8.086022	0.032620	6438.074149	
1622	5.143914	0.251227	20.874287	
1968	2.445929	0.151051	12.241731	

	operatingCashFlowPerShare	operatingCashFlowSalesRatio	\
466	3.590689	0.132411	
704	5.843144	0.433670	
1648	7.209990	0.133189	
1762	0.600570	0.019328	
1720	1.678025	0.248164	
...	...	...	
1129	258.919254	0.338499	
1478	2.718484	0.110632	
741	0.002142	0.057120	
1622	1.438326	0.145867	
1968	11.628574	0.296207	

	payablesTurnover	year-month	Rating_encoded	Year	Risk_level
466	3.439028	2012-01	7	2012	Low Risk
704	0.000000	2012-02	5	2012	High Risk
1648	1.759130	2012-02	8	2012	Low Risk
1762	7.848490	2012-02	5	2012	High Risk
1720	9.307698	2012-02	4	2012	High Risk
...	...	...	...	...	...

1129	0.526270	2016-12	6	2016	Medium Risk
1478	10.682471	2016-12	6	2016	Medium Risk
741	9.708661	2016-12	4	2016	High Risk
1622	6.286792	2016-12	6	2016	Medium Risk
1968	16.427532	2016-12	5	2016	High Risk

[1960 rows x 35 columns]

```
[264]: #encode risk level

# Label encode 'rating'

risk_mapping = {'Lowest Risk': 0,
               'Low Risk': 1,
               'Medium Risk': 2,
               'High Risk': 3,
               'Highest Risk': 4,
               'In Default': 5}

# Apply encoding
credit_rating_2012_2016['Risk_level_encoded'] =
    credit_rating_2012_2016['Risk_level'].map(risk_mapping)

# Print encoding
print("Risk Level Encoding:", risk_mapping)
```

Risk Level Encoding: {'Lowest Risk': 0, 'Low Risk': 1, 'Medium Risk': 2, 'High Risk': 3, 'Highest Risk': 4, 'In Default': 5}

/var/folders/l0/54kf6c4x4ggbwjtfk_yk4lkh0000gn/T/ipykernel_13489/2788763875.py:1

5: SettingWithCopyWarning:

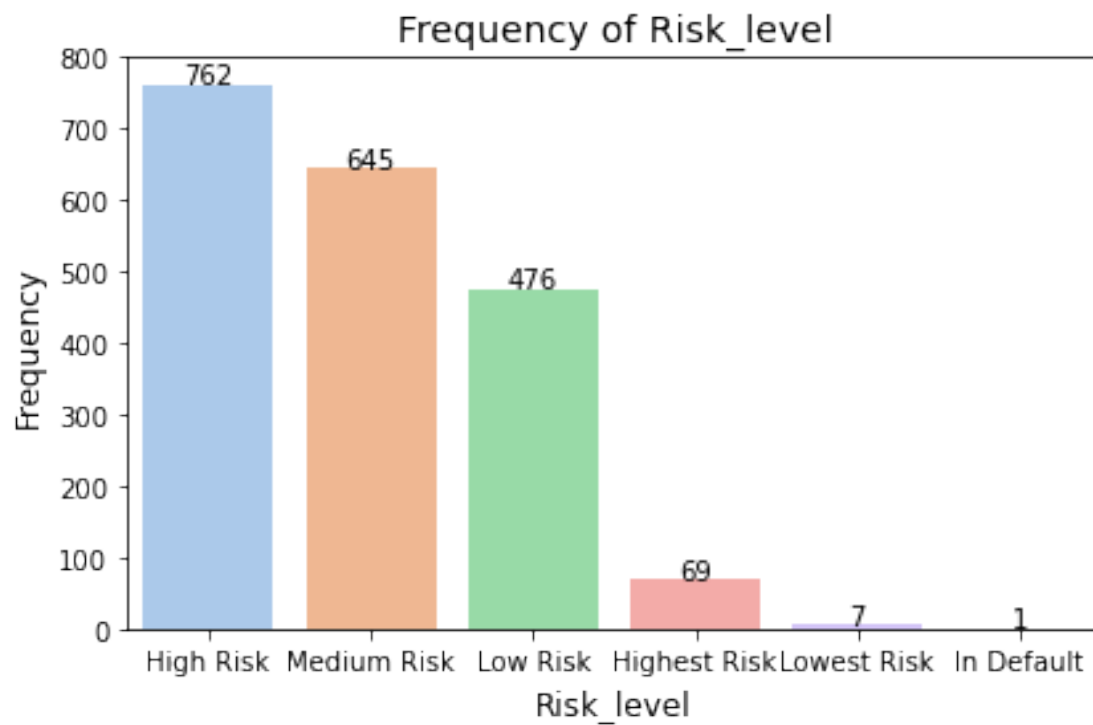
A value is trying to be set on a copy of a slice from a DataFrame.

Try using .loc[row_indexer,col_indexer] = value instead

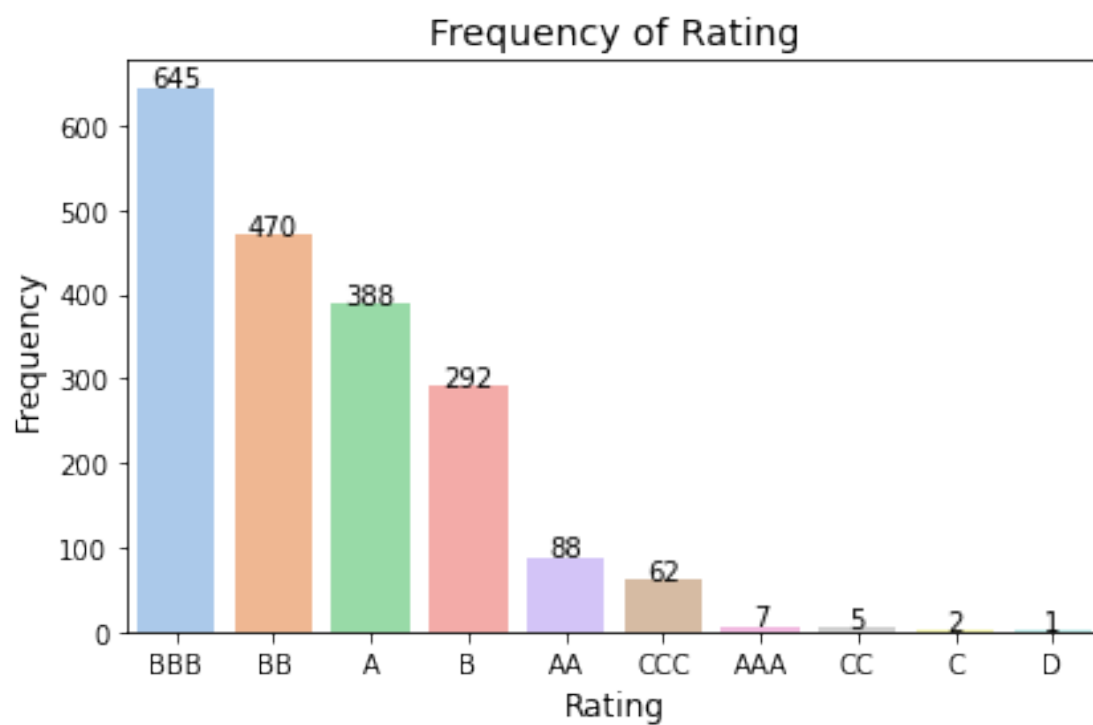
See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
credit_rating_2012_2016['Risk_level_encoded'] =
credit_rating_2012_2016['Risk_level'].map(risk_mapping)
```

```
[265]: freq_graph(credit_rating_2012_2016, 'Risk_level', 0)
```

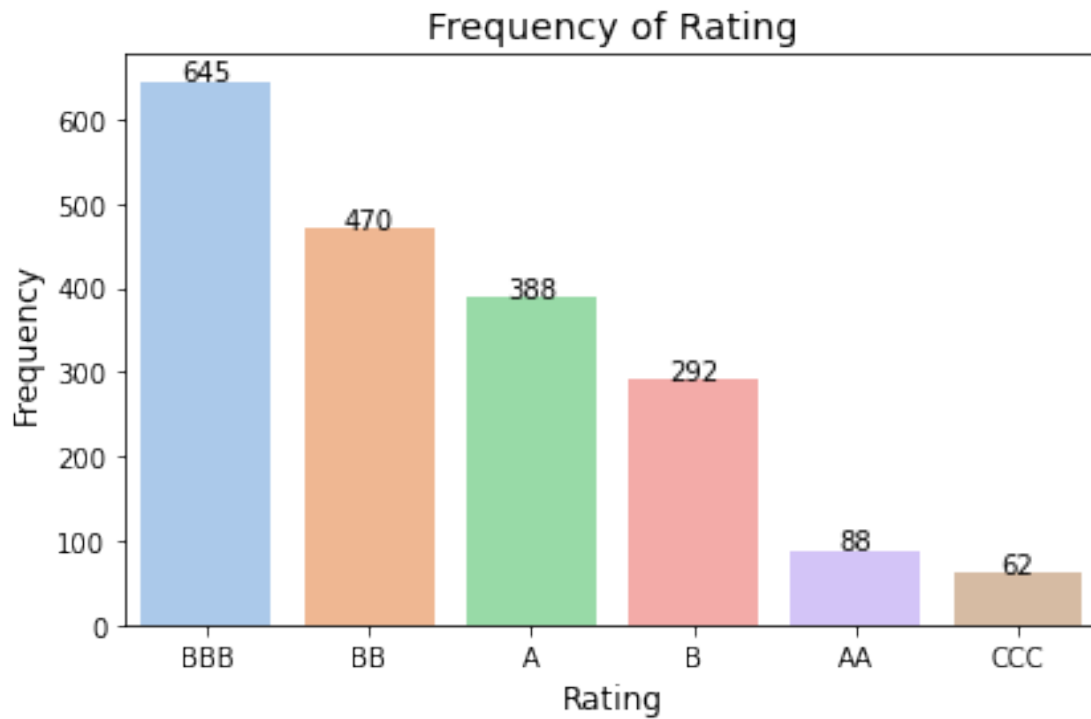



```
[266]: freq_graph(credit_rating_2012_2016, 'Rating', 0)
```

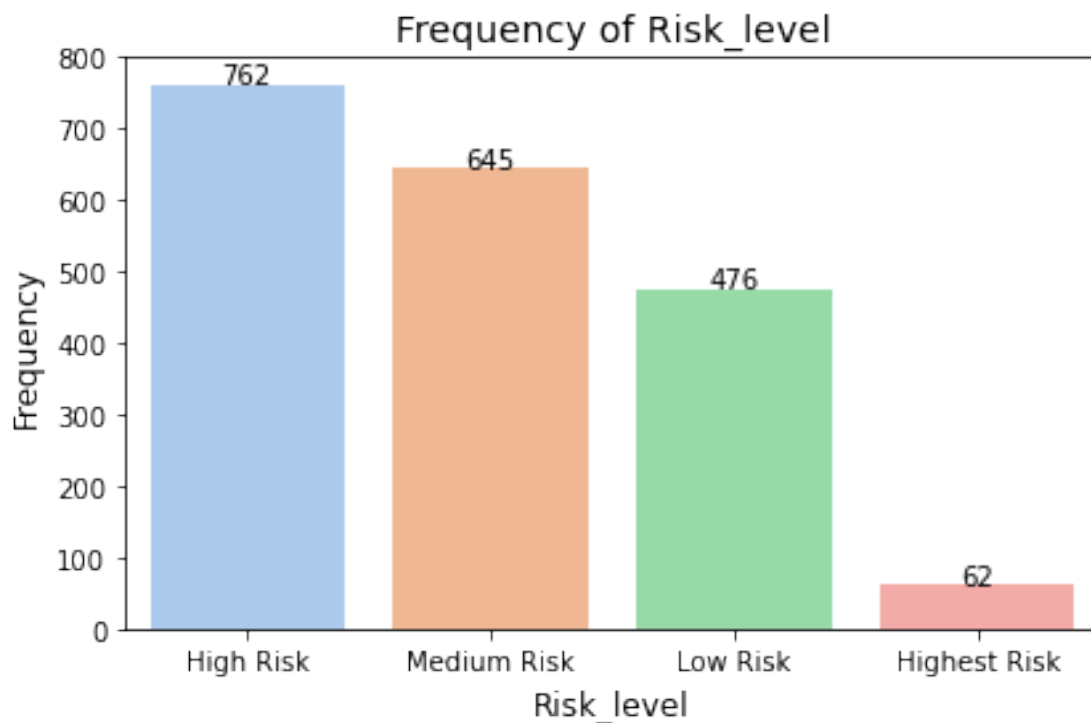


```
[267]: # Drop rows where Rating is 'C', 'D','CC','AAA'

credit_rating_new= credit_rating_2012_2016[~credit_rating_2012_2016['Rating'].
↪isin(['C', 'D','CC','AAA'])]
freq_graph(credit_rating_new, 'Rating', 0)
```



```
[268]: freq_graph(credit_rating_new, 'Risk_level', 0)
```



```
[269]: credit_rating_new.head(5)
```

```
[269]:
```

	Rating	Name	Symbol	\
466	A	ENI S.p.A.	E	
704	BB	Williams Companies, Inc. (The)	WMB	
1648	AA	United Parcel Service, Inc.	UPS	
1762	BB	Griffon Corporation	GFF	
1720	B	Cogent Communications Holdings, Inc.	CCOI	

	Rating Agency	Name	Date	Sector	\
466	Standard & Poor's Ratings Services		2012-01-17	Energy	
704	Standard & Poor's Ratings Services		2012-02-16	Public Utilities	
1648	Standard & Poor's Ratings Services		2012-02-17	Transportation	
1762	Standard & Poor's Ratings Services		2012-02-29	Capital Goods	
1720	Standard & Poor's Ratings Services		2012-02-29	Consumer Services	

	currentRatio	quickRatio	cashRatio	daysOfSalesOutstanding	...	\
466	1.071930	0.858273	0.042097	96.848531	...	
704	1.130746	0.934925	0.530746	29.319672	...	
1648	1.885784	1.615137	0.465766	42.929856	...	
1762	2.857637	1.623020	0.771817	53.435320	...	
1720	5.712163	5.502080	4.978931	29.903715	...	

	ebitPerRevenue	enterpriseValueMultiple	operatingCashFlowPerShare	\
466	0.222555	6.205352	3.590689	
704	0.151576	7.104383	5.843144	
1648	0.108766	10.531309	7.209990	
1762	-0.007838	8.590875	0.600570	
1720	0.018272	5.764237	1.678025	

	operatingCashFlowSalesRatio	payablesTurnover	year-month	\
466	0.132411	3.439028	2012-01	
704	0.433670	0.000000	2012-02	
1648	0.133189	1.759130	2012-02	
1762	0.019328	7.848490	2012-02	
1720	0.248164	9.307698	2012-02	

	Rating_encoded	Year	Risk_level	Risk_level_encoded
466	7	2012	Low Risk	1
704	5	2012	High Risk	3
1648	8	2012	Low Risk	1
1762	5	2012	High Risk	3
1720	4	2012	High Risk	3

[5 rows x 36 columns]

3 Distribution of variables in credit_rating_new and outliers

```
[270]: ### Boxplots

# def boxplt(data, category_col, prdeictor):

#     # Create boxplot
#     plt.figure(figsize=(8, 6))
#     sns.boxplot(x=category_col, y=prdeictor, data=data, whis=2)

#     # Display the plot
#     plt.title('Boxplot of ' + prdeictor + ' by ' + category_col)
#     plt.show()

def boxplt(data, category_col, predictor):
    # Create boxplot
    plt.figure(figsize=(8, 6))
```

```

sns.boxplot(x=category_col, y=predictor, data=data, whis=2)

# Increase font size
plt.xlabel(category_col, fontsize=16) # X-axis label
plt.ylabel(predictor, fontsize=16) # Y-axis label
plt.title(f'Boxplot of {predictor} by {category_col}', fontsize=18) # Title

# Increase tick label size
plt.xticks(fontsize=14)
plt.yticks(fontsize=14)

# Display the plot
plt.show()

```

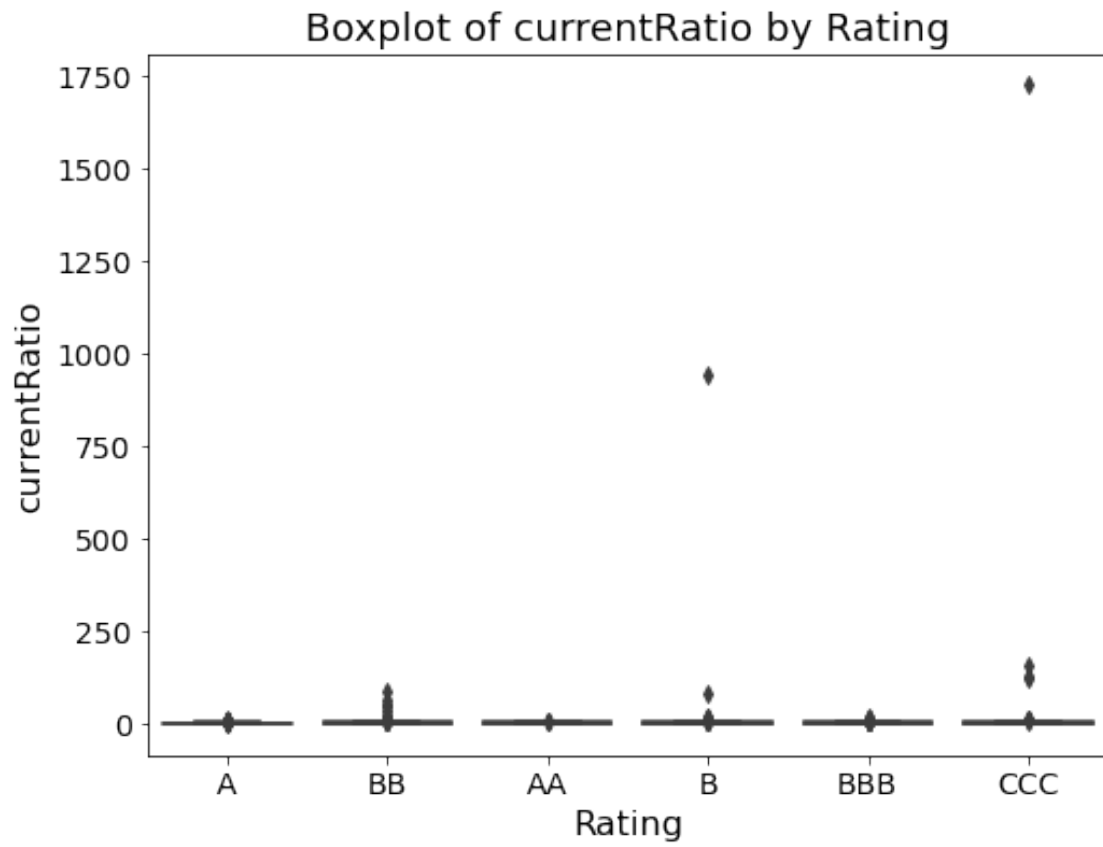
```

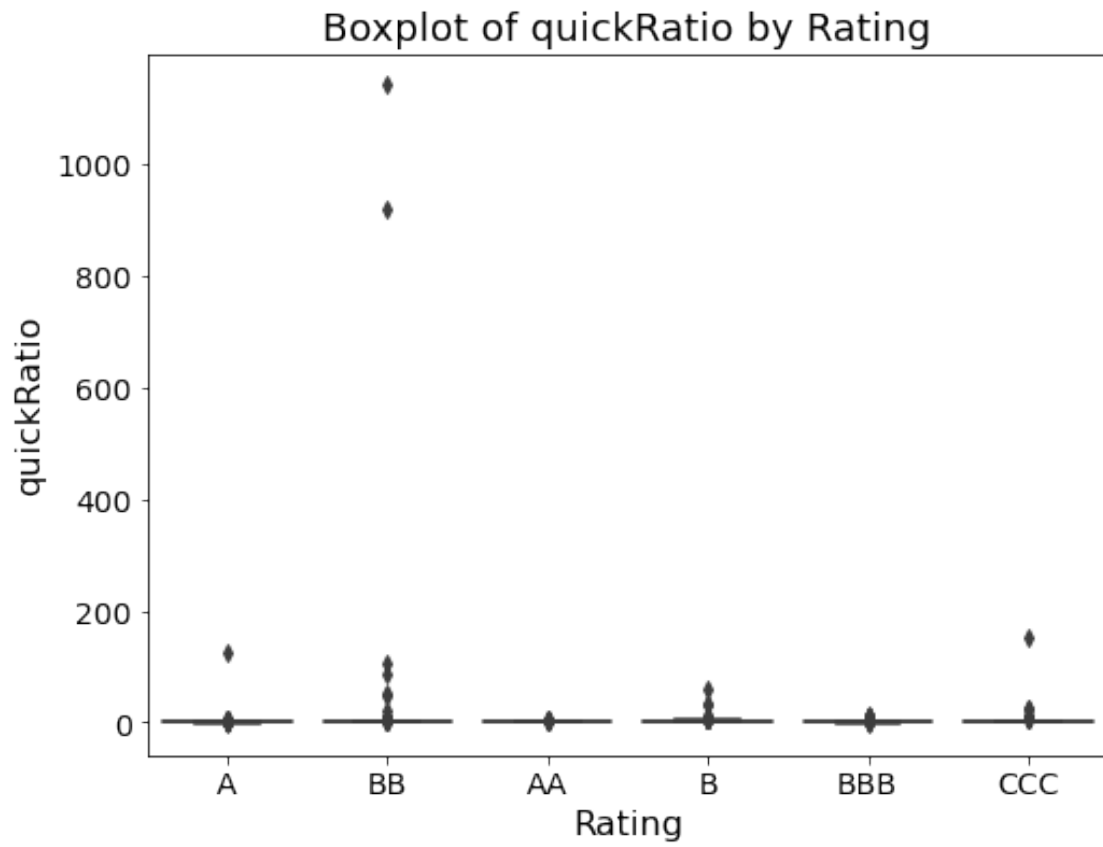
[271]: predictors=['currentRatio', 'quickRatio', 'cashRatio', 'daysOfSalesOutstanding',
    'netProfitMargin', 'pretaxProfitMargin', 'grossProfitMargin',
    'operatingProfitMargin', 'returnOnAssets', 'returnOnCapitalEmployed',
    'returnOnEquity', 'assetTurnover', 'fixedAssetTurnover',
    'debtEquityRatio', 'debtRatio', 'effectiveTaxRate',
    'freeCashFlowOperatingCashFlowRatio', 'freeCashFlowPerShare',
    'cashPerShare', 'companyEquityMultiplier', 'ebitPerRevenue',
    'enterpriseValueMultiple', 'operatingCashFlowPerShare',
    'operatingCashFlowSalesRatio', 'payablesTurnover']

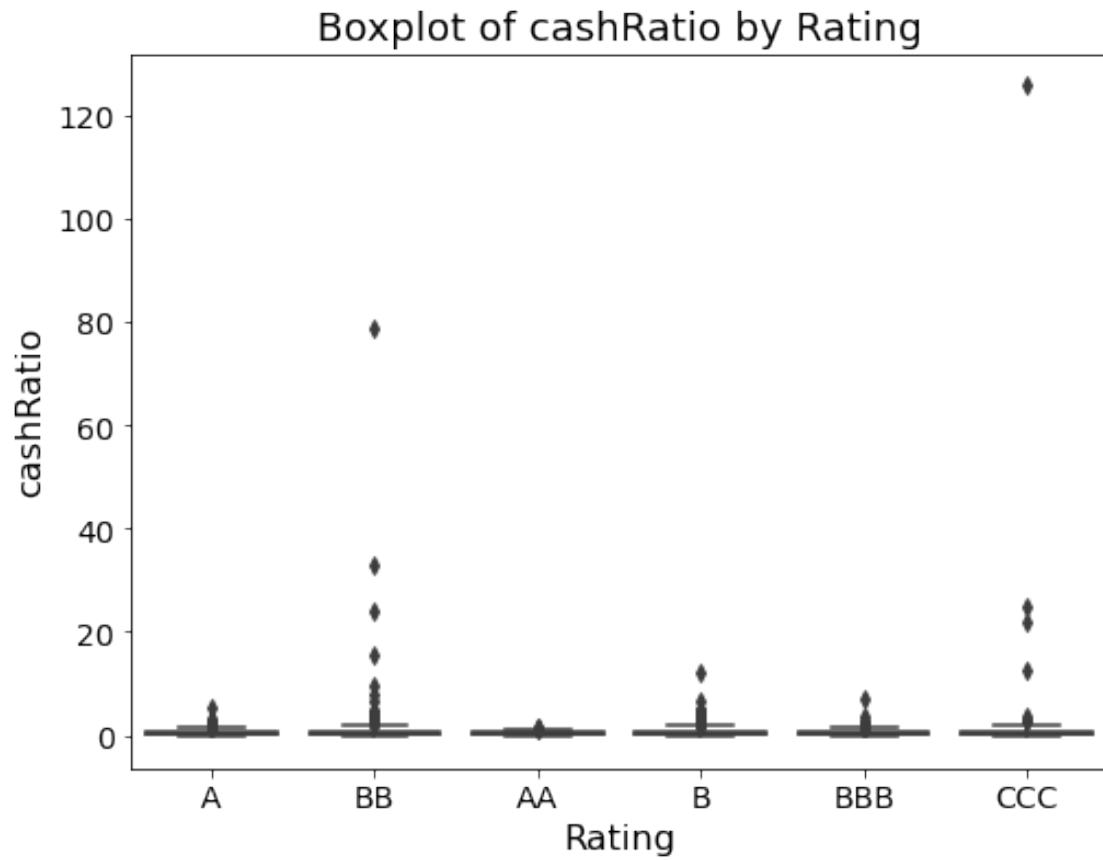
#category_col = ['Rating', 'Risk_level']

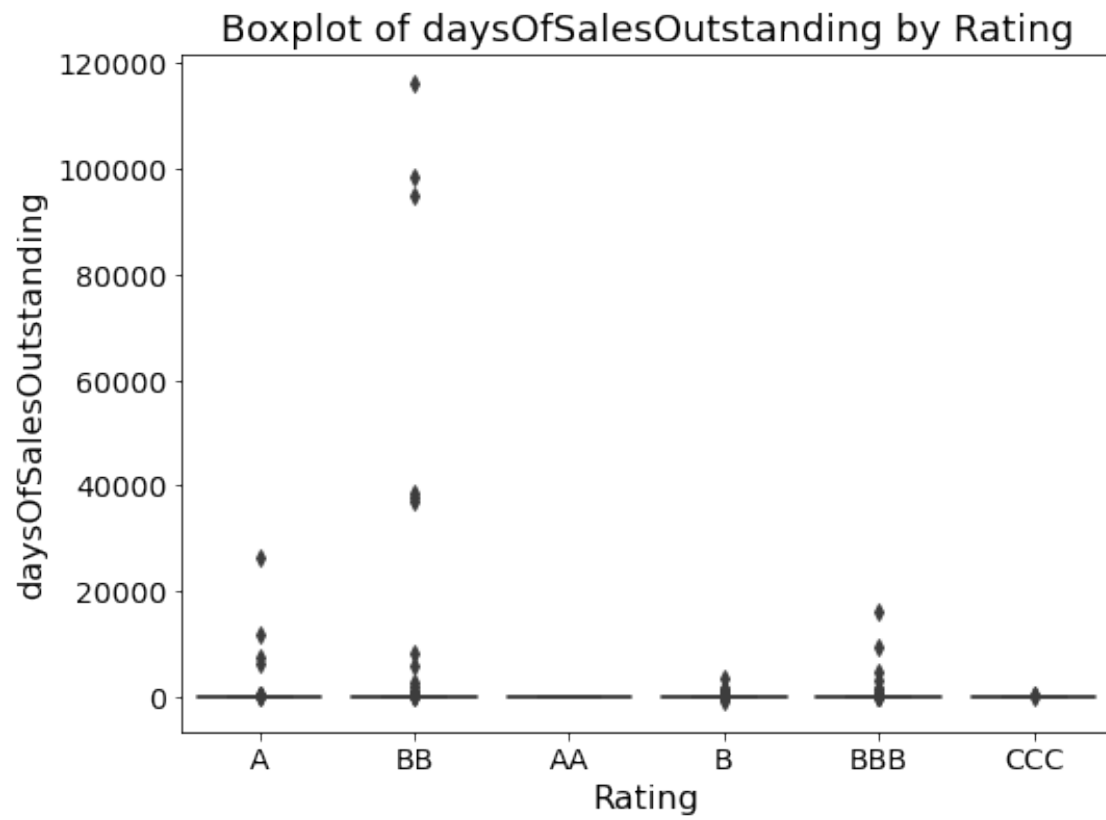
for pred in predictors:
    boxplt(credit_rating_new, 'Rating', pred)

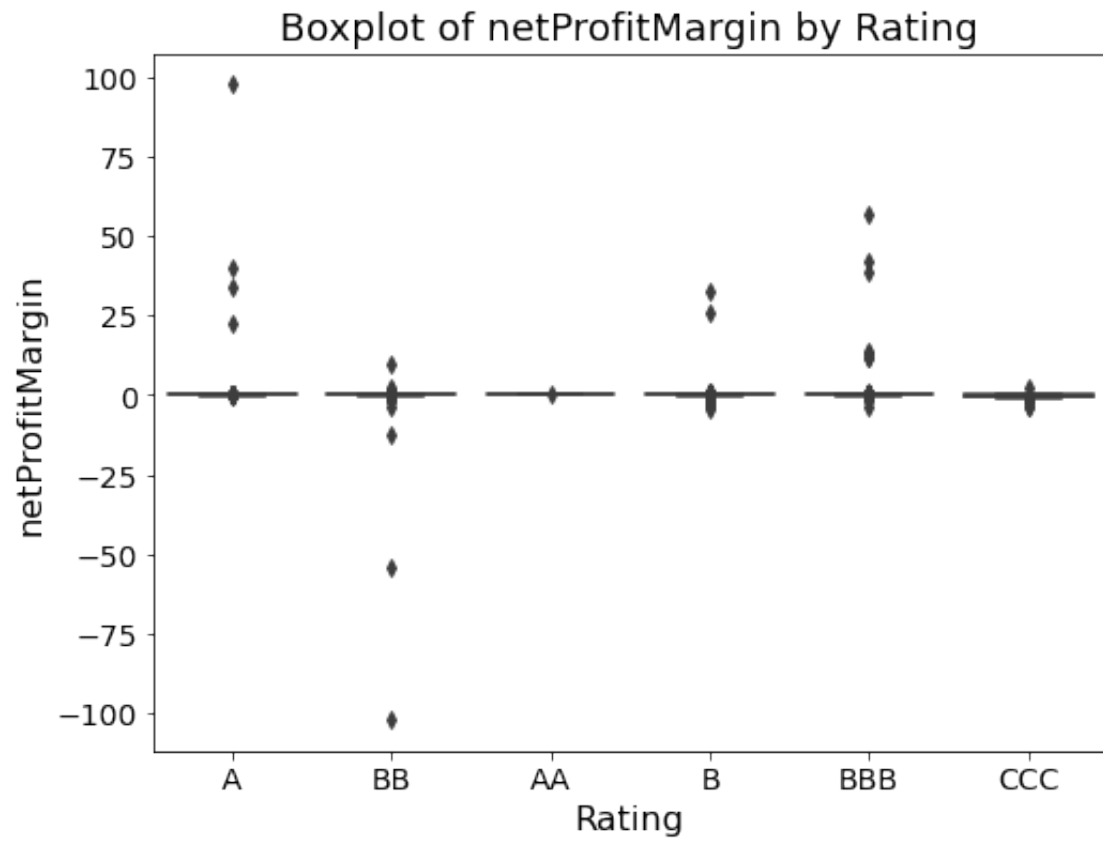
```

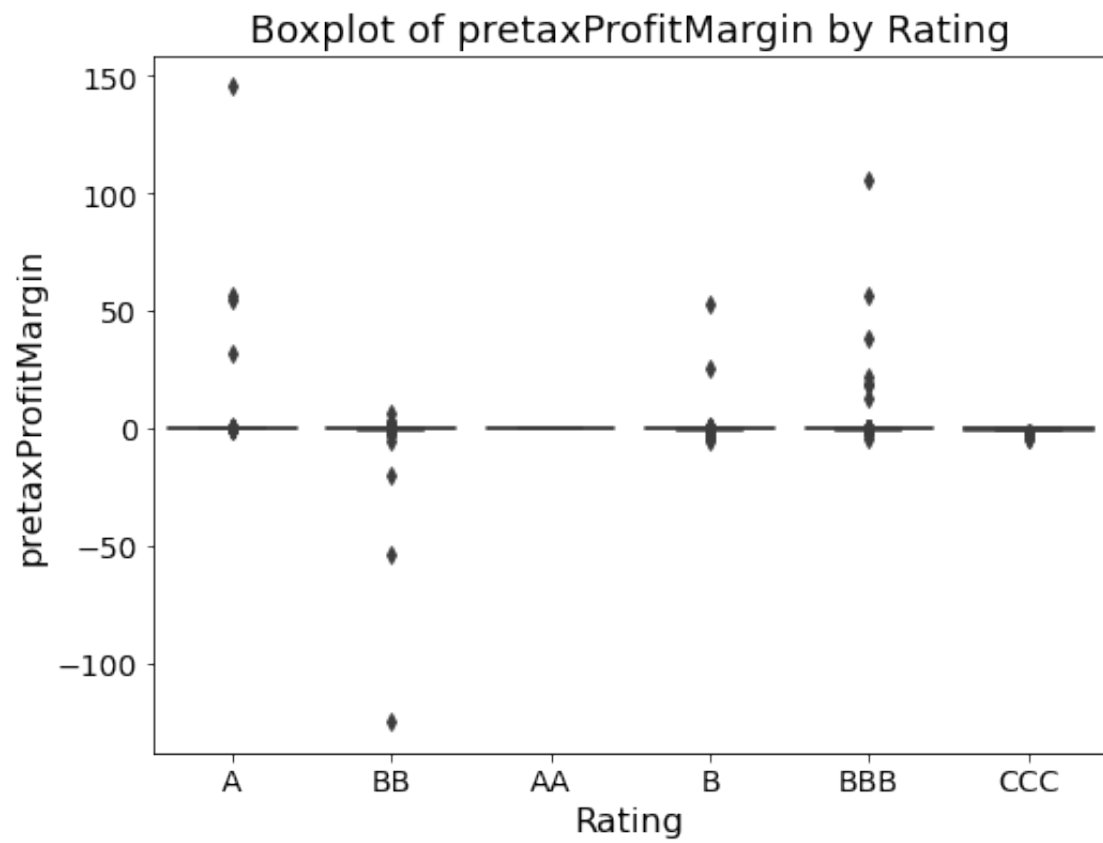


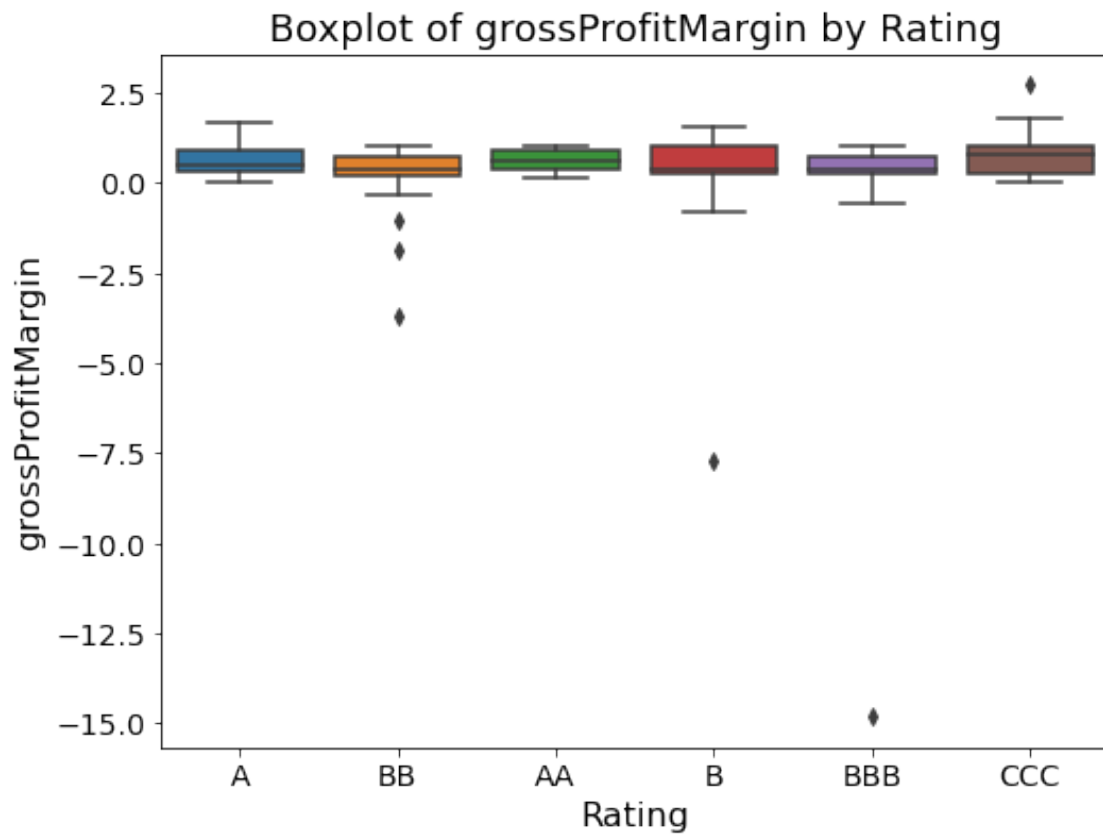


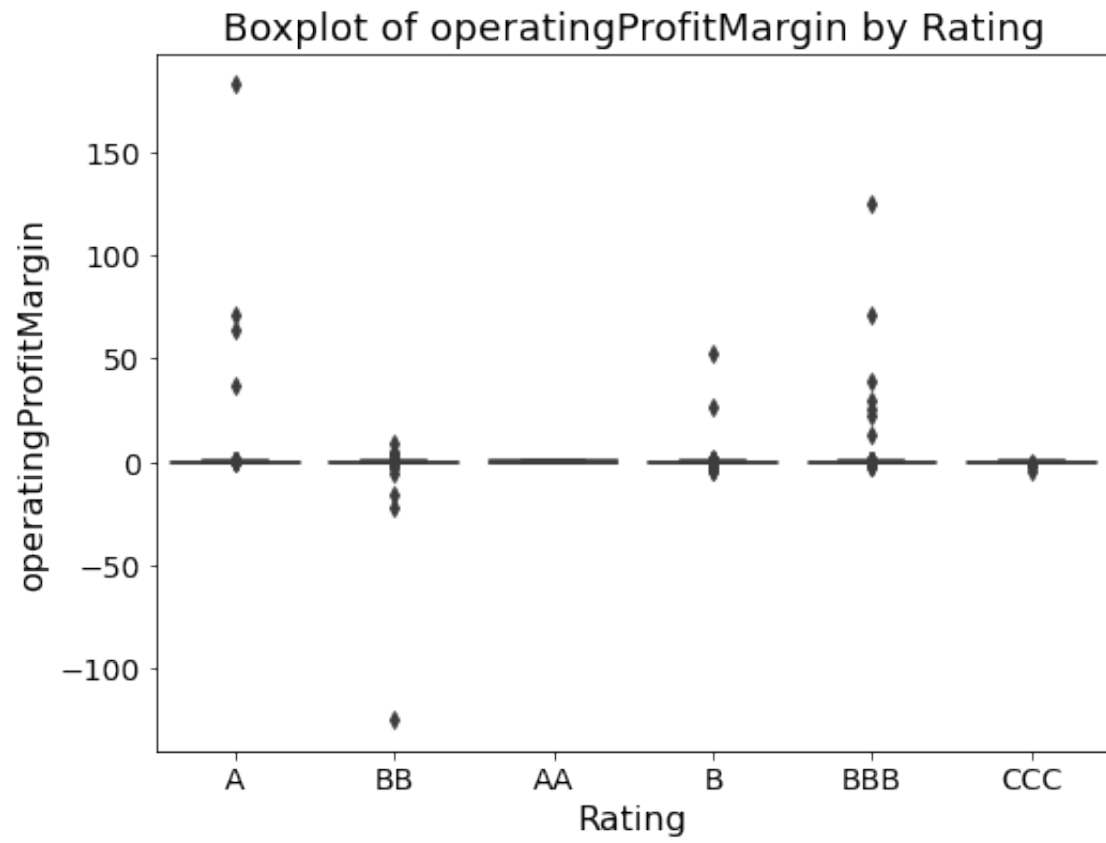


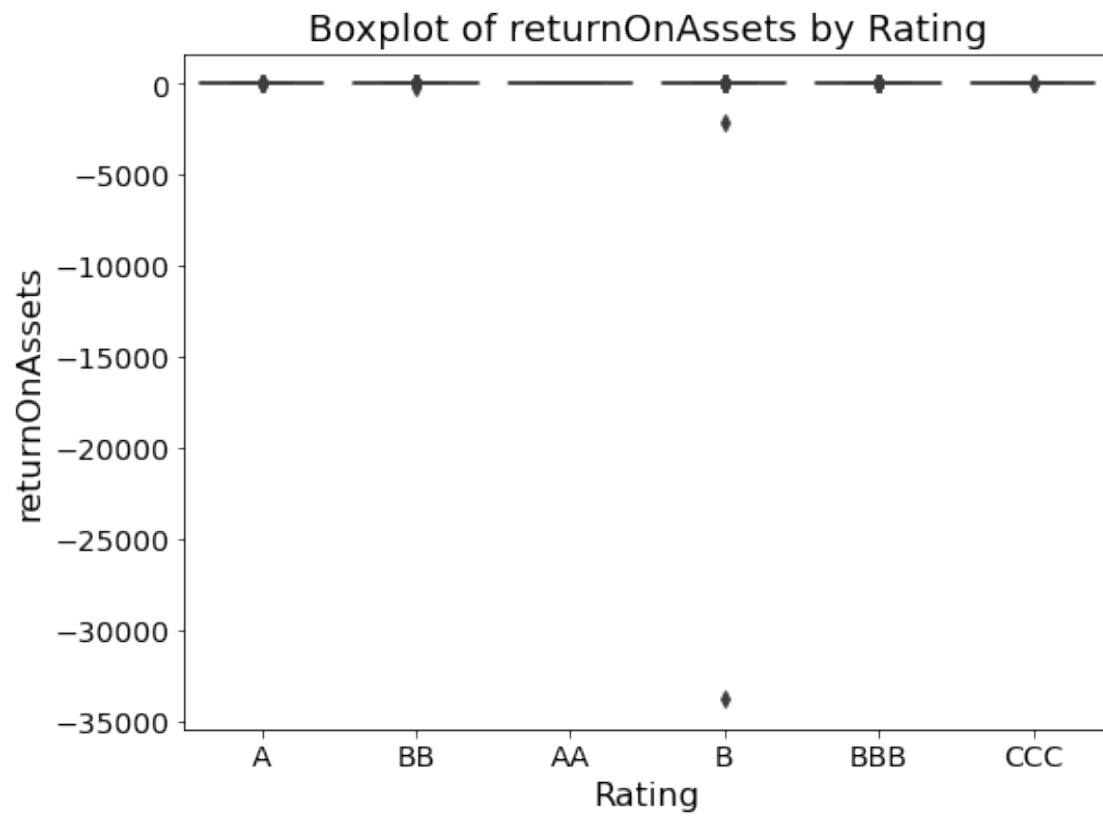


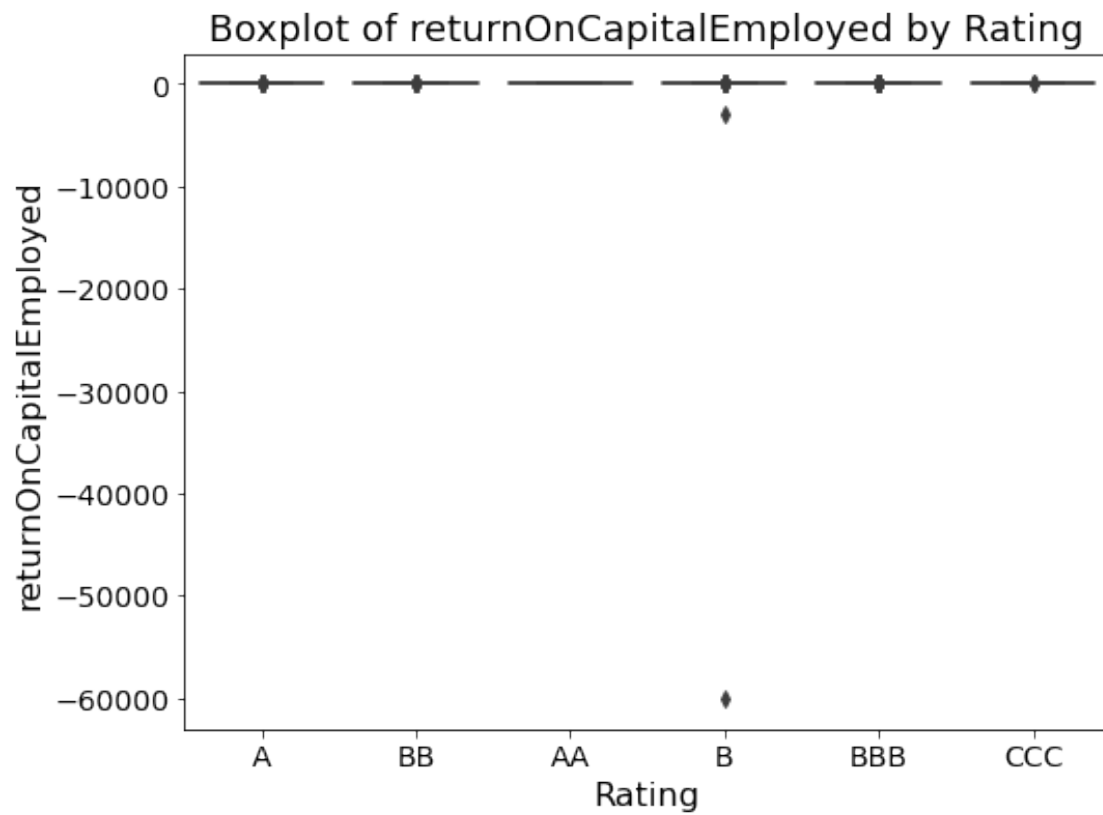


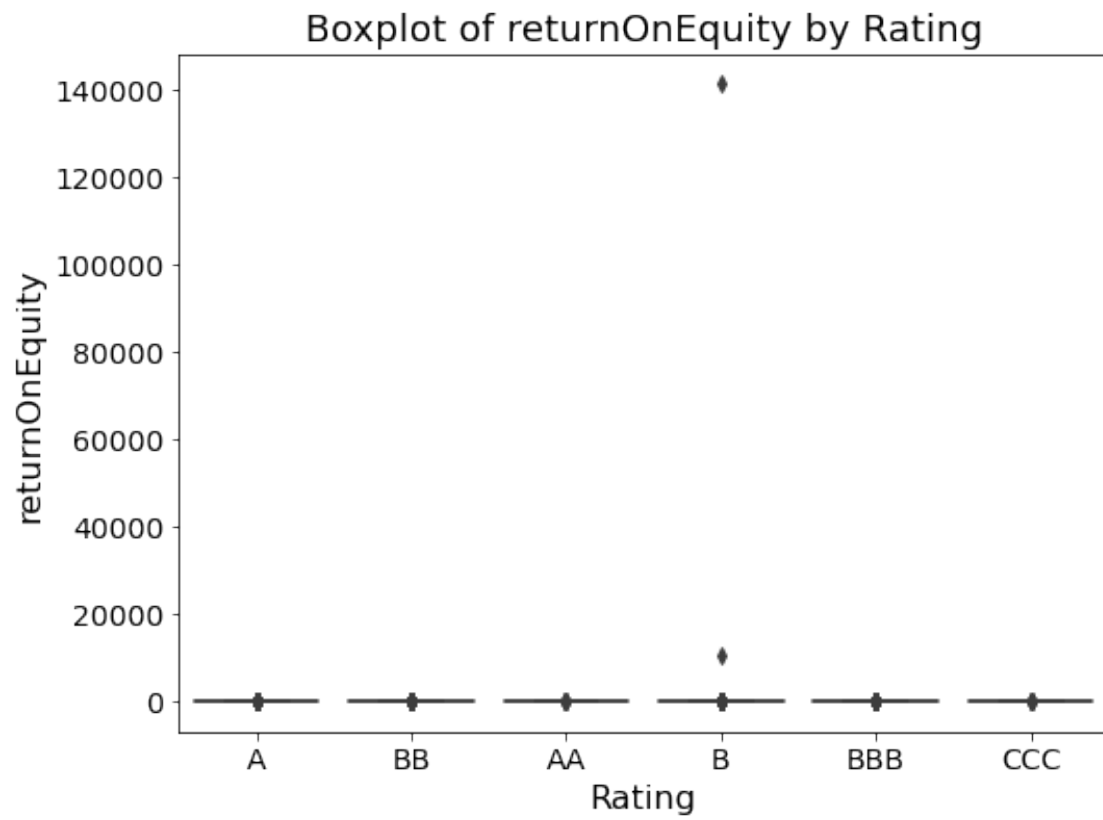


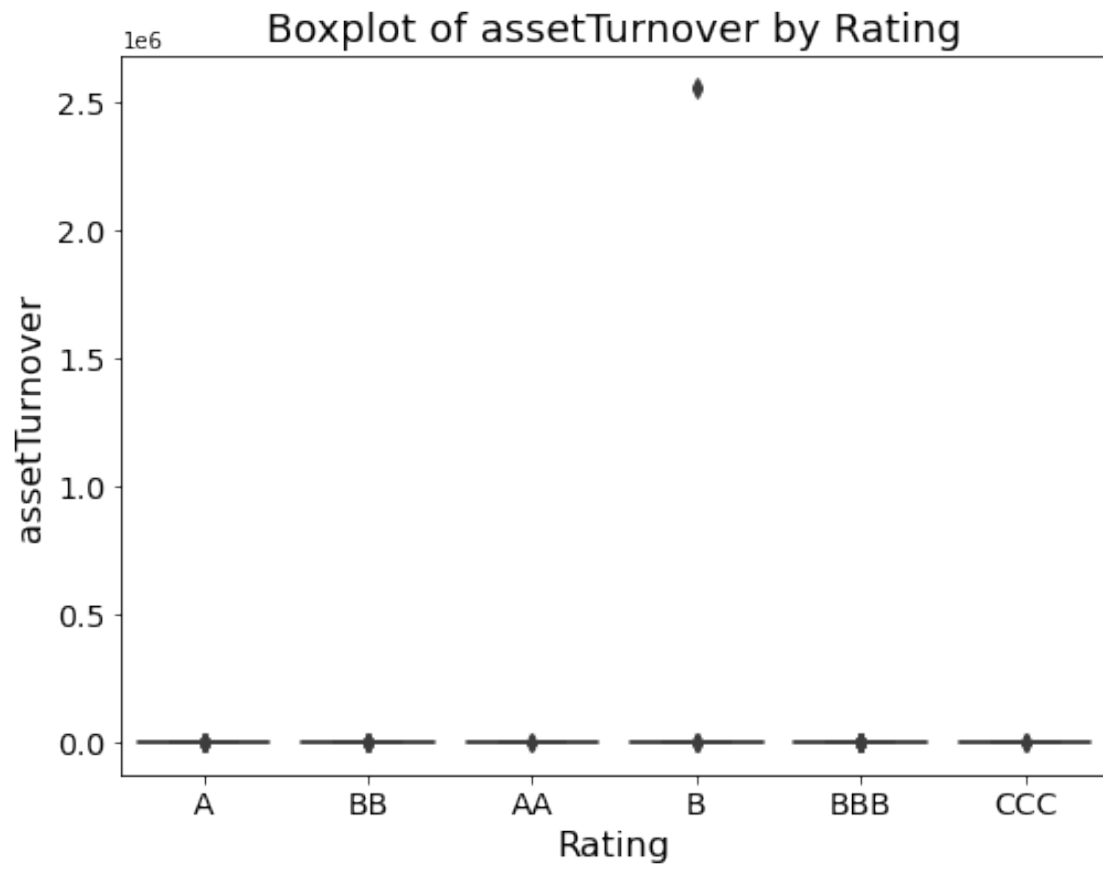


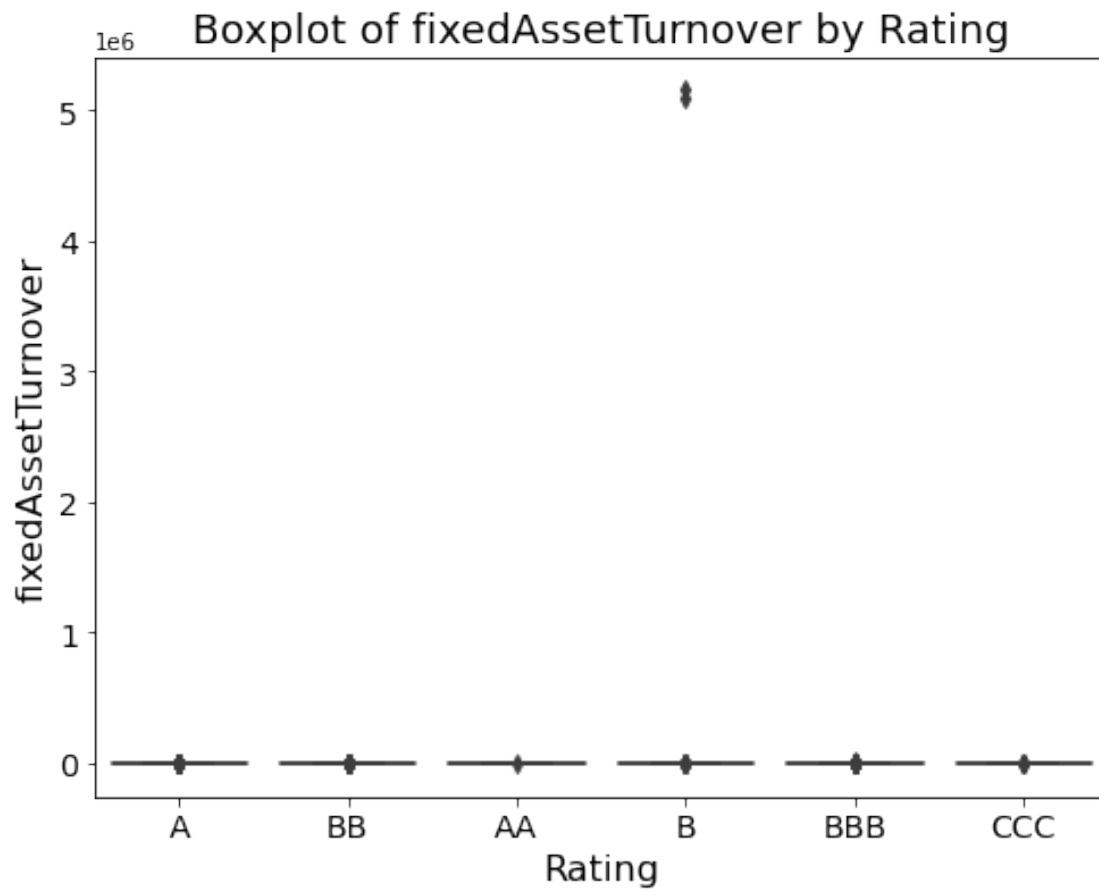


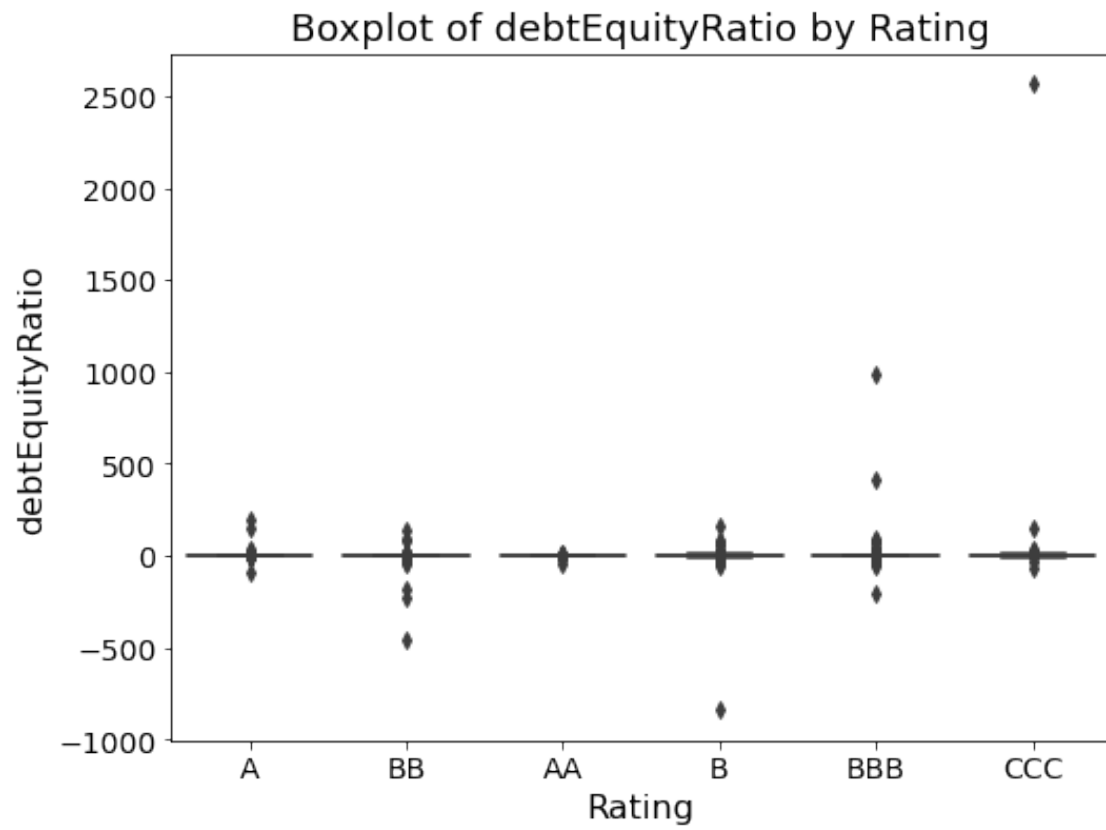


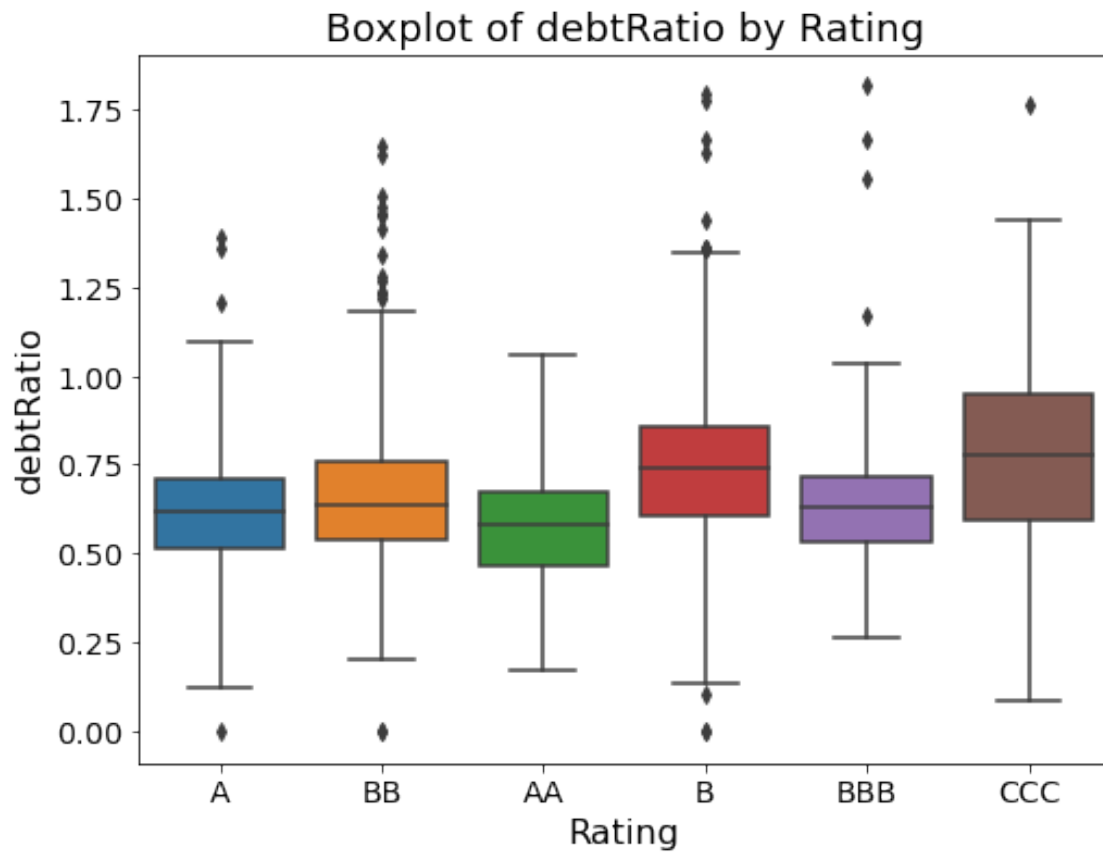


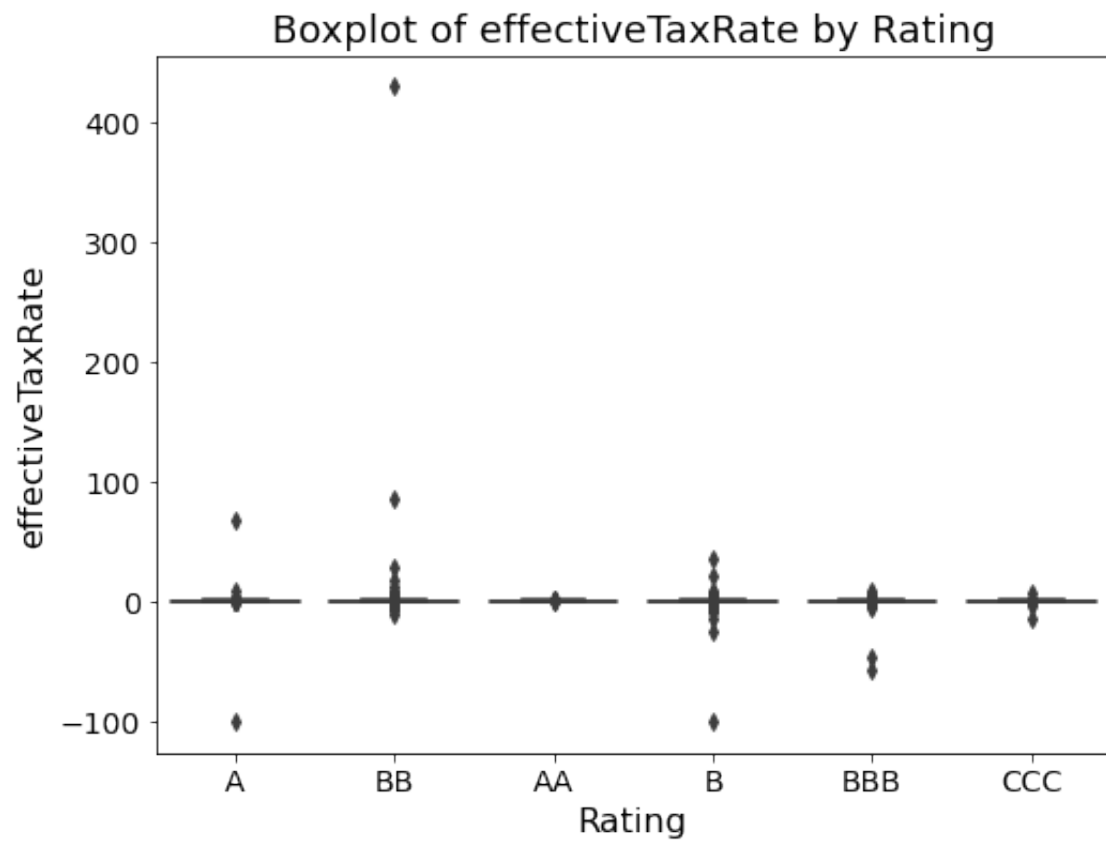


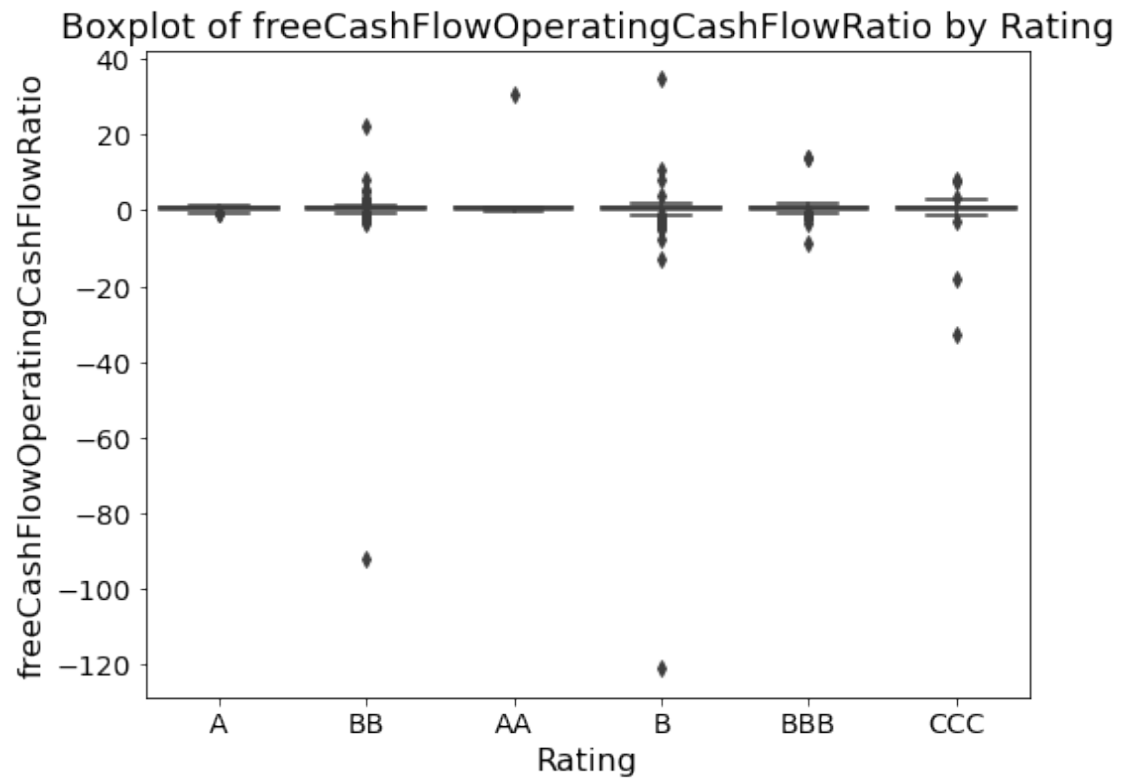


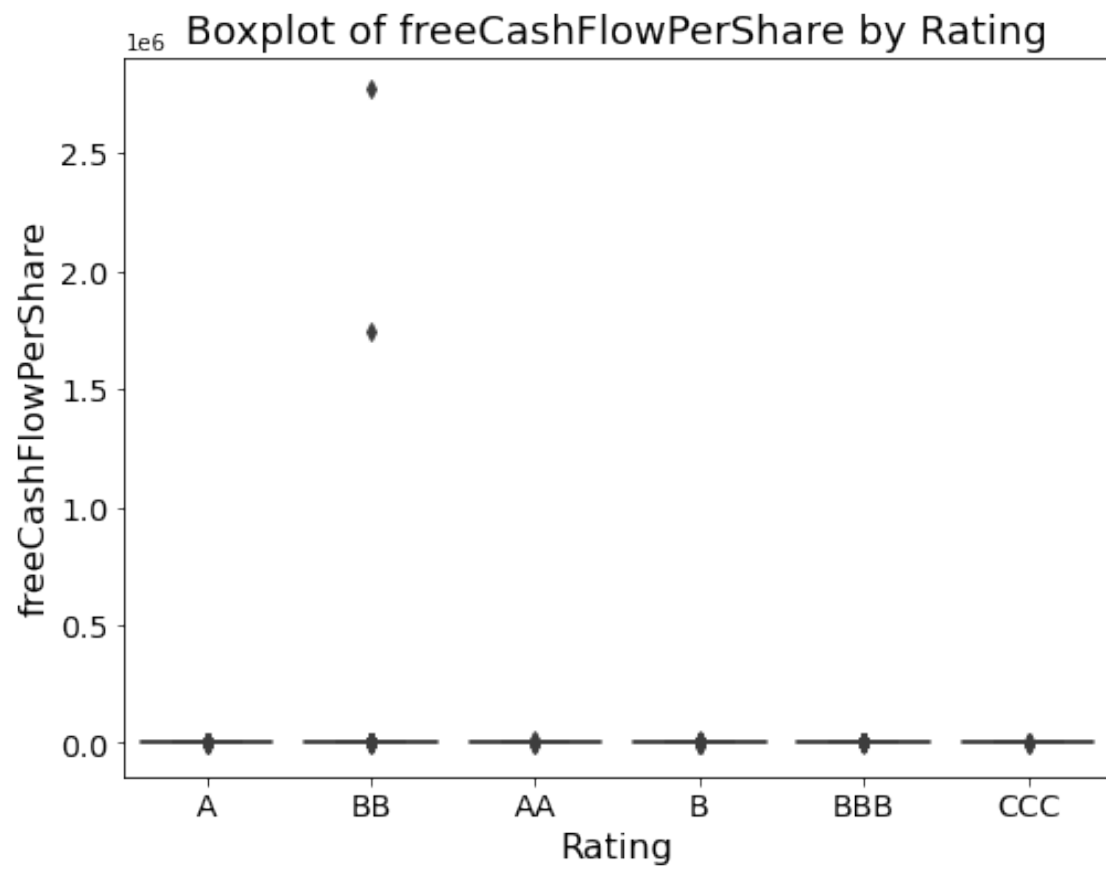


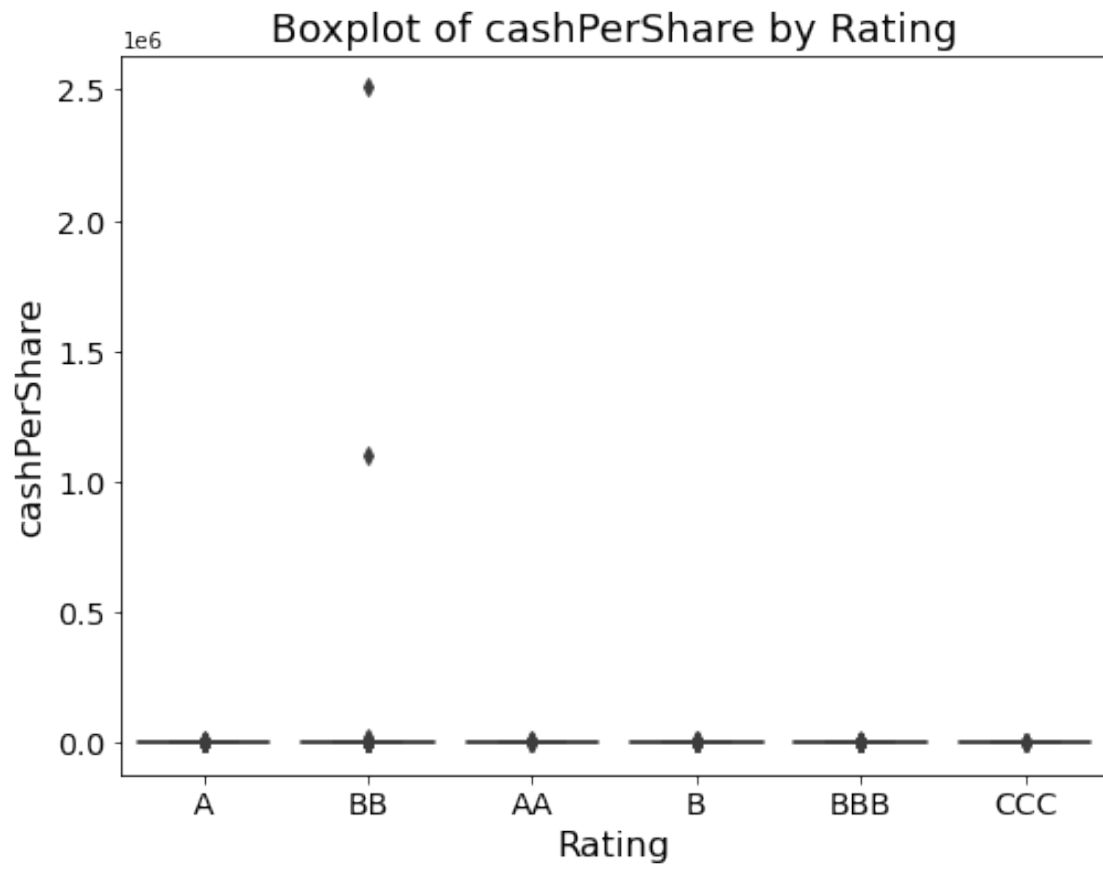


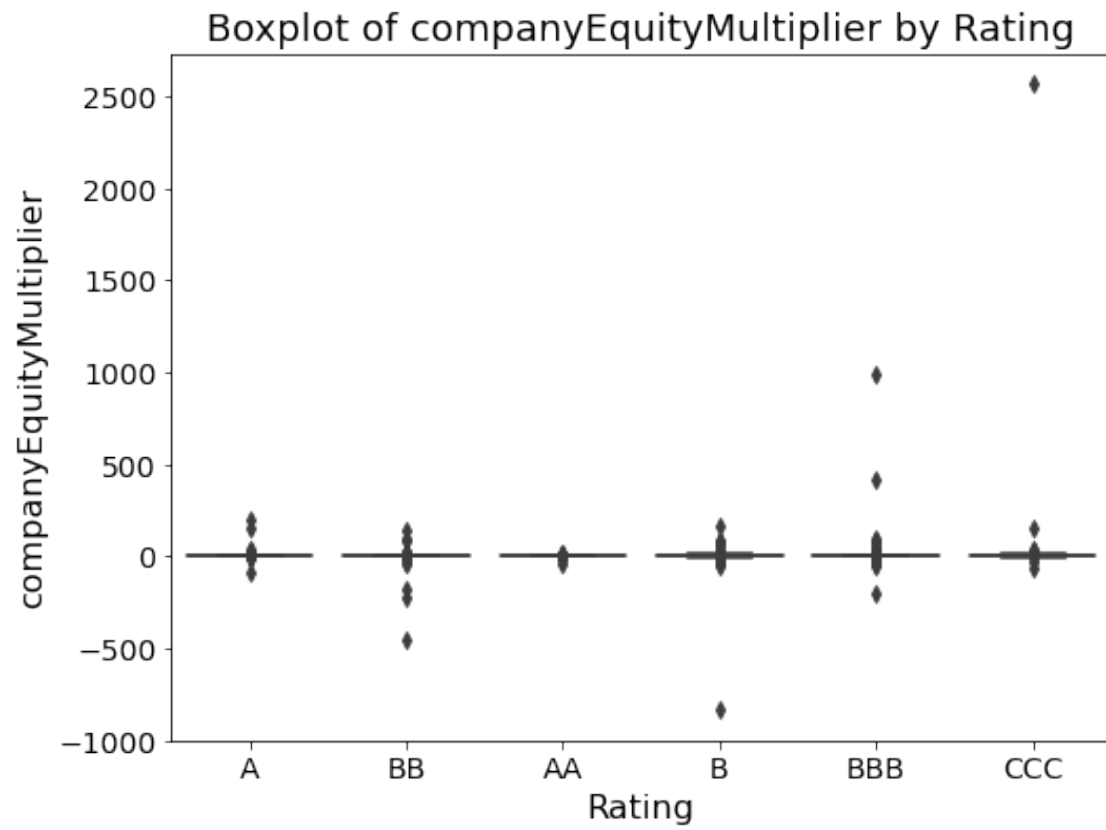


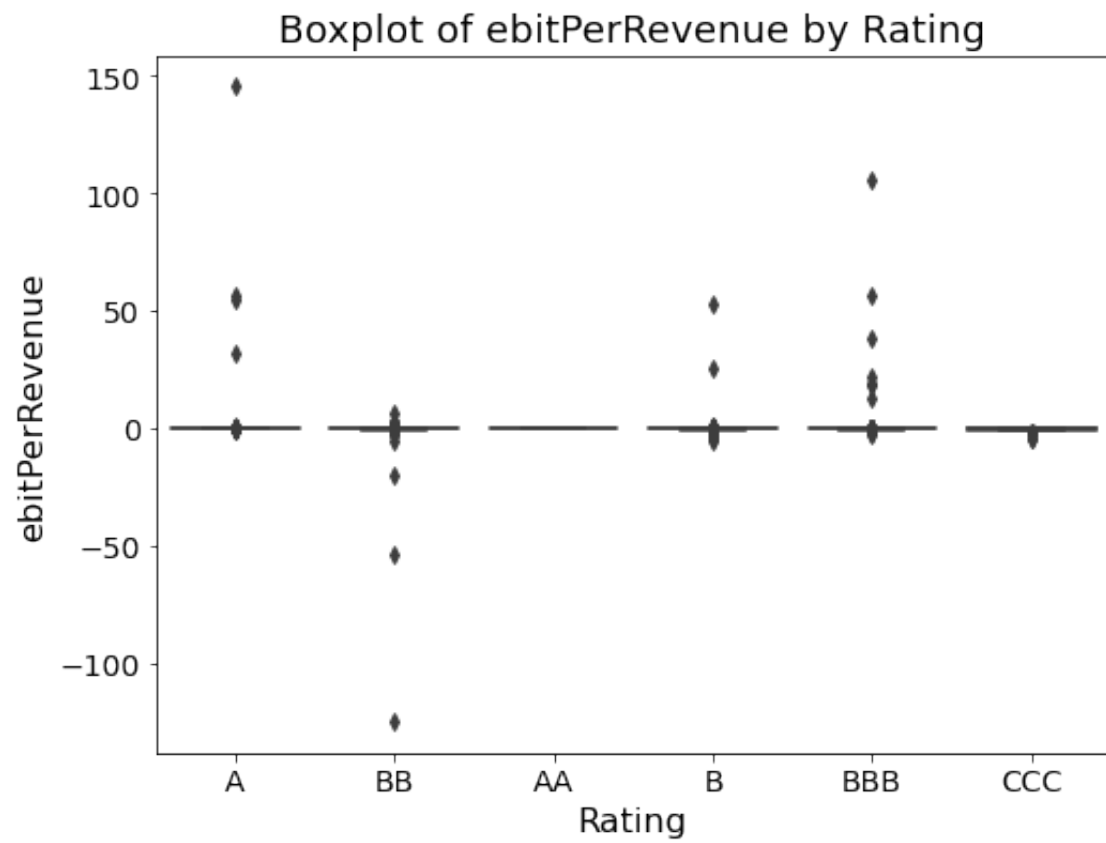


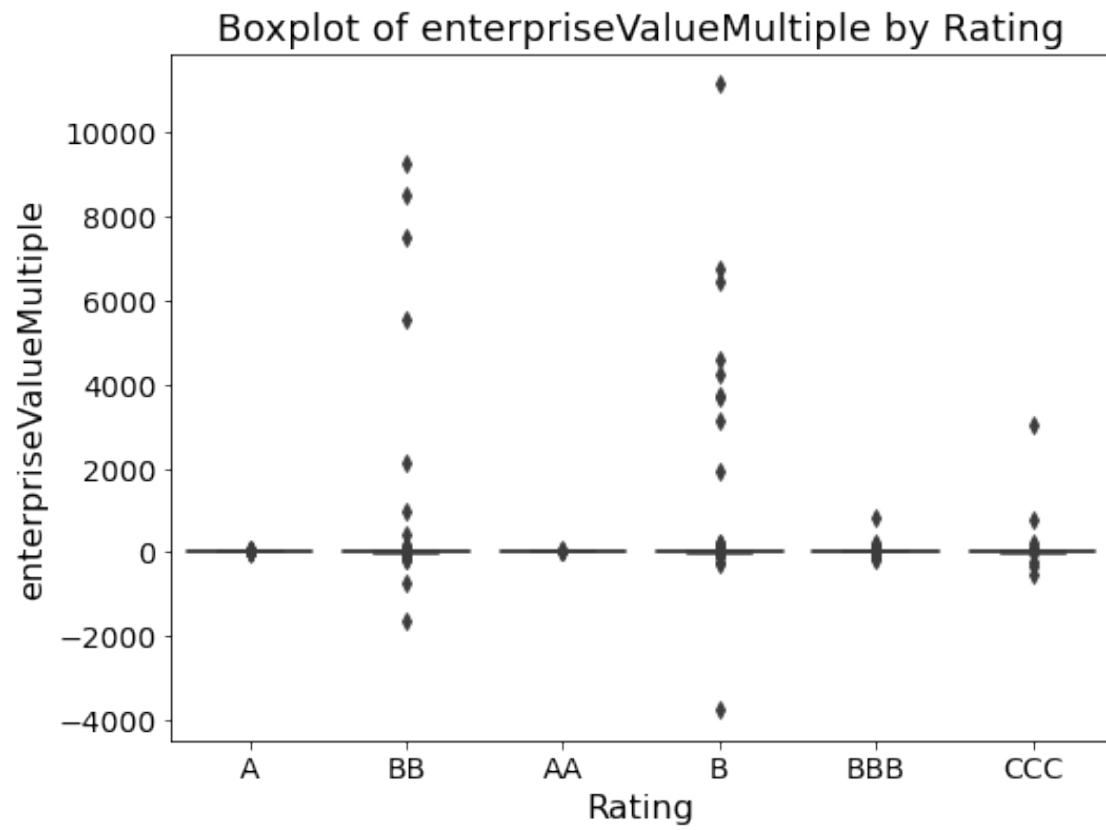


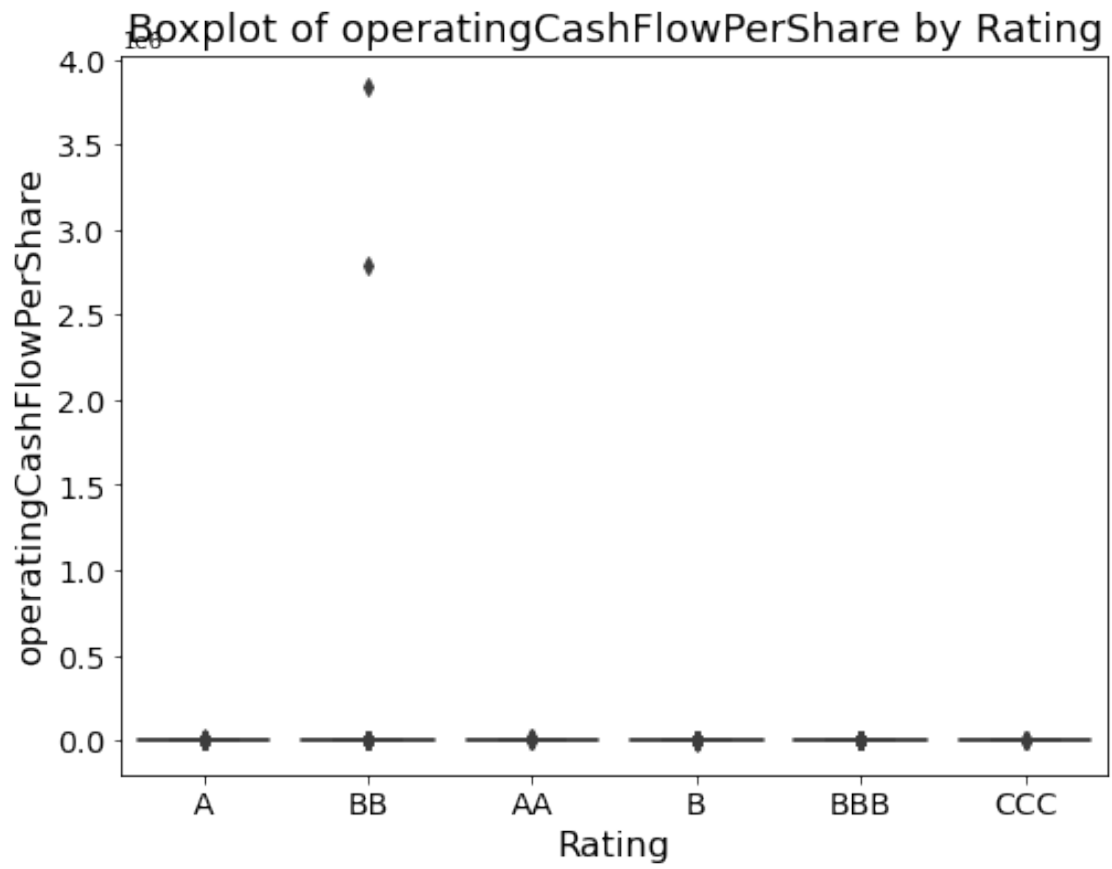


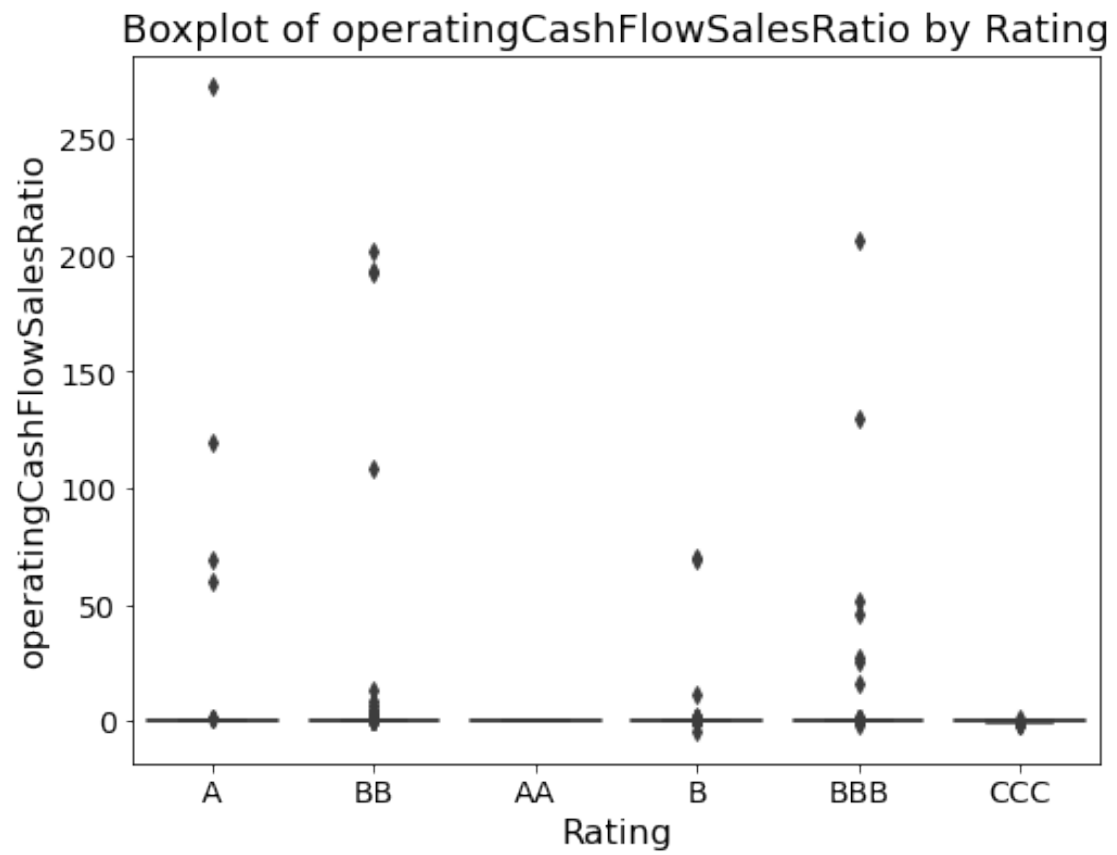


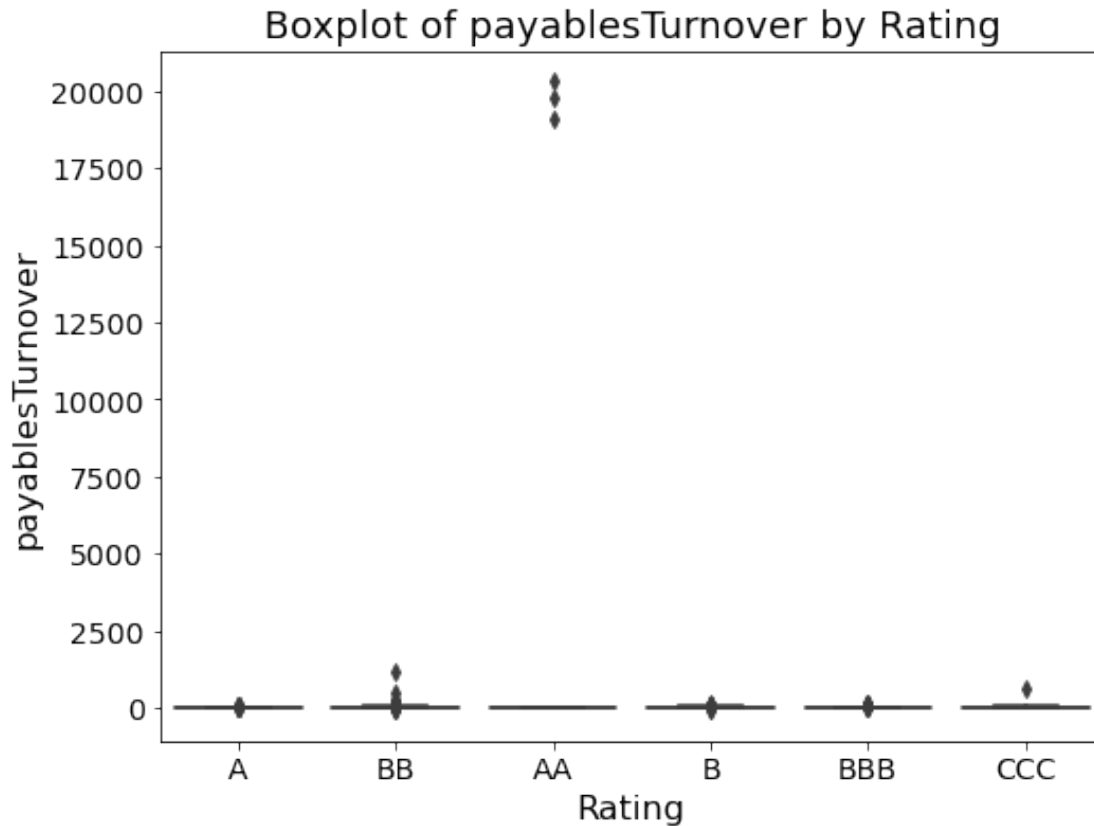












```
[272]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# def plot_histogram_distribution(data, col):

#     plt.figure(figsize=(12, 5))

#     # Histogram
#     plt.subplot(1, 2, 1)
#     sns.histplot(data[col], bins=30, kde=False, color='blue')
#     plt.title(f'Histogram of {col}')
#     plt.xlabel(col)
#     plt.ylabel('Frequency')

#     # Distribution Plot
#     plt.subplot(1, 2, 2)
#     sns.kdeplot(data[col], fill=True, color='red')
#     plt.title(f'Distribution Plot of {col}')
#     plt.xlabel(col)
```

```

# plt.tight_layout()
# plt.show()

def plot_histogram_distribution(data, col):
    plt.figure(figsize=(12, 5))

    # Histogram
    plt.subplot(1, 2, 1)
    sns.histplot(data[col], bins=30, kde=False, color='blue')
    plt.title(f'Histogram of {col}', fontsize=18) # Increased title font size
    plt.xlabel(col, fontsize=16) # Increased x-axis font size
    plt.ylabel('Frequency', fontsize=16) # Increased y-axis font size
    plt.xticks(fontsize=12)
    plt.yticks(fontsize=12)

    # Distribution Plot
    plt.subplot(1, 2, 2)
    sns.kdeplot(data[col], fill=True, color='red')
    plt.title(f'Distribution Plot of {col}', fontsize=18) # Increased title_
    plt.xlabel(col, fontsize=16) # Increased x-axis font size
    plt.ylabel('Density', fontsize=16)
    plt.xticks(fontsize=12)
    plt.yticks(fontsize=12)

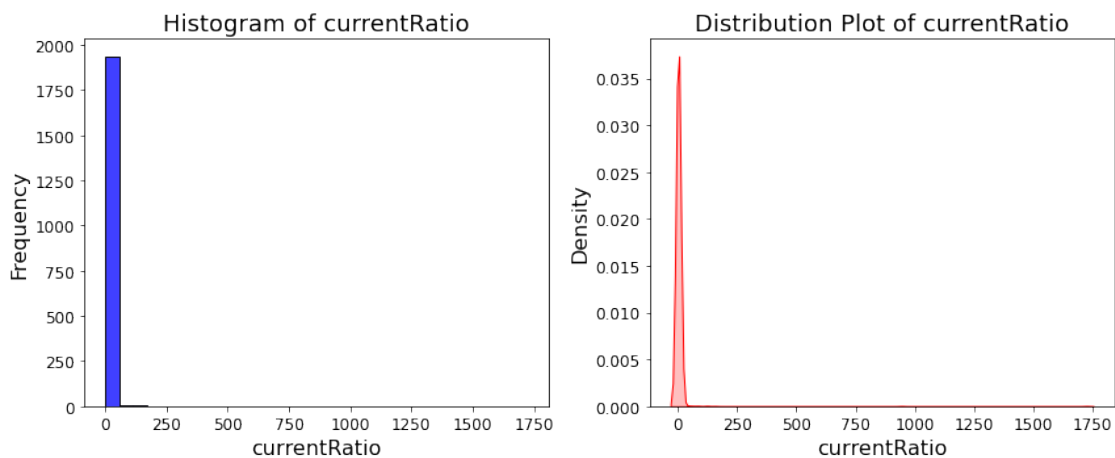
    plt.tight_layout()
    plt.show()

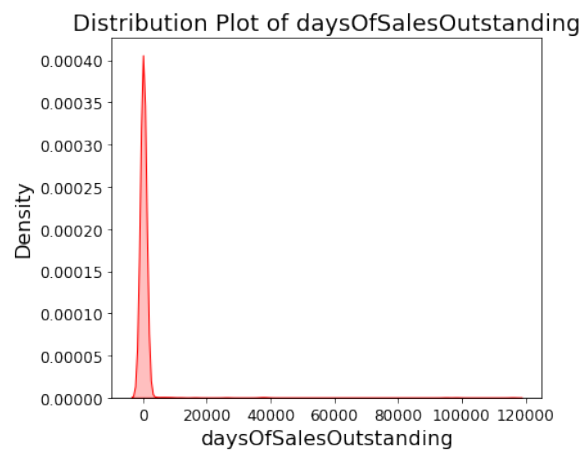
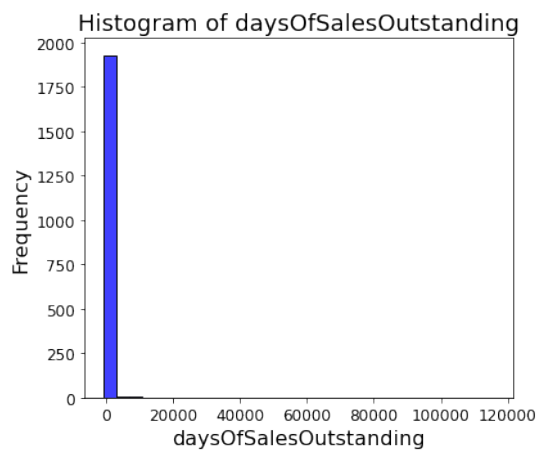
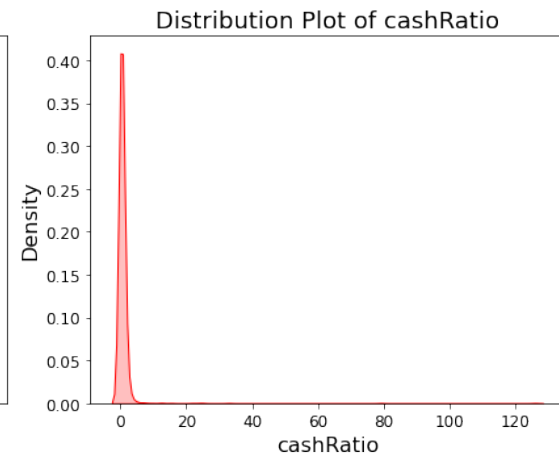
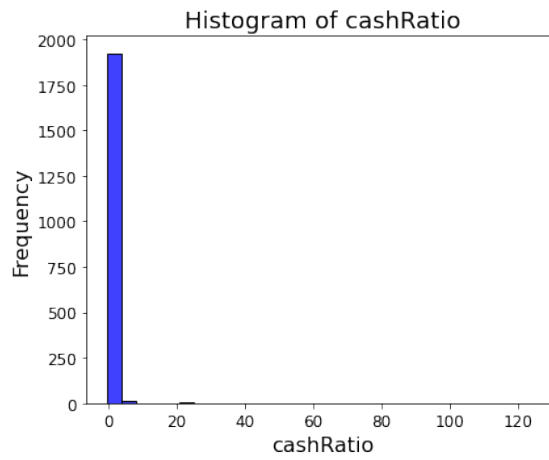
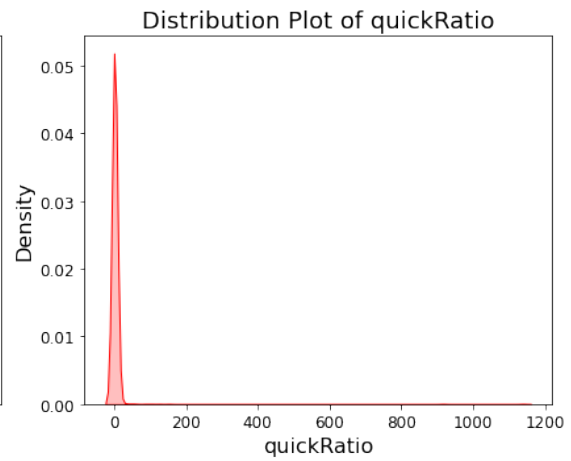
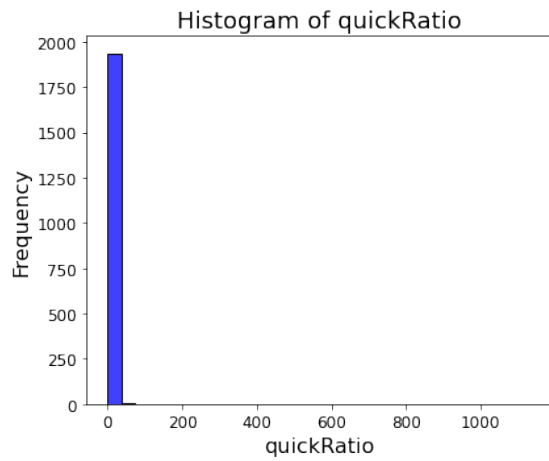
```

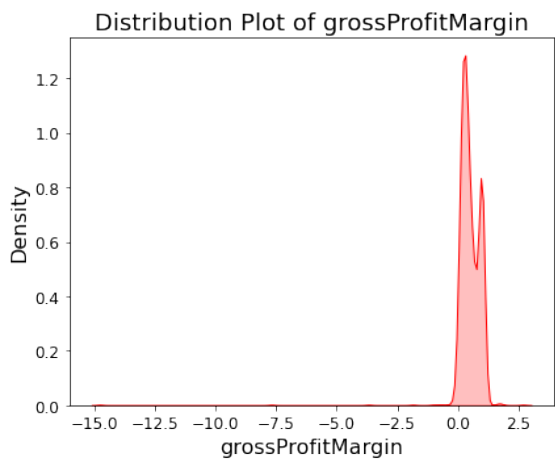
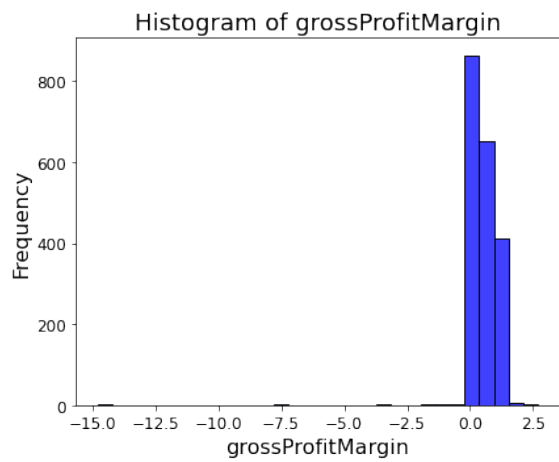
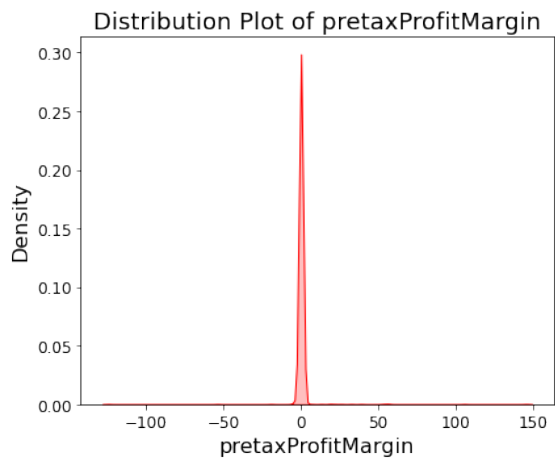
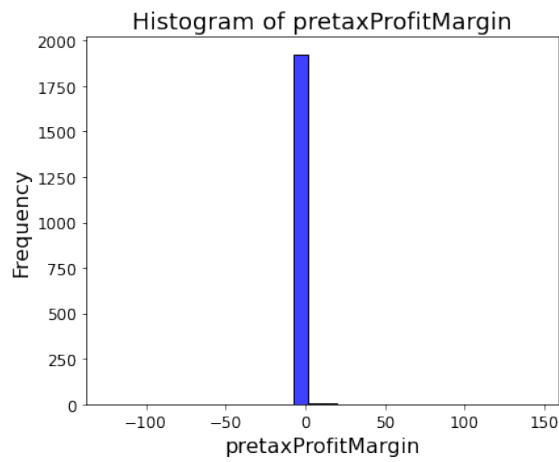
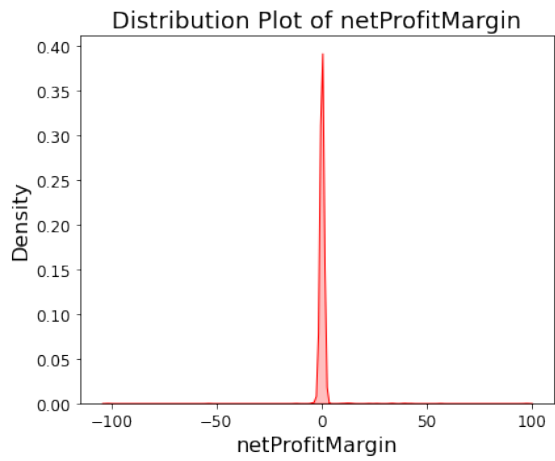
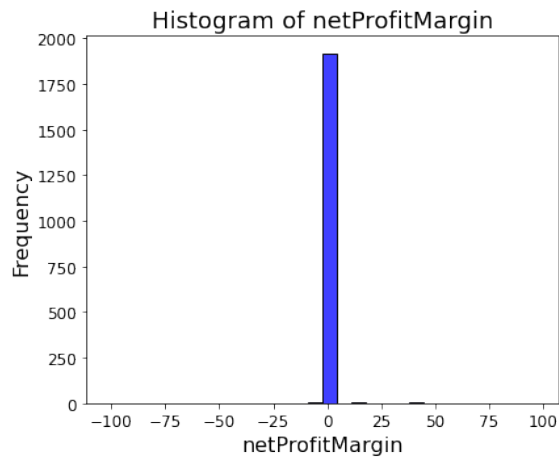
```

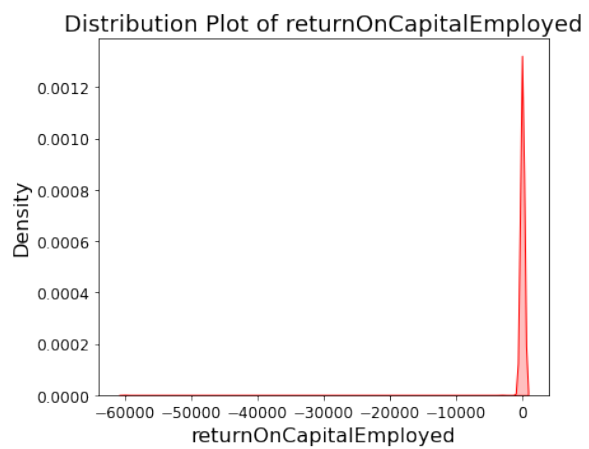
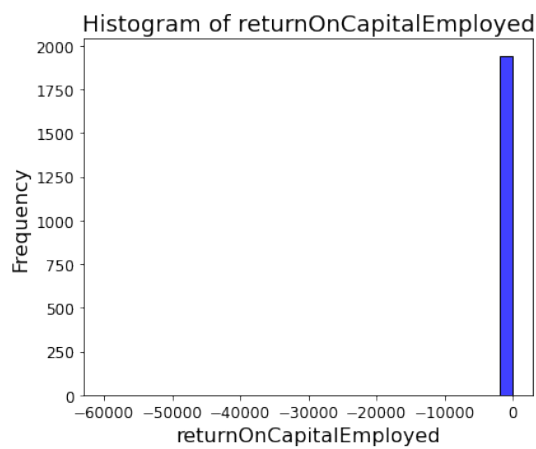
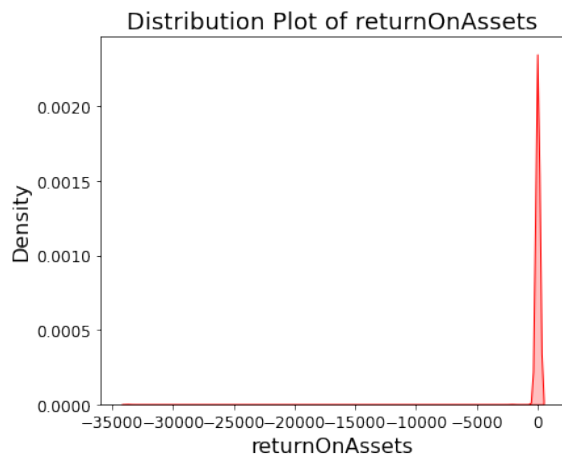
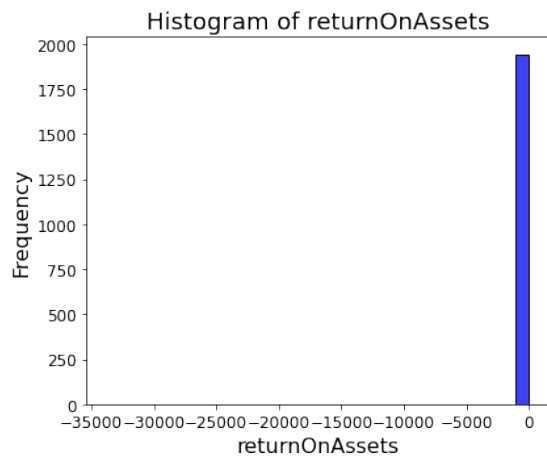
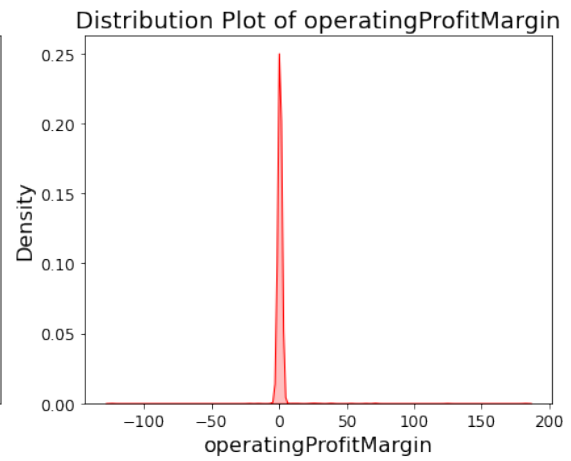
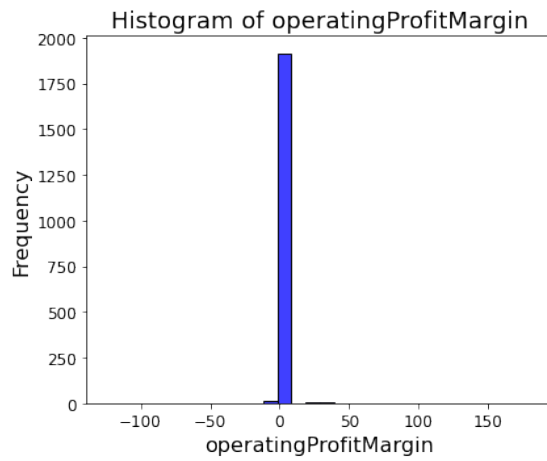
[273]: for pred in predictors:
        plot_histogram_distribution(credit_rating_new, pred)

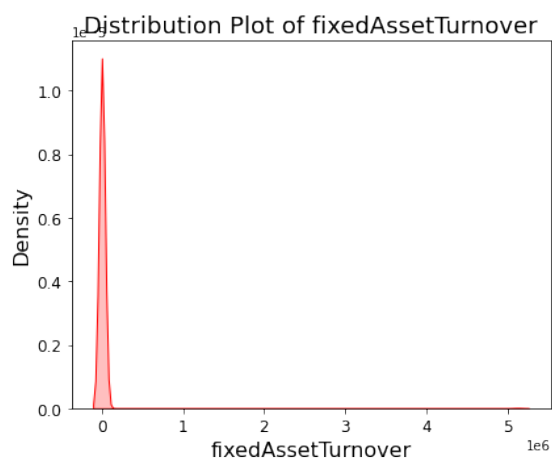
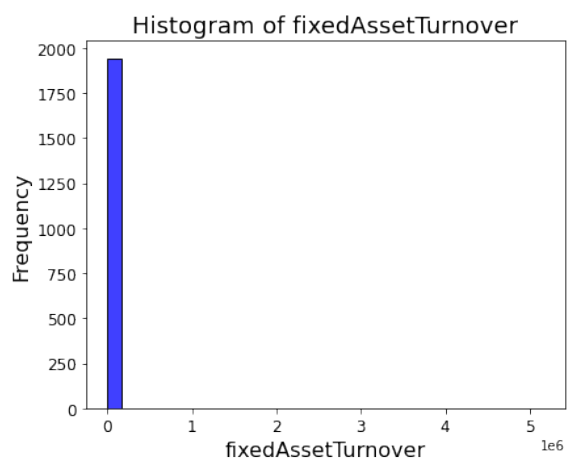
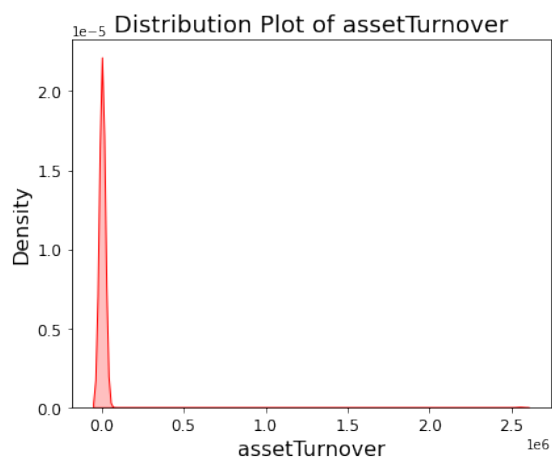
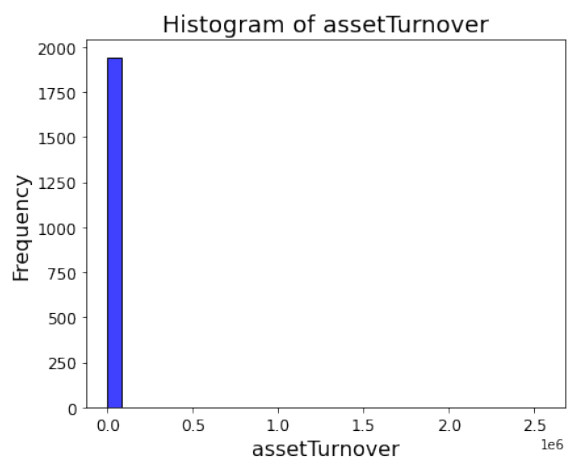
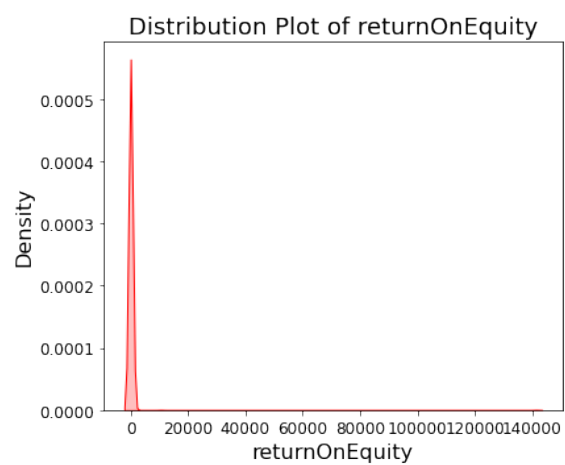
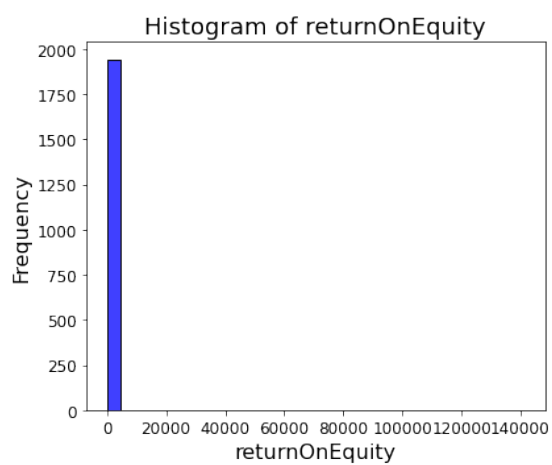
```

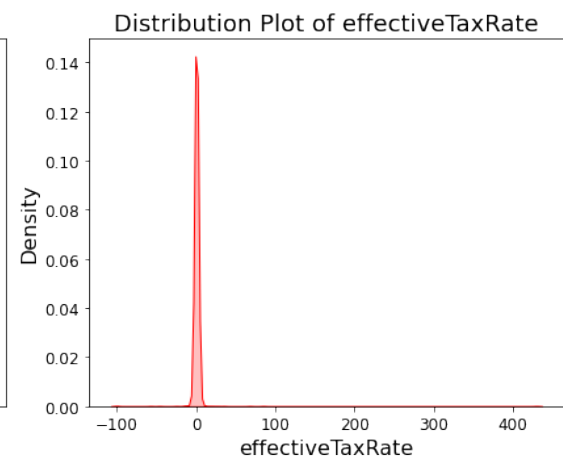
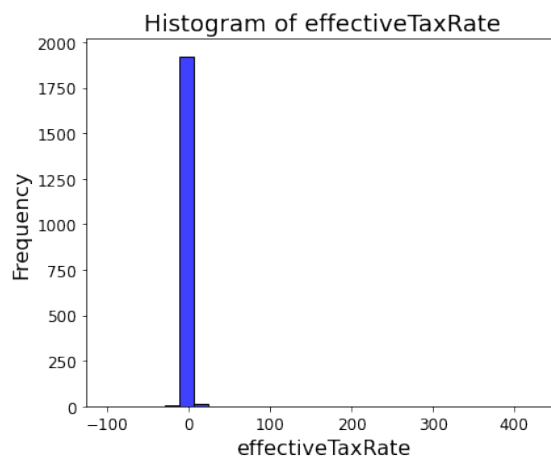
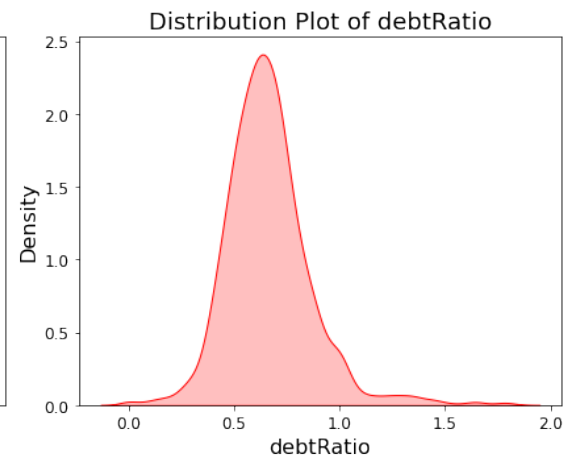
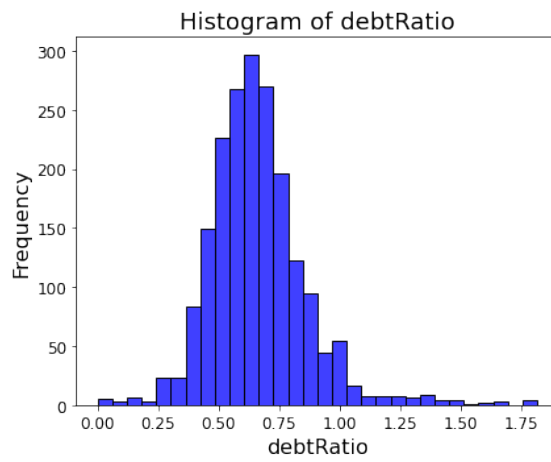
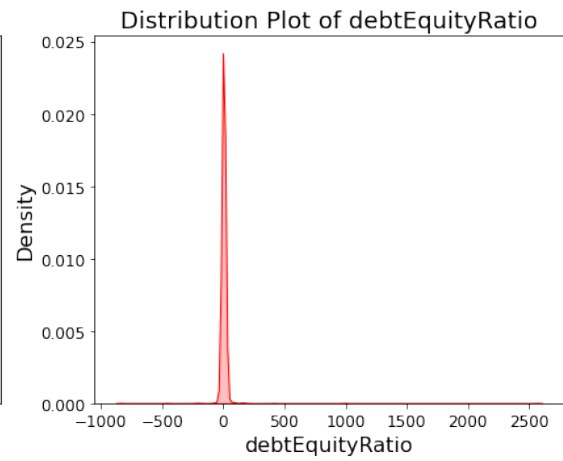
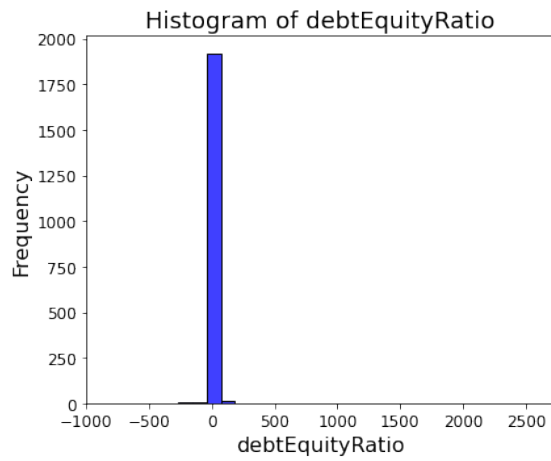




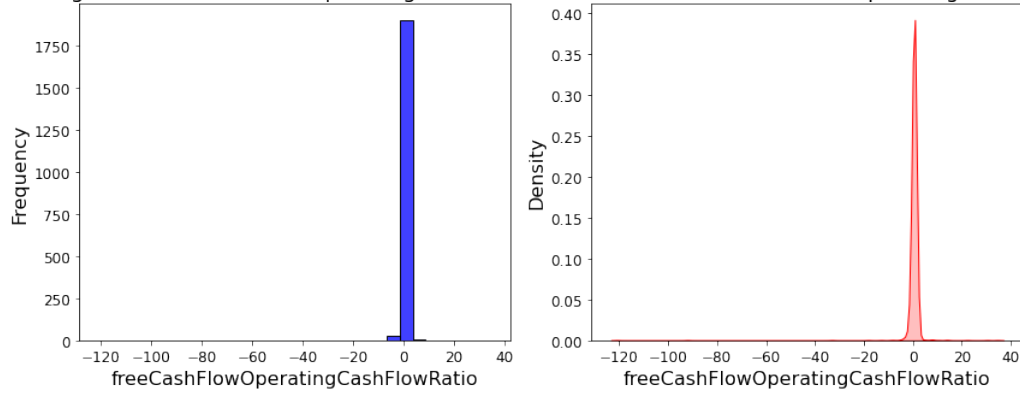




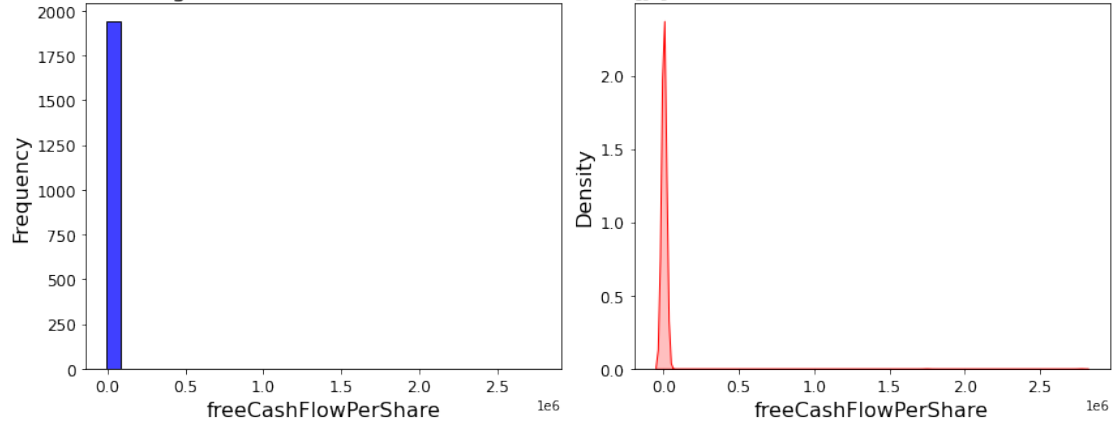




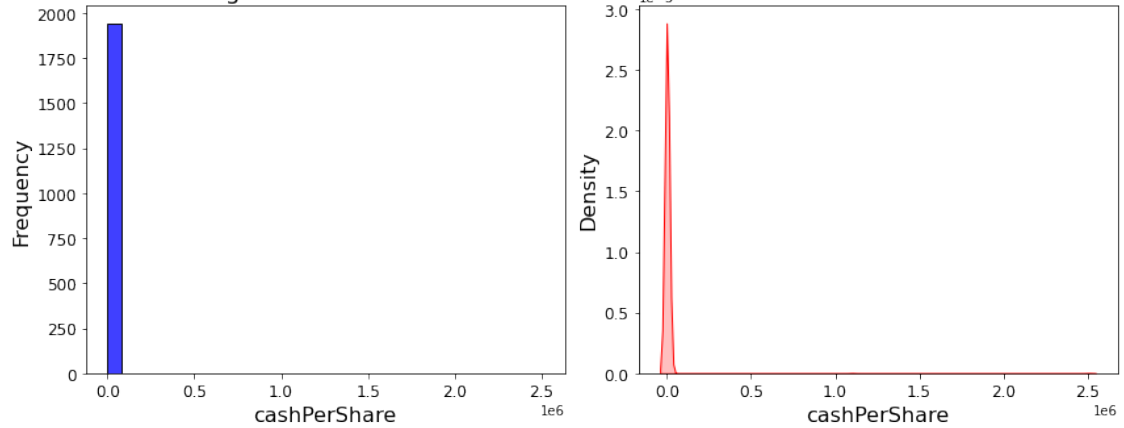
Histogram of freeCashFlowOperatingCashFlowRatio Distribution Plot of freeCashFlowOperatingCashFlowRatio

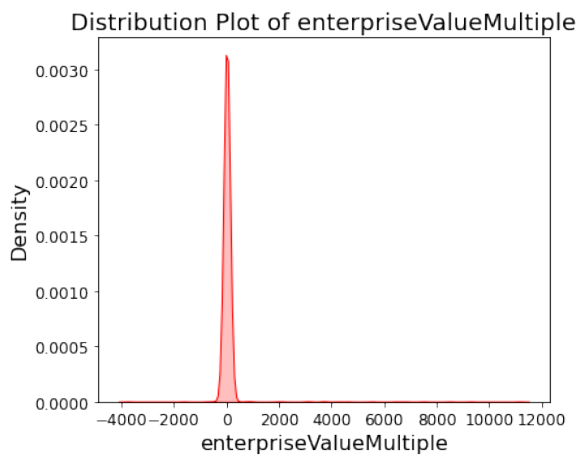
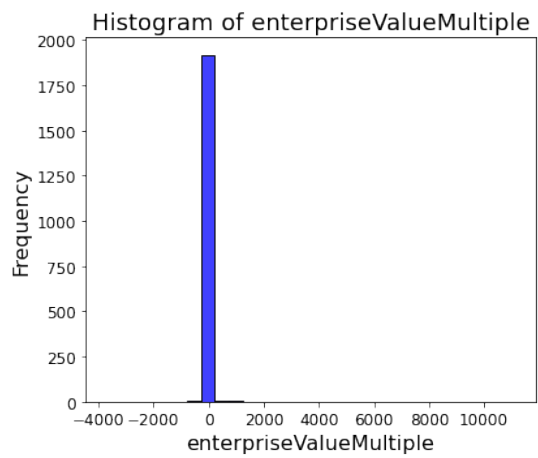
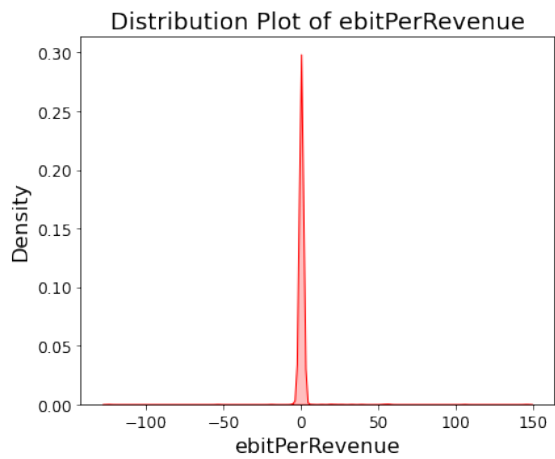
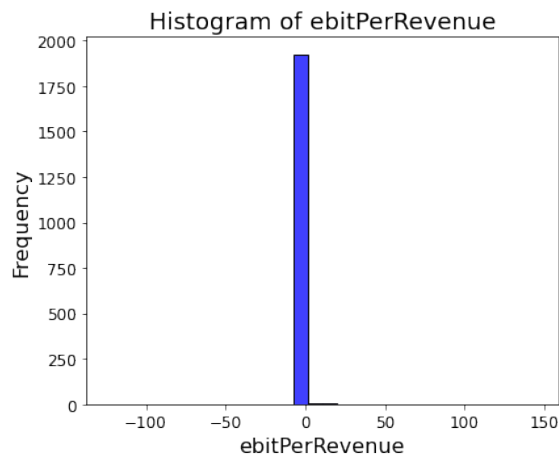
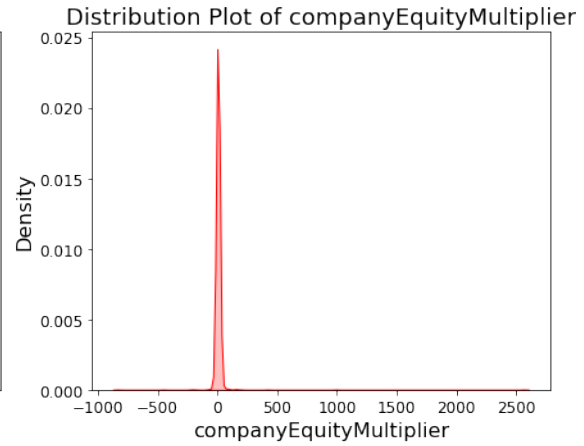
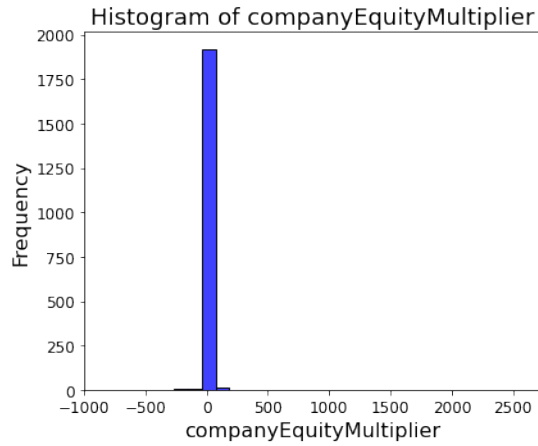


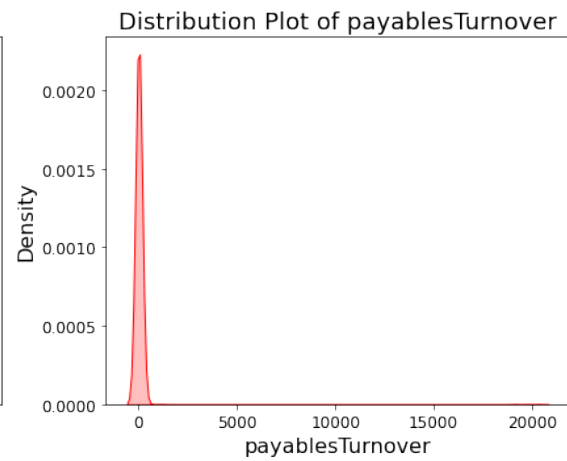
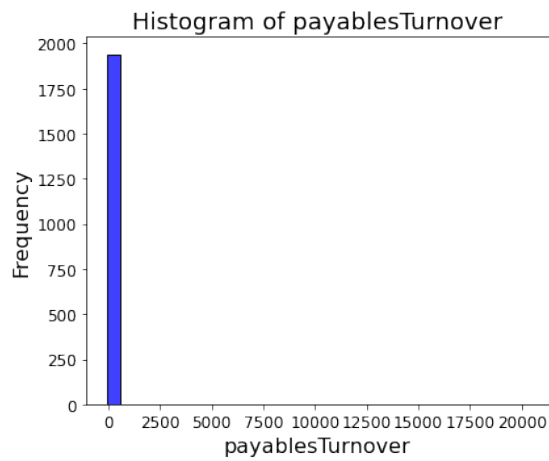
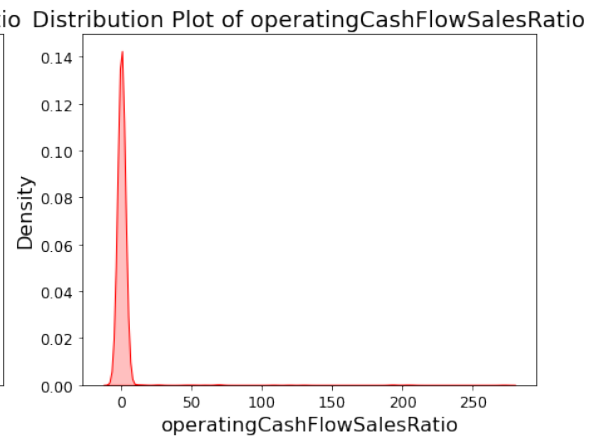
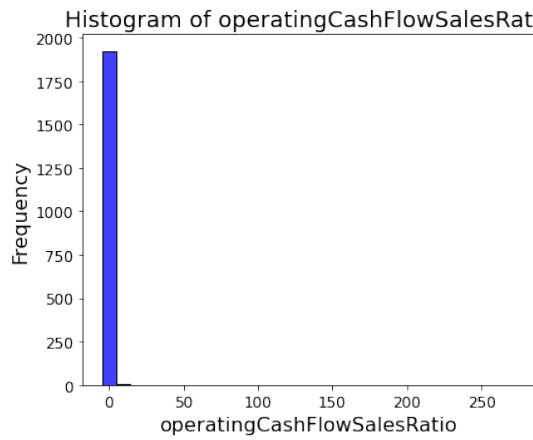
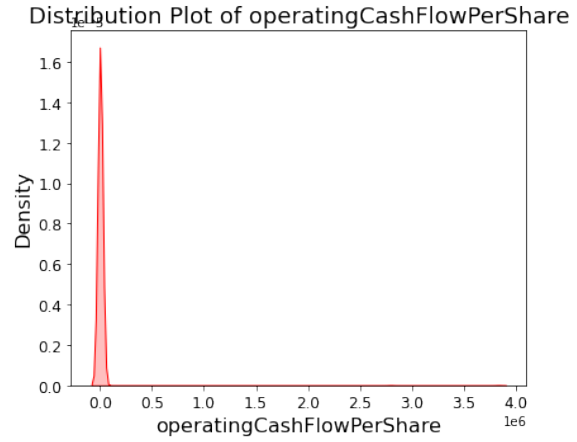
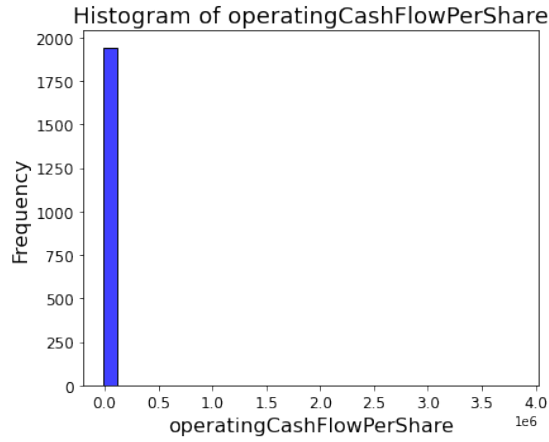
Histogram of freeCashFlowPerShare Distribution Plot of freeCashFlowPerShare



Histogram of cashPerShare Distribution Plot of cashPerShare







```
[274]: credit_rating_new[predictors].skew(axis=0).round(1)
```

```
[274]: currentRatio          33.6
      quickRatio           30.3
      cashRatio            26.7
      daysOfSalesOutstanding 21.2
      netProfitMargin       1.2
      pretaxProfitMargin    7.3
      grossProfitMargin     -14.3
      operatingProfitMargin 12.3
      returnOnAssets        -43.9
      returnOnCapitalEmployed -43.9
      returnOnEquity        43.8
      assetTurnover         31.2
      fixedAssetTurnover    31.2
      debtEquityRatio       28.0
      debtRatio            1.2
      effectiveTaxRate      31.7
      freeCashFlowOperatingCashFlowRatio -22.5
      freeCashFlowPerShare  33.4
      cashPerShare         36.7
      companyEquityMultiplier 28.0
      ebitPerRevenue        7.3
      enterpriseValueMultiple 13.7
      operatingCashFlowPerShare 32.3
      operatingCashFlowSalesRatio 15.6
      payablesTurnover      25.4
      dtype: float64
```

```
[275]: #Calculate skewness
      #It should return between 0 and 1 if a column is normally distributed.
      #Effect of outliers

      #credit_rating_new[predictors].skew(axis=0)

      pd.DataFrame(credit_rating_new[predictors].skew(axis=0).round(1),
        ↳ columns=['Skewness'])

      skewness_df = pd.DataFrame(credit_rating_new[predictors].skew(axis=0).round(1),
        ↳ columns=['Skewness'])
      skewness_df.to_csv("/Users/kamila/Desktop/tables pngs/skewness_table.csv",
        ↳ index=True)
```



```
#debtEquityRatio          0.267271
#debtRatio                1.208183
#companyEquityMultiplier  0.267356
```

```
[276]: def count_outliers(data, col):

        Q1 = data[col].quantile(0.25)
        Q3 = data[col].quantile(0.75)
        IQR = Q3 - Q1
        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR

        outliers = data[(data[col] < lower_bound) | (data[col] > upper_bound)]

        num_outliers = len(outliers)
        percentage_outliers = (num_outliers / len(data)) * 100

        print(f"Number of outliers in {col} : {num_outliers} ({percentage_outliers:.
→2f}%)")
        #print(f"Percentage of outliers in {col} : {percentage_outliers:.2f}%)")
```

```
[277]: def count_outliers2(data, cols):
        outlier_summary = []

        for col in cols:
            Q1 = data[col].quantile(0.25)
            Q3 = data[col].quantile(0.75)
            IQR = Q3 - Q1
            lower_bound = Q1 - 1.5 * IQR
            upper_bound = Q3 + 1.5 * IQR

            outliers = data[(data[col] < lower_bound) | (data[col] > upper_bound)]

            num_outliers = len(outliers)
            percentage_outliers = (num_outliers / len(data)) * 100

            outlier_summary.append({'Column': col, 'Num Outliers': num_outliers,
→'Percentage Outliers': round(percentage_outliers, 2)})

        return pd.DataFrame(outlier_summary)
```

```
[278]: for pred in predictors:
        count_outliers(credit_rating_new, pred)
```

```
Number of outliers in currentRatio : 126 (6.48%)
Number of outliers in quickRatio : 163 (8.38%)
Number of outliers in cashRatio : 135 (6.94%)
```

Number of outliers in daysOfSalesOutstanding : 80 (4.11%)
 Number of outliers in netProfitMargin : 231 (11.88%)
 Number of outliers in pretaxProfitMargin : 228 (11.72%)
 Number of outliers in grossProfitMargin : 10 (0.51%)
 Number of outliers in operatingProfitMargin : 185 (9.51%)
 Number of outliers in returnOnAssets : 164 (8.43%)
 Number of outliers in returnOnCapitalEmployed : 143 (7.35%)
 Number of outliers in returnOnEquity : 302 (15.53%)
 Number of outliers in assetTurnover : 104 (5.35%)
 Number of outliers in fixedAssetTurnover : 144 (7.40%)
 Number of outliers in debtEquityRatio : 267 (13.73%)
 Number of outliers in debtRatio : 78 (4.01%)
 Number of outliers in effectiveTaxRate : 235 (12.08%)
 Number of outliers in freeCashFlowOperatingCashFlowRatio : 116 (5.96%)
 Number of outliers in freeCashFlowPerShare : 186 (9.56%)
 Number of outliers in cashPerShare : 197 (10.13%)
 Number of outliers in companyEquityMultiplier : 266 (13.68%)
 Number of outliers in ebitPerRevenue : 219 (11.26%)
 Number of outliers in enterpriseValueMultiple : 220 (11.31%)
 Number of outliers in operatingCashFlowPerShare : 157 (8.07%)
 Number of outliers in operatingCashFlowSalesRatio : 156 (8.02%)
 Number of outliers in payablesTurnover : 113 (5.81%)

```
[279]: count_outliers2(credit_rating_new, predictors).sort_values(by='Num Outliers',
↪ascending=False)
```

```
[279]:
```

	Column	Num Outliers	Percentage Outliers
10	returnOnEquity	302	15.53
13	debtEquityRatio	267	13.73
19	companyEquityMultiplier	266	13.68
15	effectiveTaxRate	235	12.08
4	netProfitMargin	231	11.88
5	pretaxProfitMargin	228	11.72
21	enterpriseValueMultiple	220	11.31
20	ebitPerRevenue	219	11.26
18	cashPerShare	197	10.13
17	freeCashFlowPerShare	186	9.56
7	operatingProfitMargin	185	9.51
8	returnOnAssets	164	8.43
1	quickRatio	163	8.38
22	operatingCashFlowPerShare	157	8.07
23	operatingCashFlowSalesRatio	156	8.02
12	fixedAssetTurnover	144	7.40
9	returnOnCapitalEmployed	143	7.35
2	cashRatio	135	6.94
0	currentRatio	126	6.48
16	freeCashFlowOperatingCashFlowRatio	116	5.96

24	payablesTurnover	113	5.81
11	assetTurnover	104	5.35
3	daysOfSalesOutstanding	80	4.11
14	debtRatio	78	4.01
6	grossProfitMargin	10	0.51

```
[280]: outl_df = count_outliers2(credit_rating_new, predictors).sort_values(by='Num_
↳Outliers', ascending=False)
outl_df.to_csv("/Users/kamila/Desktop/tables pngs/outliers_table.csv",
↳index=True)
```

```
[281]: def count_row_outliers(data):

    outlier_counts = pd.Series(0, index=data.index)

    for col in data.columns:
        Q1 = data[col].quantile(0.25)
        Q3 = data[col].quantile(0.75)
        IQR = Q3 - Q1
        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR

        outlier_mask = (data[col] < lower_bound) | (data[col] > upper_bound)
        outlier_counts += outlier_mask.astype(int)

    return outlier_counts
```

```
[282]: outliers=pd.DataFrame(count_row_outliers(credit_rating_new[predictors]),
↳columns=['number of outliers'])
outliers
```

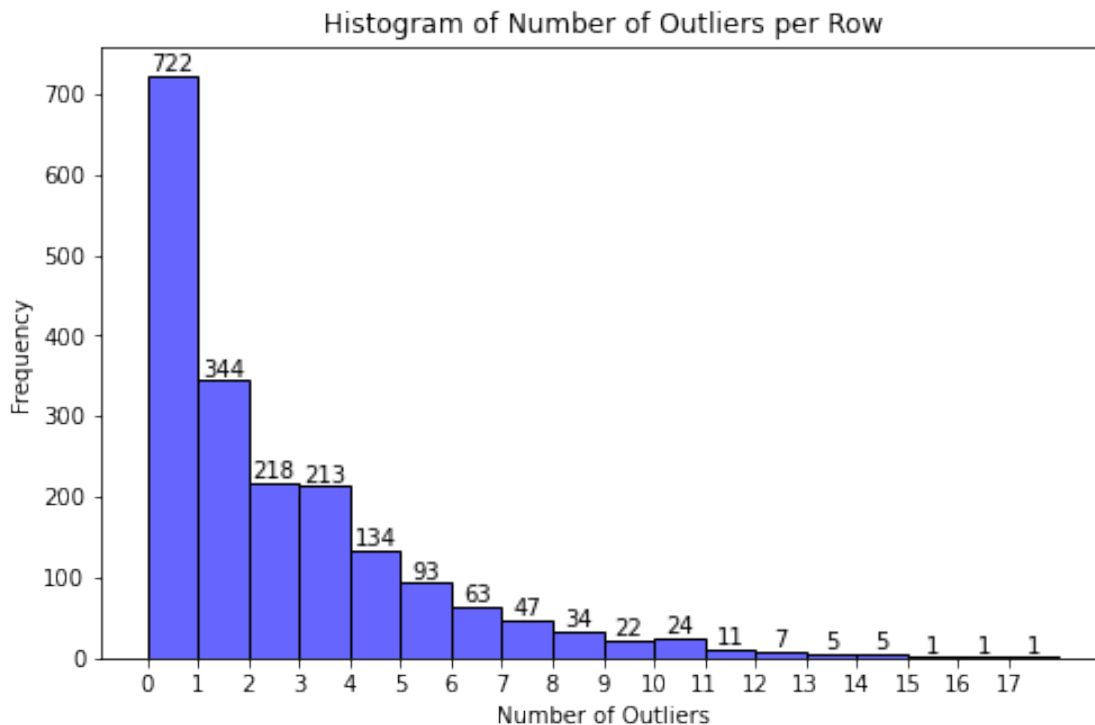
```
[282]:      number of outliers
466                0
704                2
1648               1
1762               1
1720               4
...                ...
1129               3
1478               1
741                7
1622               1
1968               0

[1945 rows x 1 columns]
```

```
[283]: outliers=count_row_outliers(credit_rating_new[predictors])
plt.figure(figsize=(8, 5))
ax = sns.histplot(outliers, bins=range(outliers.max() + 2), color='blue',
    ↪alpha=0.6)

for p in ax.patches:
    height = p.get_height()
    if height > 0:
        ax.annotate(f'{int(height)}', (p.get_x() + p.get_width() / 2., height),
            ha='center', va='bottom', fontsize=10, color='black')

plt.xticks(range(outliers.max() + 1))
plt.title('Histogram of Number of Outliers per Row')
plt.xlabel('Number of Outliers')
plt.ylabel('Frequency')
plt.show()
```



```
[284]: import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

def draw_boxplots_without_outliers(data, predictors, class_col):
```

```

# Loop through each predictor column
for predictor in predictors:
    # Calculate Q1, Q3, and IQR for each predictor
    Q1 = data[predictor].quantile(0.25)
    Q3 = data[predictor].quantile(0.75)
    IQR = Q3 - Q1

    # Define the outlier bounds
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    # Filter out the outliers
    filtered_data = data[(data[predictor] >= lower_bound) &
↳(data[predictor] <= upper_bound)]

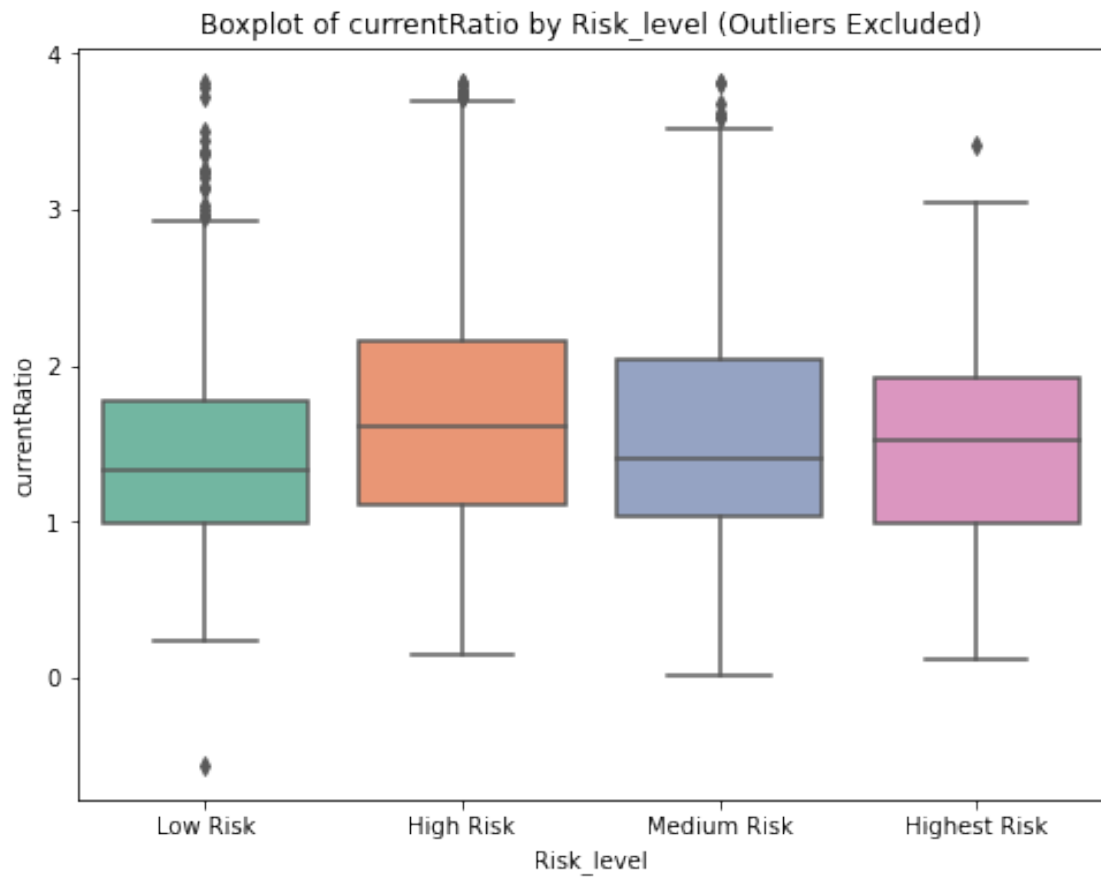
    # Create the boxplot for the predictor column with the category column
    plt.figure(figsize=(8, 6))
    sns.boxplot(x=class_col, y=predictor, data=filtered_data,
↳palette='Set2')

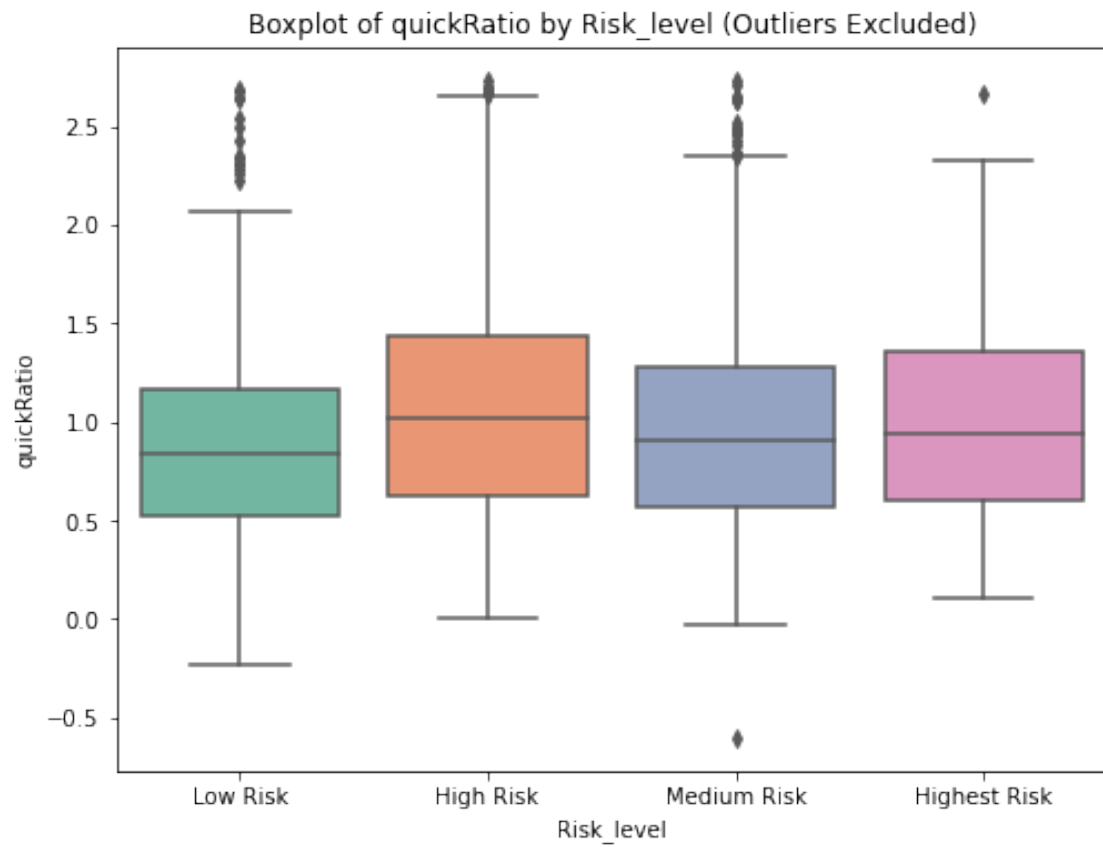
    # Set labels and title
    plt.xlabel(class_col)
    plt.ylabel(predictor)
    plt.title(f"Boxplot of {predictor} by {class_col} (Outliers Excluded)")

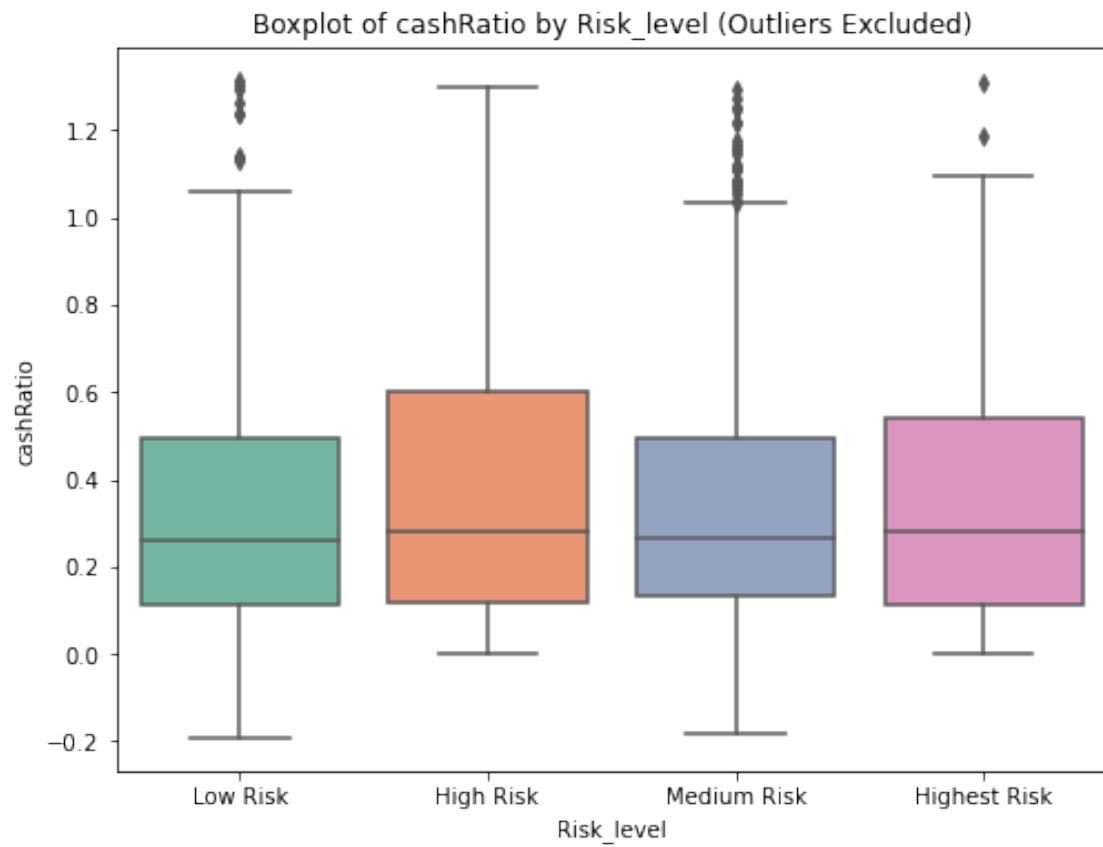
    # Show the plot
    plt.show()

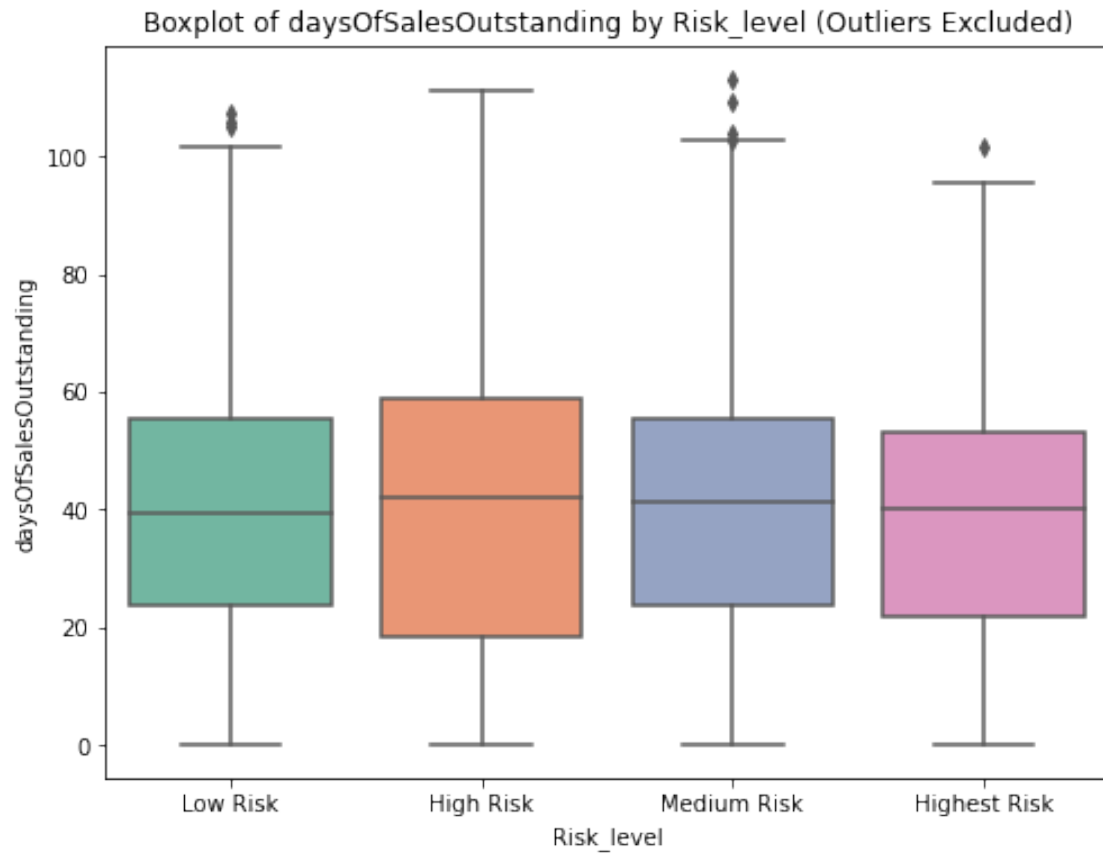
```

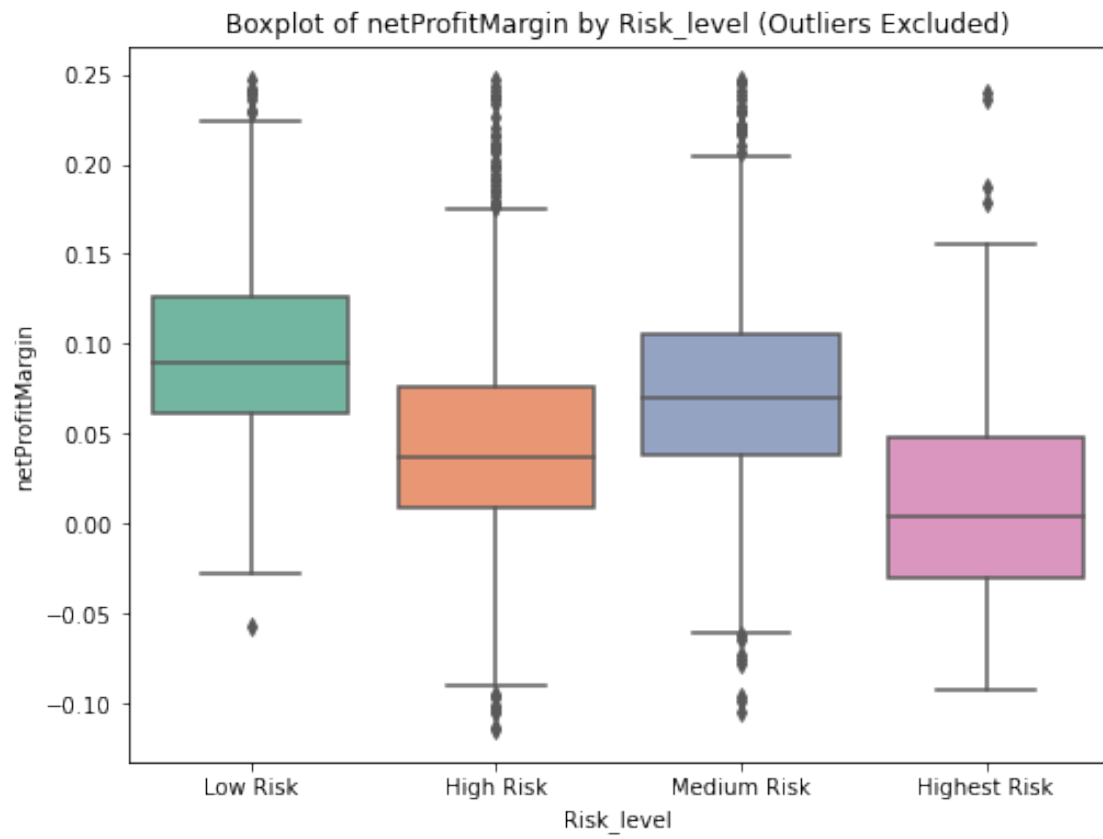
```
[285]: draw_boxplots_without_outliers(credit_rating_new, predictors, 'Risk_level')
```

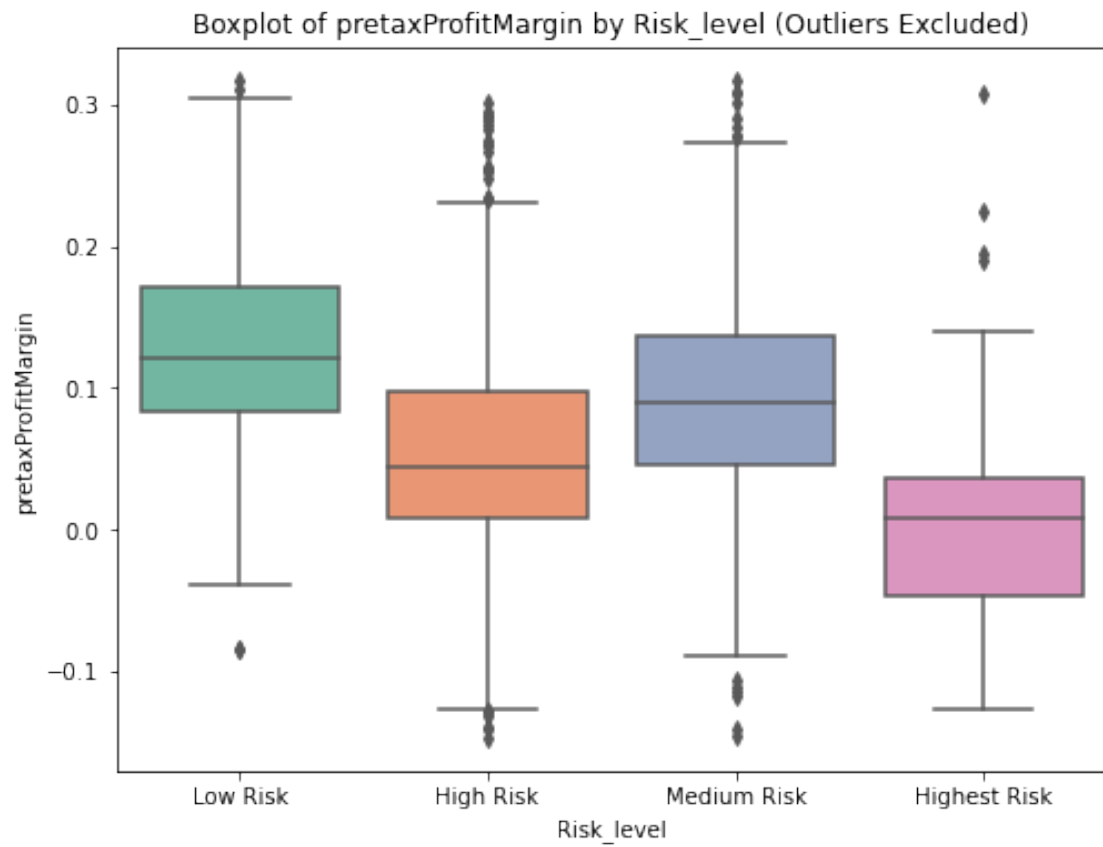


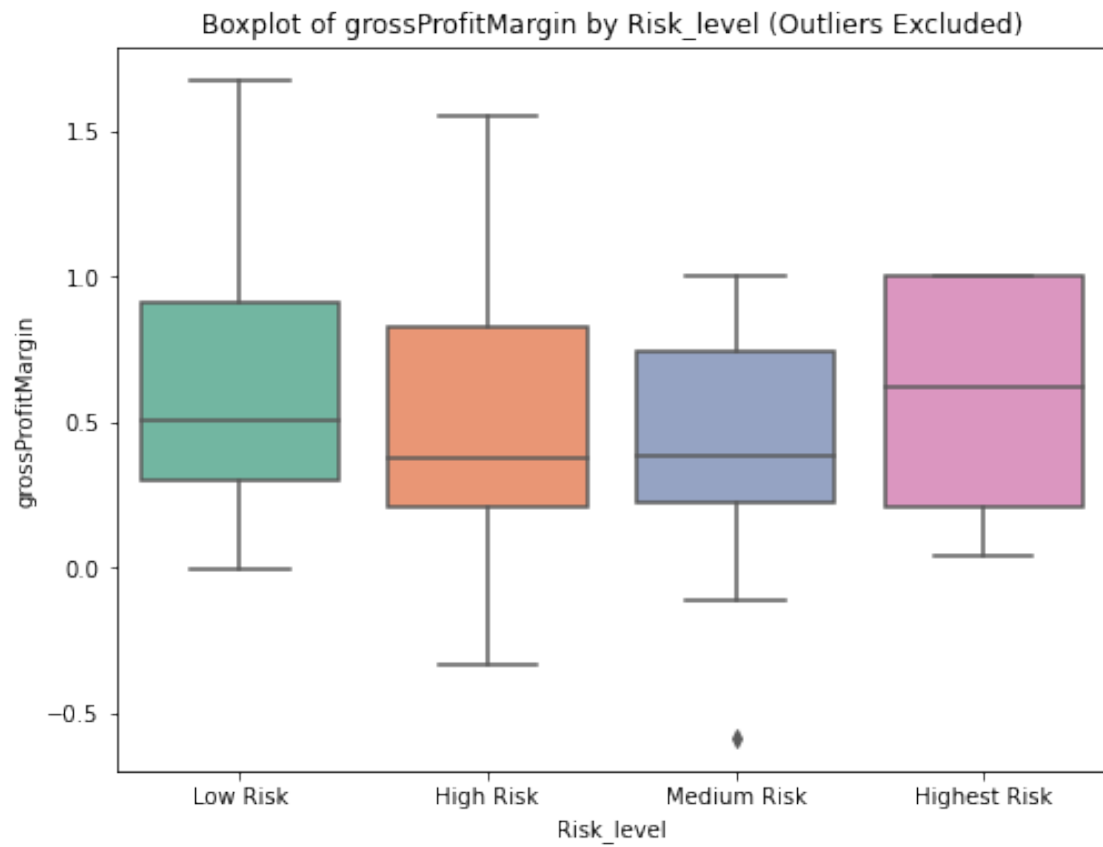


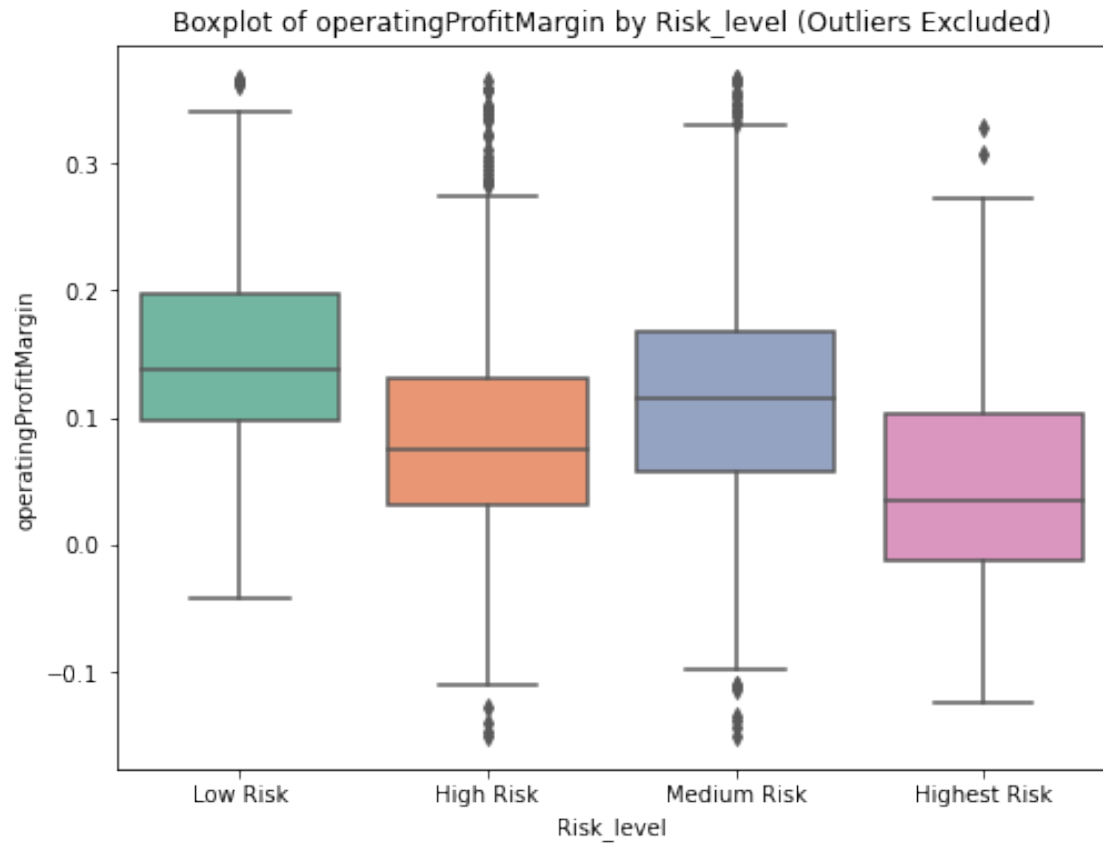


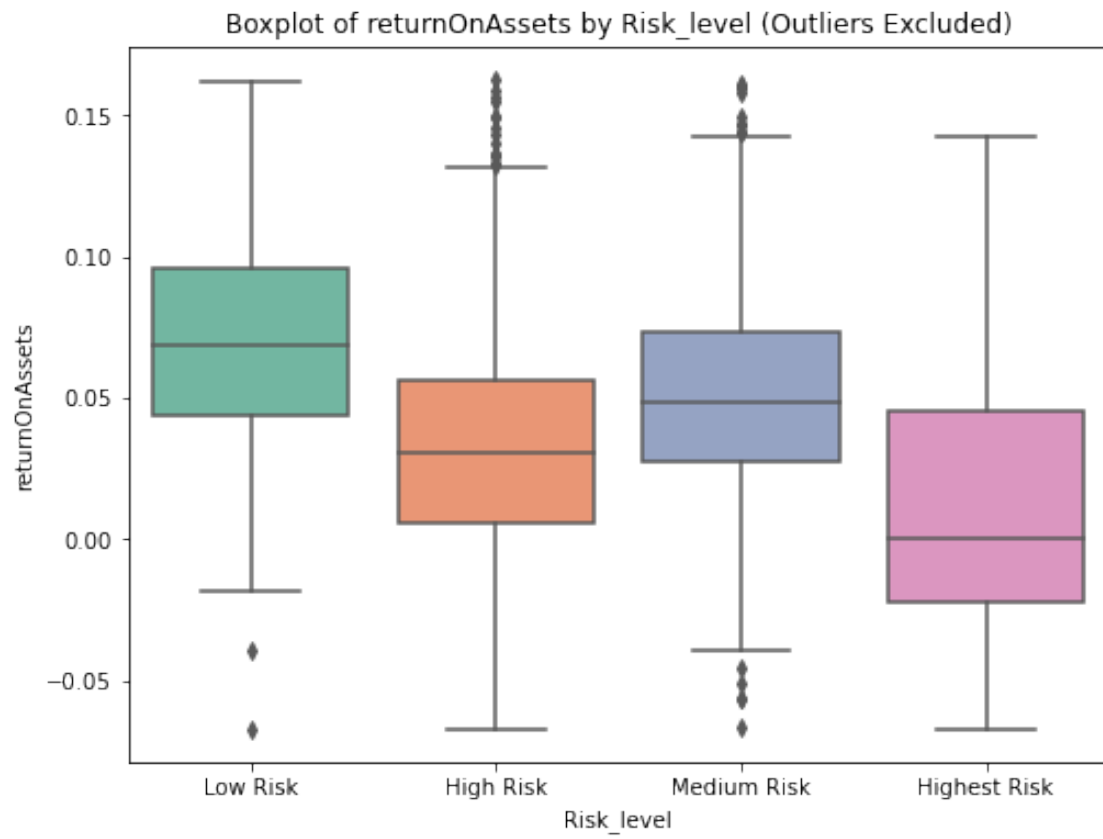




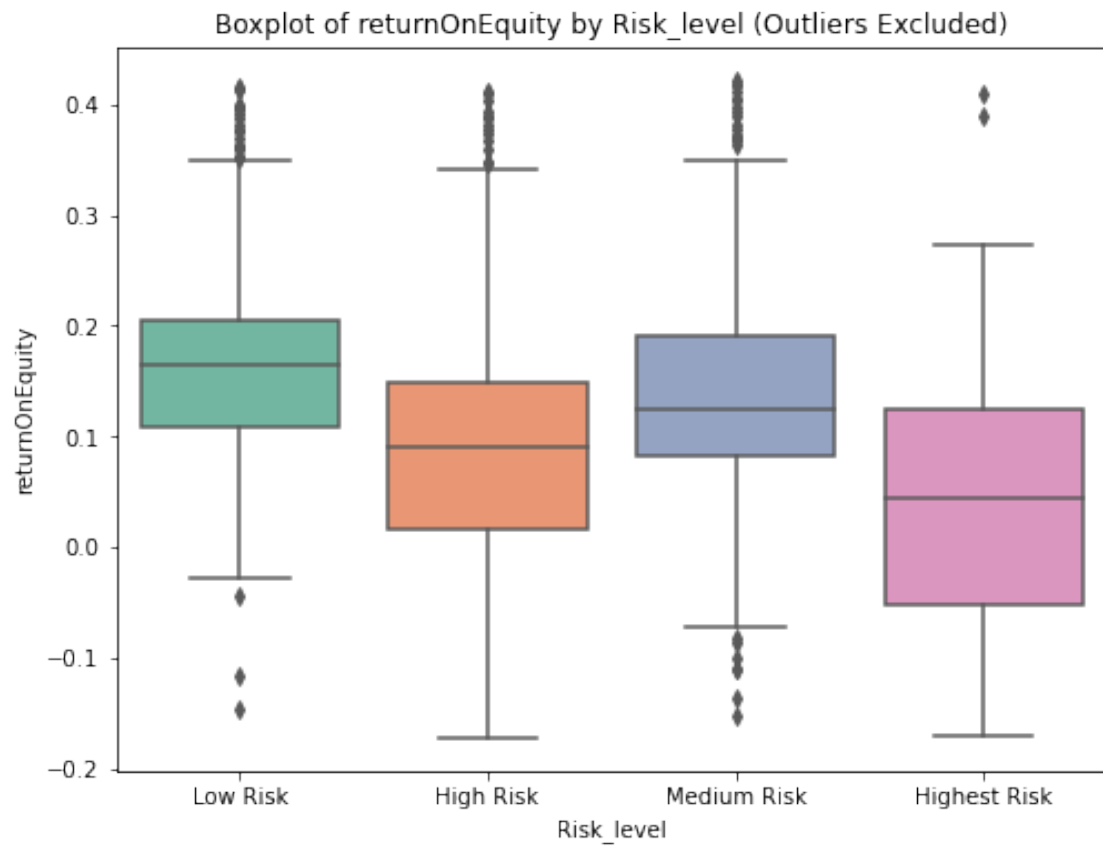


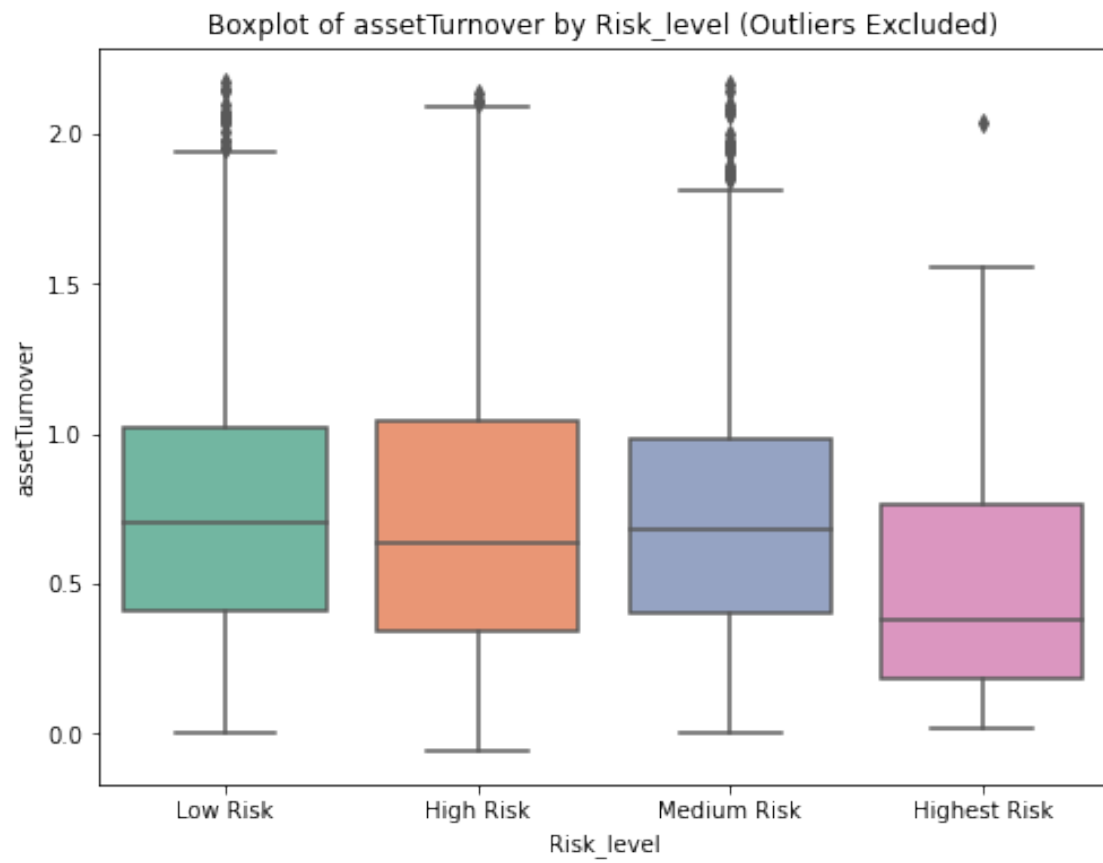


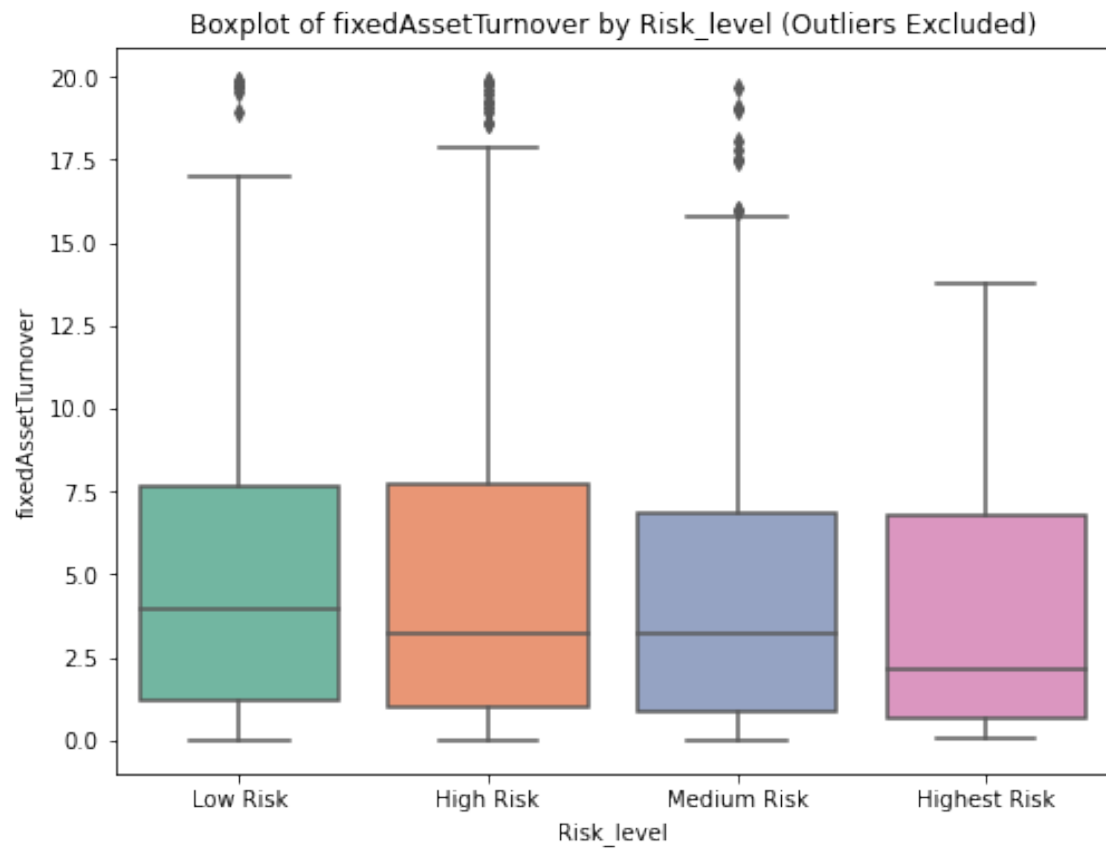


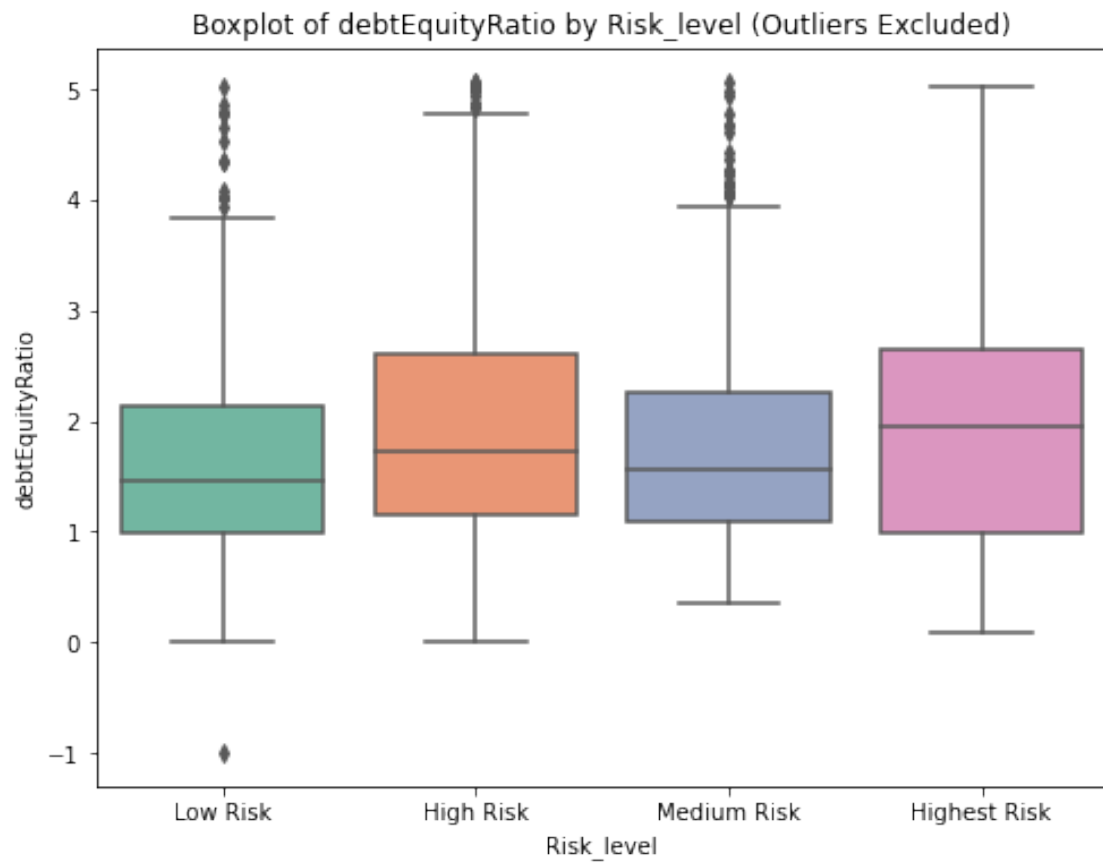


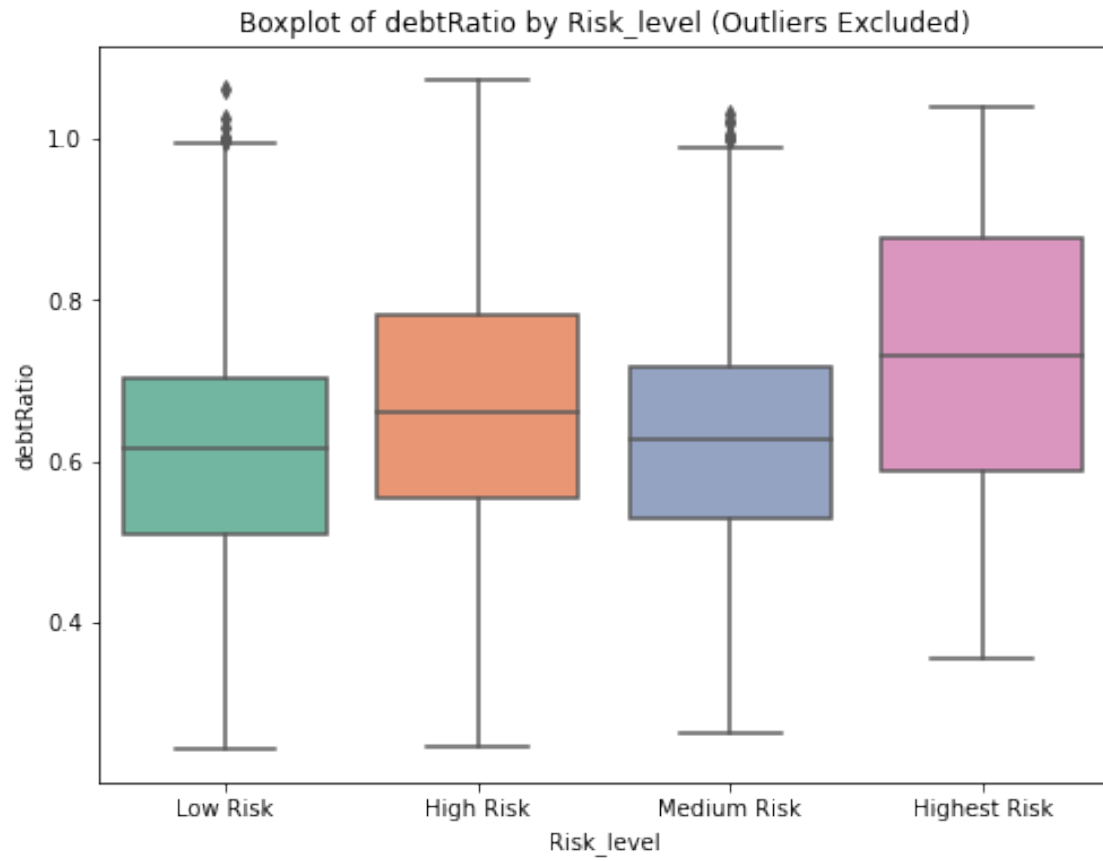


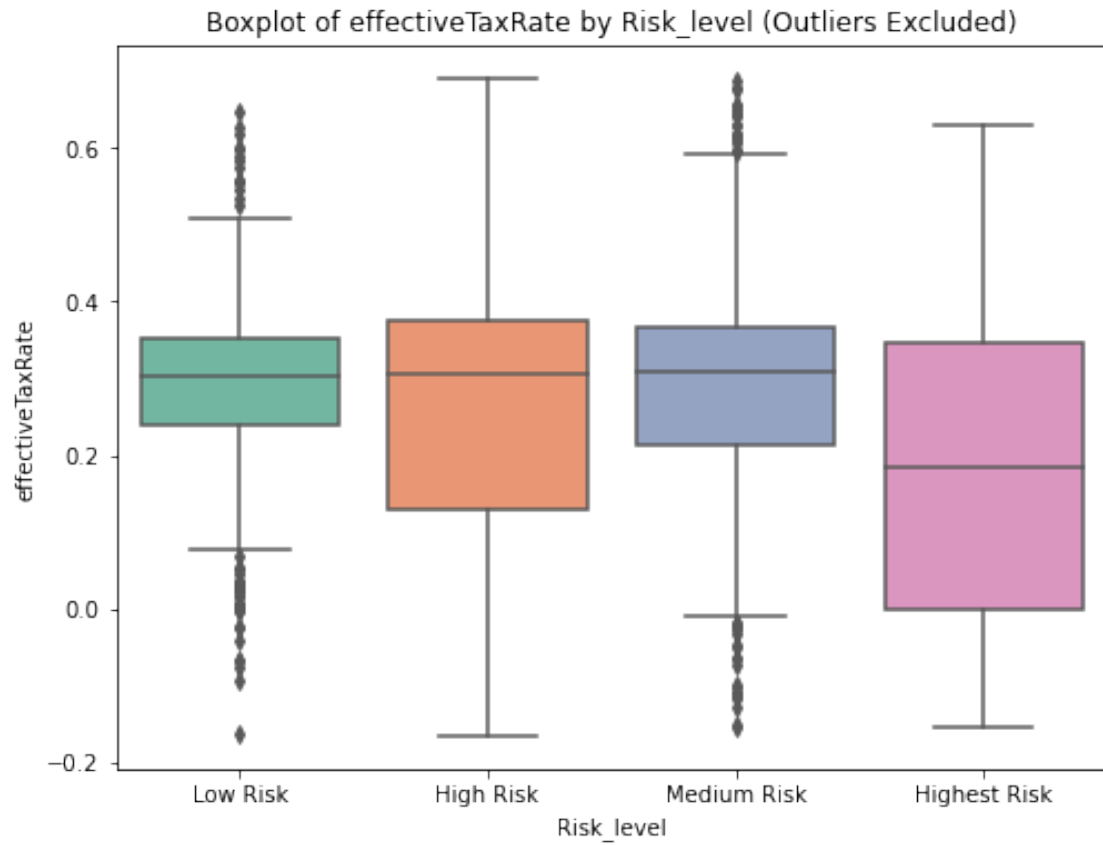


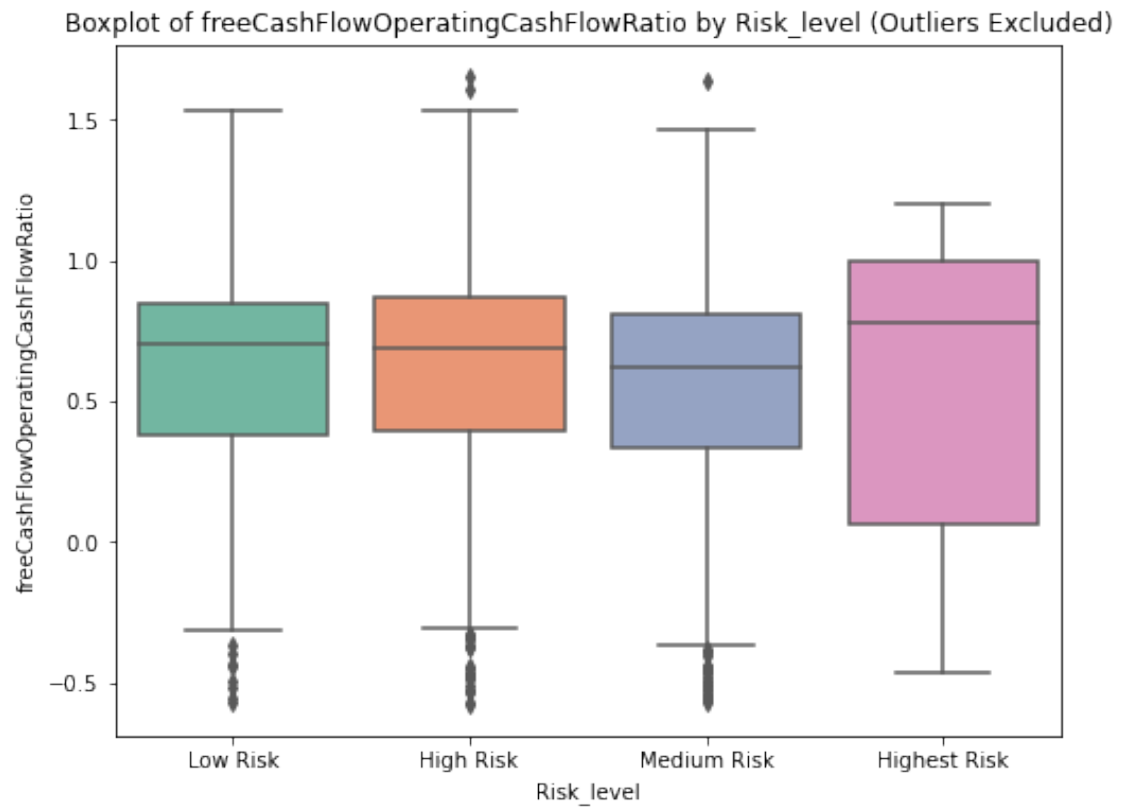


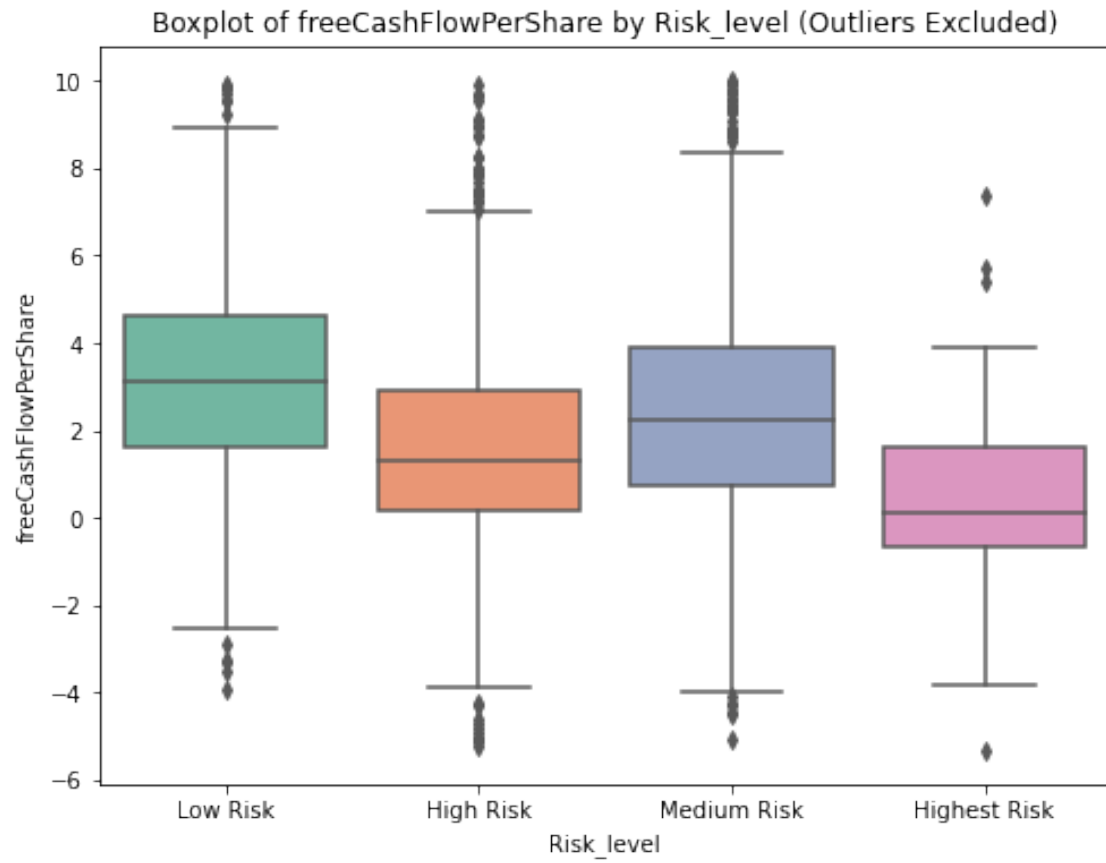


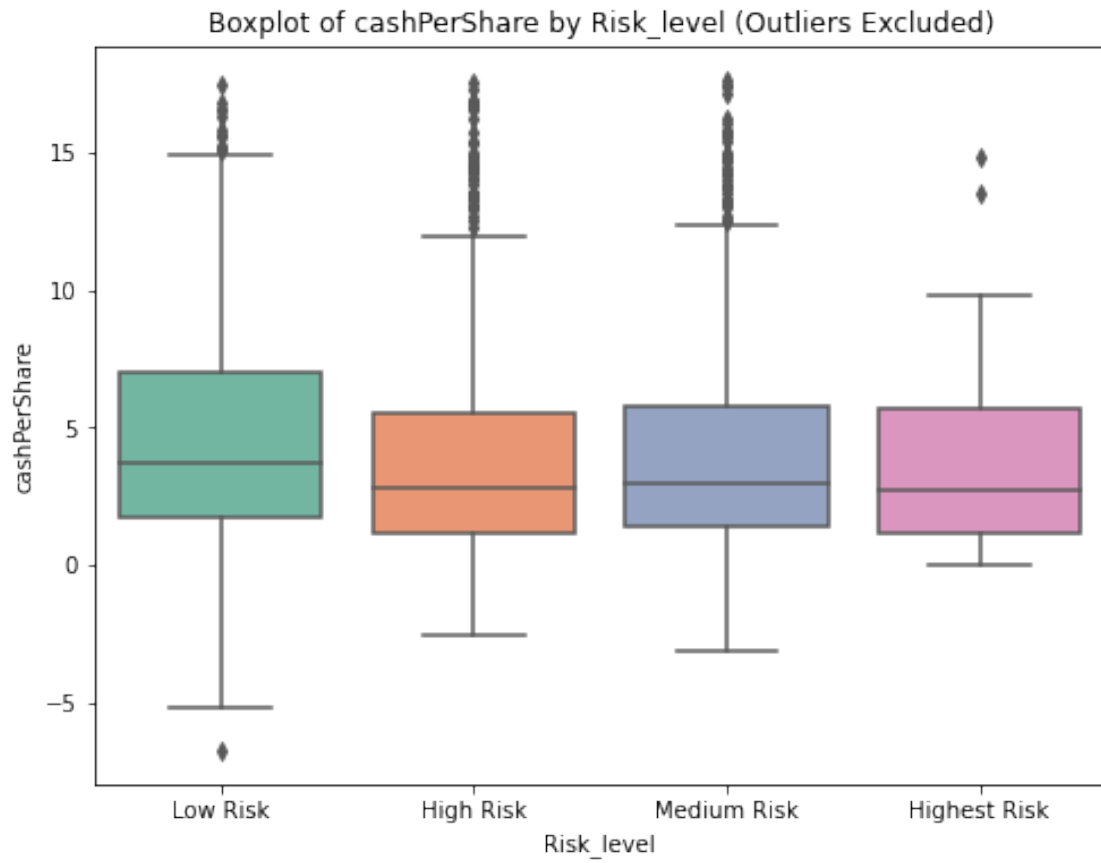


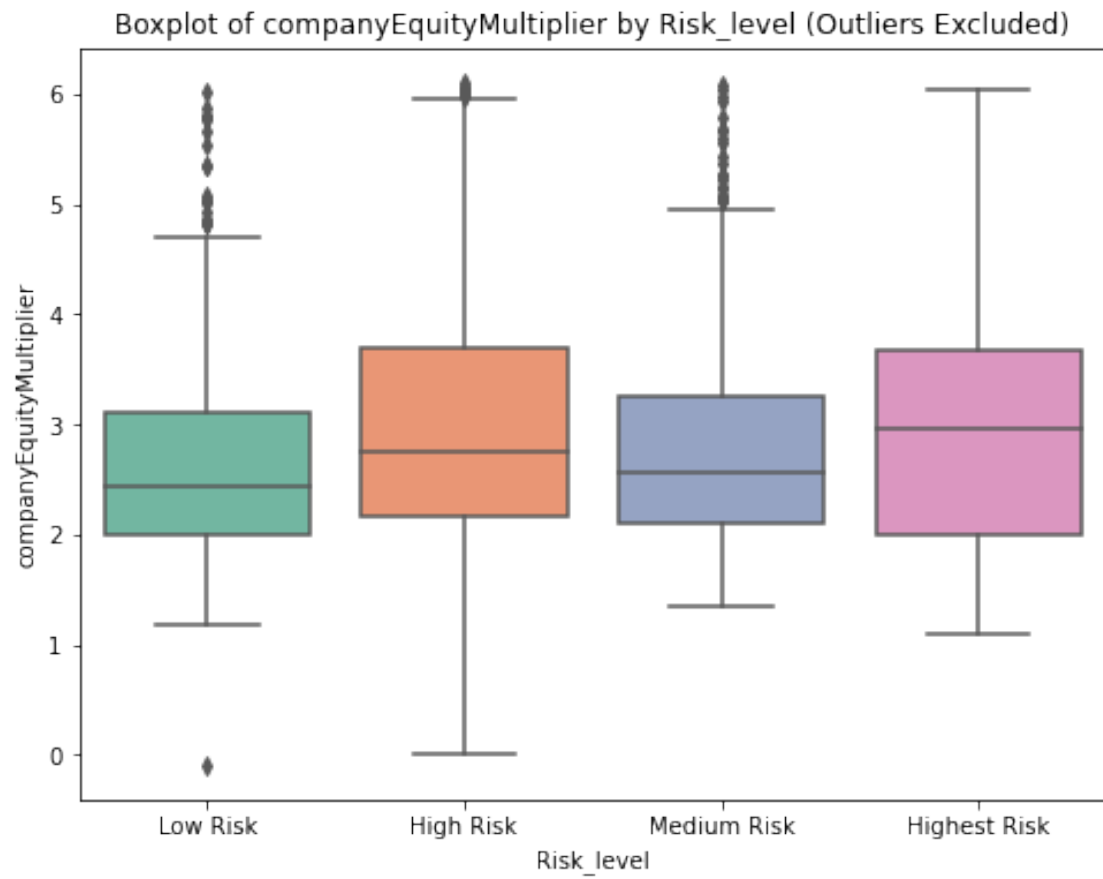


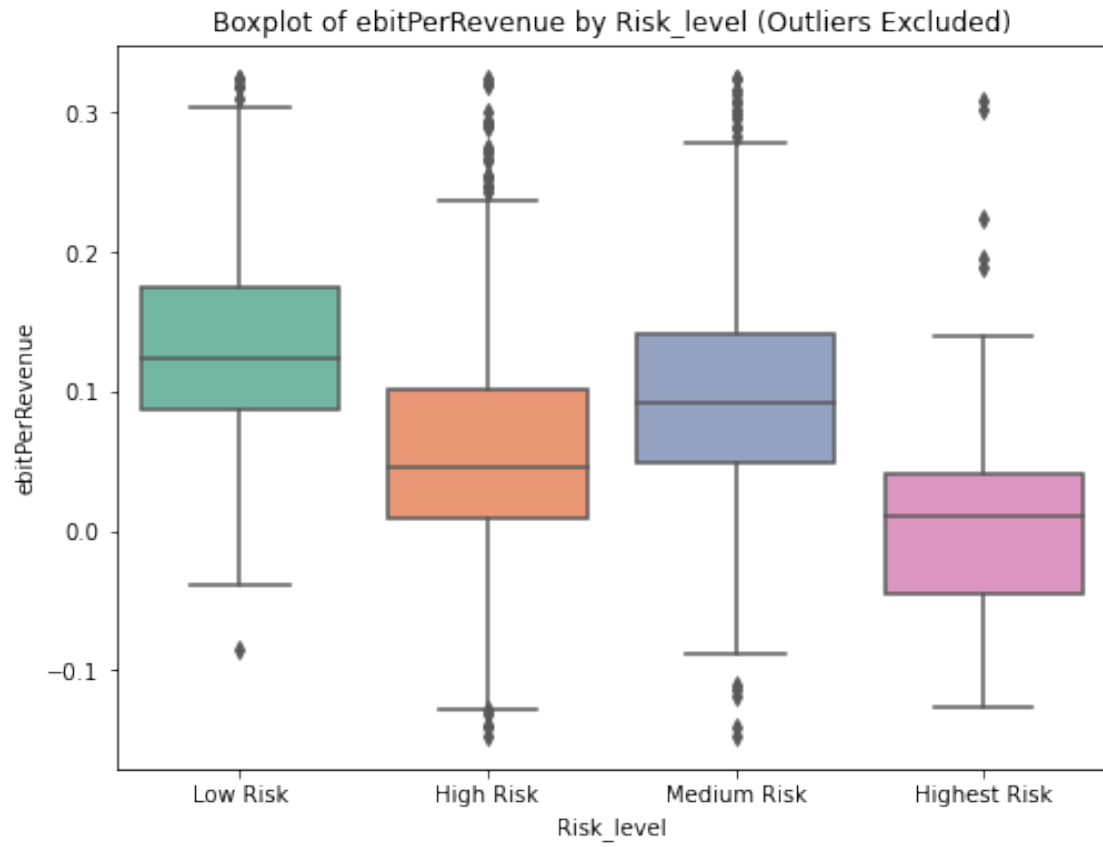


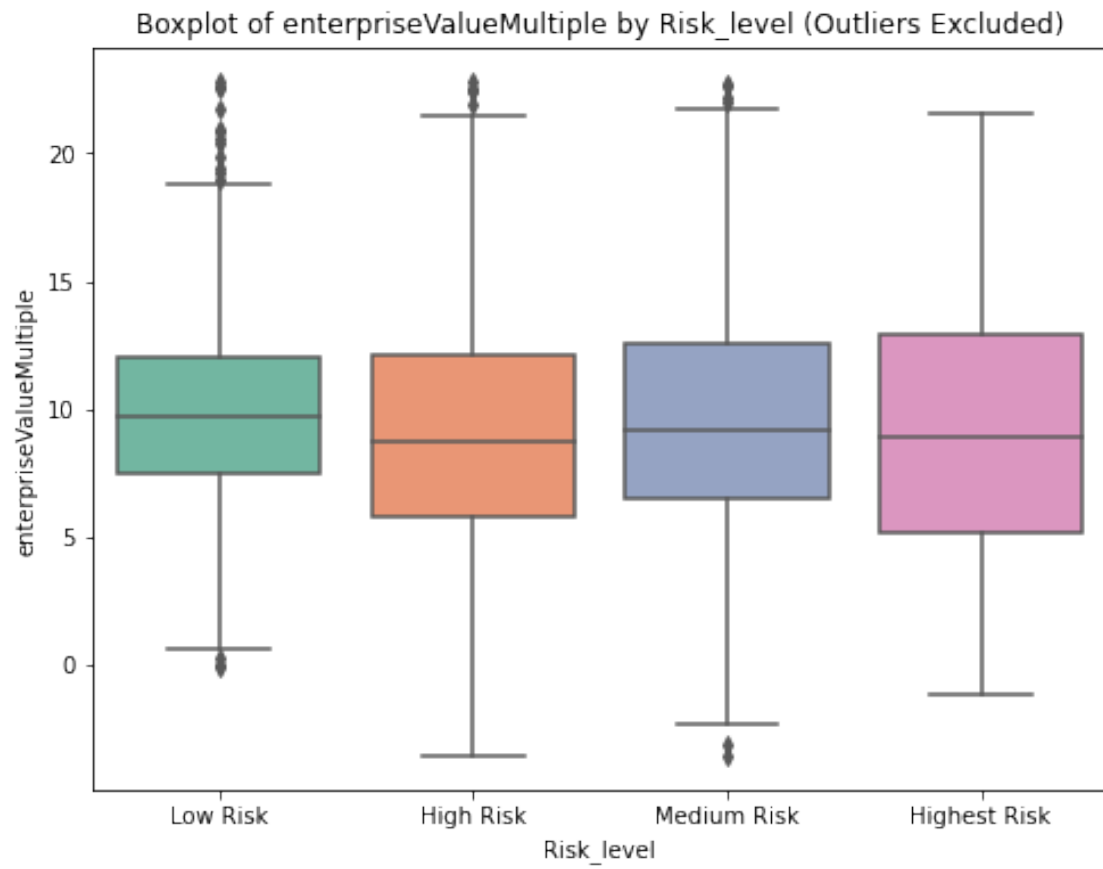


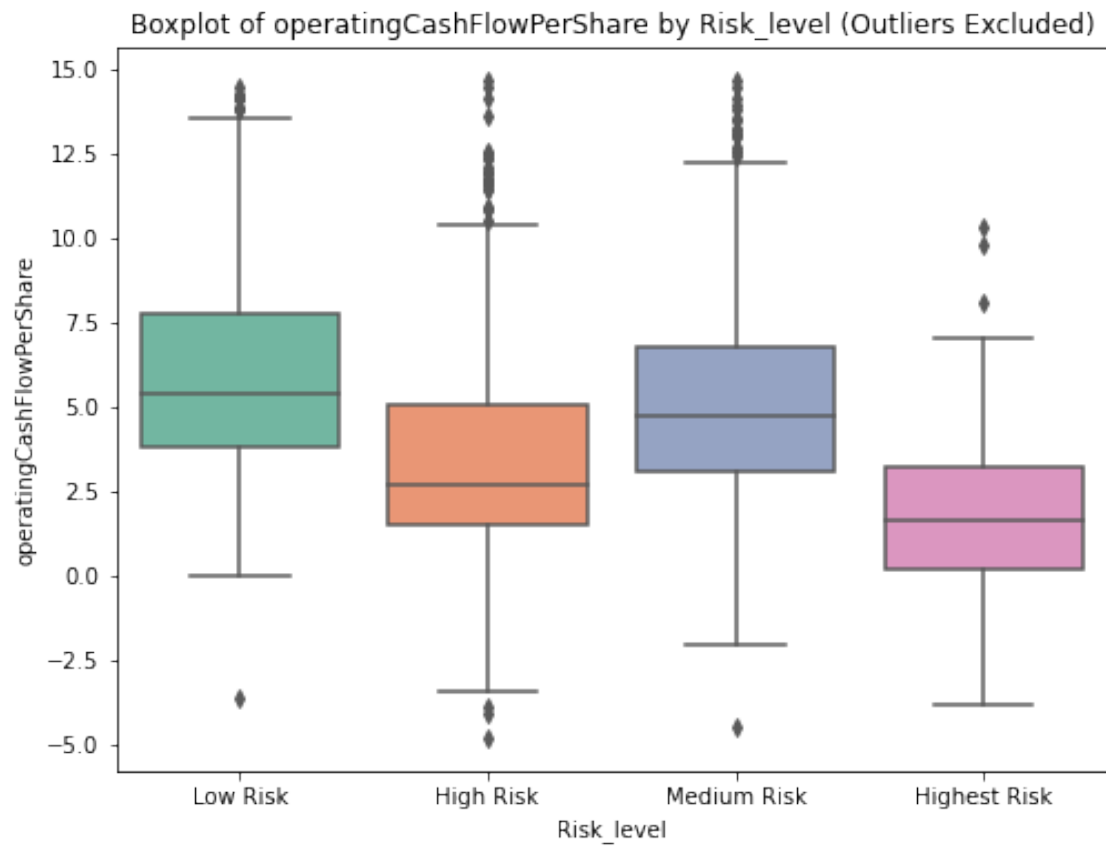


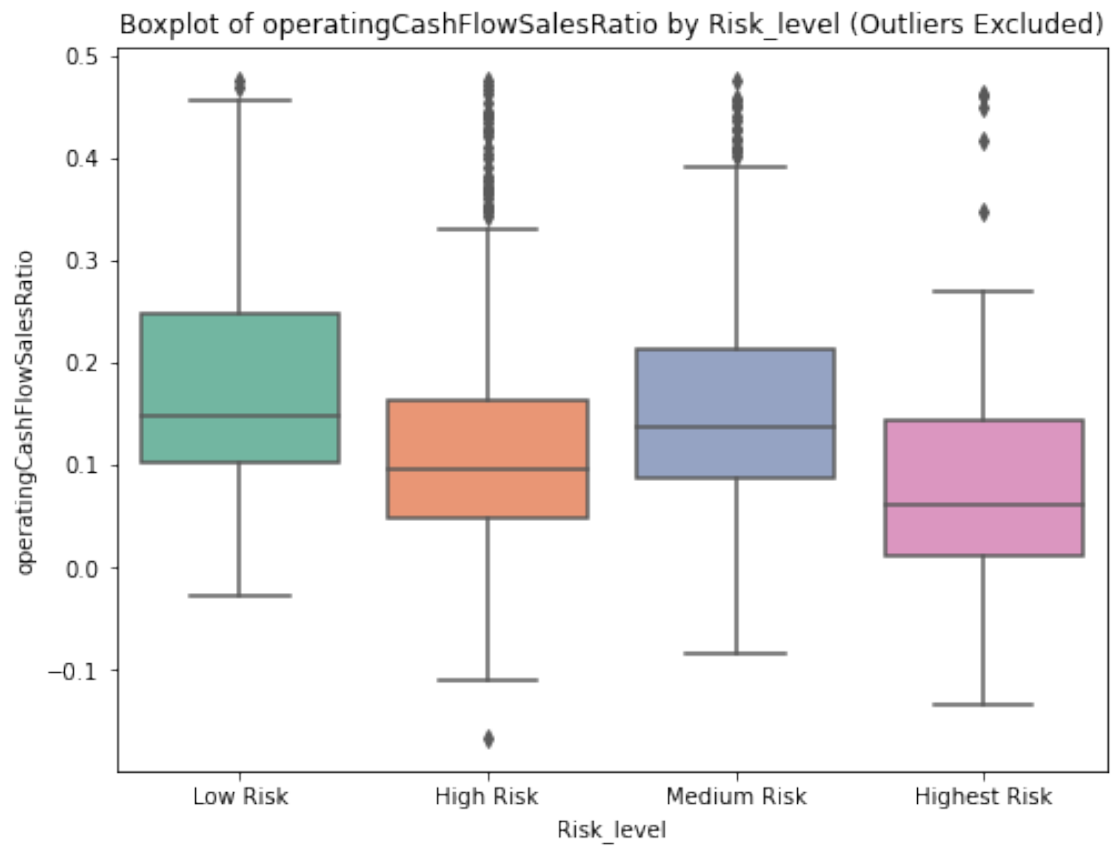


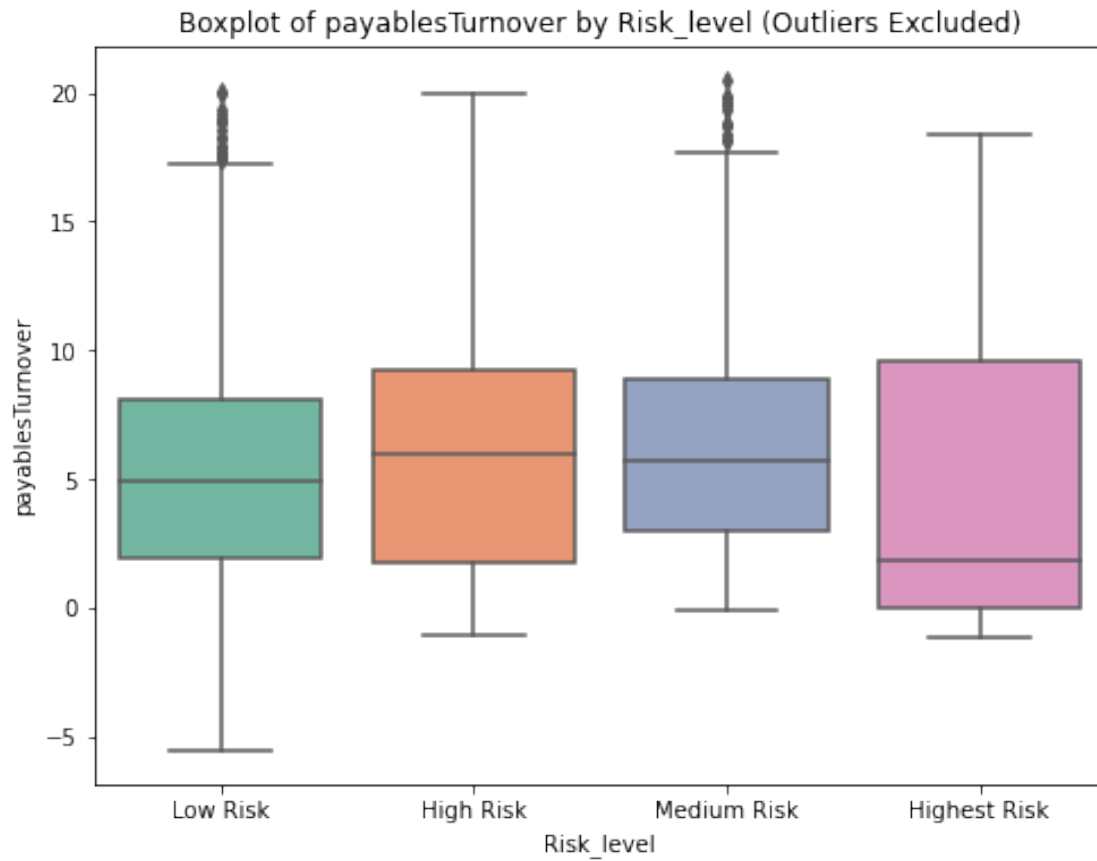




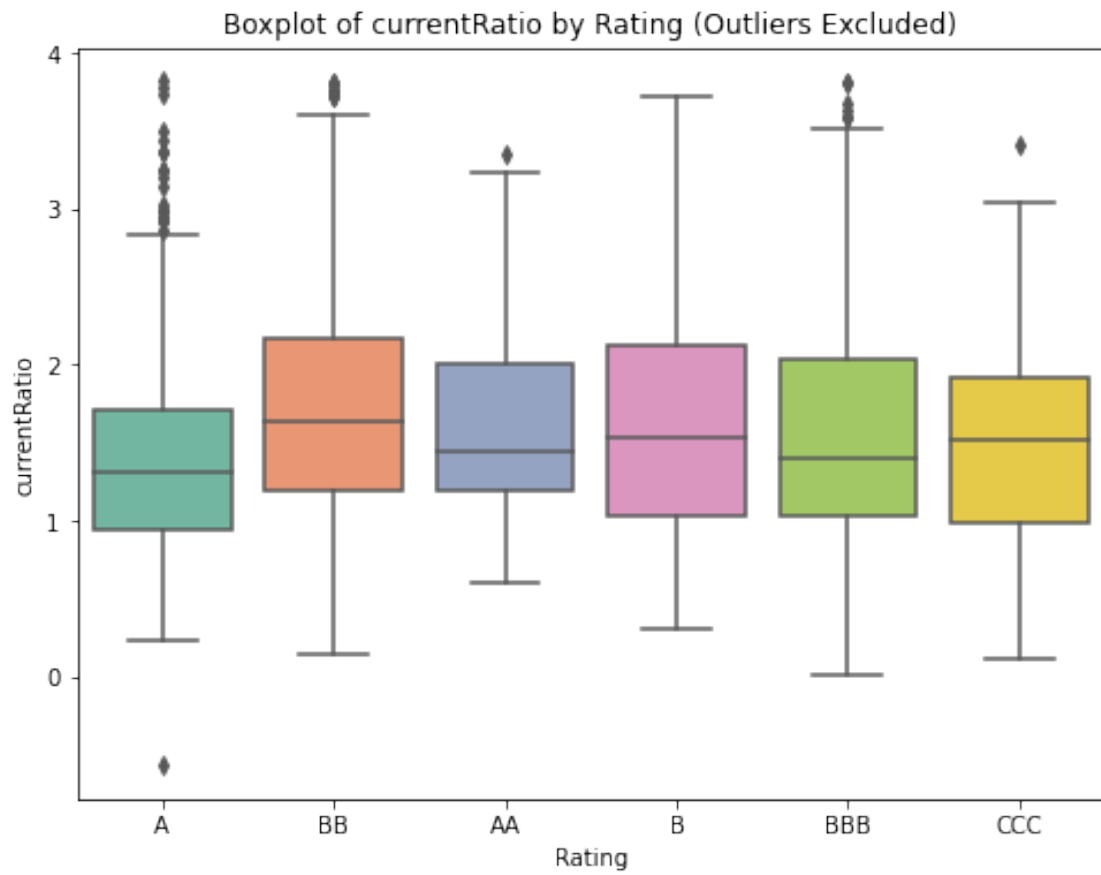


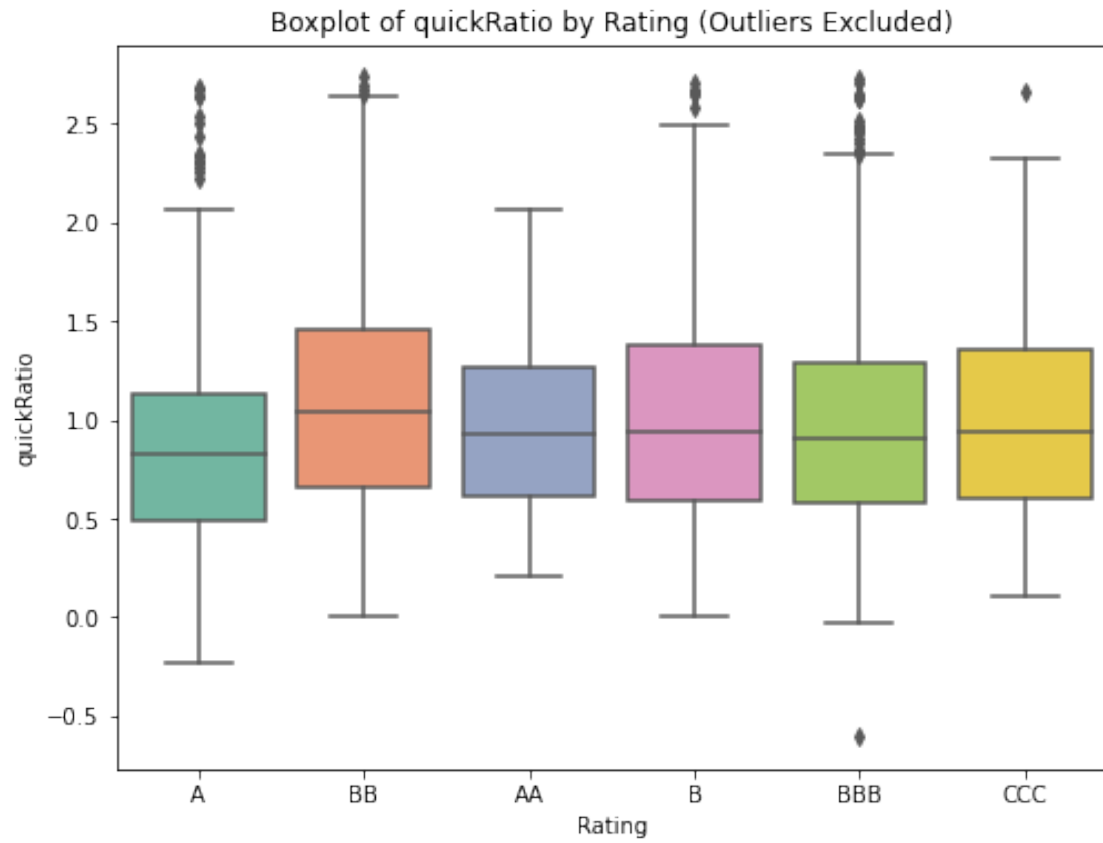


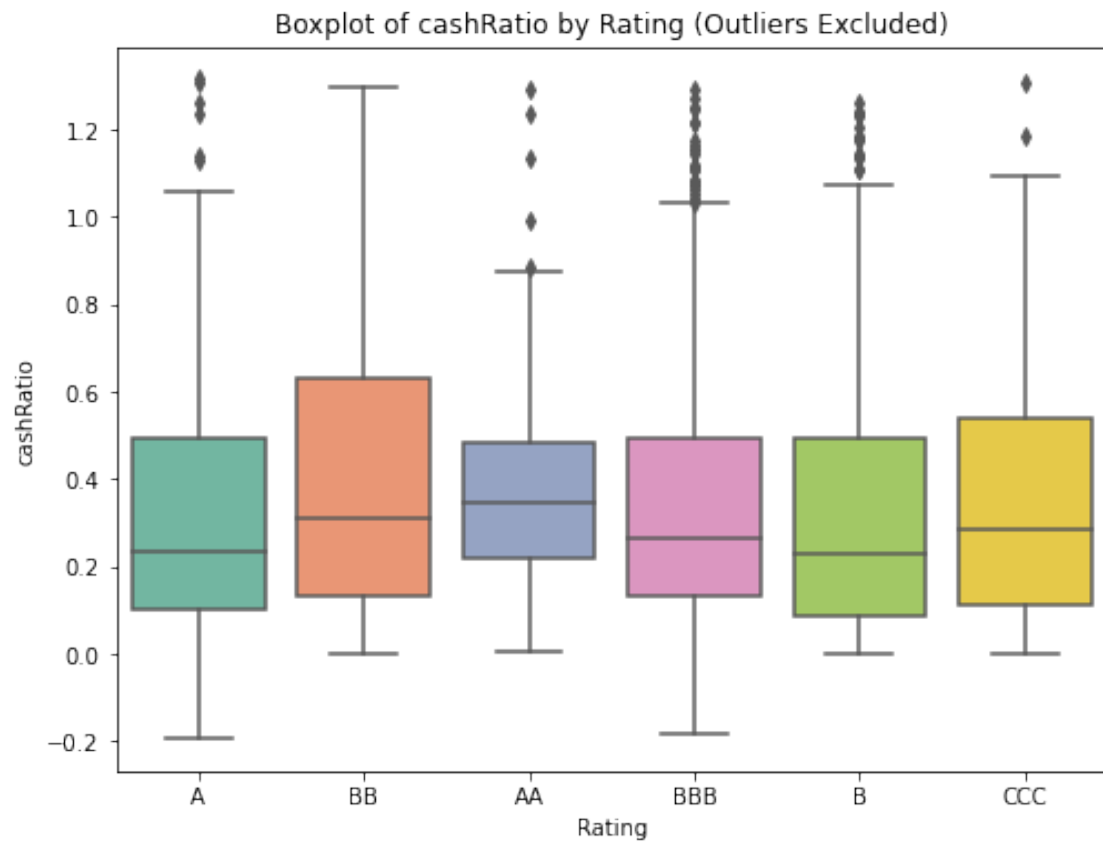


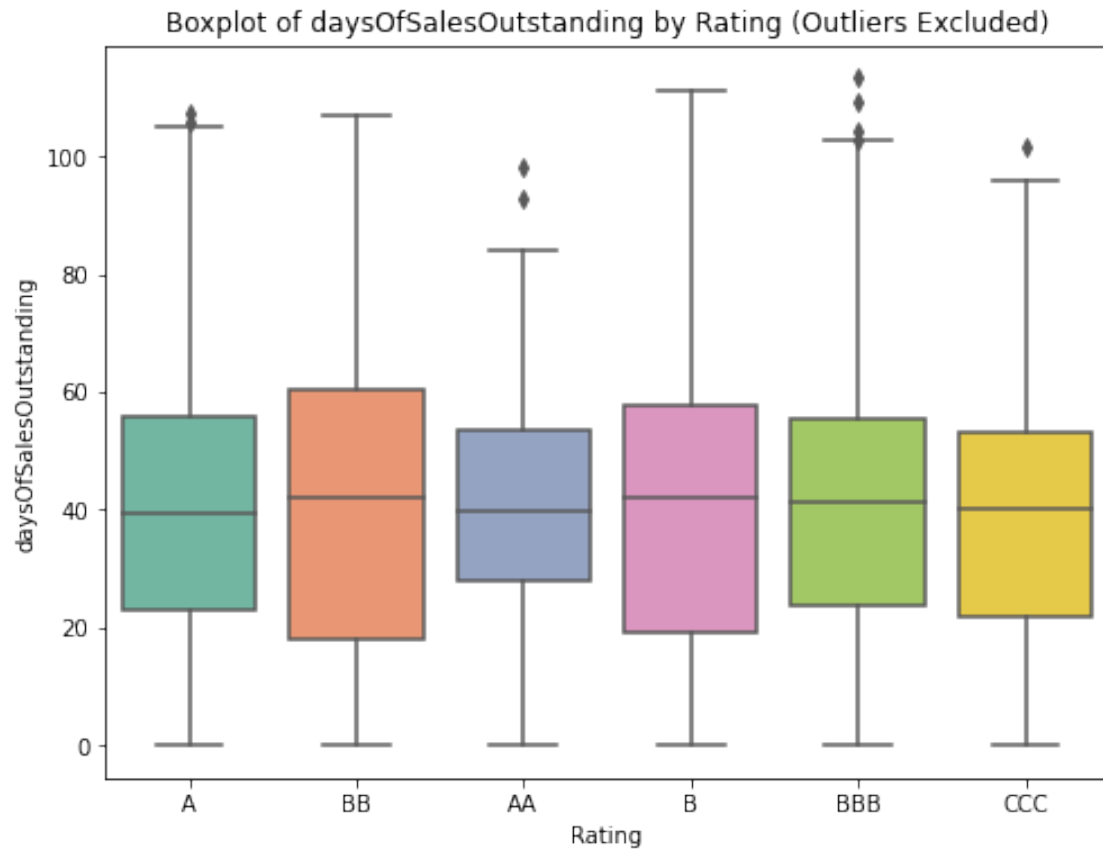


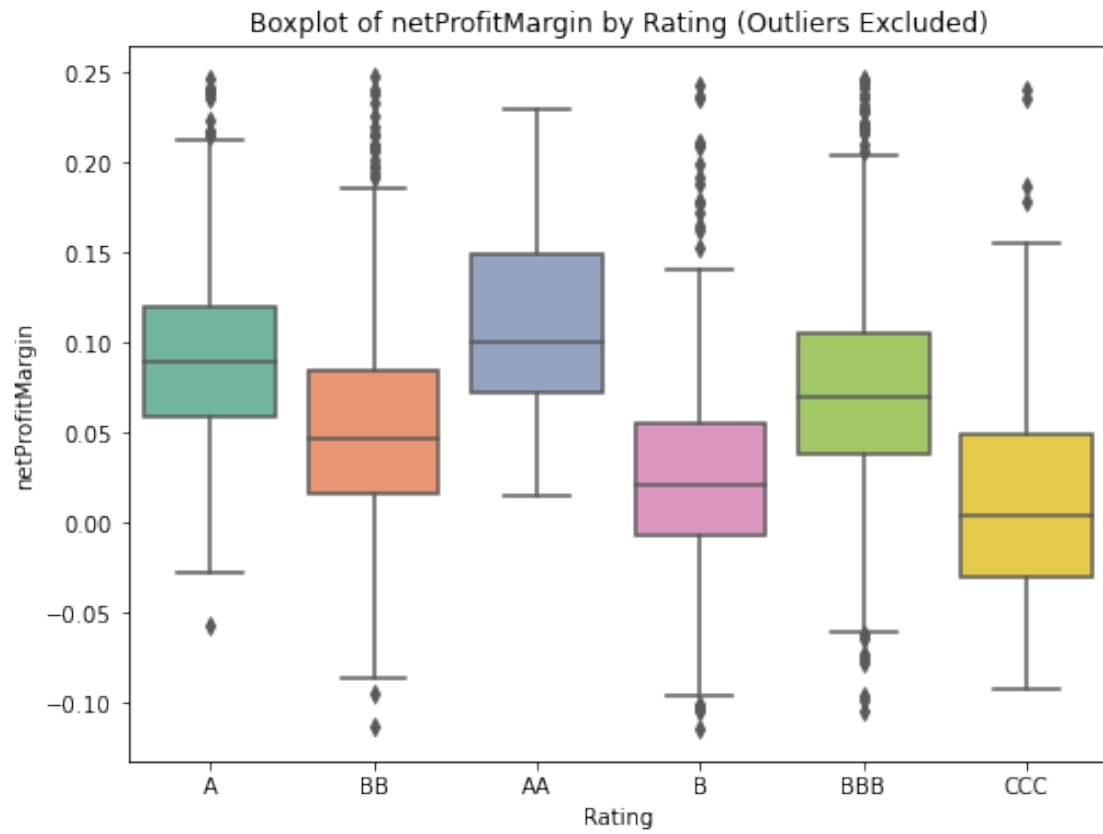
```
[286]: draw_boxplots_without_outliers(credit_rating_new, predictors, 'Rating')
```

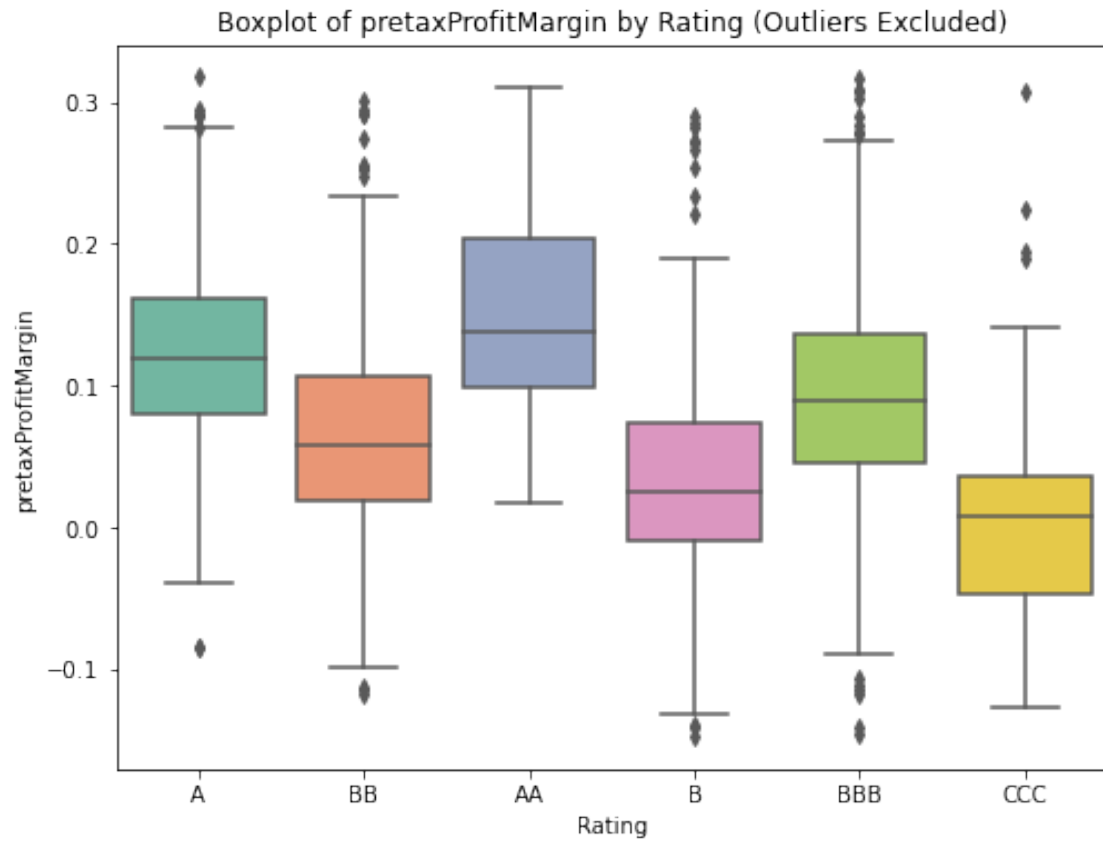


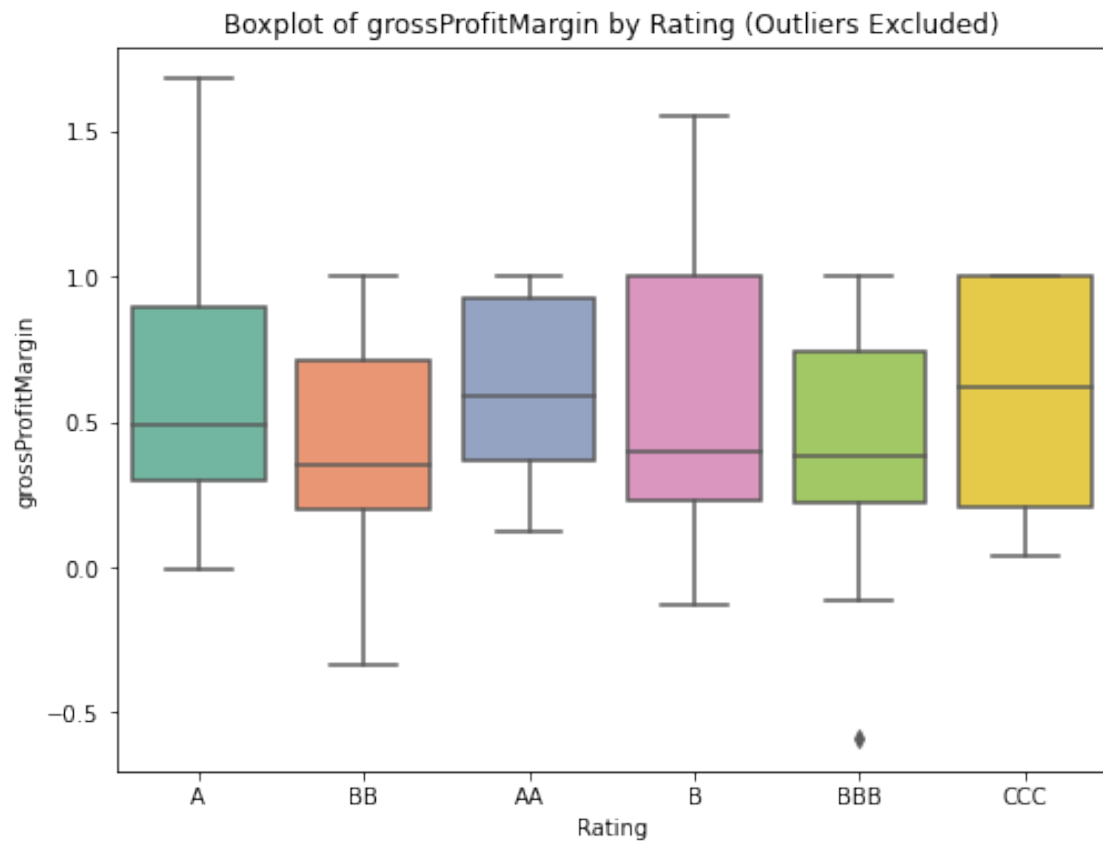


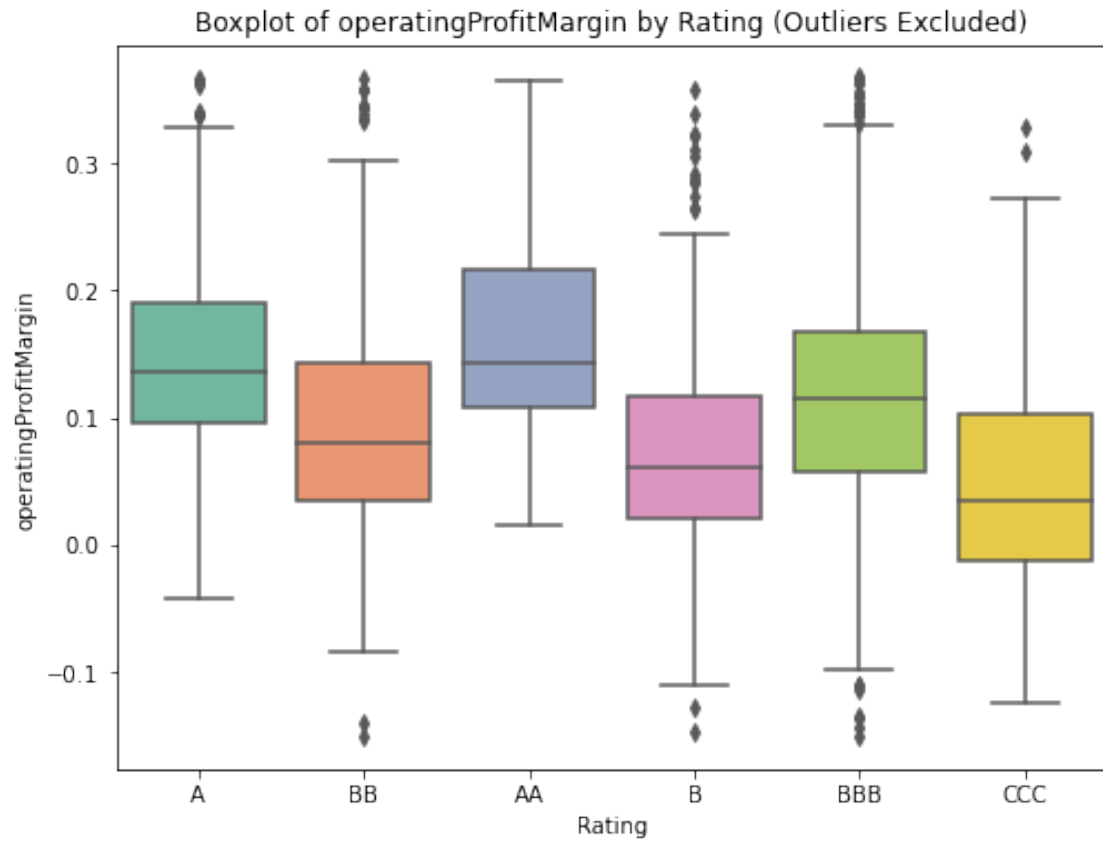


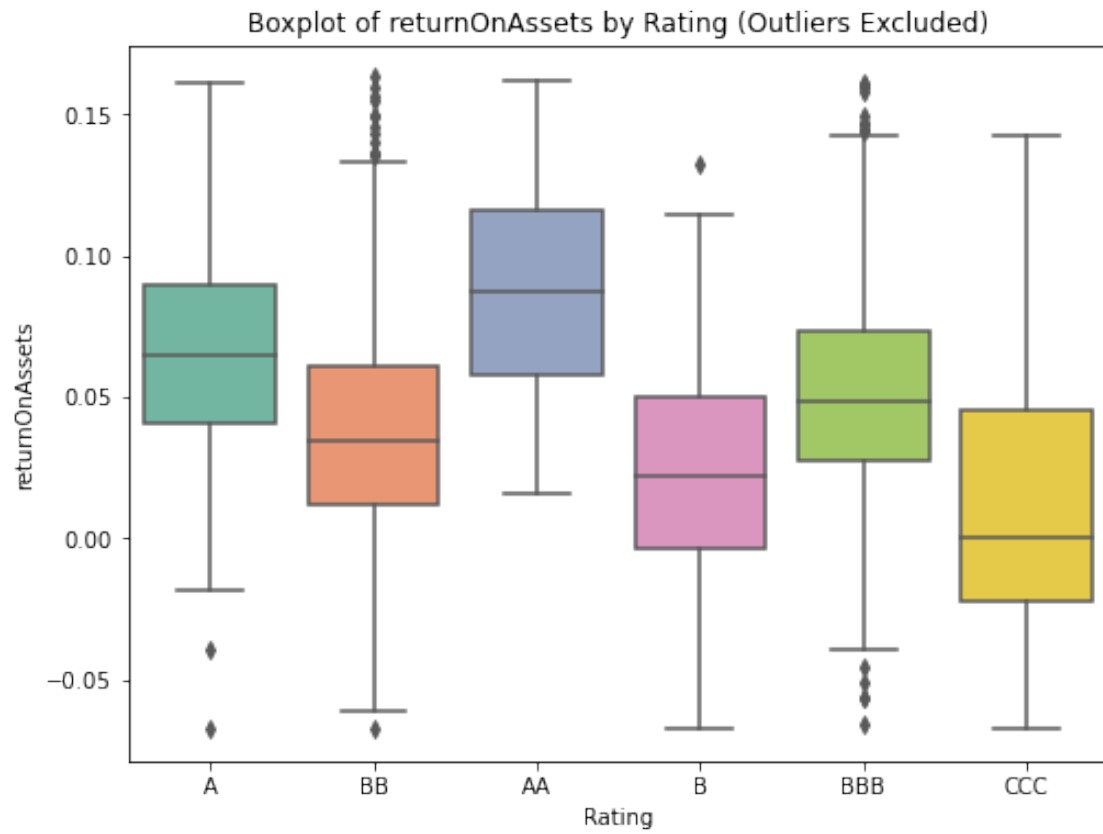


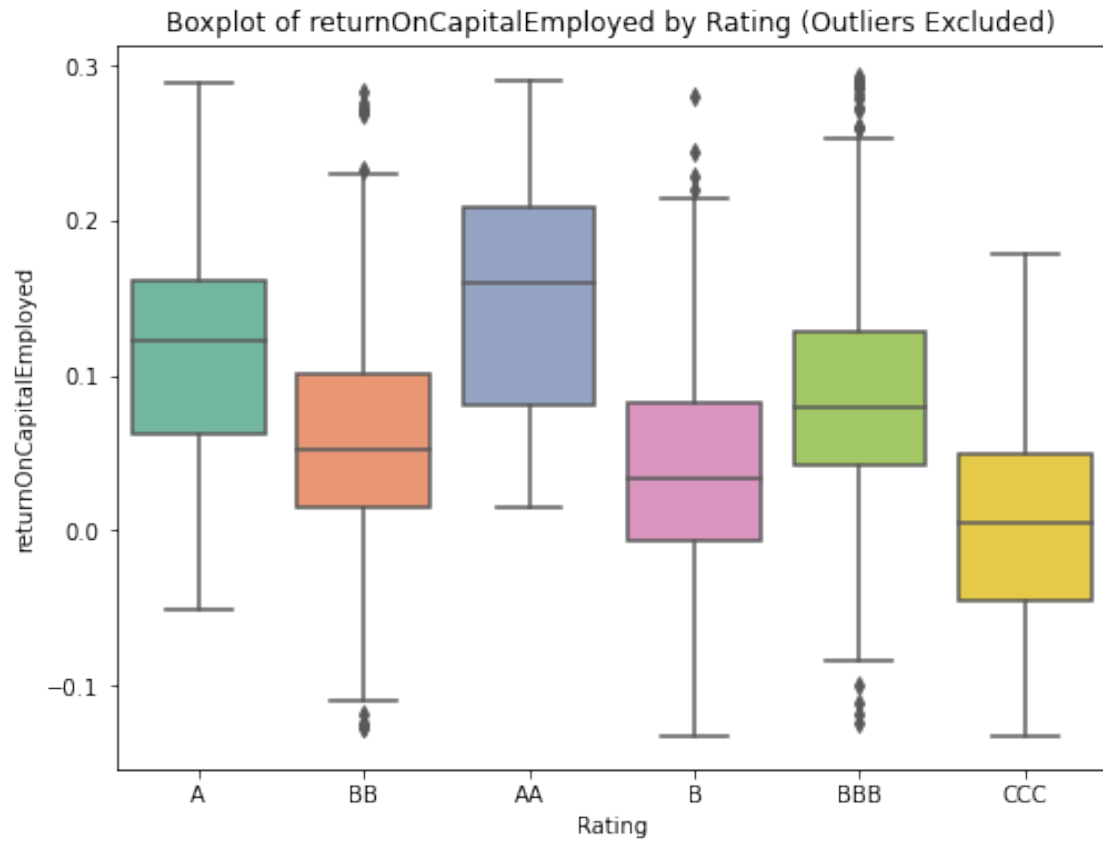




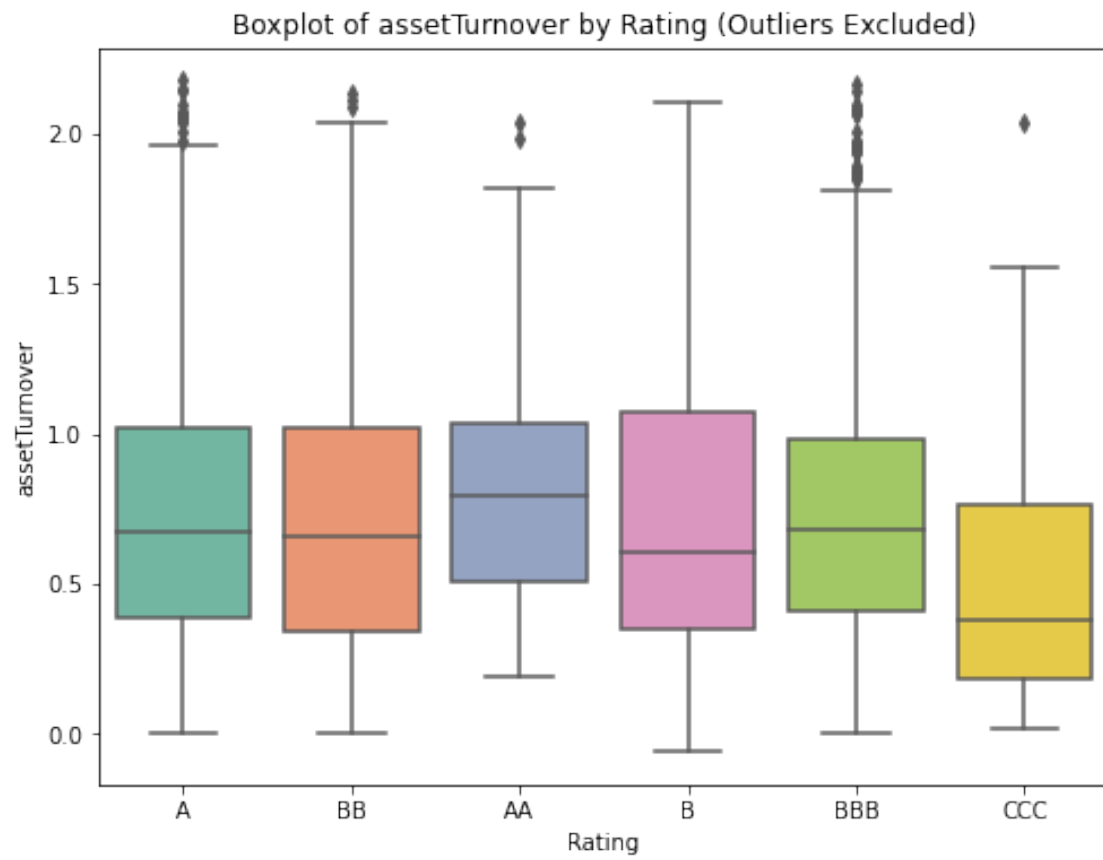


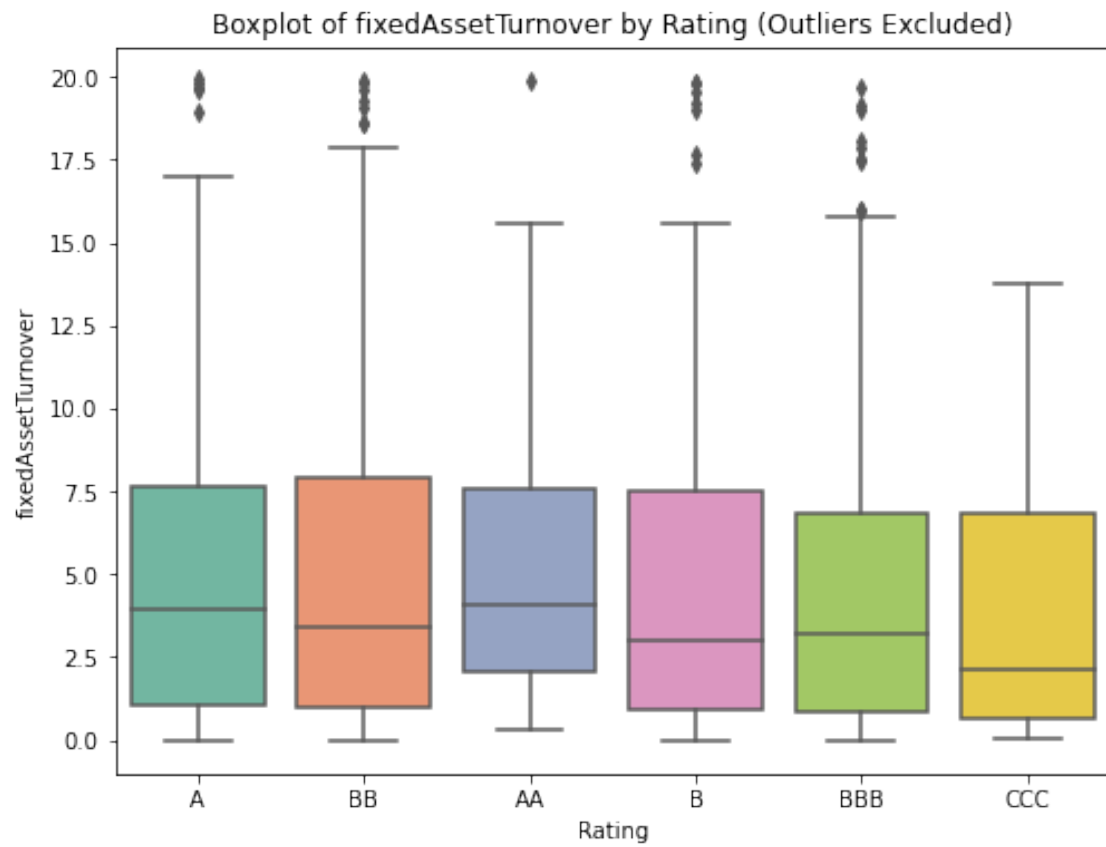


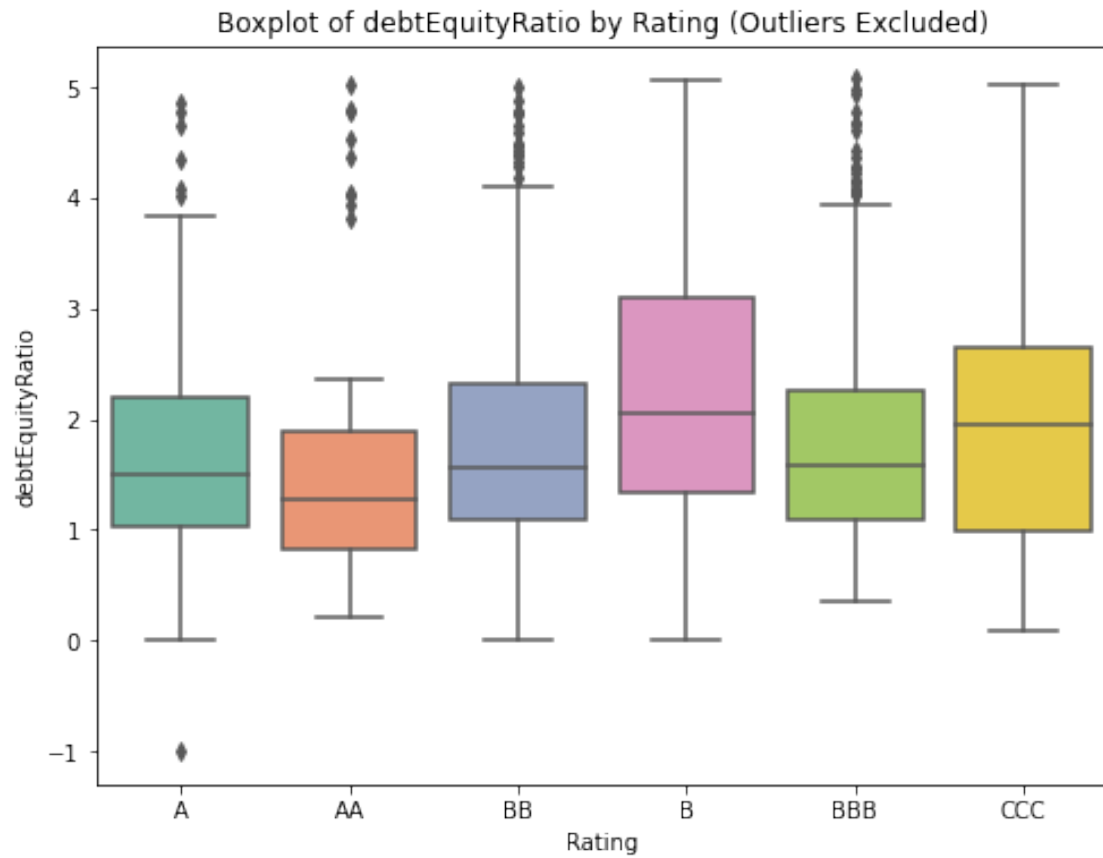


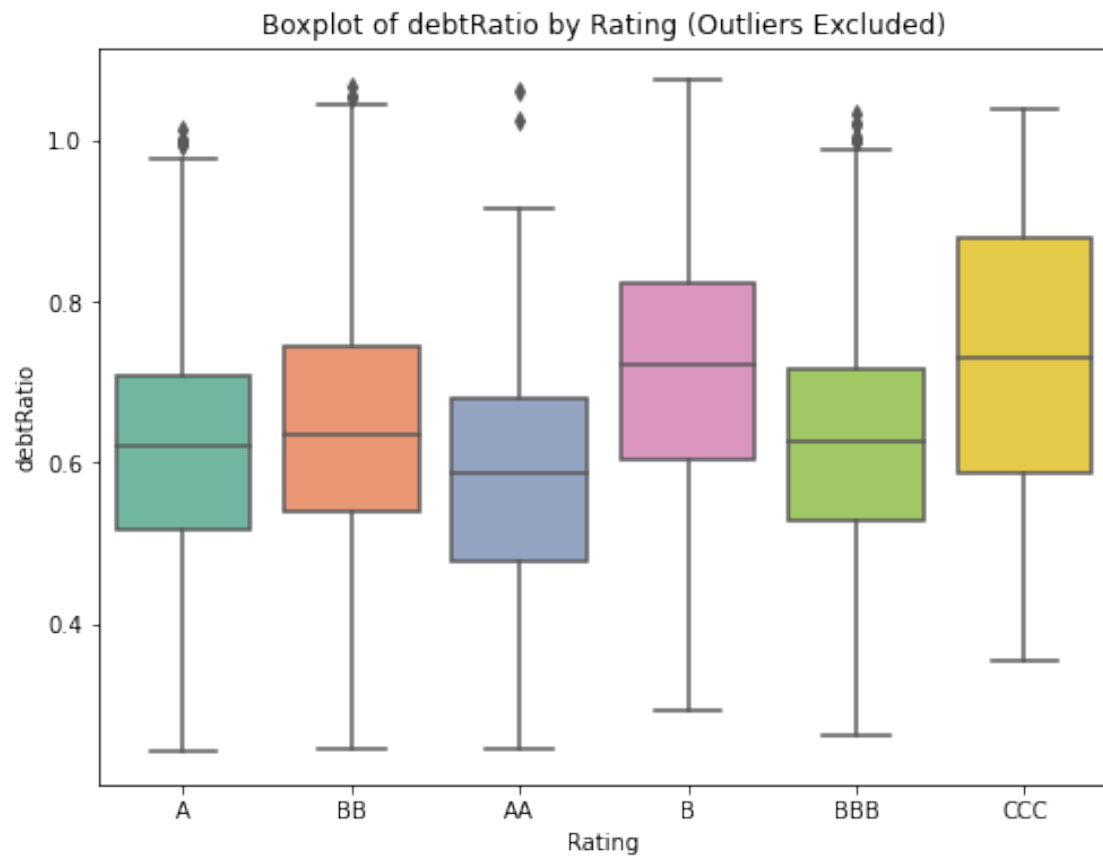


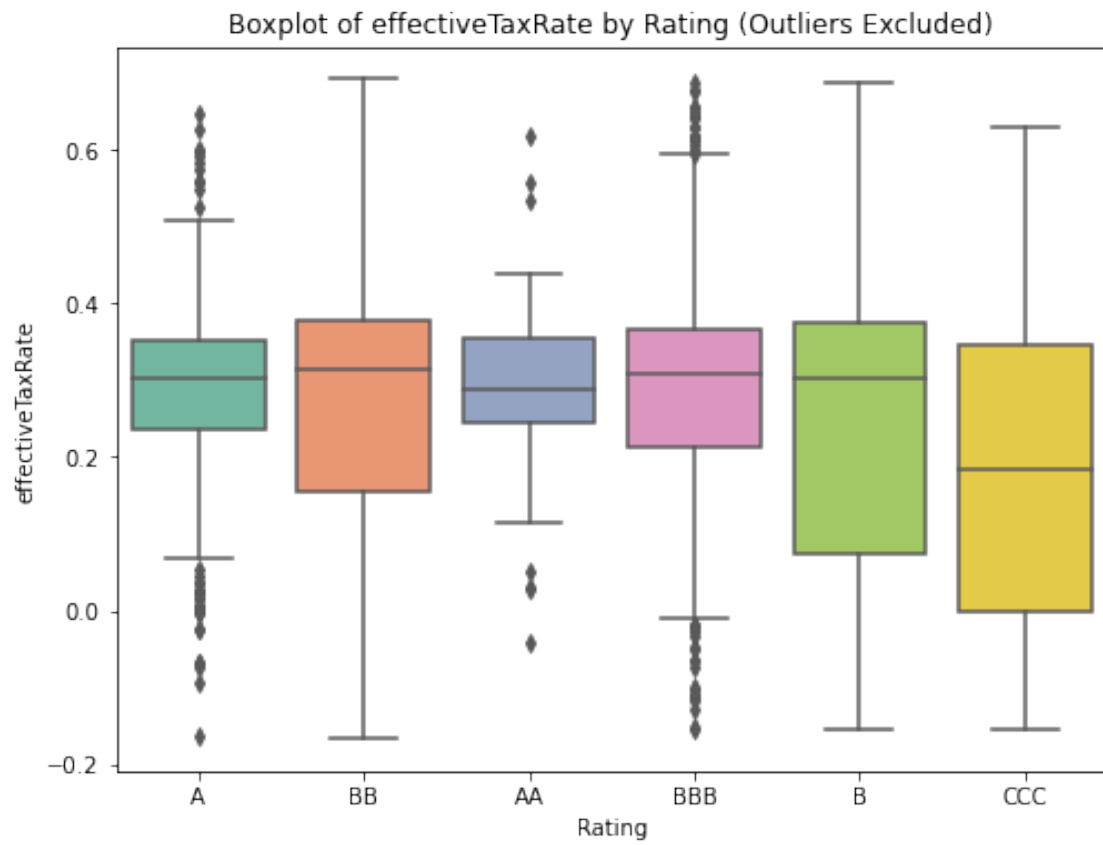


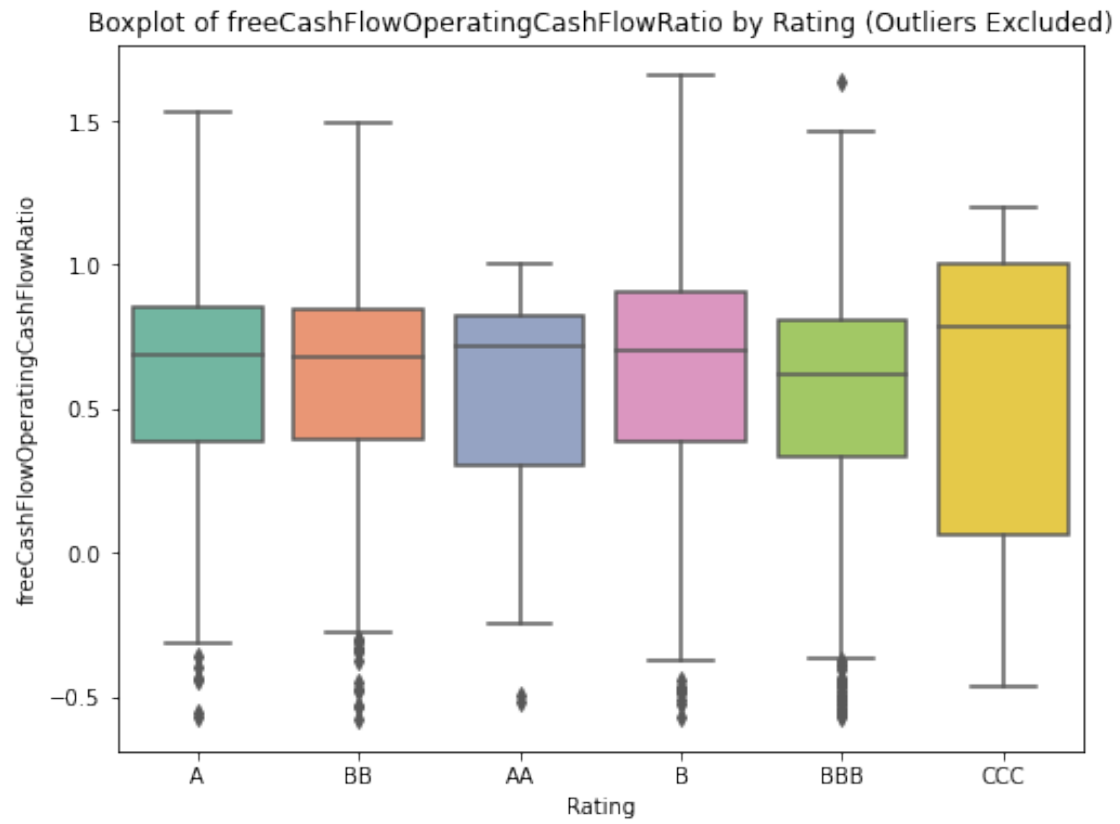


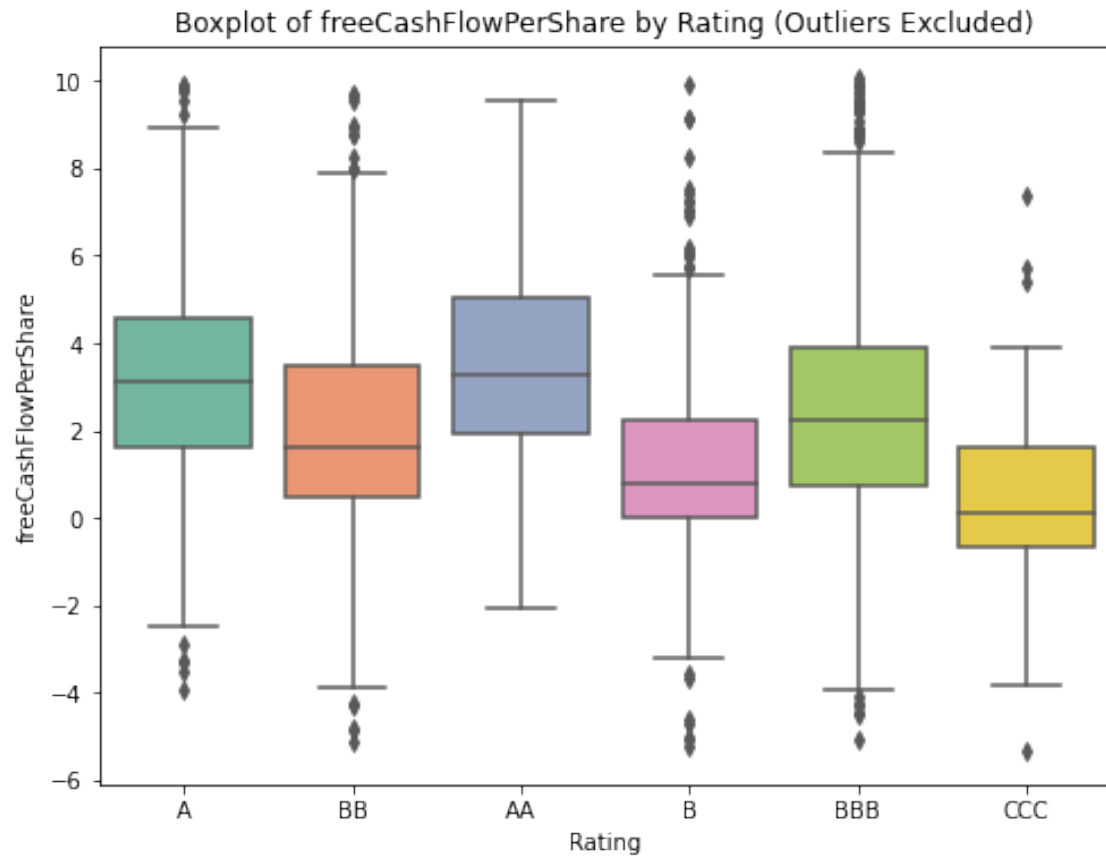


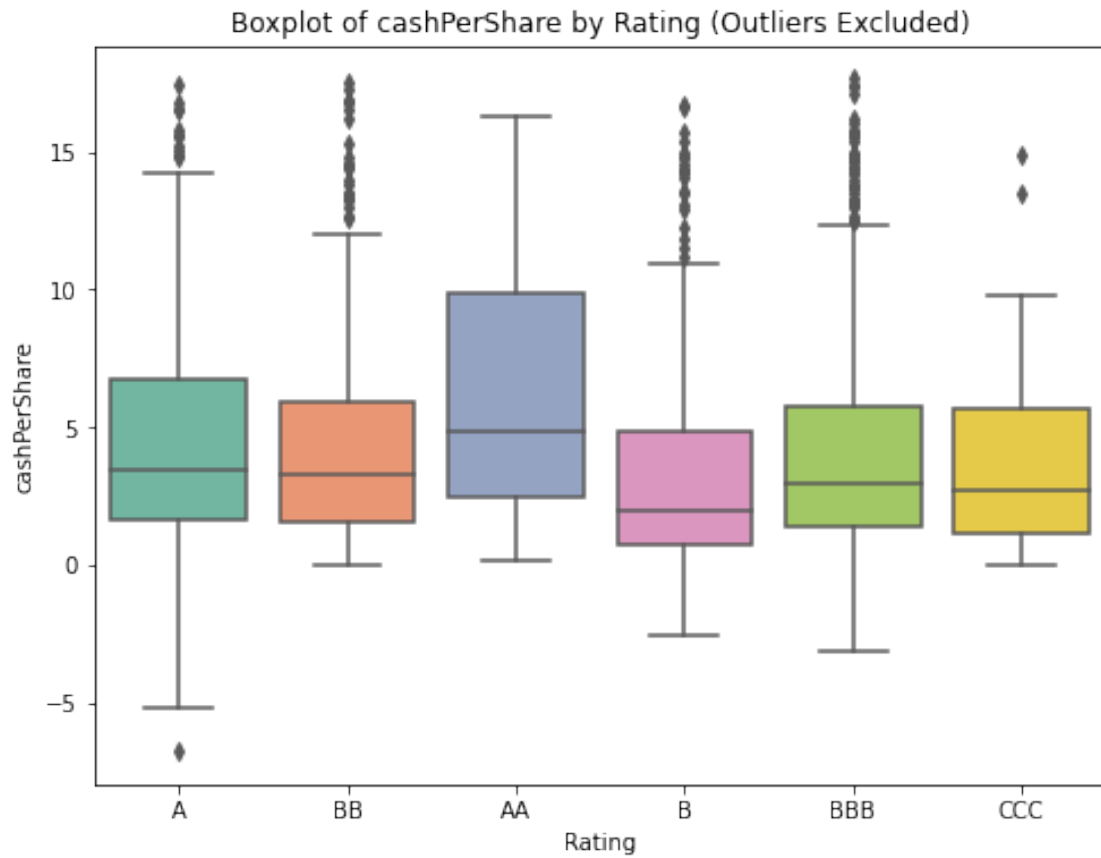


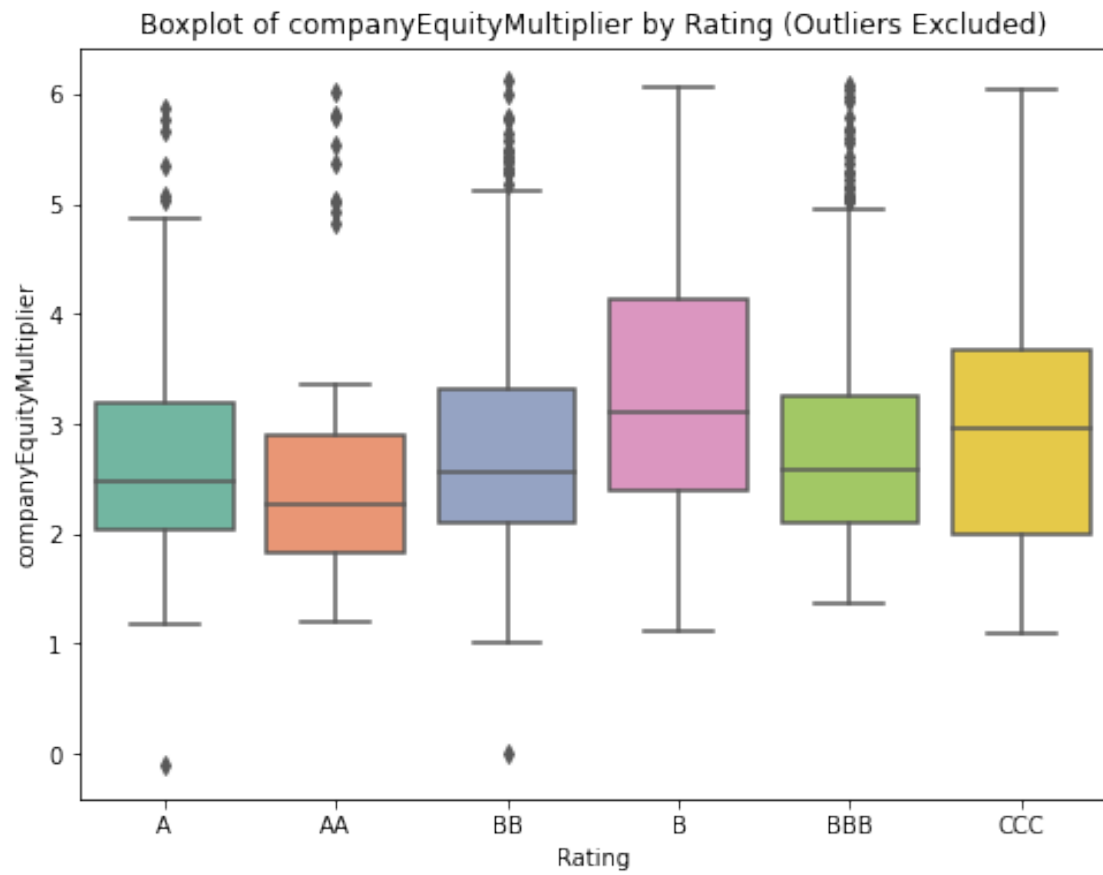


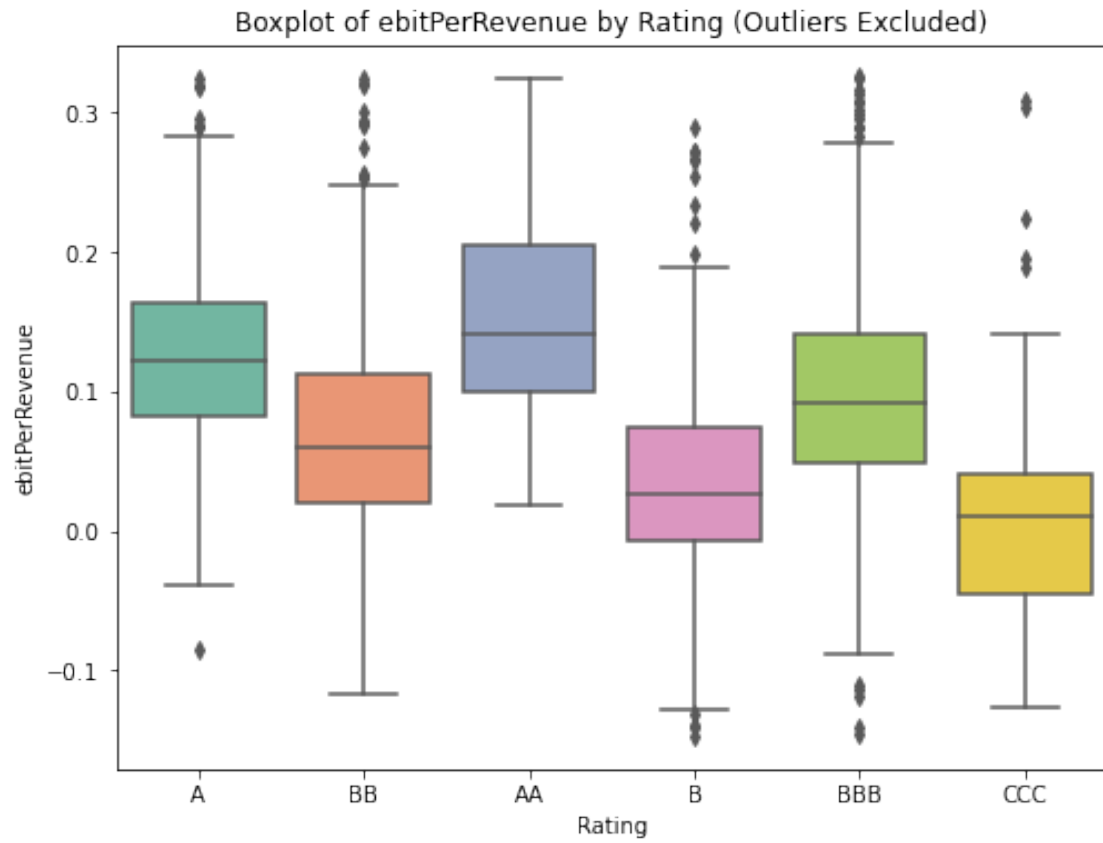


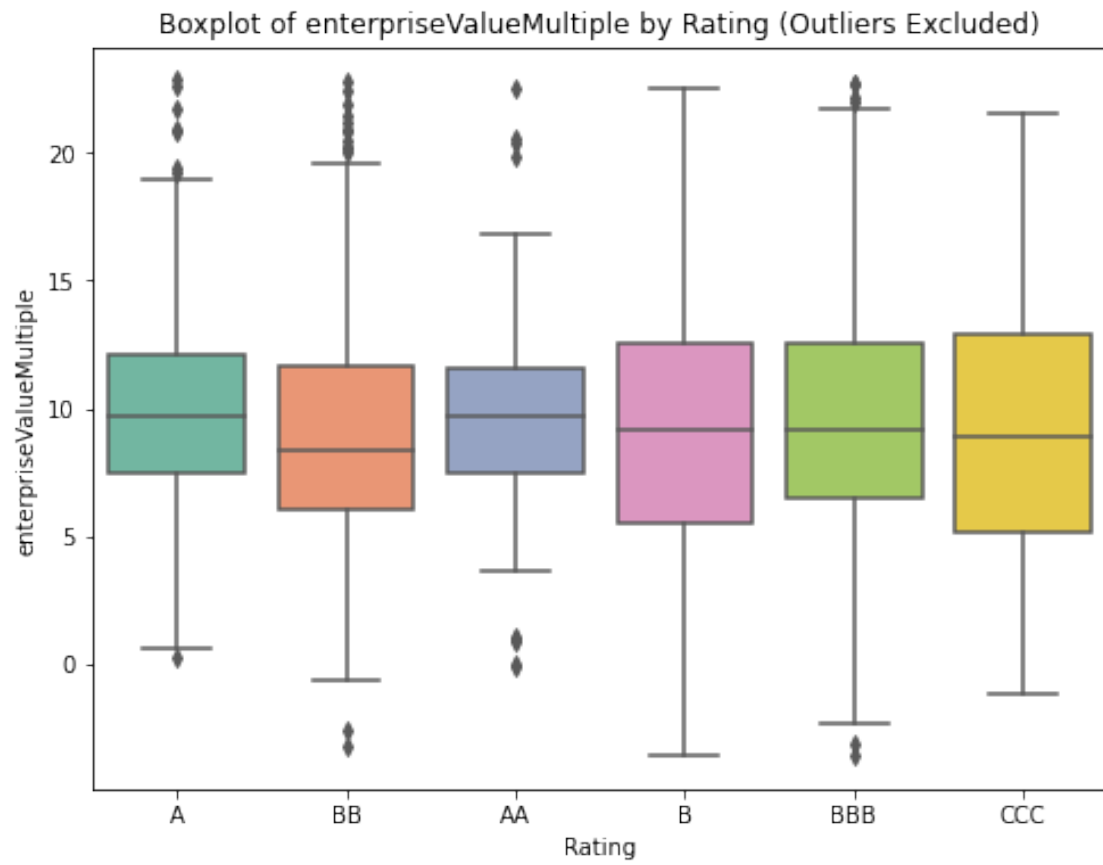


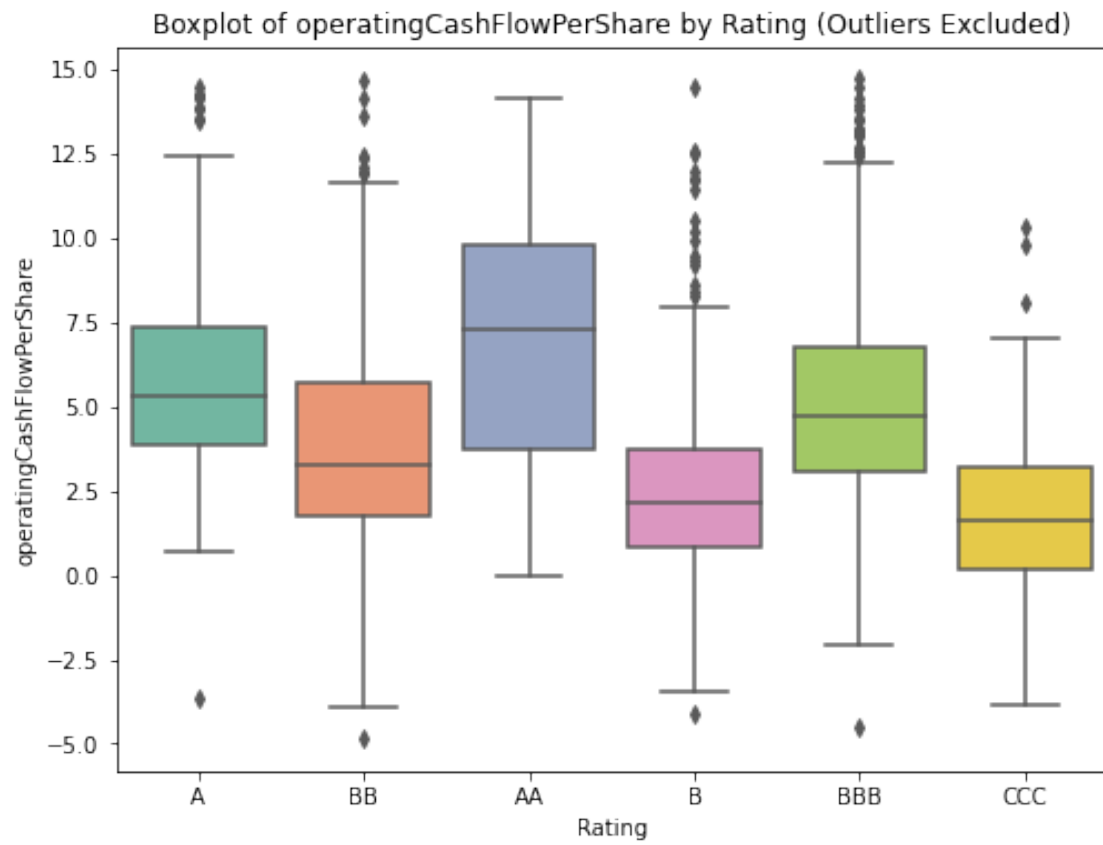


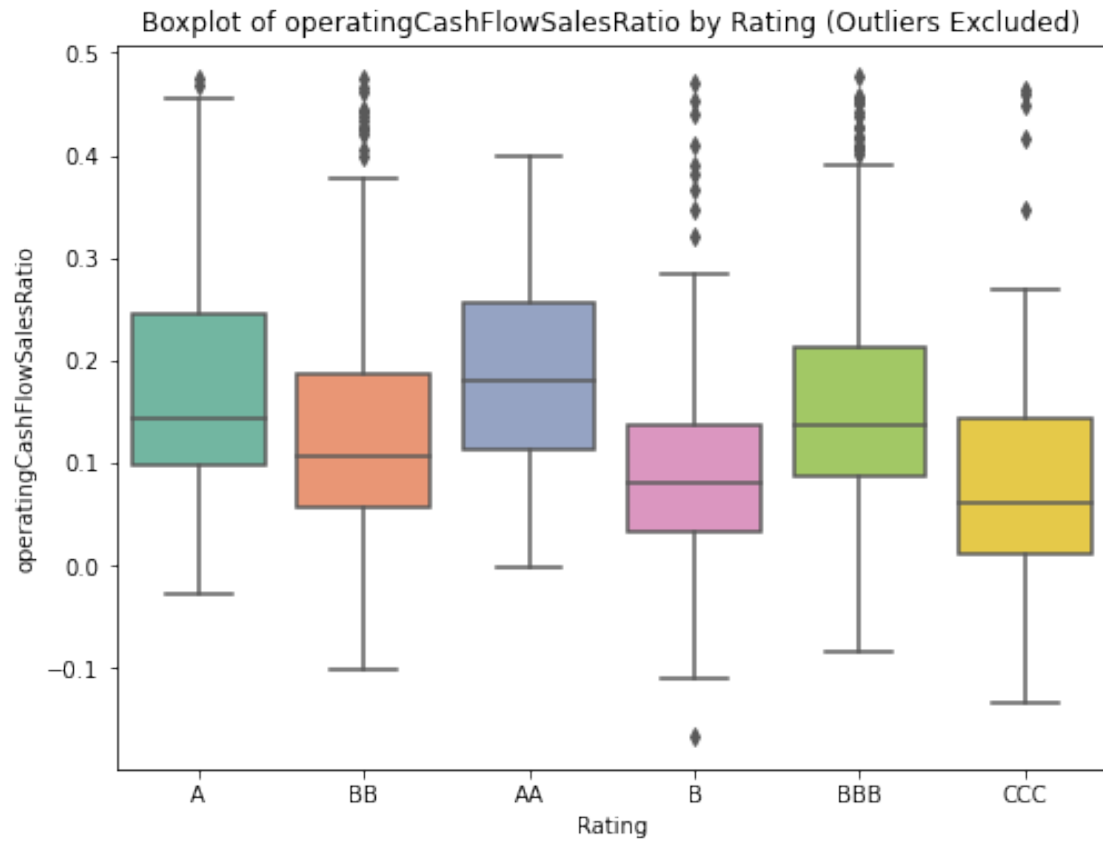


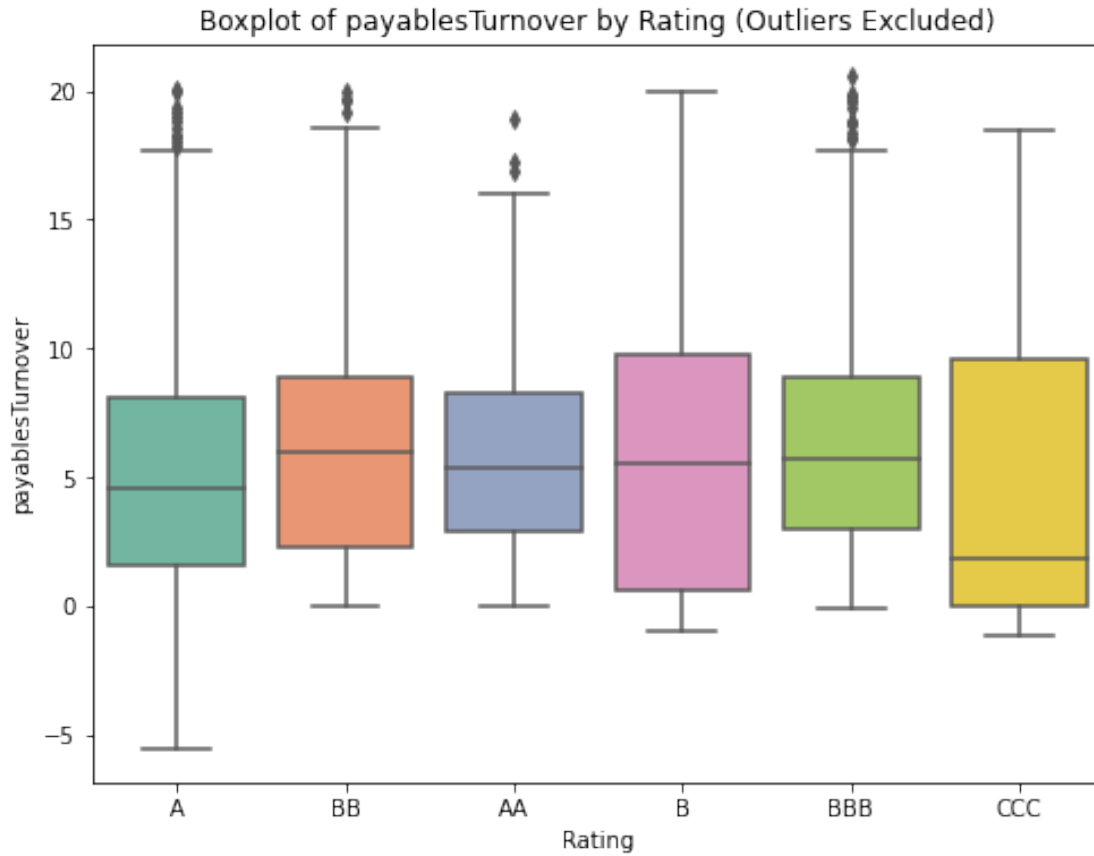












```
[287]: credit_rating_new[predictors].describe()
```

```
[287]:
```

	currentRatio	quickRatio	cashRatio	daysOfSalesOutstanding \
count	1945.000000	1945.000000	1945.000000	1945.000000
mean	3.597772	2.716067	0.670611	320.315483
std	44.987112	33.646595	3.652565	4401.394331
min	-0.932005	-1.893266	-0.192736	-811.845623
25%	1.071930	0.602825	0.130476	22.751296
50%	1.497207	0.988258	0.295357	42.285644
75%	2.172051	1.457053	0.624856	59.305426
max	1725.505005	1139.541703	125.917417	115961.637400

	netProfitMargin	pretaxProfitMargin	grossProfitMargin \
count	1945.000000	1945.000000	1945.000000
mean	0.184920	0.286879	0.493404
std	4.258280	5.915047	0.530818
min	-101.845815	-124.343612	-14.800817
25%	0.020894	0.025649	0.232565
50%	0.064219	0.084462	0.411563
75%	0.112164	0.143353	0.831541

max	97.451045	145.476348	2.702533
-----	-----------	------------	----------

	operatingProfitMargin	returnOnAssets	returnOnCapitalEmployed	...	\
count	1945.000000	1945.000000	1945.000000	...	
mean	0.396677	-18.464996	-32.358860	...	
std	6.714621	766.816695	1363.486614	...	
min	-124.343612	-33753.148610	-60057.887120	...	
25%	0.044003	0.018757	0.027902	...	
50%	0.106727	0.045163	0.074098	...	
75%	0.173932	0.076574	0.135036	...	
max	182.216722	0.487826	2.439504	...	

	effectiveTaxRate	freeCashFlowOperatingCashFlowRatio	\
count	1945.000000	1945.000000	
mean	0.411961	0.407290	
std	10.817706	3.873042	
min	-100.611015	-120.916010	
25%	0.154696	0.270574	
50%	0.301064	0.644529	
75%	0.370814	0.837470	
max	429.926282	34.594086	

	freeCashFlowPerShare	cashPerShare	companyEquityMultiplier	\
count	1.945000e+03	1.945000e+03	1945.000000	
mean	2.348574e+03	1.938737e+03	4.736067	
std	7.423563e+04	6.210600e+04	67.955950	
min	-4.912742e+03	-1.915035e+01	-833.014260	
25%	4.203961e-01	1.556122e+00	2.052242	
50%	2.144184e+00	3.681818e+00	2.663527	
75%	4.275228e+00	8.011616e+00	3.689693	
max	2.770492e+06	2.508197e+06	2562.871795	

	ebitPerRevenue	enterpriseValueMultiple	operatingCashFlowPerShare	\
count	1945.000000	1945.000000	1.945000e+03	
mean	0.292812	50.008844	3.479583e+03	
std	5.914120	540.360273	1.076161e+05	
min	-124.343612	-3749.921337	-1.195049e+04	
25%	0.028057	6.313191	2.368970e+00	
50%	0.087042	9.358311	4.376682e+00	
75%	0.147200	12.929346	7.301747e+00	
max	145.476348	11153.607090	3.836066e+06	

	operatingCashFlowSalesRatio	payablesTurnover
count	1945.000000	1945.000000
mean	1.146441	39.369944
std	12.369528	775.113340
min	-4.461837	-76.662850

25%	0.073886	2.239925
50%	0.132249	5.828326
75%	0.236323	9.615368
max	272.249725	20314.880400

[8 rows x 25 columns]

4 Substitute outliers with lower/ upper bound

```
[288]: def cap_outliers(data_inp, cols):

    data = data_inp.copy()
    for col in cols: # Process only numeric columns
        Q1 = data[col].quantile(0.25)
        Q3 = data[col].quantile(0.75)
        IQR = Q3 - Q1
        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR

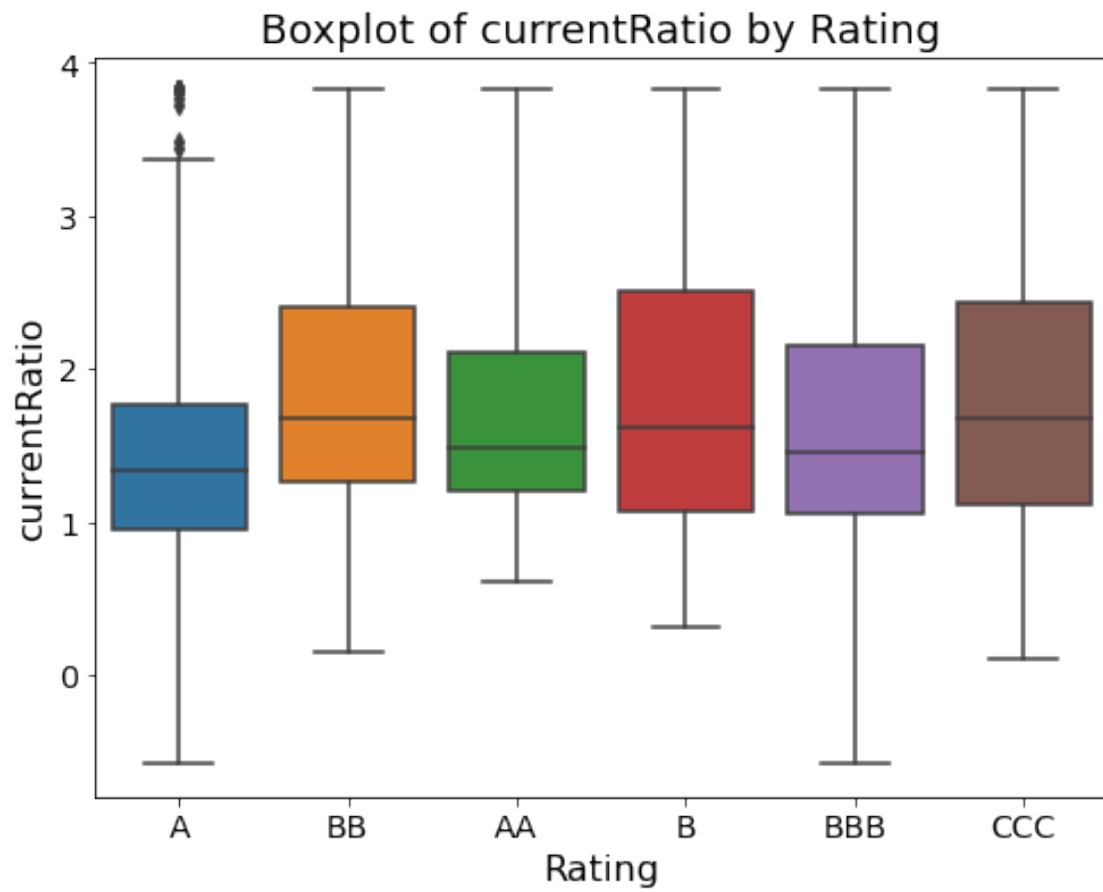
        data[col] = data[col].clip(lower=lower_bound, upper=upper_bound) #
    → Capping outliers

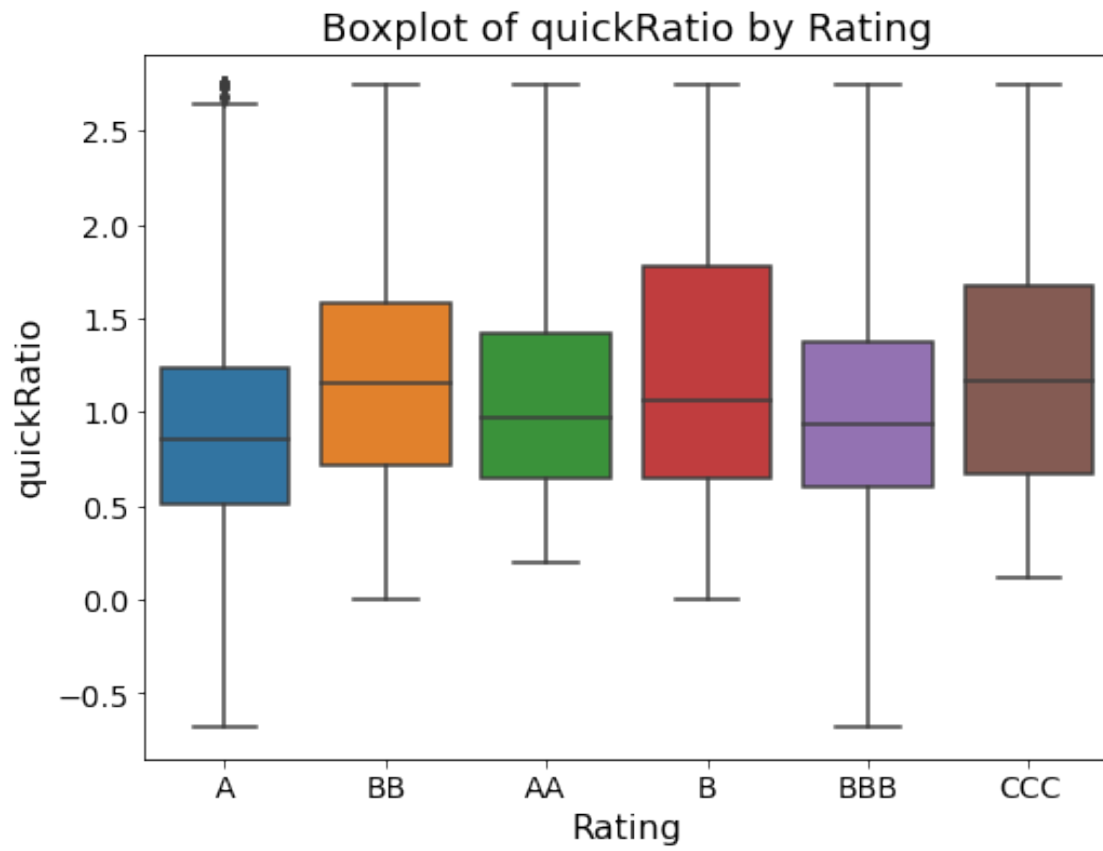
    return data

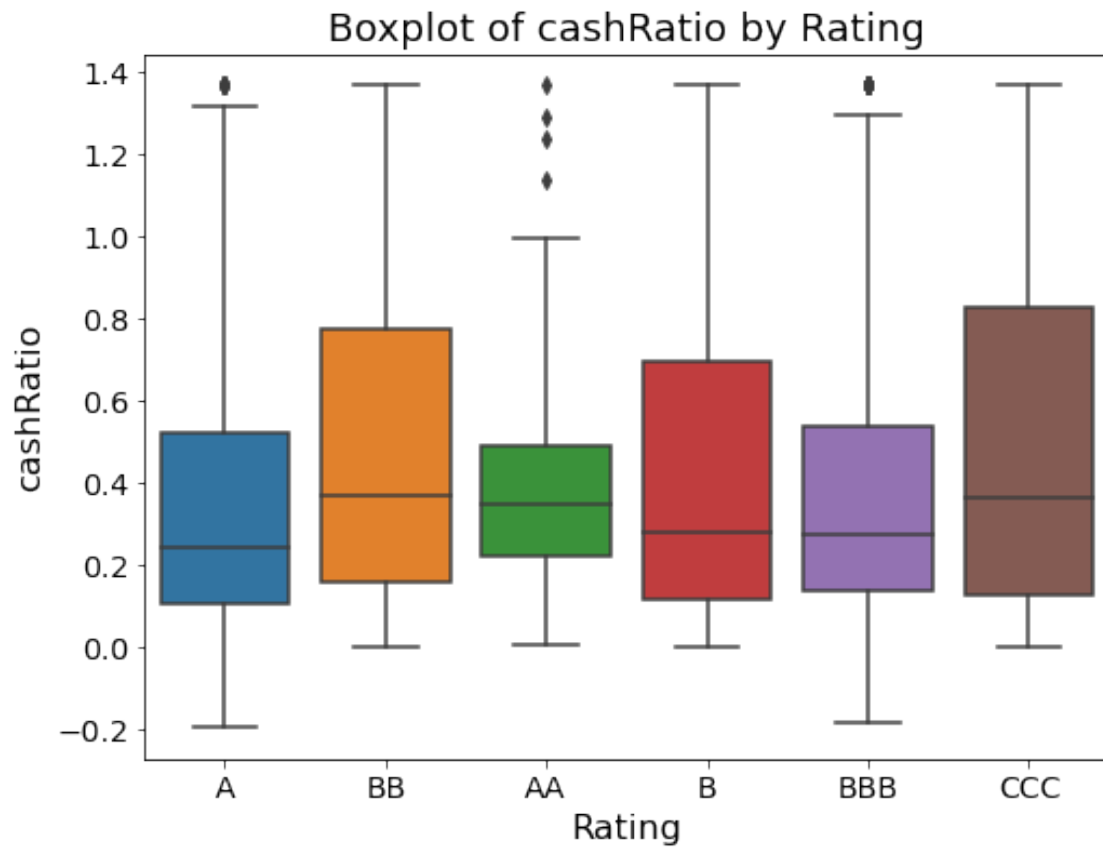
# Example usage:
# df = cap_outliers(df)
```

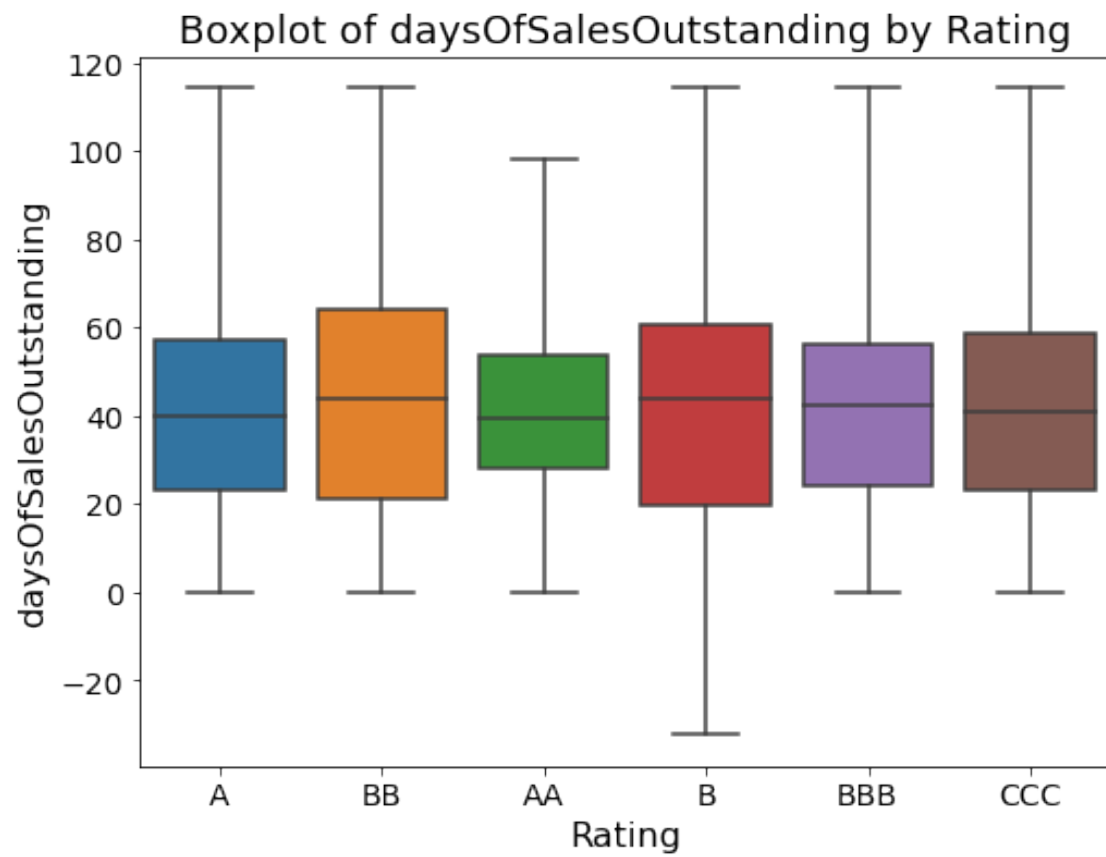
```
[289]: credit_rating_outl = cap_outliers(credit_rating_new.copy(), predictors)
```

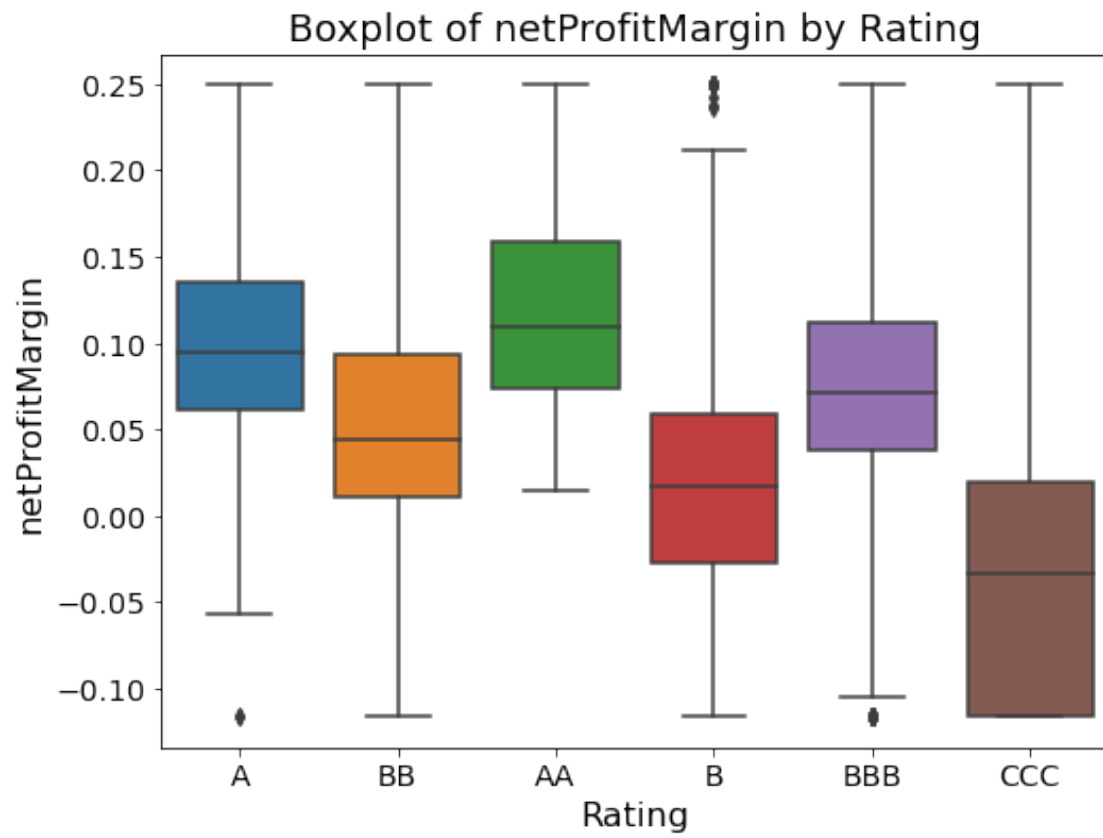
```
[290]: for pred in predictors:
        boxplt(credit_rating_outl, 'Rating', pred)
```

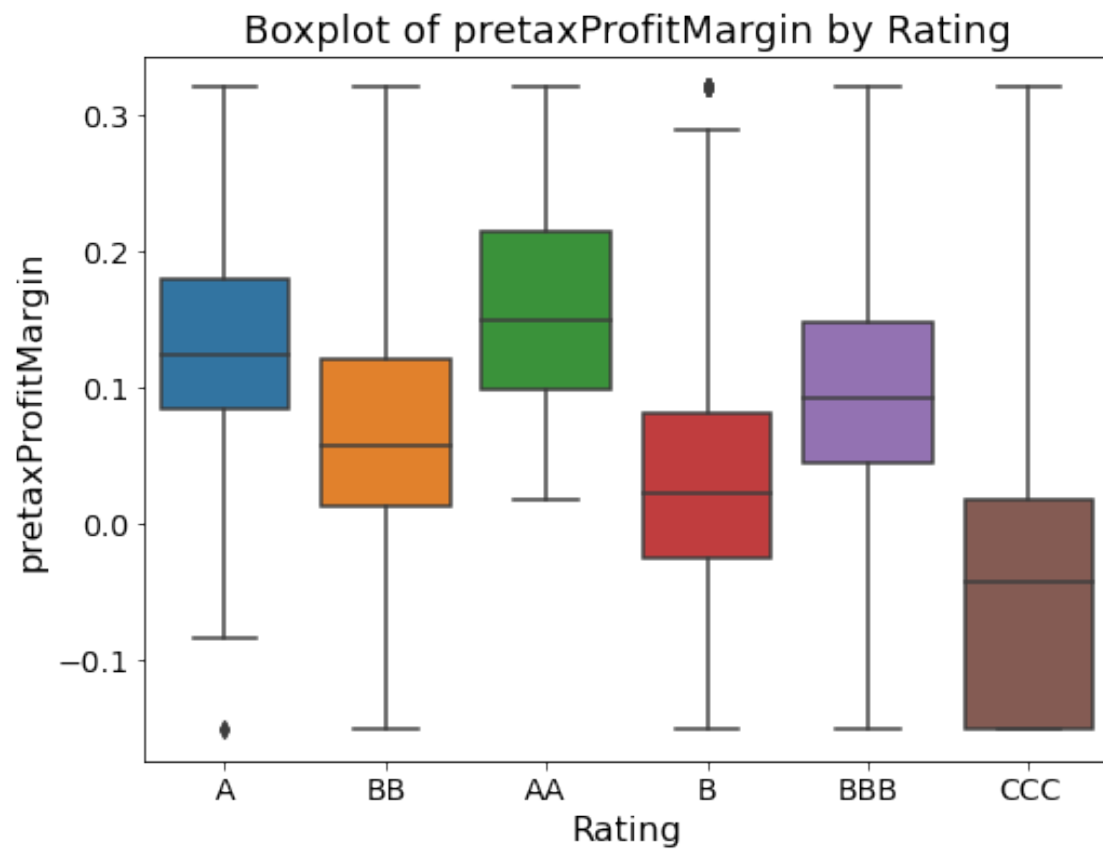


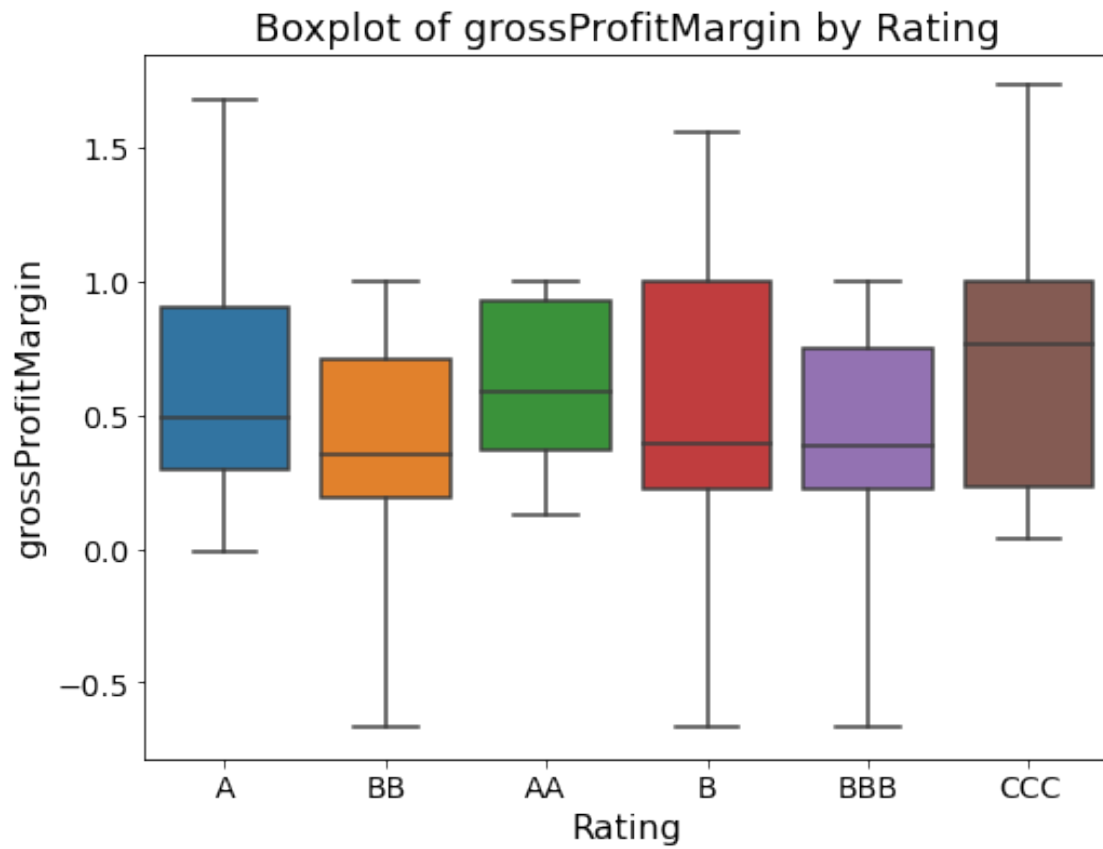


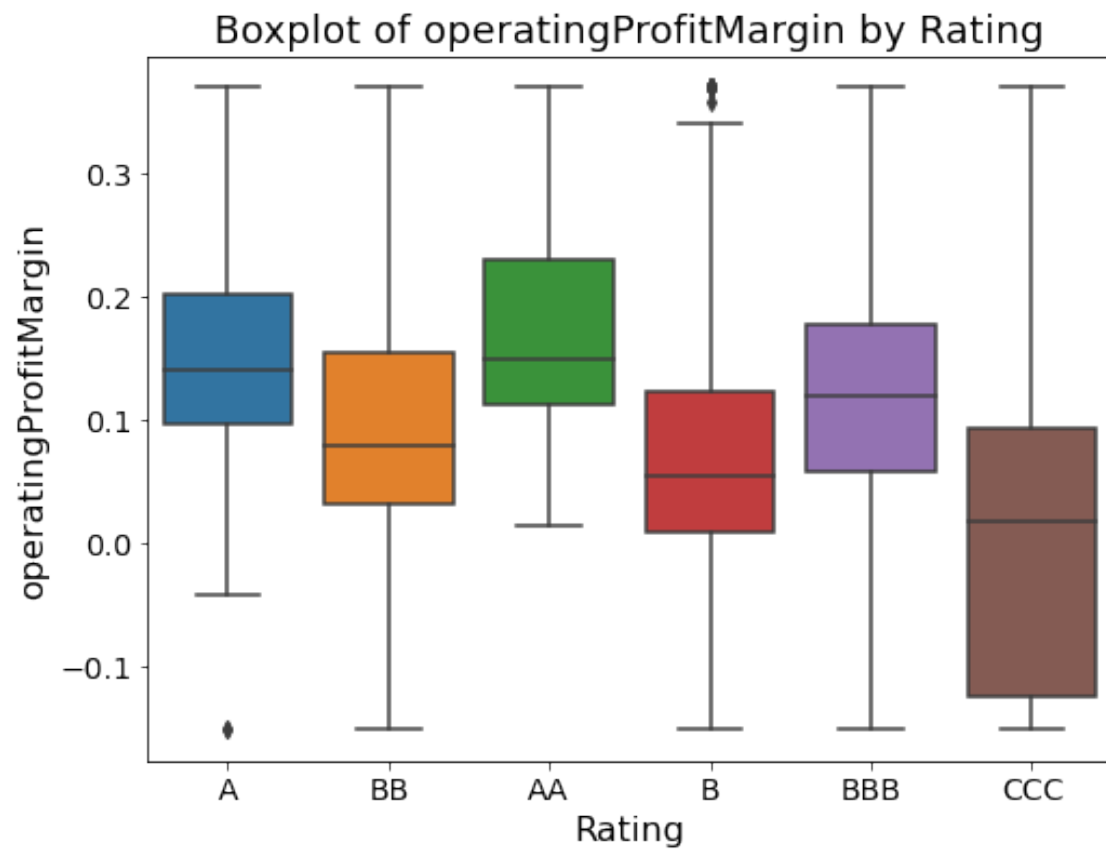


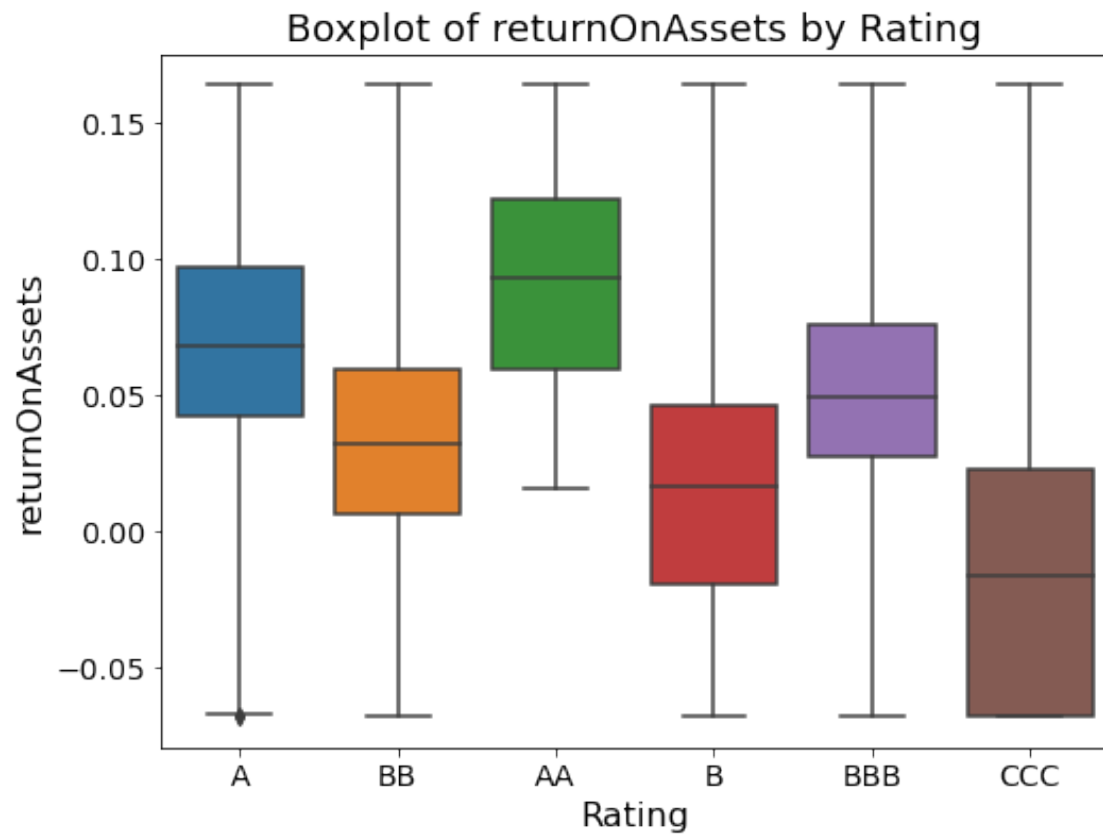




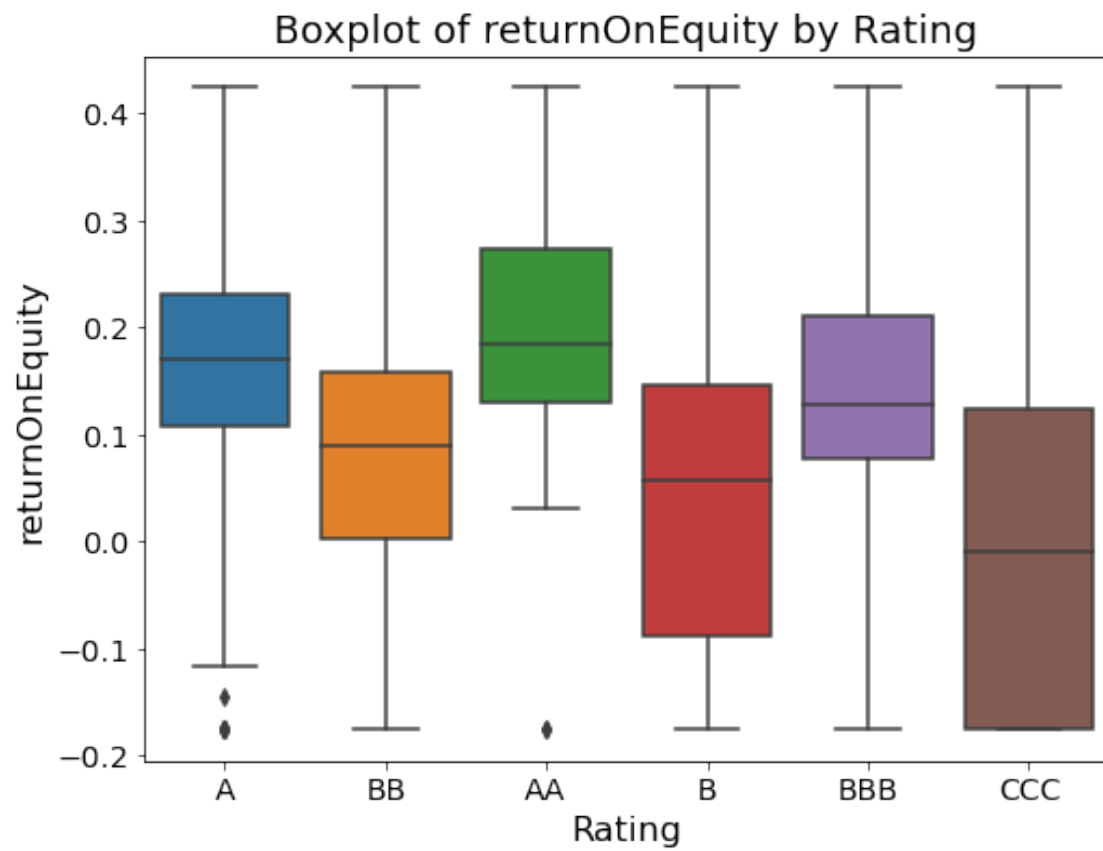


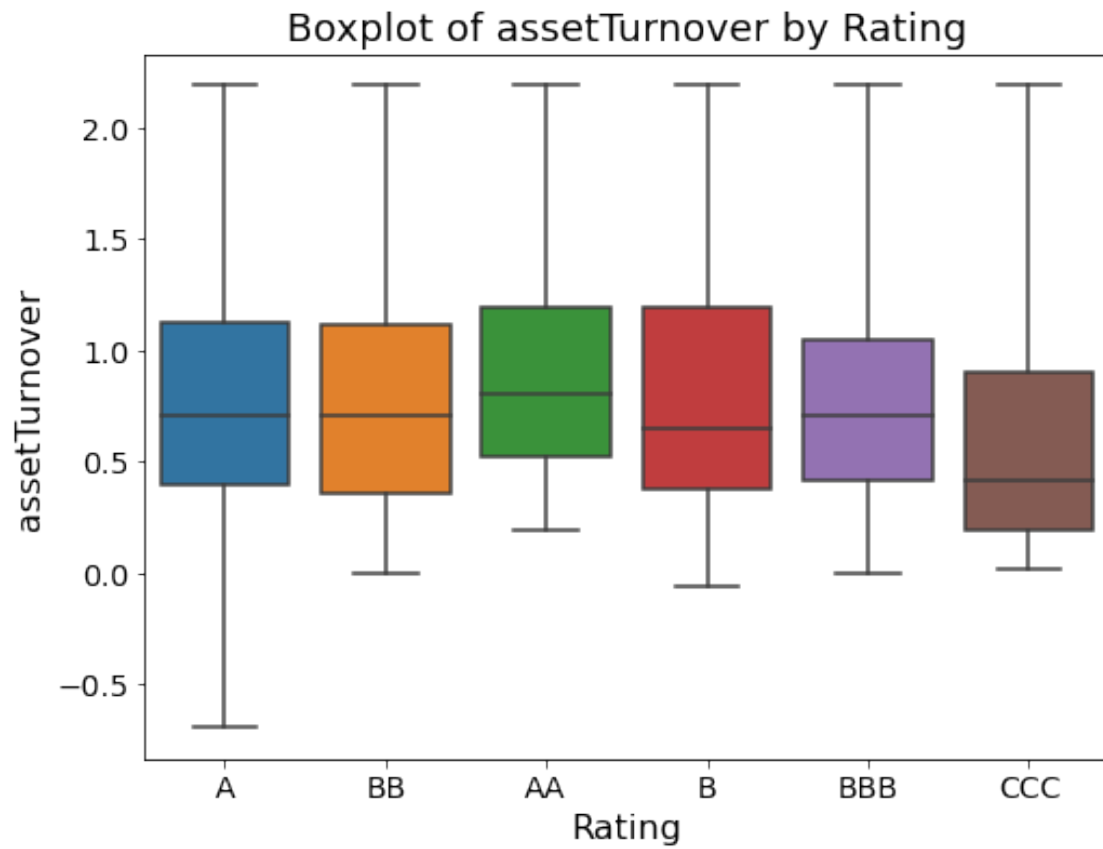


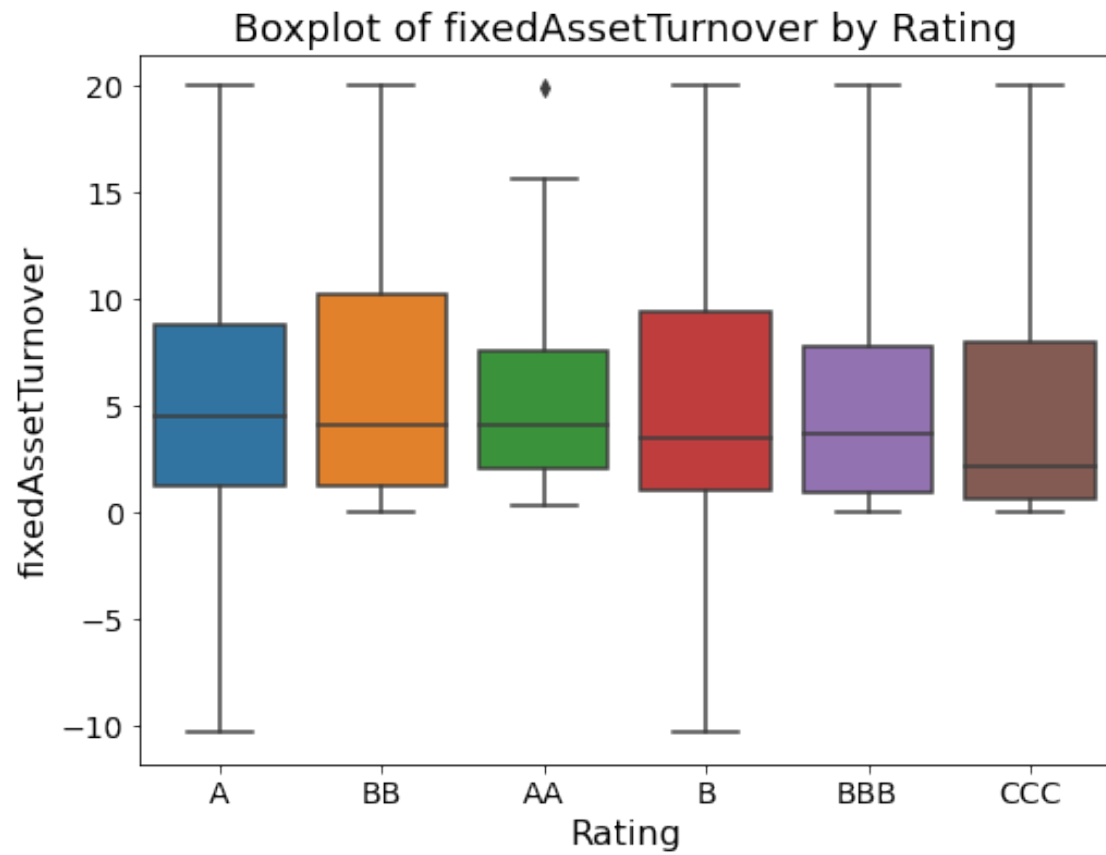


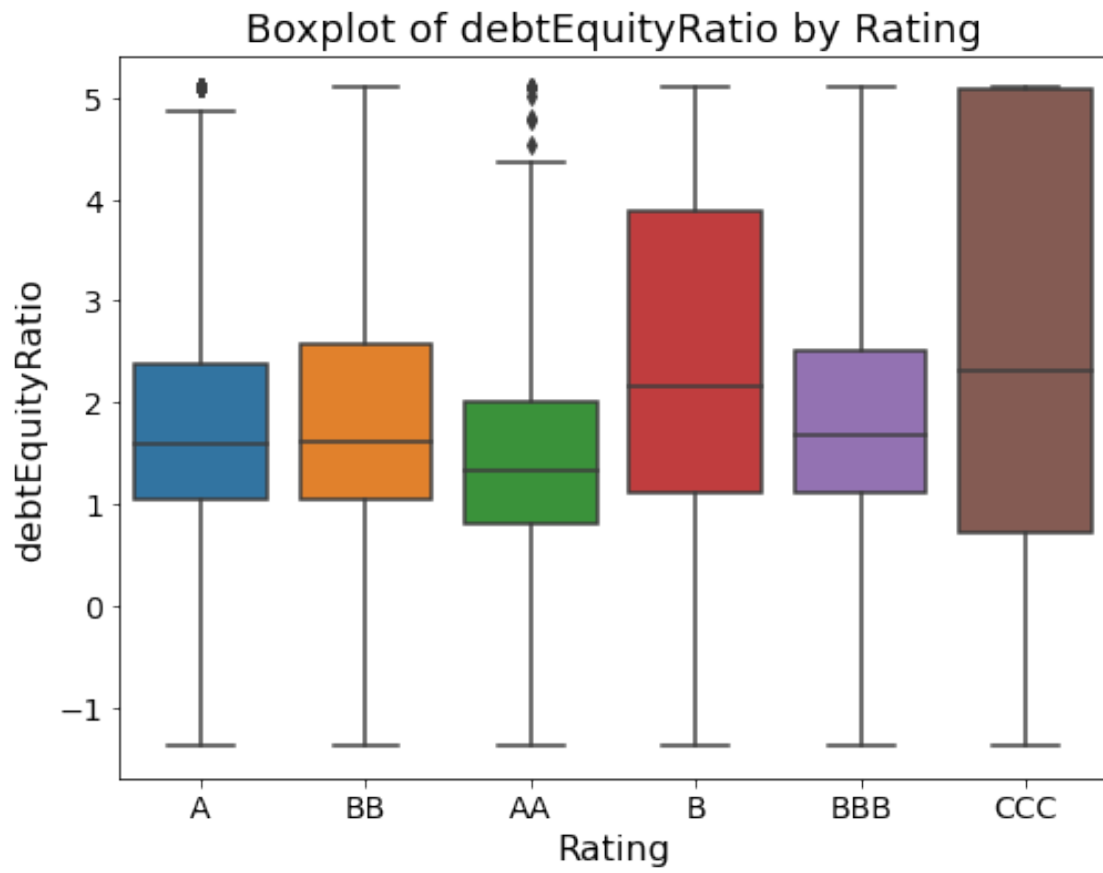


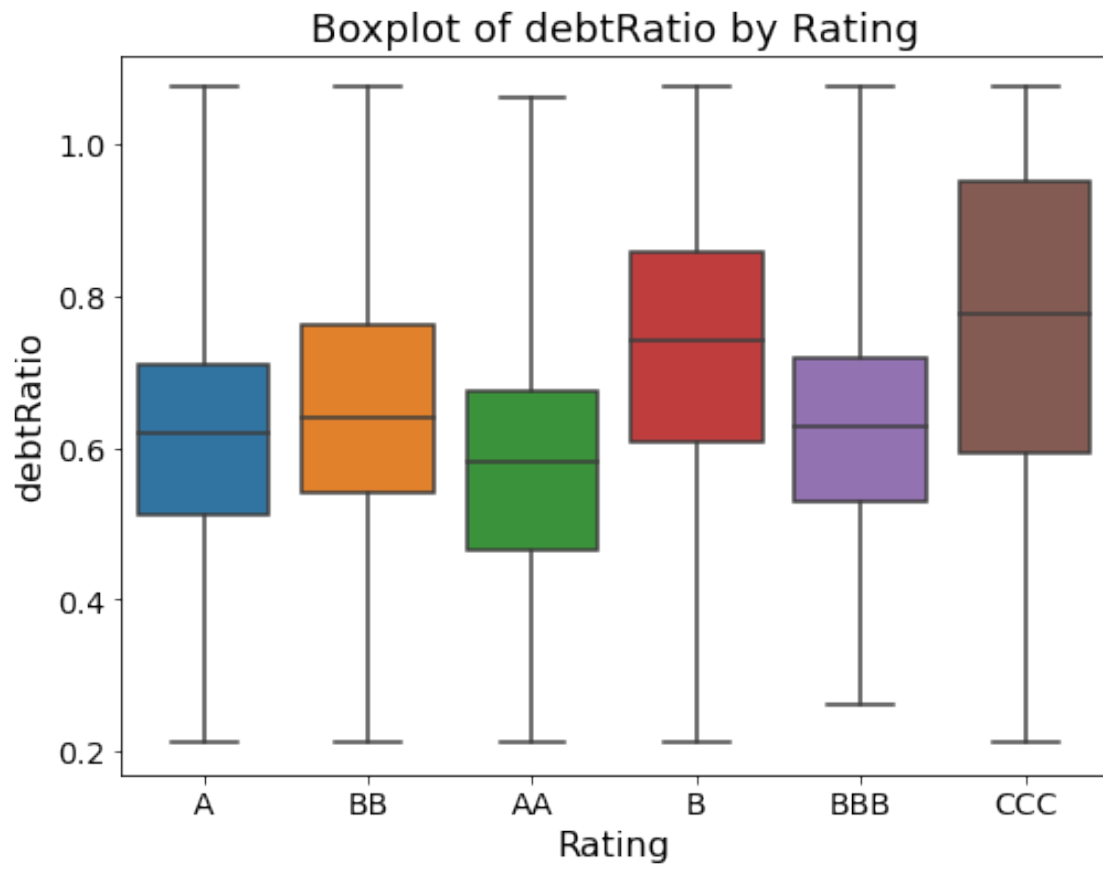


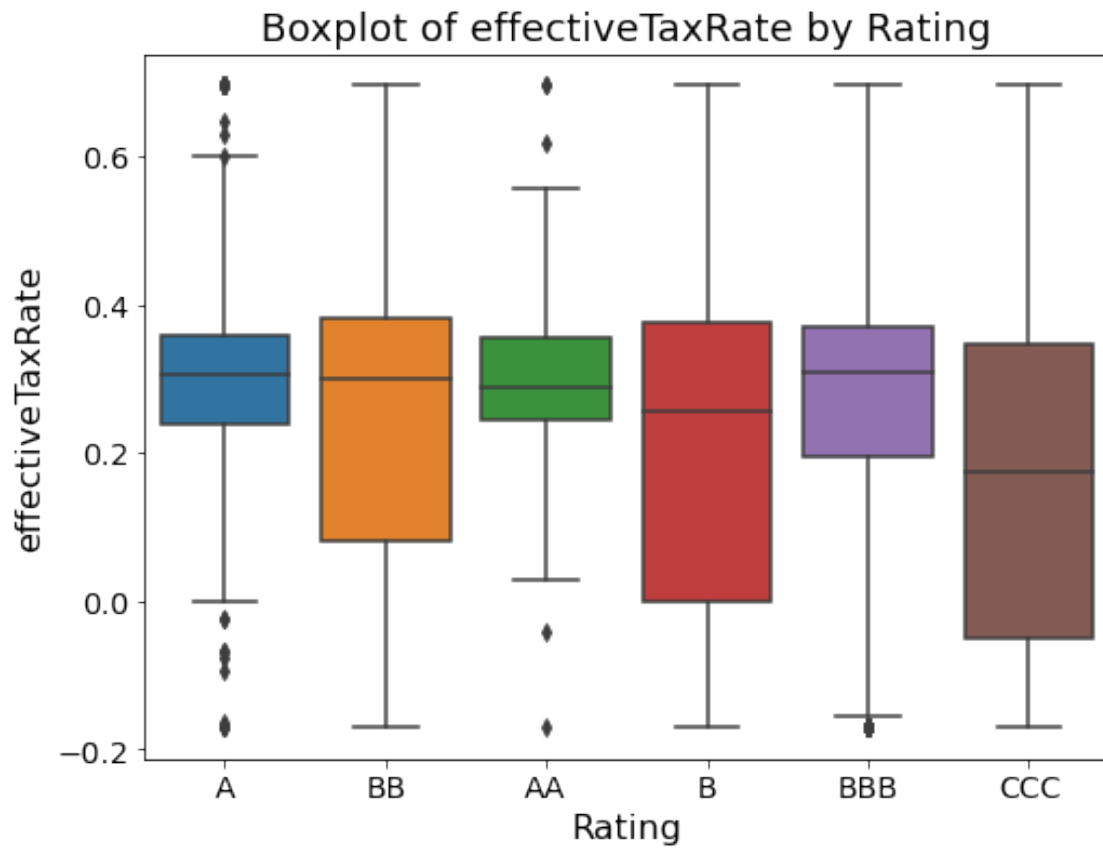


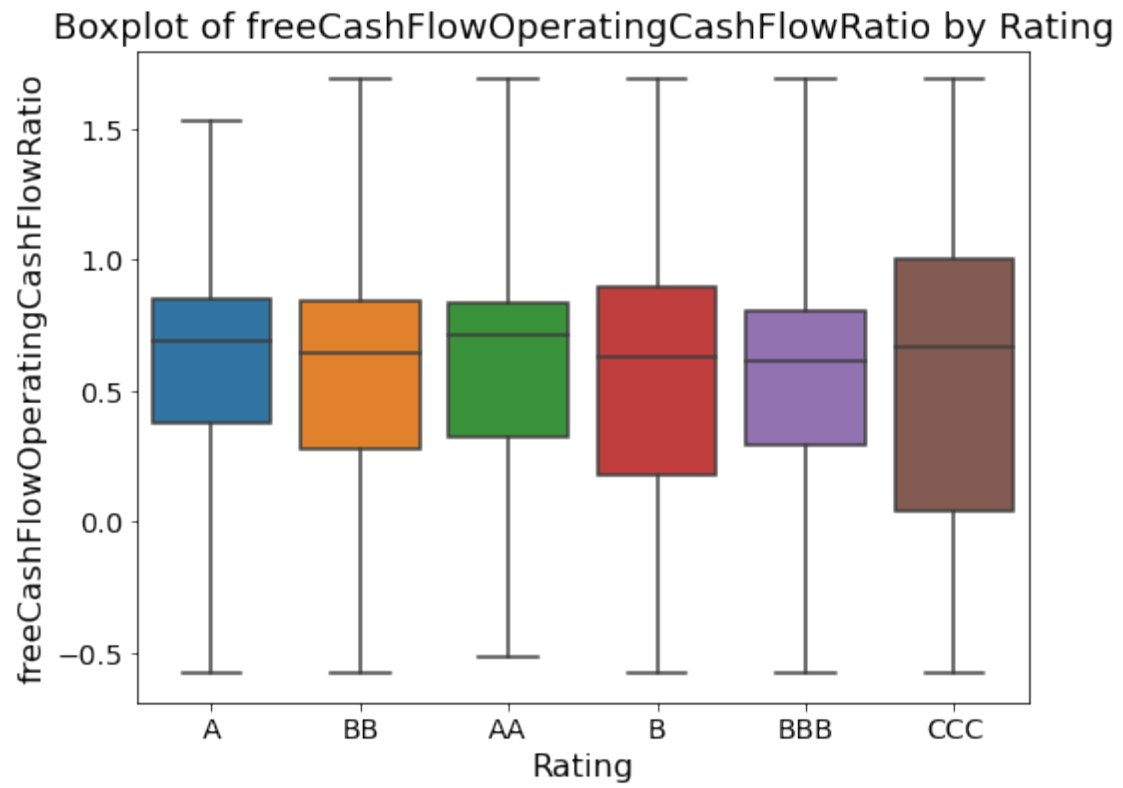


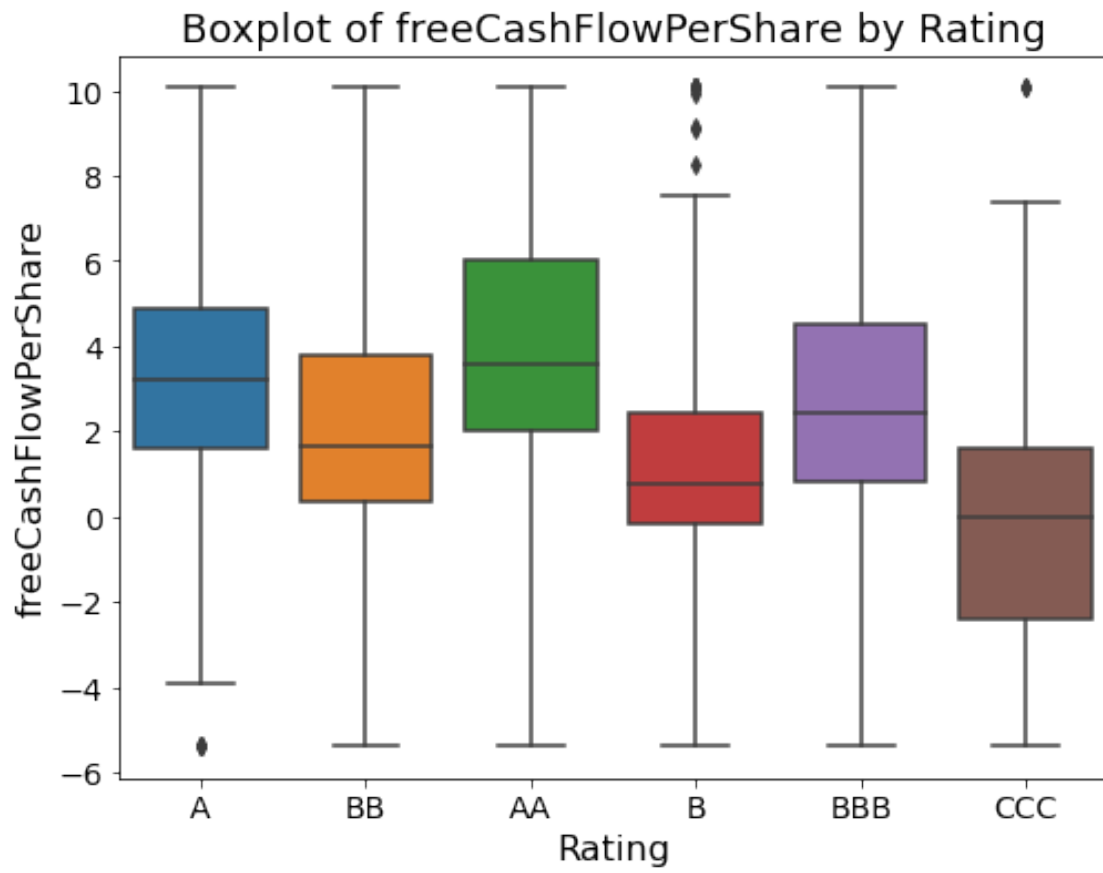


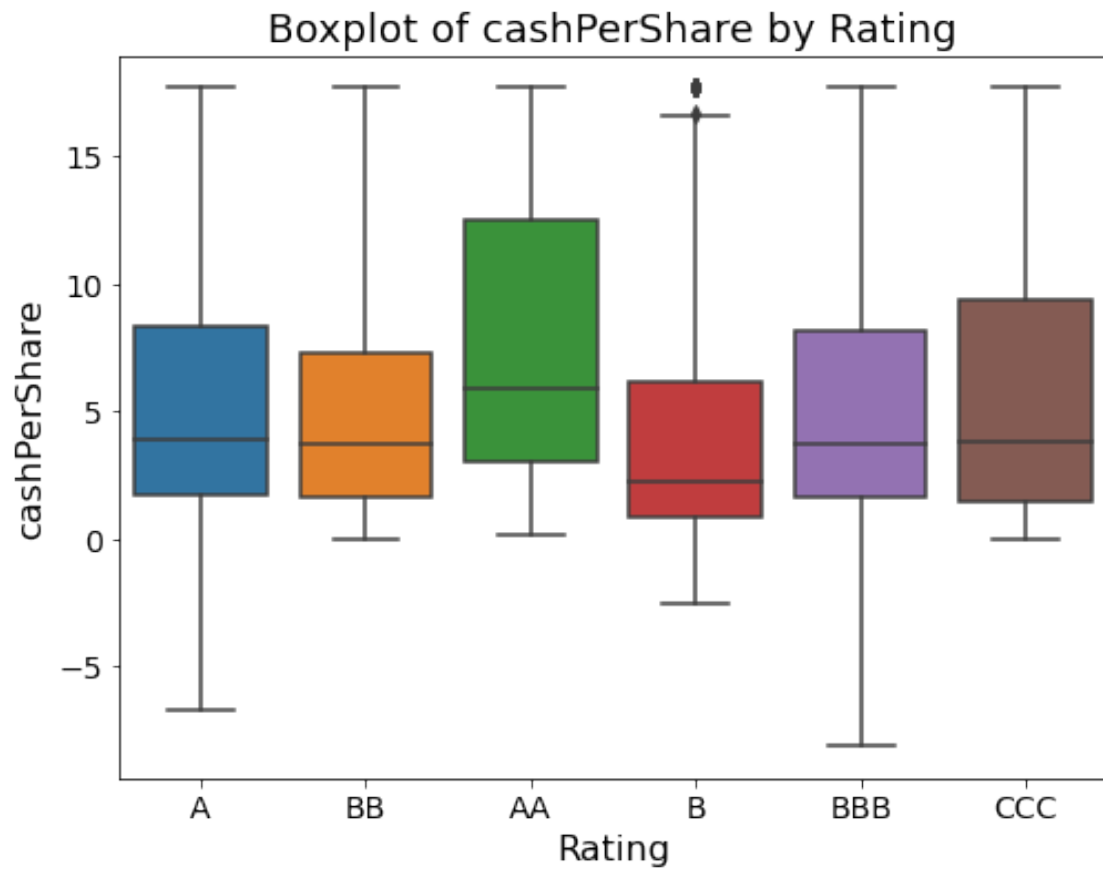


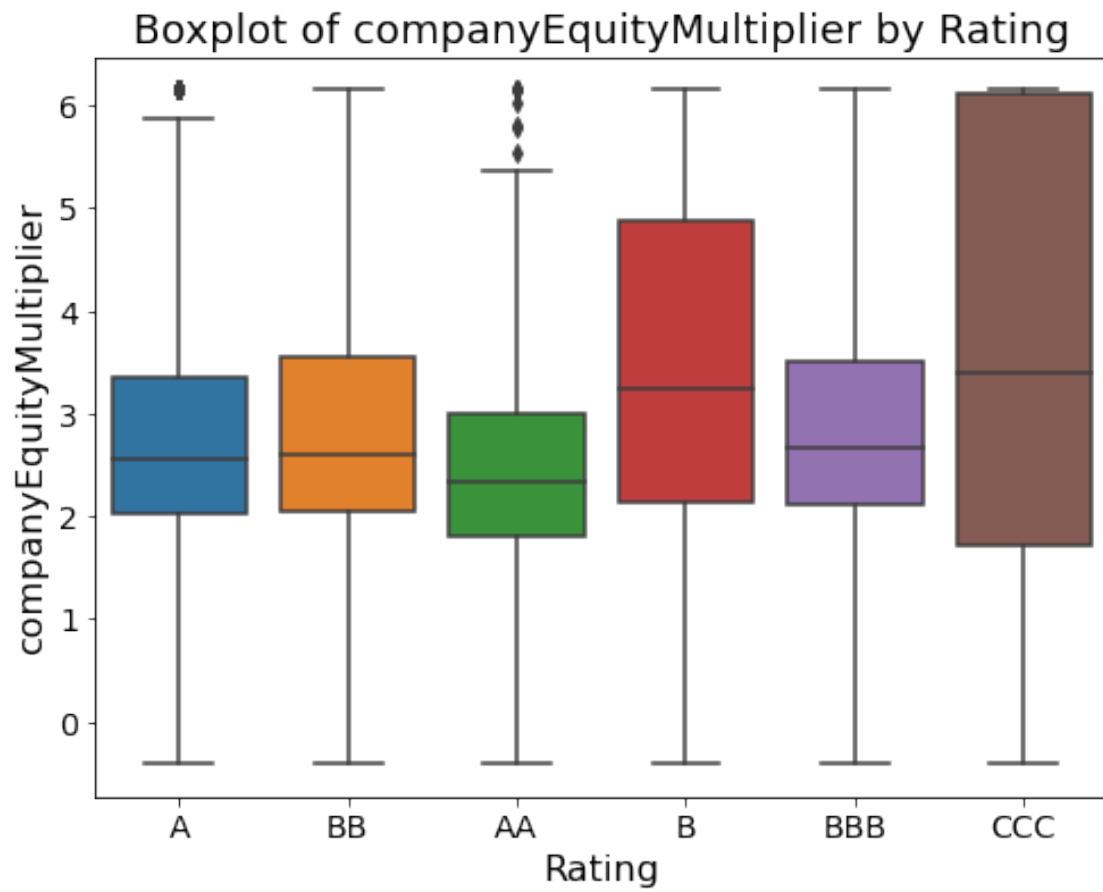


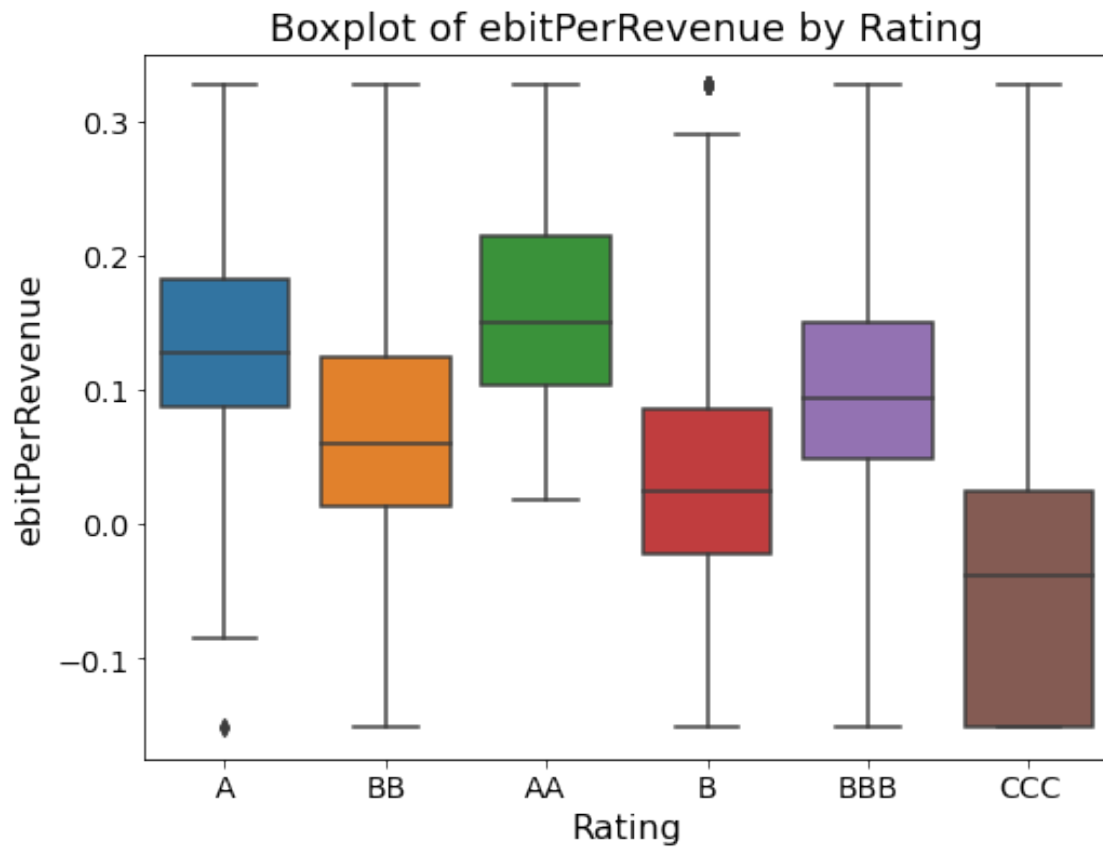


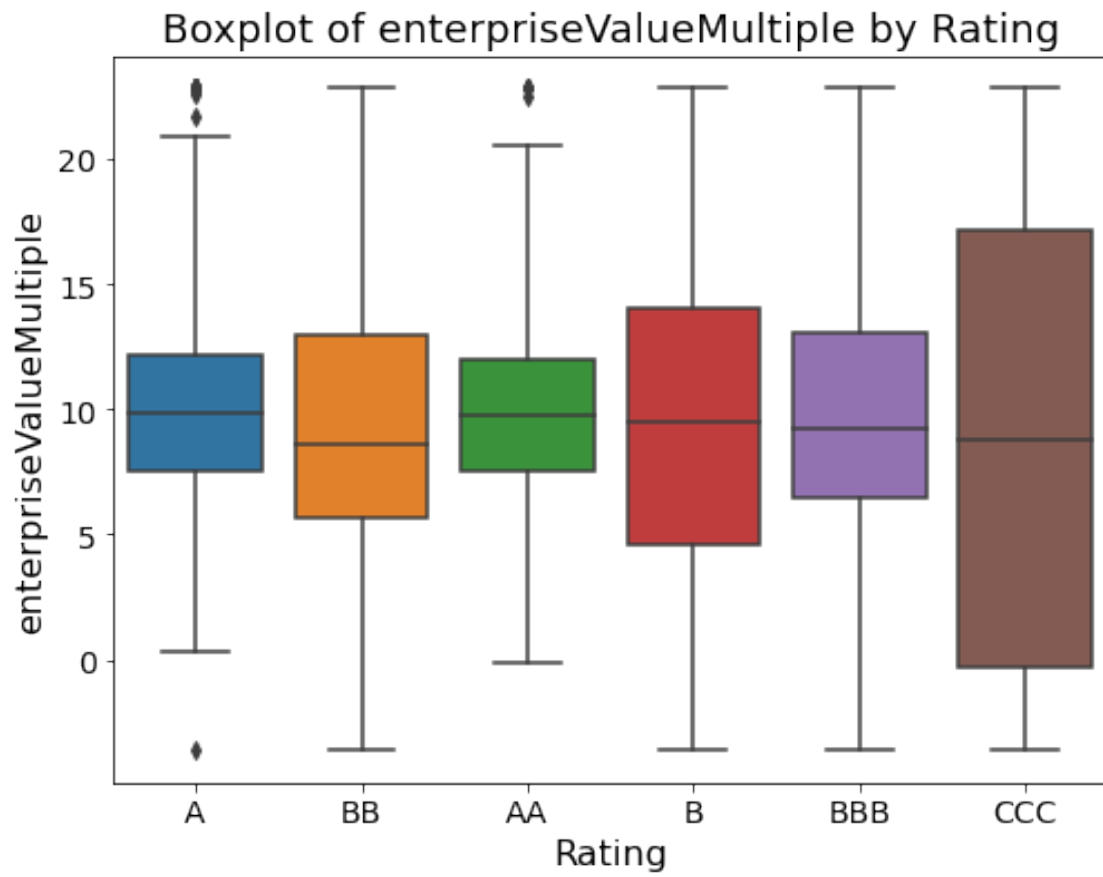


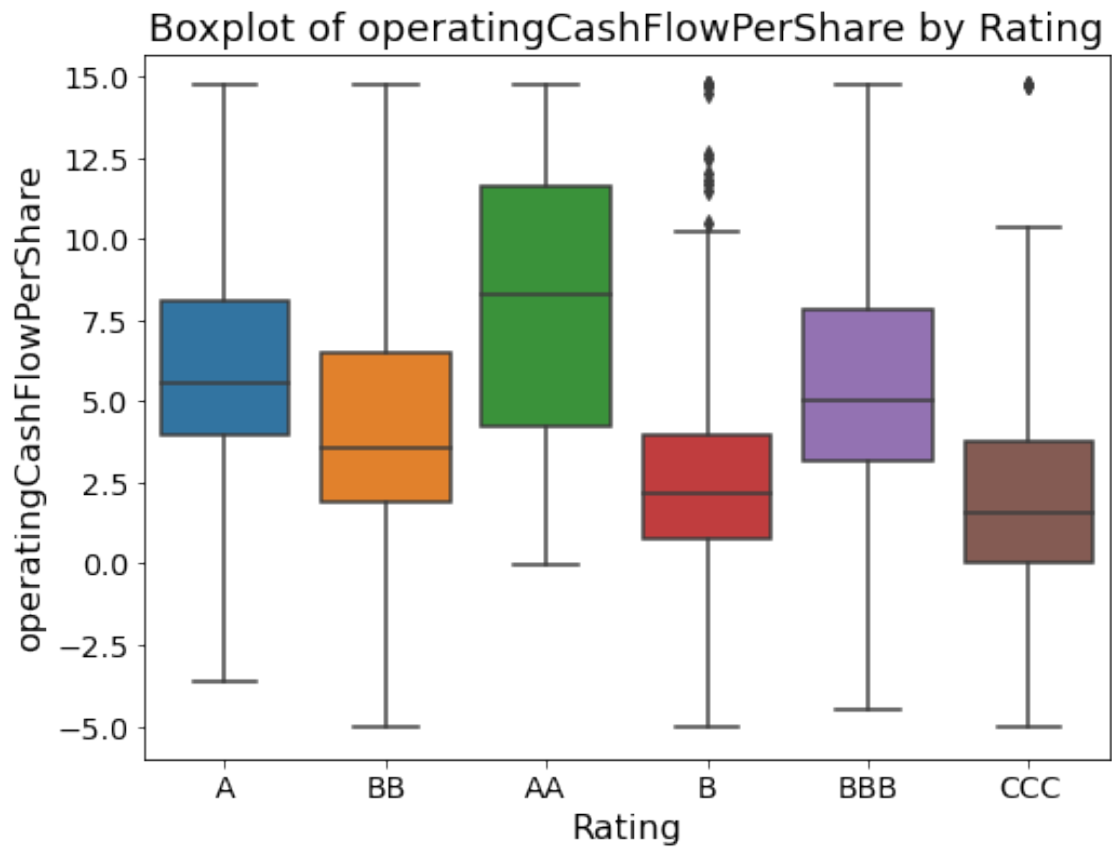


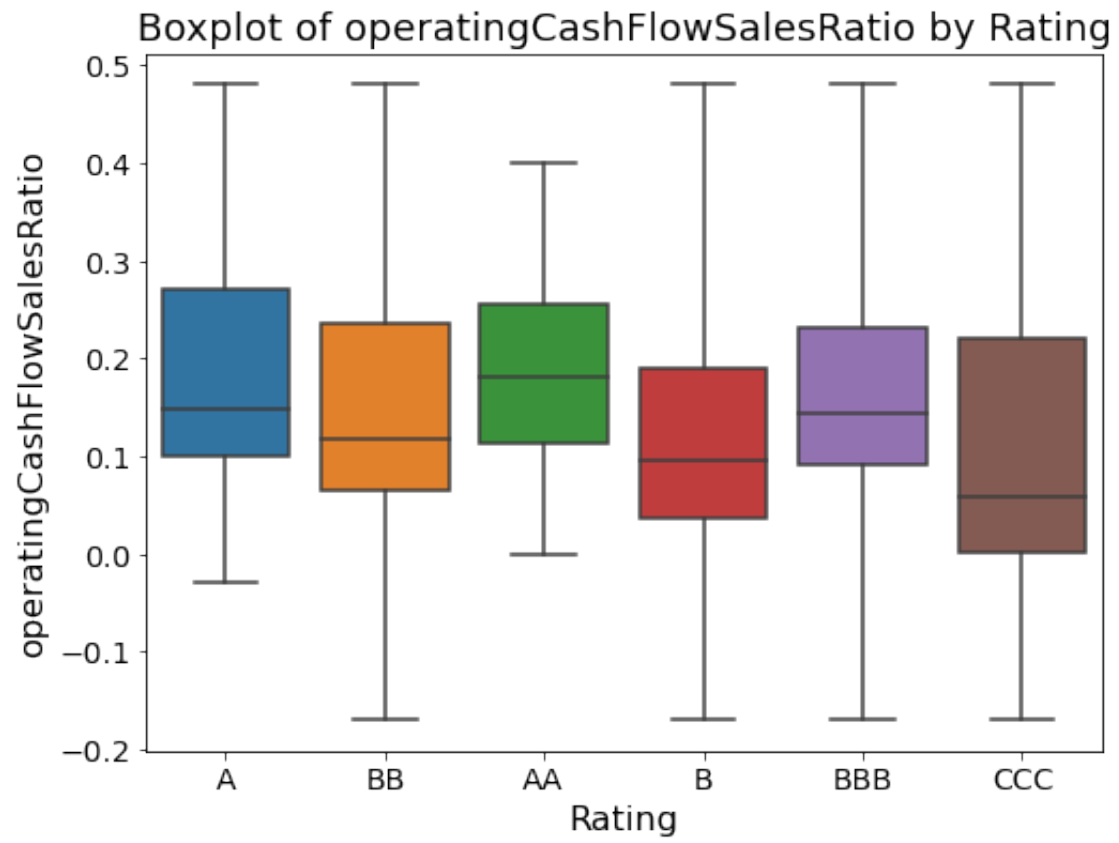


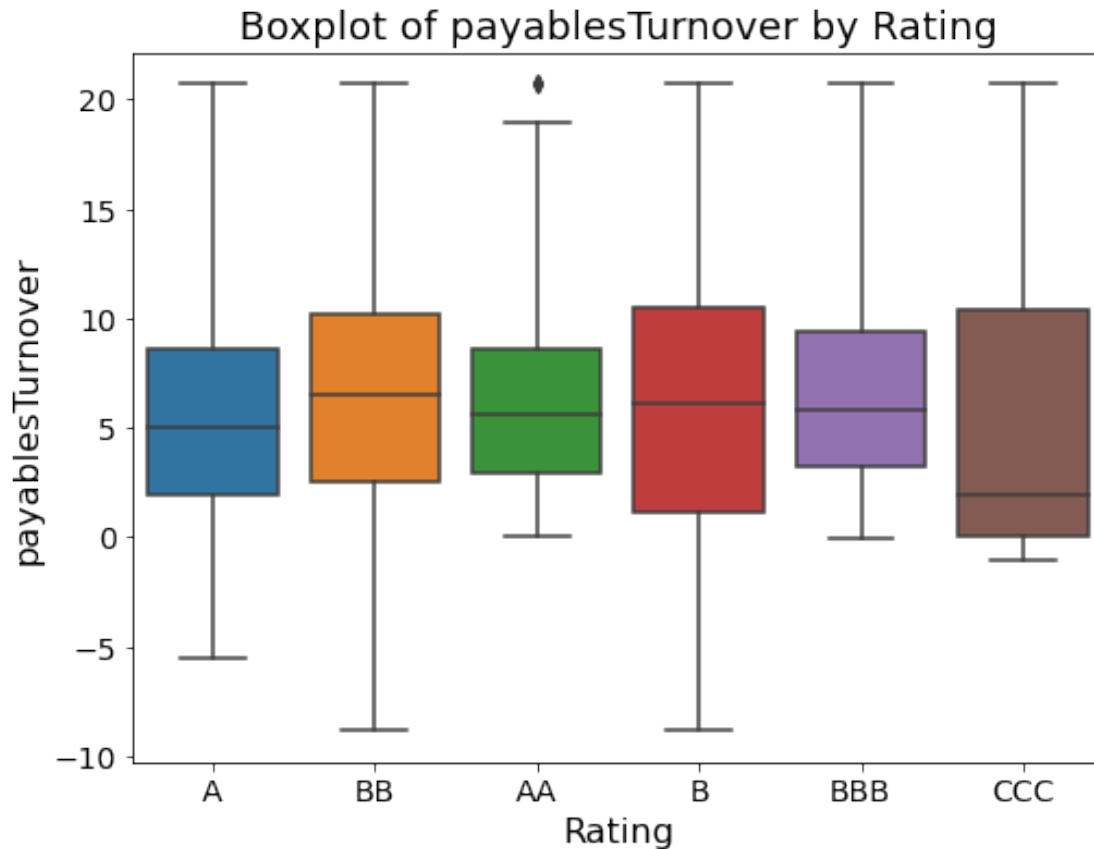












```
[291]: credit_rating_new[predictors].skew(axis=0)
```

```
[291]: currentRatio          33.632508
       quickRatio           30.280802
       cashRatio            26.695697
       daysOfSalesOutstanding 21.238001
       netProfitMargin        1.200326
       pretaxProfitMargin     7.271847
       grossProfitMargin     -14.348711
       operatingProfitMargin  12.320764
       returnOnAssets        -43.854023
       returnOnCapitalEmployed -43.939967
       returnOnEquity         43.753636
       assetTurnover          31.160855
       fixedAssetTurnover     31.162268
       debtEquityRatio        28.028643
       debtRatio              1.180238
       effectiveTaxRate       31.664193
       freeCashFlowOperatingCashFlowRatio -22.511800
       freeCashFlowPerShare   33.367829
```

```

cashPerShare                36.698888
companyEquityMultiplier     28.027029
ebitPerRevenue              7.272485
enterpriseValueMultiple     13.652395
operatingCashFlowPerShare   32.262304
operatingCashFlowSalesRatio 15.579984
payablesTurnover            25.376178
dtype: float64

```

```
[292]: credit_rating_outl[predictors].skew(axis=0)
```

```

[292]: currentRatio                0.852558
       quickRatio                 0.828302
       cashRatio                 1.114470
       daysOfSalesOutstanding    0.478921
       netProfitMargin           0.081697
       pretaxProfitMargin        0.075949
       grossProfitMargin         0.317761
       operatingProfitMargin     0.120608
       returnOnAssets            -0.049913
       returnOnCapitalEmployed   0.139242
       returnOnEquity            0.015720
       assetTurnover             0.955855
       fixedAssetTurnover        1.151884
       debtEquityRatio           0.433598
       debtRatio                 0.313160
       effectiveTaxRate          -0.274405
       freeCashFlowOperatingCashFlowRatio -0.733025
       freeCashFlowPerShare      0.315184
       cashPerShare              1.117583
       companyEquityMultiplier   0.430488
       ebitPerRevenue            0.093537
       enterpriseValueMultiple   0.243007
       operatingCashFlowPerShare 0.726208
       operatingCashFlowSalesRatio 0.816839
       payablesTurnover          0.818221
dtype: float64

```

```

[293]: # Compute skewness for original and outlier-substituted datasets
skewness_orig = pd.DataFrame(credit_rating_new[predictors].skew(axis=0).
    ↳round(1), columns=['Skewness_Original'])
skewness_outl = pd.DataFrame(credit_rating_outl[predictors].skew(axis=0).
    ↳round(1), columns=['Skewness_After_Outliers'])

# Combine both datasets
skewness_combined = pd.concat([skewness_orig, skewness_outl], axis=1)

```

```

# Add predictors as a column instead of index
skewness_combined.reset_index(inplace=True)
skewness_combined.rename(columns={'index': 'Predictors'}, inplace=True)

# Display the combined DataFrame
skewness_combined

# skewness_orig = pd.DataFrame(credit_rating_new[predictors].skew(axis=0).
↳round(1), columns=['Skewness'])
# skewness_outl = pd.DataFrame(credit_rating_outl[predictors].skew(axis=0).
↳round(1), columns=['Skewness'])

```

[293]:

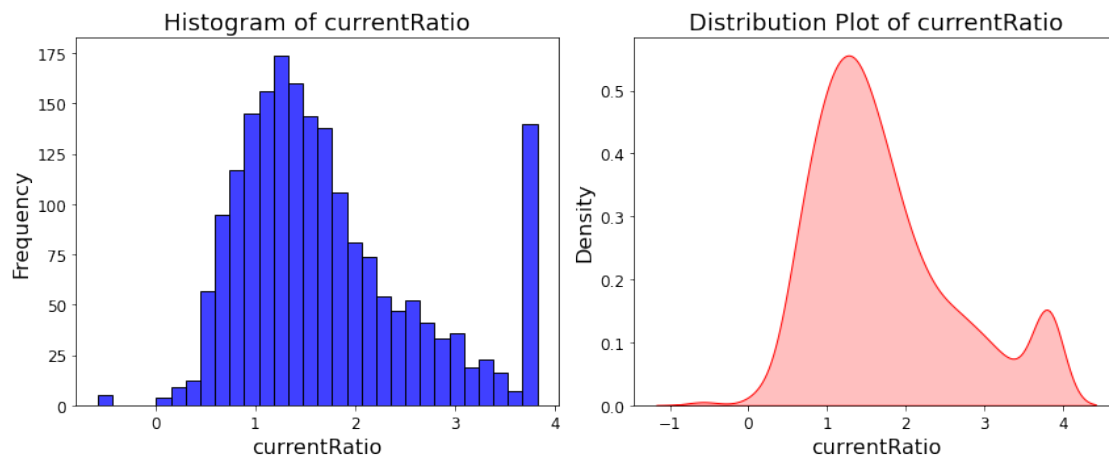
	Predictors	Skewness_Original \
0	currentRatio	33.6
1	quickRatio	30.3
2	cashRatio	26.7
3	daysOfSalesOutstanding	21.2
4	netProfitMargin	1.2
5	pretaxProfitMargin	7.3
6	grossProfitMargin	-14.3
7	operatingProfitMargin	12.3
8	returnOnAssets	-43.9
9	returnOnCapitalEmployed	-43.9
10	returnOnEquity	43.8
11	assetTurnover	31.2
12	fixedAssetTurnover	31.2
13	debtEquityRatio	28.0
14	debtRatio	1.2
15	effectiveTaxRate	31.7
16	freeCashFlowOperatingCashFlowRatio	-22.5
17	freeCashFlowPerShare	33.4
18	cashPerShare	36.7
19	companyEquityMultiplier	28.0
20	ebitPerRevenue	7.3
21	enterpriseValueMultiple	13.7
22	operatingCashFlowPerShare	32.3
23	operatingCashFlowSalesRatio	15.6
24	payablesTurnover	25.4

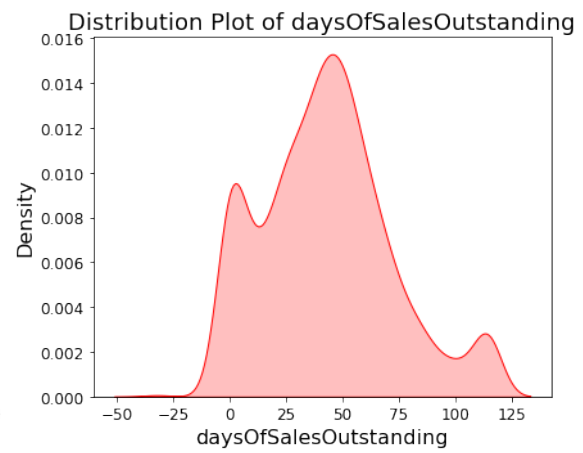
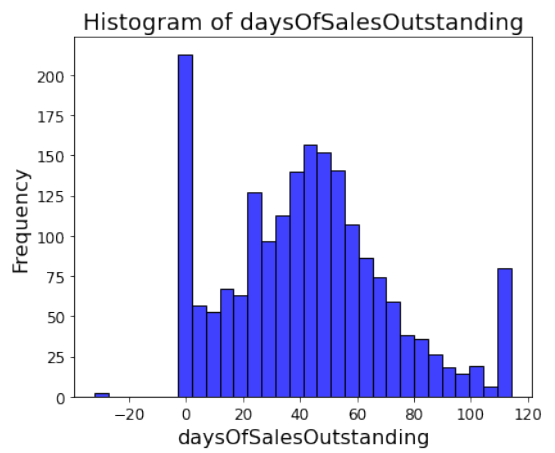
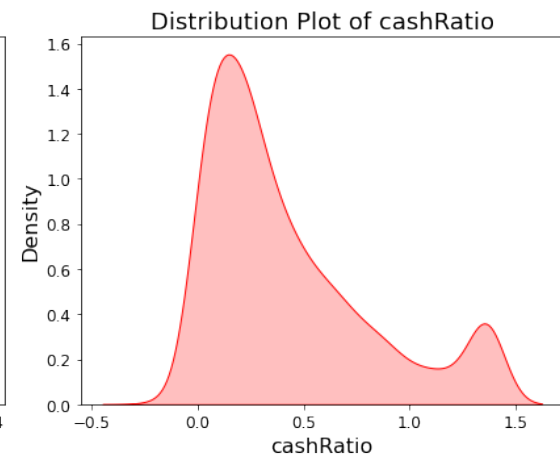
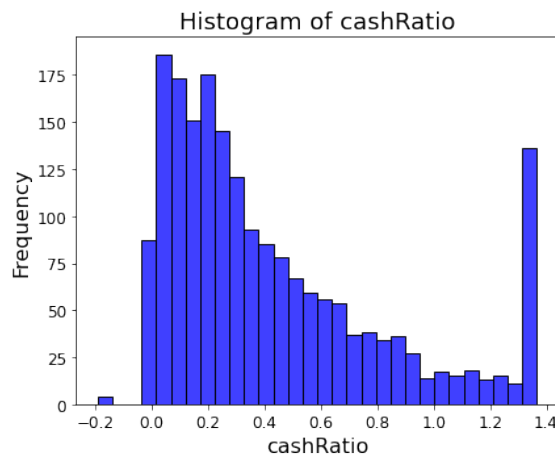
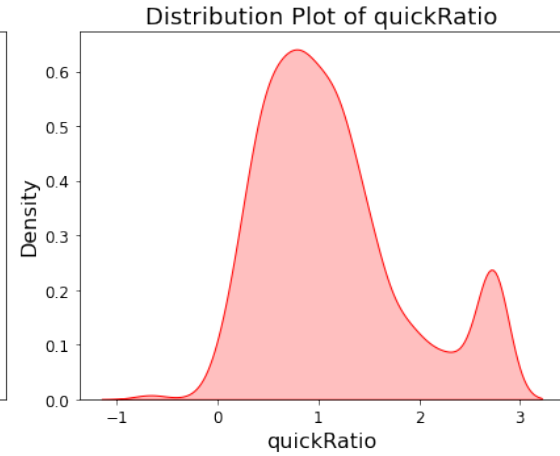
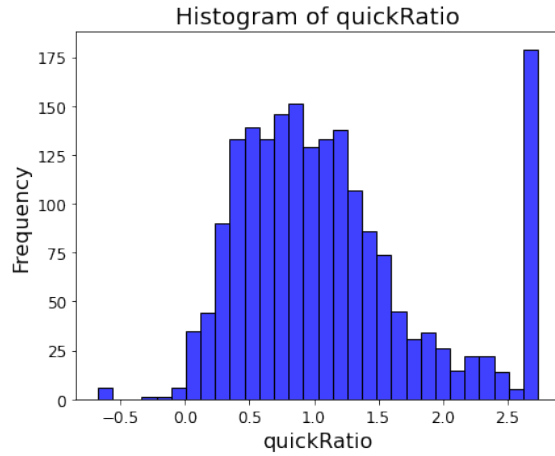
	Skewness_After_Outliers
0	0.9
1	0.8
2	1.1
3	0.5
4	0.1

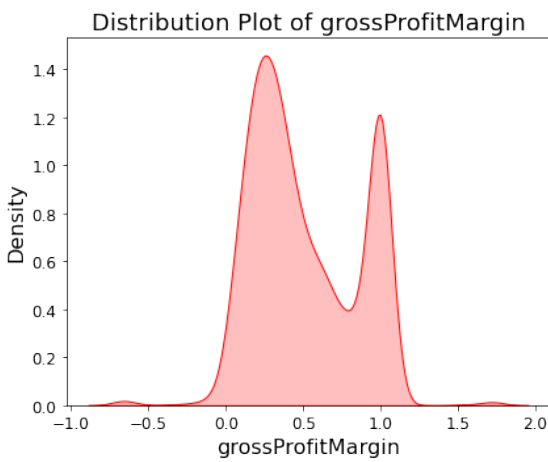
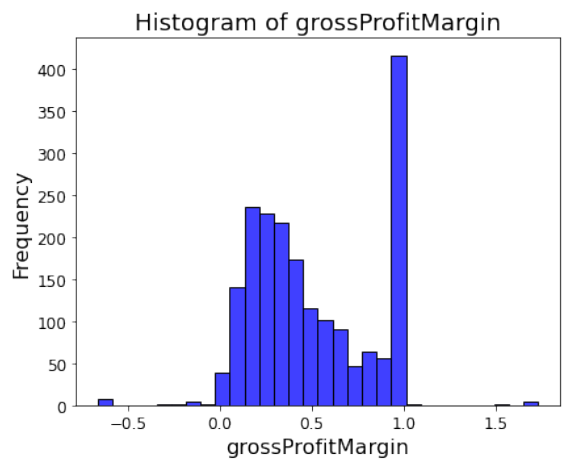
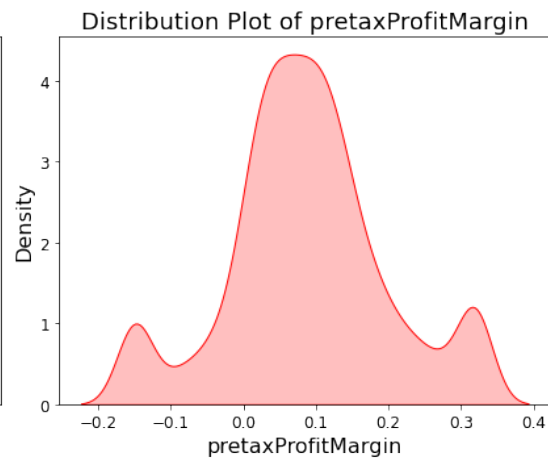
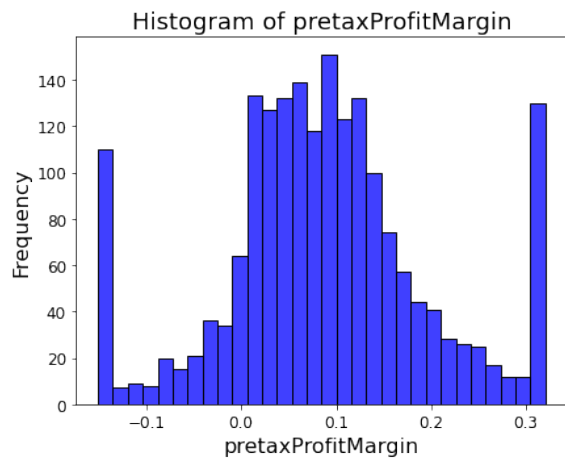
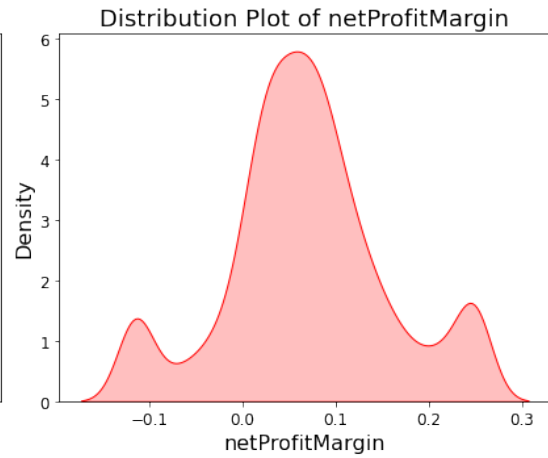
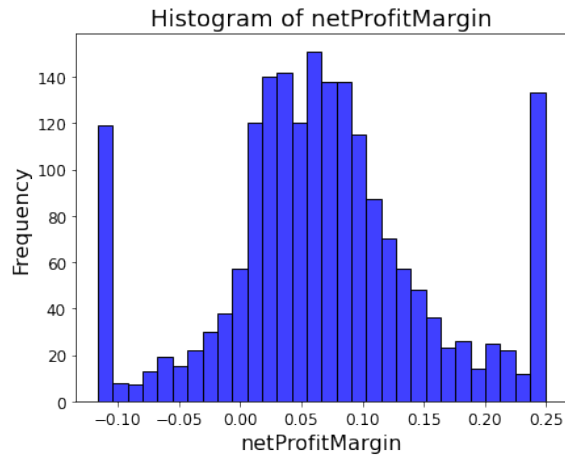
5	0.1
6	0.3
7	0.1
8	-0.0
9	0.1
10	0.0
11	1.0
12	1.2
13	0.4
14	0.3
15	-0.3
16	-0.7
17	0.3
18	1.1
19	0.4
20	0.1
21	0.2
22	0.7
23	0.8
24	0.8

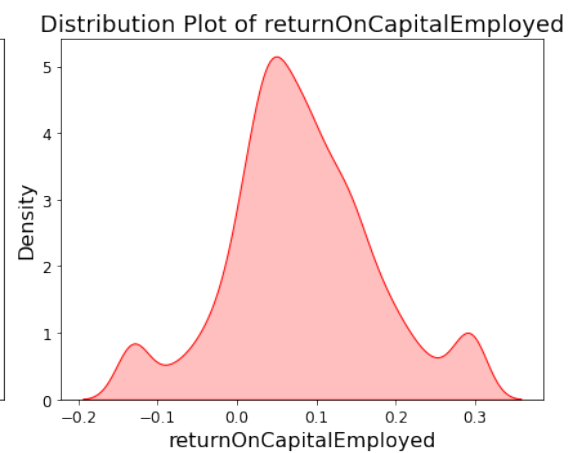
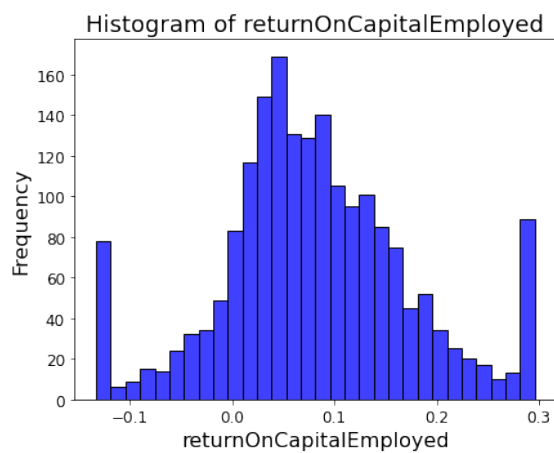
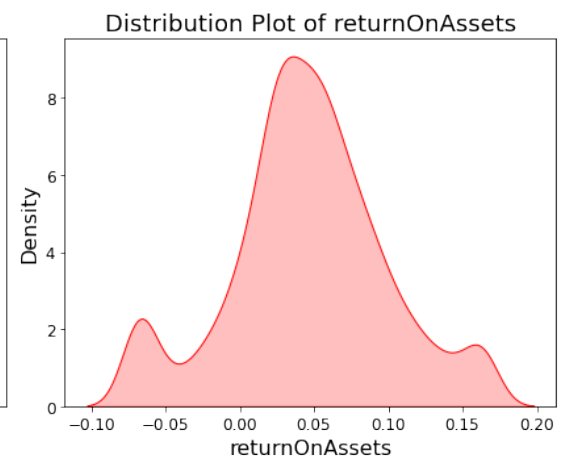
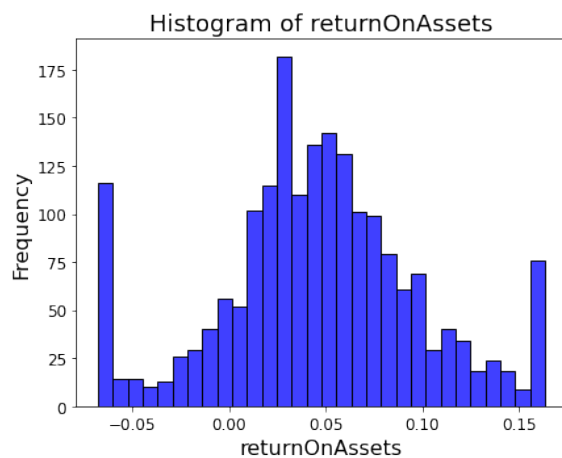
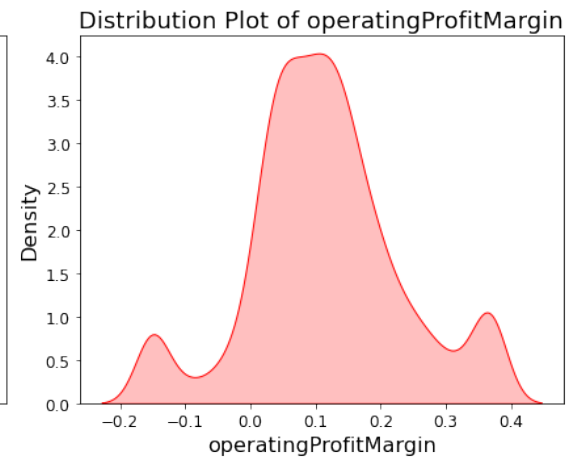
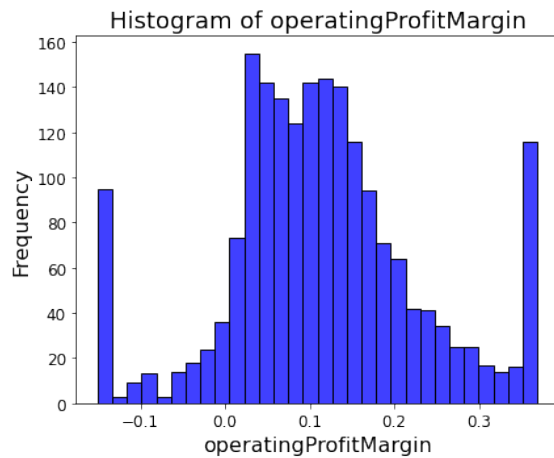
```
[294]: skewness_combined.to_csv("/Users/kamila/Desktop/tables pngs/
↳skewness_orig_outl_table.csv", index=True)
```

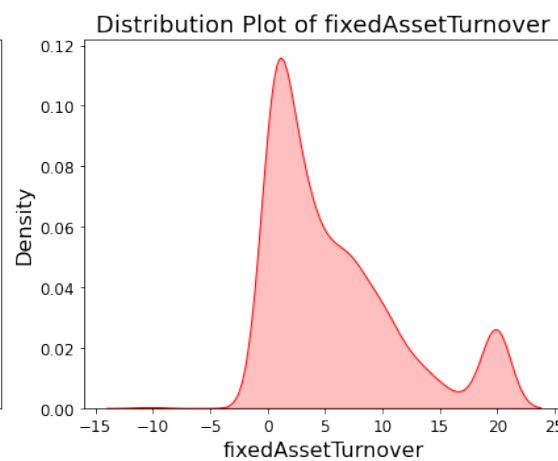
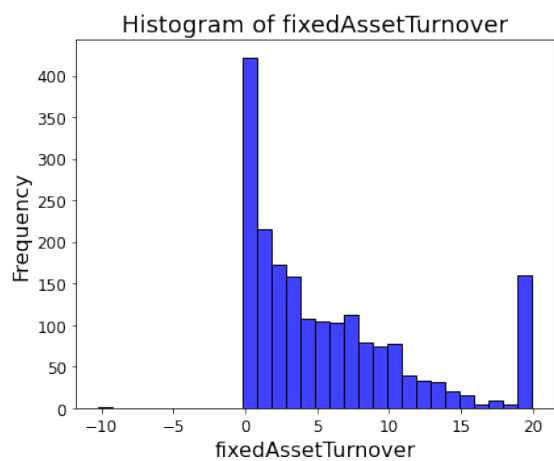
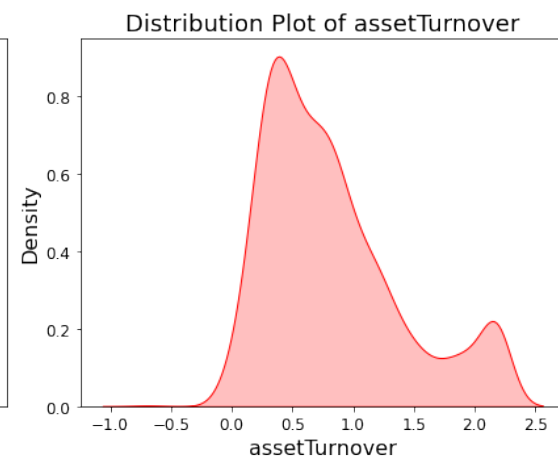
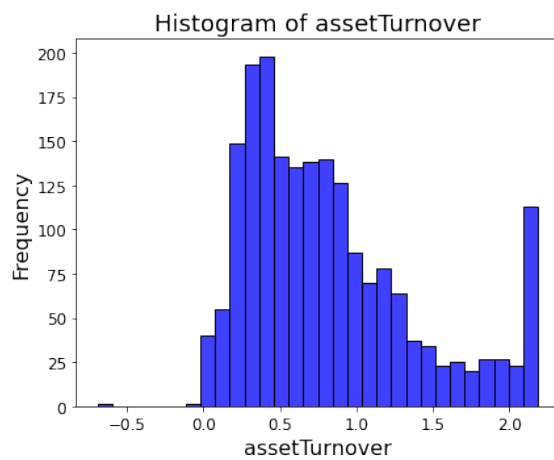
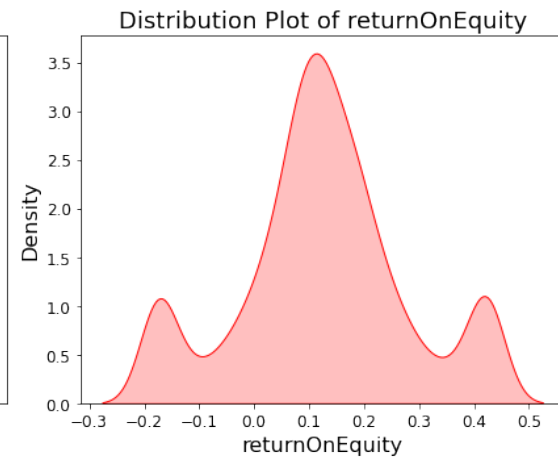
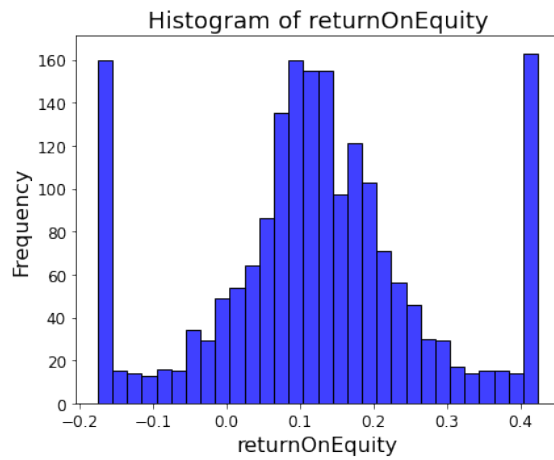
```
[295]: for pred in predictors:
        plot_histogram_distribution(credit_rating_outl, pred)
```

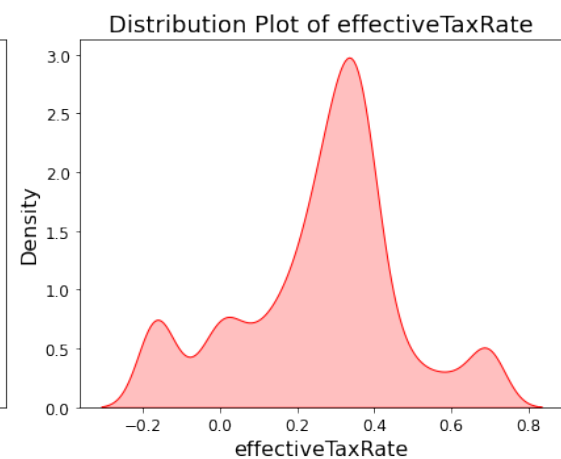
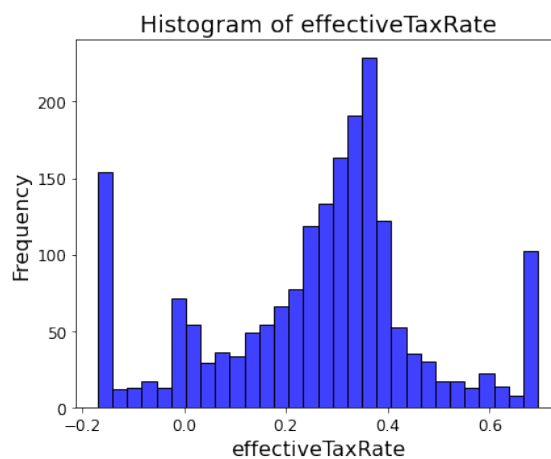
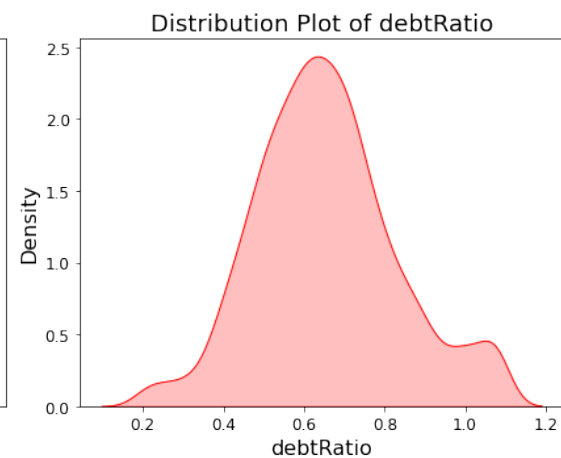
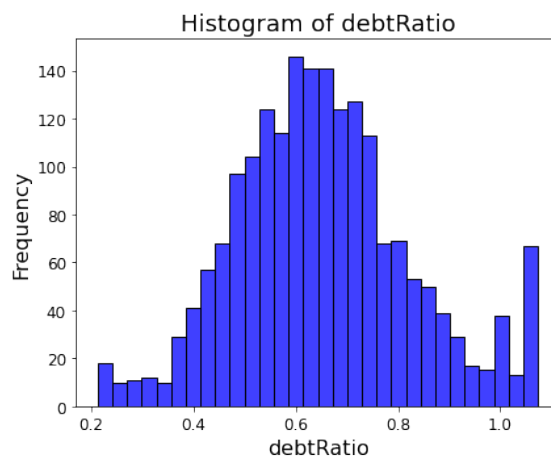
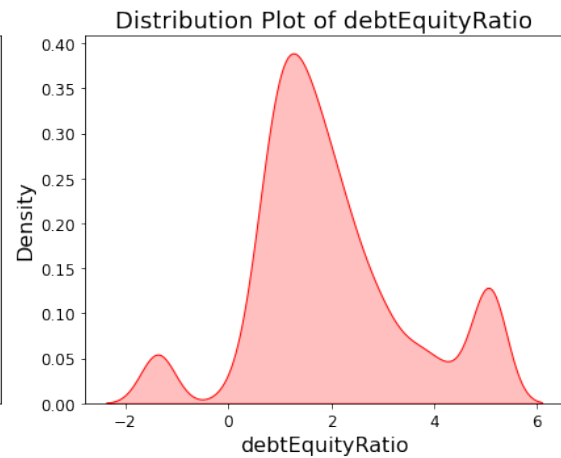
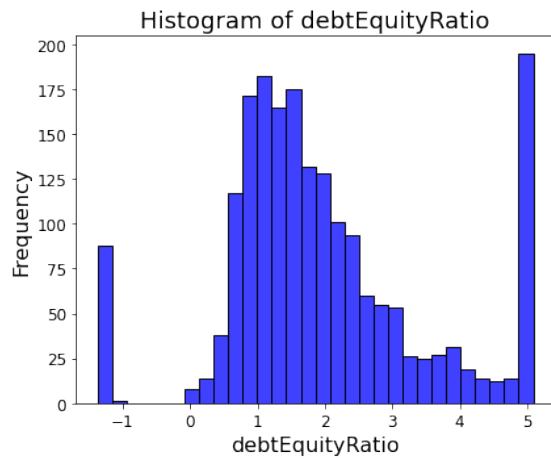




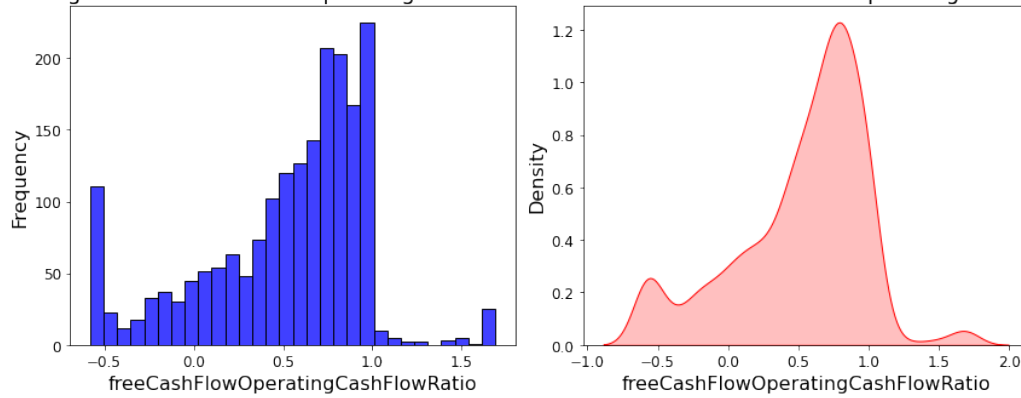




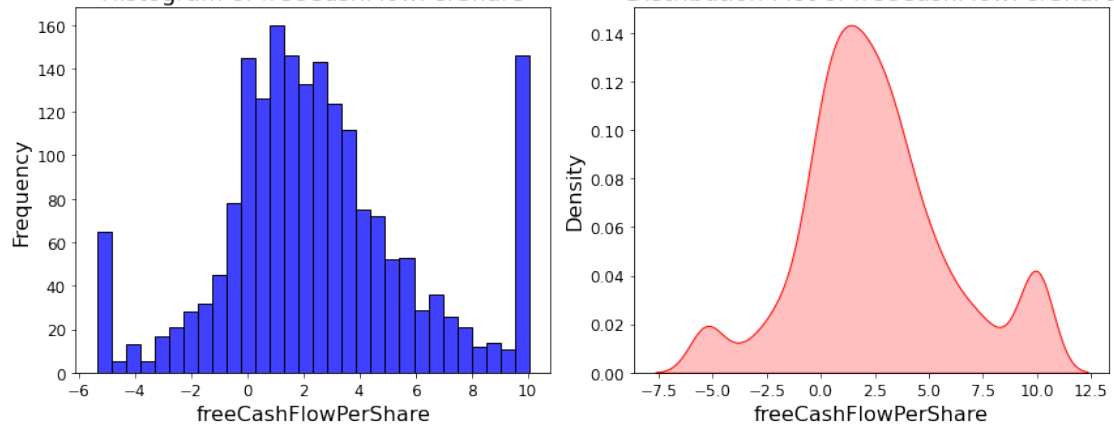




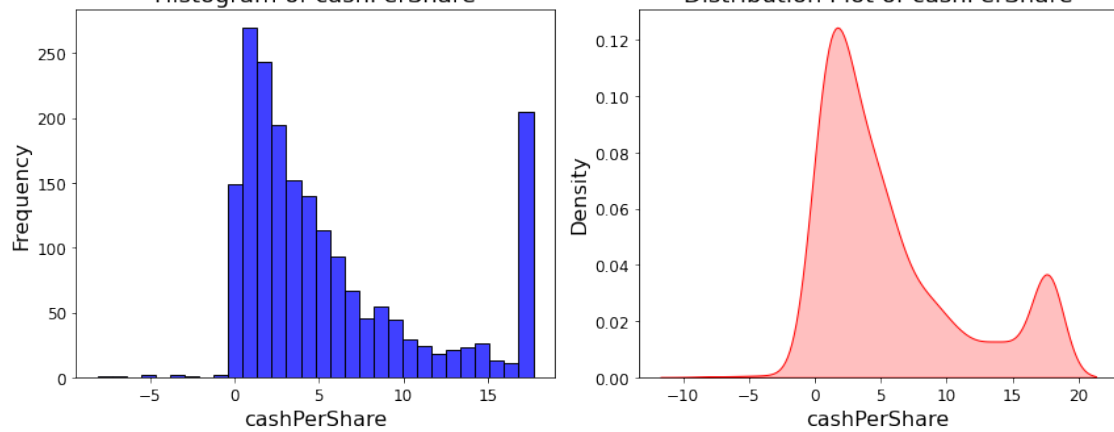
Histogram of freeCashFlowOperatingCashFlowRatio Distribution Plot of freeCashFlowOperatingCashFlowRatio

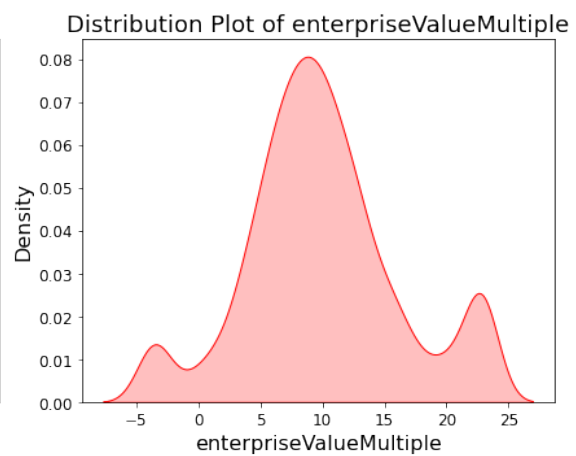
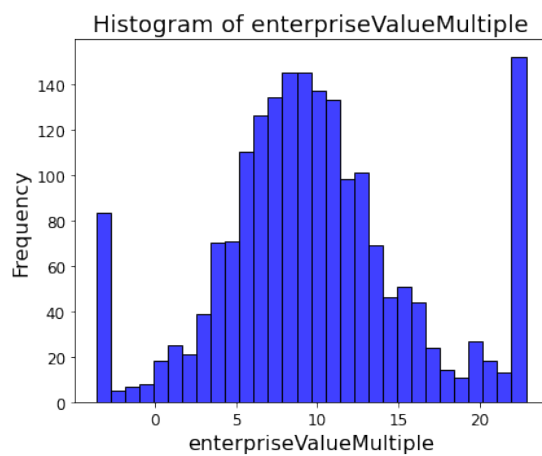
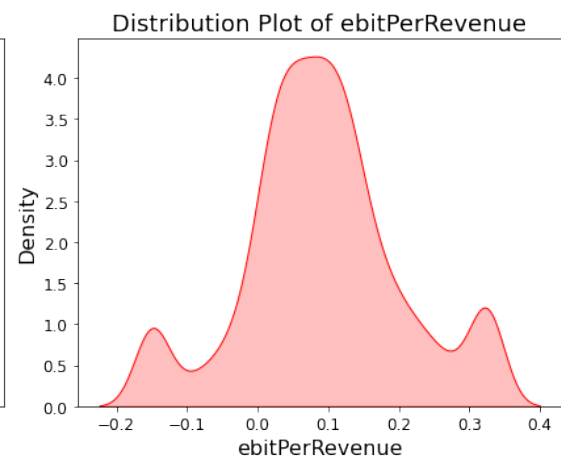
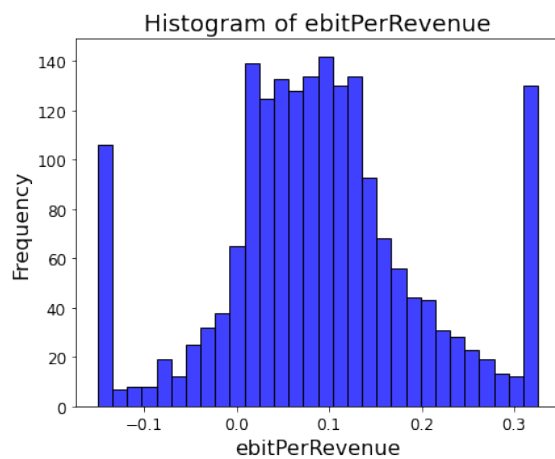
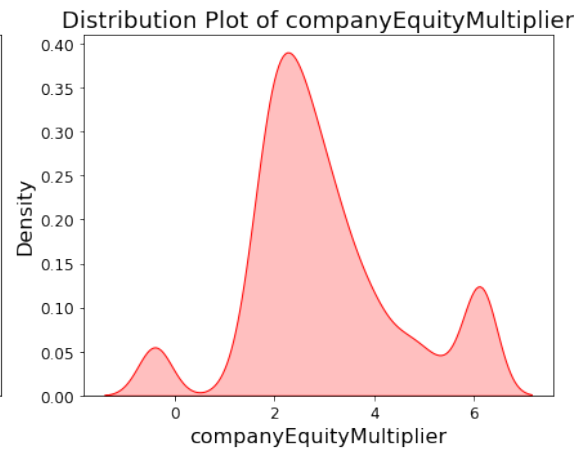
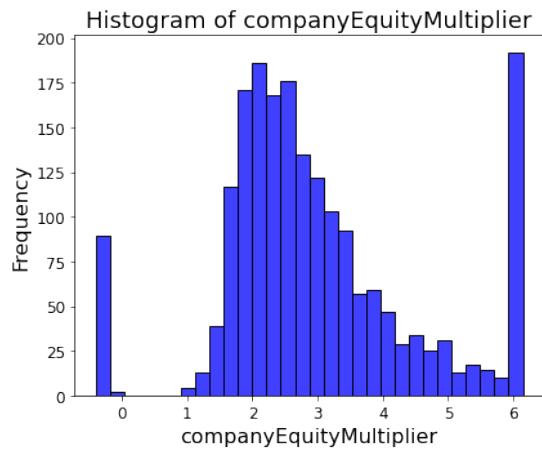


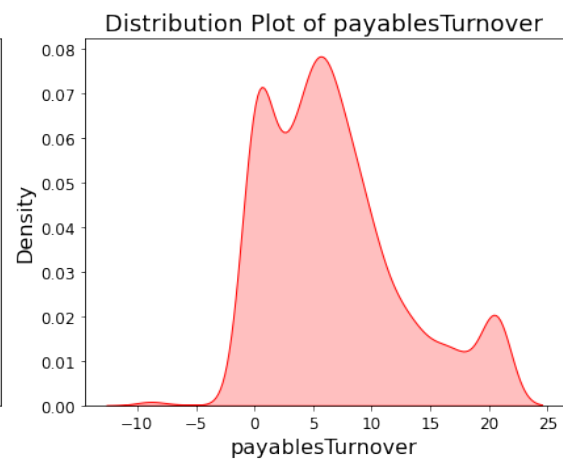
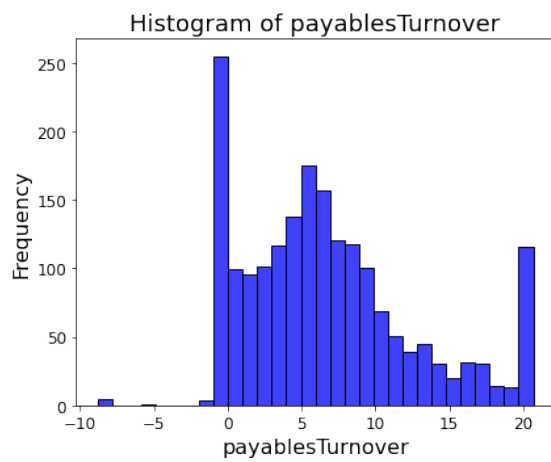
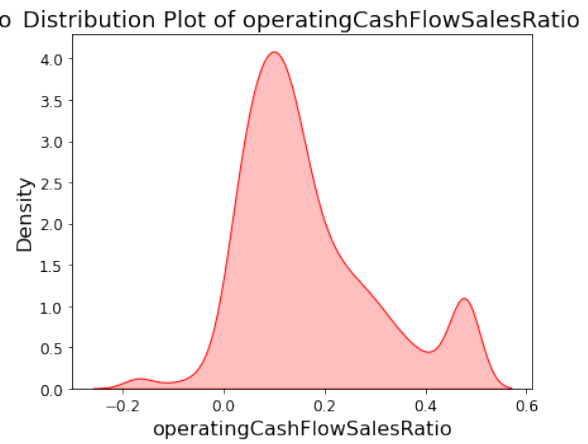
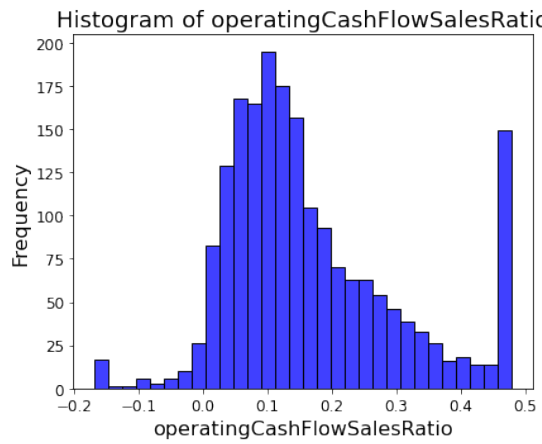
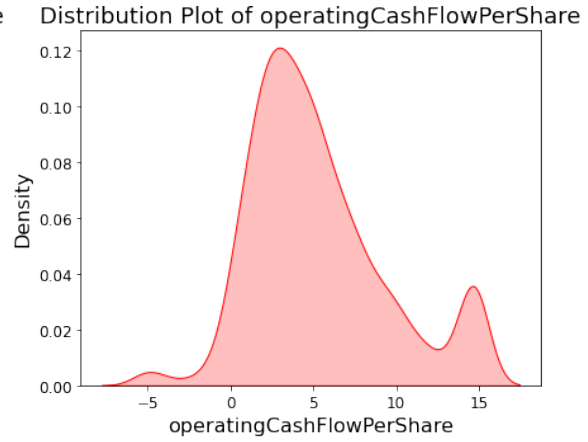
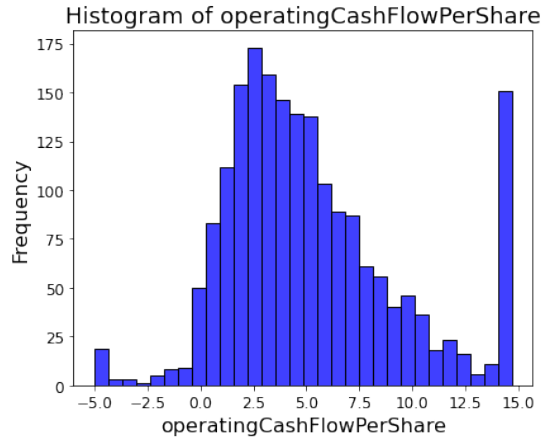
Histogram of freeCashFlowPerShare Distribution Plot of freeCashFlowPerShare



Histogram of cashPerShare Distribution Plot of cashPerShare







```
[296]: pd.set_option('display.max_columns', None)
credit_rating_outl[predictors].describe()
```

```
[296]:
```

	currentRatio	quickRatio	cashRatio	daysOfSalesOutstanding \
count	1945.000000	1945.000000	1945.000000	1945.000000
mean	1.725496	1.134234	0.427865	42.901474
std	0.904069	0.723911	0.390735	28.857953
min	-0.578253	-0.678516	-0.192736	-32.079899
25%	1.071930	0.602825	0.130476	22.751296
50%	1.497207	0.988258	0.295357	42.285644
75%	2.172051	1.457053	0.624856	59.305426
max	3.822234	2.738394	1.366426	114.136621

	netProfitMargin	pretaxProfitMargin	grossProfitMargin \
count	1945.000000	1945.000000	1945.000000
mean	0.068353	0.087874	0.506154
std	0.087175	0.110627	0.339364
min	-0.116012	-0.150908	-0.665901
25%	0.020894	0.025649	0.232565
50%	0.064219	0.084462	0.411563
75%	0.112164	0.143353	0.831541
max	0.249069	0.319909	1.730006

	operatingProfitMargin	returnOnAssets	returnOnCapitalEmployed \
count	1945.000000	1945.000000	1945.000000
mean	0.114310	0.045658	0.080740
std	0.117133	0.052777	0.093745
min	-0.150891	-0.067969	-0.132799
25%	0.044003	0.018757	0.027902
50%	0.106727	0.045163	0.074098
75%	0.173932	0.076574	0.135036
max	0.368826	0.163300	0.295738

	returnOnEquity	assetTurnover	fixedAssetTurnover	debtEquityRatio \
count	1945.000000	1945.000000	1945.000000	1945.000000
mean	0.123975	0.823209	5.800394	1.986201
std	0.153155	0.567019	5.805639	1.518669
min	-0.175470	-0.692477	-10.270060	-1.377470
25%	0.049234	0.387236	1.053337	1.048583
50%	0.121813	0.703145	3.834966	1.668163
75%	0.199037	1.107044	8.602268	2.665952
max	0.423741	2.186756	19.925665	5.092006

	debtRatio	effectiveTaxRate	freeCashFlowOperatingCashFlowRatio \
count	1945.000000	1945.000000	1945.000000
mean	0.653568	0.261562	0.519337
std	0.175802	0.210632	0.465289

min	0.212711	-0.169481	-0.579768
25%	0.535562	0.154696	0.270574
50%	0.642010	0.301064	0.644529
75%	0.750796	0.370814	0.837470
max	1.073647	0.694992	1.687812

	freeCashFlowPerShare	cashPerShare	companyEquityMultiplier \
count	1945.000000	1945.000000	1945.000000
mean	2.549573	5.633543	2.991957
std	3.488241	5.486699	1.531562
min	-5.361851	-8.127117	-0.403934
25%	0.420396	1.556122	2.052242
50%	2.144184	3.681818	2.663527
75%	4.275228	8.011616	3.689693
max	10.057475	17.694855	6.145869

	ebitPerRevenue	enterpriseValueMultiple	operatingCashFlowPerShare \
count	1945.000000	1945.000000	1945.000000
mean	0.091632	9.947585	5.251449
std	0.111624	6.201708	4.146342
min	-0.150657	-3.611042	-5.030194
25%	0.028057	6.313191	2.368970
50%	0.087042	9.358311	4.376682
75%	0.147200	12.929346	7.301747
max	0.325914	22.853579	14.700912

	operatingCashFlowSalesRatio	payablesTurnover
count	1945.000000	1945.000000
mean	0.167938	6.790366
std	0.136334	5.809955
min	-0.169770	-8.823239
25%	0.073886	2.239925
50%	0.132249	5.828326
75%	0.236323	9.615368
max	0.479980	20.678532

[297]: predictors

```
[297]: ['currentRatio',
        'quickRatio',
        'cashRatio',
        'daysOfSalesOutstanding',
        'netProfitMargin',
        'pretaxProfitMargin',
        'grossProfitMargin',
        'operatingProfitMargin',
        'returnOnAssets',
```

```

'returnOnCapitalEmployed',
'returnOnEquity',
'assetTurnover',
'fixedAssetTurnover',
'debtEquityRatio',
'debtRatio',
'effectiveTaxRate',
'freeCashFlowOperatingCashFlowRatio',
'freeCashFlowPerShare',
'cashPerShare',
'companyEquityMultiplier',
'ebitPerRevenue',
'enterpriseValueMultiple',
'operatingCashFlowPerShare',
'operatingCashFlowSalesRatio',
'payablesTurnover']

```

```

[298]: credit_rating_outl[[
'cashRatio',
'daysOfSalesOutstanding',
'netProfitMargin',
'pretaxProfitMargin',
'fixedAssetTurnover',
'cashPerShare',
'enterpriseValueMultiple',
'operatingCashFlowPerShare',
'operatingCashFlowSalesRatio',
'payablesTurnover']] .describe()

```

```

[298]:
      cashRatio  daysOfSalesOutstanding  netProfitMargin  \
count  1945.000000      1945.000000      1945.000000
mean    0.427865        42.901474        0.068353
std     0.390735        28.857953        0.087175
min     -0.192736       -32.079899       -0.116012
25%     0.130476        22.751296        0.020894
50%     0.295357        42.285644        0.064219
75%     0.624856        59.305426        0.112164
max      1.366426       114.136621        0.249069

      pretaxProfitMargin  fixedAssetTurnover  cashPerShare  \
count      1945.000000      1945.000000      1945.000000
mean         0.087874         5.800394         5.633543
std          0.110627         5.805639         5.486699
min         -0.150908        -10.270060        -8.127117
25%          0.025649         1.053337         1.556122
50%          0.084462         3.834966         3.681818
75%          0.143353         8.602268         8.011616

```


max	0.319909	19.925665	17.694855
-----	----------	-----------	-----------

	enterpriseValueMultiple	operatingCashFlowPerShare \
count	1945.000000	1945.000000
mean	9.947585	5.251449
std	6.201708	4.146342
min	-3.611042	-5.030194
25%	6.313191	2.368970
50%	9.358311	4.376682
75%	12.929346	7.301747
max	22.853579	14.700912

	operatingCashFlowSalesRatio	payablesTurnover
count	1945.000000	1945.000000
mean	0.167938	6.790366
std	0.136334	5.809955
min	-0.169770	-8.823239
25%	0.073886	2.239925
50%	0.132249	5.828326
75%	0.236323	9.615368
max	0.479980	20.678532

```
[299]: credit_rating_outl[[
    'cashRatio', 'daysOfSalesOutstanding', 'netProfitMargin',
    ↪ 'pretaxProfitMargin',
    'fixedAssetTurnover', 'cashPerShare', 'enterpriseValueMultiple',
    'operatingCashFlowPerShare', 'operatingCashFlowSalesRatio',
    ↪ 'payablesTurnover'
]].describe().round(2)
```

[299]:	cashRatio	daysOfSalesOutstanding	netProfitMargin	pretaxProfitMargin \
count	1945.00	1945.00	1945.00	1945.00
mean	0.43	42.90	0.07	0.09
std	0.39	28.86	0.09	0.11
min	-0.19	-32.08	-0.12	-0.15
25%	0.13	22.75	0.02	0.03
50%	0.30	42.29	0.06	0.08
75%	0.62	59.31	0.11	0.14
max	1.37	114.14	0.25	0.32

	fixedAssetTurnover	cashPerShare	enterpriseValueMultiple \
count	1945.00	1945.00	1945.00
mean	5.80	5.63	9.95
std	5.81	5.49	6.20
min	-10.27	-8.13	-3.61
25%	1.05	1.56	6.31
50%	3.83	3.68	9.36

75%	8.60	8.01	12.93
max	19.93	17.69	22.85

	operatingCashFlowPerShare	operatingCashFlowSalesRatio \
count	1945.00	1945.00
mean	5.25	0.17
std	4.15	0.14
min	-5.03	-0.17
25%	2.37	0.07
50%	4.38	0.13
75%	7.30	0.24
max	14.70	0.48

	payablesTurnover
count	1945.00
mean	6.79
std	5.81
min	-8.82
25%	2.24
50%	5.83
75%	9.62
max	20.68

```
[300]: credit_rating_outl.nunique()
```

```
[300]: Rating      6
      Name      590
      Symbol     590
      Rating Agency Name    5
      Date      843
      Sector     12
      currentRatio    1821
      quickRatio     1784
      cashRatio      1808
      daysOfSalesOutstanding  1682
      netProfitMargin  1714
      pretaxProfitMargin  1717
      grossProfitMargin  1556
      operatingProfitMargin  1760
      returnOnAssets    1783
      returnOnCapitalEmployed  1804
      returnOnEquity    1645
      assetTurnover     1843
      fixedAssetTurnover  1803
      debtEquityRatio    1676
      debtRatio        1852
      effectiveTaxRate   1680
```

freeCashFlowOperatingCashFlowRatio	1665
freeCashFlowPerShare	1760
cashPerShare	1750
companyEquityMultiplier	1681
ebitPerRevenue	1726
enterpriseValueMultiple	1727
operatingCashFlowPerShare	1788
operatingCashFlowSalesRatio	1789
payablesTurnover	1590
year-month	60
Rating_encoded	6
Year	5
Risk_level	4
Risk_level_encoded	4
dtype:	int64

```
[301]: #Standardization
from sklearn.preprocessing import StandardScaler, LabelEncoder

X_num = ['currentRatio', 'quickRatio', 'cashRatio', 'daysOfSalesOutstanding',
         'netProfitMargin', 'pretaxProfitMargin', 'grossProfitMargin',
         'operatingProfitMargin', 'returnOnAssets', 'returnOnCapitalEmployed',
         'returnOnEquity', 'assetTurnover', 'fixedAssetTurnover',
         'debtEquityRatio', 'debtRatio', 'effectiveTaxRate',
         'freeCashFlowOperatingCashFlowRatio', 'freeCashFlowPerShare',
         'cashPerShare', 'companyEquityMultiplier', 'ebitPerRevenue',
         'enterpriseValueMultiple', 'operatingCashFlowPerShare',
         'operatingCashFlowSalesRatio', 'payablesTurnover']

# Standardize predictors (mean=0, std=1)
scaler = StandardScaler()
credit_rating_outl_scaled = credit_rating_outl.copy()
credit_rating_outl_scaled[X_num] = scaler.
    ↪fit_transform(credit_rating_outl[X_num])
```

```
[302]: credit_rating_outl[X_num].skew(axis=0)
```

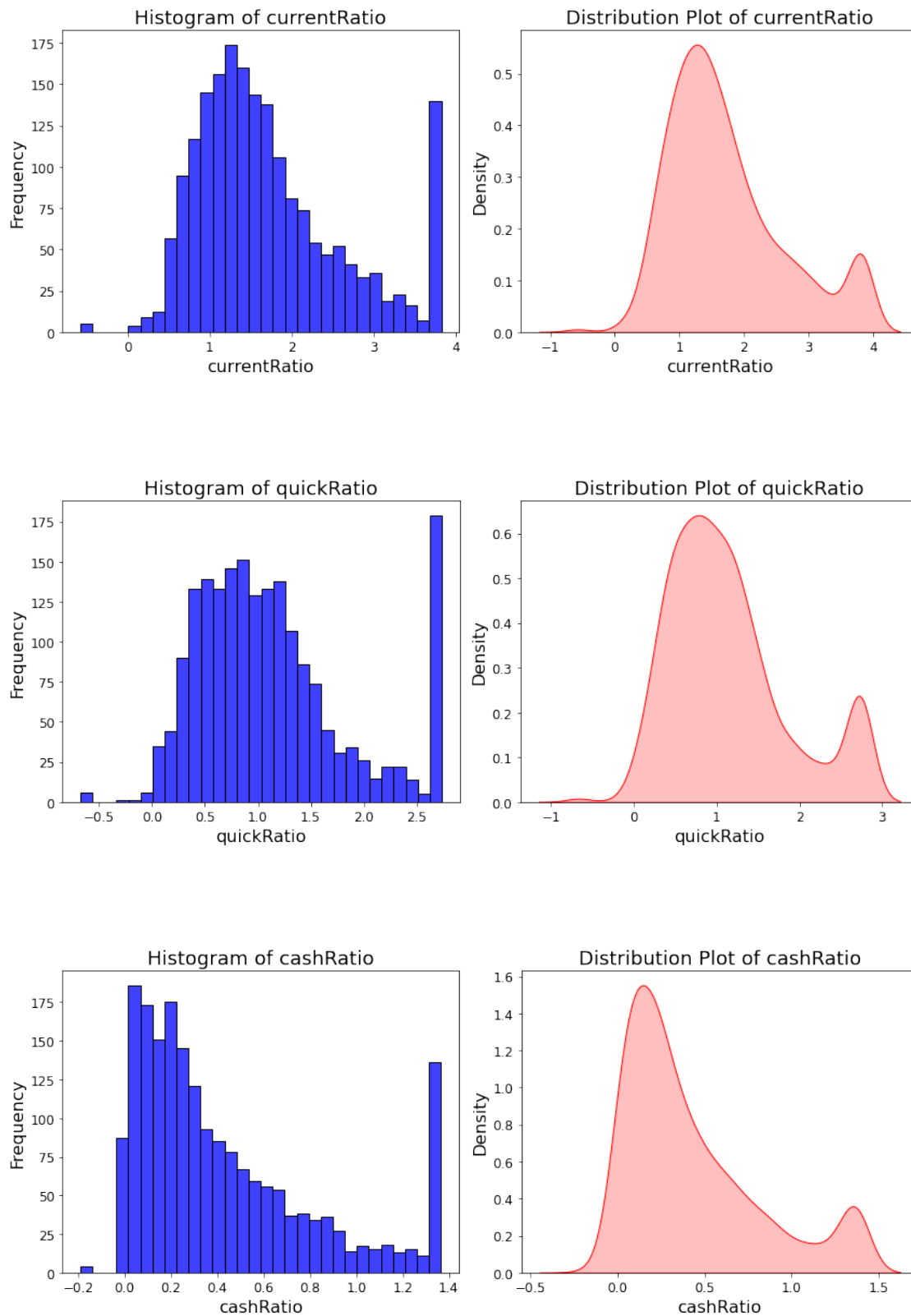
currentRatio	0.852558
quickRatio	0.828302
cashRatio	1.114470
daysOfSalesOutstanding	0.478921
netProfitMargin	0.081697
pretaxProfitMargin	0.075949
grossProfitMargin	0.317761
operatingProfitMargin	0.120608
returnOnAssets	-0.049913

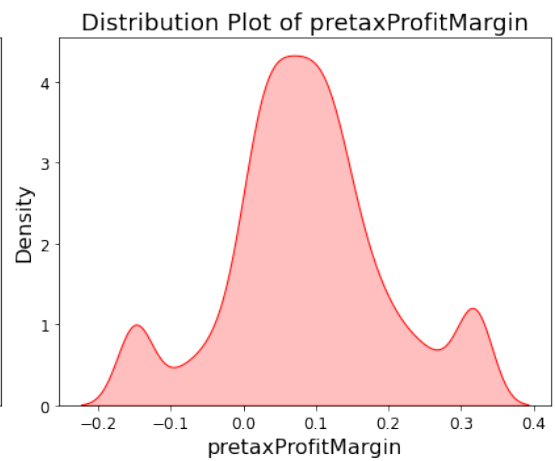
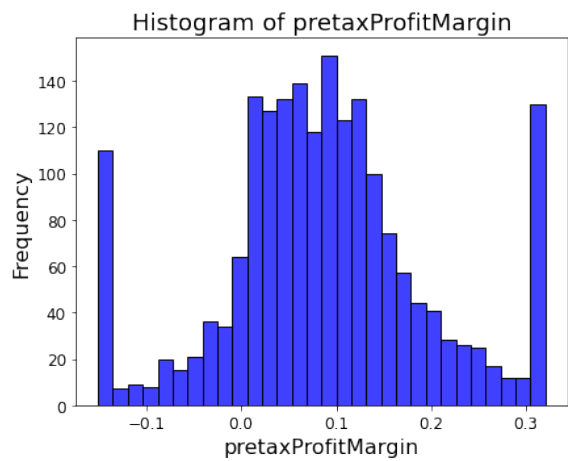
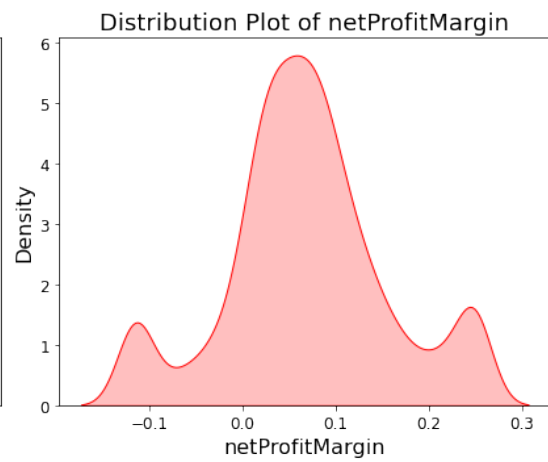
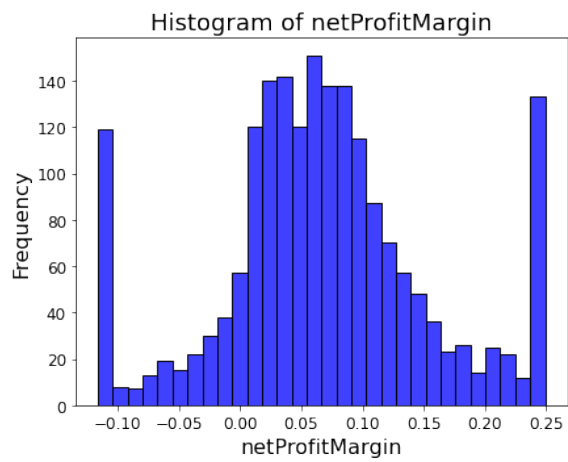
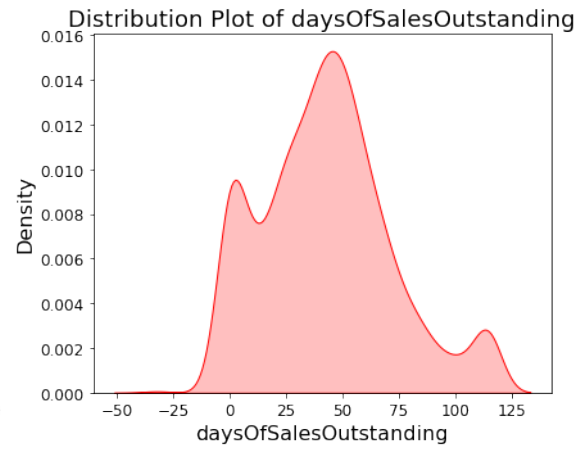
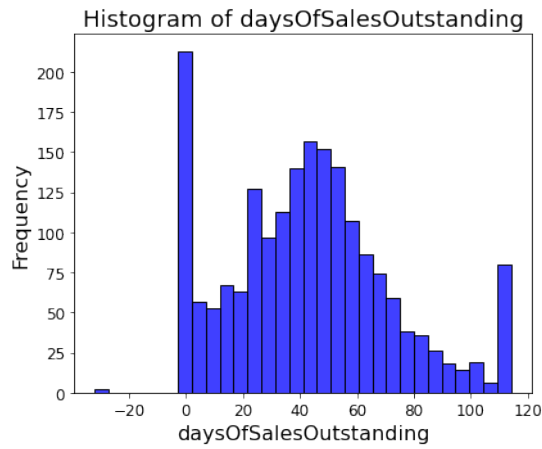
returnOnCapitalEmployed	0.139242
returnOnEquity	0.015720
assetTurnover	0.955855
fixedAssetTurnover	1.151884
debtEquityRatio	0.433598
debtRatio	0.313160
effectiveTaxRate	-0.274405
freeCashFlowOperatingCashFlowRatio	-0.733025
freeCashFlowPerShare	0.315184
cashPerShare	1.117583
companyEquityMultiplier	0.430488
ebitPerRevenue	0.093537
enterpriseValueMultiple	0.243007
operatingCashFlowPerShare	0.726208
operatingCashFlowSalesRatio	0.816839
payablesTurnover	0.818221
dtype: float64	

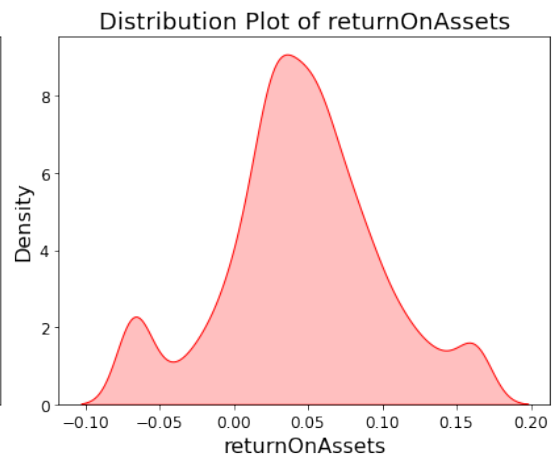
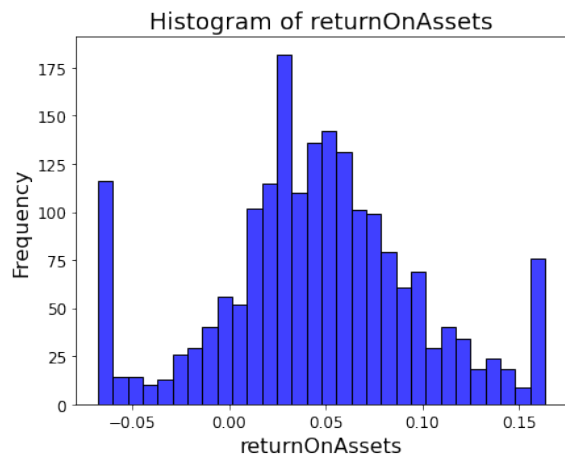
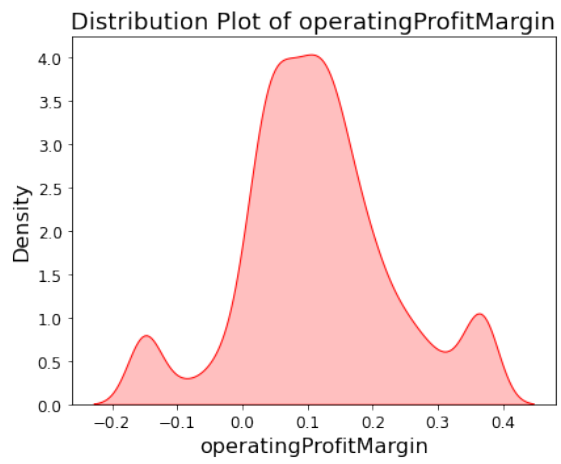
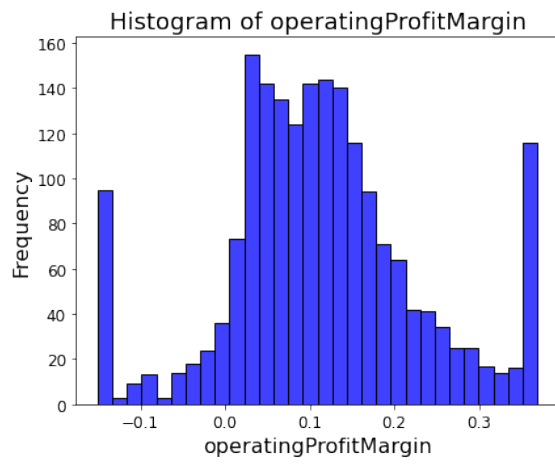
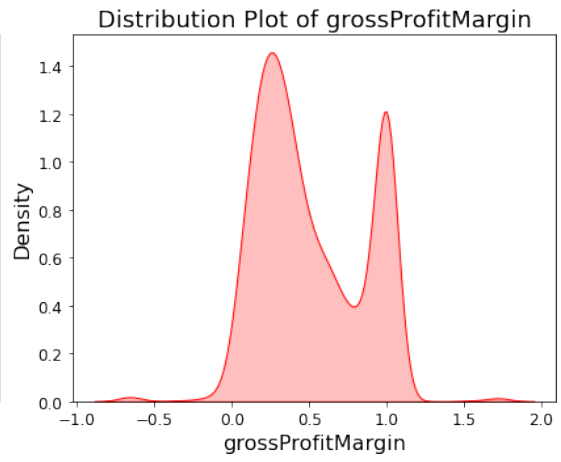
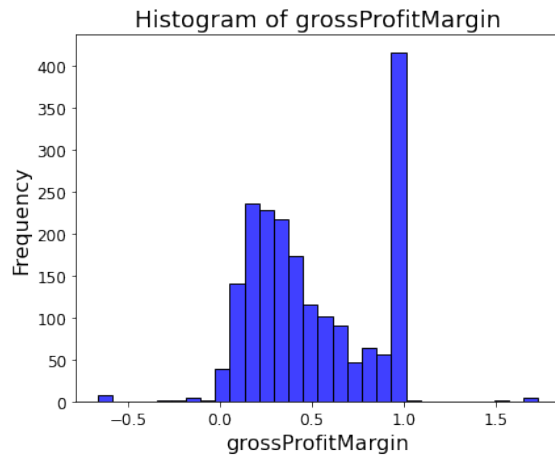
```
[303]: credit_rating_outl_scaled[X_num].skew(axis=0)
```

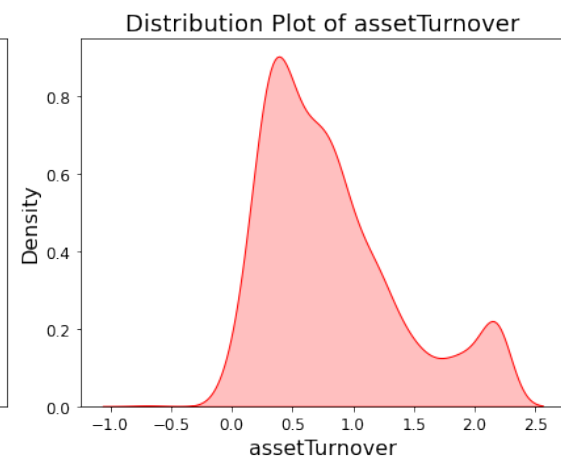
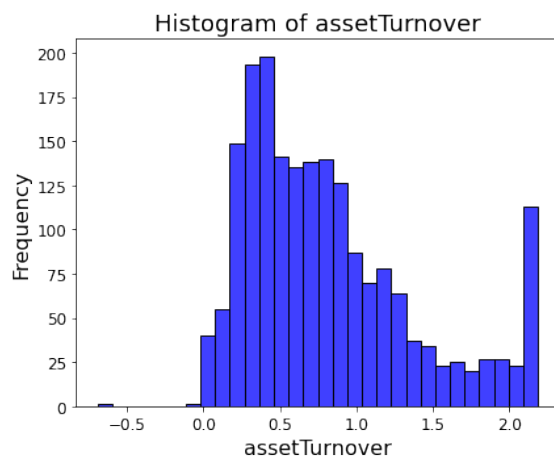
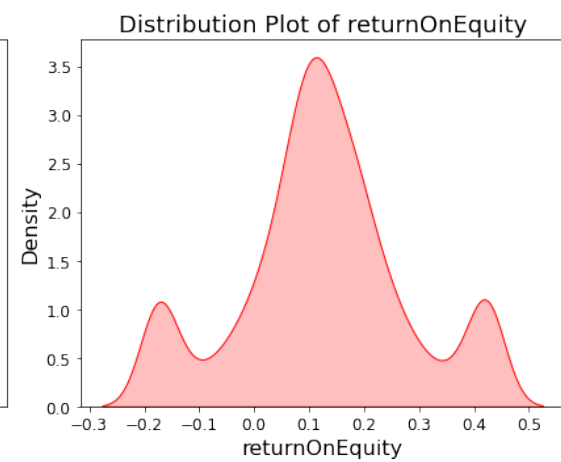
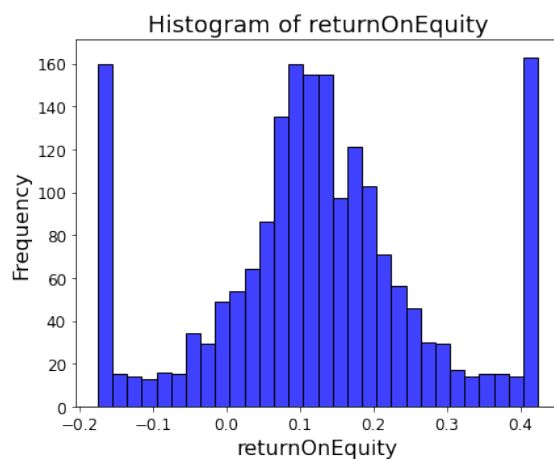
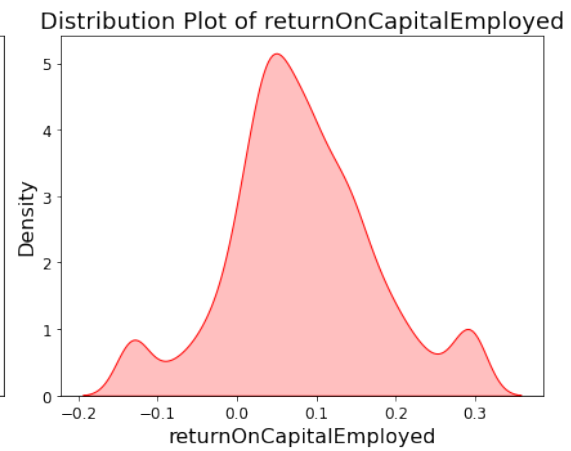
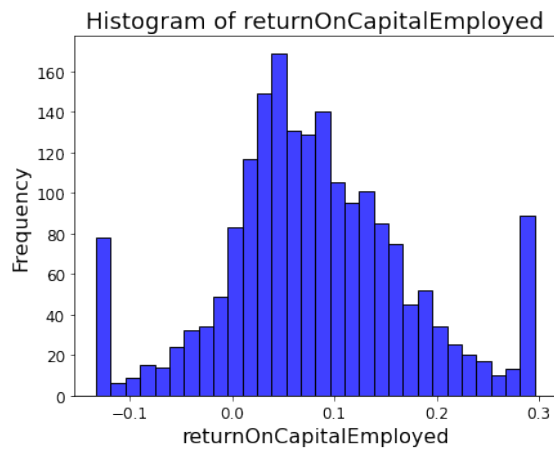
currentRatio	0.852558
quickRatio	0.828302
cashRatio	1.114470
daysOfSalesOutstanding	0.478921
netProfitMargin	0.081697
pretaxProfitMargin	0.075949
grossProfitMargin	0.317761
operatingProfitMargin	0.120608
returnOnAssets	-0.049913
returnOnCapitalEmployed	0.139242
returnOnEquity	0.015720
assetTurnover	0.955855
fixedAssetTurnover	1.151884
debtEquityRatio	0.433598
debtRatio	0.313160
effectiveTaxRate	-0.274405
freeCashFlowOperatingCashFlowRatio	-0.733025
freeCashFlowPerShare	0.315184
cashPerShare	1.117583
companyEquityMultiplier	0.430488
ebitPerRevenue	0.093537
enterpriseValueMultiple	0.243007
operatingCashFlowPerShare	0.726208
operatingCashFlowSalesRatio	0.816839
payablesTurnover	0.818221
dtype: float64	

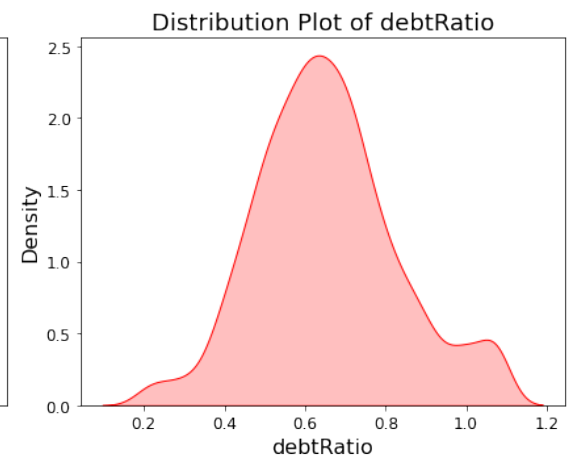
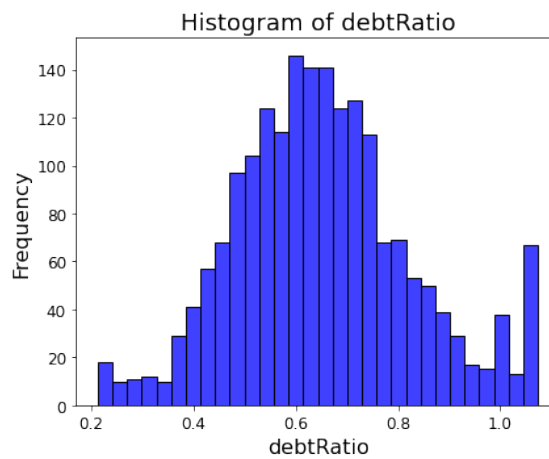
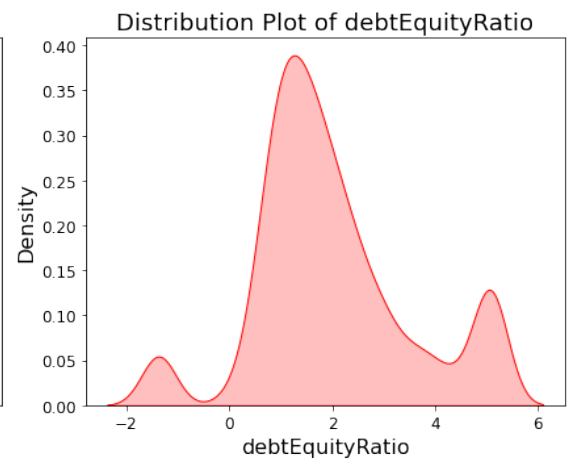
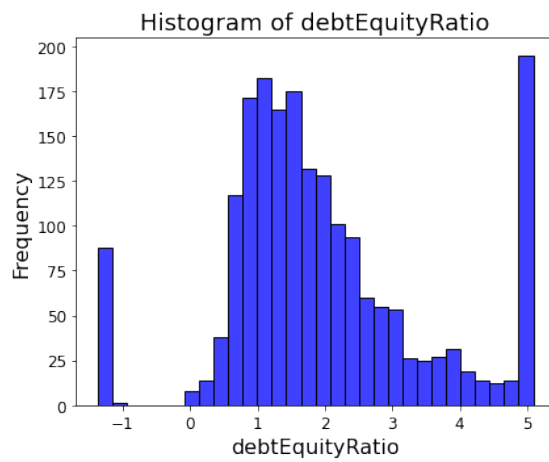
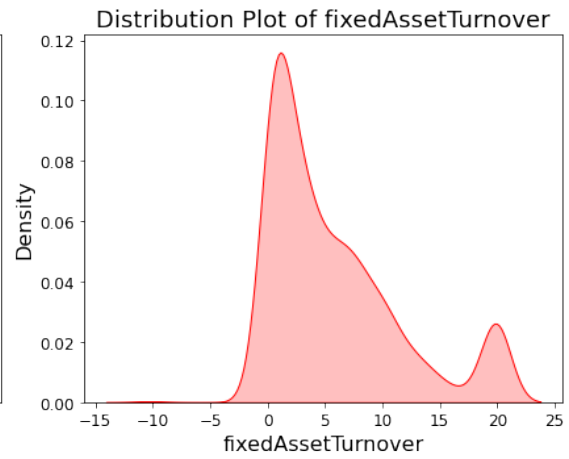
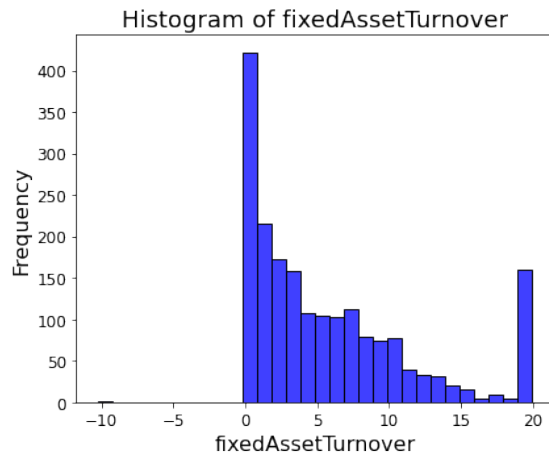
```
[304]: for pred in predictors:
        plot_histogram_distribution(credit_rating_out1, pred)
```

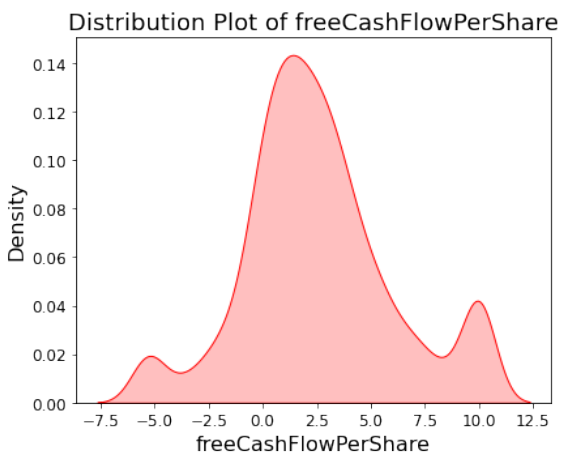
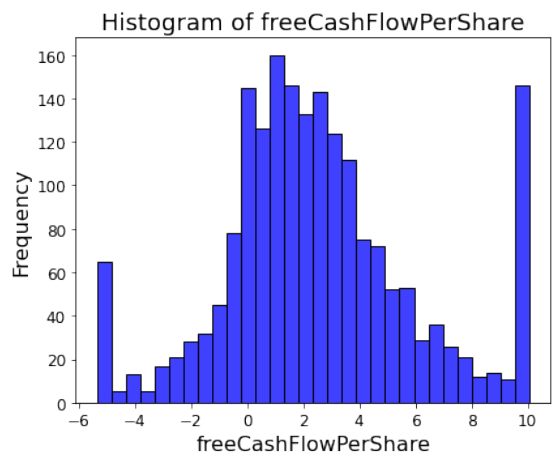
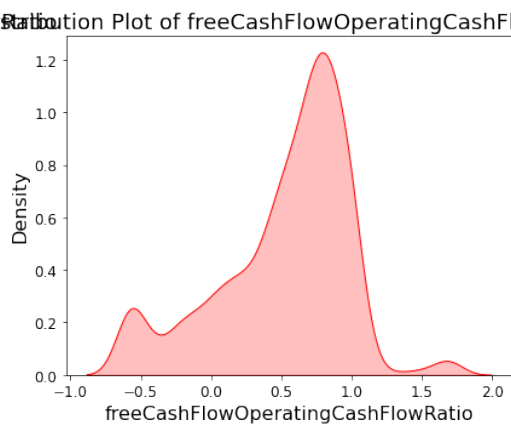
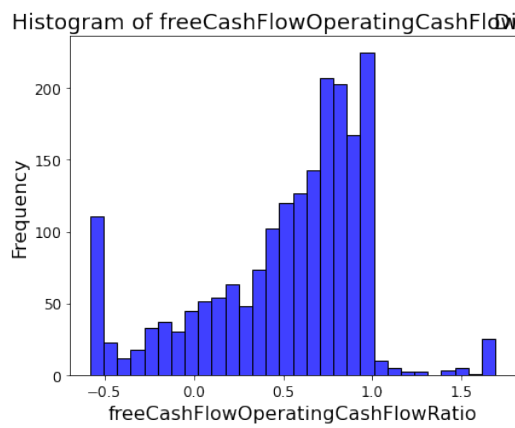
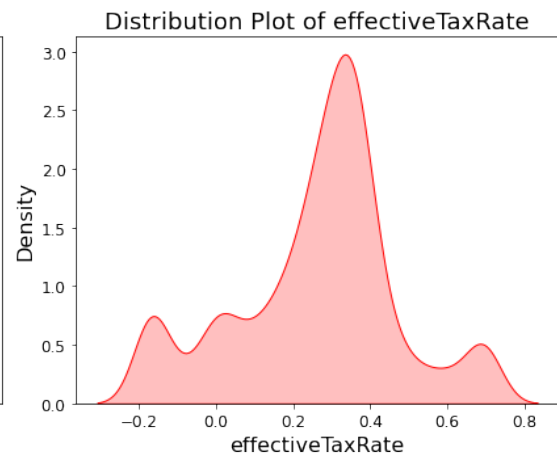
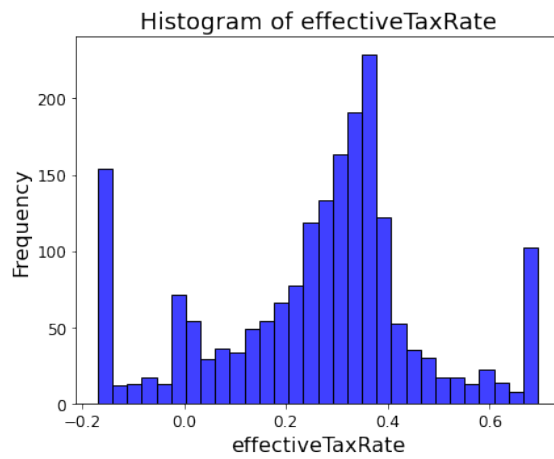


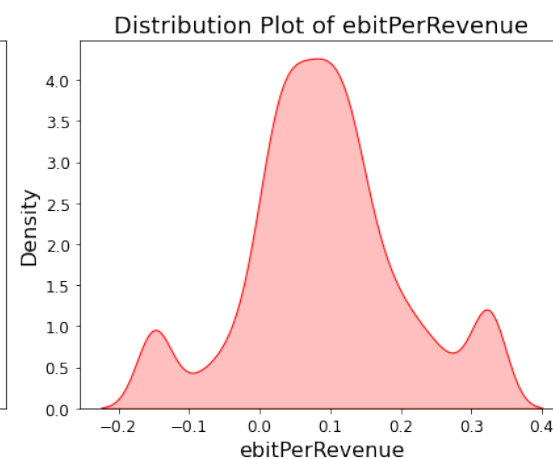
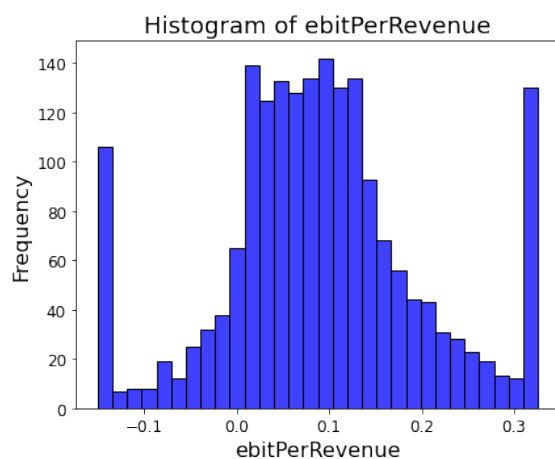
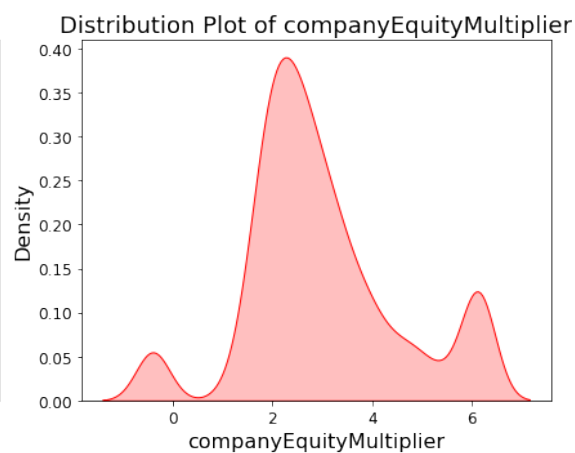
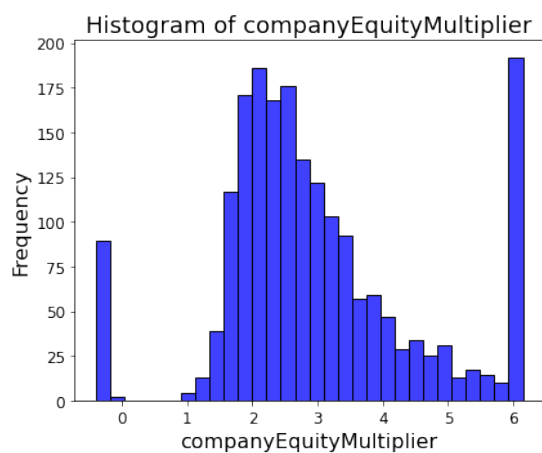
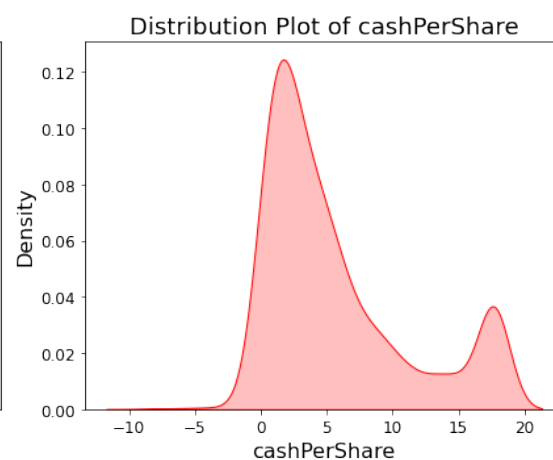
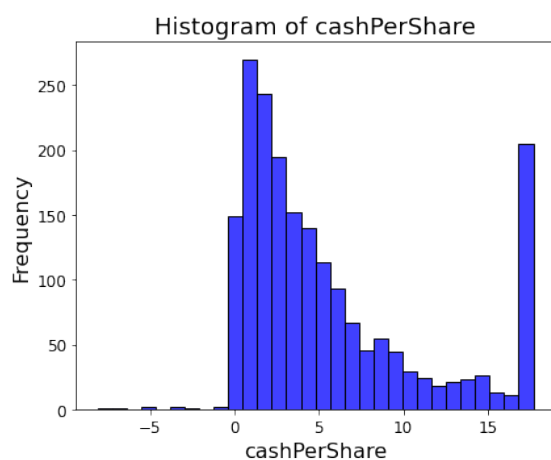


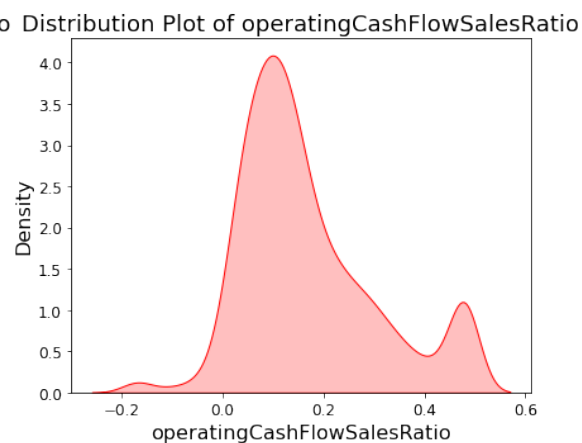
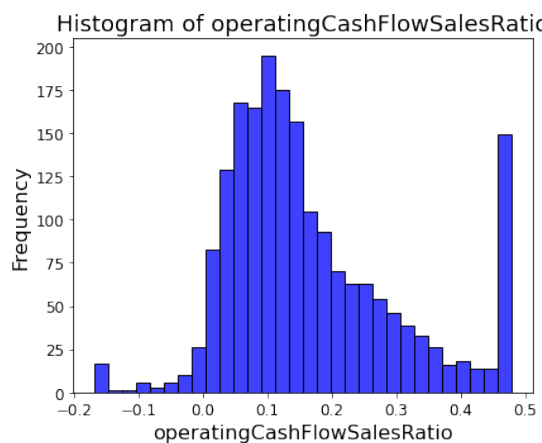
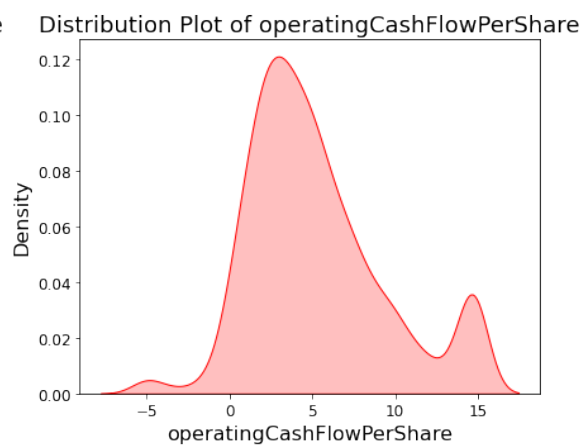
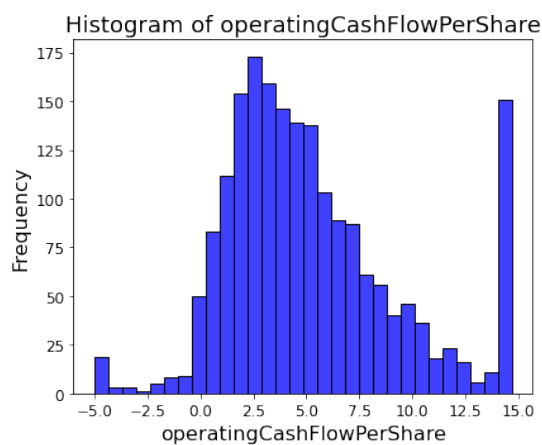
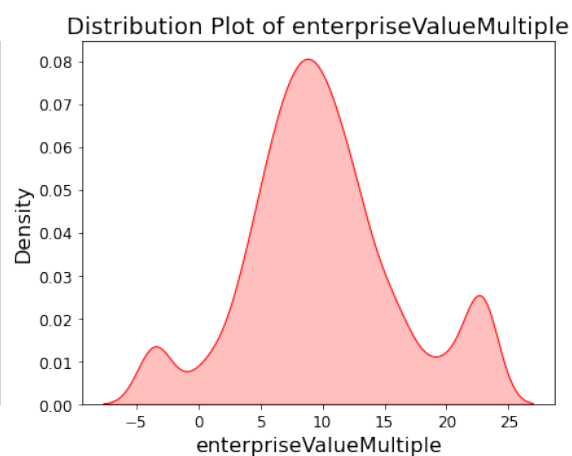
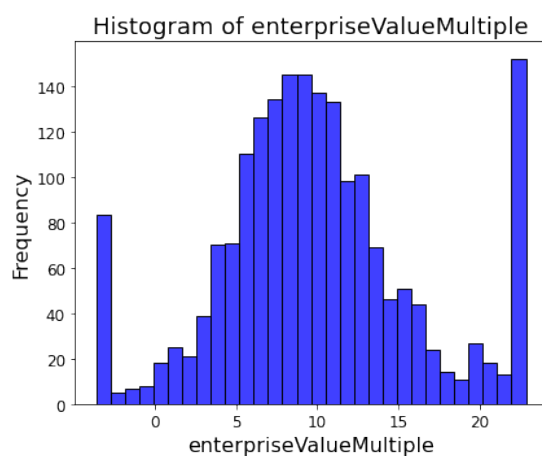


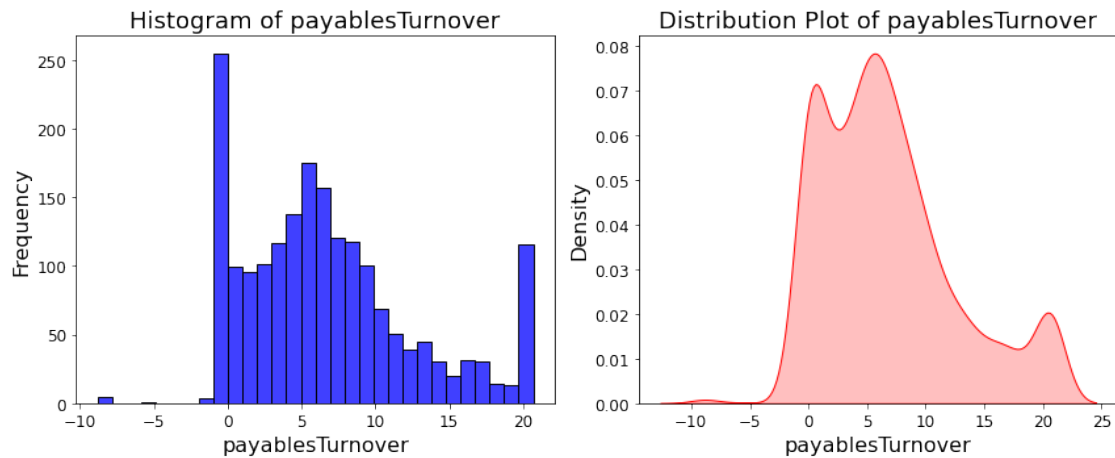




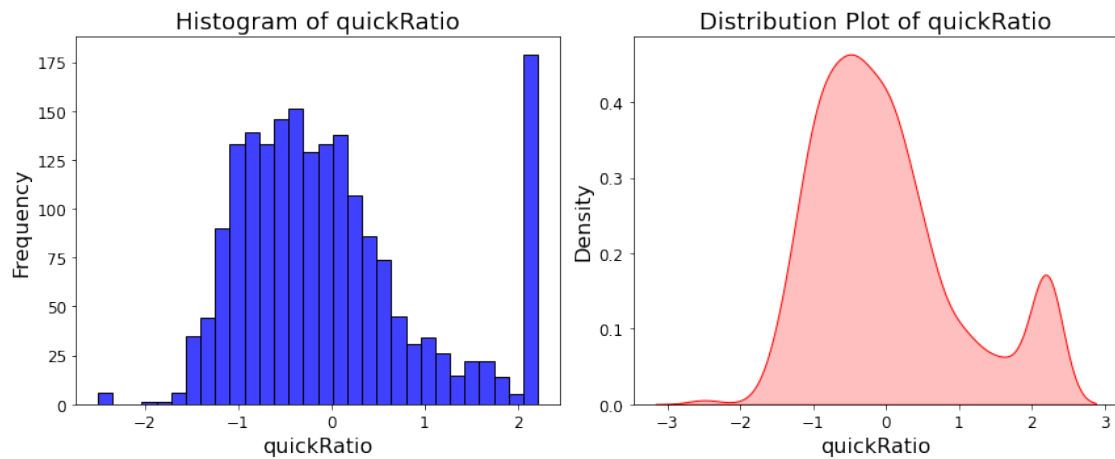
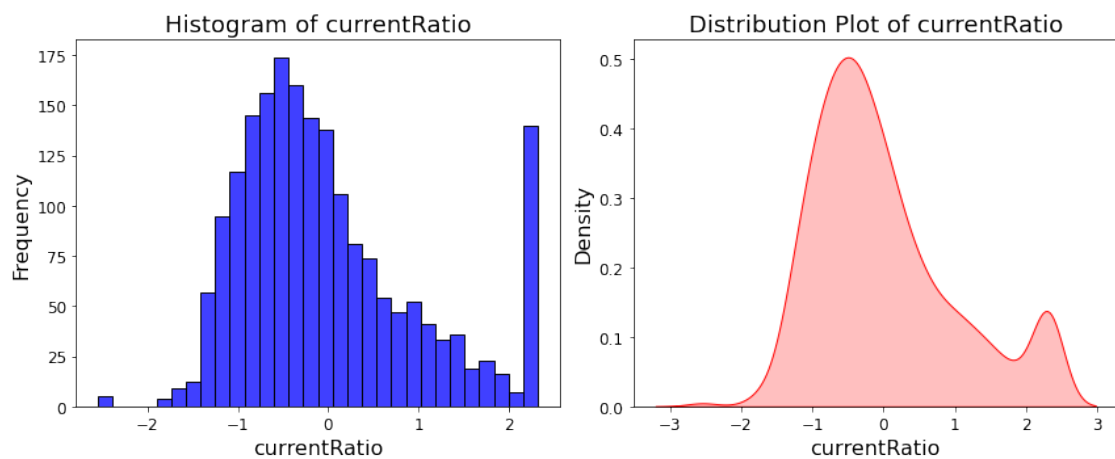


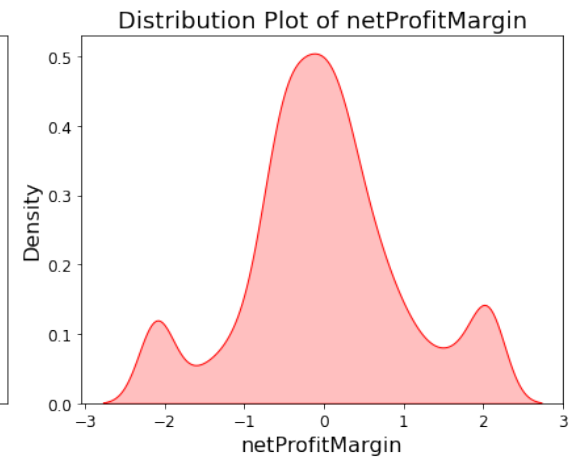
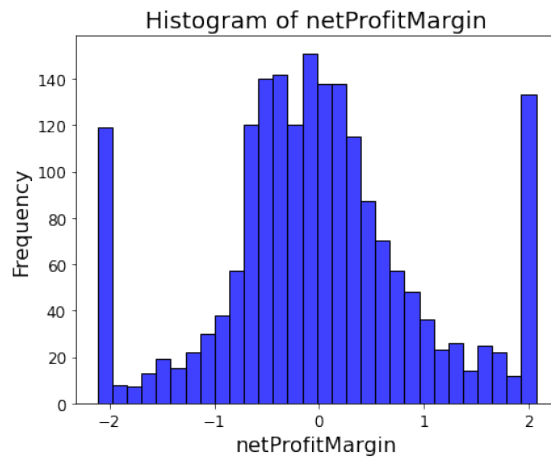
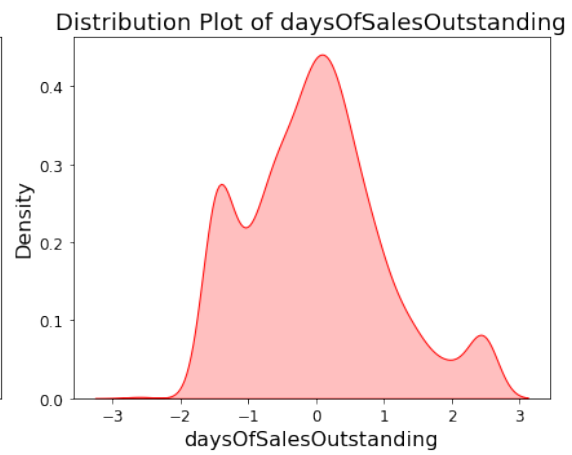
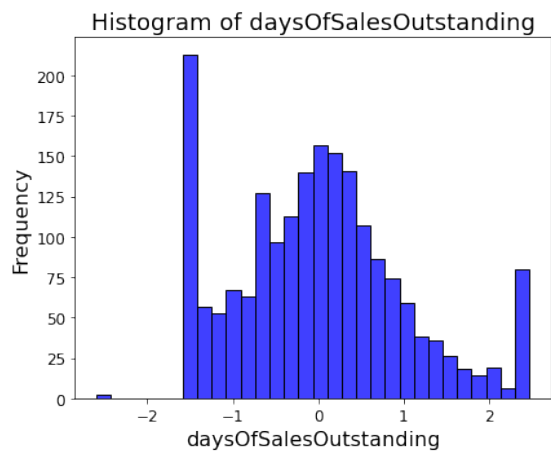
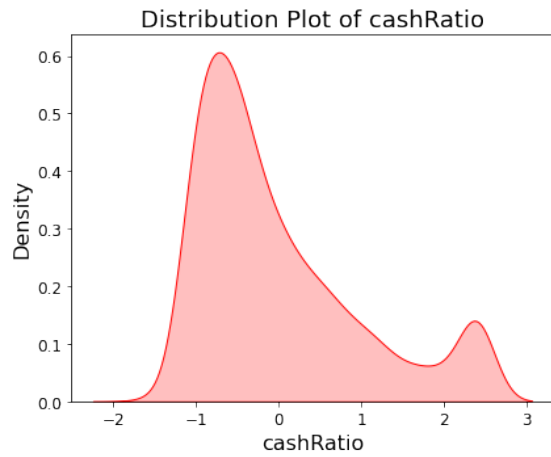
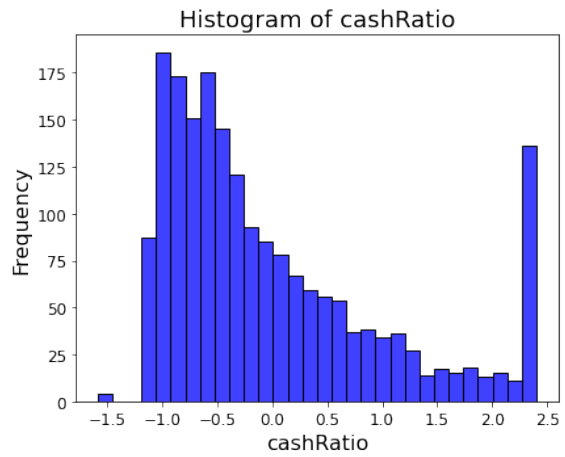


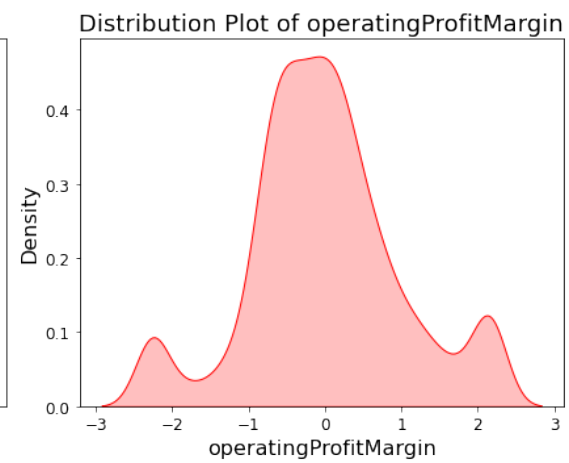
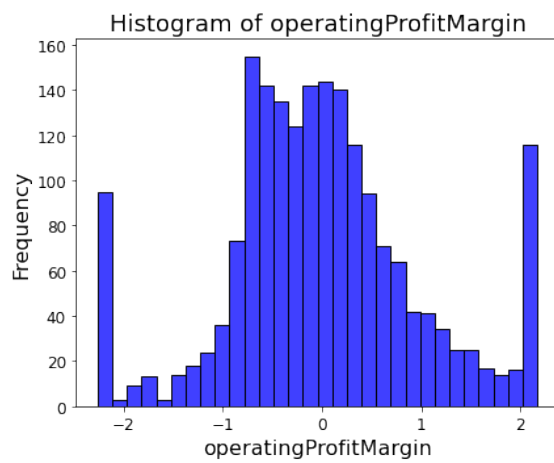
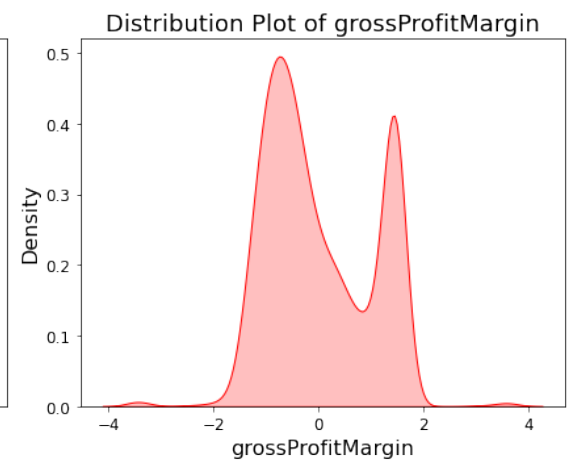
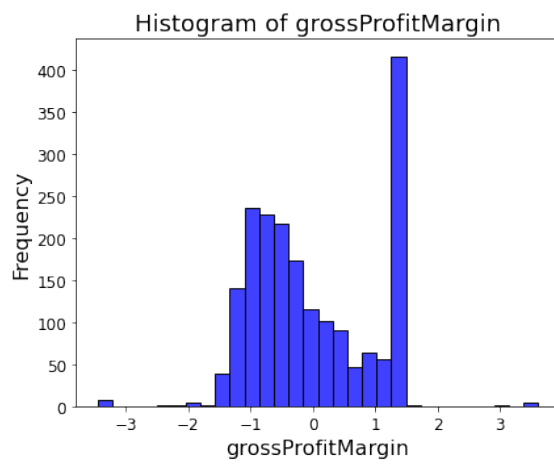
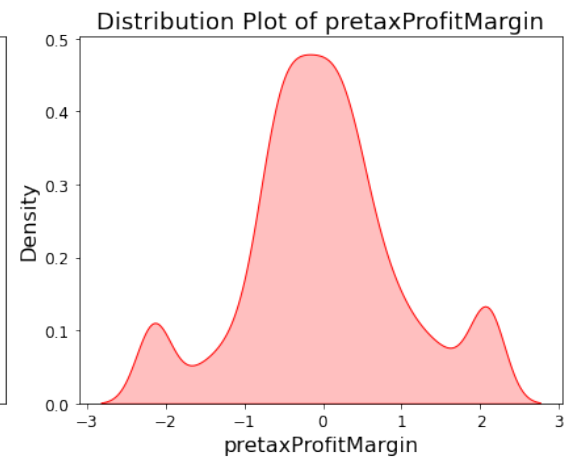
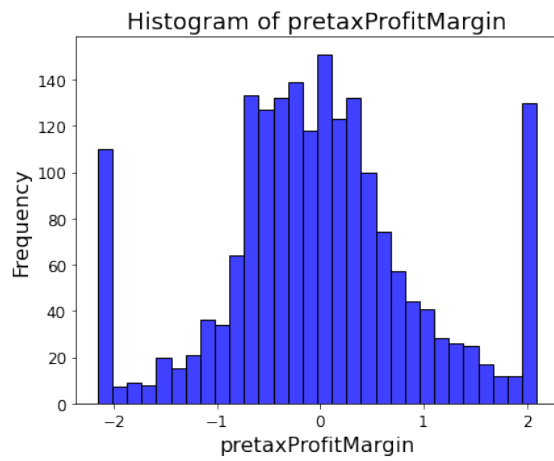


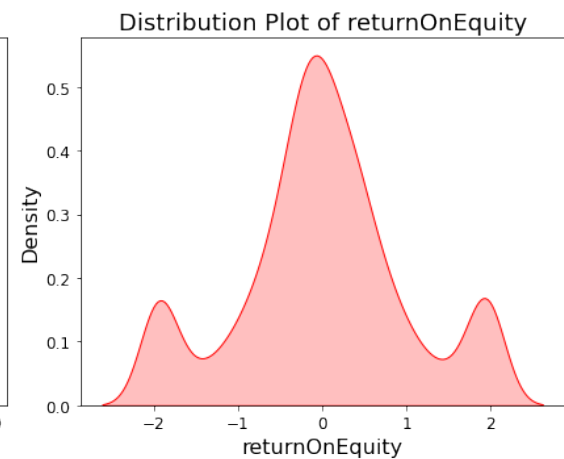
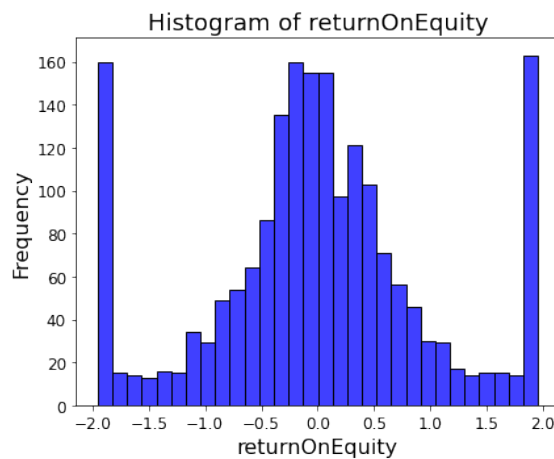
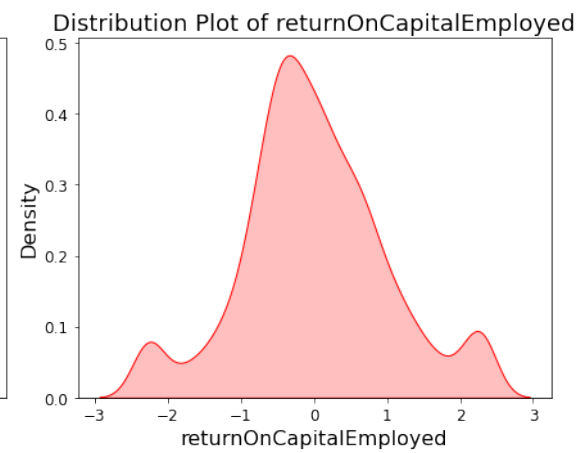
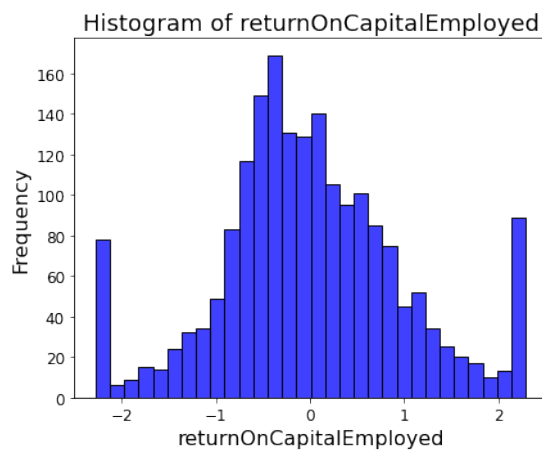
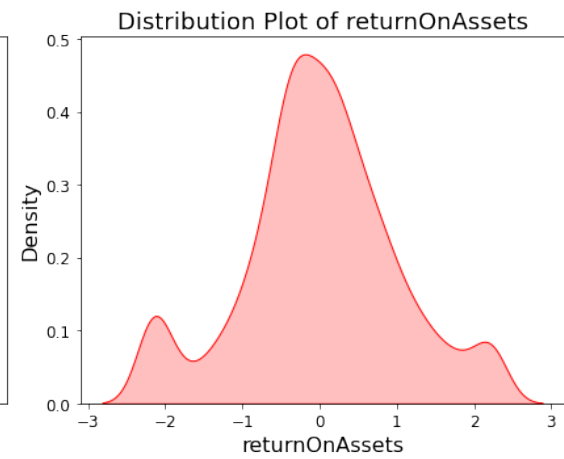
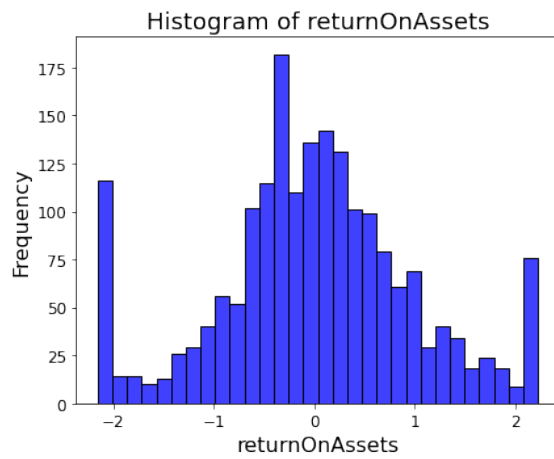


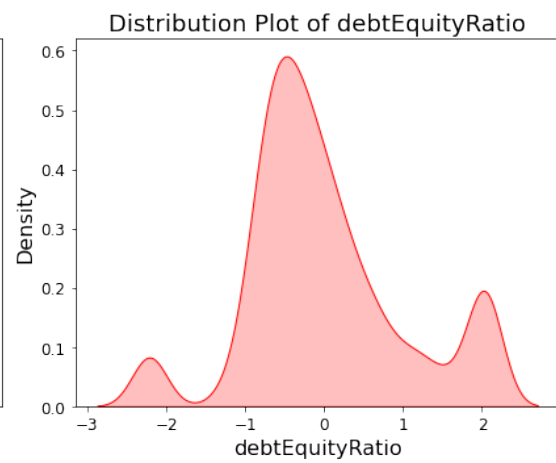
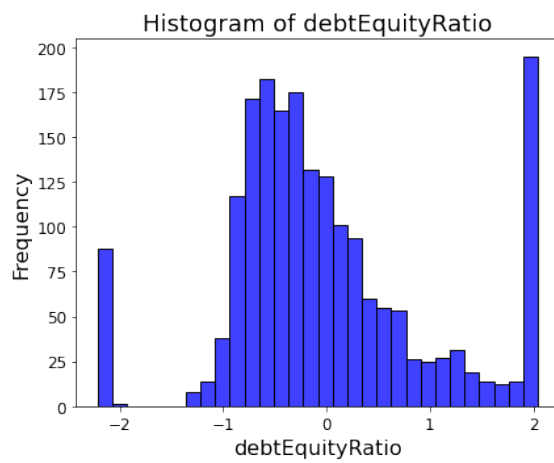
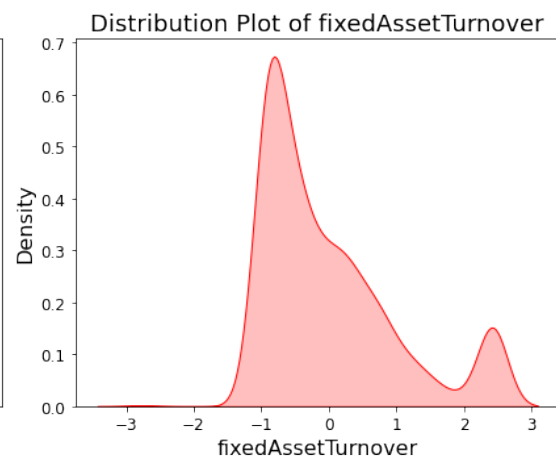
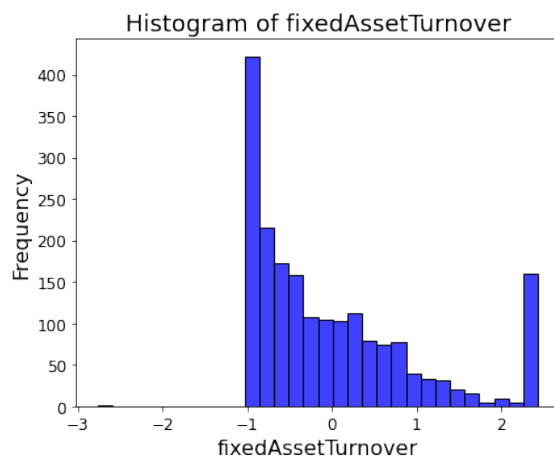
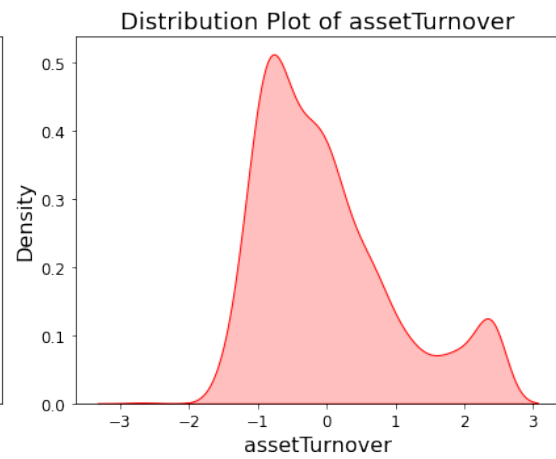
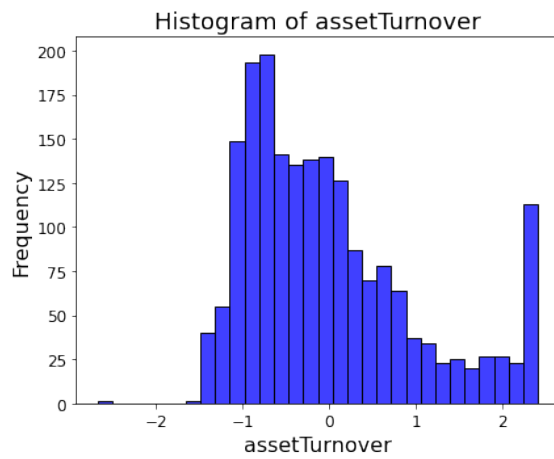
```
[305]: for pred in predictors:
        plot_histogram_distribution(credit_rating_outl_scaled, pred)
```

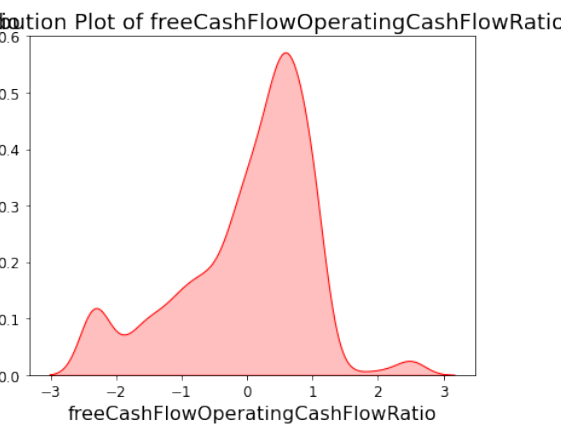
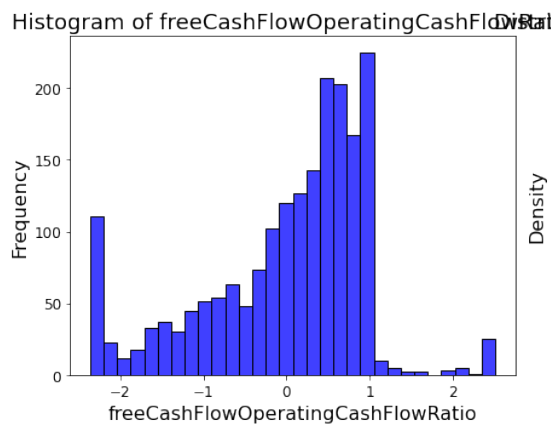
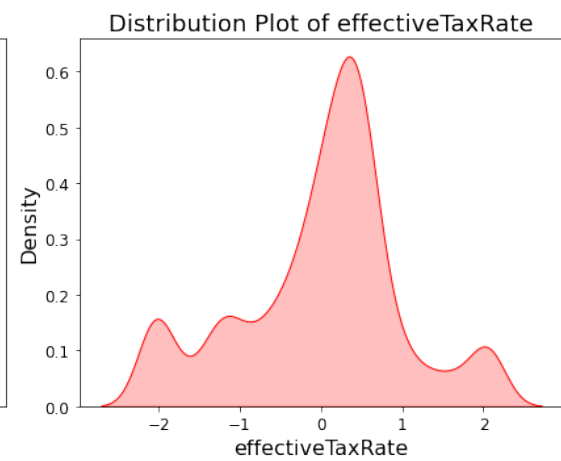
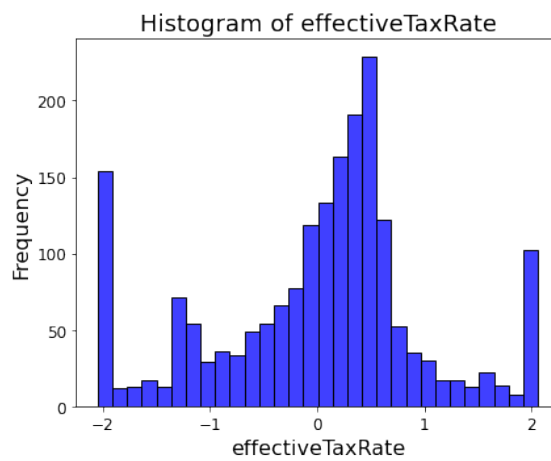
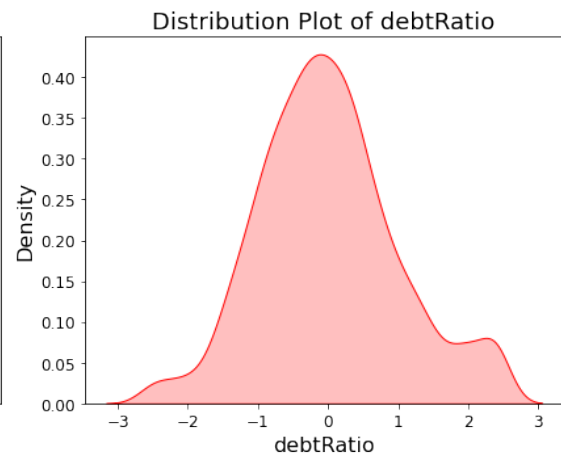
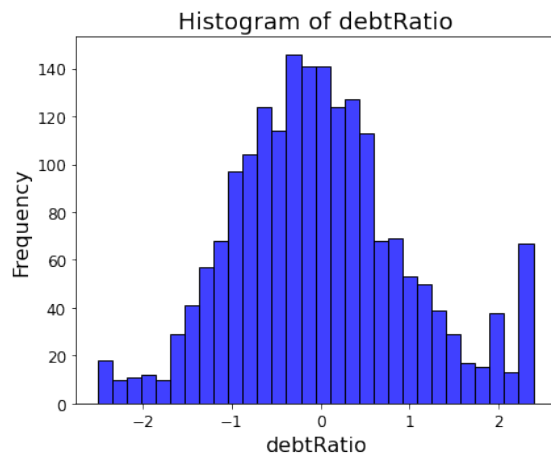


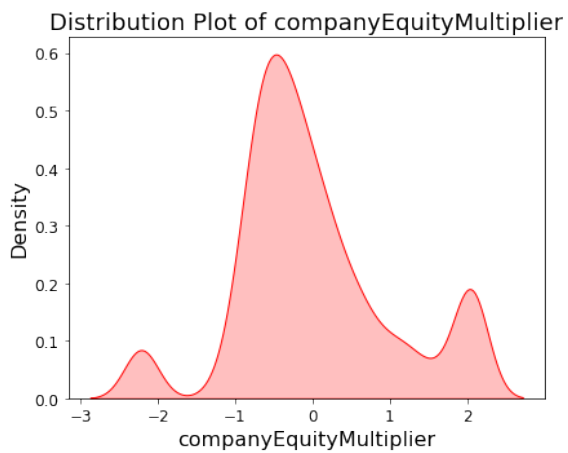
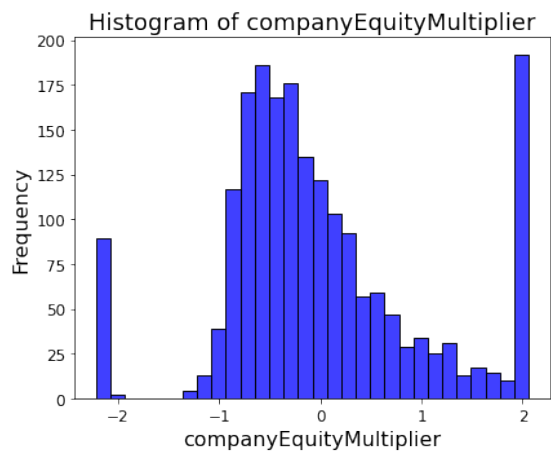
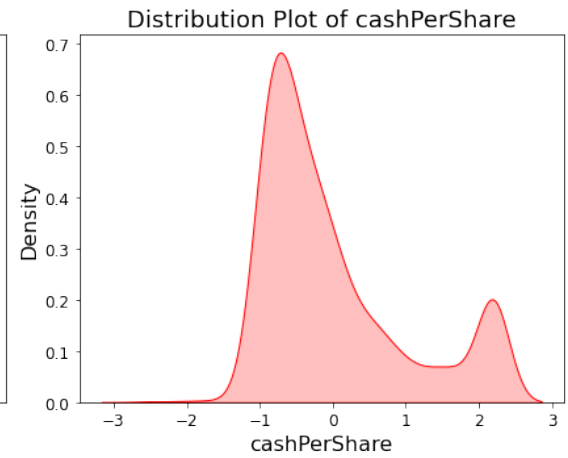
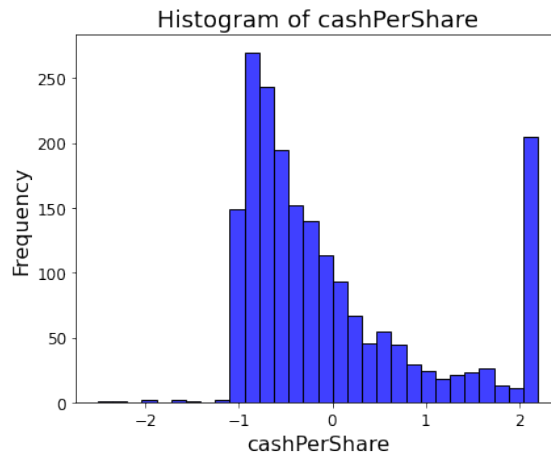
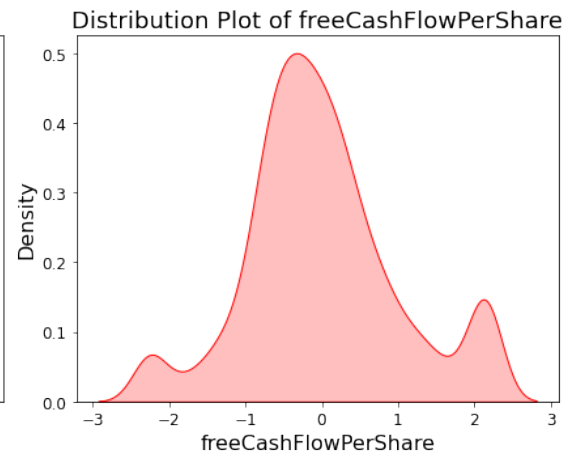
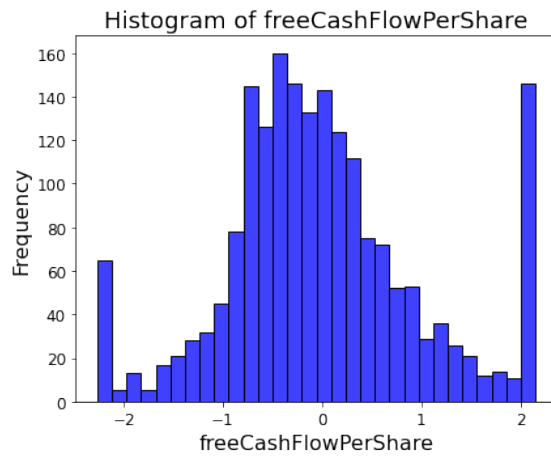


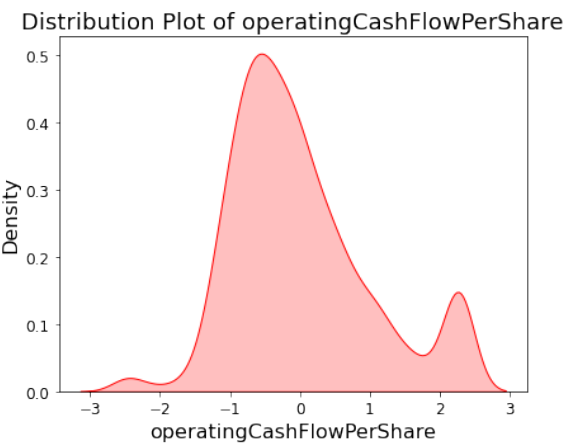
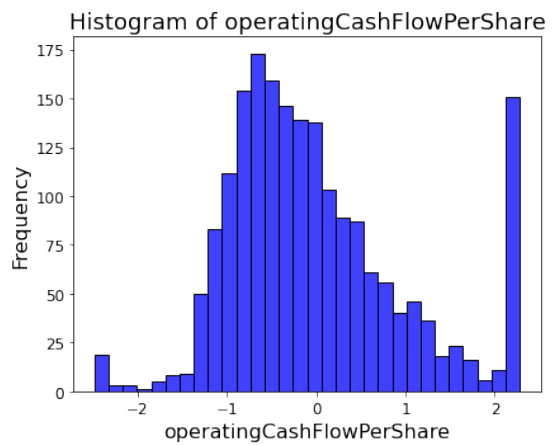
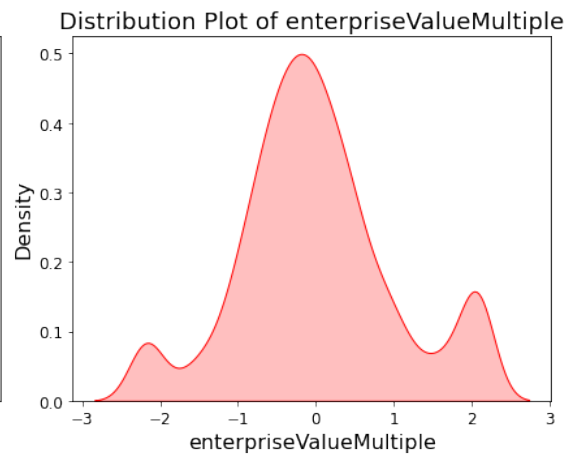
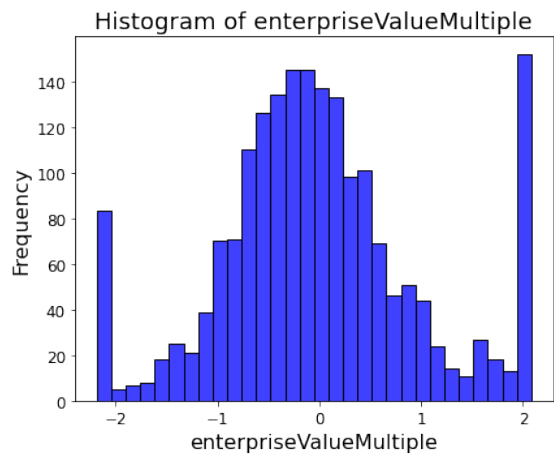
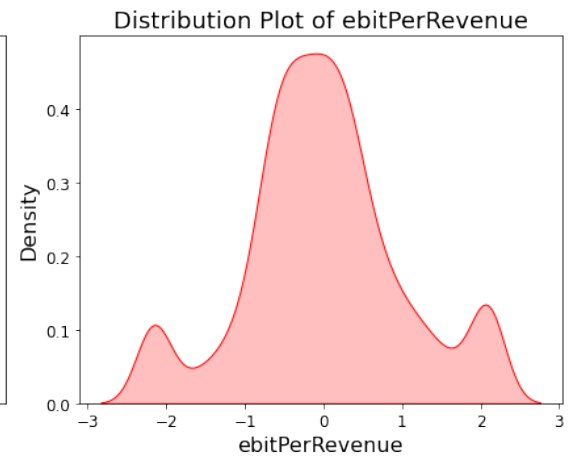
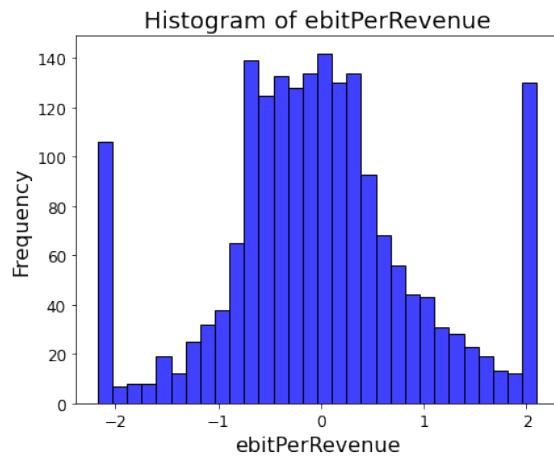


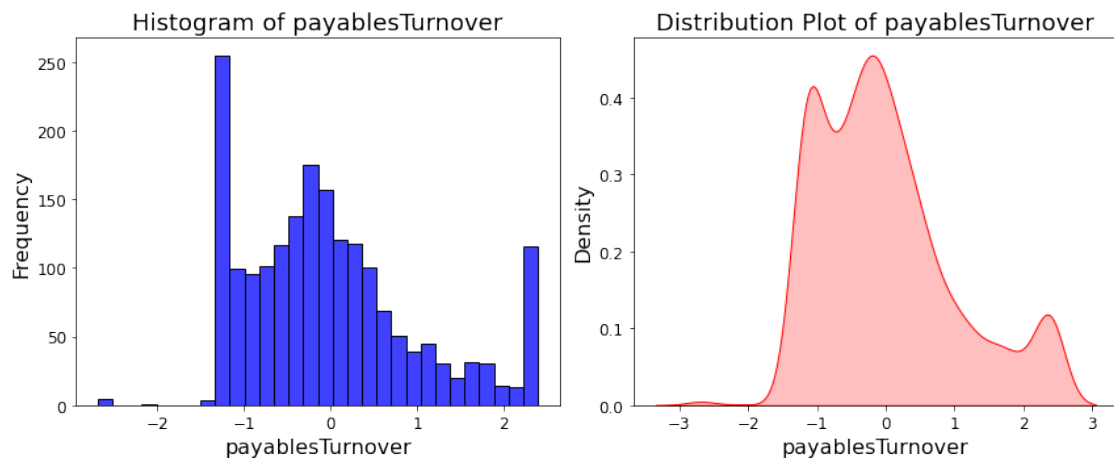
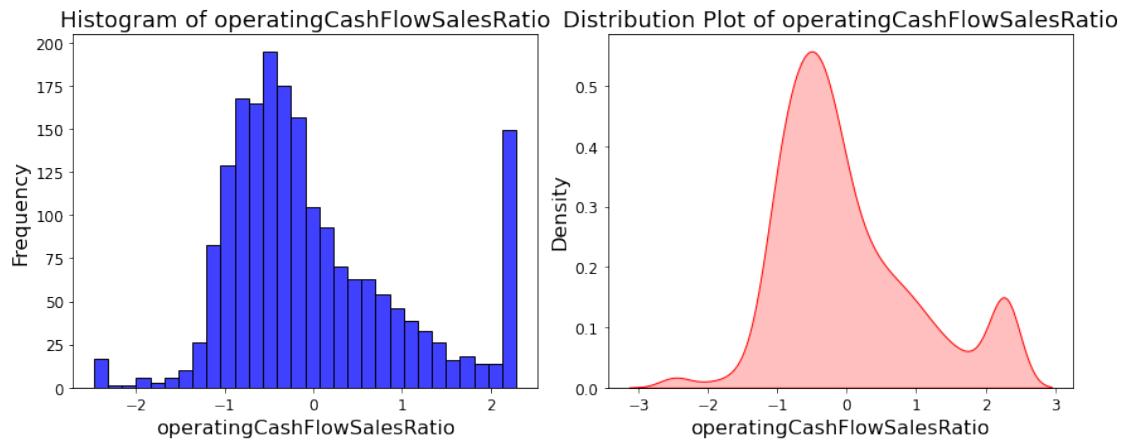












```
[306]: credit_rating_outl_scaled.describe()
```

```
[306]:
```

	currentRatio	quickRatio	cashRatio	daysOfSalesOutstanding \
count	1.945000e+03	1.945000e+03	1.945000e+03	1.945000e+03
mean	1.714709e-16	2.751298e-17	4.326730e-17	3.082367e-17
std	1.000257e+00	1.000257e+00	1.000257e+00	1.000257e+00
min	-2.548855e+00	-2.504751e+00	-1.588700e+00	-2.598960e+00
25%	-7.231023e-01	-7.342695e-01	-7.612970e-01	-6.984335e-01
50%	-2.525775e-01	-2.017022e-01	-3.392127e-01	-2.134553e-02
75%	4.940661e-01	4.460512e-01	5.042833e-01	5.685840e-01
max	2.319819e+00	2.216532e+00	2.402654e+00	2.469110e+00

	netProfitMargin	pretaxProfitMargin	grossProfitMargin \
count	1.945000e+03	1.945000e+03	1.945000e+03

mean	4.503681e-17	2.454478e-16	-2.511559e-17
std	1.000257e+00	1.000257e+00	1.000257e+00
min	-2.115429e+00	-2.158991e+00	-3.454567e+00
25%	-5.445530e-01	-5.626231e-01	-8.063889e-01
50%	-4.743245e-02	-3.084675e-02	-2.788009e-01
75%	5.026978e-01	5.016224e-01	9.590629e-01
max	2.073574e+00	2.097991e+00	3.607241e+00

	operatingProfitMargin	returnOnAssets	returnOnCapitalEmployed \
count	1.945000e+03	1.945000e+03	1.945000e+03
mean	-3.522461e-16	2.934242e-16	-3.425138e-16
std	1.000257e+00	1.000257e+00	1.000257e+00
min	-2.264685e+00	-2.153533e+00	-2.278456e+00
25%	-6.003879e-01	-5.098531e-01	-5.637810e-01
50%	-6.475154e-02	-9.384824e-03	-7.086951e-02
75%	5.091435e-01	5.859331e-01	5.793358e-01
max	2.173441e+00	2.229612e+00	2.294011e+00

	returnOnEquity	assetTurnover	fixedAssetTurnover	debtEquityRatio \
count	1.945000e+03	1.945000e+03	1.945000e+03	1.945000e+03
mean	7.277812e-19	-1.959586e-16	5.668131e-17	3.778754e-17
std	1.000257e+00	1.000257e+00	1.000257e+00	1.000257e+00
min	-1.955675e+00	-2.673765e+00	-2.768789e+00	-2.215450e+00
25%	-4.881339e-01	-7.690849e-01	-8.178734e-01	-6.175530e-01
50%	-1.412227e-02	-2.118018e-01	-3.386247e-01	-2.094728e-01
75%	4.902269e-01	5.007018e-01	4.827368e-01	4.477117e-01
max	1.957768e+00	2.405382e+00	2.433652e+00	2.045609e+00

	debtRatio	effectiveTaxRate	freeCashFlowOperatingCashFlowRatio \
count	1.945000e+03	1.945000e+03	1.945000e+03
mean	9.795078e-17	-1.391632e-16	-1.143901e-16
std	1.000257e+00	1.000257e+00	1.000257e+00
min	-2.508341e+00	-2.046952e+00	-2.362806e+00
25%	-6.714177e-01	-5.074876e-01	-5.347786e-01
50%	-6.576264e-02	1.875903e-01	2.691310e-01
75%	5.531981e-01	5.188217e-01	6.839061e-01
max	2.390122e+00	2.058286e+00	2.511933e+00

	freeCashFlowPerShare	cashPerShare	companyEquityMultiplier \
count	1.945000e+03	1.945000e+03	1.945000e+03
mean	-1.318568e-16	9.989153e-18	-4.129801e-17
std	1.000257e+00	1.000257e+00	1.000257e+00
min	-2.268610e+00	-2.508648e+00	-2.217843e+00
25%	-6.105439e-01	-7.433375e-01	-6.137241e-01
50%	-1.162456e-01	-3.558108e-01	-2.144966e-01
75%	4.948336e-01	4.335365e-01	4.556885e-01
max	2.152900e+00	2.198847e+00	2.059807e+00

	ebitPerRevenue	enterpriseValueMultiple	operatingCashFlowPerShare	\
count	1.945000e+03	1.945000e+03	1.945000e+03	
mean	-3.676008e-17	-4.486557e-17	3.196529e-18	
std	1.000257e+00	1.000257e+00	1.000257e+00	
min	-2.171141e+00	-2.186835e+00	-2.480328e+00	
25%	-5.696900e-01	-5.861817e-01	-6.953648e-01	
50%	-4.113526e-02	-9.504245e-02	-2.110277e-01	
75%	4.979439e-01	4.809204e-01	4.946106e-01	
max	2.099395e+00	2.081574e+00	2.279574e+00	

	operatingCashFlowSalesRatio	payablesTurnover	Rating_encoded	\
count	1.945000e+03	1.945000e+03	1945.000000	
mean	-5.536845e-18	-4.269649e-17	5.652442	
std	1.000257e+00	1.000257e+00	1.178741	
min	-2.477703e+00	-2.688079e+00	3.000000	
25%	-6.900441e-01	-7.834159e-01	5.000000	
50%	-2.618471e-01	-1.656273e-01	6.000000	
75%	5.017285e-01	4.863597e-01	6.000000	
max	2.289387e+00	2.391023e+00	8.000000	

	Year	Risk_level_encoded
count	1945.000000	1945.000000
mean	2014.156298	2.210797
std	1.408105	0.848496
min	2012.000000	1.000000
25%	2013.000000	2.000000
50%	2014.000000	2.000000
75%	2015.000000	3.000000
max	2016.000000	4.000000

```
[310]: from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn import preprocessing, metrics
from sklearn.model_selection import train_test_split
from sklearn.ensemble import GradientBoostingClassifier, RandomForestClassifier
from sklearn.svm import SVC
from sklearn.naive_bayes import GaussianNB
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis, QuadraticDiscriminantAnalysis
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
import xgboost as xgb
from sklearn.metrics import f1_score
from sklearn.utils.class_weight import compute_sample_weight
from sklearn.metrics import make_scorer, accuracy_score, classification_report
```

```

from sklearn.preprocessing import LabelEncoder
from sklearn import metrics, preprocessing
import numpy as np

def class_models(data_inp, X_cols, y_col, test_size, num_class):

    data = data_inp.copy()

    le = preprocessing.LabelEncoder()
    le.fit(data.Sector)
    data.Sector = le.transform(data.Sector) # Encode sector

    # Split dataset into training and testing while maintaining category
    → proportions
    X = data[X_cols] # Features
    y = data[y_col] # Target

    X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                         test_size=test_size, stratify=y, random_state=123)

    # Compute class distribution for LDA
    classes, counts = np.unique(y_train, return_counts=True)
    class_priors = counts / np.sum(counts) # Compute priors based on training
    → data

    # Initialize models
    models = {
        "Gradient Boosting": GradientBoostingClassifier(random_state=123),
        "Random Forest": RandomForestClassifier(n_estimators=200,
    → class_weight='balanced', random_state=123),
        "Linear Discriminant Analysis":
    → LinearDiscriminantAnalysis(priors=class_priors),
        "Logistic Regression": LogisticRegression(class_weight='balanced',
    → random_state=123,
                                                    multi_class='multinomial',
    → solver='newton-cg')
    }

    accuracies = {}
    sample_weights = compute_sample_weight(class_weight="balanced", y=y_train)

    # Train and evaluate each model
    for name, model in models.items():

```



```

if(name == "Gradient Boosting"):
    model.fit(X_train, y_train, sample_weight=sample_weights)

else:
    model.fit(X_train, y_train)

y_pred = model.predict(X_test)
accuracy = metrics.accuracy_score(y_test, y_pred)
accuracies[name] = accuracy
print(f"{name} Accuracy: {accuracy:.4f}")

# Train KNN models with k from 1 to 10 and find the best k
best_knn_k = 1
best_knn_accuracy = 0

for k in range(1, 11):
    knn_model = KNeighborsClassifier(n_neighbors=k, weights='distance')
    knn_model.fit(X_train, y_train)
    y_pred_knn = knn_model.predict(X_test)
    accuracy_knn = metrics.accuracy_score(y_test, y_pred_knn)

    if accuracy_knn > best_knn_accuracy:
        best_knn_accuracy = accuracy_knn
        best_knn_k = k

# Store the best KNN accuracy
accuracies[f"KNN (k={best_knn_k})"] = best_knn_accuracy
print(f"Best KNN Accuracy ({best_knn_k} neighbors): {best_knn_accuracy:.4f}")

# Convert to DataFrame and sort by accuracy
df_accuracy = pd.DataFrame(list(accuracies.items()), columns=['Model',
→'Accuracy'])
df_accuracy = df_accuracy.sort_values(by='Accuracy', ascending=False)

# Plot the accuracies
plt.figure(figsize=(12, 6))
ax = sns.barplot(x='Accuracy', y='Model', data=df_accuracy,
→palette='viridis')

# Add percentage labels on bars
for index, value in enumerate(df_accuracy["Accuracy"]):
    ax.text(value + 0.01, index, f"{value:.2%}", va='center', fontsize=12,
→fontweight='bold', color='black')

plt.xlabel("Accuracy")
plt.ylabel("Model")

```

```

plt.title("Comparison of Model Accuracies for " + y_col)
plt.xlim(0, 1) # Accuracy ranges from 0 to 1
plt.grid(axis='x', linestyle='--', alpha=0.7)
plt.show()

return df_accuracy


def class_models_accur_confus_matrix(data_inp, X_cols, y_col, test_size,
    ↪num_class):
    data = data_inp.copy()

    le = preprocessing.LabelEncoder()
    le.fit(data.Sector)
    data.Sector = le.transform(data.Sector) # Encode sector

    # Split dataset into training and testing while maintaining category
    ↪proportions
    X = data[X_cols] # Features
    y = data[y_col] # Target

    X_train, X_test, y_train, y_test = train_test_split(X, y,
        test_size=test_size, stratify=y, random_state=123)

    # Compute class distribution for LDA
    classes, counts = np.unique(y_train, return_counts=True)
    class_priors = counts / np.sum(counts) # Compute priors based on training
    ↪data

    # Initialize models
    models = {
        "Gradient Boosting": GradientBoostingClassifier(random_state=123),
        "Random Forest": RandomForestClassifier(n_estimators=200,
    ↪class_weight='balanced', random_state=123),
        "Linear Discriminant Analysis":
    ↪LinearDiscriminantAnalysis(priors=class_priors),
        "Logistic Regression": LogisticRegression(class_weight='balanced',
    ↪random_state=123,
        multi_class='multinomial',
    ↪solver='newton-cg')
    }

    accuracies = {}

```

```

sample_weights = compute_sample_weight(class_weight="balanced", y=y_train)

# Train and evaluate each model
for name, model in models.items():

    if name == "Gradient Boosting":
        model.fit(X_train, y_train, sample_weight=sample_weights)
    else:
        model.fit(X_train, y_train)

    y_pred = model.predict(X_test)
    accuracy = metrics.accuracy_score(y_test, y_pred)
    accuracies[name] = accuracy

    print(f"{name} Accuracy: {accuracy:.4f}")
    print(f"{name} Classification Report:\n{metrics.
→classification_report(y_test, y_pred)}")
    print(f"{name} Confusion Matrix:\n{metrics.confusion_matrix(y_test,
→y_pred)}\n")

# Train KNN models with k from 1 to 10 and find the best k
best_knn_k = 1
best_knn_accuracy = 0

for k in range(1, 11):
    knn_model = KNeighborsClassifier(n_neighbors=k, weights='distance')
    knn_model.fit(X_train, y_train)
    y_pred_knn = knn_model.predict(X_test)
    accuracy_knn = metrics.accuracy_score(y_test, y_pred_knn)

    if accuracy_knn > best_knn_accuracy:
        best_knn_accuracy = accuracy_knn
        best_knn_k = k

# Store the best KNN accuracy
accuracies[f"KNN (k={best_knn_k})"] = best_knn_accuracy
print(f"Best KNN Accuracy ({best_knn_k} neighbors): {best_knn_accuracy:.
→4f}")

# Convert to DataFrame and sort by accuracy
df_accuracy = pd.DataFrame(list(accuracies.items()), columns=['Model',
→'Accuracy'])
df_accuracy = df_accuracy.sort_values(by='Accuracy', ascending=False)

# Plot the accuracies
plt.figure(figsize=(12, 6))

```

```

    ax = sns.barplot(x='Accuracy', y='Model', data=df_accuracy,
→palette='viridis')

    # Add percentage labels on bars
    for index, value in enumerate(df_accuracy["Accuracy"]):
        ax.text(value + 0.01, index, f"{value:.2%}", va='center', fontsize=12,
→fontweight='bold', color='black')

    plt.xlabel("Accuracy")
    plt.ylabel("Model")
    plt.title("Comparison of Model Accuracies for " + y_col)
    plt.xlim(0, 1) # Accuracy ranges from 0 to 1
    plt.grid(axis='x', linestyle='--', alpha=0.7)
    plt.show()

    return df_accuracy

def class_models_f1(data_inp, X_cols, y_col, test_size, num_class):

    data = data_inp.copy()

    le = preprocessing.LabelEncoder()
    le.fit(data.Sector)
    data.Sector = le.transform(data.Sector) # Encode sector

    # Split dataset into training and testing while maintaining category
→proportions
    X = data[X_cols] # Features
    y = data[y_col] # Target

    X_train, X_test, y_train, y_test = train_test_split(X, y,
        test_size=test_size, stratify=y, random_state=123)

    # Compute class distribution for LDA
    classes, counts = np.unique(y_train, return_counts=True)
    class_priors = counts / np.sum(counts) # Compute priors based on training
→data

    # Initialize models
    models = {

        "Gradient Boosting": GradientBoostingClassifier(random_state=123),
        "Random Forest": RandomForestClassifier(n_estimators=200,
→class_weight='balanced', random_state=123),

```

```

        "Linear Discriminant Analysis":  

        ↳LinearDiscriminantAnalysis(priors=class_priors),  

        "Logistic Regression": LogisticRegression(class_weight='balanced',  

        ↳random_state=123,  

        multi_class='multinomial',  

        ↳solver='newton-cg')  

    }  
  

    f1_scores = {}  

    sample_weights = compute_sample_weight(class_weight="balanced", y=y_train)  
  

    # Train and evaluate each model  

    for name, model in models.items():  
  

        if(name == "Gradient Boosting"):  

            model.fit(X_train, y_train, sample_weight=sample_weights)  
  

        else:  

            model.fit(X_train, y_train)  
  

        y_pred = model.predict(X_test)  

        f1 = f1_score(y_test, y_pred, average='weighted')  

        f1_scores[name] = f1  
  

        print(f"{name} F1 Score: {f1:.4f}")  
  

    # Train KNN models with k from 1 to 10 and find the best k  

    best_knn_k = 1  

    best_knn_f1 = 0  
  

    for k in range(1, 11):  

        knn_model = KNeighborsClassifier(n_neighbors=k, weights='distance')  

        knn_model.fit(X_train, y_train)  

        y_pred_knn = knn_model.predict(X_test)  

        f1_knn = f1_score(y_test, y_pred_knn, average='weighted')  
  

        if f1_knn > best_knn_f1:  

            best_knn_f1 = f1_knn  

            best_knn_k = k  
  

    # Store the best KNN F1 score  

    f1_scores[f"KNN (k={best_knn_k})"] = best_knn_f1  

    print(f"Best KNN F1 Score ({best_knn_k} neighbors): {best_knn_f1:.4f}")

```

```

# Convert to DataFrame and sort by F1 score
df_f1 = pd.DataFrame(list(f1_scores.items()), columns=['Model', 'F1 Score'])
df_f1 = df_f1.sort_values(by='F1 Score', ascending=False)

# Plot the F1 scores
plt.figure(figsize=(12, 6))
ax = sns.barplot(x='F1 Score', y='Model', data=df_f1, palette='viridis')

# Add percentage labels on bars
for index, value in enumerate(df_f1["F1 Score"]):
    ax.text(value + 0.01, index, f"{value:.2f}", va='center', fontsize=12,
    →fontweight='bold', color='black')

plt.xlabel("F1 Score")
plt.ylabel("Model")
plt.title("Comparison of Model F1 Scores for " + y_col)
plt.xlim(0, 1) # F1 score ranges from 0 to 1
plt.grid(axis='x', linestyle='--', alpha=0.7)
plt.show()

return df_f1

def class_models_macro_avg(data_inp, X_cols, y_col, test_size, num_class):

    data = data_inp.copy()

    # Encode categorical variables
    le = preprocessing.LabelEncoder()
    le.fit(data.Sector)
    data.Sector = le.transform(data.Sector) # Encode sector

    # Split dataset into training and testing while maintaining category
    →proportions
    X = data[X_cols] # Features
    y = data[y_col] # Target

    X_train, X_test, y_train, y_test = train_test_split(X, y,
        test_size=test_size, stratify=y, random_state=123)

    # Compute class distribution for LDA
    classes, counts = np.unique(y_train, return_counts=True)
    class_priors = counts / np.sum(counts) # Compute priors based on training
    →data

```

```

# Initialize models
models = {
    "Gradient Boosting": GradientBoostingClassifier(random_state=123),
    "Random Forest": RandomForestClassifier(n_estimators=200,
    ↪class_weight='balanced', random_state=123),
    "Linear Discriminant Analysis":
    ↪LinearDiscriminantAnalysis(priors=class_priors),
    "Logistic Regression": LogisticRegression(class_weight='balanced',
    ↪random_state=123,
    ↪multi_class='multinomial',
    ↪solver='newton-cg')
}

accuracies = {}

sample_weights = compute_sample_weight(class_weight="balanced", y=y_train)

# Train and evaluate each model
for name, model in models.items():

    if(name == "Gradient Boosting"):
        model.fit(X_train, y_train, sample_weight=sample_weights)

    else:
        model.fit(X_train, y_train)

    y_pred = model.predict(X_test)

    # Compute macro-average accuracy
    macro_avg_accuracy = metrics.balanced_accuracy_score(y_test, y_pred)
    accuracies[name] = macro_avg_accuracy
    print(f"{name} Macro-Average Accuracy: {macro_avg_accuracy:.4f}")

# Train KNN models with k from 1 to 10 and find the best k
best_knn_k = 1
best_knn_accuracy = 0

for k in range(1, 11):
    knn_model = KNeighborsClassifier(n_neighbors=k, weights='distance')
    knn_model.fit(X_train, y_train)
    y_pred_knn = knn_model.predict(X_test)

    # Compute macro-average accuracy for KNN
    accuracy_knn = metrics.balanced_accuracy_score(y_test, y_pred_knn)

```

```

        if accuracy_knn > best_knn_accuracy:
            best_knn_accuracy = accuracy_knn
            best_knn_k = k

    # Store the best KNN accuracy
    accuracies[f"KNN (k={best_knn_k})"] = best_knn_accuracy
    print(f"Best KNN Macro-Average Accuracy ({best_knn_k} neighbors):␣
→{best_knn_accuracy:.4f}")

    # Convert to DataFrame and sort by accuracy
    df_accuracy = pd.DataFrame(list(accuracies.items()), columns=['Model',␣
→'Macro Avg Accuracy'])
    df_accuracy = df_accuracy.sort_values(by='Macro Avg Accuracy',␣
→ascending=False)

    # Plot the accuracies
    plt.figure(figsize=(12, 6))
    ax = sns.barplot(x='Macro Avg Accuracy', y='Model', data=df_accuracy,␣
→palette='viridis')

    # Add percentage labels on bars
    for index, value in enumerate(df_accuracy["Macro Avg Accuracy"]):
        ax.text(value + 0.01, index, f"{value:.2%}", va='center', fontsize=12,␣
→fontweight='bold', color='black')

    plt.xlabel("Macro-Average Accuracy")
    plt.ylabel("Model")
    plt.title("Comparison of Model Macro Avg Accuracies for " + y_col)
    plt.xlim(0, 1) # Accuracy ranges from 0 to 1
    plt.grid(axis='x', linestyle='--', alpha=0.7)
    plt.show()

    return df_accuracy

def class_models_macro_f1(data_inp, X_cols, y_col, test_size, num_class):

    data = data_inp.copy()

    le = preprocessing.LabelEncoder()
    le.fit(data.Sector)
    data.Sector = le.transform(data.Sector) # Encode sector

```



```

    # Split dataset into training and testing while maintaining category
    ↳ proportions
    X = data[X_cols] # Features
    y = data[y_col] # Target

    X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                         test_size=test_size, stratify=y, random_state=123)

    # Compute class distribution for LDA
    classes, counts = np.unique(y_train, return_counts=True)
    class_priors = counts / np.sum(counts) # Compute priors based on training
    ↳ data

    # Initialize models
    models = {
        "Gradient Boosting": GradientBoostingClassifier(random_state=123),
        "Random Forest": RandomForestClassifier(n_estimators=200,
    ↳ class_weight='balanced', random_state=123),
        "Linear Discriminant Analysis":
    ↳ LinearDiscriminantAnalysis(priors=class_priors),
        "Logistic Regression": LogisticRegression(class_weight='balanced',
    ↳ random_state=123, multi_class='multinomial', solver='newton-cg')
    }

    f1_scores = {}
    sample_weights = compute_sample_weight(class_weight="balanced", y=y_train)

    # Train and evaluate each model using MACRO F1
    for name, model in models.items():

        if(name == "Gradient Boosting"):
            model.fit(X_train, y_train, sample_weight=sample_weights)

        else:
            model.fit(X_train, y_train)

    y_pred = model.predict(X_test)

    #Compute macro avg F1
    f1 = f1_score(y_test, y_pred, average='macro')
    f1_scores[name] = f1
    print(f"{name} Macro F1 Score: {f1:.4f}")

    # Train KNN models with k from 1 to 10 and find the best k
    best_knn_k = 1

```

```

best_knn_f1 = 0

for k in range(1, 11):
    knn_model = KNeighborsClassifier(n_neighbors=k, weights='distance')
    knn_model.fit(X_train, y_train)
    y_pred_knn = knn_model.predict(X_test)

    f1_knn = f1_score(y_test, y_pred_knn, average='macro')

    if f1_knn > best_knn_f1:
        best_knn_f1 = f1_knn
        best_knn_k = k

# Store the best KNN Macro F1 score
f1_scores[f"KNN (k={best_knn_k})"] = best_knn_f1
print(f"Best KNN Macro F1 Score ({best_knn_k} neighbors): {best_knn_f1:.4f}")

# Convert to DataFrame and sort by F1 score
df_f1 = pd.DataFrame(list(f1_scores.items()), columns=['Model', 'F1 Score'])
df_f1 = df_f1.sort_values(by='F1 Score', ascending=False)

# Plot the F1 scores
plt.figure(figsize=(12, 6))
ax = sns.barplot(x='F1 Score', y='Model', data=df_f1, palette='viridis')

# Add percentage labels on bars
for index, value in enumerate(df_f1["F1 Score"]):
    ax.text(value + 0.01, index, f"{value:.2f}", va='center', fontsize=12,
    fontweight='bold', color='black')

plt.xlabel("F1 Score")
plt.ylabel("Model")
plt.title("Comparison of Model Macro F1 Scores for " + y_col)
plt.xlim(0, 1) # F1 score ranges from 0 to 1
plt.grid(axis='x', linestyle='--', alpha=0.7)
plt.show()

return df_f1

```

```

[311]: X_cols=['Sector', 'currentRatio', 'quickRatio', 'cashRatio',
    → 'daysOfSalesOutstanding',
        'netProfitMargin', 'pretaxProfitMargin', 'grossProfitMargin',
        'operatingProfitMargin', 'returnOnAssets', 'returnOnCapitalEmployed',
        'returnOnEquity', 'assetTurnover', 'fixedAssetTurnover',
        'debtEquityRatio', 'debtRatio', 'effectiveTaxRate',
        'freeCashFlowOperatingCashFlowRatio', 'freeCashFlowPerShare',

```

```
'cashPerShare', 'companyEquityMultiplier', 'ebitPerRevenue',
'enterpriseValueMultiple', 'operatingCashFlowPerShare',
'operatingCashFlowSalesRatio', 'payablesTurnover']
```

```
[312]: class_models(data_inp=credit_rating_new, X_cols=X_cols, y_col =
↳ 'Rating_encoded', test_size=0.2, num_class = 6)
```

Gradient Boosting Accuracy: 0.5039

Random Forest Accuracy: 0.5964

Linear Discriminant Analysis Accuracy: 0.3342

/Users/kamila/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.

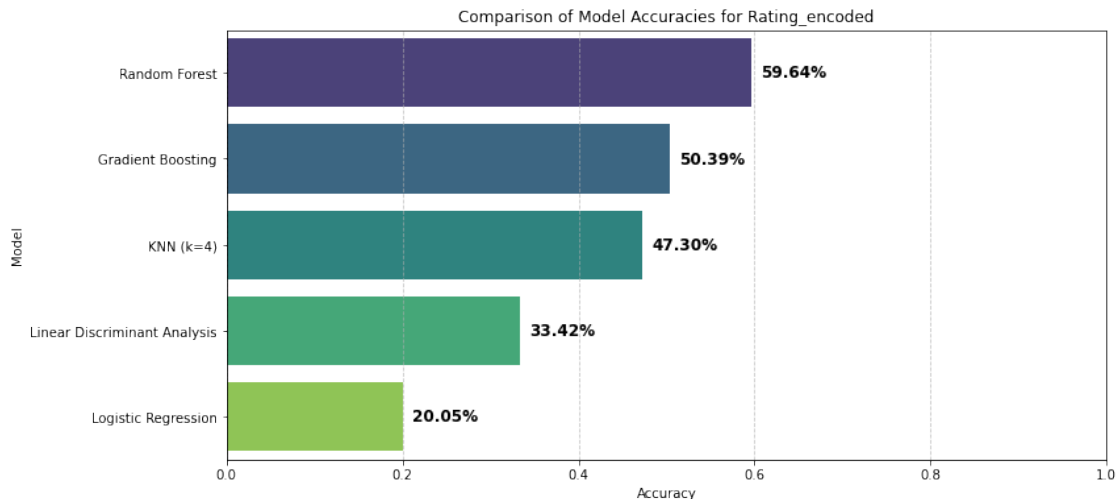
warnings.warn(

/Users/kamila/opt/anaconda3/lib/python3.9/site-packages/sklearn/utils/optimize.py:319: ConvergenceWarning: newton-cg failed to converge at loss = 1.5397029451003554. Increase the number of iterations.

warnings.warn(

Logistic Regression Accuracy: 0.2005

Best KNN Accuracy (4 neighbors): 0.4730



```
[312]:
1          Model  Accuracy
0      Random Forest  0.596401
      Gradient Boosting  0.503856
```

```

4           KNN (k=4)  0.473008
2  Linear Discriminant Analysis  0.334190
3           Logistic Regression  0.200514

```

```
[313]: class_models(data_inp=credit_rating_outl, X_cols=X_cols,
                y_col = 'Rating_encoded', test_size=0.2, num_class = 6)
```

Gradient Boosting Accuracy: 0.5270

Random Forest Accuracy: 0.5758

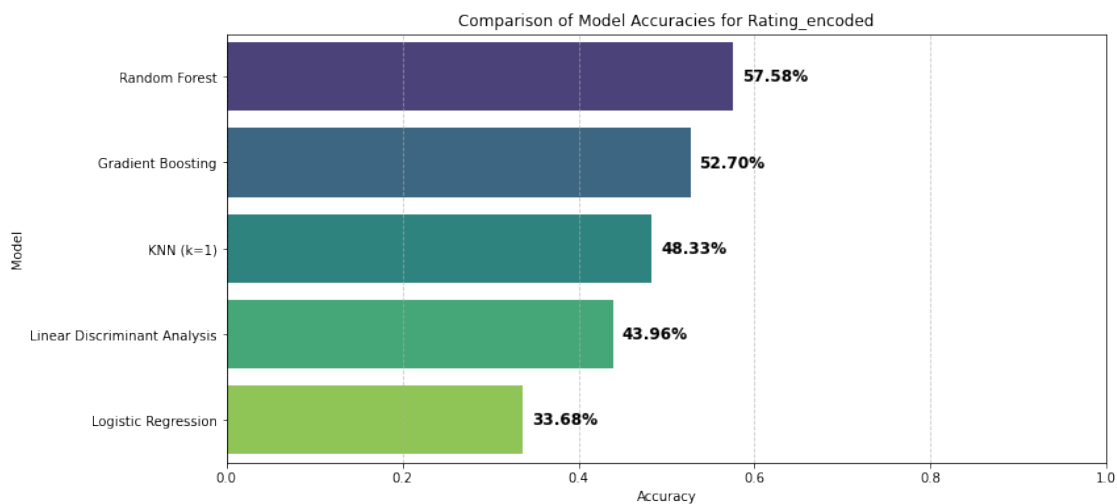
Linear Discriminant Analysis Accuracy: 0.4396

/Users/kamila/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.

warnings.warn(

Logistic Regression Accuracy: 0.3368

Best KNN Accuracy (1 neighbors): 0.4833



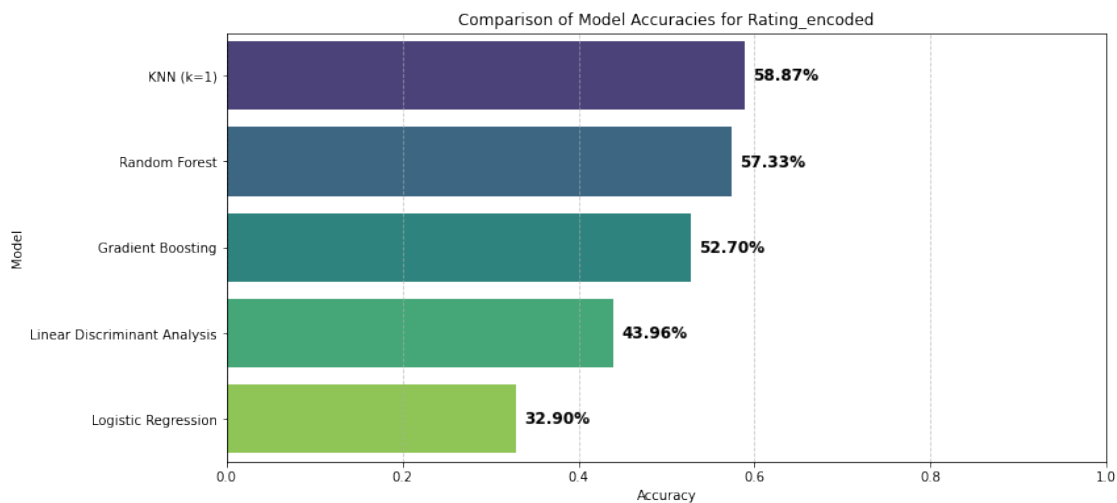
```
[313]:
Model  Accuracy
1      Random Forest  0.575835
0      Gradient Boosting  0.526992
4      KNN (k=1)  0.483290
2  Linear Discriminant Analysis  0.439589
3      Logistic Regression  0.336761

```

```
[314]: class_models(data_inp=credit_rating_outl_scaled, X_cols=X_cols, y_col =
        ↳ 'Rating_encoded',
        test_size=0.2, num_class = 6)
```

Gradient Boosting Accuracy: 0.5270
 Random Forest Accuracy: 0.5733
 Linear Discriminant Analysis Accuracy: 0.4396
 Logistic Regression Accuracy: 0.3290
 Best KNN Accuracy (1 neighbors): 0.5887

/Users/kamila/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.
 warnings.warn(



```
[314]:
```

	Model	Accuracy
4	KNN (k=1)	0.588689
1	Random Forest	0.573265
0	Gradient Boosting	0.526992
2	Linear Discriminant Analysis	0.439589
3	Logistic Regression	0.329049

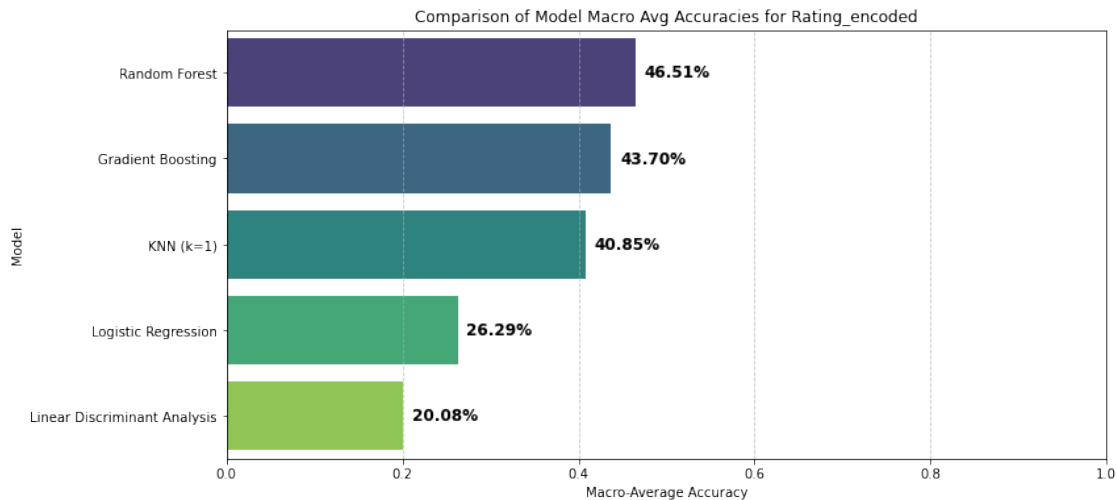
```
[315]: class_models_macro_avg(data_inp=credit_rating_new, X_cols=X_cols, y_col =
↳ 'Rating_encoded', test_size=0.2, num_class = 6)
```

Gradient Boosting Macro-Average Accuracy: 0.4370
 Random Forest Macro-Average Accuracy: 0.4651
 Linear Discriminant Analysis Macro-Average Accuracy: 0.2008

/Users/kamila/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.
 warnings.warn(

Logistic Regression Macro-Average Accuracy: 0.2629
 Best KNN Macro-Average Accuracy (1 neighbors): 0.4085

/Users/kamila/opt/anaconda3/lib/python3.9/site-packages/sklearn/utils/optimize.py:319: ConvergenceWarning: newton-cg failed to converge at loss = 1.5397029451003554. Increase the number of iterations.
 warnings.warn(



```
[315]:
```

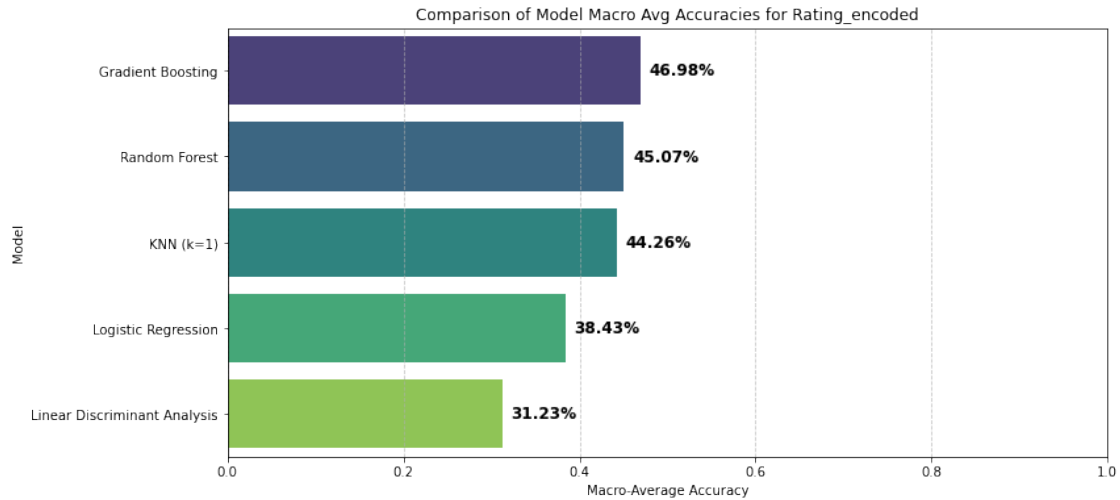
	Model	Macro Avg Accuracy
1	Random Forest	0.465123
0	Gradient Boosting	0.437038
4	KNN (k=1)	0.408524
3	Logistic Regression	0.262878
2	Linear Discriminant Analysis	0.200781

```
[316]: class_models_macro_avg(data_inp=credit_rating_outl, X_cols=X_cols, y_col =
↳ 'Rating_encoded', test_size=0.2, num_class = 6)
```

Gradient Boosting Macro-Average Accuracy: 0.4698
 Random Forest Macro-Average Accuracy: 0.4507
 Linear Discriminant Analysis Macro-Average Accuracy: 0.3123

/Users/kamila/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.
 warnings.warn(

Logistic Regression Macro-Average Accuracy: 0.3843
 Best KNN Macro-Average Accuracy (1 neighbors): 0.4426



```
[316]:
```

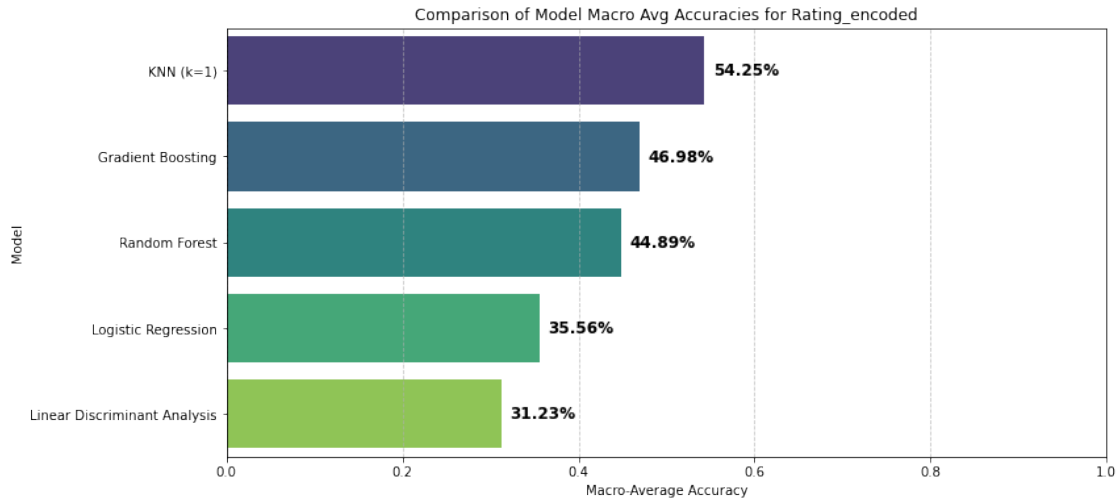
	Model	Macro Avg Accuracy
0	Gradient Boosting	0.469750
1	Random Forest	0.450693
4	KNN (k=1)	0.442575
3	Logistic Regression	0.384307
2	Linear Discriminant Analysis	0.312337

```
[317]: class_models_macro_avg(data_inp=credit_rating_outl_scaled, X_cols=X_cols, y_col=
↳ 'Rating_encoded', test_size=0.2, num_class = 6)
```

```
Gradient Boosting Macro-Average Accuracy: 0.4698
Random Forest Macro-Average Accuracy: 0.4489
Linear Discriminant Analysis Macro-Average Accuracy: 0.3123
Logistic Regression Macro-Average Accuracy: 0.3556
```

```
/Users/kamila/opt/anaconda3/lib/python3.9/site-
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will
always use 'multinomial'. Leave it to its default value to avoid this warning.
warnings.warn(
```

```
Best KNN Macro-Average Accuracy (1 neighbors): 0.5425
```



```
[317]:
```

	Model	Macro Avg Accuracy
4	KNN (k=1)	0.542530
0	Gradient Boosting	0.469750
1	Random Forest	0.448920
3	Logistic Regression	0.355561
2	Linear Discriminant Analysis	0.312337

```
[318]: class_models_macro_f1(data_inp=credit_rating_new, X_cols=X_cols, y_col =
↳ 'Rating_encoded', test_size=0.2, num_class = 6)
```

Gradient Boosting Macro F1 Score: 0.4334

Random Forest Macro F1 Score: 0.4910

Linear Discriminant Analysis Macro F1 Score: 0.1570

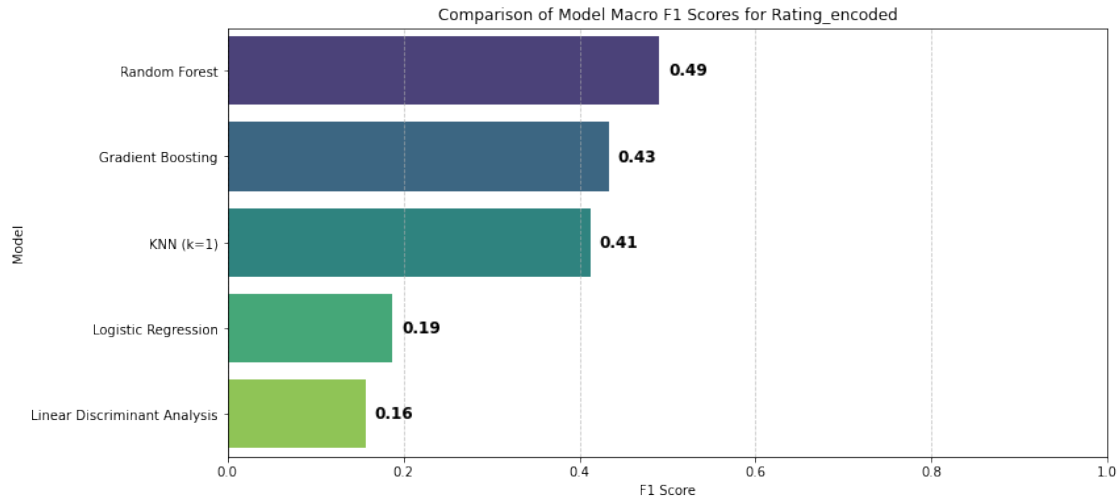
/Users/kamila/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.

warnings.warn(
/Users/kamila/opt/anaconda3/lib/python3.9/site-packages/sklearn/utils/optimize.py:319: ConvergenceWarning: newton-cg failed to converge at loss = 1.5397029451003554. Increase the number of iterations.

warnings.warn(

Logistic Regression Macro F1 Score: 0.1878

Best KNN Macro F1 Score (1 neighbors): 0.4120



```
[318]:
```

	Model	F1 Score
1	Random Forest	0.491034
0	Gradient Boosting	0.433415
4	KNN (k=1)	0.412018
3	Logistic Regression	0.187845
2	Linear Discriminant Analysis	0.157008

```
[319]: class_models_macro_f1(data_inp=credit_rating_out1, X_cols=X_cols, y_col =
↳ 'Rating_encoded', test_size=0.2, num_class = 6)
```

Gradient Boosting Macro F1 Score: 0.4683

Random Forest Macro F1 Score: 0.4758

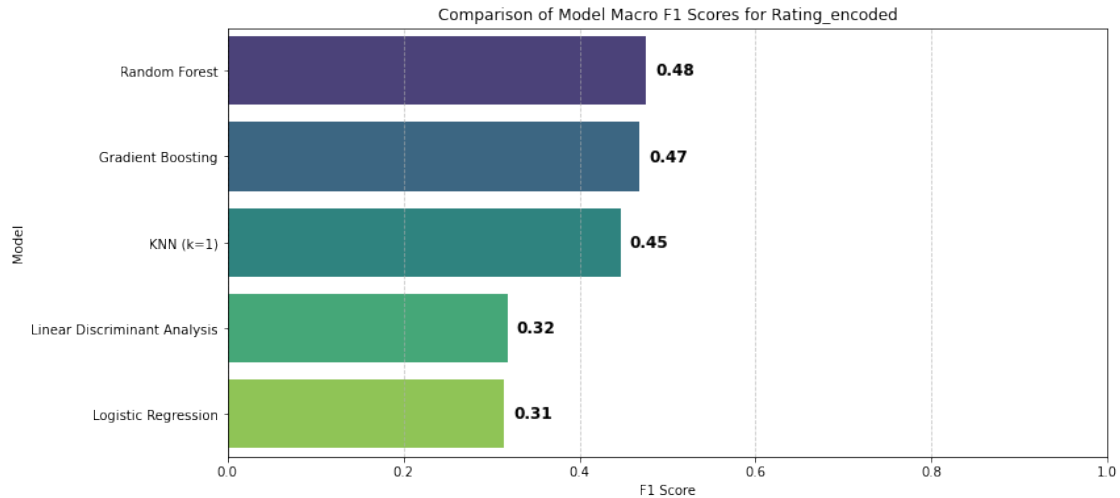
Linear Discriminant Analysis Macro F1 Score: 0.3188

/Users/kamila/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.

warnings.warn(

Logistic Regression Macro F1 Score: 0.3148

Best KNN Macro F1 Score (1 neighbors): 0.4466



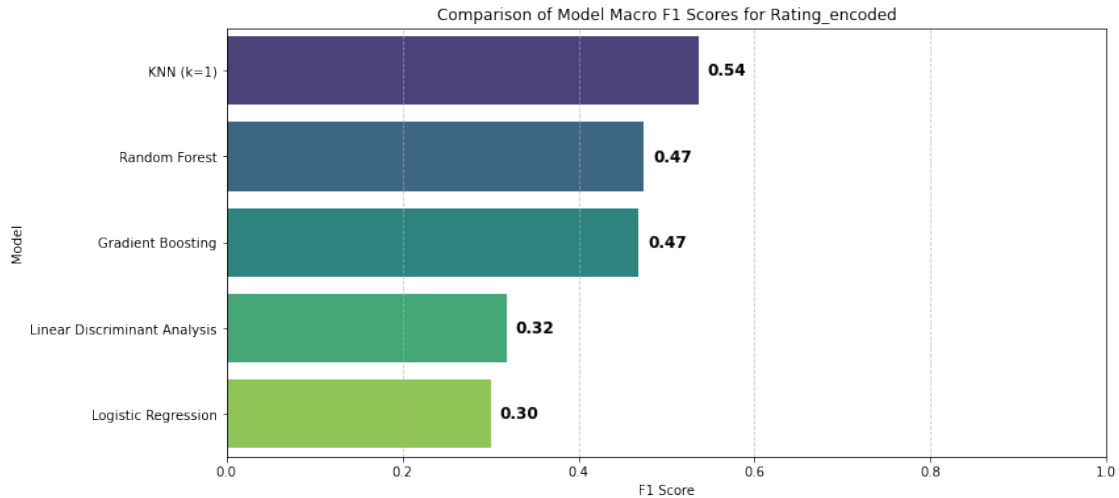
```
[319]:
```

	Model	F1 Score
1	Random Forest	0.475825
0	Gradient Boosting	0.468263
4	KNN (k=1)	0.446554
2	Linear Discriminant Analysis	0.318815
3	Logistic Regression	0.314759

```
[320]: class_models_macro_f1(data_inp=credit_rating_outl_scaled, X_cols=X_cols, y_col_
↳ 'Rating_encoded', test_size=0.2, num_class = 6)
```

```
Gradient Boosting Macro F1 Score: 0.4683
Random Forest Macro F1 Score: 0.4746
Linear Discriminant Analysis Macro F1 Score: 0.3188
Logistic Regression Macro F1 Score: 0.3006
Best KNN Macro F1 Score (1 neighbors): 0.5368
```

```
/Users/kamila/opt/anaconda3/lib/python3.9/site-
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will
always use 'multinomial'. Leave it to its default value to avoid this warning.
warnings.warn(
```



```
[320]:
```

	Model	F1 Score
4	KNN (k=1)	0.536838
1	Random Forest	0.474598
0	Gradient Boosting	0.468263
2	Linear Discriminant Analysis	0.318815
3	Logistic Regression	0.300554

```
[321]: class_models(data_inp=credit_rating_new , X_cols=X_cols, y_col =
↳ 'Risk_level_encoded', test_size=0.2, num_class = 4)
```

Gradient Boosting Accuracy: 0.6247

Random Forest Accuracy: 0.6684

Linear Discriminant Analysis Accuracy: 0.4139

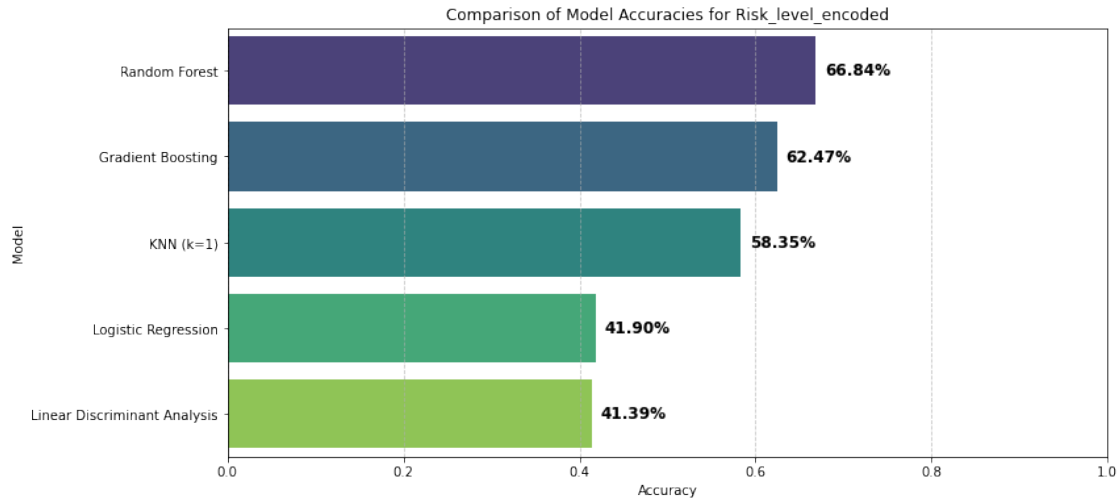
/Users/kamila/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.

warnings.warn(
/Users/kamila/opt/anaconda3/lib/python3.9/site-packages/sklearn/utils/optimize.py:319: ConvergenceWarning: newton-cg failed to converge at loss = 1.1444958118143944. Increase the number of iterations.

warnings.warn(

Logistic Regression Accuracy: 0.4190

Best KNN Accuracy (1 neighbors): 0.5835



```
[321]:
```

	Model	Accuracy
1	Random Forest	0.668380
0	Gradient Boosting	0.624679
4	KNN (k=1)	0.583548
3	Logistic Regression	0.419023
2	Linear Discriminant Analysis	0.413882

```
[322]: class_models(data_inp=credit_rating_outl, X_cols=X_cols, y_col =
↳ 'Risk_level_encoded',
        test_size=0.2, num_class = 4)
```

Gradient Boosting Accuracy: 0.6555

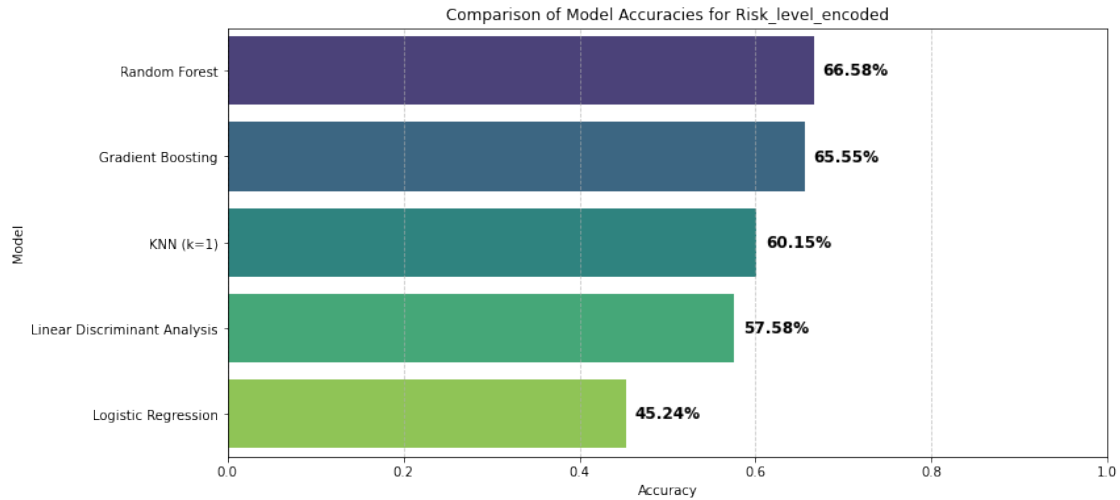
Random Forest Accuracy: 0.6658

Linear Discriminant Analysis Accuracy: 0.5758

/Users/kamila/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.
warnings.warn(

Logistic Regression Accuracy: 0.4524

Best KNN Accuracy (1 neighbors): 0.6015



```
[322]:
```

	Model	Accuracy
1	Random Forest	0.665810
0	Gradient Boosting	0.655527
4	KNN (k=1)	0.601542
2	Linear Discriminant Analysis	0.575835
3	Logistic Regression	0.452442

```
[323]: class_models_accur_confus_matrix(data_inp=credit_rating_outl, X_cols=X_cols,
→y_col = 'Risk_level_encoded',
      test_size=0.2, num_class = 4)
```

Gradient Boosting Accuracy: 0.6555

Gradient Boosting Classification Report:

	precision	recall	f1-score	support
1	0.65	0.66	0.66	95
2	0.59	0.64	0.61	129
3	0.76	0.70	0.73	153
4	0.27	0.25	0.26	12
accuracy			0.66	389
macro avg	0.57	0.56	0.56	389
weighted avg	0.66	0.66	0.66	389

Gradient Boosting Confusion Matrix:

```
[[ 63  27   5   0]
 [ 24  82  22   1]
 [ 10  29 107   7]
 [   0   2   7   3]]
```

Random Forest Accuracy: 0.6658

Random Forest Classification Report:

	precision	recall	f1-score	support
1	0.72	0.63	0.67	95
2	0.60	0.60	0.60	129
3	0.69	0.80	0.74	153
4	0.00	0.00	0.00	12
accuracy			0.67	389
macro avg	0.50	0.51	0.50	389
weighted avg	0.65	0.67	0.65	389

Random Forest Confusion Matrix:

```
[[ 60  26   9   0]
 [ 17  77  35   0]
 [   6  24 122   1]
 [   0   1  11   0]]
```

Linear Discriminant Analysis Accuracy: 0.5758

Linear Discriminant Analysis Classification Report:

	precision	recall	f1-score	support
1	0.58	0.40	0.47	95
2	0.49	0.64	0.55	129
3	0.66	0.65	0.66	153
4	0.67	0.33	0.44	12
accuracy			0.58	389
macro avg	0.60	0.51	0.53	389
weighted avg	0.59	0.58	0.57	389

Linear Discriminant Analysis Confusion Matrix:

```
[[ 38  47  10   0]
 [ 14  82  33   0]
 [ 13  38 100   2]
 [   0   0   8   4]]
```

Logistic Regression Accuracy: 0.4524

```
/Users/kamila/opt/anaconda3/lib/python3.9/site-
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will
always use 'multinomial'. Leave it to its default value to avoid this warning.
  warnings.warn(
```

Logistic Regression Classification Report:

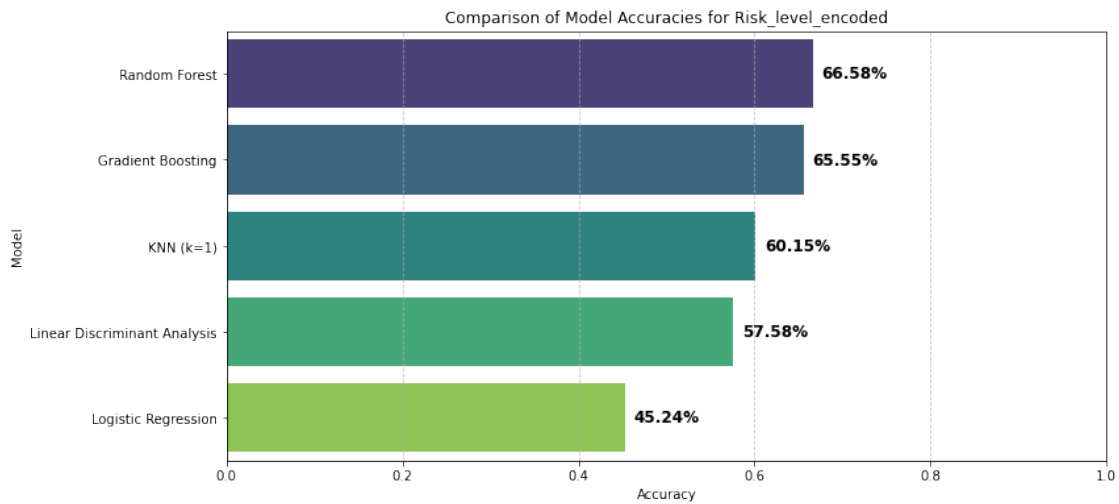
	precision	recall	f1-score	support
--	-----------	--------	----------	---------

1	0.44	0.55	0.49	95
2	0.50	0.50	0.50	129
3	0.67	0.35	0.46	153
4	0.11	0.58	0.18	12
accuracy			0.45	389
macro avg	0.43	0.49	0.41	389
weighted avg	0.54	0.45	0.47	389

Logistic Regression Confusion Matrix:

```
[[52 35  2  6]
 [37 64 19  9]
 [28 29 53 43]
 [ 0  0  5  7]]
```

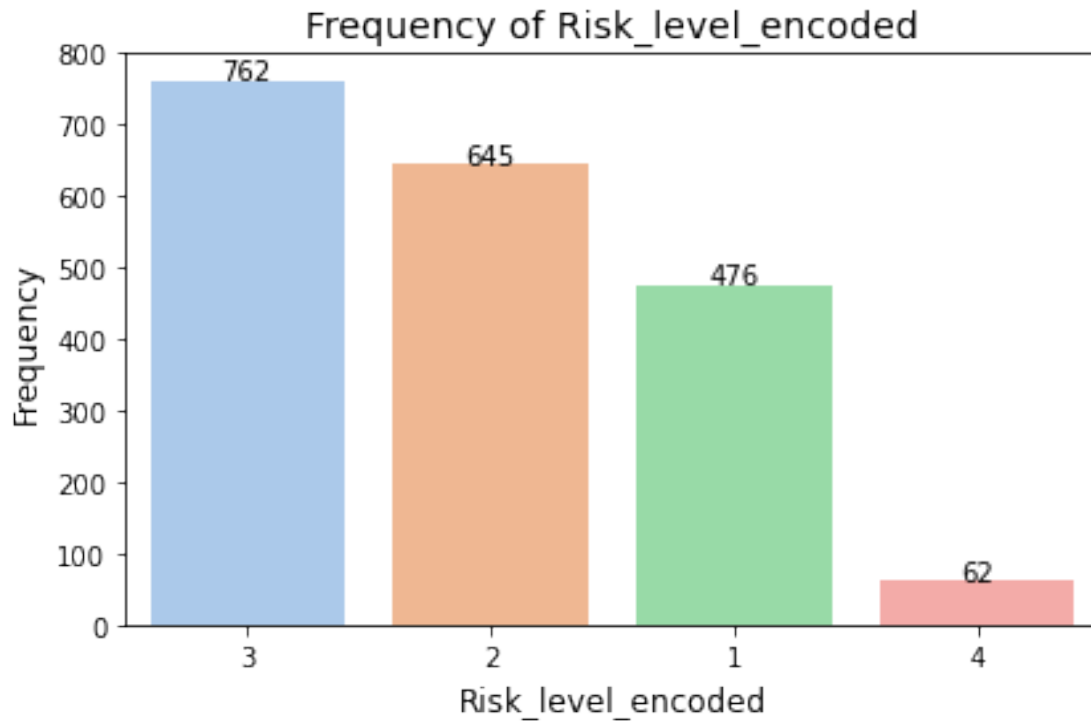
Best KNN Accuracy (1 neighbors): 0.6015



```
[323]:
```

	Model	Accuracy
1	Random Forest	0.665810
0	Gradient Boosting	0.655527
4	KNN (k=1)	0.601542
2	Linear Discriminant Analysis	0.575835
3	Logistic Regression	0.452442

```
[324]: freq_graph(credit_rating_out1, 'Risk_level_encoded',0)
```



```
[325]: class_models(data_inp=credit_rating_outl_scaled, X_cols=X_cols, y_col =  
↳ 'Risk_level_encoded', test_size=0.2, num_class = 4)
```

Gradient Boosting Accuracy: 0.6555

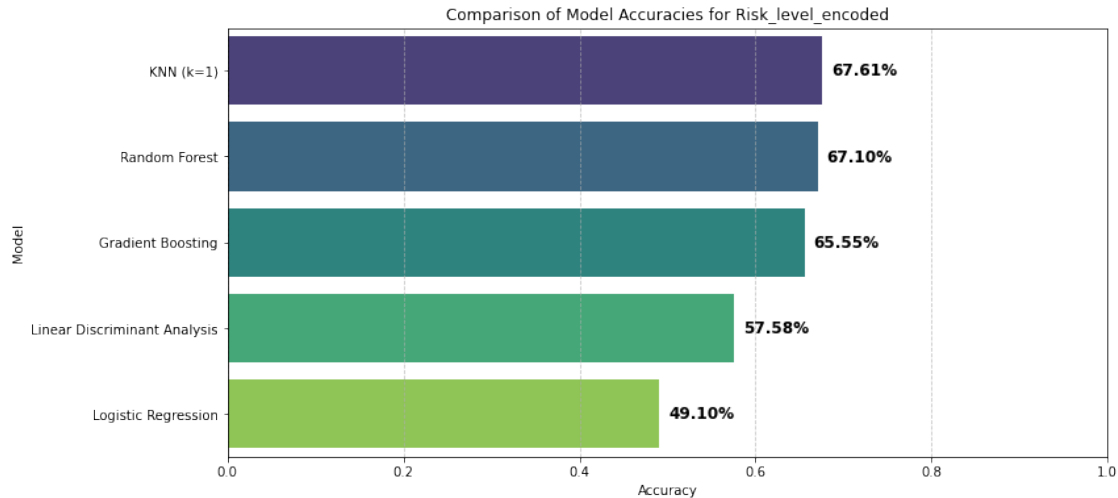
Random Forest Accuracy: 0.6710

Linear Discriminant Analysis Accuracy: 0.5758

Logistic Regression Accuracy: 0.4910

Best KNN Accuracy (1 neighbors): 0.6761

```
/Users/kamila/opt/anaconda3/lib/python3.9/site-  
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'  
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will  
always use 'multinomial'. Leave it to its default value to avoid this warning.  
warnings.warn(
```

```
[325]:
```

	Model	Accuracy
4	KNN (k=1)	0.676093
1	Random Forest	0.670951
0	Gradient Boosting	0.655527
2	Linear Discriminant Analysis	0.575835
3	Logistic Regression	0.491003

```
[326]: class_models_macro_avg(data_inp=credit_rating_new, X_cols=X_cols, y_col = ↳
↳ 'Risk_level_encoded', test_size=0.2, num_class = 4)
```

Gradient Boosting Macro-Average Accuracy: 0.5170

Random Forest Macro-Average Accuracy: 0.5257

Linear Discriminant Analysis Macro-Average Accuracy: 0.3160

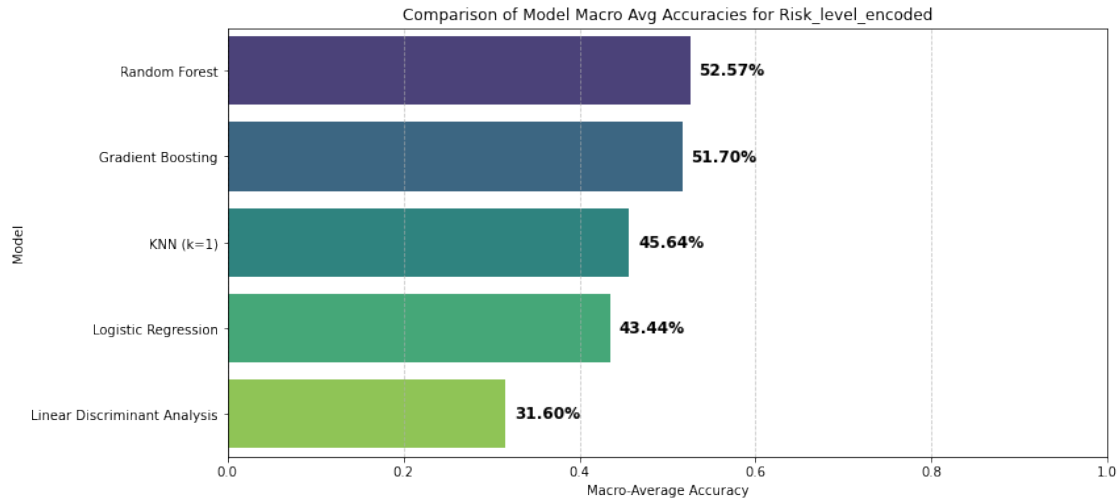
```
/Users/kamila/opt/anaconda3/lib/python3.9/site-  
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'  
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will  
always use 'multinomial'. Leave it to its default value to avoid this warning.
```

```
warnings.warn(
/Users/kamila/opt/anaconda3/lib/python3.9/site-
packages/sklearn/utils/optimize.py:319: ConvergenceWarning: newton-cg failed to
converge at loss = 1.1444958118143944. Increase the number of iterations.
```

```
warnings.warn(
```

Logistic Regression Macro-Average Accuracy: 0.4344

Best KNN Macro-Average Accuracy (1 neighbors): 0.4564



```
[326]:
```

	Model	Macro Avg Accuracy
1	Random Forest	0.525694
0	Gradient Boosting	0.517030
4	KNN (k=1)	0.456430
3	Logistic Regression	0.434399
2	Linear Discriminant Analysis	0.316036

```
[327]: class_models_macro_avg(data_inp=credit_rating_outl, X_cols=X_cols, y_col = 'Risk_level_encoded', test_size=0.2, num_class = 4)
```

Gradient Boosting Macro-Average Accuracy: 0.5620

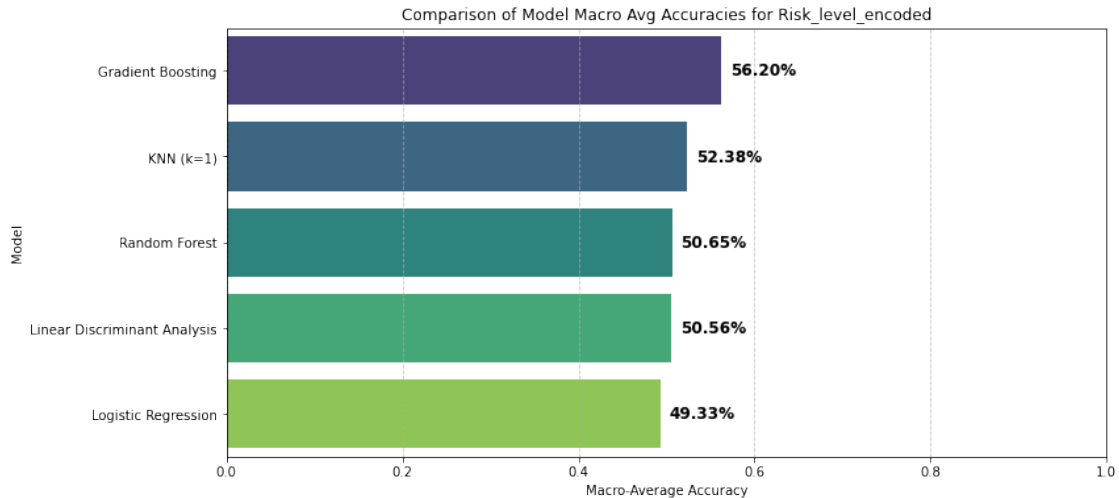
Random Forest Macro-Average Accuracy: 0.5065

Linear Discriminant Analysis Macro-Average Accuracy: 0.5056

/Users/kamila/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.
warnings.warn(

Logistic Regression Macro-Average Accuracy: 0.4933

Best KNN Macro-Average Accuracy (1 neighbors): 0.5238



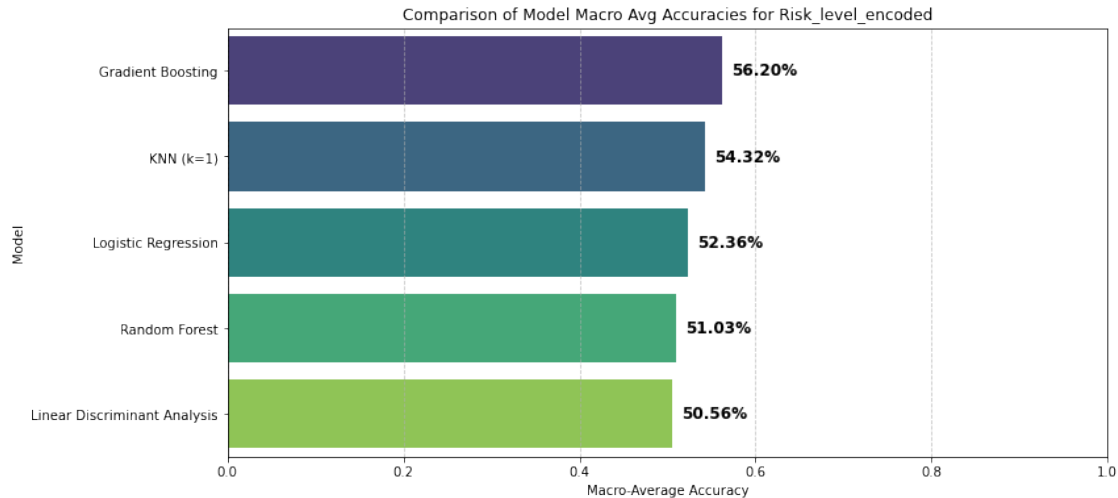
```
[327]:
```

	Model	Macro Avg Accuracy
0	Gradient Boosting	0.562041
4	KNN (k=1)	0.523775
1	Random Forest	0.506466
2	Linear Discriminant Analysis	0.505647
3	Logistic Regression	0.493308

```
[328]: class_models_macro_avg(data_inp=credit_rating_outl_scaled, X_cols=X_cols, y_col=
↳= 'Risk_level_encoded', test_size=0.2, num_class = 4)
```

```
Gradient Boosting Macro-Average Accuracy: 0.5620
Random Forest Macro-Average Accuracy: 0.5103
Linear Discriminant Analysis Macro-Average Accuracy: 0.5056
Logistic Regression Macro-Average Accuracy: 0.5236
Best KNN Macro-Average Accuracy (1 neighbors): 0.5432
```

```
/Users/kamila/opt/anaconda3/lib/python3.9/site-
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will
always use 'multinomial'. Leave it to its default value to avoid this warning.
warnings.warn(
```



[328]:	Model	Macro Avg Accuracy
0	Gradient Boosting	0.562041
4	KNN (k=1)	0.543175
3	Logistic Regression	0.523632
1	Random Forest	0.510342
2	Linear Discriminant Analysis	0.505647

```
[329]: class_models_macro_f1(data_inp=credit_rating_new, X_cols=X_cols, y_col = ↳
↳ 'Risk_level_encoded', test_size=0.2, num_class = 4)
```

Gradient Boosting Macro F1 Score: 0.5238

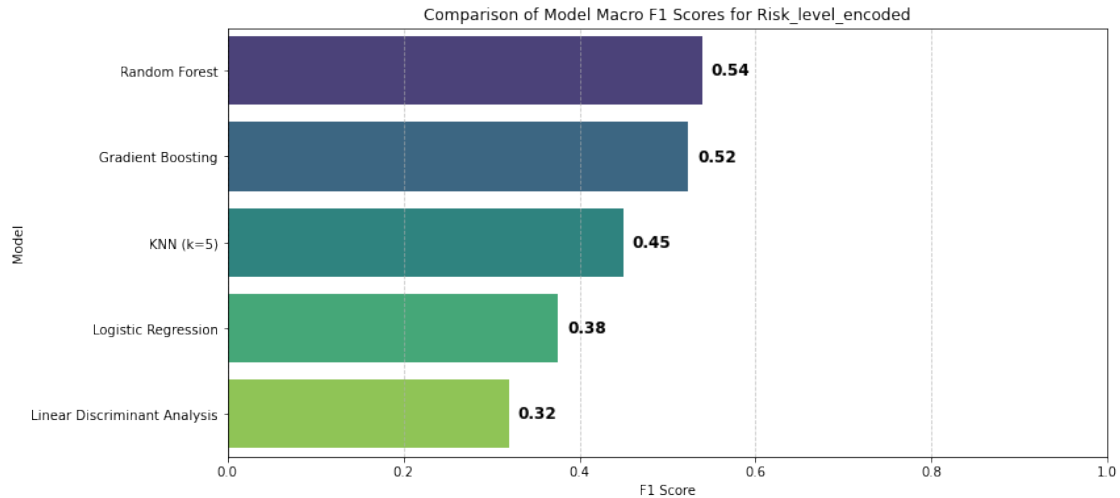
Random Forest Macro F1 Score: 0.5389

Linear Discriminant Analysis Macro F1 Score: 0.3196

```
/Users/kamila/opt/anaconda3/lib/python3.9/site-  
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'  
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will  
always use 'multinomial'. Leave it to its default value to avoid this warning.
```

```
warnings.warn(
/Users/kamila/opt/anaconda3/lib/python3.9/site-
packages/sklearn/utils/optimize.py:319: ConvergenceWarning: newton-cg failed to
converge at loss = 1.1444958118143944. Increase the number of iterations.
```

```
warnings.warn(
Logistic Regression Macro F1 Score: 0.3757
Best KNN Macro F1 Score (5 neighbors): 0.4493
```



```
[329]:
```

	Model	F1 Score
1	Random Forest	0.538918
0	Gradient Boosting	0.523812
4	KNN (k=5)	0.449292
3	Logistic Regression	0.375692
2	Linear Discriminant Analysis	0.319585

```
[330]: class_models_macro_f1(data_inp=credit_rating_out1, X_cols=X_cols, y_col =
↳ 'Risk_level_encoded', test_size=0.2, num_class = 4)
```

Gradient Boosting Macro F1 Score: 0.5637

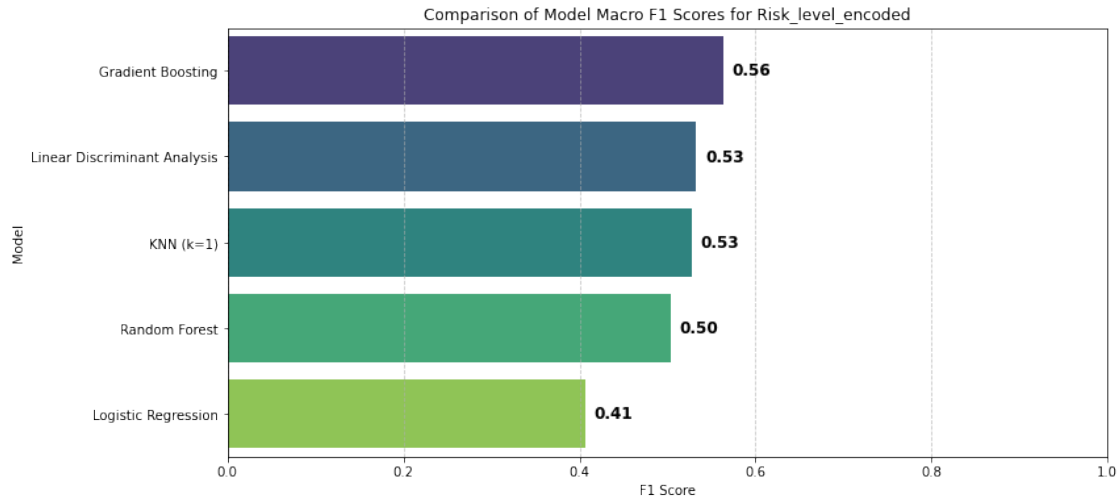
Random Forest Macro F1 Score: 0.5032

Linear Discriminant Analysis Macro F1 Score: 0.5328

/Users/kamila/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.
warnings.warn(

Logistic Regression Macro F1 Score: 0.4068

Best KNN Macro F1 Score (1 neighbors): 0.5277



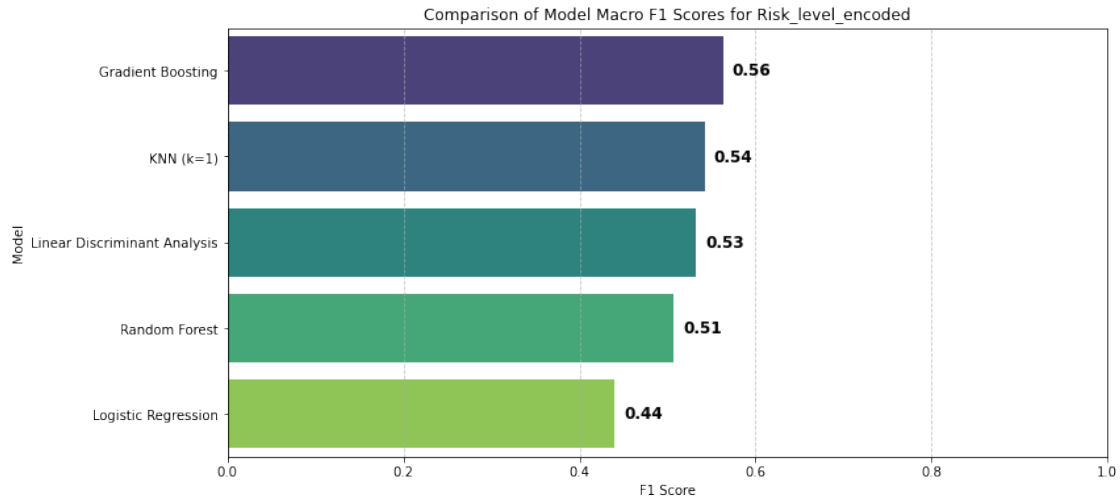
```
[330]:
```

	Model	F1 Score
0	Gradient Boosting	0.563669
2	Linear Discriminant Analysis	0.532848
4	KNN (k=1)	0.527727
1	Random Forest	0.503193
3	Logistic Regression	0.406834

```
[331]: class_models_macro_f1(data_inp=credit_rating_outl_scaled, X_cols=X_cols, y_col_
↳ 'Risk_level_encoded', test_size=0.2, num_class = 4)
```

```
Gradient Boosting Macro F1 Score: 0.5637
Random Forest Macro F1 Score: 0.5074
Linear Discriminant Analysis Macro F1 Score: 0.5328
Logistic Regression Macro F1 Score: 0.4400
Best KNN Macro F1 Score (1 neighbors): 0.5421
```

```
/Users/kamila/opt/anaconda3/lib/python3.9/site-
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will
always use 'multinomial'. Leave it to its default value to avoid this warning.
warnings.warn(
```



```
[331]:
```

	Model	F1 Score
0	Gradient Boosting	0.563669
4	KNN (k=1)	0.542134
2	Linear Discriminant Analysis	0.532848
1	Random Forest	0.507412
3	Logistic Regression	0.439982

```
[376]: # def compare_bar_charts(df1, df2, title, y_col, label1, label2, xlim,
        # color1='blue', color2='red'):
        # """
        #     Creates a horizontal bar chart comparing values from two datasets.

        #     Parameters:
        #     df1, df2: DataFrames with the same index (categories) and one column with
        #     numerical values.
        #     title: Title of the chart (default: 'Comparison of Scores')
        #     color1, color2: Colors for the bars (default: blue and red)
        #     """
        #     categories = df1[y_col]
        #     values1 = df1.iloc[:, 1].values # Extract first column as values
        #     values2 = df2.iloc[:, 1].values # Extract first column as values

        #     y_pos = np.arange(len(categories))

        #     plt.figure(figsize=(8, 6))

        #     # Plot bars for both datasets
        #     plt.barh(y_pos + 0.2, values2, height=0.4, color=color2, alpha=0.7,
        #             label=label2)
```

```

#     plt.barh(y_pos - 0.2, values1, height=0.4, color=color1, alpha=0.7,
→label=label1)

#     # Add labels
#     plt.yticks(y_pos, categories)
#     plt.xlabel('Score')
#     plt.title(title + ' for ' + y_col)

#     # Add value labels to bars
#     for i in range(len(categories)):
#         plt.text(values1[i] + 0.01, y_pos[i] - 0.2, f"{values1[i]:.2f}",
→va='center', fontsize=10, fontweight='bold', color='black')
#         plt.text(values2[i] + 0.01, y_pos[i] + 0.2, f"{values2[i]:.2f}",
→va='center', fontsize=10, fontweight='bold', color='black')

#     plt.xlim(0, xlim)
#     plt.legend()
#     plt.grid(axis='x', linestyle='--', alpha=0.7)
#     plt.show()

# Example usage:
# compare_bar_charts(df1, df2)

import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

def compare_bar_charts(df1, df2, title, y_col, label1, label2, xlim,
→color1='blue', color2='red'):
    """
    Creates a horizontal bar chart comparing values from two datasets, aligning
→them by the first column.

    Parameters:
    df1, df2: DataFrames with a common first column (e.g., 'Model'), which will
→be used for merging.
    title: Title of the chart.
    y_col: Column name being compared.
    label1, label2: Labels for the datasets.
    xlim: Maximum value for the x-axis.
    color1, color2: Colors for the bars (default: blue and red).
    """

```



```

# Clean 'Model' column to remove any text after 'KNN'
df1_cleaned = df1.copy()
df2_cleaned = df2.copy()

df1_cleaned['Model'] = df1_cleaned['Model'].replace(r'KNN.*', 'KNN',
→regex=True)
df2_cleaned['Model'] = df2_cleaned['Model'].replace(r'KNN.*', 'KNN',
→regex=True)

# Merge datasets on the cleaned 'Model' column
merged_df = pd.merge(df1_cleaned, df2_cleaned, on=df1.columns[0],
→suffixes=('_1', '_2'))

categories = merged_df.iloc[:, 0] # First column (e.g., 'Model')
values1 = merged_df.iloc[:, 1].values # Second column (values from df1)
values2 = merged_df.iloc[:, 2].values # Third column (values from df2)

y_pos = np.arange(len(categories))

plt.figure(figsize=(8, 6))

# Plot bars for both datasets
plt.barh(y_pos + 0.2, values2, height=0.4, color=color2, alpha=0.7,
→label=label2)
plt.barh(y_pos - 0.2, values1, height=0.4, color=color1, alpha=0.7,
→label=label1)

# Add labels
plt.yticks(y_pos, categories)
plt.xlabel('Macro Avg F1 Score')
plt.title(f"{title} for {y_col}")

# Add value labels to bars
for i in range(len(categories)):
    plt.text(values1[i] + 0.01, y_pos[i] - 0.2, f"{values1[i]:.2f}",
→va='center', fontsize=10, fontweight='bold', color='black')
    plt.text(values2[i] + 0.01, y_pos[i] + 0.2, f"{values2[i]:.2f}",
→va='center', fontsize=10, fontweight='bold', color='black')

plt.xlim(0, xlim)
plt.legend()
plt.grid(axis='x', linestyle='--', alpha=0.7)
plt.show()

```

```
[379]: data1 = class_models_macro_f1(data_inp=credit_rating_out1, X_cols=X_cols,
                                     y_col = 'Risk_level_encoded', test_size=0.2,
                                     ↪num_class = 4)

data2 = class_models_macro_f1(data_inp=credit_rating_out1_scaled, X_cols=X_cols,
                               y_col = 'Risk_level_encoded', test_size=0.2,
                               ↪num_class = 4)
```

Gradient Boosting Macro F1 Score: 0.5637

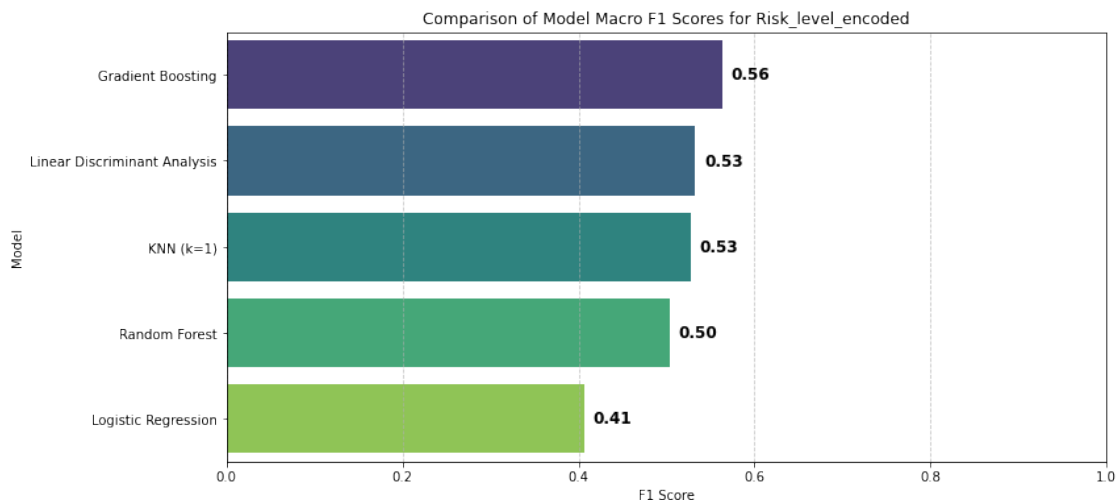
Random Forest Macro F1 Score: 0.5032

Linear Discriminant Analysis Macro F1 Score: 0.5328

/Users/kamila/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.
warnings.warn(

Logistic Regression Macro F1 Score: 0.4068

Best KNN Macro F1 Score (1 neighbors): 0.5277



Gradient Boosting Macro F1 Score: 0.5637

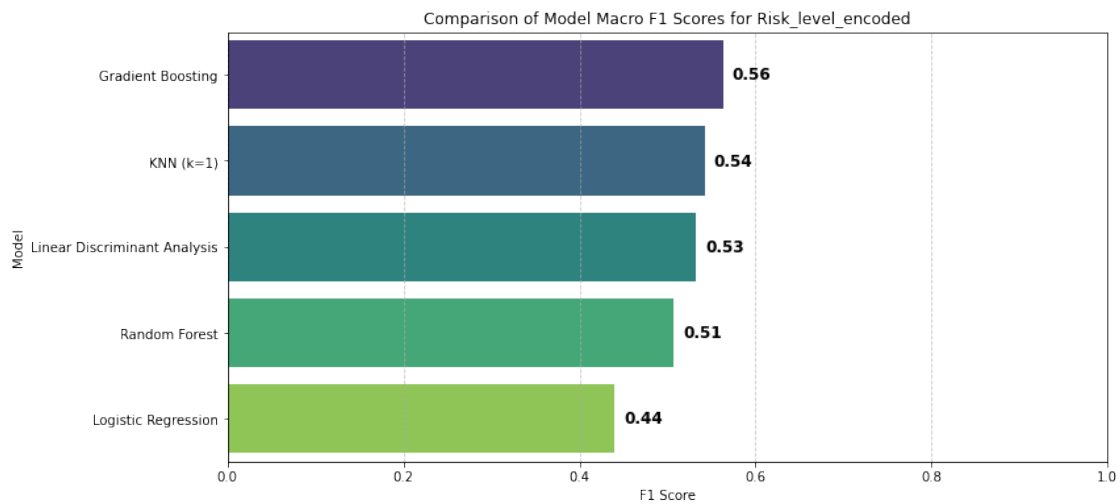
Random Forest Macro F1 Score: 0.5074

Linear Discriminant Analysis Macro F1 Score: 0.5328

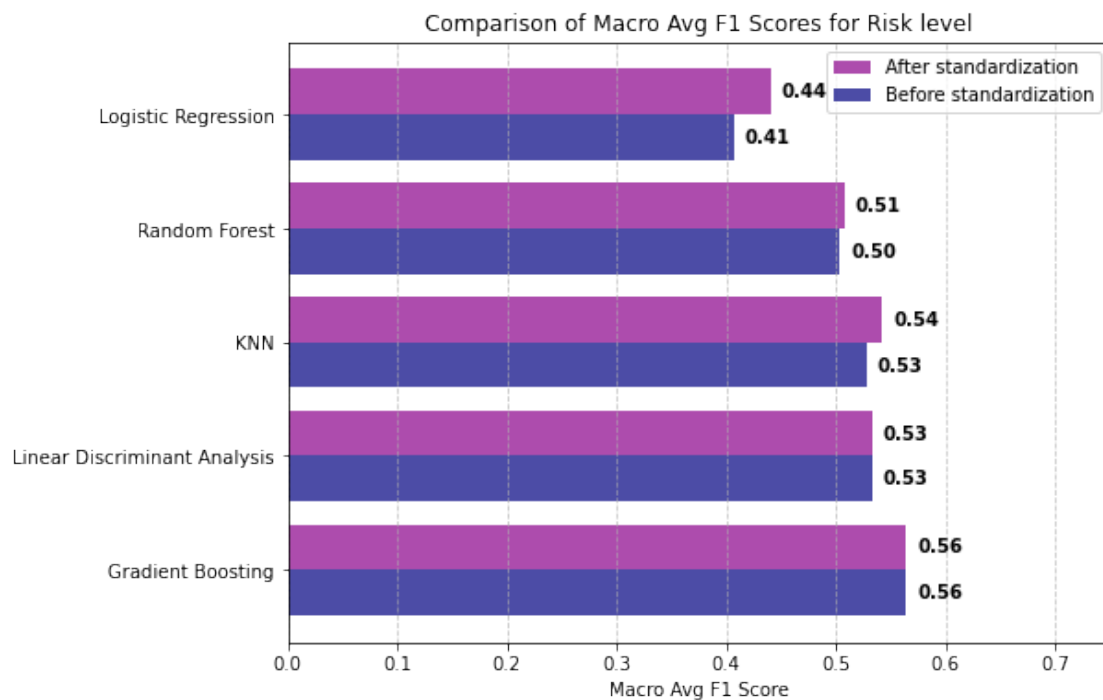
Logistic Regression Macro F1 Score: 0.4400

/Users/kamila/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.
warnings.warn(

Best KNN Macro F1 Score (1 neighbors): 0.5421



```
[380]: #df1, df2, title, y_col, label1, label2, color1='blue', color2='red'
compare_bar_charts(data1, data2, title='Comparison of Macro Avg F1 Scores',
                    y_col='Risk level',
                    label1='Before standardization', label2='After
                    standardization', xlim=0.75,
                    color1='navy', color2='darkmagenta')
```



```
[381]: data1 = class_models_macro_f1(data_inp=credit_rating_out1, X_cols=X_cols,
                                     y_col = 'Rating_encoded', test_size=0.2,
                                     ↪ num_class = 6)

data2 = class_models_macro_f1(data_inp=credit_rating_out1_scaled, X_cols=X_cols,
                               y_col = 'Rating_encoded', test_size=0.2,
                               ↪ num_class = 6)
```

Gradient Boosting Macro F1 Score: 0.4683

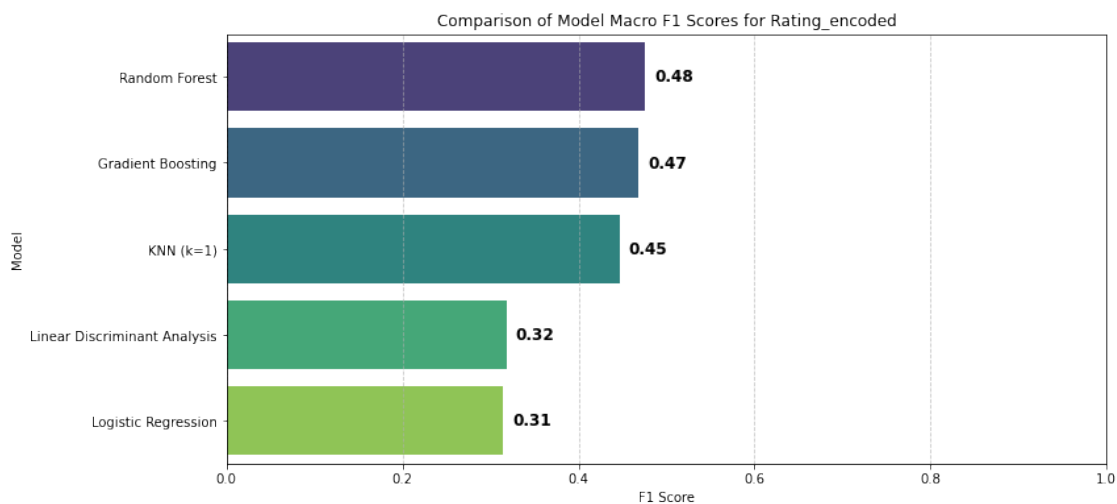
Random Forest Macro F1 Score: 0.4758

Linear Discriminant Analysis Macro F1 Score: 0.3188

/Users/kamila/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.
warnings.warn(

Logistic Regression Macro F1 Score: 0.3148

Best KNN Macro F1 Score (1 neighbors): 0.4466



Gradient Boosting Macro F1 Score: 0.4683

Random Forest Macro F1 Score: 0.4746

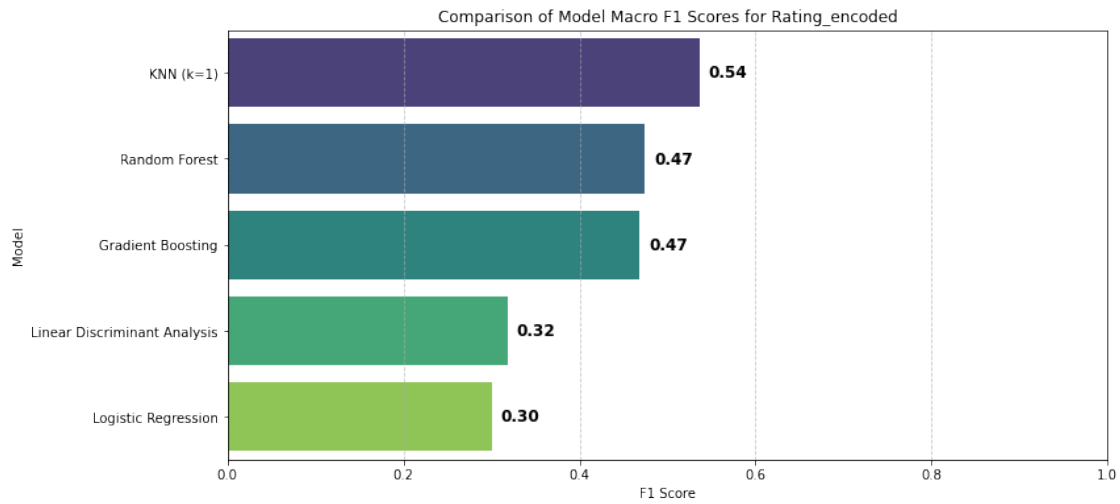
Linear Discriminant Analysis Macro F1 Score: 0.3188

Logistic Regression Macro F1 Score: 0.3006

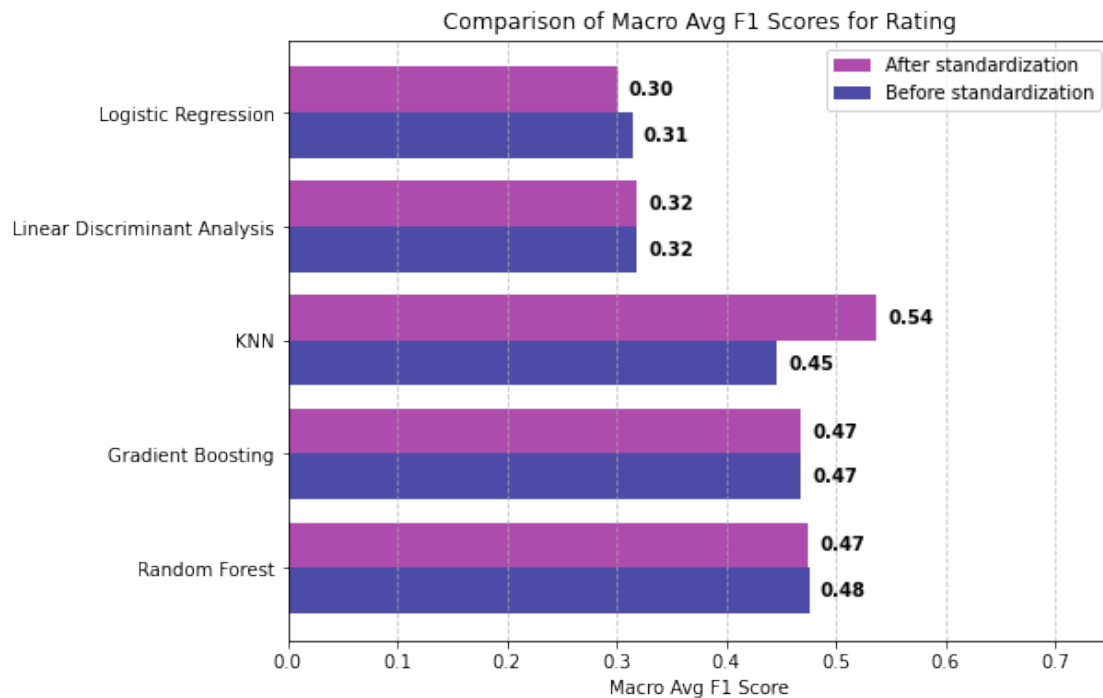
Best KNN Macro F1 Score (1 neighbors): 0.5368

/Users/kamila/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.7. From then on, it will

always use 'multinomial'. Leave it to its default value to avoid this warning.
warnings.warn(



```
[382]: #df1, df2, title, y_col, label1, label2, color1='blue', color2='red'
compare_bar_charts(data1, data2, title='Comparison of Macro Avg F1 Scores',
    y_col='Rating',
    label1='Before standardization', label2='After
    standardization', xlim=0.75,
    color1='navy', color2='darkmagenta')
```



5 Subset Selection

```
[420]: import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from statsmodels.stats.outliers_influence import variance_inflation_factor

def cor(data, cols):
    correlation_matrix = data[cols].corr()
    print(correlation_matrix)

    plt.figure(figsize=(14, 10))
    sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt='.2f')
    #plt.savefig("correlation_heatmap.png", dpi=300, bbox_inches='tight') # Save as PNG
    plt.show()

def vif(data, cols, save_path=None, font = 10):

    X = data[cols]
    vif_data = pd.DataFrame()
    vif_data["feature"] = X.columns
    vif_data["VIF"] = [round(variance_inflation_factor(X.values, i), 2) for i in range(len(X.columns))]
    vif_data = vif_data.sort_values(by='VIF', ascending=True).reset_index(drop=True)

    if save_path:

        fig, ax = plt.subplots(figsize=(6, len(cols) * 0.4))
        ax.axis('tight')
        ax.axis('off')
        table = ax.table(cellText=vif_data.values, colLabels=vif_data.columns, cellLoc='center', loc='center')
        table.auto_set_font_size(False)
        table.set_fontsize(font)
        plt.savefig(save_path, bbox_inches='tight', dpi=300) # Save as PNG
    return vif_data

[590]: cor(credit_rating_new, predictors)
```

```
# credit_rating_outl
# credit_rating_new
```

	currentRatio	quickRatio	cashRatio	\
currentRatio	1.000000	0.104169	0.736740	
quickRatio	0.104169	1.000000	0.125612	
cashRatio	0.736740	0.125612	1.000000	
daysOfSalesOutstanding	0.000179	0.760659	0.007757	
netProfitMargin	0.006295	-0.001157	-0.007174	
pretaxProfitMargin	-0.002540	-0.001566	-0.005627	
grossProfitMargin	0.068765	-0.028185	-0.049862	
operatingProfitMargin	-0.000651	-0.000959	0.004494	
returnOnAssets	0.001259	0.001181	0.002496	
returnOnCapitalEmployed	0.001242	0.001169	0.002513	
returnOnEquity	-0.001269	-0.001197	-0.002566	
assetTurnover	-0.001666	-0.001584	-0.003320	
fixedAssetTurnover	-0.001659	-0.001576	-0.003264	
debtEquityRatio	-0.000549	-0.001320	-0.003815	
debtRatio	0.006484	0.003707	-0.007211	
effectiveTaxRate	-0.026544	-0.004000	-0.024584	
freeCashFlowOperatingCashFlowRatio	0.005820	0.001708	-0.013977	
freeCashFlowPerShare	-0.001701	-0.001839	-0.004235	
cashPerShare	-0.001611	-0.000828	-0.003967	
companyEquityMultiplier	-0.000517	-0.001319	-0.003825	
ebitPerRevenue	-0.002594	-0.001616	-0.005699	
enterpriseValueMultiple	-0.002166	-0.003001	-0.008160	
operatingCashFlowPerShare	-0.001735	-0.001883	-0.004373	
operatingCashFlowSalesRatio	-0.001115	-0.000465	0.012164	
payablesTurnover	-0.002228	-0.001841	-0.006993	
	daysOfSalesOutstanding	netProfitMargin	\	
currentRatio	0.000179	0.006295		
quickRatio	0.760659	-0.001157		
cashRatio	0.007757	-0.007174		
daysOfSalesOutstanding	1.000000	0.124081		
netProfitMargin	0.124081	1.000000		
pretaxProfitMargin	0.142620	0.983997		
grossProfitMargin	-0.076092	-0.169054		
operatingProfitMargin	0.159409	0.950951		
returnOnAssets	0.001754	0.001161		
returnOnCapitalEmployed	0.001730	0.001126		
returnOnEquity	-0.001771	-0.001088		
assetTurnover	-0.002336	-0.001447		
fixedAssetTurnover	-0.002337	-0.001449		
debtEquityRatio	-0.001478	-0.002778		

debtRatio	0.004200	-0.021202
effectiveTaxRate	-0.001018	0.002241
freeCashFlowOperatingCashFlowRatio	0.004884	-0.000778
freeCashFlowPerShare	-0.001793	-0.000556
cashPerShare	0.000959	-0.000585
companyEquityMultiplier	-0.001495	-0.002783
ebitPerRevenue	0.142586	0.983879
enterpriseValueMultiple	-0.020206	-0.003604
operatingCashFlowPerShare	-0.001861	-0.000563
operatingCashFlowSalesRatio	0.352455	0.507362
payablesTurnover	-0.002946	-0.001315

	pretaxProfitMargin	grossProfitMargin \
currentRatio	-0.002540	0.068765
quickRatio	-0.001566	-0.028185
cashRatio	-0.005627	-0.049862
daysOfSalesOutstanding	0.142620	-0.076092
netProfitMargin	0.983997	-0.169054
pretaxProfitMargin	1.000000	-0.238116
grossProfitMargin	-0.238116	1.000000
operatingProfitMargin	0.986223	-0.239716
returnOnAssets	0.001266	-0.022966
returnOnCapitalEmployed	0.001236	-0.022718
returnOnEquity	-0.001208	0.023179
assetTurnover	-0.001604	0.030625
fixedAssetTurnover	-0.001606	0.030652
debtEquityRatio	-0.002931	0.000091
debtRatio	-0.025123	0.021135
effectiveTaxRate	-0.000106	0.019232
freeCashFlowOperatingCashFlowRatio	-0.001563	0.017506
freeCashFlowPerShare	-0.001128	-0.005020
cashPerShare	-0.001136	-0.004935
companyEquityMultiplier	-0.002931	0.000094
ebitPerRevenue	0.999941	-0.238210
enterpriseValueMultiple	-0.004071	-0.007408
operatingCashFlowPerShare	-0.001142	-0.004969
operatingCashFlowSalesRatio	0.581345	-0.198116
payablesTurnover	-0.001472	0.035066

	operatingProfitMargin	returnOnAssets \
currentRatio	-0.000651	0.001259
quickRatio	-0.000959	0.001181
cashRatio	0.004494	0.002496
daysOfSalesOutstanding	0.159409	0.001754
netProfitMargin	0.950951	0.001161
pretaxProfitMargin	0.986223	0.001266
grossProfitMargin	-0.239716	-0.022966
operatingProfitMargin	1.000000	0.001420

returnOnAssets	0.001420	1.000000
returnOnCapitalEmployed	0.001389	0.999920
returnOnEquity	-0.001373	-0.999920
assetTurnover	-0.001829	-0.749961
fixedAssetTurnover	-0.001832	-0.745933
debtEquityRatio	-0.002834	0.002827
debtRatio	-0.027598	-0.039559
effectiveTaxRate	0.000346	0.000927
freeCashFlowOperatingCashFlowRatio	-0.002086	0.011002
freeCashFlowPerShare	-0.001337	-0.004570
cashPerShare	-0.001339	0.000757
companyEquityMultiplier	-0.002831	0.003181
ebitPerRevenue	0.986253	0.001290
enterpriseValueMultiple	-0.004081	0.001926
operatingCashFlowPerShare	-0.001354	0.003299
operatingCashFlowSalesRatio	0.650407	0.002222
payablesTurnover	-0.002023	0.001230

	returnOnCapitalEmployed	returnOnEquity \
currentRatio	0.001242	-0.001269
quickRatio	0.001169	-0.001197
cashRatio	0.002513	-0.002566
daysOfSalesOutstanding	0.001730	-0.001771
netProfitMargin	0.001126	-0.001088
pretaxProfitMargin	0.001236	-0.001208
grossProfitMargin	-0.022718	0.023179
operatingProfitMargin	0.001389	-0.001373
returnOnAssets	0.999920	-0.999920
returnOnCapitalEmployed	1.000000	-0.999708
returnOnEquity	-0.999708	1.000000
assetTurnover	-0.741899	0.757875
fixedAssetTurnover	-0.737817	0.753903
debtEquityRatio	0.002790	-0.002882
debtRatio	-0.039387	0.040237
effectiveTaxRate	0.000906	-0.000919
freeCashFlowOperatingCashFlowRatio	0.011008	-0.011009
freeCashFlowPerShare	-0.004587	0.004477
cashPerShare	0.000744	-0.000823
companyEquityMultiplier	0.003139	-0.003239
ebitPerRevenue	0.001260	-0.001233
enterpriseValueMultiple	0.002214	-0.002256
operatingCashFlowPerShare	0.003287	-0.003396
operatingCashFlowSalesRatio	0.002190	-0.002243
payablesTurnover	0.001214	-0.001232

	assetTurnover	fixedAssetTurnover \
currentRatio	-0.001666	-0.001659
quickRatio	-0.001584	-0.001576

cashRatio	-0.003320	-0.003264
daysOfSalesOutstanding	-0.002336	-0.002337
netProfitMargin	-0.001447	-0.001449
pretaxProfitMargin	-0.001604	-0.001606
grossProfitMargin	0.030625	0.030652
operatingProfitMargin	-0.001829	-0.001832
returnOnAssets	-0.749961	-0.745933
returnOnCapitalEmployed	-0.741899	-0.737817
returnOnEquity	0.757875	0.753903
assetTurnover	1.000000	0.999981
fixedAssetTurnover	0.999981	1.000000
debtEquityRatio	-0.003935	-0.003938
debtRatio	0.053092	0.053062
effectiveTaxRate	-0.001222	-0.001223
freeCashFlowOperatingCashFlowRatio	-0.008497	-0.008447
freeCashFlowPerShare	0.002769	0.002744
cashPerShare	-0.001002	-0.001003
companyEquityMultiplier	-0.004405	-0.004408
ebitPerRevenue	-0.001636	-0.001638
enterpriseValueMultiple	-0.003116	-0.003121
operatingCashFlowPerShare	-0.002820	-0.002811
operatingCashFlowSalesRatio	-0.002931	-0.002936
payablesTurnover	-0.001630	-0.001632

	debtEquityRatio	debtRatio \
currentRatio	-0.000549	0.006484
quickRatio	-0.001320	0.003707
cashRatio	-0.003815	-0.007211
daysOfSalesOutstanding	-0.001478	0.004200
netProfitMargin	-0.002778	-0.021202
pretaxProfitMargin	-0.002931	-0.025123
grossProfitMargin	0.000091	0.021135
operatingProfitMargin	-0.002834	-0.027598
returnOnAssets	0.002827	-0.039559
returnOnCapitalEmployed	0.002790	-0.039387
returnOnEquity	-0.002882	0.040237
assetTurnover	-0.003935	0.053092
fixedAssetTurnover	-0.003938	0.053062
debtEquityRatio	1.000000	0.044153
debtRatio	0.044153	1.000000
effectiveTaxRate	0.003056	-0.000366
freeCashFlowOperatingCashFlowRatio	0.000873	-0.058359
freeCashFlowPerShare	-0.031590	0.040977
cashPerShare	-0.023416	0.037454
companyEquityMultiplier	0.999991	0.043791
ebitPerRevenue	-0.002949	-0.025168
enterpriseValueMultiple	0.013027	0.054001
operatingCashFlowPerShare	-0.035074	0.041962

operatingCashFlowSalesRatio	-0.003084	-0.021647
payablesTurnover	-0.001102	-0.004247

	effectiveTaxRate	\
currentRatio	-0.026544	
quickRatio	-0.004000	
cashRatio	-0.024584	
daysOfSalesOutstanding	-0.001018	
netProfitMargin	0.002241	
pretaxProfitMargin	-0.000106	
grossProfitMargin	0.019232	
operatingProfitMargin	0.000346	
returnOnAssets	0.000927	
returnOnCapitalEmployed	0.000906	
returnOnEquity	-0.000919	
assetTurnover	-0.001222	
fixedAssetTurnover	-0.001223	
debtEquityRatio	0.003056	
debtRatio	-0.000366	
effectiveTaxRate	1.000000	
freeCashFlowOperatingCashFlowRatio	0.005541	
freeCashFlowPerShare	-0.002657	
cashPerShare	-0.002714	
companyEquityMultiplier	0.003080	
ebitPerRevenue	-0.000001	
enterpriseValueMultiple	-0.004656	
operatingCashFlowPerShare	-0.002662	
operatingCashFlowSalesRatio	-0.001170	
payablesTurnover	-0.000266	

	freeCashFlowOperatingCashFlowRatio	\
currentRatio	0.005820	
quickRatio	0.001708	
cashRatio	-0.013977	
daysOfSalesOutstanding	0.004884	
netProfitMargin	-0.000778	
pretaxProfitMargin	-0.001563	
grossProfitMargin	0.017506	
operatingProfitMargin	-0.002086	
returnOnAssets	0.011002	
returnOnCapitalEmployed	0.011008	
returnOnEquity	-0.011009	
assetTurnover	-0.008497	
fixedAssetTurnover	-0.008447	
debtEquityRatio	0.000873	
debtRatio	-0.058359	
effectiveTaxRate	0.005541	
freeCashFlowOperatingCashFlowRatio	1.000000	

freeCashFlowPerShare	0.002162
cashPerShare	0.002430
companyEquityMultiplier	0.000870
ebitPerRevenue	-0.001443
enterpriseValueMultiple	-0.002964
operatingCashFlowPerShare	0.002236
operatingCashFlowSalesRatio	0.001574
payablesTurnover	-0.003618

	freeCashFlowPerShare	cashPerShare \
currentRatio	-0.001701	-0.001611
quickRatio	-0.001839	-0.000828
cashRatio	-0.004235	-0.003967
daysOfSalesOutstanding	-0.001793	0.000959
netProfitMargin	-0.000556	-0.000585
pretaxProfitMargin	-0.001128	-0.001136
grossProfitMargin	-0.005020	-0.004935
operatingProfitMargin	-0.001337	-0.001339
returnOnAssets	-0.004570	0.000757
returnOnCapitalEmployed	-0.004587	0.000744
returnOnEquity	0.004477	-0.000823
assetTurnover	0.002769	-0.001002
fixedAssetTurnover	0.002744	-0.001003
debtEquityRatio	-0.031590	-0.023416
debtRatio	0.040977	0.037454
effectiveTaxRate	-0.002657	-0.002714
freeCashFlowOperatingCashFlowRatio	0.002162	0.002430
freeCashFlowPerShare	1.000000	0.988936
cashPerShare	0.988936	1.000000
companyEquityMultiplier	-0.031695	-0.023489
ebitPerRevenue	-0.001160	-0.001168
enterpriseValueMultiple	-0.002910	-0.002930
operatingCashFlowPerShare	0.997661	0.976681
operatingCashFlowSalesRatio	-0.002381	-0.000592
payablesTurnover	-0.001207	-0.000809

	companyEquityMultiplier	ebitPerRevenue \
currentRatio	-0.000517	-0.002594
quickRatio	-0.001319	-0.001616
cashRatio	-0.003825	-0.005699
daysOfSalesOutstanding	-0.001495	0.142586
netProfitMargin	-0.002783	0.983879
pretaxProfitMargin	-0.002931	0.999941
grossProfitMargin	0.000094	-0.238210
operatingProfitMargin	-0.002831	0.986253
returnOnAssets	0.003181	0.001290
returnOnCapitalEmployed	0.003139	0.001260
returnOnEquity	-0.003239	-0.001233

assetTurnover	-0.004405	-0.001636
fixedAssetTurnover	-0.004408	-0.001638
debtEquityRatio	0.999991	-0.002949
debtRatio	0.043791	-0.025168
effectiveTaxRate	0.003080	-0.000001
freeCashFlowOperatingCashFlowRatio	0.000870	-0.001443
freeCashFlowPerShare	-0.031695	-0.001160
cashPerShare	-0.023489	-0.001168
companyEquityMultiplier	1.000000	-0.002947
ebitPerRevenue	-0.002947	1.000000
enterpriseValueMultiple	0.013004	-0.004161
operatingCashFlowPerShare	-0.035191	-0.001175
operatingCashFlowSalesRatio	-0.003137	0.581333
payablesTurnover	-0.001097	-0.001519

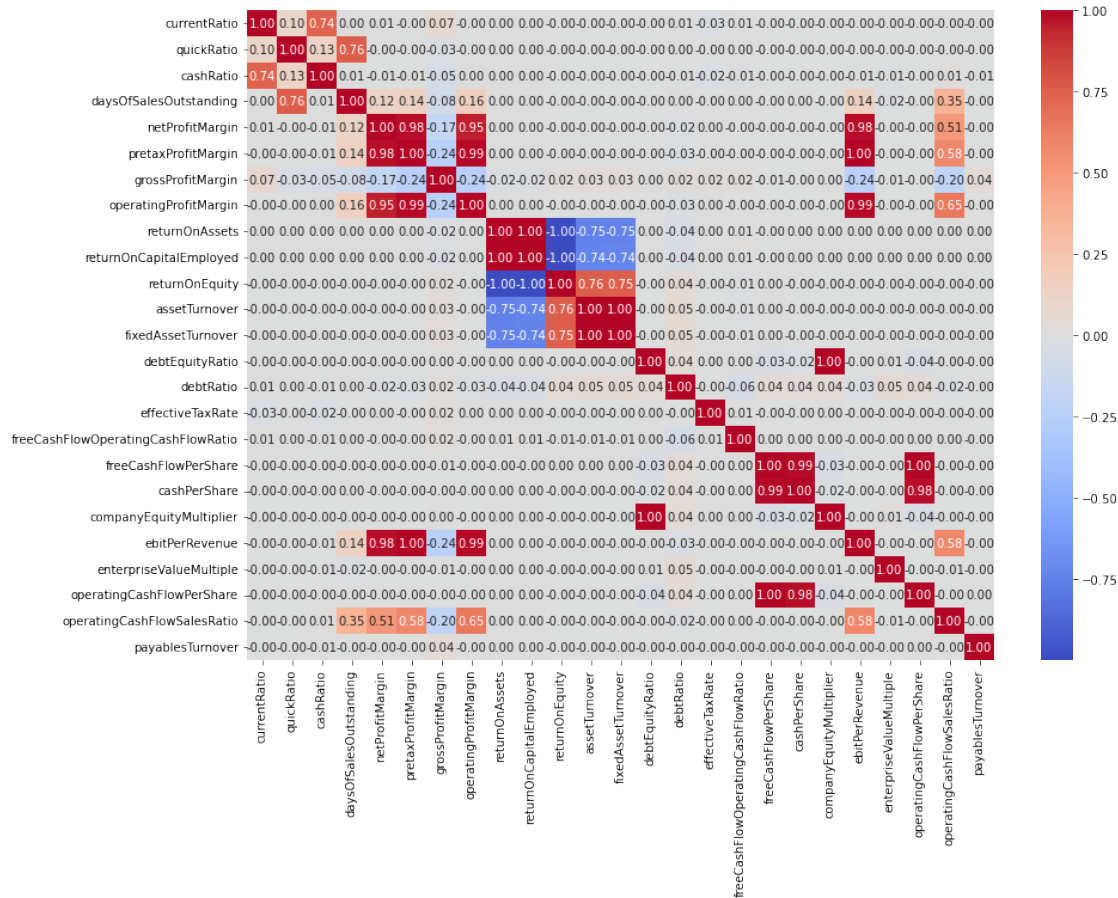
	enterpriseValueMultiple \
currentRatio	-0.002166
quickRatio	-0.003001
cashRatio	-0.008160
daysOfSalesOutstanding	-0.020206
netProfitMargin	-0.003604
pretaxProfitMargin	-0.004071
grossProfitMargin	-0.007408
operatingProfitMargin	-0.004081
returnOnAssets	0.001926
returnOnCapitalEmployed	0.002214
returnOnEquity	-0.002256
assetTurnover	-0.003116
fixedAssetTurnover	-0.003121
debtEquityRatio	0.013027
debtRatio	0.054001
effectiveTaxRate	-0.004656
freeCashFlowOperatingCashFlowRatio	-0.002964
freeCashFlowPerShare	-0.002910
cashPerShare	-0.002930
companyEquityMultiplier	0.013004
ebitPerRevenue	-0.004161
enterpriseValueMultiple	1.000000
operatingCashFlowPerShare	-0.002972
operatingCashFlowSalesRatio	-0.006137
payablesTurnover	-0.001319

	operatingCashFlowPerShare \
currentRatio	-0.001735
quickRatio	-0.001883
cashRatio	-0.004373
daysOfSalesOutstanding	-0.001861
netProfitMargin	-0.000563

pretaxProfitMargin	-0.001142
grossProfitMargin	-0.004969
operatingProfitMargin	-0.001354
returnOnAssets	0.003299
returnOnCapitalEmployed	0.003287
returnOnEquity	-0.003396
assetTurnover	-0.002820
fixedAssetTurnover	-0.002811
debtEquityRatio	-0.035074
debtRatio	0.041962
effectiveTaxRate	-0.002662
freeCashFlowOperatingCashFlowRatio	0.002236
freeCashFlowPerShare	0.997661
cashPerShare	0.976681
companyEquityMultiplier	-0.035191
ebitPerRevenue	-0.001175
enterpriseValueMultiple	-0.002972
operatingCashFlowPerShare	1.000000
operatingCashFlowSalesRatio	-0.002482
payablesTurnover	0.000847

	operatingCashFlowSalesRatio \
currentRatio	-0.001115
quickRatio	-0.000465
cashRatio	0.012164
daysOfSalesOutstanding	0.352455
netProfitMargin	0.507362
pretaxProfitMargin	0.581345
grossProfitMargin	-0.198116
operatingProfitMargin	0.650407
returnOnAssets	0.002222
returnOnCapitalEmployed	0.002190
returnOnEquity	-0.002243
assetTurnover	-0.002931
fixedAssetTurnover	-0.002936
debtEquityRatio	-0.003084
debtRatio	-0.021647
effectiveTaxRate	-0.001170
freeCashFlowOperatingCashFlowRatio	0.001574
freeCashFlowPerShare	-0.002381
cashPerShare	-0.000592
companyEquityMultiplier	-0.003137
ebitPerRevenue	0.581333
enterpriseValueMultiple	-0.006137
operatingCashFlowPerShare	-0.002482
operatingCashFlowSalesRatio	1.000000
payablesTurnover	-0.004092

	payablesTurnover
currentRatio	-0.002228
quickRatio	-0.001841
cashRatio	-0.006993
daysOfSalesOutstanding	-0.002946
netProfitMargin	-0.001315
pretaxProfitMargin	-0.001472
grossProfitMargin	0.035066
operatingProfitMargin	-0.002023
returnOnAssets	0.001230
returnOnCapitalEmployed	0.001214
returnOnEquity	-0.001232
assetTurnover	-0.001630
fixedAssetTurnover	-0.001632
debtEquityRatio	-0.001102
debtRatio	-0.004247
effectiveTaxRate	-0.000266
freeCashFlowOperatingCashFlowRatio	-0.003618
freeCashFlowPerShare	-0.001207
cashPerShare	-0.000809
companyEquityMultiplier	-0.001097
ebitPerRevenue	-0.001519
enterpriseValueMultiple	-0.001319
operatingCashFlowPerShare	0.000847
operatingCashFlowSalesRatio	-0.004092
payablesTurnover	1.000000



```
[591]: vif(credit_rating_new, predictors)
```

	feature	VIF
0	currentRatio	2.267300
1	quickRatio	3.236023
2	cashRatio	2.362426
3	daysOfSalesOutstanding	3.729159
4	netProfitMargin	73.120973
5	pretaxProfitMargin	8663.512061
6	grossProfitMargin	2.545193
7	operatingProfitMargin	88.660961
8	returnOnAssets	75147.467237
9	returnOnCapitalEmployed	877710.202114
10	returnOnEquity	868054.057457
11	assetTurnover	501663.486240
12	fixedAssetTurnover	493771.840008
13	debtEquityRatio	27773.689095
14	debtRatio	6.371824
15	effectiveTaxRate	1.003133

16	freeCashFlowOperatingCashFlowRatio	1.012810
17	freeCashFlowPerShare	30481.370280
18	cashPerShare	3046.886403
19	companyEquityMultiplier	27829.818788
20	ebitPerRevenue	8505.510191
21	enterpriseValueMultiple	1.021144
22	operatingCashFlowPerShare	14536.381778
23	operatingCashFlowSalesRatio	3.190283
24	payablesTurnover	1.075601

```
[592]: cor(credit_rating_outl_scaled, predictors)
```

	currentRatio	quickRatio	cashRatio	\
currentRatio	1.000000	0.744222	0.701051	
quickRatio	0.744222	1.000000	0.683437	
cashRatio	0.701051	0.683437	1.000000	
daysOfSalesOutstanding	0.069528	0.235014	0.022535	
netProfitMargin	-0.007933	0.071742	0.058272	
pretaxProfitMargin	-0.036303	0.048278	0.011482	
grossProfitMargin	-0.219900	-0.029425	-0.066873	
operatingProfitMargin	-0.041878	0.071856	0.041486	
returnOnAssets	0.039200	0.038224	0.043907	
returnOnCapitalEmployed	-0.033028	-0.032124	-0.024113	
returnOnEquity	-0.084788	-0.041947	-0.040515	
assetTurnover	0.021907	-0.138763	-0.132404	
fixedAssetTurnover	0.062071	0.044912	0.017586	
debtEquityRatio	-0.214413	-0.118206	-0.163952	
debtRatio	-0.282656	-0.183789	-0.201487	
effectiveTaxRate	-0.052083	-0.034838	-0.080134	
freeCashFlowOperatingCashFlowRatio	0.164350	0.153927	0.139856	
freeCashFlowPerShare	0.011959	0.029721	-0.018395	
cashPerShare	0.248653	0.394053	0.274601	
companyEquityMultiplier	-0.213520	-0.113647	-0.162281	
ebitPerRevenue	-0.049665	0.041701	0.007996	
enterpriseValueMultiple	0.003440	0.028574	-0.004555	
operatingCashFlowPerShare	-0.166358	-0.107110	-0.141165	
operatingCashFlowSalesRatio	-0.147103	0.038236	0.048369	
payablesTurnover	0.241996	0.130883	0.110186	

	daysOfSalesOutstanding	netProfitMargin	\
currentRatio	0.069528	-0.007933	
quickRatio	0.235014	0.071742	
cashRatio	0.022535	0.058272	
daysOfSalesOutstanding	1.000000	0.039782	
netProfitMargin	0.039782	1.000000	
pretaxProfitMargin	0.025871	0.898153	
grossProfitMargin	0.034664	0.225224	
operatingProfitMargin	0.067209	0.868786	

returnOnAssets	-0.091511	0.755582
returnOnCapitalEmployed	-0.104176	0.585488
returnOnEquity	-0.070288	0.525543
assetTurnover	-0.286222	-0.225795
fixedAssetTurnover	-0.000213	-0.034598
debtEquityRatio	-0.013111	-0.094078
debtRatio	-0.041900	-0.168280
effectiveTaxRate	0.027738	0.078650
freeCashFlowOperatingCashFlowRatio	0.087399	0.142584
freeCashFlowPerShare	-0.003424	0.212631
cashPerShare	-0.044628	0.011755
companyEquityMultiplier	-0.004273	-0.095925
ebitPerRevenue	0.056672	0.884276
enterpriseValueMultiple	0.142073	0.109897
operatingCashFlowPerShare	-0.096817	0.207209
operatingCashFlowSalesRatio	0.140246	0.432589
payablesTurnover	-0.169389	-0.096028

	pretaxProfitMargin	grossProfitMargin \
currentRatio	-0.036303	-0.219900
quickRatio	0.048278	-0.029425
cashRatio	0.011482	-0.066873
daysOfSalesOutstanding	0.025871	0.034664
netProfitMargin	0.898153	0.225224
pretaxProfitMargin	1.000000	0.226039
grossProfitMargin	0.226039	1.000000
operatingProfitMargin	0.900086	0.299462
returnOnAssets	0.701867	-0.004122
returnOnCapitalEmployed	0.662520	-0.053749
returnOnEquity	0.481483	-0.020315
assetTurnover	-0.197486	-0.451027
fixedAssetTurnover	-0.038207	-0.142053
debtEquityRatio	-0.085239	0.003641
debtRatio	-0.142243	0.048218
effectiveTaxRate	0.128690	0.063061
freeCashFlowOperatingCashFlowRatio	0.118873	0.017484
freeCashFlowPerShare	0.249365	0.017095
cashPerShare	0.002174	-0.024191
companyEquityMultiplier	-0.085156	0.003464
ebitPerRevenue	0.984009	0.216260
enterpriseValueMultiple	0.071846	0.074222
operatingCashFlowPerShare	0.254659	0.128265
operatingCashFlowSalesRatio	0.447026	0.432852
payablesTurnover	-0.096216	-0.541091

	operatingProfitMargin	returnOnAssets \
currentRatio	-0.041878	0.039200
quickRatio	0.071856	0.038224

cashRatio	0.041486	0.043907
daysOfSalesOutstanding	0.067209	-0.091511
netProfitMargin	0.868786	0.755582
pretaxProfitMargin	0.900086	0.701867
grossProfitMargin	0.299462	-0.004122
operatingProfitMargin	1.000000	0.617132
returnOnAssets	0.617132	1.000000
returnOnCapitalEmployed	0.543895	0.887018
returnOnEquity	0.420494	0.680274
assetTurnover	-0.293889	0.241855
fixedAssetTurnover	-0.079582	0.170188
debtEquityRatio	-0.034722	-0.097793
debtRatio	-0.088765	-0.124251
effectiveTaxRate	0.155647	0.131880
freeCashFlowOperatingCashFlowRatio	0.113644	0.229095
freeCashFlowPerShare	0.190910	0.335259
cashPerShare	-0.031033	0.086414
companyEquityMultiplier	-0.031203	-0.102530
ebitPerRevenue	0.902936	0.681974
enterpriseValueMultiple	0.104629	0.055121
operatingCashFlowPerShare	0.188598	0.238936
operatingCashFlowSalesRatio	0.494354	0.085074
payablesTurnover	-0.124597	0.138222

	returnOnCapitalEmployed	returnOnEquity \
currentRatio	-0.033028	-0.084788
quickRatio	-0.032124	-0.041947
cashRatio	-0.024113	-0.040515
daysOfSalesOutstanding	-0.104176	-0.070288
netProfitMargin	0.585488	0.525543
pretaxProfitMargin	0.662520	0.481483
grossProfitMargin	-0.053749	-0.020315
operatingProfitMargin	0.543895	0.420494
returnOnAssets	0.887018	0.680274
returnOnCapitalEmployed	1.000000	0.616245
returnOnEquity	0.616245	1.000000
assetTurnover	0.359703	0.205379
fixedAssetTurnover	0.235372	0.150816
debtEquityRatio	-0.058467	0.245354
debtRatio	-0.059022	0.063213
effectiveTaxRate	0.184837	0.059564
freeCashFlowOperatingCashFlowRatio	0.222603	0.155077
freeCashFlowPerShare	0.364512	0.242287
cashPerShare	0.105993	0.076421
companyEquityMultiplier	-0.063422	0.240397
ebitPerRevenue	0.659554	0.463969
enterpriseValueMultiple	0.018585	-0.000555
operatingCashFlowPerShare	0.260795	0.171138

operatingCashFlowSalesRatio	0.018203	-0.021566
payablesTurnover	0.149818	0.082724

	assetTurnover	fixedAssetTurnover \
currentRatio	0.021907	0.062071
quickRatio	-0.138763	0.044912
cashRatio	-0.132404	0.017586
daysOfSalesOutstanding	-0.286222	-0.000213
netProfitMargin	-0.225795	-0.034598
pretaxProfitMargin	-0.197486	-0.038207
grossProfitMargin	-0.451027	-0.142053
operatingProfitMargin	-0.293889	-0.079582
returnOnAssets	0.241855	0.170188
returnOnCapitalEmployed	0.359703	0.235372
returnOnEquity	0.205379	0.150816
assetTurnover	1.000000	0.442086
fixedAssetTurnover	0.442086	1.000000
debtEquityRatio	0.002146	0.027987
debtRatio	0.105760	0.101206
effectiveTaxRate	0.040895	0.019901
freeCashFlowOperatingCashFlowRatio	0.084412	0.360065
freeCashFlowPerShare	0.154155	0.189949
cashPerShare	0.081337	0.163465
companyEquityMultiplier	-0.004379	0.027088
ebitPerRevenue	-0.217376	-0.059737
enterpriseValueMultiple	-0.109994	0.055767
operatingCashFlowPerShare	0.013965	-0.111880
operatingCashFlowSalesRatio	-0.582522	-0.368431
payablesTurnover	0.413496	0.273091

	debtEquityRatio	debtRatio \
currentRatio	-0.214413	-0.282656
quickRatio	-0.118206	-0.183789
cashRatio	-0.163952	-0.201487
daysOfSalesOutstanding	-0.013111	-0.041900
netProfitMargin	-0.094078	-0.168280
pretaxProfitMargin	-0.085239	-0.142243
grossProfitMargin	0.003641	0.048218
operatingProfitMargin	-0.034722	-0.088765
returnOnAssets	-0.097793	-0.124251
returnOnCapitalEmployed	-0.058467	-0.059022
returnOnEquity	0.245354	0.063213
assetTurnover	0.002146	0.105760
fixedAssetTurnover	0.027987	0.101206
debtEquityRatio	1.000000	0.453393
debtRatio	0.453393	1.000000
effectiveTaxRate	-0.040646	-0.036203
freeCashFlowOperatingCashFlowRatio	-0.037066	-0.054729

freeCashFlowPerShare	-0.051063	-0.099855
cashPerShare	0.017483	0.015853
companyEquityMultiplier	0.994576	0.433926
ebitPerRevenue	-0.085753	-0.150097
enterpriseValueMultiple	0.019386	0.014406
operatingCashFlowPerShare	0.002887	-0.082871
operatingCashFlowSalesRatio	-0.103118	-0.179817
payablesTurnover	-0.068000	-0.102072

	effectiveTaxRate	\
currentRatio	-0.052083	
quickRatio	-0.034838	
cashRatio	-0.080134	
daysOfSalesOutstanding	0.027738	
netProfitMargin	0.078650	
pretaxProfitMargin	0.128690	
grossProfitMargin	0.063061	
operatingProfitMargin	0.155647	
returnOnAssets	0.131880	
returnOnCapitalEmployed	0.184837	
returnOnEquity	0.059564	
assetTurnover	0.040895	
fixedAssetTurnover	0.019901	
debtEquityRatio	-0.040646	
debtRatio	-0.036203	
effectiveTaxRate	1.000000	
freeCashFlowOperatingCashFlowRatio	-0.010237	
freeCashFlowPerShare	0.106856	
cashPerShare	-0.006112	
companyEquityMultiplier	-0.040219	
ebitPerRevenue	0.133130	
enterpriseValueMultiple	-0.087641	
operatingCashFlowPerShare	0.159807	
operatingCashFlowSalesRatio	0.076522	
payablesTurnover	0.033023	

	freeCashFlowOperatingCashFlowRatio	\
currentRatio	0.164350	
quickRatio	0.153927	
cashRatio	0.139856	
daysOfSalesOutstanding	0.087399	
netProfitMargin	0.142584	
pretaxProfitMargin	0.118873	
grossProfitMargin	0.017484	
operatingProfitMargin	0.113644	
returnOnAssets	0.229095	
returnOnCapitalEmployed	0.222603	
returnOnEquity	0.155077	

assetTurnover	0.084412
fixedAssetTurnover	0.360065
debtEquityRatio	-0.037066
debtRatio	-0.054729
effectiveTaxRate	-0.010237
freeCashFlowOperatingCashFlowRatio	1.000000
freeCashFlowPerShare	0.514967
cashPerShare	0.109546
companyEquityMultiplier	-0.035663
ebitPerRevenue	0.113496
enterpriseValueMultiple	0.005655
operatingCashFlowPerShare	-0.044821
operatingCashFlowSalesRatio	-0.030186
payablesTurnover	0.105935

	freeCashFlowPerShare	cashPerShare \
currentRatio	0.011959	0.248653
quickRatio	0.029721	0.394053
cashRatio	-0.018395	0.274601
daysOfSalesOutstanding	-0.003424	-0.044628
netProfitMargin	0.212631	0.011755
pretaxProfitMargin	0.249365	0.002174
grossProfitMargin	0.017095	-0.024191
operatingProfitMargin	0.190910	-0.031033
returnOnAssets	0.335259	0.086414
returnOnCapitalEmployed	0.364512	0.105993
returnOnEquity	0.242287	0.076421
assetTurnover	0.154155	0.081337
fixedAssetTurnover	0.189949	0.163465
debtEquityRatio	-0.051063	0.017483
debtRatio	-0.099855	0.015853
effectiveTaxRate	0.106856	-0.006112
freeCashFlowOperatingCashFlowRatio	0.514967	0.109546
freeCashFlowPerShare	1.000000	0.300591
cashPerShare	0.300591	1.000000
companyEquityMultiplier	-0.052820	0.012489
ebitPerRevenue	0.239990	-0.003805
enterpriseValueMultiple	-0.103399	-0.192484
operatingCashFlowPerShare	0.624753	0.407656
operatingCashFlowSalesRatio	0.180502	-0.124759
payablesTurnover	0.043334	0.012914

	companyEquityMultiplier	ebitPerRevenue \
currentRatio	-0.213520	-0.049665
quickRatio	-0.113647	0.041701
cashRatio	-0.162281	0.007996
daysOfSalesOutstanding	-0.004273	0.056672
netProfitMargin	-0.095925	0.884276

pretaxProfitMargin	-0.085156	0.984009
grossProfitMargin	0.003464	0.216260
operatingProfitMargin	-0.031203	0.902936
returnOnAssets	-0.102530	0.681974
returnOnCapitalEmployed	-0.063422	0.659554
returnOnEquity	0.240397	0.463969
assetTurnover	-0.004379	-0.217376
fixedAssetTurnover	0.027088	-0.059737
debtEquityRatio	0.994576	-0.085753
debtRatio	0.433926	-0.150097
effectiveTaxRate	-0.040219	0.133130
freeCashFlowOperatingCashFlowRatio	-0.035663	0.113496
freeCashFlowPerShare	-0.052820	0.239990
cashPerShare	0.012489	-0.003805
companyEquityMultiplier	1.000000	-0.082366
ebitPerRevenue	-0.082366	1.000000
enterpriseValueMultiple	0.020992	0.073494
operatingCashFlowPerShare	-0.000698	0.246873
operatingCashFlowSalesRatio	-0.099772	0.457347
payablesTurnover	-0.066155	-0.110682

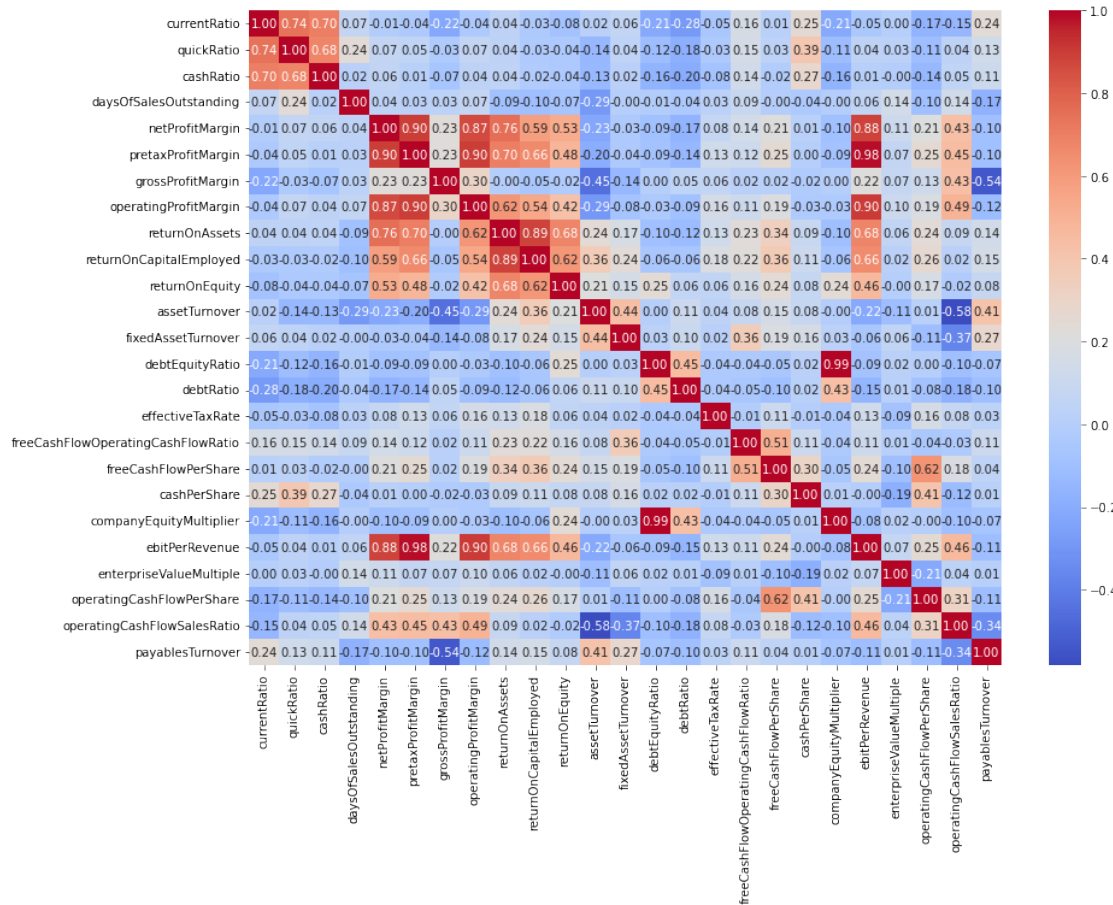
	enterpriseValueMultiple	\
currentRatio	0.003440	
quickRatio	0.028574	
cashRatio	-0.004555	
daysOfSalesOutstanding	0.142073	
netProfitMargin	0.109897	
pretaxProfitMargin	0.071846	
grossProfitMargin	0.074222	
operatingProfitMargin	0.104629	
returnOnAssets	0.055121	
returnOnCapitalEmployed	0.018585	
returnOnEquity	-0.000555	
assetTurnover	-0.109994	
fixedAssetTurnover	0.055767	
debtEquityRatio	0.019386	
debtRatio	0.014406	
effectiveTaxRate	-0.087641	
freeCashFlowOperatingCashFlowRatio	0.005655	
freeCashFlowPerShare	-0.103399	
cashPerShare	-0.192484	
companyEquityMultiplier	0.020992	
ebitPerRevenue	0.073494	
enterpriseValueMultiple	1.000000	
operatingCashFlowPerShare	-0.214936	
operatingCashFlowSalesRatio	0.035597	
payablesTurnover	0.007727	

	operatingCashFlowPerShare \
currentRatio	-0.166358
quickRatio	-0.107110
cashRatio	-0.141165
daysOfSalesOutstanding	-0.096817
netProfitMargin	0.207209
pretaxProfitMargin	0.254659
grossProfitMargin	0.128265
operatingProfitMargin	0.188598
returnOnAssets	0.238936
returnOnCapitalEmployed	0.260795
returnOnEquity	0.171138
assetTurnover	0.013965
fixedAssetTurnover	-0.111880
debtEquityRatio	0.002887
debtRatio	-0.082871
effectiveTaxRate	0.159807
freeCashFlowOperatingCashFlowRatio	-0.044821
freeCashFlowPerShare	0.624753
cashPerShare	0.407656
companyEquityMultiplier	-0.000698
ebitPerRevenue	0.246873
enterpriseValueMultiple	-0.214936
operatingCashFlowPerShare	1.000000
operatingCashFlowSalesRatio	0.309117
payablesTurnover	-0.108446

	operatingCashFlowSalesRatio \
currentRatio	-0.147103
quickRatio	0.038236
cashRatio	0.048369
daysOfSalesOutstanding	0.140246
netProfitMargin	0.432589
pretaxProfitMargin	0.447026
grossProfitMargin	0.432852
operatingProfitMargin	0.494354
returnOnAssets	0.085074
returnOnCapitalEmployed	0.018203
returnOnEquity	-0.021566
assetTurnover	-0.582522
fixedAssetTurnover	-0.368431
debtEquityRatio	-0.103118
debtRatio	-0.179817
effectiveTaxRate	0.076522
freeCashFlowOperatingCashFlowRatio	-0.030186
freeCashFlowPerShare	0.180502
cashPerShare	-0.124759
companyEquityMultiplier	-0.099772

ebitPerRevenue	0.457347
enterpriseValueMultiple	0.035597
operatingCashFlowPerShare	0.309117
operatingCashFlowSalesRatio	1.000000
payablesTurnover	-0.342826

	payablesTurnover
currentRatio	0.241996
quickRatio	0.130883
cashRatio	0.110186
daysOfSalesOutstanding	-0.169389
netProfitMargin	-0.096028
pretaxProfitMargin	-0.096216
grossProfitMargin	-0.541091
operatingProfitMargin	-0.124597
returnOnAssets	0.138222
returnOnCapitalEmployed	0.149818
returnOnEquity	0.082724
assetTurnover	0.413496
fixedAssetTurnover	0.273091
debtEquityRatio	-0.068000
debtRatio	-0.102072
effectiveTaxRate	0.033023
freeCashFlowOperatingCashFlowRatio	0.105935
freeCashFlowPerShare	0.043334
cashPerShare	0.012914
companyEquityMultiplier	-0.066155
ebitPerRevenue	-0.110682
enterpriseValueMultiple	0.007727
operatingCashFlowPerShare	-0.108446
operatingCashFlowSalesRatio	-0.342826
payablesTurnover	1.000000



```
[301]: vif(credit_rating_outl_scaled, predictors, '/Users/kamila/Desktop/Fidelity_
↪final_project/vif_table.png')
```

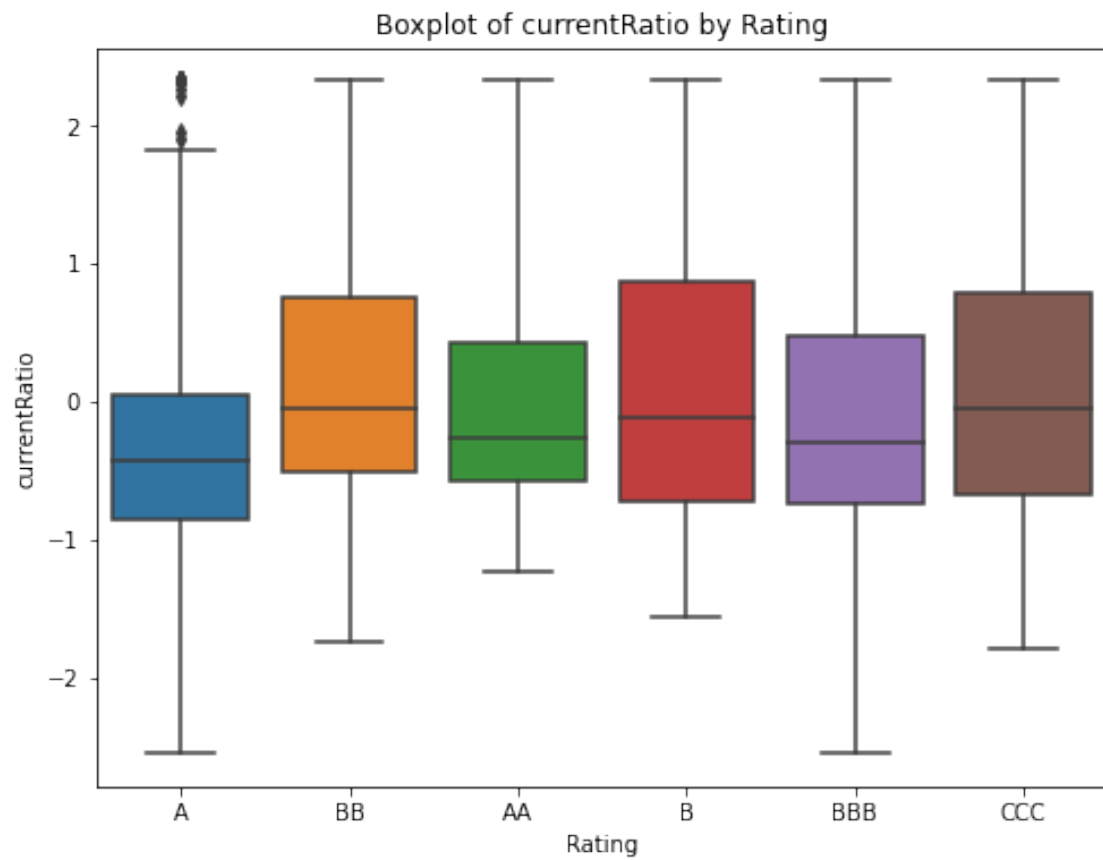
```
[301]:
```

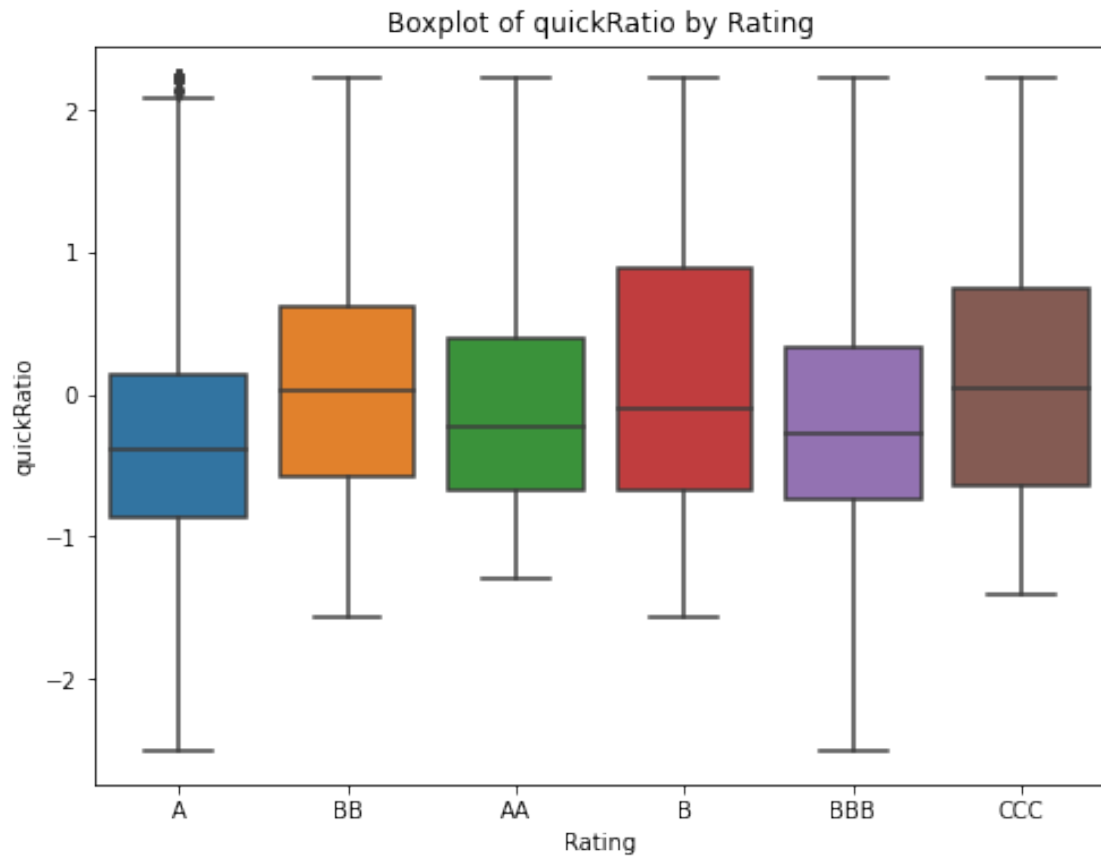
	feature	VIF
0	currentRatio	3.47
1	quickRatio	3.57
2	cashRatio	2.53
3	daysOfSalesOutstanding	1.38
4	netProfitMargin	12.18
5	pretaxProfitMargin	39.53
6	grossProfitMargin	1.92
7	operatingProfitMargin	7.41
8	returnOnAssets	12.50
9	returnOnCapitalEmployed	9.89
10	returnOnEquity	2.38
11	assetTurnover	3.17
12	fixedAssetTurnover	1.67
13	debtEquityRatio	103.52
14	debtRatio	1.53

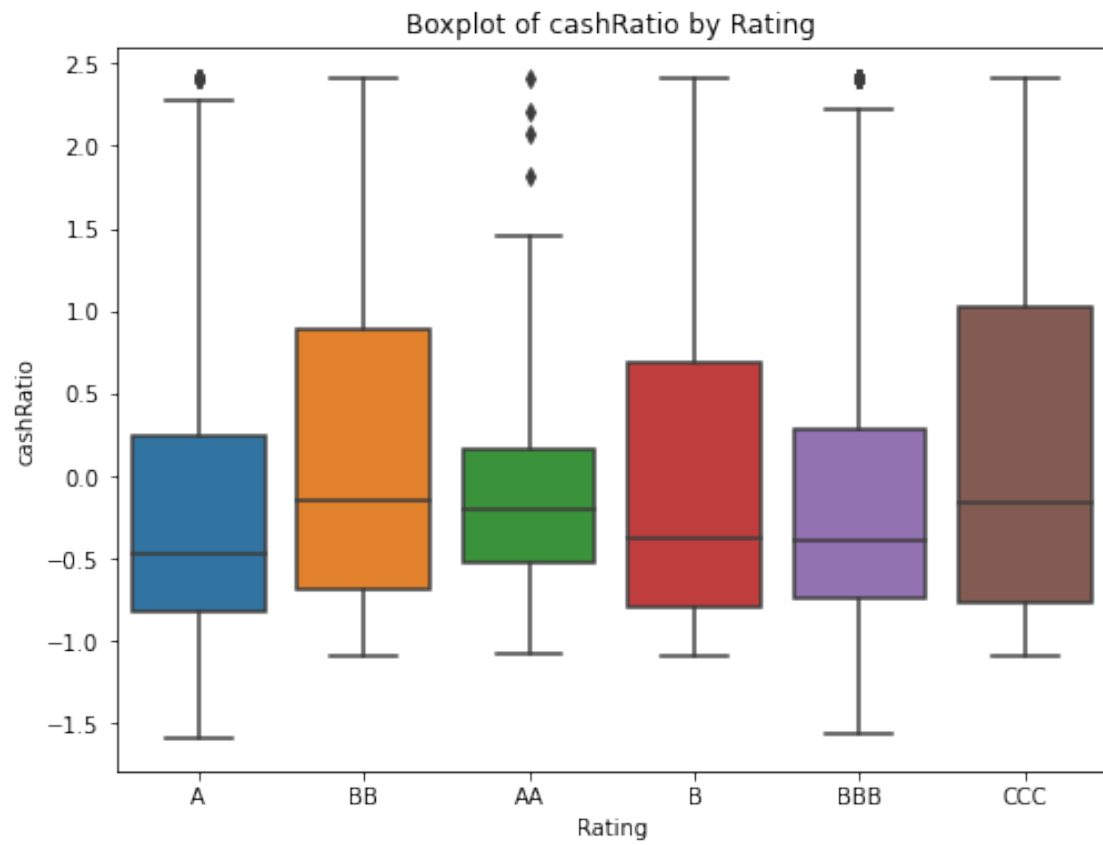
15	effectiveTaxRate	1.12
16	freeCashFlowOperatingCashFlowRatio	2.18
17	freeCashFlowPerShare	3.50
18	cashPerShare	2.08
19	companyEquityMultiplier	101.57
20	ebitPerRevenue	40.46
21	enterpriseValueMultiple	1.16
22	operatingCashFlowPerShare	3.52
23	operatingCashFlowSalesRatio	2.74
24	payablesTurnover	1.75

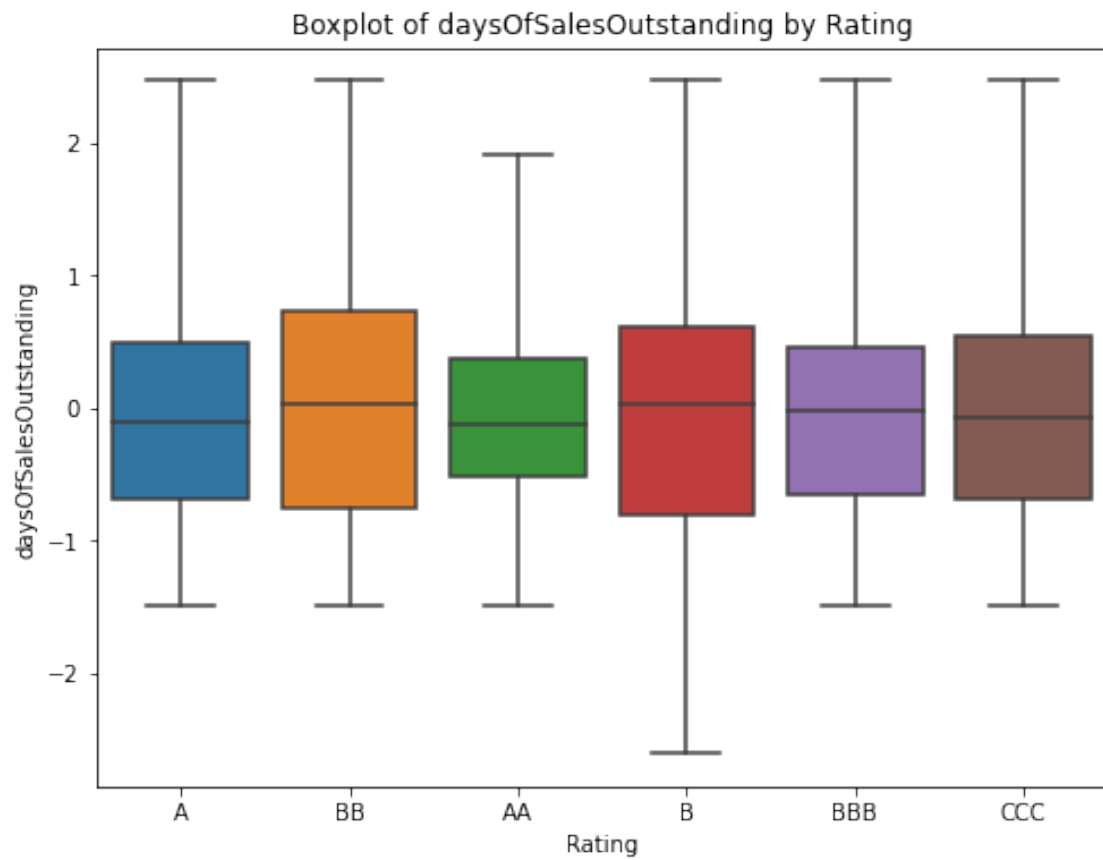
feature	VIF
currentRatio	3.47
quickRatio	3.57
cashRatio	2.53
daysOfSalesOutstanding	1.38
netProfitMargin	12.18
pretaxProfitMargin	39.53
grossProfitMargin	1.92
operatingProfitMargin	7.41
returnOnAssets	12.5
returnOnCapitalEmployed	9.89
returnOnEquity	2.38
assetTurnover	3.17
fixedAssetTurnover	1.67
debtEquityRatio	103.52
debtRatio	1.53
effectiveTaxRate	1.12
freeCashFlowOperatingCashFlowRatio	2.18
freeCashFlowPerShare	3.5
cashPerShare	2.08
companyEquityMultiplier	101.57
ebitPerRevenue	40.46
enterpriseValueMultiple	1.16
operatingCashFlowPerShare	3.52
operatingCashFlowSalesRatio	2.74
payablesTurnover	1.75

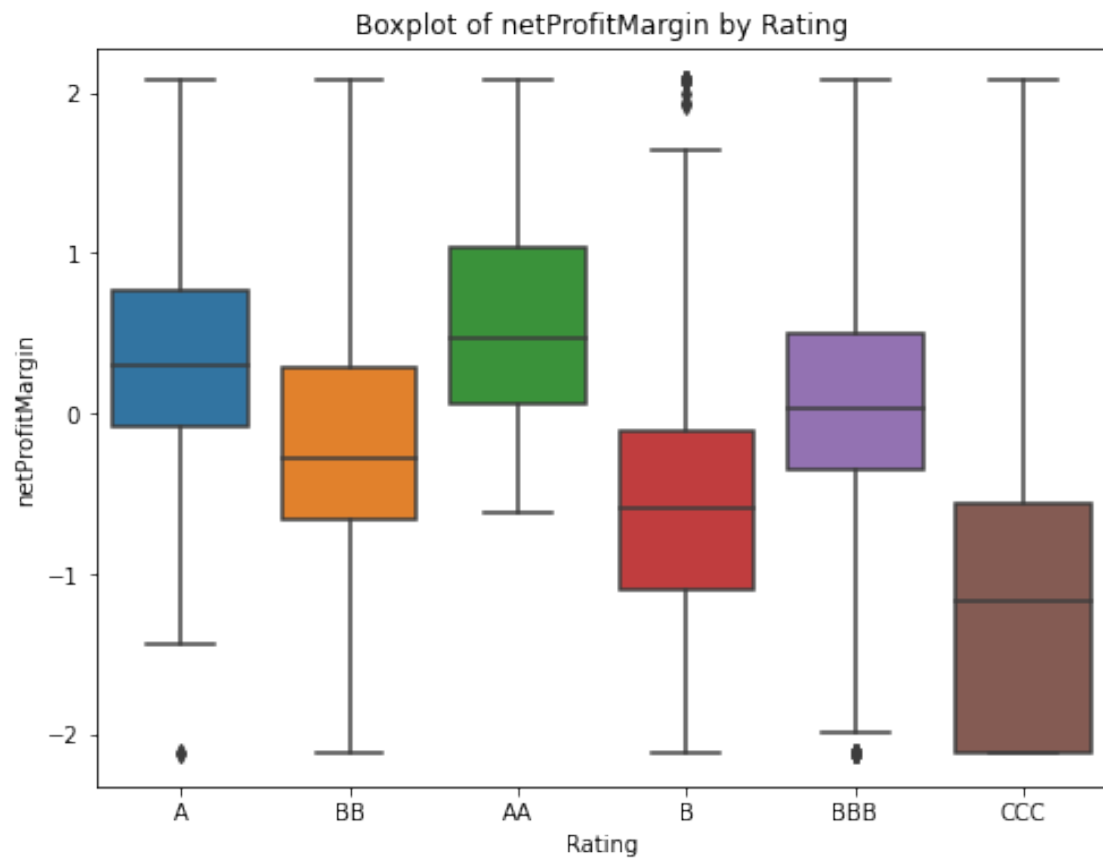
```
[594]: for pred in predictors:
        boxplt(credit_rating_outl_scaled, 'Rating', pred)
```

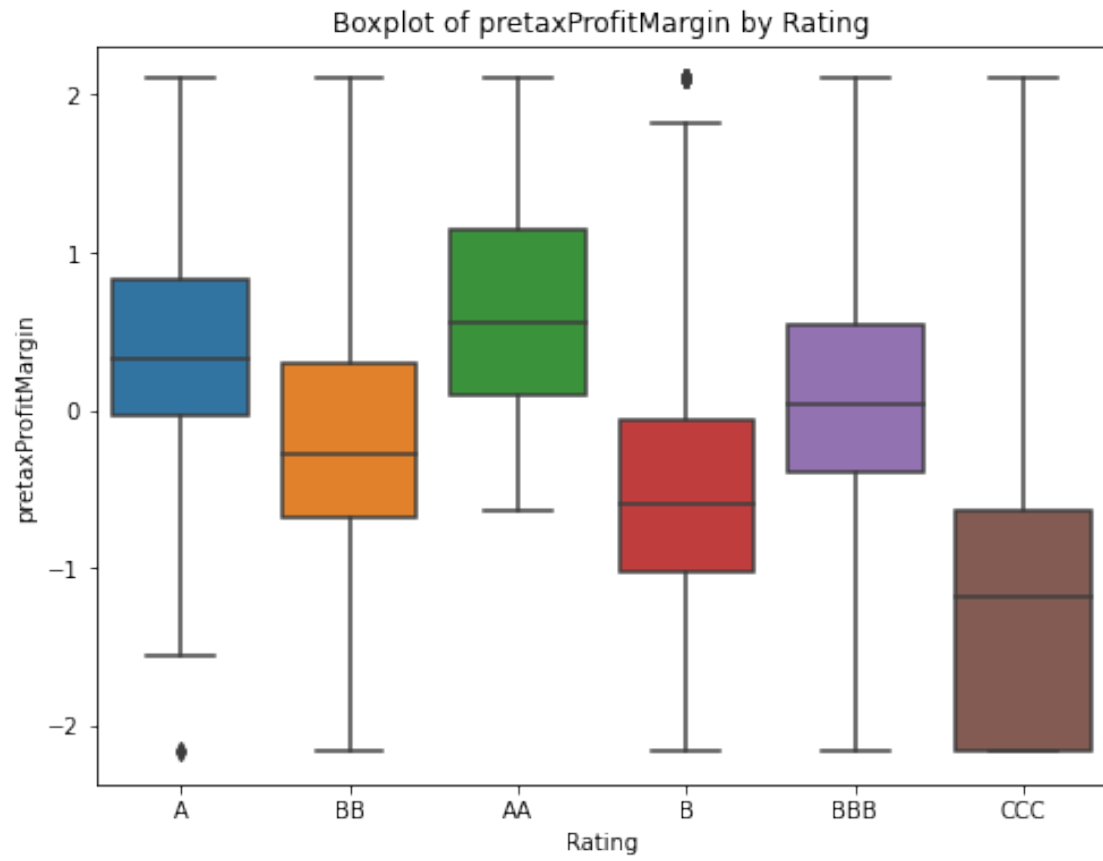


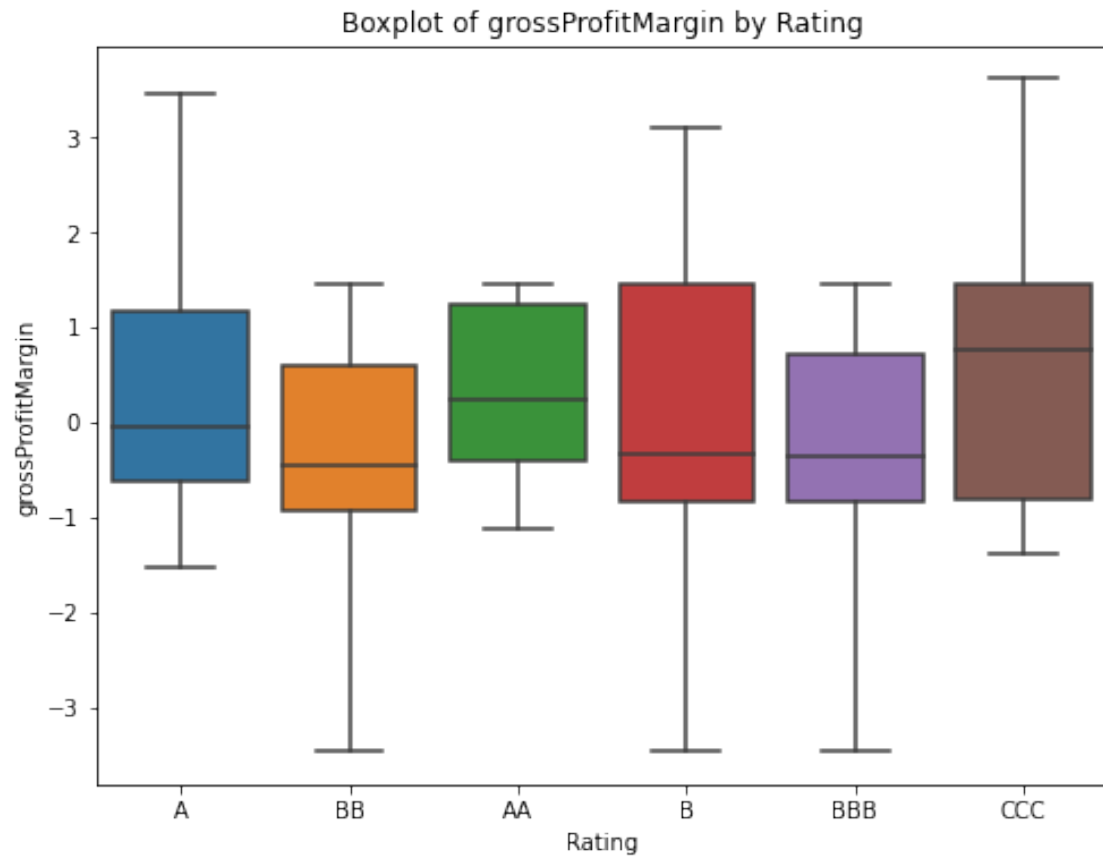


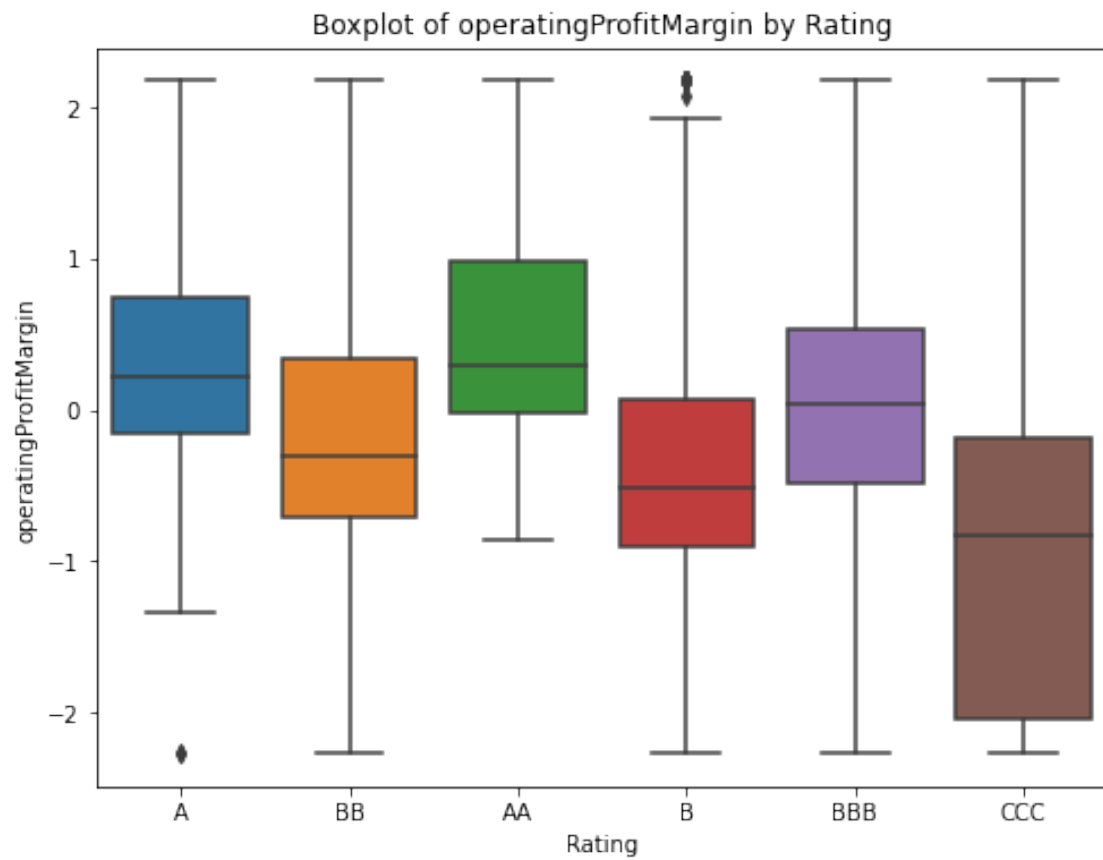


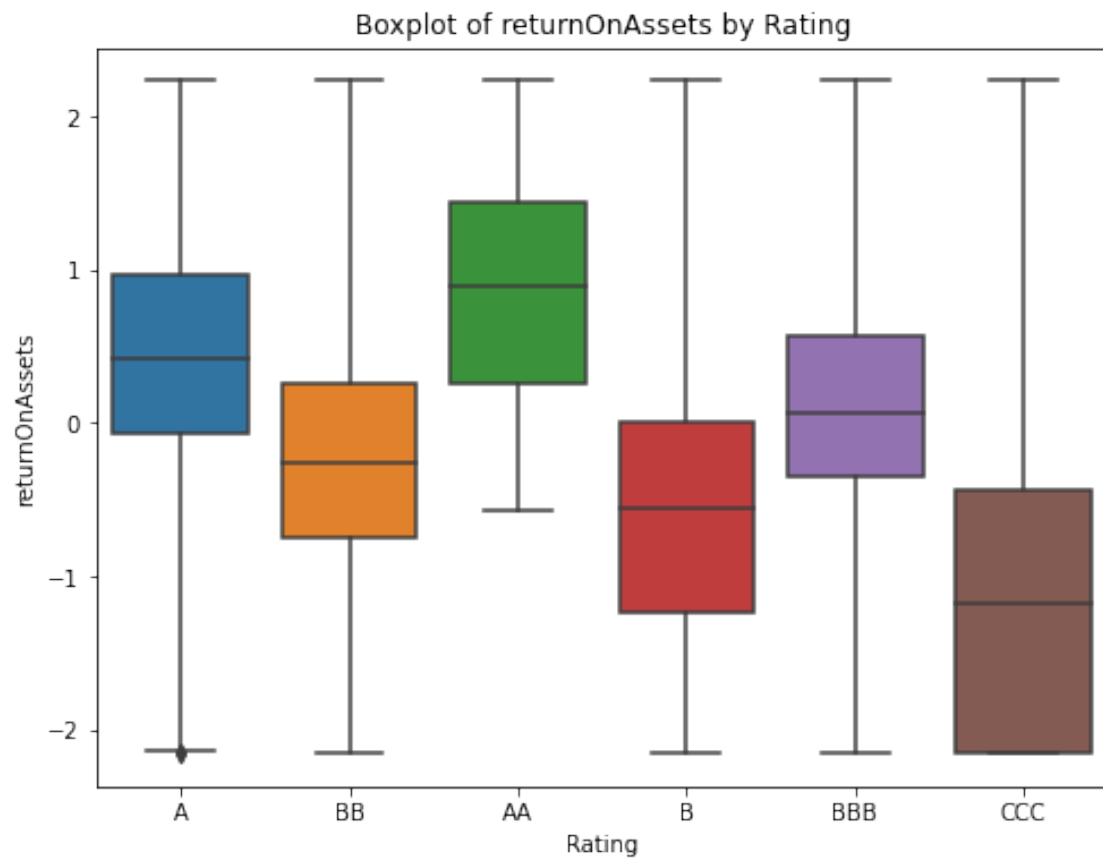


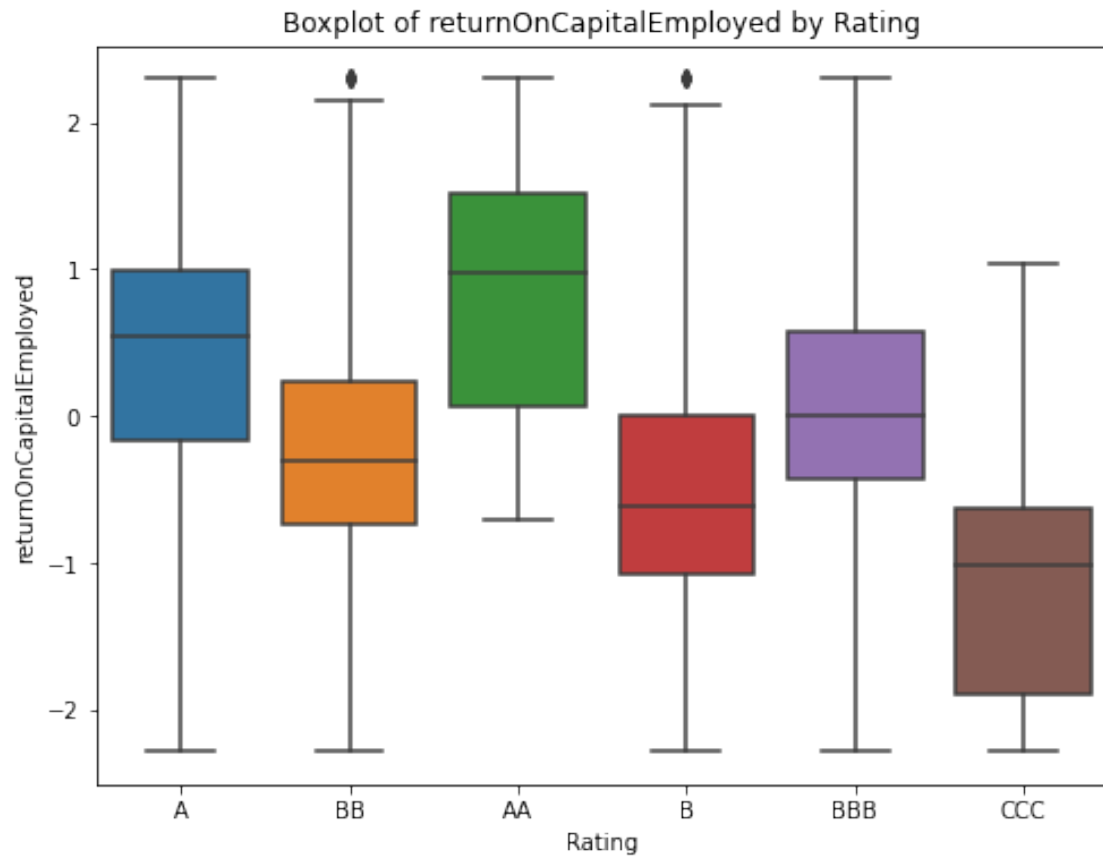


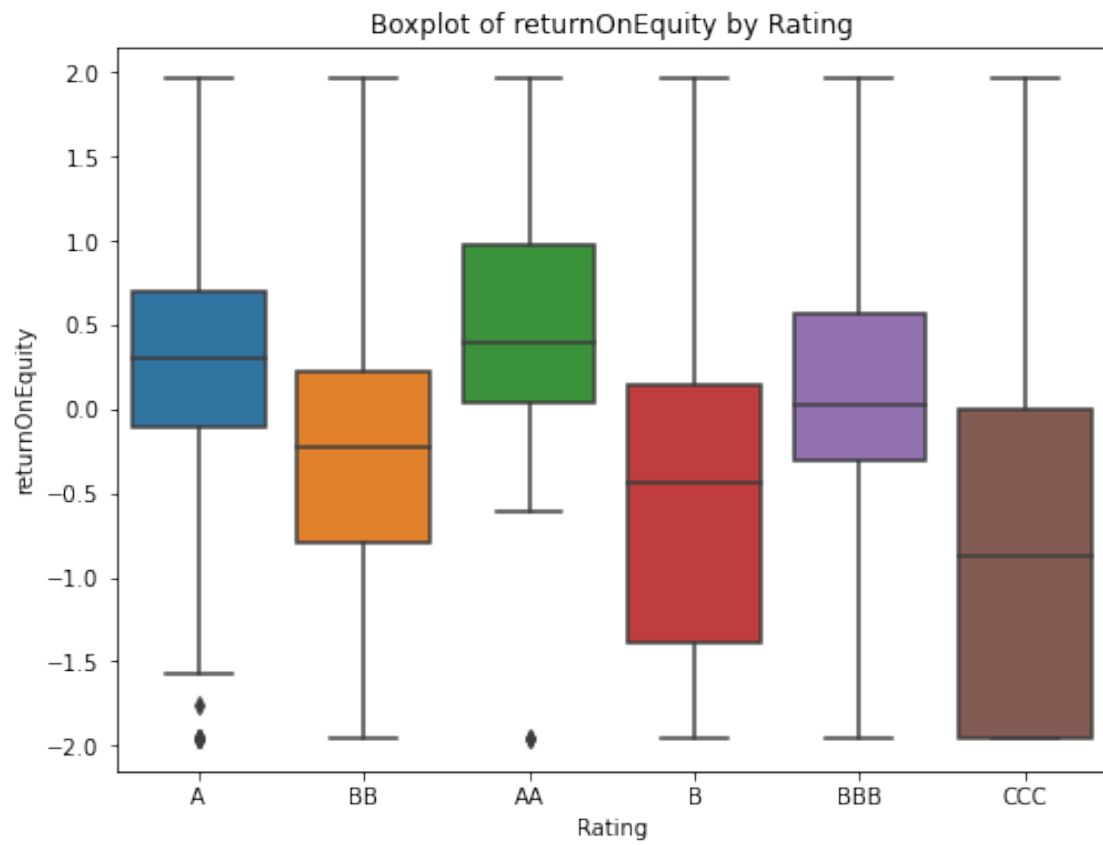


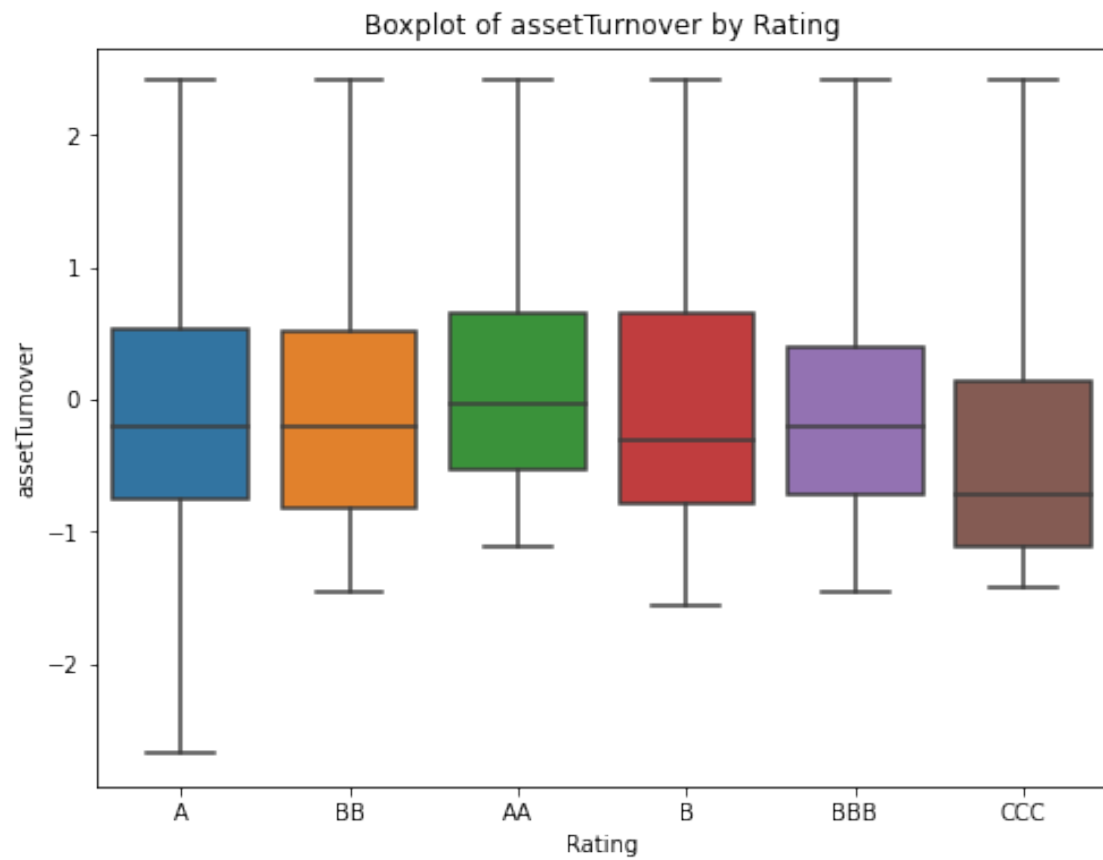


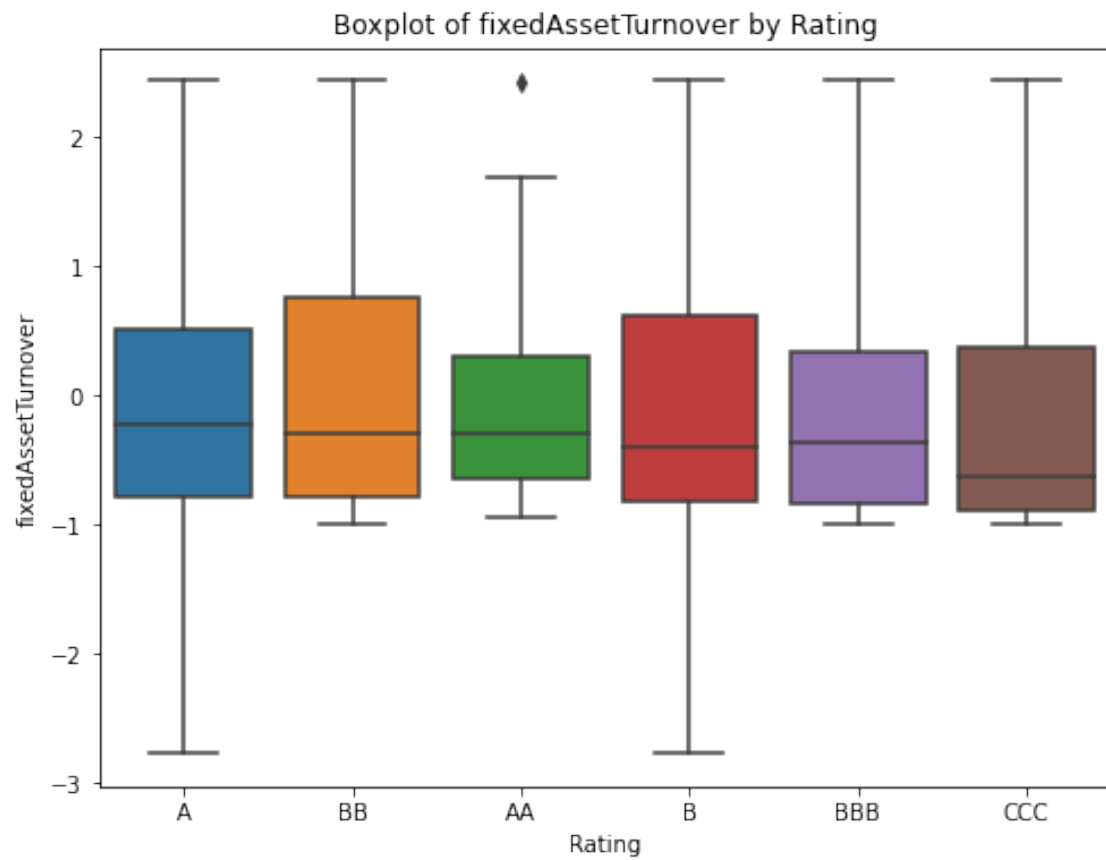


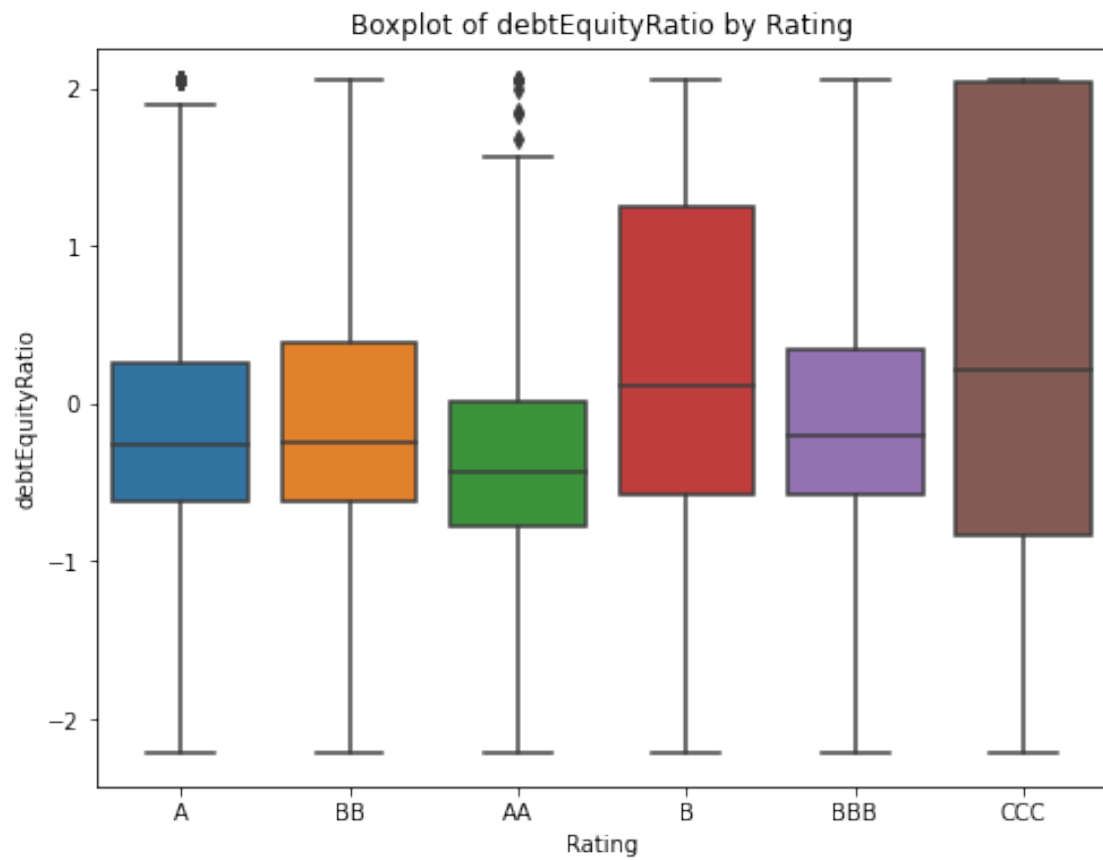


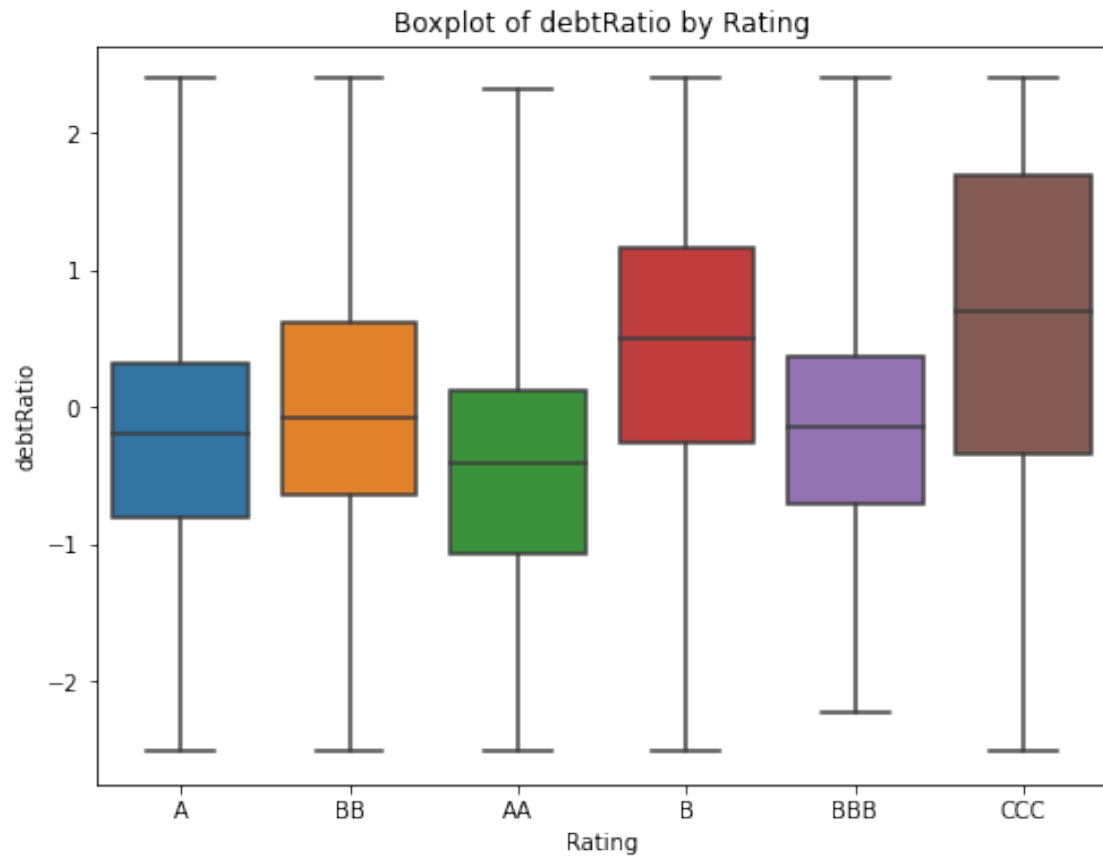


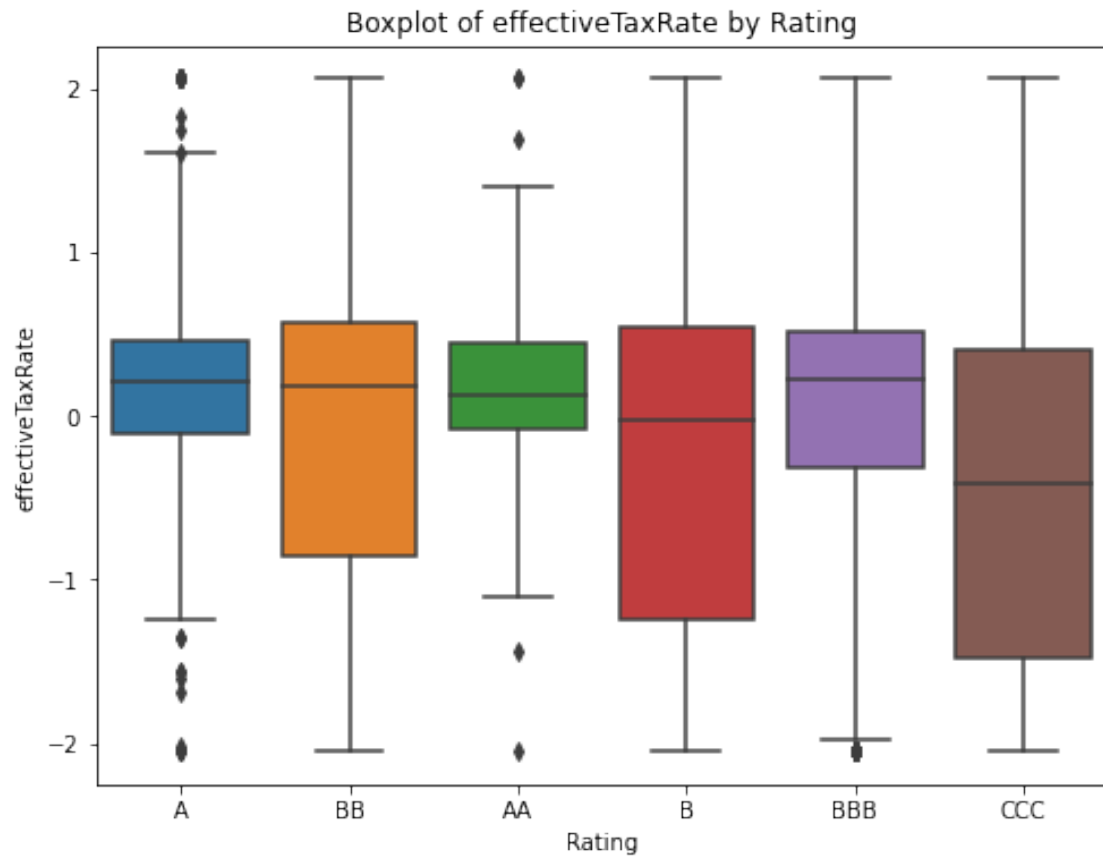


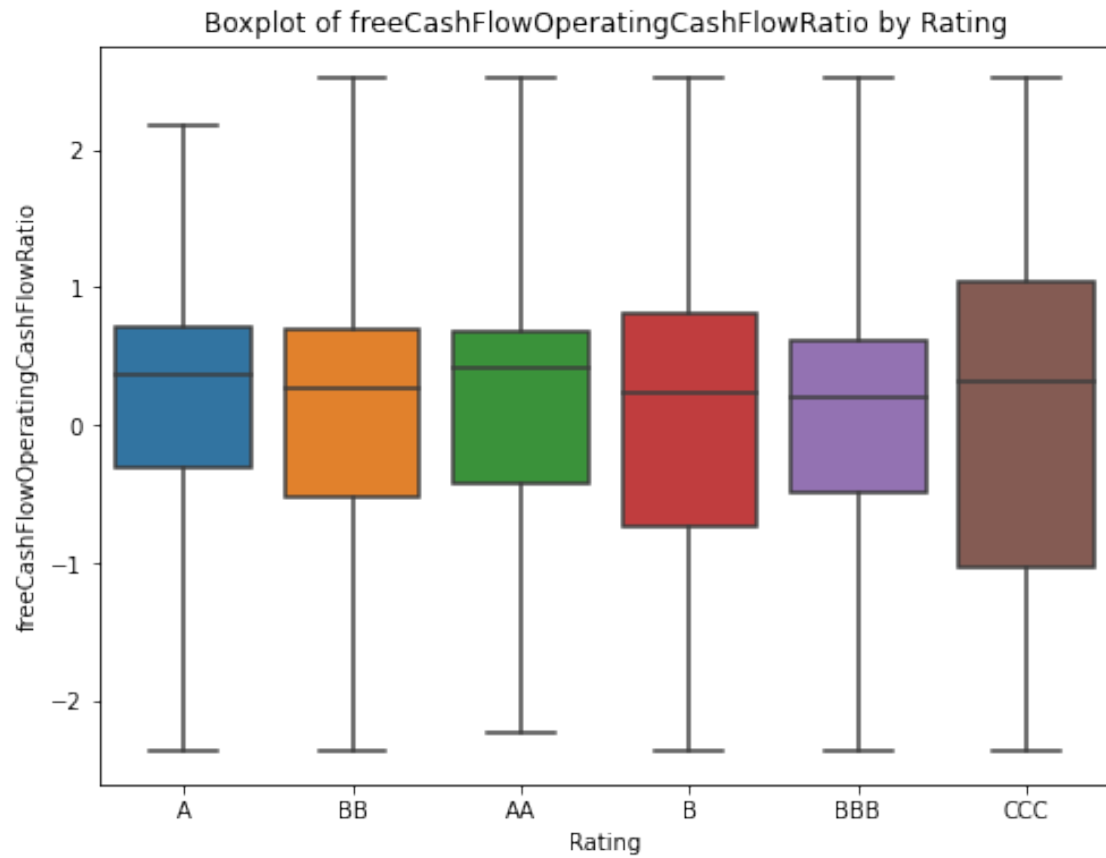


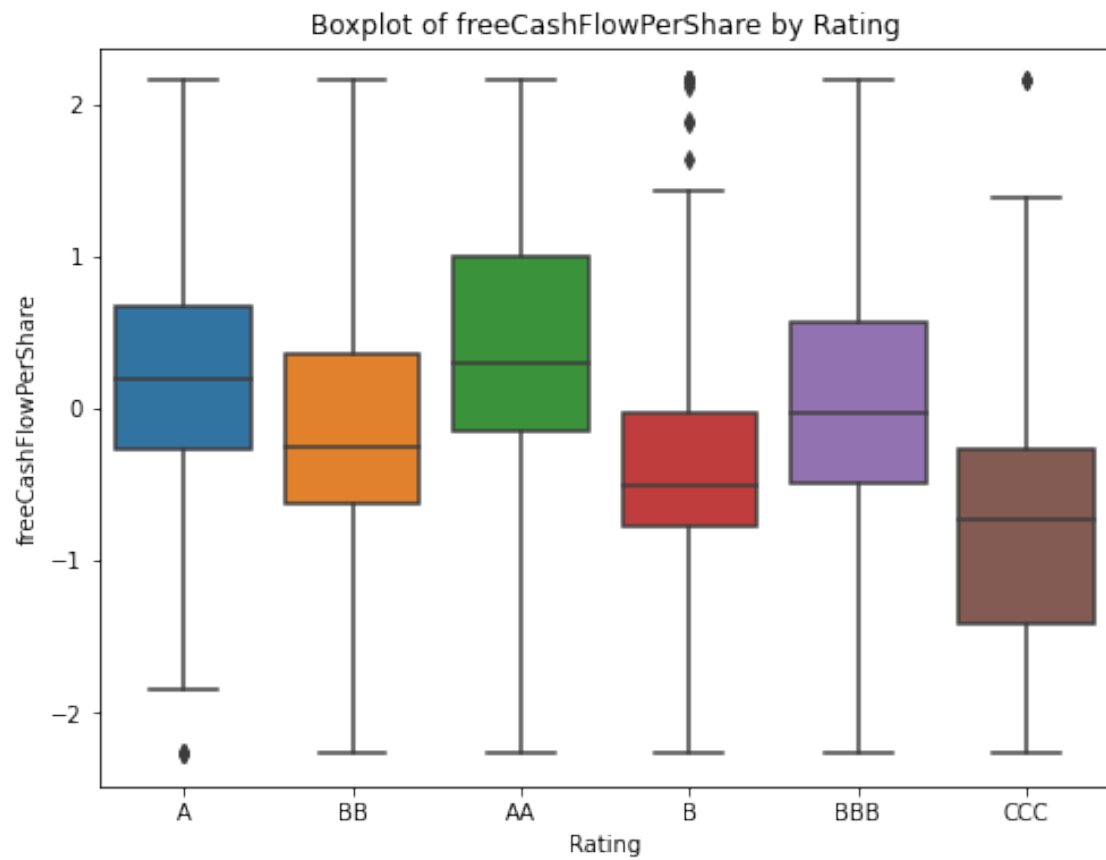


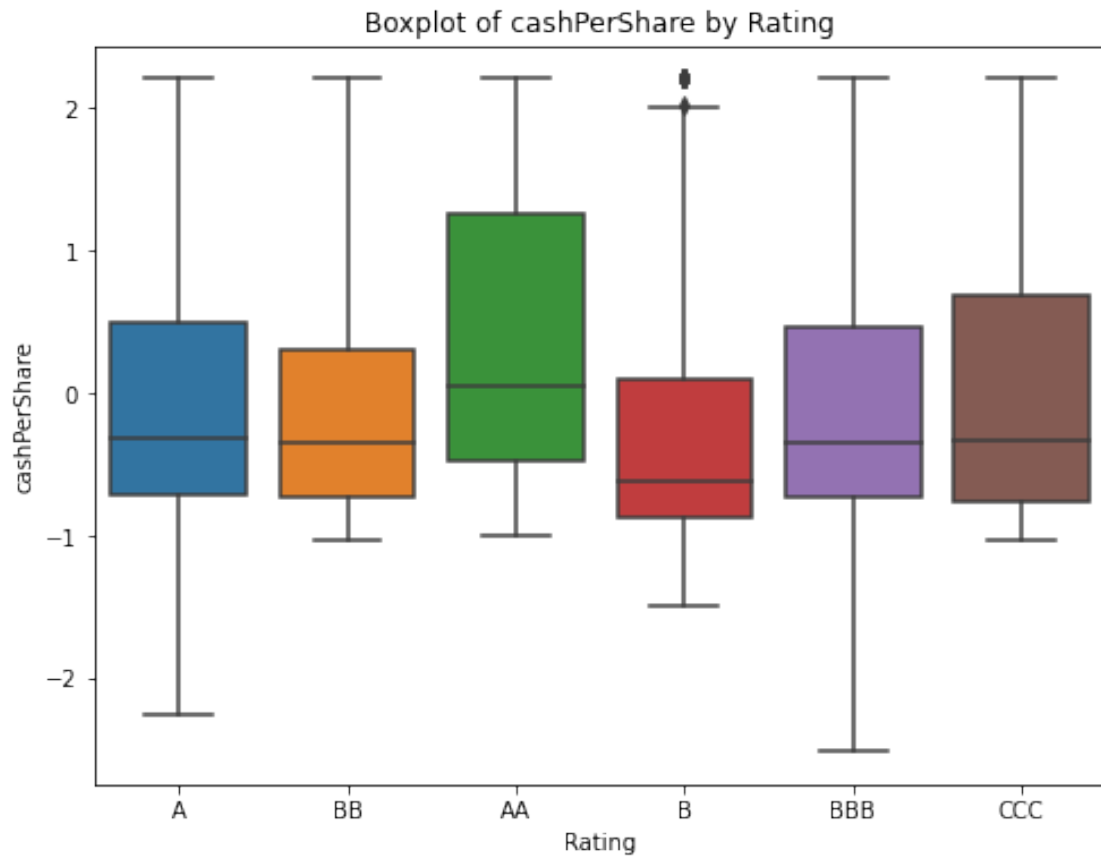


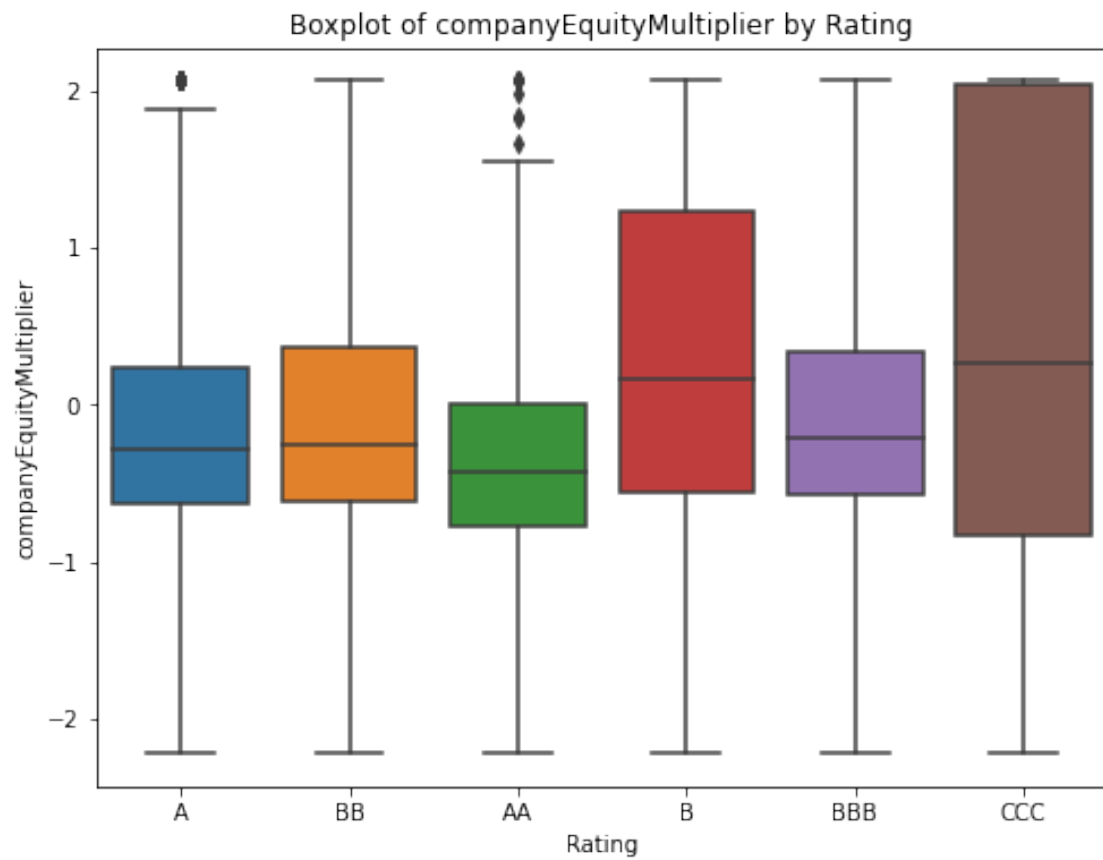


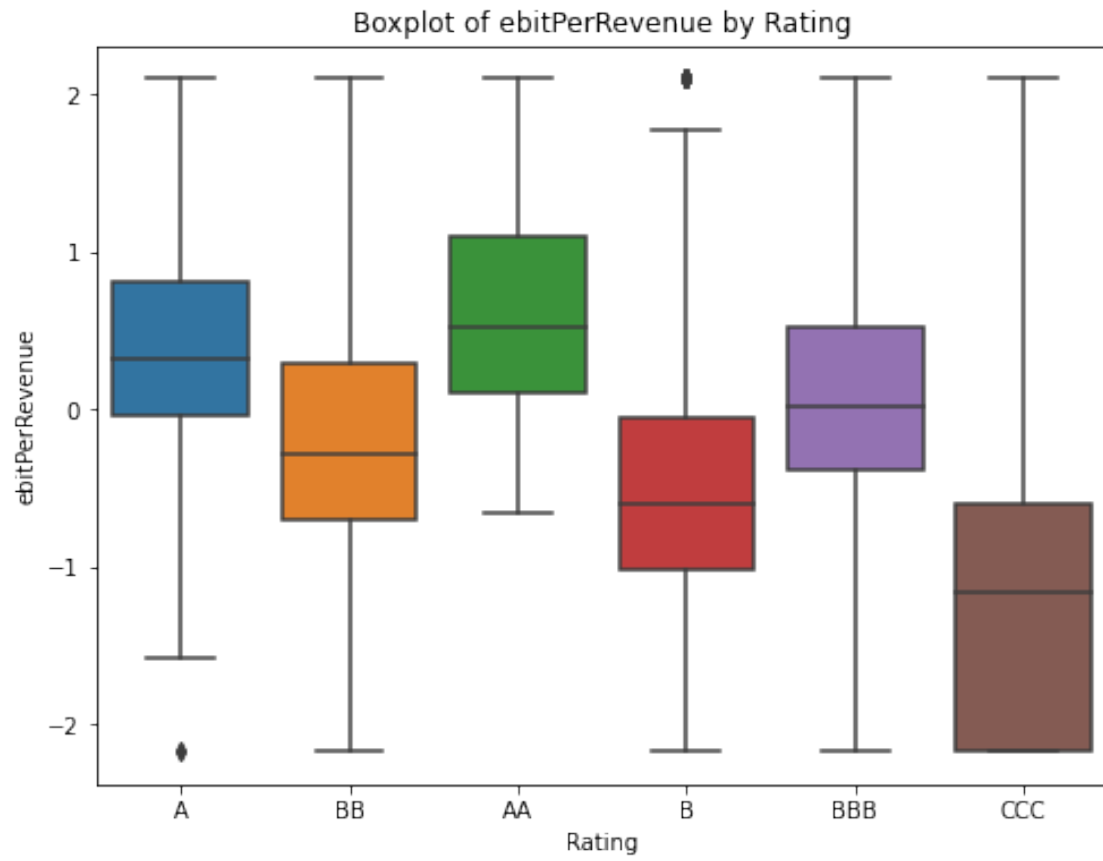


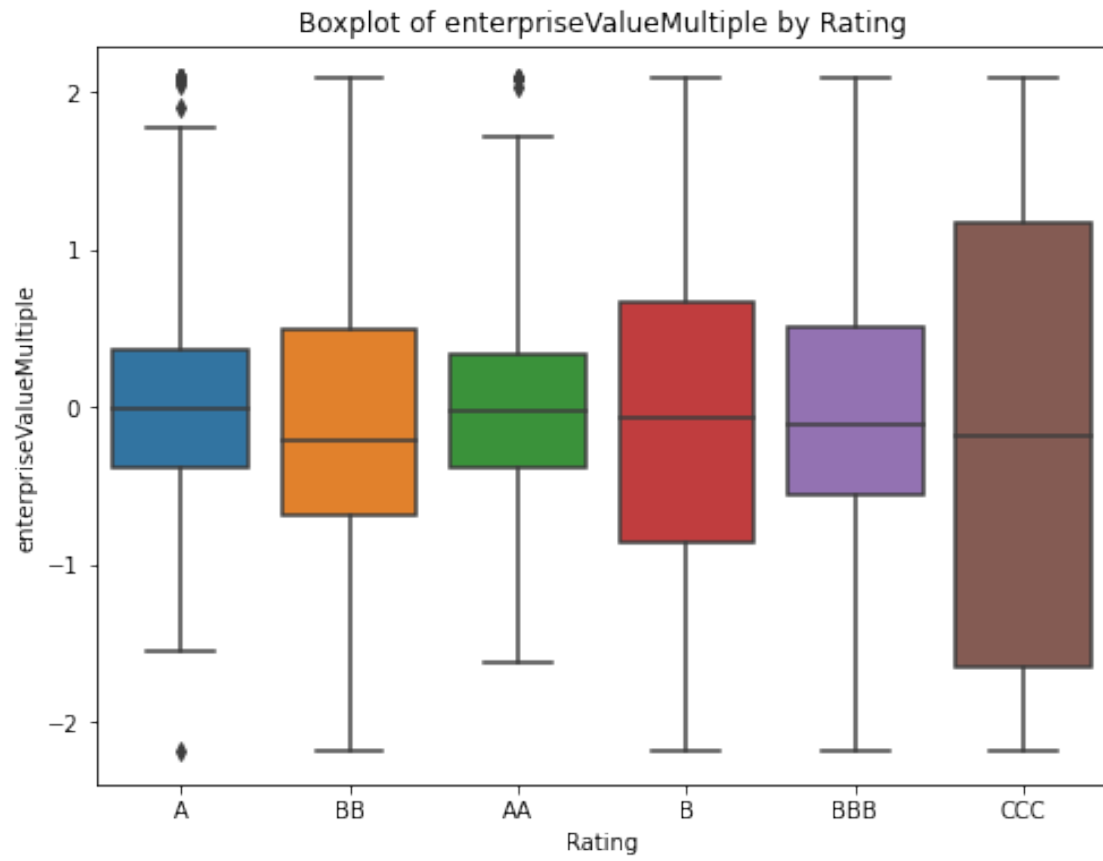


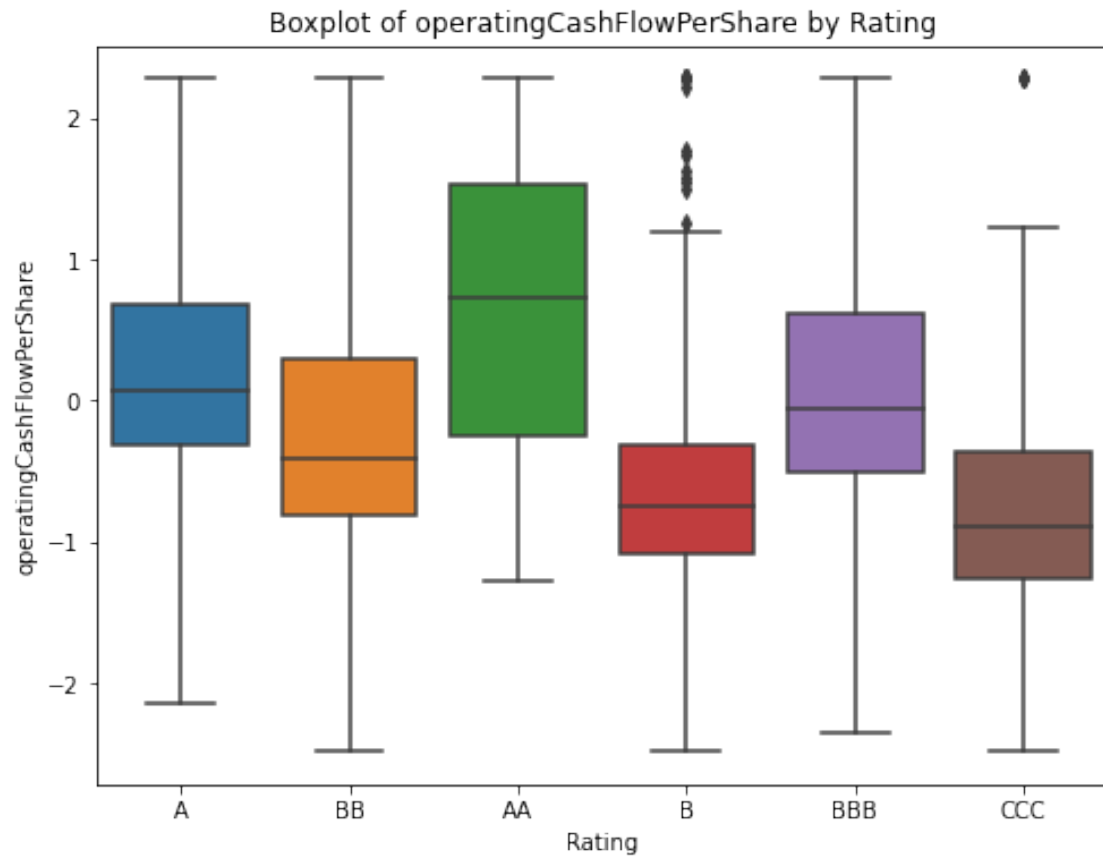


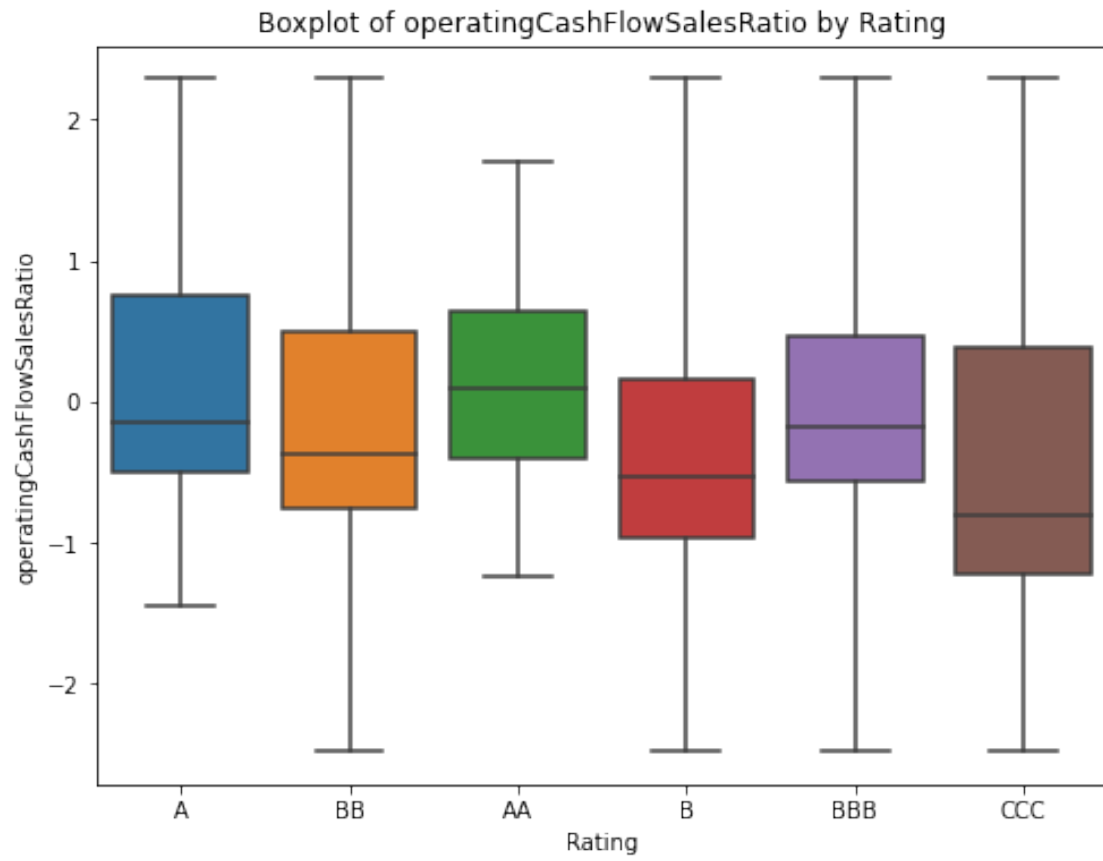


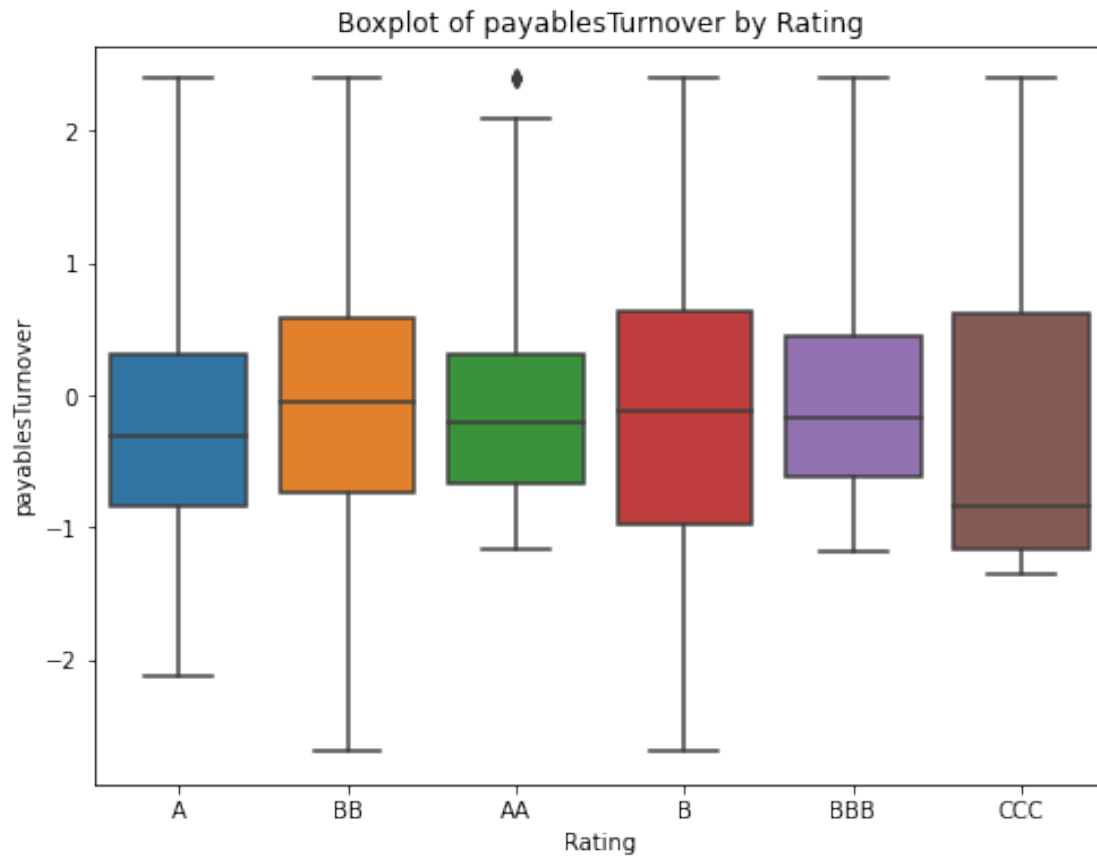




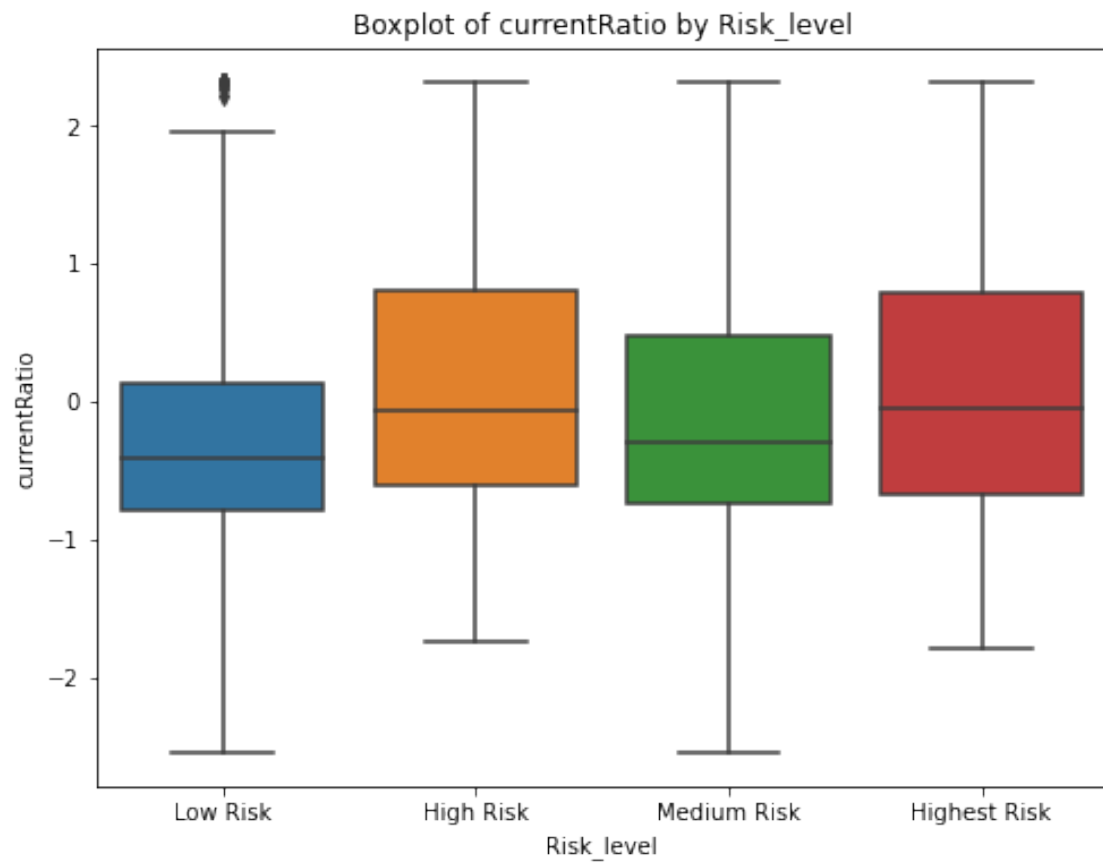


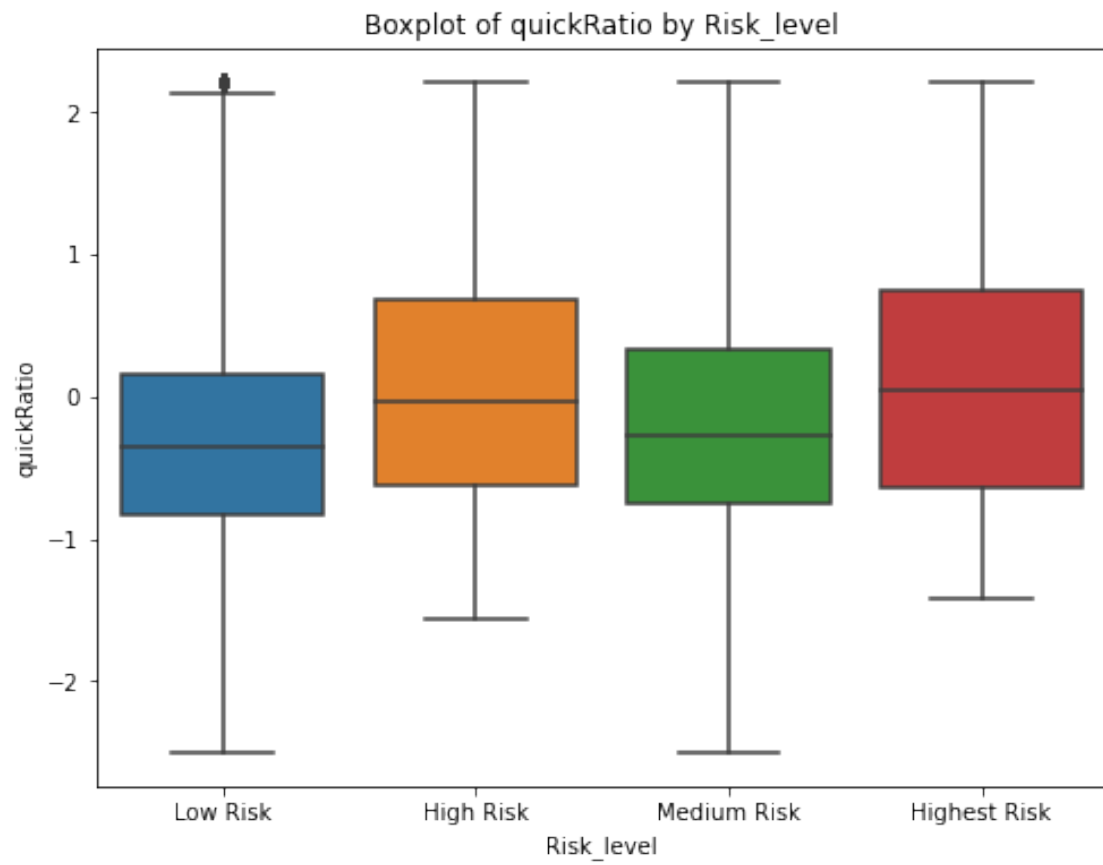


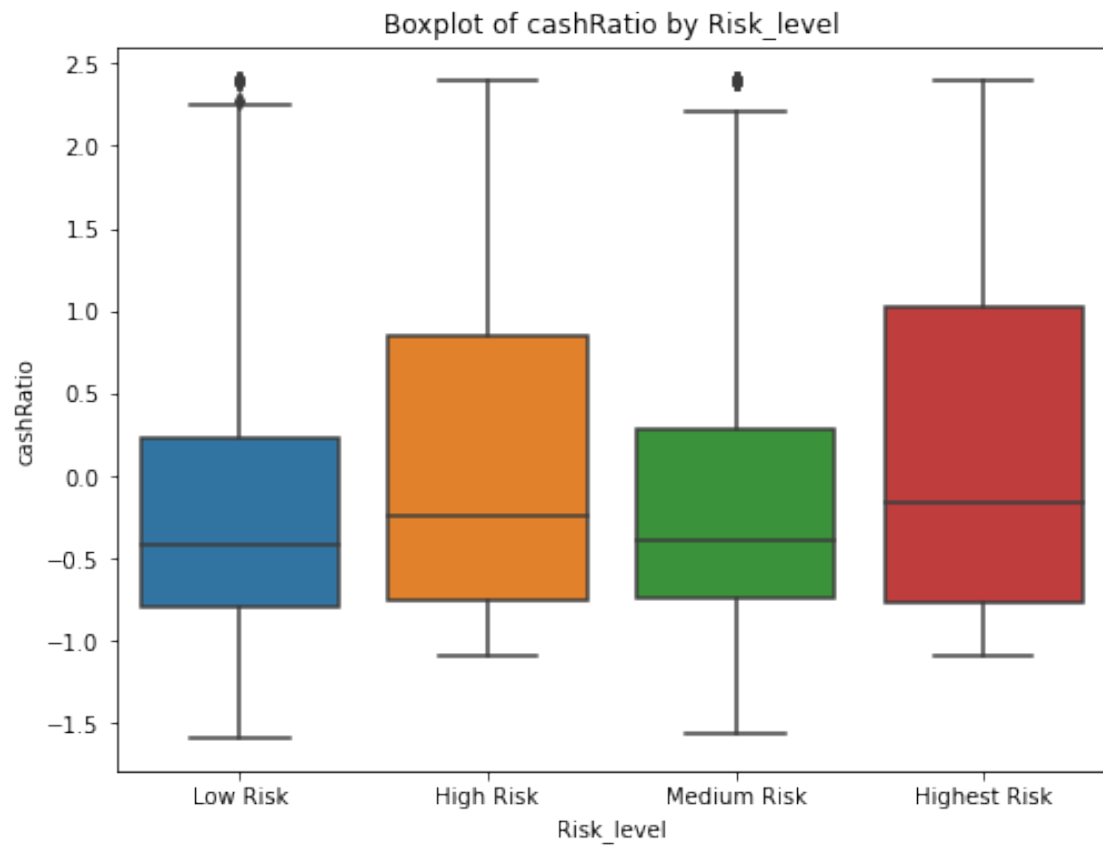


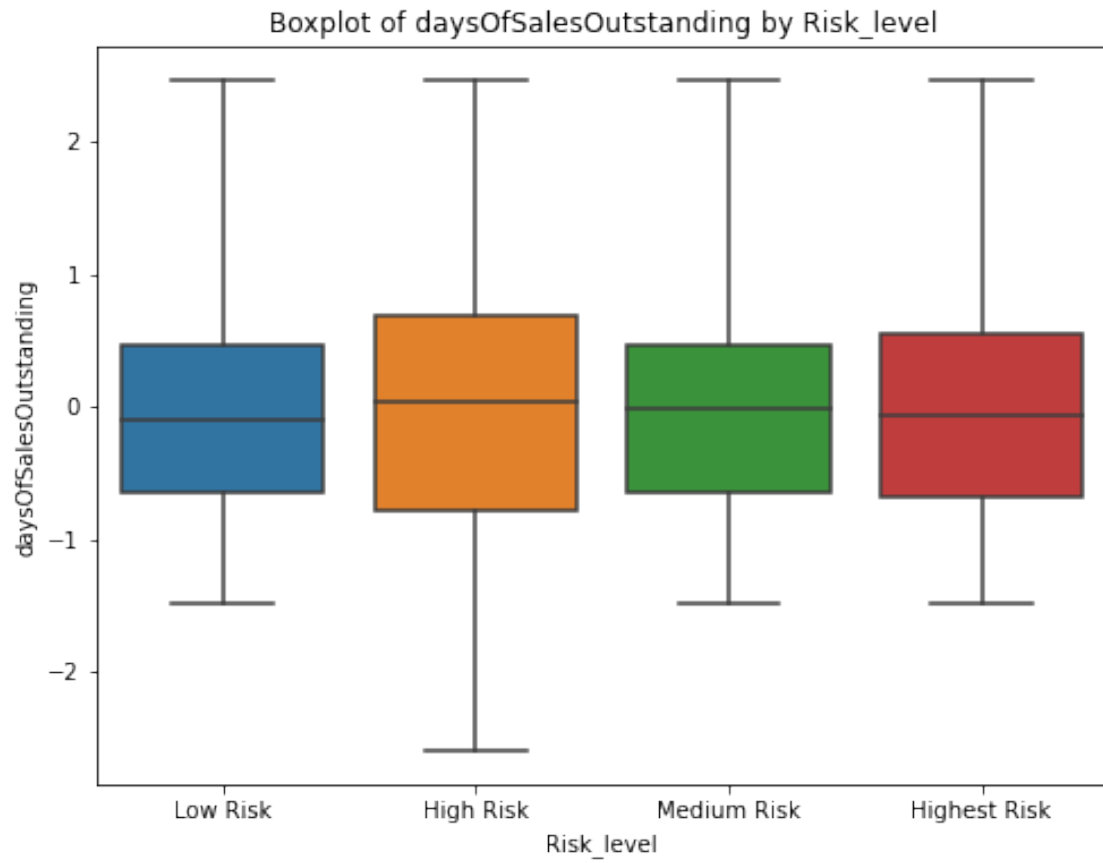


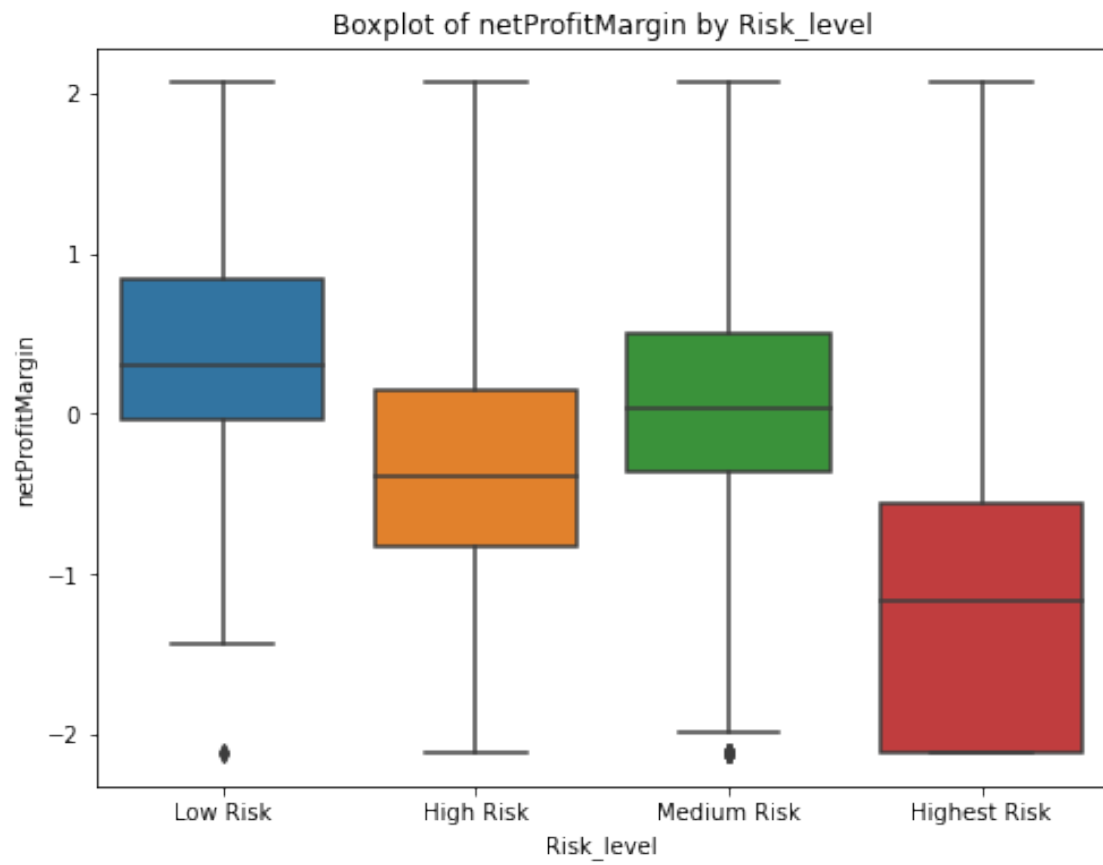
```
[595]: for pred in predictors:  
        boxplt(credit_rating_outl_scaled, 'Risk_level', pred)
```

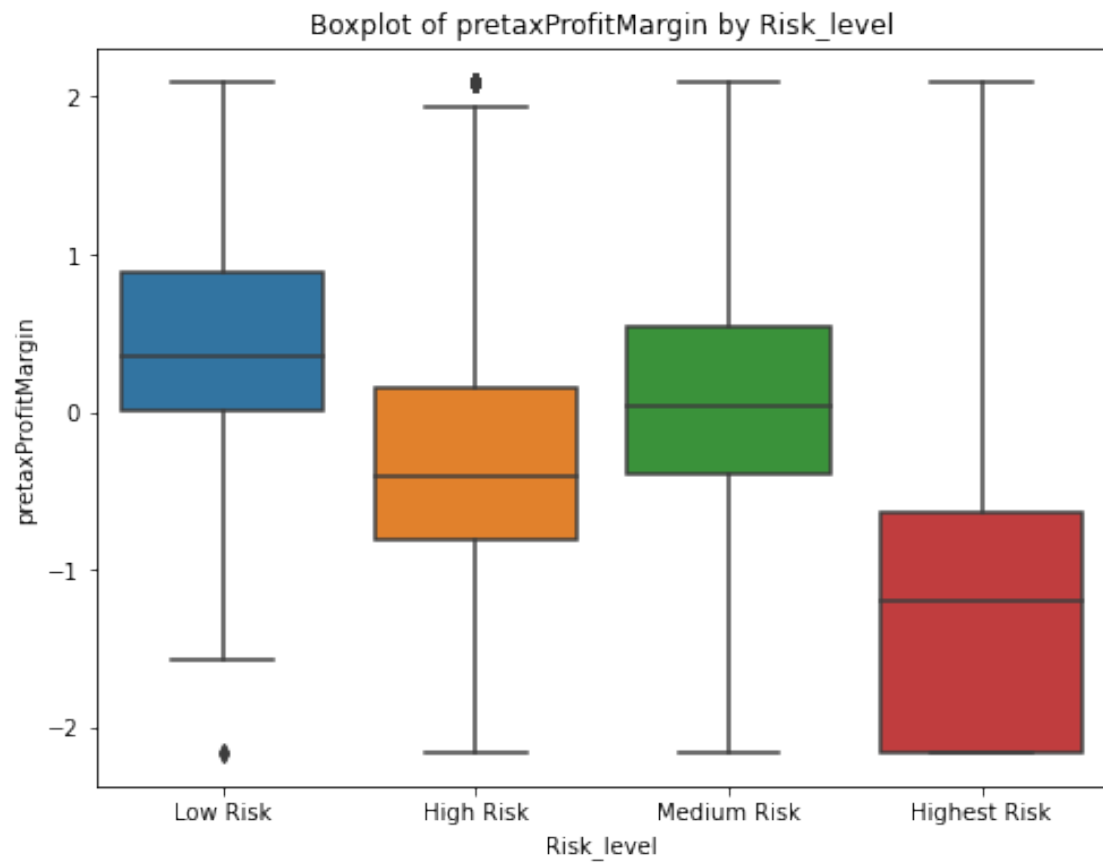


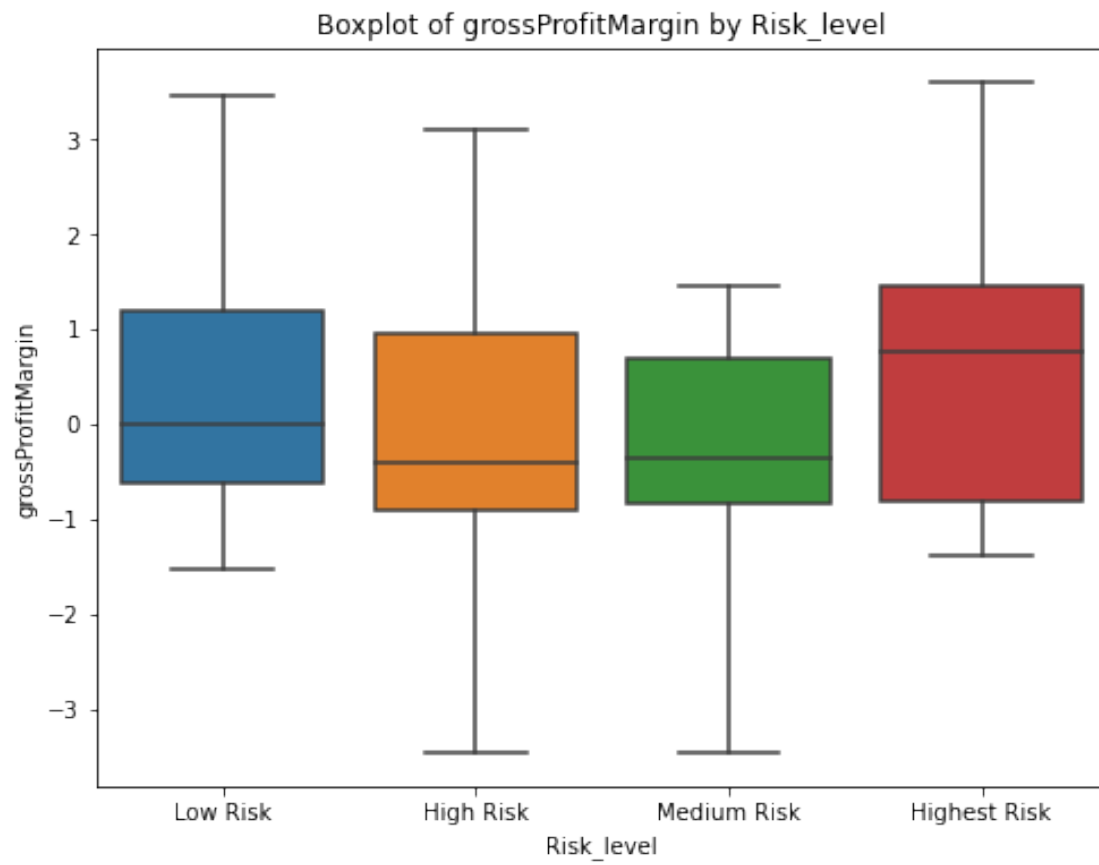


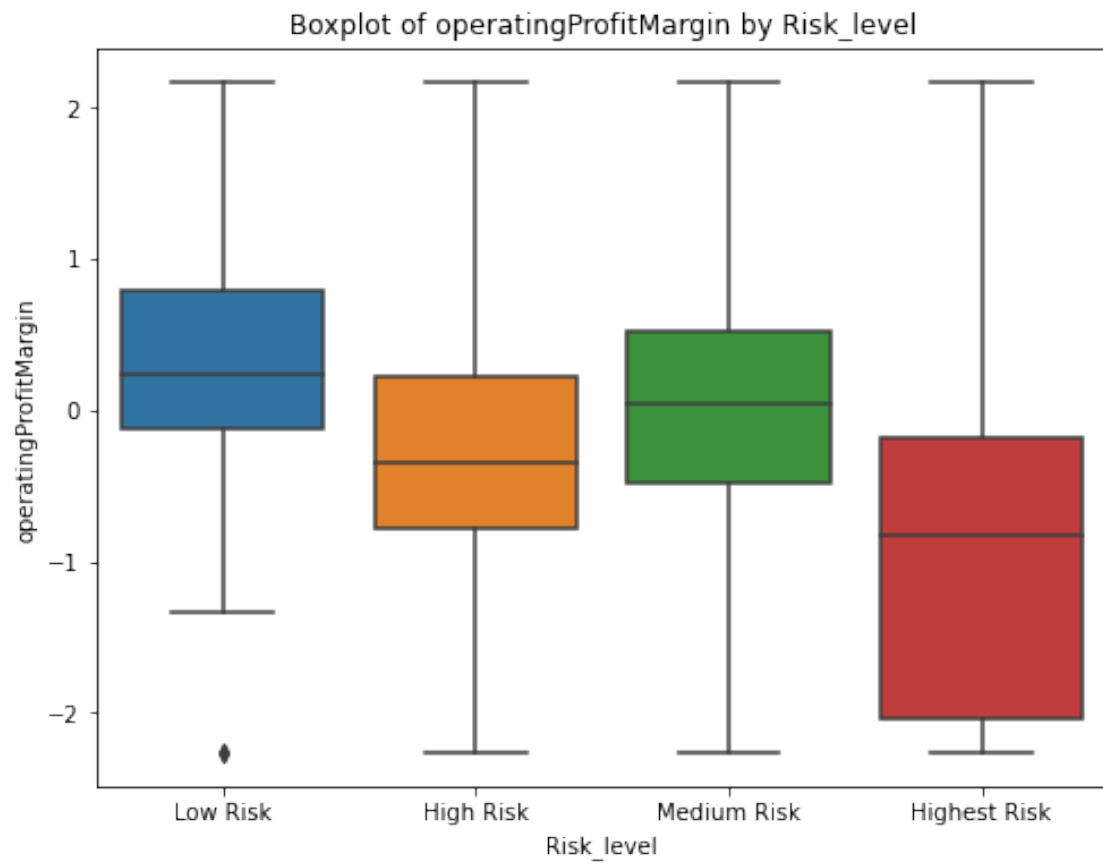


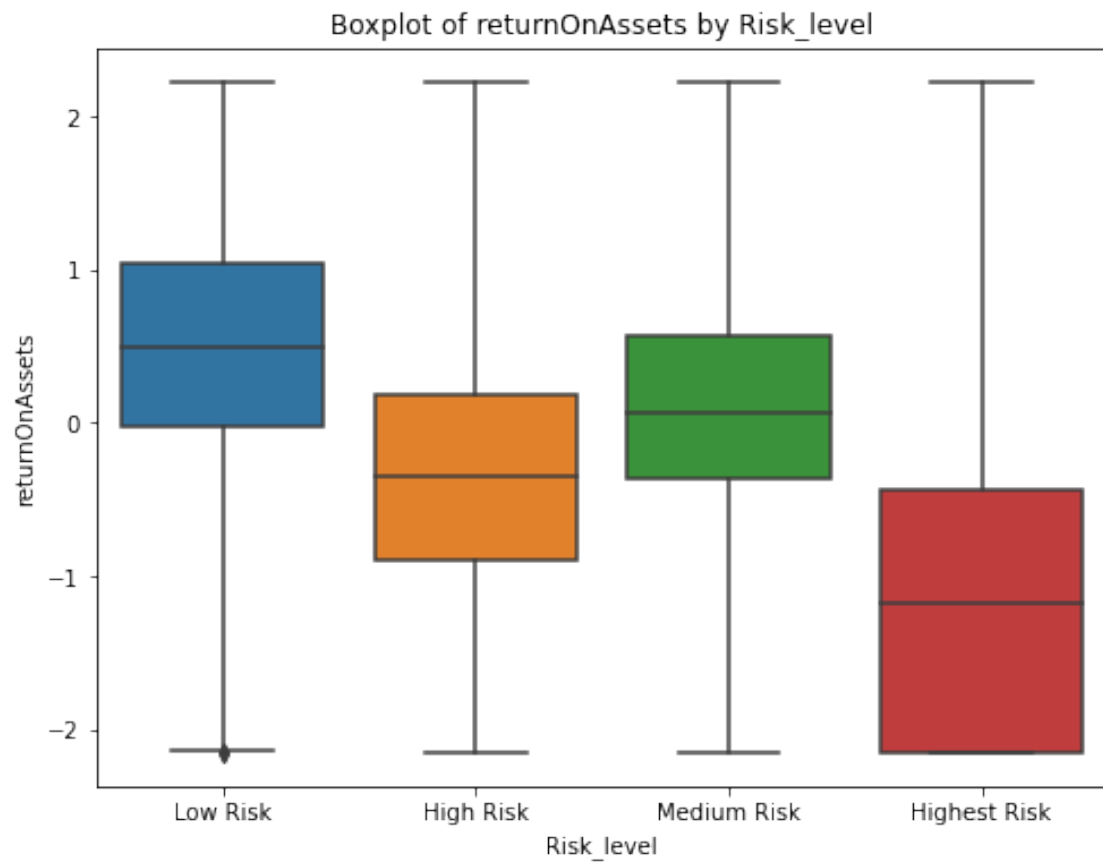


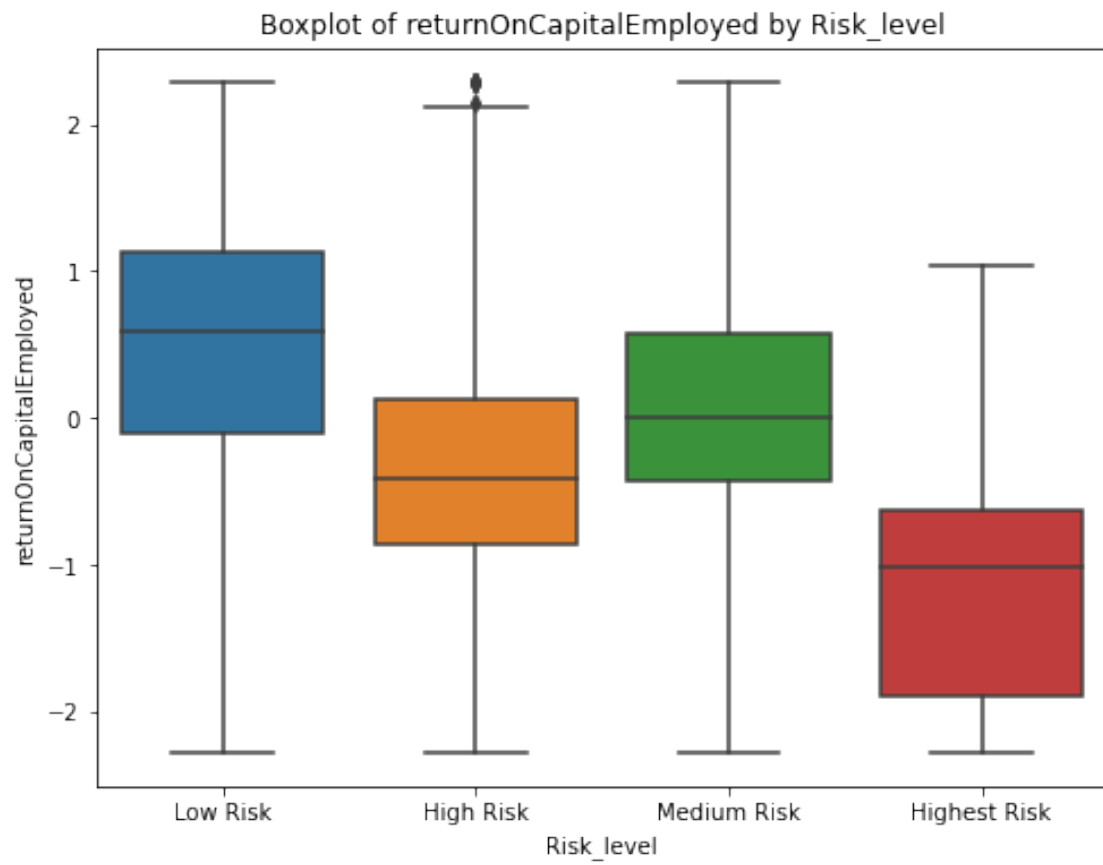


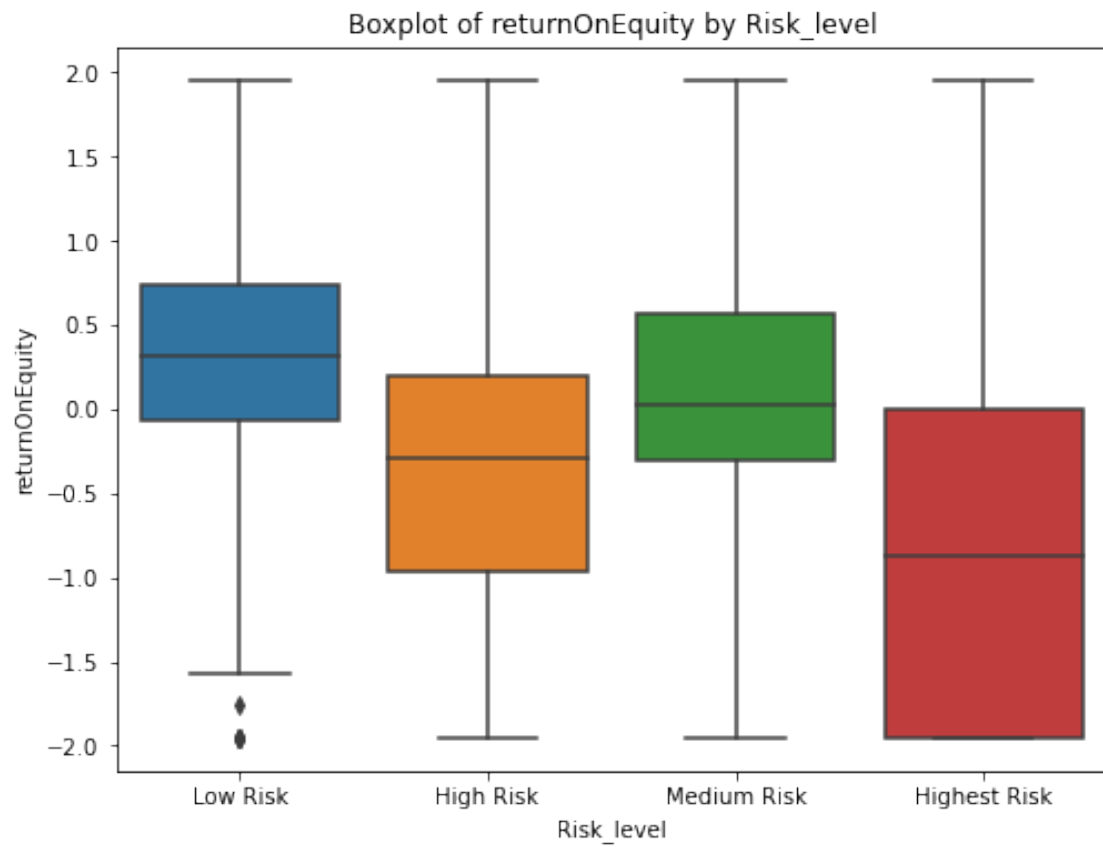


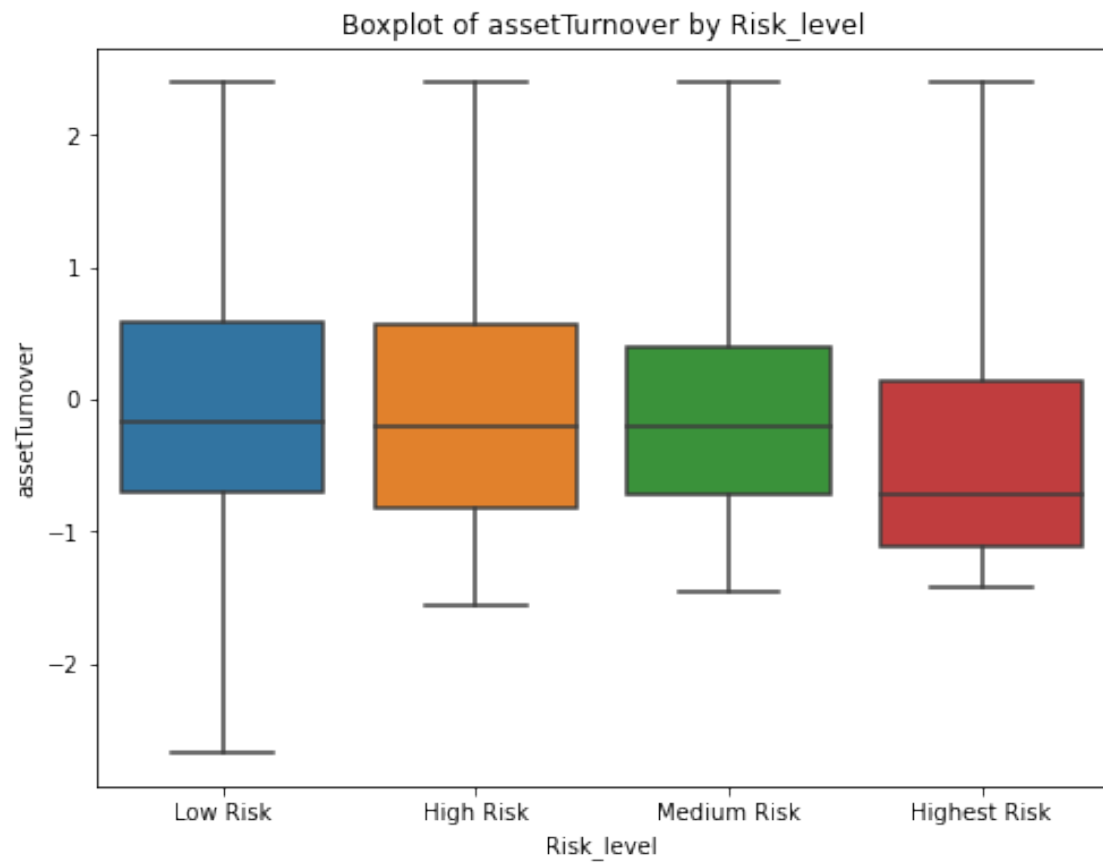


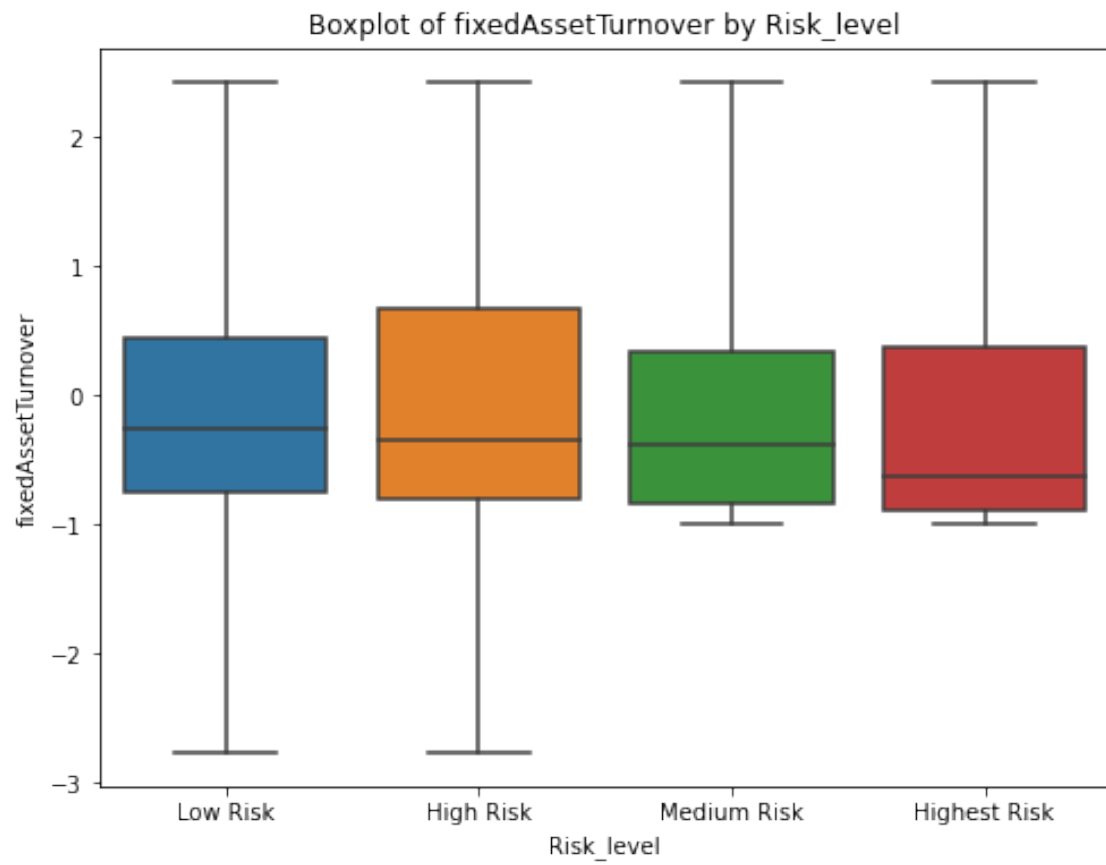


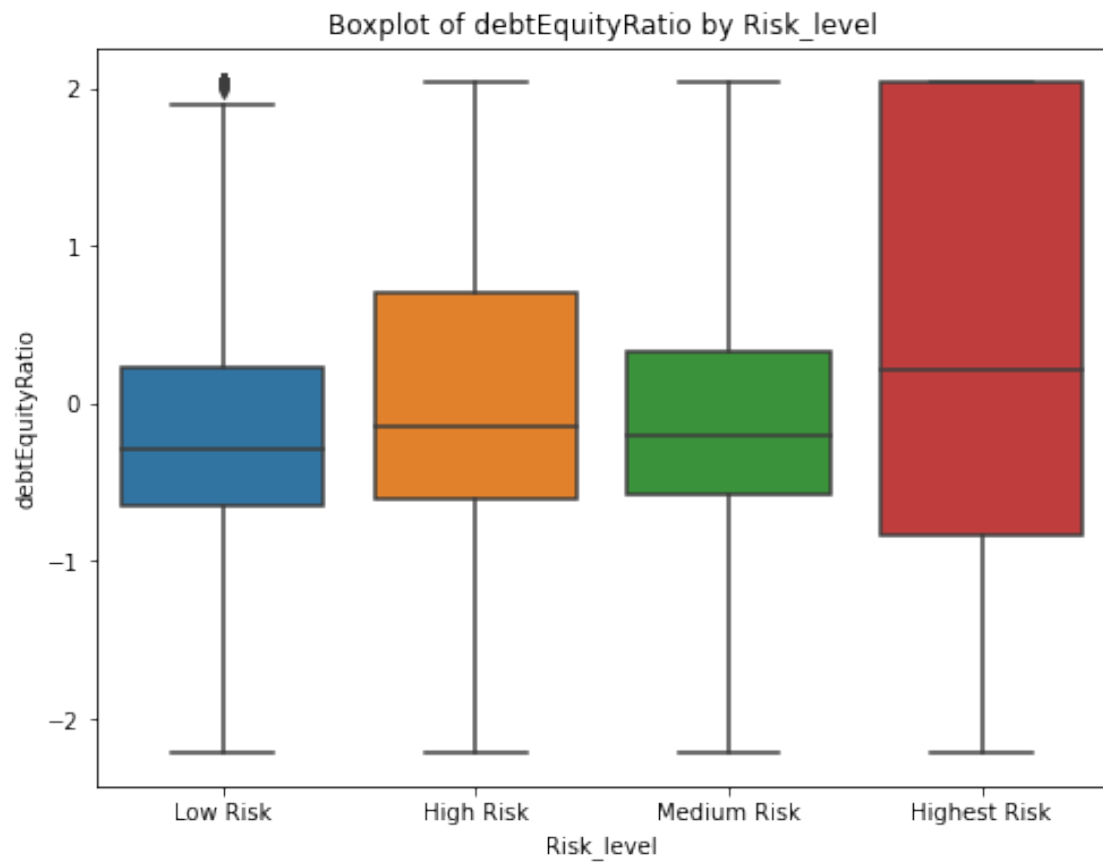


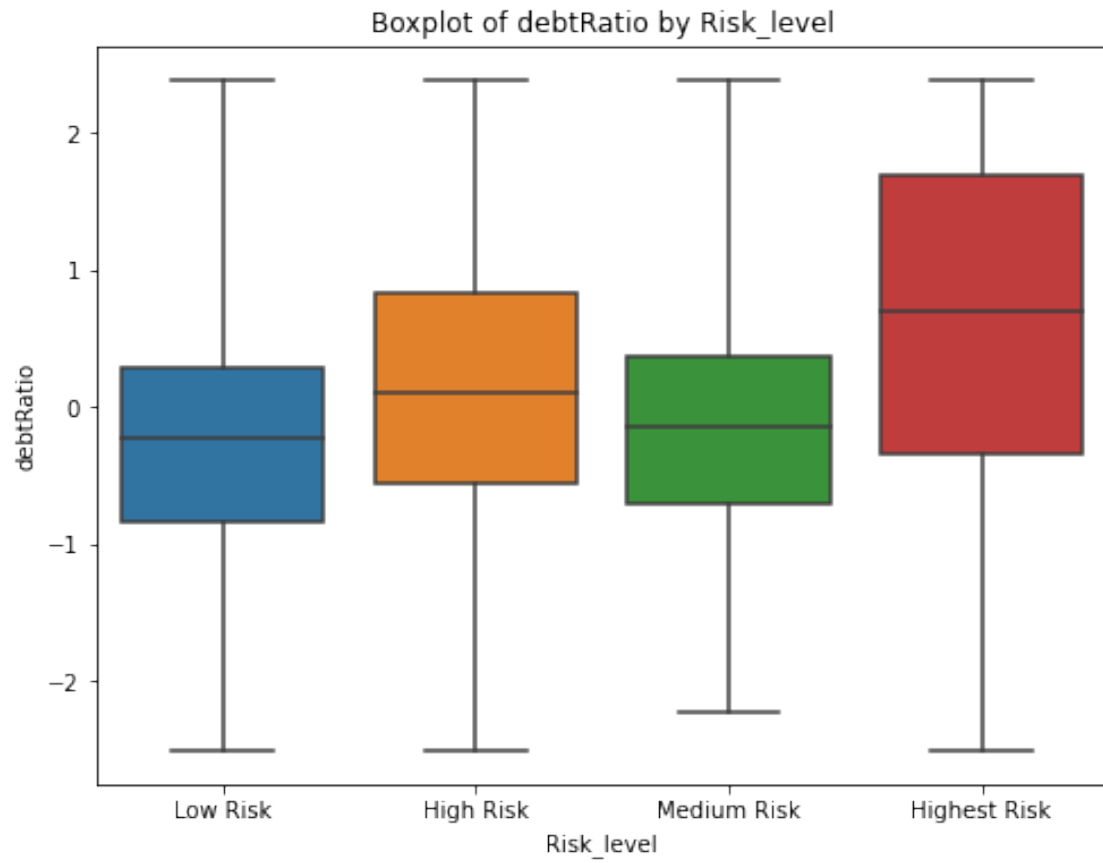


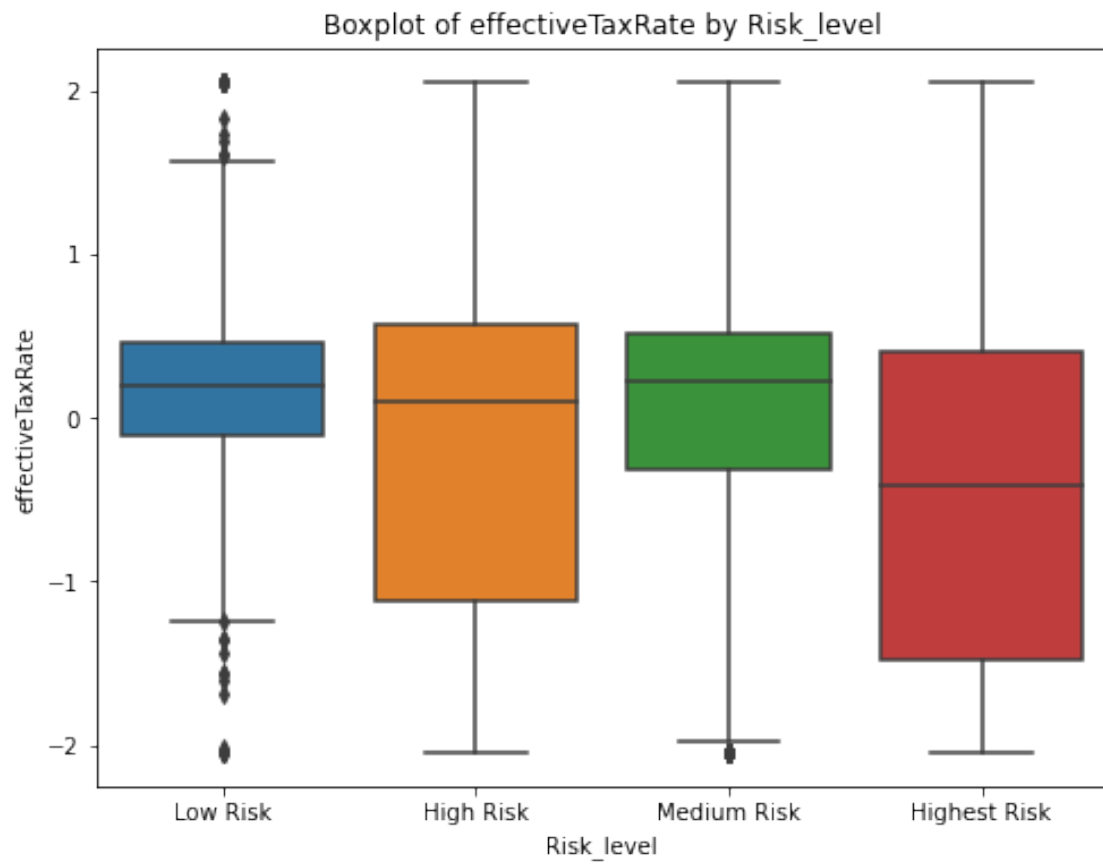


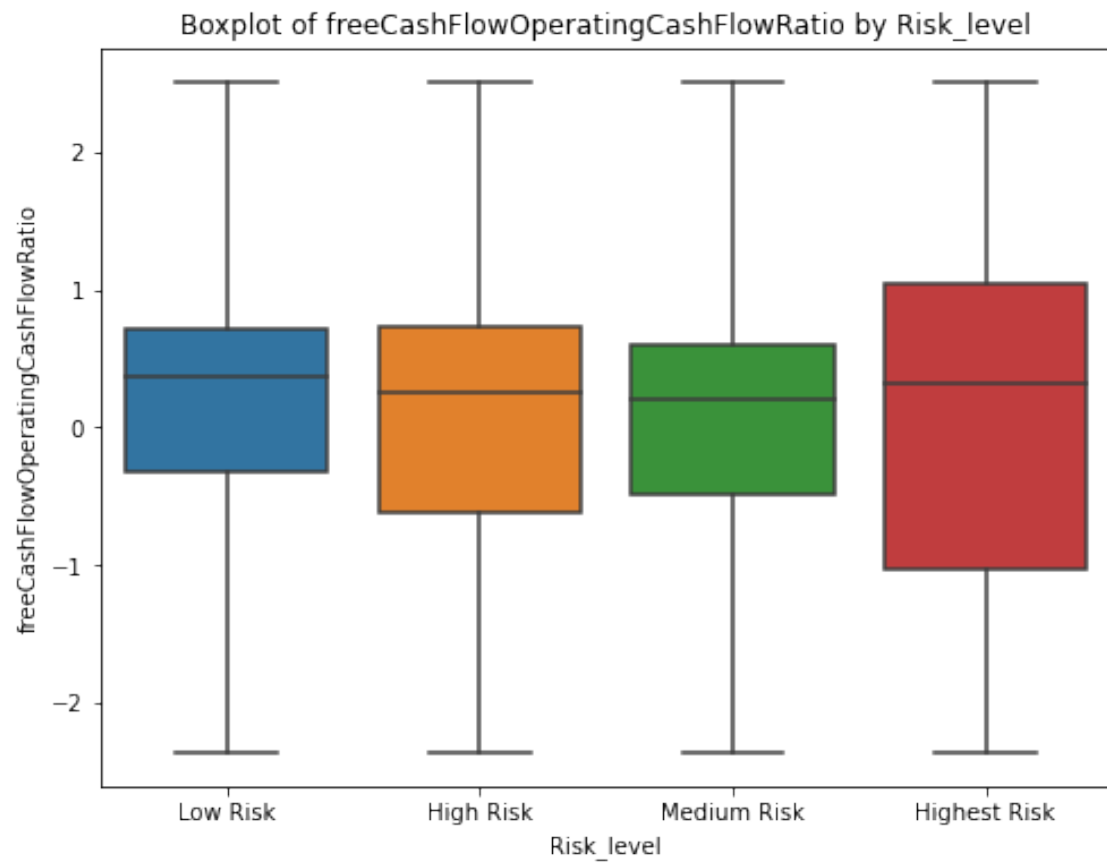


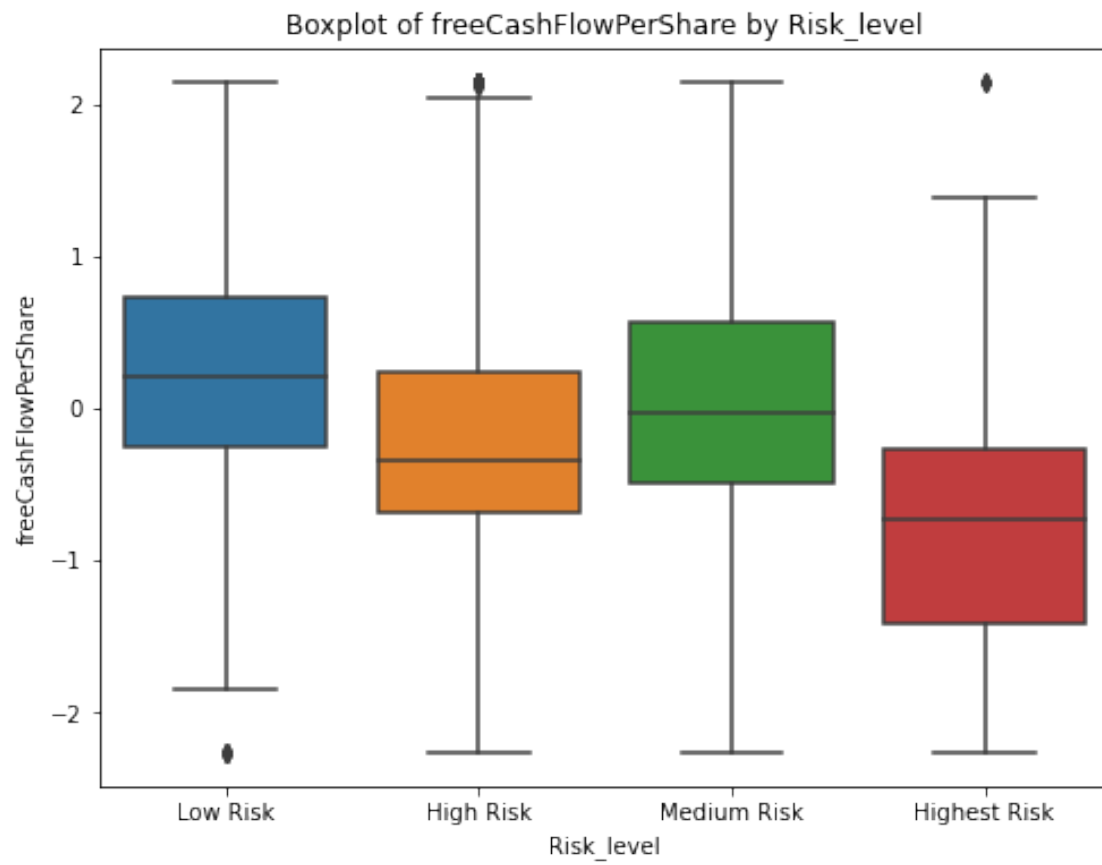


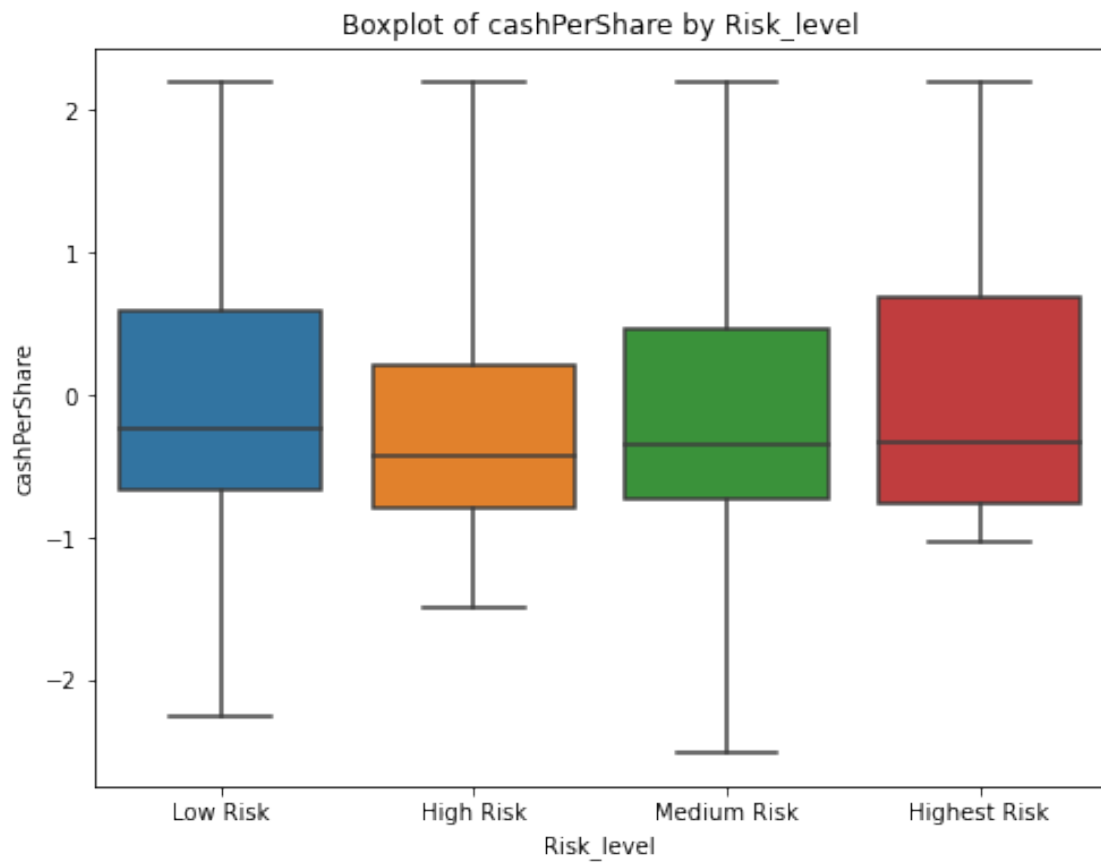


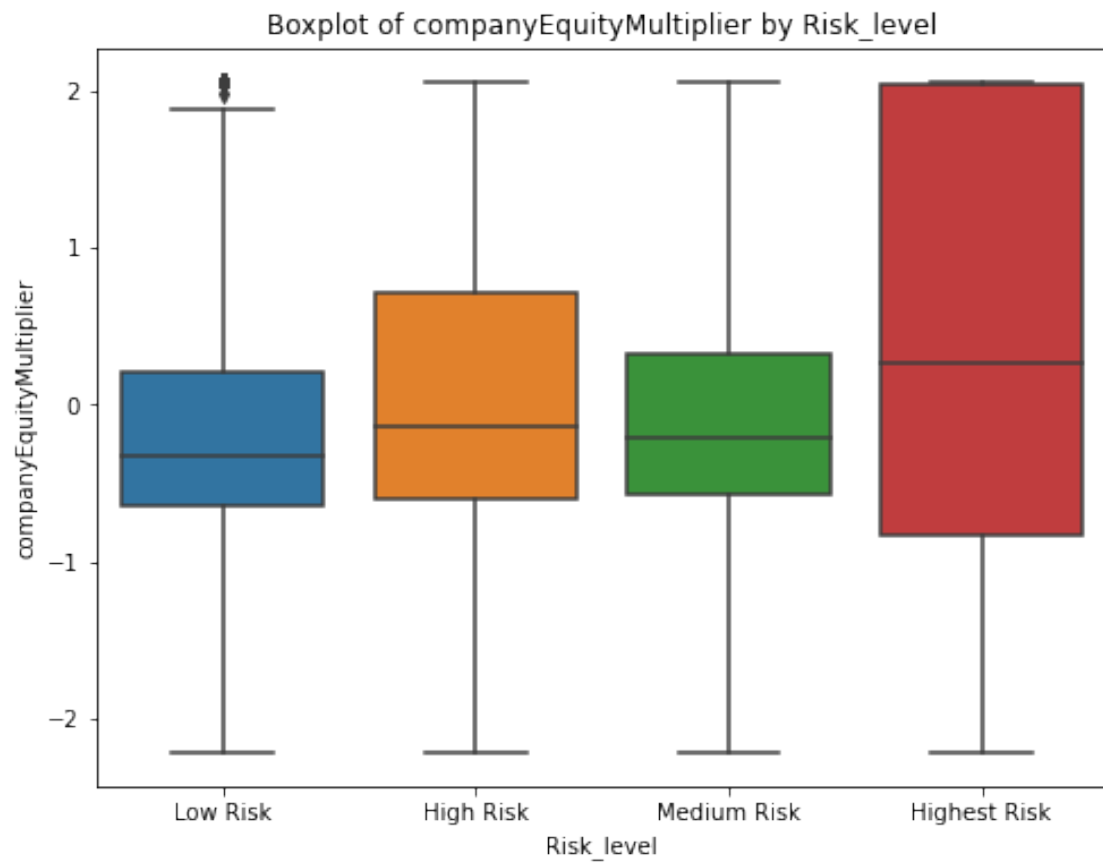


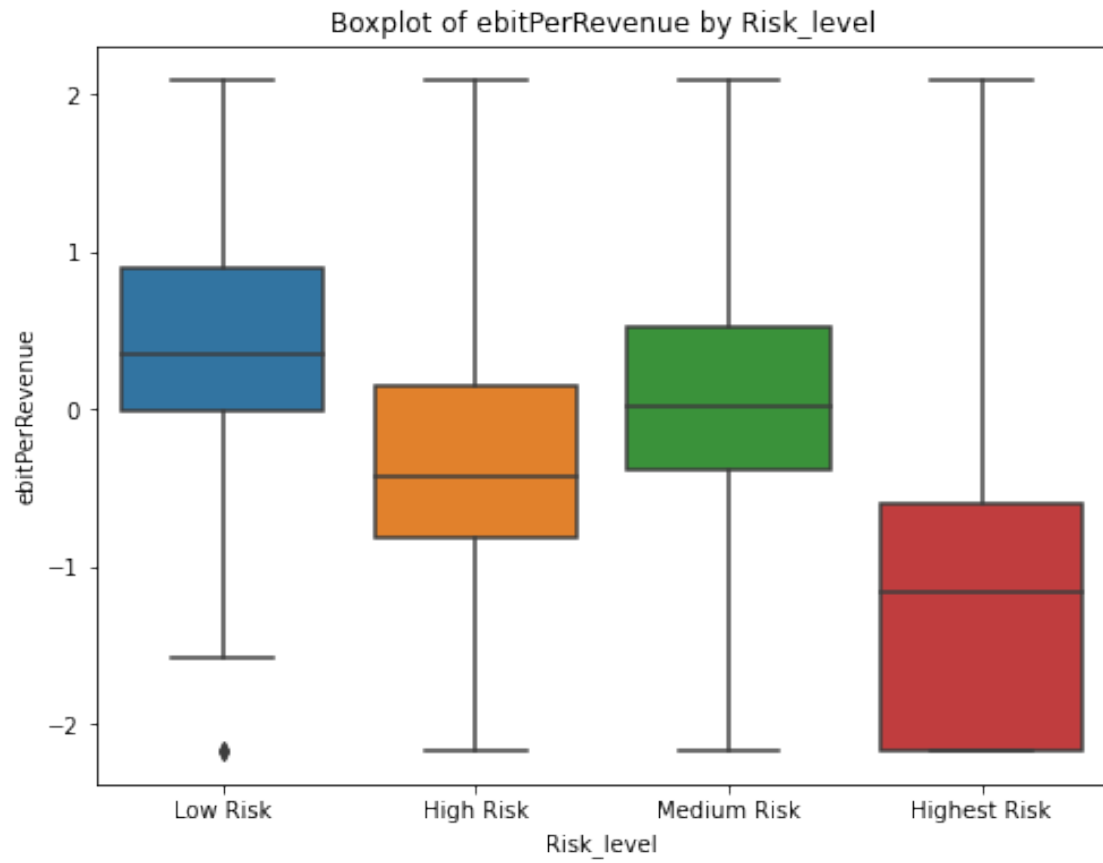


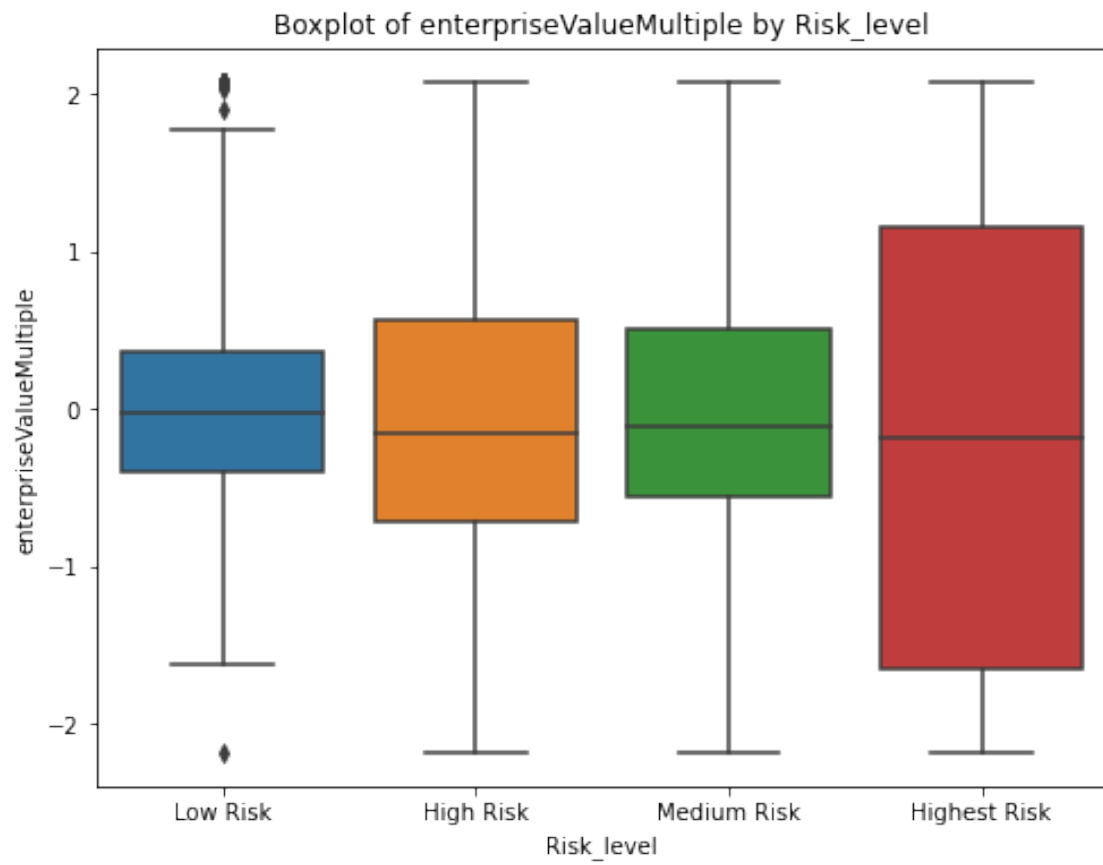


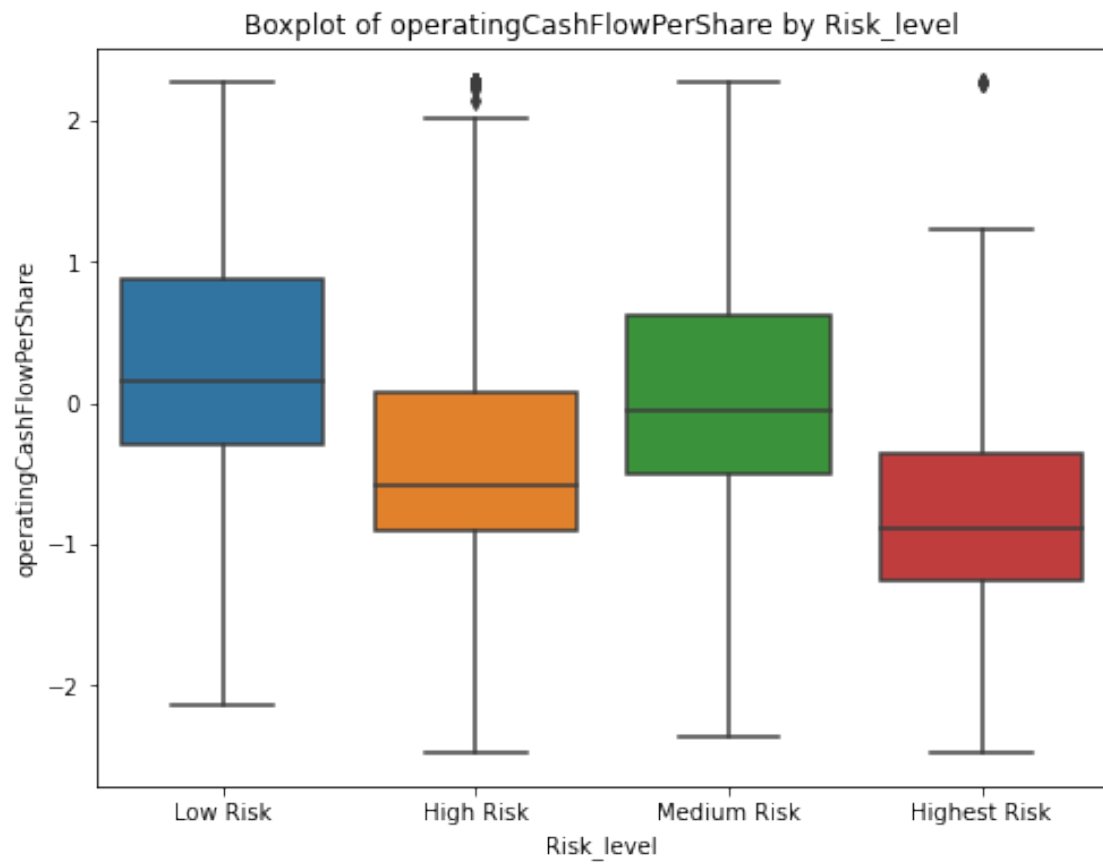


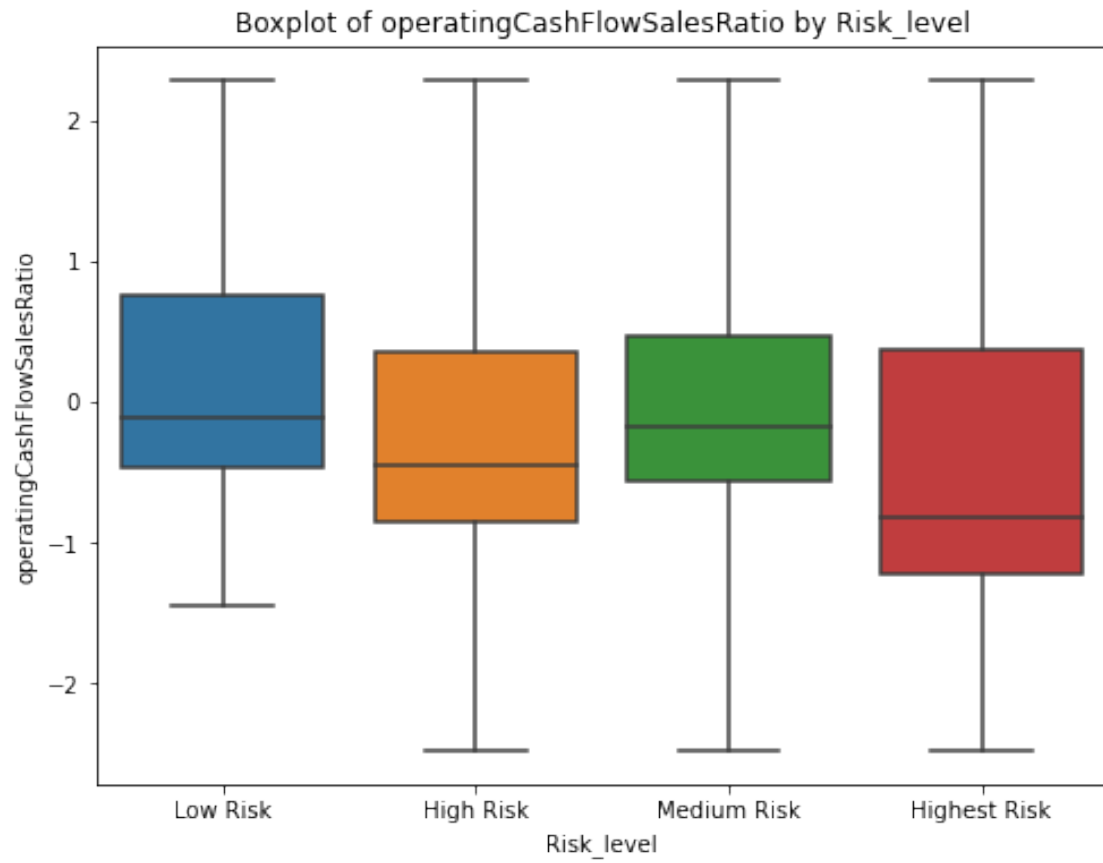


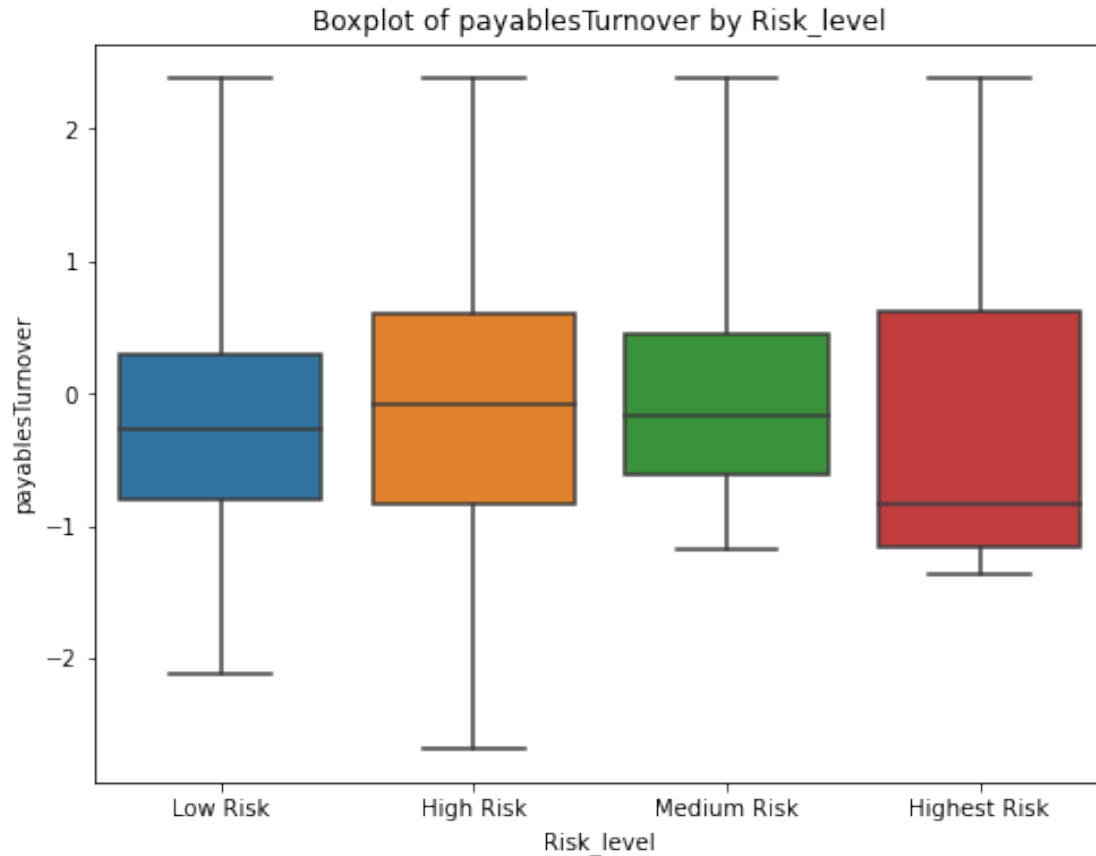












```
[45]: # ['currentRatio', +
# 'quickRatio', +
# 'cashRatio', +
# 'daysOfSalesOutstanding', ?
# 'netProfitMargin', ++
# 'pretaxProfitMargin', ++
# 'grossProfitMargin', ++
# 'operatingProfitMargin', ++
# 'returnOnAssets', ++
# 'returnOnCapitalEmployed', ++
# 'returnOnEquity', ++
# 'assetTurnover', +
# 'fixedAssetTurnover', +?
# 'debtEquityRatio', ++
# 'debtRatio', ++
# 'effectiveTaxRate', ++
# 'freeCashFlowOperatingCashFlowRatio', ?
# 'freeCashFlowPerShare', ++
# 'cashPerShare', +?
# 'companyEquityMultiplier', ++
```

```
# 'ebitPerRevenue',++
# 'enterpriseValueMultiple', ?
# 'operatingCashFlowPerShare',++
# 'operatingCashFlowSalesRatio',++
# 'payablesTurnover' ++
#]

#Based on boxplots: 'netProfitMargin', 'pretaxProfitMargin', (2)
#'grossProfitMargin','operatingProfitMargin','returnOnAssets', (3)
# 'returnOnCapitalEmployed', 'returnOnEquity', 'debtEquityRatio', (3)
# 'debtRatio', 'effectiveTaxRate', 'freeCashFlowPerShare', (3)
# 'companyEquityMultiplier', 'ebitPerRevenue', 'operatingCashFlowPerShare', (3)
# 'operatingCashFlowSalesRatio', 'payablesTurnover' (2) + Sector
# Total: 17/26
```

```
[603]: pred_box = ['netProfitMargin', 'pretaxProfitMargin',␣
↳ 'grossProfitMargin','operatingProfitMargin','returnOnAssets',
↳ 'returnOnCapitalEmployed', 'returnOnEquity', 'debtEquityRatio', 'debtRatio',␣
↳ 'effectiveTaxRate',
↳ 'freeCashFlowPerShare', 'companyEquityMultiplier',
↳ 'ebitPerRevenue', 'operatingCashFlowPerShare',
↳ 'operatingCashFlowSalesRatio', 'payablesTurnover']
#+SECTOR
pred_box_sector= ['netProfitMargin', 'pretaxProfitMargin',␣
↳ 'grossProfitMargin','operatingProfitMargin','returnOnAssets',
↳ 'returnOnCapitalEmployed', 'returnOnEquity', 'debtEquityRatio', 'debtRatio',␣
↳ 'effectiveTaxRate',
↳ 'freeCashFlowPerShare', 'companyEquityMultiplier',
↳ 'ebitPerRevenue', 'operatingCashFlowPerShare',
↳ 'operatingCashFlowSalesRatio', 'payablesTurnover', 'Sector']
```

```
[599]: vif(credit_rating_outl_scaled, pred_box)
```

	feature	VIF
0	netProfitMargin	11.669540
1	pretaxProfitMargin	37.781459
2	grossProfitMargin	1.685899
3	operatingProfitMargin	7.326014
4	returnOnAssets	11.995430
5	returnOnCapitalEmployed	8.709443
6	returnOnEquity	2.336476
7	debtEquityRatio	102.594644
8	debtRatio	1.418940
9	effectiveTaxRate	1.095096
10	freeCashFlowPerShare	1.816288
11	companyEquityMultiplier	100.518008
12	ebitPerRevenue	38.565151

```

13   operatingCashFlowPerShare    1.850446
14  operatingCashFlowSalesRatio    2.046188
15           payablesTurnover      1.586727

```

```
[600]: cor(credit_rating_outl_scaled, pred_box)
```

	netProfitMargin	pretaxProfitMargin \
netProfitMargin	1.000000	0.898153
pretaxProfitMargin	0.898153	1.000000
grossProfitMargin	0.225224	0.226039
operatingProfitMargin	0.868786	0.900086
returnOnAssets	0.755582	0.701867
returnOnCapitalEmployed	0.585488	0.662520
returnOnEquity	0.525543	0.481483
debtEquityRatio	-0.094078	-0.085239
debtRatio	-0.168280	-0.142243
effectiveTaxRate	0.078650	0.128690
freeCashFlowPerShare	0.212631	0.249365
companyEquityMultiplier	-0.095925	-0.085156
ebitPerRevenue	0.884276	0.984009
operatingCashFlowPerShare	0.207209	0.254659
operatingCashFlowSalesRatio	0.432589	0.447026
payablesTurnover	-0.096028	-0.096216

	grossProfitMargin	operatingProfitMargin \
netProfitMargin	0.225224	0.868786
pretaxProfitMargin	0.226039	0.900086
grossProfitMargin	1.000000	0.299462
operatingProfitMargin	0.299462	1.000000
returnOnAssets	-0.004122	0.617132
returnOnCapitalEmployed	-0.053749	0.543895
returnOnEquity	-0.020315	0.420494
debtEquityRatio	0.003641	-0.034722
debtRatio	0.048218	-0.088765
effectiveTaxRate	0.063061	0.155647
freeCashFlowPerShare	0.017095	0.190910
companyEquityMultiplier	0.003464	-0.031203
ebitPerRevenue	0.216260	0.902936
operatingCashFlowPerShare	0.128265	0.188598
operatingCashFlowSalesRatio	0.432852	0.494354
payablesTurnover	-0.541091	-0.124597

	returnOnAssets	returnOnCapitalEmployed \
netProfitMargin	0.755582	0.585488
pretaxProfitMargin	0.701867	0.662520
grossProfitMargin	-0.004122	-0.053749
operatingProfitMargin	0.617132	0.543895
returnOnAssets	1.000000	0.887018

returnOnCapitalEmployed	0.887018	1.000000
returnOnEquity	0.680274	0.616245
debtEquityRatio	-0.097793	-0.058467
debtRatio	-0.124251	-0.059022
effectiveTaxRate	0.131880	0.184837
freeCashFlowPerShare	0.335259	0.364512
companyEquityMultiplier	-0.102530	-0.063422
ebitPerRevenue	0.681974	0.659554
operatingCashFlowPerShare	0.238936	0.260795
operatingCashFlowSalesRatio	0.085074	0.018203
payablesTurnover	0.138222	0.149818

	returnOnEquity	debtEquityRatio	debtRatio	\
netProfitMargin	0.525543	-0.094078	-0.168280	
pretaxProfitMargin	0.481483	-0.085239	-0.142243	
grossProfitMargin	-0.020315	0.003641	0.048218	
operatingProfitMargin	0.420494	-0.034722	-0.088765	
returnOnAssets	0.680274	-0.097793	-0.124251	
returnOnCapitalEmployed	0.616245	-0.058467	-0.059022	
returnOnEquity	1.000000	0.245354	0.063213	
debtEquityRatio	0.245354	1.000000	0.453393	
debtRatio	0.063213	0.453393	1.000000	
effectiveTaxRate	0.059564	-0.040646	-0.036203	
freeCashFlowPerShare	0.242287	-0.051063	-0.099855	
companyEquityMultiplier	0.240397	0.994576	0.433926	
ebitPerRevenue	0.463969	-0.085753	-0.150097	
operatingCashFlowPerShare	0.171138	0.002887	-0.082871	
operatingCashFlowSalesRatio	-0.021566	-0.103118	-0.179817	
payablesTurnover	0.082724	-0.068000	-0.102072	

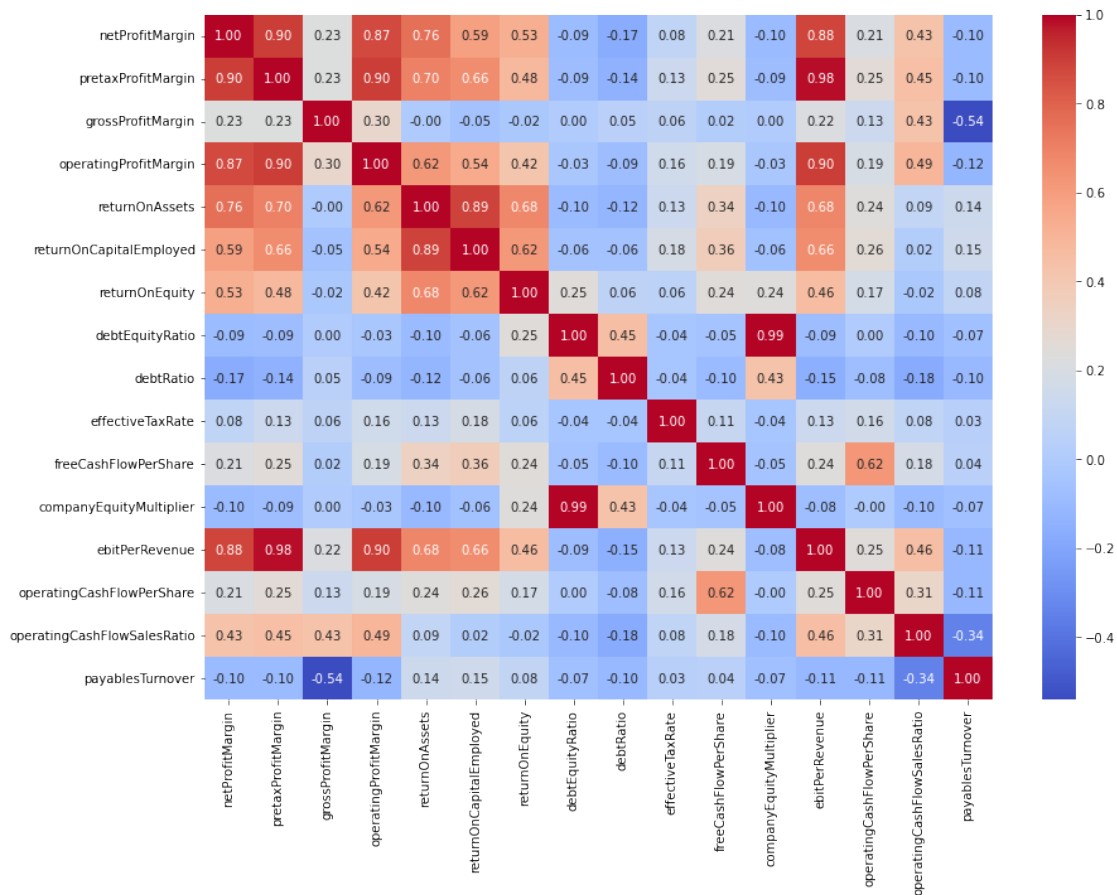
	effectiveTaxRate	freeCashFlowPerShare	\
netProfitMargin	0.078650	0.212631	
pretaxProfitMargin	0.128690	0.249365	
grossProfitMargin	0.063061	0.017095	
operatingProfitMargin	0.155647	0.190910	
returnOnAssets	0.131880	0.335259	
returnOnCapitalEmployed	0.184837	0.364512	
returnOnEquity	0.059564	0.242287	
debtEquityRatio	-0.040646	-0.051063	
debtRatio	-0.036203	-0.099855	
effectiveTaxRate	1.000000	0.106856	
freeCashFlowPerShare	0.106856	1.000000	
companyEquityMultiplier	-0.040219	-0.052820	
ebitPerRevenue	0.133130	0.239990	
operatingCashFlowPerShare	0.159807	0.624753	
operatingCashFlowSalesRatio	0.076522	0.180502	
payablesTurnover	0.033023	0.043334	

	companyEquityMultiplier	ebitPerRevenue \
netProfitMargin	-0.095925	0.884276
pretaxProfitMargin	-0.085156	0.984009
grossProfitMargin	0.003464	0.216260
operatingProfitMargin	-0.031203	0.902936
returnOnAssets	-0.102530	0.681974
returnOnCapitalEmployed	-0.063422	0.659554
returnOnEquity	0.240397	0.463969
debtEquityRatio	0.994576	-0.085753
debtRatio	0.433926	-0.150097
effectiveTaxRate	-0.040219	0.133130
freeCashFlowPerShare	-0.052820	0.239990
companyEquityMultiplier	1.000000	-0.082366
ebitPerRevenue	-0.082366	1.000000
operatingCashFlowPerShare	-0.000698	0.246873
operatingCashFlowSalesRatio	-0.099772	0.457347
payablesTurnover	-0.066155	-0.110682

	operatingCashFlowPerShare \
netProfitMargin	0.207209
pretaxProfitMargin	0.254659
grossProfitMargin	0.128265
operatingProfitMargin	0.188598
returnOnAssets	0.238936
returnOnCapitalEmployed	0.260795
returnOnEquity	0.171138
debtEquityRatio	0.002887
debtRatio	-0.082871
effectiveTaxRate	0.159807
freeCashFlowPerShare	0.624753
companyEquityMultiplier	-0.000698
ebitPerRevenue	0.246873
operatingCashFlowPerShare	1.000000
operatingCashFlowSalesRatio	0.309117
payablesTurnover	-0.108446

	operatingCashFlowSalesRatio	payablesTurnover
netProfitMargin	0.432589	-0.096028
pretaxProfitMargin	0.447026	-0.096216
grossProfitMargin	0.432852	-0.541091
operatingProfitMargin	0.494354	-0.124597
returnOnAssets	0.085074	0.138222
returnOnCapitalEmployed	0.018203	0.149818
returnOnEquity	-0.021566	0.082724
debtEquityRatio	-0.103118	-0.068000
debtRatio	-0.179817	-0.102072
effectiveTaxRate	0.076522	0.033023
freeCashFlowPerShare	0.180502	0.043334

companyEquityMultiplier	-0.099772	-0.066155
ebitPerRevenue	0.457347	-0.110682
operatingCashFlowPerShare	0.309117	-0.108446
operatingCashFlowSalesRatio	1.000000	-0.342826
payablesTurnover	-0.342826	1.000000



```
[601]: class_models_macro_avg(data_inp=credit_rating_outl_scaled, X_cols=X_cols, y_col='Risk_level_encoded', test_size=0.2, num_class = 4)
```

Gradient Boosting Macro-Average Accuracy: 0.5620

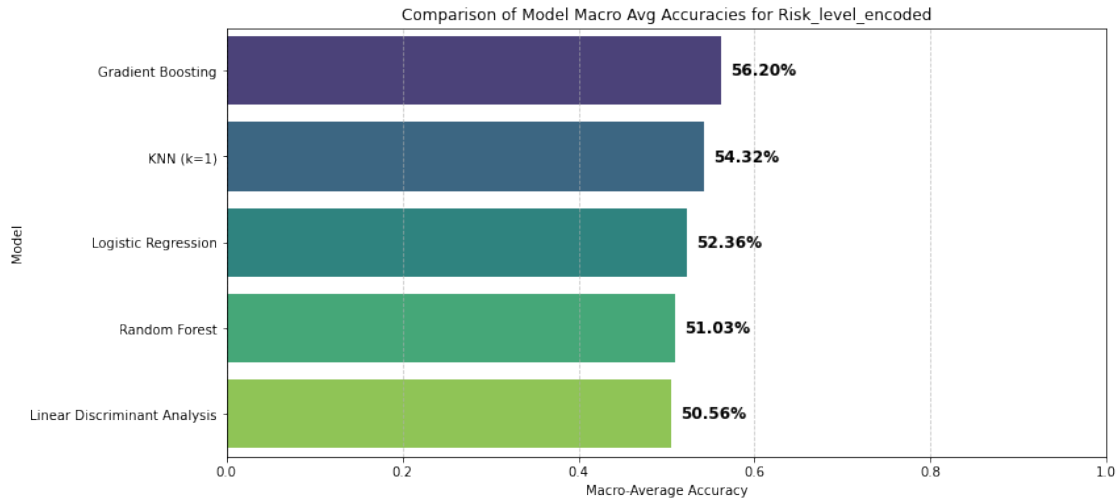
Random Forest Macro-Average Accuracy: 0.5103

Linear Discriminant Analysis Macro-Average Accuracy: 0.5056

Logistic Regression Macro-Average Accuracy: 0.5236

/Users/kamila/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.
warnings.warn(

Best KNN Macro-Average Accuracy (1 neighbors): 0.5432



```
[601]:
```

	Model	Macro Avg Accuracy
0	Gradient Boosting	0.562041
4	KNN (k=1)	0.543175
3	Logistic Regression	0.523632
1	Random Forest	0.510342
2	Linear Discriminant Analysis	0.505647

```
[604]: class_models_macro_avg(data_inp=credit_rating_outl_scaled,
    ↳X_cols=pred_box_sector, y_col = 'Risk_level_encoded', test_size=0.2,
    ↳num_class = 4)
```

Gradient Boosting Macro-Average Accuracy: 0.4853

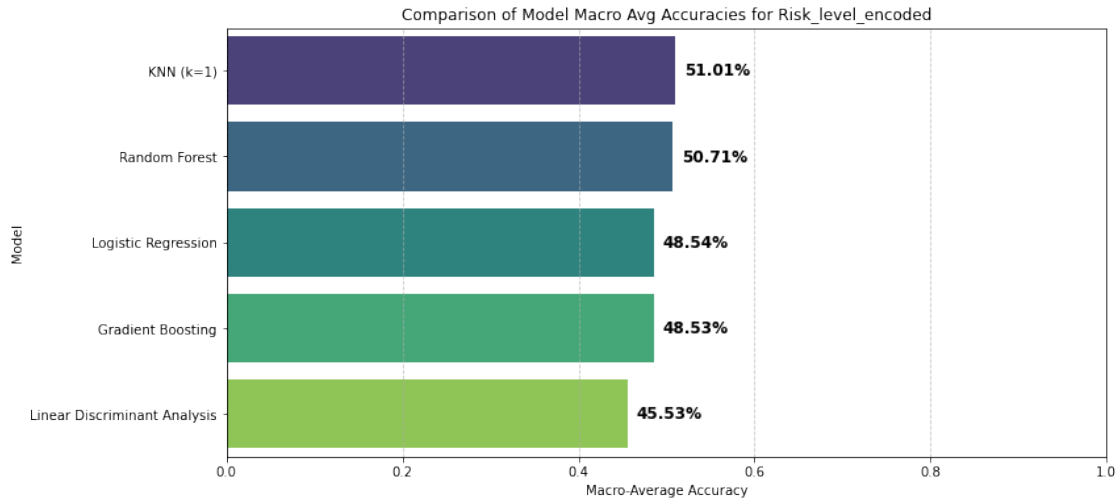
Random Forest Macro-Average Accuracy: 0.5071

Linear Discriminant Analysis Macro-Average Accuracy: 0.4553

Logistic Regression Macro-Average Accuracy: 0.4854

```
/Users/kamila/opt/anaconda3/lib/python3.9/site-
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will
always use 'multinomial'. Leave it to its default value to avoid this warning.
warnings.warn(
```

Best KNN Macro-Average Accuracy (1 neighbors): 0.5101



```
[604]:
```

	Model	Macro Avg Accuracy
4	KNN (k=1)	0.510105
1	Random Forest	0.507055
3	Logistic Regression	0.485357
0	Gradient Boosting	0.485272
2	Linear Discriminant Analysis	0.455325

```
[605]: class_models_macro_f1(data_inp=credit_rating_outl_scaled, X_cols=X_cols,
                             y_col = 'Risk_level_encoded', test_size=0.2, num_class = 4)
→4)
```

Gradient Boosting Macro F1 Score: 0.5637

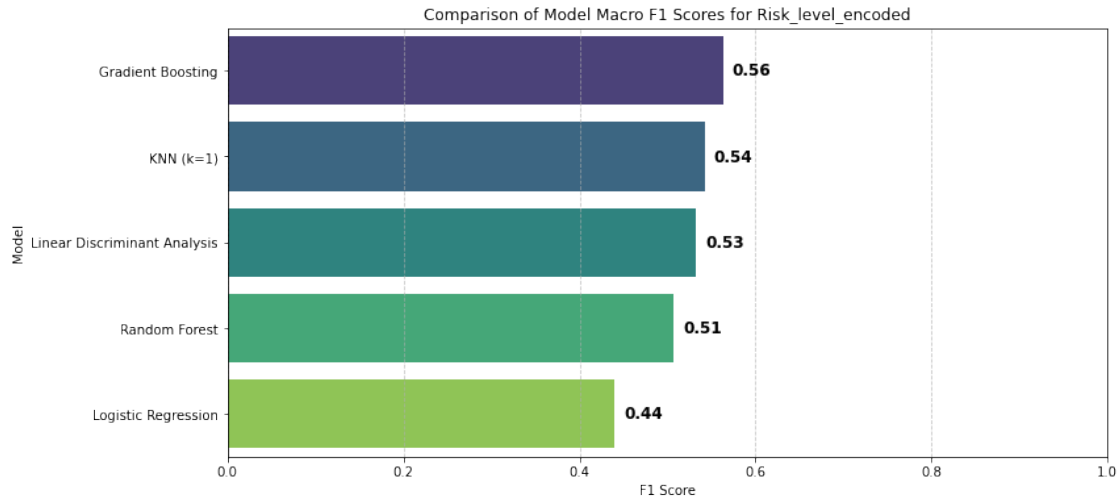
Random Forest Macro F1 Score: 0.5074

Linear Discriminant Analysis Macro F1 Score: 0.5328

Logistic Regression Macro F1 Score: 0.4400

```
/Users/kamila/opt/anaconda3/lib/python3.9/site-
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will
always use 'multinomial'. Leave it to its default value to avoid this warning.
warnings.warn(
```

Best KNN Macro F1 Score (1 neighbors): 0.5421



```
[605]:
      Model  F1 Score
0      Gradient Boosting  0.563669
4              KNN (k=1)  0.542134
2  Linear Discriminant Analysis  0.532848
1              Random Forest  0.507412
3      Logistic Regression  0.439982
```

```
[606]: class_models_macro_f1(data_inp=credit_rating_outl_scaled,
    ↪X_cols=pred_box_sector,
    y_col = 'Risk_level_encoded', test_size=0.2, num_class =
    ↪4)
```

Gradient Boosting Macro F1 Score: 0.4849

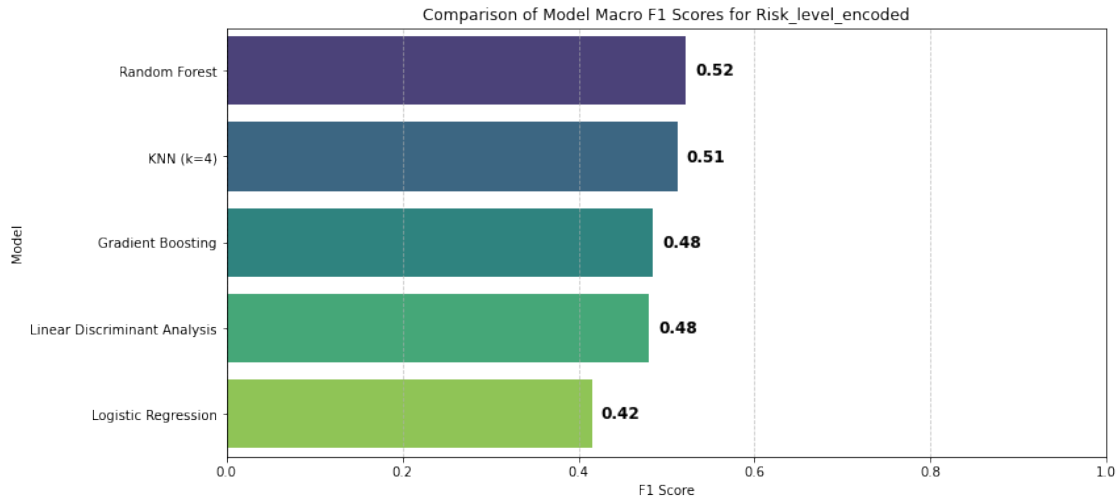
Random Forest Macro F1 Score: 0.5220

Linear Discriminant Analysis Macro F1 Score: 0.4805

Logistic Regression Macro F1 Score: 0.4155

Best KNN Macro F1 Score (4 neighbors): 0.5125

```
/Users/kamila/opt/anaconda3/lib/python3.9/site-
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will
always use 'multinomial'. Leave it to its default value to avoid this warning.
  warnings.warn(
```



```
[606]:
```

	Model	F1 Score
1	Random Forest	0.522032
4	KNN (k=4)	0.512490
0	Gradient Boosting	0.484870
2	Linear Discriminant Analysis	0.480459
3	Logistic Regression	0.415516

```
[693]: import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.feature_selection import SelectKBest, f_classif

def anova_feature_selection(data, target_col, predictor_cols):

    # # Standardize predictors (mean=0, std=1)
    # scaler = StandardScaler()
    # X_scaled = scaler.fit_transform(data[predictor_cols])

    X = data[predictor_cols]
    y = data[target_col]

    # Perform ANOVA F-test
    anova_selector = SelectKBest(score_func=f_classif, k='all')
    anova_selector.fit(X, y)

    # Store results in a DataFrame
    anova_results = pd.DataFrame({
```

```

        'Feature': predictor_cols,
        'F-Score': anova_selector.scores_,
        'p-Value': anova_selector.pvalues_
    })

    # Sort by F-score (higher is better)
    anova_results = anova_results.sort_values(by='F-Score', ascending=False)

    # Print results
    #print(anova_results)

    # Plot feature importance
    plt.figure(figsize=(10, 5))
    ax = sns.barplot(x='F-Score', y='Feature', data=anova_results,
→palette='viridis')

    # Add F-score values on bars
    for index, value in enumerate(anova_results["F-Score"]):
        ax.text(value + 0.1, index, f"{value:.2f}", va='center', fontsize=12,
→fontweight='bold', color='black')

    plt.xlabel("ANOVA F-Score")
    plt.ylabel("Feature")
    plt.title("ANOVA Feature Selection for " + target_col)
    plt.grid(axis='x', linestyle='--', alpha=0.7)
    plt.show()

    return anova_results

def anova_feature_selection_train(data, target_col, predictor_cols, test_size):

    # Split dataset into training and testing while maintaining category
→proportions
    X = data[predictor_cols] # Features
    y = data[target_col]

    X_train, X_test, y_train, y_test = train_test_split(X, y,
        test_size=test_size, random_state=123)

```



```

# Perform ANOVA F-test
anova_selector = SelectKBest(score_func=f_classif, k='all')
anova_selector.fit(X_train, y_train)

# Store results in a DataFrame
anova_results = pd.DataFrame({
    'Feature': predictor_cols,
    'F-Score': anova_selector.scores_,
    'p-Value': anova_selector.pvalues_
})

# Sort by F-score (higher is better)
anova_results = anova_results.sort_values(by='F-Score', ascending=False)

# Print results
#print(anova_results)

# Plot feature importance
plt.figure(figsize=(10, 5))
ax = sns.barplot(x='F-Score', y='Feature', data=anova_results,
→palette='viridis')

# Add F-score values on bars
for index, value in enumerate(anova_results["F-Score"]):
    ax.text(value + 0.1, index, f"{value:.2f}", va='center', fontsize=12,
→fontweight='bold', color='black')

plt.xlabel("ANOVA F-Score")
plt.ylabel("Feature")
plt.title("ANOVA Feature Selection for " + target_col)
plt.grid(axis='x', linestyle='--', alpha=0.7)
plt.show()

return anova_results

```

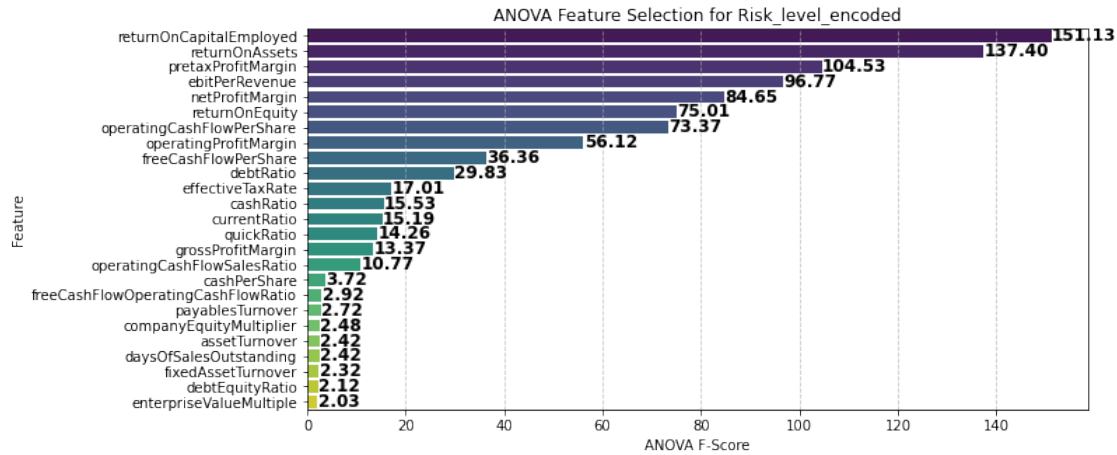
```

[686]: # target_col: ['Rating', 'Risk_level']
predictors=['currentRatio', 'quickRatio', 'cashRatio', 'daysOfSalesOutstanding',
    'netProfitMargin', 'pretaxProfitMargin', 'grossProfitMargin',
    'operatingProfitMargin', 'returnOnAssets', 'returnOnCapitalEmployed',
    'returnOnEquity', 'assetTurnover', 'fixedAssetTurnover',
    'debtEquityRatio', 'debtRatio', 'effectiveTaxRate',
    'freeCashFlowOperatingCashFlowRatio', 'freeCashFlowPerShare',
    'cashPerShare', 'companyEquityMultiplier', 'ebitPerRevenue',

```

```
'enterpriseValueMultiple', 'operatingCashFlowPerShare',  
'operatingCashFlowSalesRatio', 'payablesTurnover']
```

```
anova_feature_selection(credit_rating_outl_scaled, 'Risk_level_encoded',  
→predictors)
```

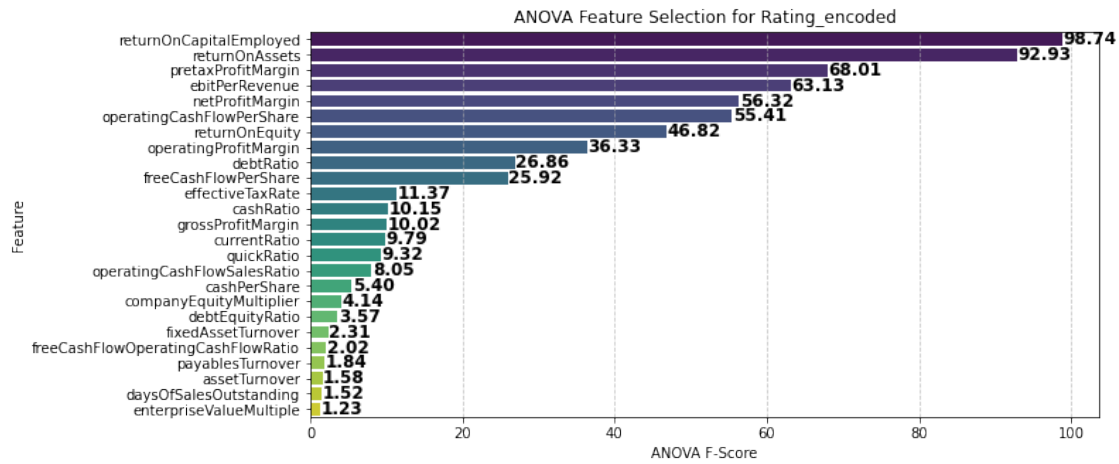


[686] :

	Feature	F-Score	p-Value
9	returnOnCapitalEmployed	151.134079	5.054808e-88
8	returnOnAssets	137.402470	1.003340e-80
5	pretaxProfitMargin	104.525593	9.961106e-63
20	ebitPerRevenue	96.765651	2.283505e-58
4	netProfitMargin	84.649660	1.802408e-51
10	returnOnEquity	75.007243	6.673329e-46
22	operatingCashFlowPerShare	73.372763	5.961117e-45
7	operatingProfitMargin	56.124077	8.665853e-35
17	freeCashFlowPerShare	36.362346	7.358854e-23
14	debtRatio	29.831238	7.488015e-19
15	effectiveTaxRate	17.008405	6.634068e-11
2	cashRatio	15.525699	5.564587e-10
0	currentRatio	15.194885	8.944657e-10
1	quickRatio	14.263494	3.403754e-09
6	grossProfitMargin	13.370870	1.225137e-08
23	operatingCashFlowSalesRatio	10.768888	5.110518e-07
18	cashPerShare	3.722842	1.099557e-02
16	freeCashFlowOperatingCashFlowRatio	2.923130	3.277812e-02
24	payablesTurnover	2.722948	4.293780e-02
19	companyEquityMultiplier	2.479663	5.947251e-02
11	assetTurnover	2.417304	6.462155e-02
3	daysOfSalesOutstanding	2.415658	6.476318e-02
12	fixedAssetTurnover	2.315220	7.399714e-02
13	debtEquityRatio	2.120000	9.571766e-02

21 enterpriseValueMultiple 2.026681 1.081544e-01

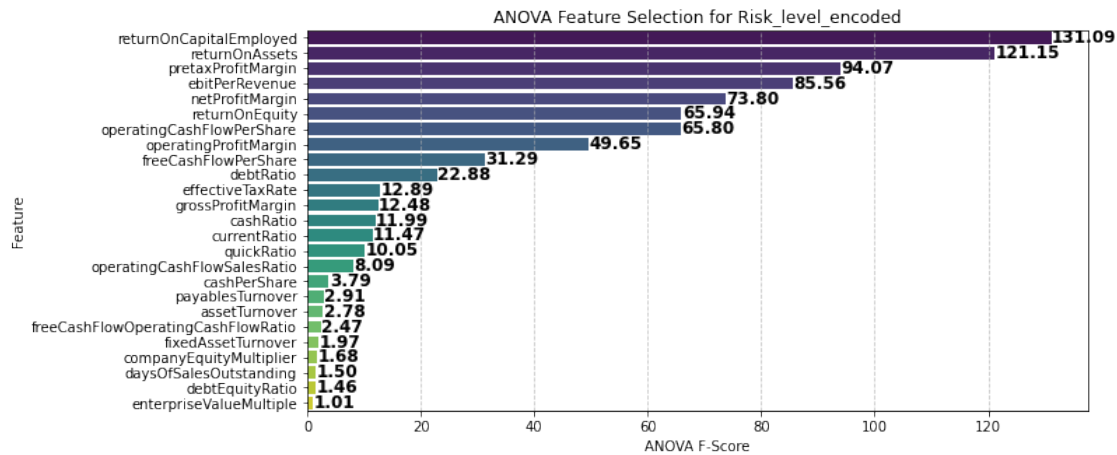
```
[687]: anova_feature_selection(credit_rating_outl_scaled, 'Rating_encoded', predictors)
```



```
[687]:
```

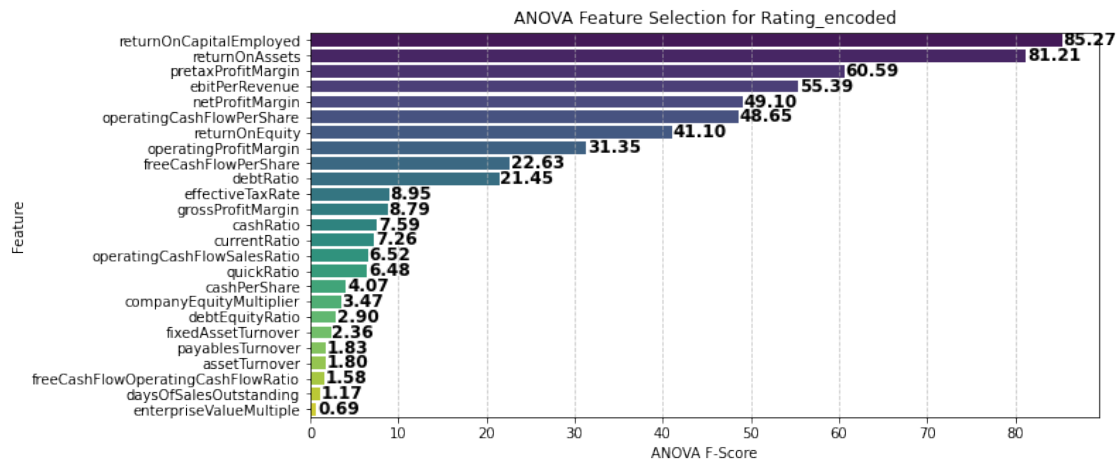
	Feature	F-Score	p-Value
9	returnOnCapitalEmployed	98.737046	6.562855e-93
8	returnOnAssets	92.930910	6.925354e-88
5	pretaxProfitMargin	68.009096	1.220907e-65
20	ebitPerRevenue	63.130668	3.768593e-61
4	netProfitMargin	56.315843	8.394148e-55
22	operatingCashFlowPerShare	55.409281	5.958886e-54
10	returnOnEquity	46.822268	8.256404e-46
7	operatingProfitMargin	36.333914	1.133680e-35
14	debtRatio	26.857909	2.446870e-26
17	freeCashFlowPerShare	25.917775	2.103454e-25
15	effectiveTaxRate	11.371832	7.806740e-11
2	cashRatio	10.145705	1.312740e-09
6	grossProfitMargin	10.016286	1.767287e-09
0	currentRatio	9.788811	2.979433e-09
1	quickRatio	9.322297	8.684004e-09
23	operatingCashFlowSalesRatio	8.046573	1.599665e-07
18	cashPerShare	5.403388	6.097773e-05
19	companyEquityMultiplier	4.142088	9.569147e-04
13	debtEquityRatio	3.566209	3.258201e-03
12	fixedAssetTurnover	2.310653	4.186193e-02
16	freeCashFlowOperatingCashFlowRatio	2.023308	7.250451e-02
24	payablesTurnover	1.838676	1.021435e-01
11	assetTurnover	1.583675	1.613590e-01
3	daysOfSalesOutstanding	1.518958	1.805880e-01
21	enterpriseValueMultiple	1.225537	2.944569e-01

```
[697]: anova_feature_selection_train(credit_rating_outl_scaled, 'Risk_level_encoded',
    ↪predictors, 0.2)
```



	Feature	F-Score	p-Value
9	returnOnCapitalEmployed	131.088626	1.086027e-75
8	returnOnAssets	121.148410	1.692482e-70
5	pretaxProfitMargin	94.073538	6.112895e-56
20	ebitPerRevenue	85.563924	3.107211e-51
4	netProfitMargin	73.799308	1.276702e-44
10	returnOnEquity	65.944321	3.917766e-40
22	operatingCashFlowPerShare	65.802156	4.729107e-40
7	operatingProfitMargin	49.654419	1.214438e-30
17	freeCashFlowPerShare	31.288040	1.231526e-19
14	debtRatio	22.884197	1.697669e-14
15	effectiveTaxRate	12.891186	2.549122e-08
6	grossProfitMargin	12.478987	4.591390e-08
2	cashRatio	11.993641	9.179787e-08
0	currentRatio	11.472727	1.930764e-07
1	quickRatio	10.045033	1.479836e-06
23	operatingCashFlowSalesRatio	8.086149	2.405417e-05
18	cashPerShare	3.788266	1.008274e-02
24	payablesTurnover	2.906913	3.357052e-02
11	assetTurnover	2.783943	3.962402e-02
16	freeCashFlowOperatingCashFlowRatio	2.469873	6.033562e-02
12	fixedAssetTurnover	1.966014	1.171495e-01
19	companyEquityMultiplier	1.680837	1.691829e-01
3	daysOfSalesOutstanding	1.500671	2.125632e-01
13	debtEquityRatio	1.459643	2.237922e-01
21	enterpriseValueMultiple	1.007181	3.886166e-01

```
[698]: anova_feature_selection_train(credit_rating_outl_scaled, 'Rating_encoded',
    ↪predictors, 0.2)
```



```
[698]:
```

	Feature	F-Score	p-Value
9	returnOnCapitalEmployed	85.271384	2.671000e-79
8	returnOnAssets	81.205208	7.621873e-76
5	pretaxProfitMargin	60.593100	8.827249e-58
20	ebitPerRevenue	55.386960	4.557659e-53
4	netProfitMargin	49.101499	2.706700e-47
22	operatingCashFlowPerShare	48.654221	7.027883e-47
10	returnOnEquity	41.099750	8.259520e-40
7	operatingProfitMargin	31.351690	1.729605e-30
17	freeCashFlowPerShare	22.628506	5.735699e-22
14	debtRatio	21.446327	8.412534e-21
15	effectiveTaxRate	8.949856	2.151251e-08
6	grossProfitMargin	8.786715	3.117233e-08
2	cashRatio	7.590451	4.686296e-07
0	currentRatio	7.255752	9.967159e-07
23	operatingCashFlowSalesRatio	6.518051	5.220542e-06
1	quickRatio	6.478120	5.708181e-06
18	cashPerShare	4.071340	1.124710e-03
19	companyEquityMultiplier	3.471769	3.999884e-03
13	debtEquityRatio	2.901403	1.297138e-02
12	fixedAssetTurnover	2.363837	3.784465e-02
24	payablesTurnover	1.828528	1.041965e-01
11	assetTurnover	1.800750	1.096086e-01
16	freeCashFlowOperatingCashFlowRatio	1.582997	1.616928e-01
3	daysOfSalesOutstanding	1.172364	3.204416e-01
21	enterpriseValueMultiple	0.694749	6.274448e-01

```
[ ]: returnOnCapitalEmployed (1)(1) +
returnOnAssets (2)(2) +
pretaxProfitMargin (3)(3) +
ebitPerRevenue (4)(4) +
netProfitMargin (5)(5) +
returnOnEquity (6)(7) +
operatingProfitMargin (8)(8)I +
operatingCashFlowPerShare (7)(6)+
debtRatio (10)(9)+
freeCashFlowPerShare (9)(10)+
effectiveTaxRate (11)(11)+
quickRatio (14)(15)+
operatingCashFlowSalesRatio (16)(16)+
currentRatio (13)(14)+
cashRatio (12)(12)+
grossProfitMargin (15)(13)+

freeCashFlowOperatingCashFlowRatio (18)()
companyEquityMultiplier ()(18)
debtEquityRatio ()(19)
cashPerShare (17)(17)
Sector
fixedAssetTurnover (20)

#anova_no_training
payablesTurnover(19)

#remove
enterpriseValueMultiple
assetTurnover
daysOfSalesOutstanding
```

```
[304]: pred_anova_common = ['returnOnCapitalEmployed', 'returnOnAssets',
↳ 'pretaxProfitMargin', 'ebitPerRevenue',
'netProfitMargin', 'returnOnEquity', 'operatingProfitMargin',
↳ 'operatingCashFlowPerShare',
'debtRatio', 'freeCashFlowPerShare', 'effectiveTaxRate', 'quickRatio',
↳ 'operatingCashFlowSalesRatio',
'currentRatio', 'cashRatio', 'grossProfitMargin' ]
```

```
[307]: vif(credit_rating_outl_scaled, pred_anova_common, '/Users/kamila/Desktop/
↳ Fidelity final project/vif_table_pred_anova_common.png')
```

```
[307]:
```

	feature	VIF
0	returnOnCapitalEmployed	8.88
1	returnOnAssets	11.87

```

2      pretaxProfitMargin  37.10
3      ebitPerRevenue     37.14
4      netProfitMargin    11.53
5      returnOnEquity     2.03
6      operatingProfitMargin  7.17
7      operatingCashFlowPerShare  1.89
8      debtRatio          1.25
9      freeCashFlowPerShare  1.85
10     effectiveTaxRate    1.10
11     quickRatio          2.68
12     operatingCashFlowSalesRatio  2.15
13     currentRatio        3.37
14     cashRatio           2.35
15     grossProfitMargin   1.38

```

feature	VIF
returnOnCapitalEmployed	8.88
returnOnAssets	11.87
pretaxProfitMargin	37.1
ebitPerRevenue	37.14
netProfitMargin	11.53
returnOnEquity	2.03
operatingProfitMargin	7.17
operatingCashFlowPerShare	1.89
debtRatio	1.25
freeCashFlowPerShare	1.85
effectiveTaxRate	1.1
quickRatio	2.68
operatingCashFlowSalesRatio	2.15
currentRatio	3.37
cashRatio	2.35
grossProfitMargin	1.38

```
[306]: cor(credit_rating_outl_scaled, pred_anova_common)
```

```

returnOnCapitalEmployed  returnOnAssets  \

```

returnOnCapitalEmployed	1.000000	0.887018
returnOnAssets	0.887018	1.000000
pretaxProfitMargin	0.662520	0.701867
ebitPerRevenue	0.659554	0.681974
netProfitMargin	0.585488	0.755582
returnOnEquity	0.616245	0.680274
operatingProfitMargin	0.543895	0.617132
operatingCashFlowPerShare	0.260795	0.238936
debtRatio	-0.059022	-0.124251
freeCashFlowPerShare	0.364512	0.335259
effectiveTaxRate	0.184837	0.131880
quickRatio	-0.032124	0.038224
operatingCashFlowSalesRatio	0.018203	0.085074
currentRatio	-0.033028	0.039200
cashRatio	-0.024113	0.043907
grossProfitMargin	-0.053749	-0.004122

	pretaxProfitMargin	ebitPerRevenue \
returnOnCapitalEmployed	0.662520	0.659554
returnOnAssets	0.701867	0.681974
pretaxProfitMargin	1.000000	0.984009
ebitPerRevenue	0.984009	1.000000
netProfitMargin	0.898153	0.884276
returnOnEquity	0.481483	0.463969
operatingProfitMargin	0.900086	0.902936
operatingCashFlowPerShare	0.254659	0.246873
debtRatio	-0.142243	-0.150097
freeCashFlowPerShare	0.249365	0.239990
effectiveTaxRate	0.128690	0.133130
quickRatio	0.048278	0.041701
operatingCashFlowSalesRatio	0.447026	0.457347
currentRatio	-0.036303	-0.049665
cashRatio	0.011482	0.007996
grossProfitMargin	0.226039	0.216260

	netProfitMargin	returnOnEquity \
returnOnCapitalEmployed	0.585488	0.616245
returnOnAssets	0.755582	0.680274
pretaxProfitMargin	0.898153	0.481483
ebitPerRevenue	0.884276	0.463969
netProfitMargin	1.000000	0.525543
returnOnEquity	0.525543	1.000000
operatingProfitMargin	0.868786	0.420494
operatingCashFlowPerShare	0.207209	0.171138
debtRatio	-0.168280	0.063213
freeCashFlowPerShare	0.212631	0.242287
effectiveTaxRate	0.078650	0.059564
quickRatio	0.071742	-0.041947

operatingCashFlowSalesRatio	0.432589	-0.021566
currentRatio	-0.007933	-0.084788
cashRatio	0.058272	-0.040515
grossProfitMargin	0.225224	-0.020315

	operatingProfitMargin	operatingCashFlowPerShare	\
returnOnCapitalEmployed	0.543895	0.260795	
returnOnAssets	0.617132	0.238936	
pretaxProfitMargin	0.900086	0.254659	
ebitPerRevenue	0.902936	0.246873	
netProfitMargin	0.868786	0.207209	
returnOnEquity	0.420494	0.171138	
operatingProfitMargin	1.000000	0.188598	
operatingCashFlowPerShare	0.188598	1.000000	
debtRatio	-0.088765	-0.082871	
freeCashFlowPerShare	0.190910	0.624753	
effectiveTaxRate	0.155647	0.159807	
quickRatio	0.071856	-0.107110	
operatingCashFlowSalesRatio	0.494354	0.309117	
currentRatio	-0.041878	-0.166358	
cashRatio	0.041486	-0.141165	
grossProfitMargin	0.299462	0.128265	

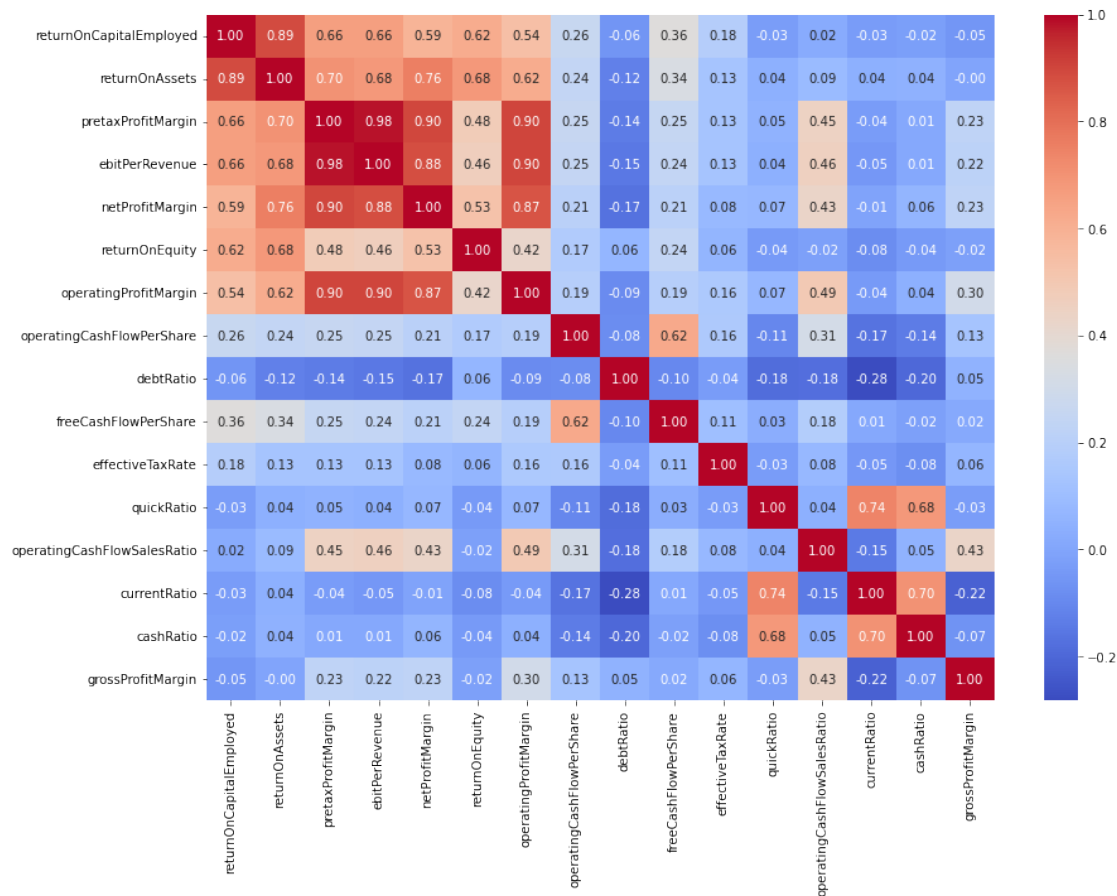
	debtRatio	freeCashFlowPerShare	\
returnOnCapitalEmployed	-0.059022	0.364512	
returnOnAssets	-0.124251	0.335259	
pretaxProfitMargin	-0.142243	0.249365	
ebitPerRevenue	-0.150097	0.239990	
netProfitMargin	-0.168280	0.212631	
returnOnEquity	0.063213	0.242287	
operatingProfitMargin	-0.088765	0.190910	
operatingCashFlowPerShare	-0.082871	0.624753	
debtRatio	1.000000	-0.099855	
freeCashFlowPerShare	-0.099855	1.000000	
effectiveTaxRate	-0.036203	0.106856	
quickRatio	-0.183789	0.029721	
operatingCashFlowSalesRatio	-0.179817	0.180502	
currentRatio	-0.282656	0.011959	
cashRatio	-0.201487	-0.018395	
grossProfitMargin	0.048218	0.017095	

	effectiveTaxRate	quickRatio	\
returnOnCapitalEmployed	0.184837	-0.032124	
returnOnAssets	0.131880	0.038224	
pretaxProfitMargin	0.128690	0.048278	
ebitPerRevenue	0.133130	0.041701	
netProfitMargin	0.078650	0.071742	
returnOnEquity	0.059564	-0.041947	

operatingProfitMargin	0.155647	0.071856
operatingCashFlowPerShare	0.159807	-0.107110
debtRatio	-0.036203	-0.183789
freeCashFlowPerShare	0.106856	0.029721
effectiveTaxRate	1.000000	-0.034838
quickRatio	-0.034838	1.000000
operatingCashFlowSalesRatio	0.076522	0.038236
currentRatio	-0.052083	0.744222
cashRatio	-0.080134	0.683437
grossProfitMargin	0.063061	-0.029425

	operatingCashFlowSalesRatio	currentRatio \
returnOnCapitalEmployed	0.018203	-0.033028
returnOnAssets	0.085074	0.039200
pretaxProfitMargin	0.447026	-0.036303
ebitPerRevenue	0.457347	-0.049665
netProfitMargin	0.432589	-0.007933
returnOnEquity	-0.021566	-0.084788
operatingProfitMargin	0.494354	-0.041878
operatingCashFlowPerShare	0.309117	-0.166358
debtRatio	-0.179817	-0.282656
freeCashFlowPerShare	0.180502	0.011959
effectiveTaxRate	0.076522	-0.052083
quickRatio	0.038236	0.744222
operatingCashFlowSalesRatio	1.000000	-0.147103
currentRatio	-0.147103	1.000000
cashRatio	0.048369	0.701051
grossProfitMargin	0.432852	-0.219900

	cashRatio	grossProfitMargin
returnOnCapitalEmployed	-0.024113	-0.053749
returnOnAssets	0.043907	-0.004122
pretaxProfitMargin	0.011482	0.226039
ebitPerRevenue	0.007996	0.216260
netProfitMargin	0.058272	0.225224
returnOnEquity	-0.040515	-0.020315
operatingProfitMargin	0.041486	0.299462
operatingCashFlowPerShare	-0.141165	0.128265
debtRatio	-0.201487	0.048218
freeCashFlowPerShare	-0.018395	0.017095
effectiveTaxRate	-0.080134	0.063061
quickRatio	0.683437	-0.029425
operatingCashFlowSalesRatio	0.048369	0.432852
currentRatio	0.701051	-0.219900
cashRatio	1.000000	-0.066873
grossProfitMargin	-0.066873	1.000000



```
[322]: #removed 'returnOnAssets', 'ebitPerRevenue', 'operatingProfitMargin',
        ↪ 'netProfitMargin'
```

```
pred_anova_common2 = ['returnOnCapitalEmployed', 'pretaxProfitMargin',
        ↪ 'returnOnEquity', 'operatingCashFlowPerShare',
        'debtRatio', 'freeCashFlowPerShare', 'effectiveTaxRate', 'quickRatio',
        ↪ 'operatingCashFlowSalesRatio',
        'currentRatio', 'cashRatio', 'grossProfitMargin' ]
```

```
[315]: cor(credit_rating_outl_scaled, pred_anova_common2)
```

	returnOnCapitalEmployed	pretaxProfitMargin	\
returnOnCapitalEmployed	1.000000	0.662520	
pretaxProfitMargin	0.662520	1.000000	
returnOnEquity	0.616245	0.481483	
operatingCashFlowPerShare	0.260795	0.254659	
debtRatio	-0.059022	-0.142243	

freeCashFlowPerShare	0.364512	0.249365
effectiveTaxRate	0.184837	0.128690
quickRatio	-0.032124	0.048278
operatingCashFlowSalesRatio	0.018203	0.447026
currentRatio	-0.033028	-0.036303
cashRatio	-0.024113	0.011482
grossProfitMargin	-0.053749	0.226039

	returnOnEquity	operatingCashFlowPerShare \
returnOnCapitalEmployed	0.616245	0.260795
pretaxProfitMargin	0.481483	0.254659
returnOnEquity	1.000000	0.171138
operatingCashFlowPerShare	0.171138	1.000000
debtRatio	0.063213	-0.082871
freeCashFlowPerShare	0.242287	0.624753
effectiveTaxRate	0.059564	0.159807
quickRatio	-0.041947	-0.107110
operatingCashFlowSalesRatio	-0.021566	0.309117
currentRatio	-0.084788	-0.166358
cashRatio	-0.040515	-0.141165
grossProfitMargin	-0.020315	0.128265

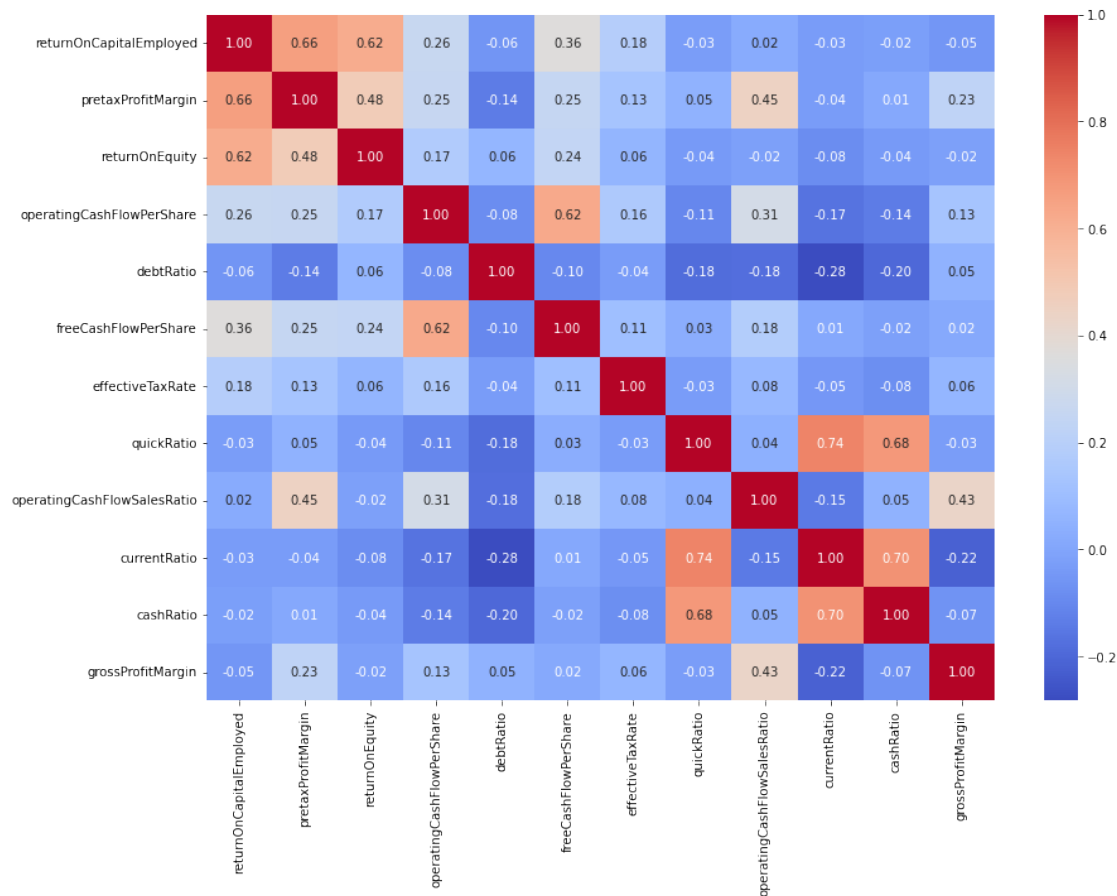
	debtRatio	freeCashFlowPerShare \
returnOnCapitalEmployed	-0.059022	0.364512
pretaxProfitMargin	-0.142243	0.249365
returnOnEquity	0.063213	0.242287
operatingCashFlowPerShare	-0.082871	0.624753
debtRatio	1.000000	-0.099855
freeCashFlowPerShare	-0.099855	1.000000
effectiveTaxRate	-0.036203	0.106856
quickRatio	-0.183789	0.029721
operatingCashFlowSalesRatio	-0.179817	0.180502
currentRatio	-0.282656	0.011959
cashRatio	-0.201487	-0.018395
grossProfitMargin	0.048218	0.017095

	effectiveTaxRate	quickRatio \
returnOnCapitalEmployed	0.184837	-0.032124
pretaxProfitMargin	0.128690	0.048278
returnOnEquity	0.059564	-0.041947
operatingCashFlowPerShare	0.159807	-0.107110
debtRatio	-0.036203	-0.183789
freeCashFlowPerShare	0.106856	0.029721
effectiveTaxRate	1.000000	-0.034838
quickRatio	-0.034838	1.000000
operatingCashFlowSalesRatio	0.076522	0.038236
currentRatio	-0.052083	0.744222
cashRatio	-0.080134	0.683437

grossProfitMargin	0.063061	-0.029425
-------------------	----------	-----------

	operatingCashFlowSalesRatio	currentRatio \
returnOnCapitalEmployed	0.018203	-0.033028
pretaxProfitMargin	0.447026	-0.036303
returnOnEquity	-0.021566	-0.084788
operatingCashFlowPerShare	0.309117	-0.166358
debtRatio	-0.179817	-0.282656
freeCashFlowPerShare	0.180502	0.011959
effectiveTaxRate	0.076522	-0.052083
quickRatio	0.038236	0.744222
operatingCashFlowSalesRatio	1.000000	-0.147103
currentRatio	-0.147103	1.000000
cashRatio	0.048369	0.701051
grossProfitMargin	0.432852	-0.219900

	cashRatio	grossProfitMargin
returnOnCapitalEmployed	-0.024113	-0.053749
pretaxProfitMargin	0.011482	0.226039
returnOnEquity	-0.040515	-0.020315
operatingCashFlowPerShare	-0.141165	0.128265
debtRatio	-0.201487	0.048218
freeCashFlowPerShare	-0.018395	0.017095
effectiveTaxRate	-0.080134	0.063061
quickRatio	0.683437	-0.029425
operatingCashFlowSalesRatio	0.048369	0.432852
currentRatio	0.701051	-0.219900
cashRatio	1.000000	-0.066873
grossProfitMargin	-0.066873	1.000000



```
[323]: vif(credit_rating_outl_scaled,pred_anova_common2, '/Users/kamila/Desktop/
↳Fidelity final project/vif_table_pred_anova_common_red.png')
```

```
[323]:
```

	feature	VIF
0	returnOnCapitalEmployed	2.92
1	pretaxProfitMargin	2.97
2	returnOnEquity	1.73
3	operatingCashFlowPerShare	1.86
4	debtRatio	1.19
5	freeCashFlowPerShare	1.84
6	effectiveTaxRate	1.07
7	quickRatio	2.67
8	operatingCashFlowSalesRatio	2.05
9	currentRatio	3.31
10	cashRatio	2.34
11	grossProfitMargin	1.34

feature	VIF
returnOnCapitalEmployed	2.92
pretaxProfitMargin	2.97
returnOnEquity	1.73
operatingCashFlowPerShare	1.86
debtRatio	1.19
freeCashFlowPerShare	1.84
effectiveTaxRate	1.07
quickRatio	2.67
operatingCashFlowSalesRatio	2.05
currentRatio	3.31
cashRatio	2.34
grossProfitMargin	1.34

```
[369]: pred_anova_common2 = ['returnOnCapitalEmployed', 'pretaxProfitMargin',
    ↪ 'returnOnEquity', 'operatingCashFlowPerShare',
    'debtRatio', 'freeCashFlowPerShare', 'effectiveTaxRate', 'quickRatio',
    ↪ 'operatingCashFlowSalesRatio',
    'currentRatio', 'cashRatio', 'grossProfitMargin', 'Sector' ]

class_models_macro_f1(data_inp = credit_rating_outl_scaled, X_cols = X_cols ,
    y_col = 'Risk_level_encoded', test_size=0.2, num_class =
    ↪ 4)
```

Gradient Boosting Macro F1 Score: 0.5637

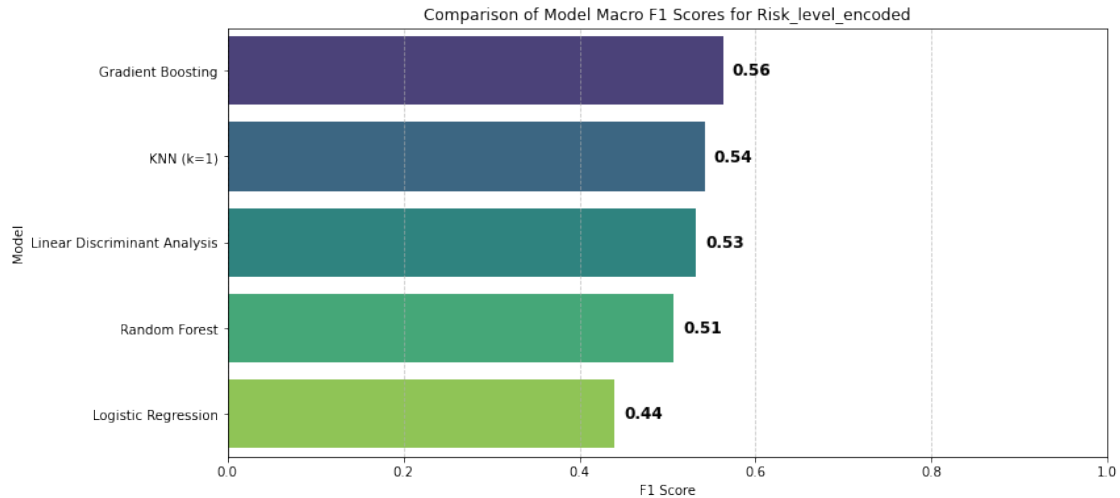
Random Forest Macro F1 Score: 0.5074

Linear Discriminant Analysis Macro F1 Score: 0.5328

Logistic Regression Macro F1 Score: 0.4400

```
/Users/kamila/opt/anaconda3/lib/python3.9/site-
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will
always use 'multinomial'. Leave it to its default value to avoid this warning.
    warnings.warn(
```

Best KNN Macro F1 Score (1 neighbors): 0.5421



```
[369]:
          Model  F1 Score
0      Gradient Boosting 0.563669
4          KNN (k=1) 0.542134
2  Linear Discriminant Analysis 0.532848
1          Random Forest 0.507412
3      Logistic Regression 0.439982
```

```
[318]: class_models_macro_f1(data_inp = credit_rating_outl_scaled, X_cols =
      ↪pred_anova_common2 ,
          y_col = 'Risk_level_encoded', test_size=0.2, num_class =
      ↪4)
```

Gradient Boosting Macro F1 Score: 0.4675

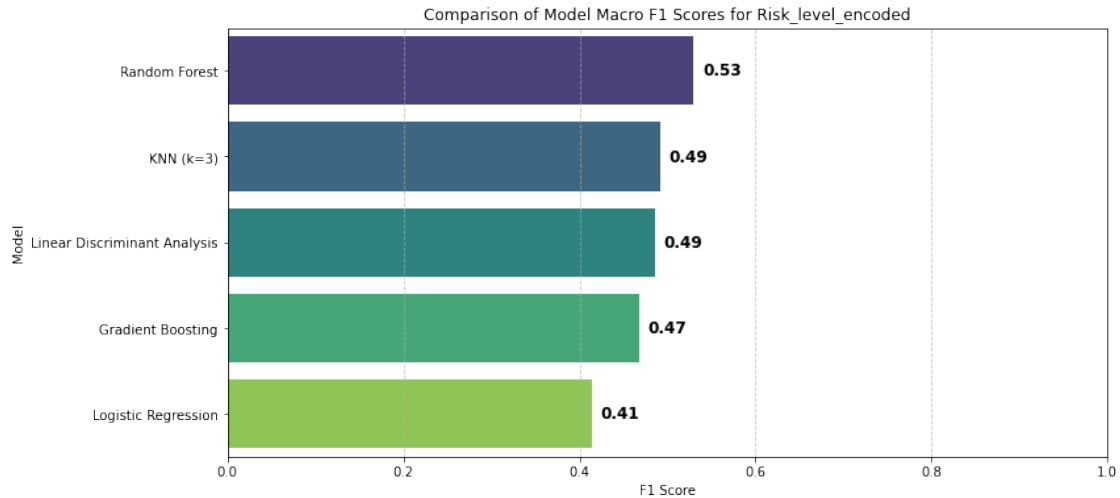
Random Forest Macro F1 Score: 0.5297

Linear Discriminant Analysis Macro F1 Score: 0.4851

Logistic Regression Macro F1 Score: 0.4136

Best KNN Macro F1 Score (3 neighbors): 0.4917

```
/Users/kamila/opt/anaconda3/lib/python3.9/site-
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will
always use 'multinomial'. Leave it to its default value to avoid this warning.
  warnings.warn(
```

```
[318]:
```

	Model	F1 Score
1	Random Forest	0.529701
4	KNN (k=3)	0.491720
2	Linear Discriminant Analysis	0.485087
0	Gradient Boosting	0.467547
3	Logistic Regression	0.413611

```
[390]: data1 = class_models_macro_f1(data_inp = credit_rating_outl_scaled, X_cols = X_cols,
    ↪X_cols,
    y_col = 'Risk_level_encoded', test_size=0.2, num_class = 4)
data2 = class_models_macro_f1(data_inp = credit_rating_outl_scaled, X_cols = pred_anova_common2,
    ↪pred_anova_common2,
    y_col = 'Risk_level_encoded', test_size=0.2, num_class = 4)
    ↪4)
```

Gradient Boosting Macro F1 Score: 0.5637

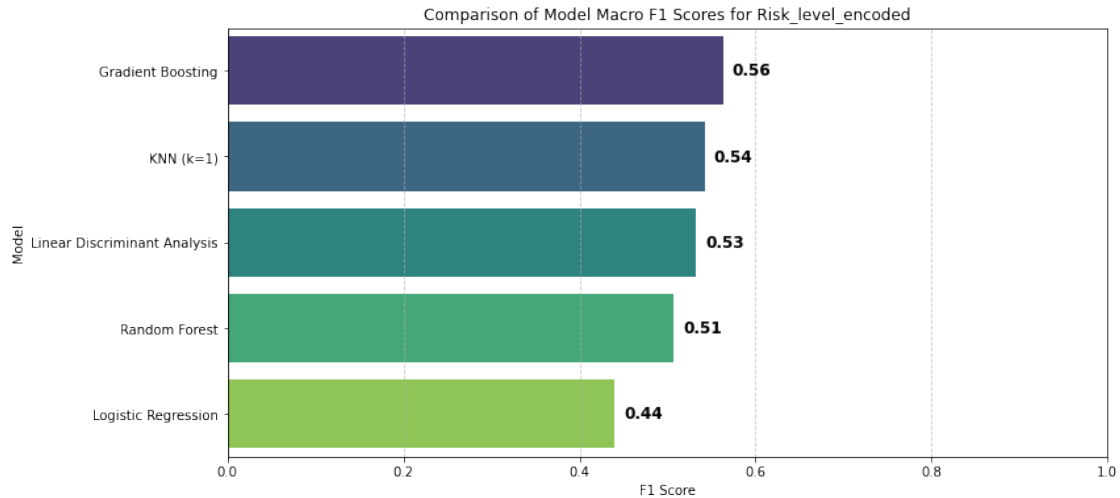
Random Forest Macro F1 Score: 0.5074

Linear Discriminant Analysis Macro F1 Score: 0.5328

Logistic Regression Macro F1 Score: 0.4400

```
/Users/kamila/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.
  warnings.warn(
```

Best KNN Macro F1 Score (1 neighbors): 0.5421



Gradient Boosting Macro F1 Score: 0.4675

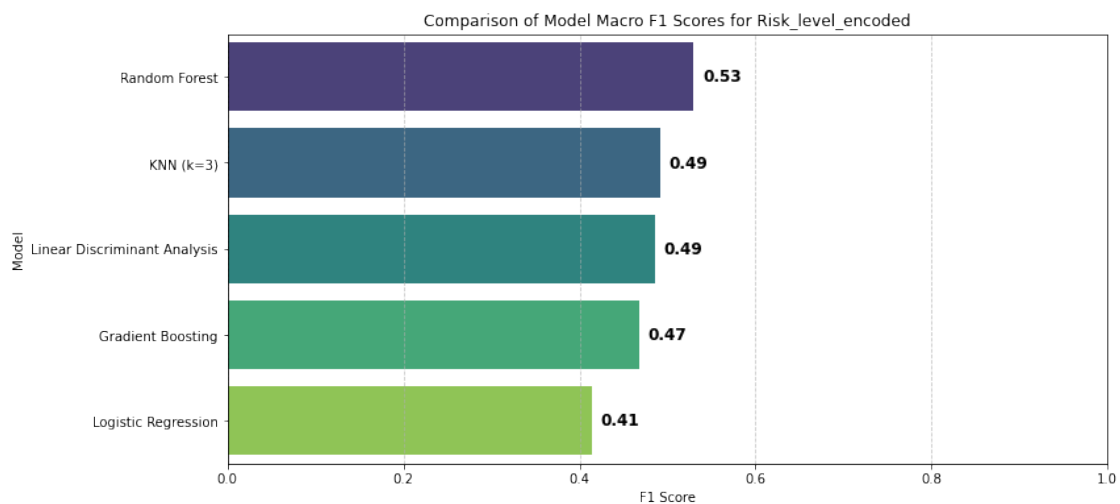
Random Forest Macro F1 Score: 0.5297

Linear Discriminant Analysis Macro F1 Score: 0.4851

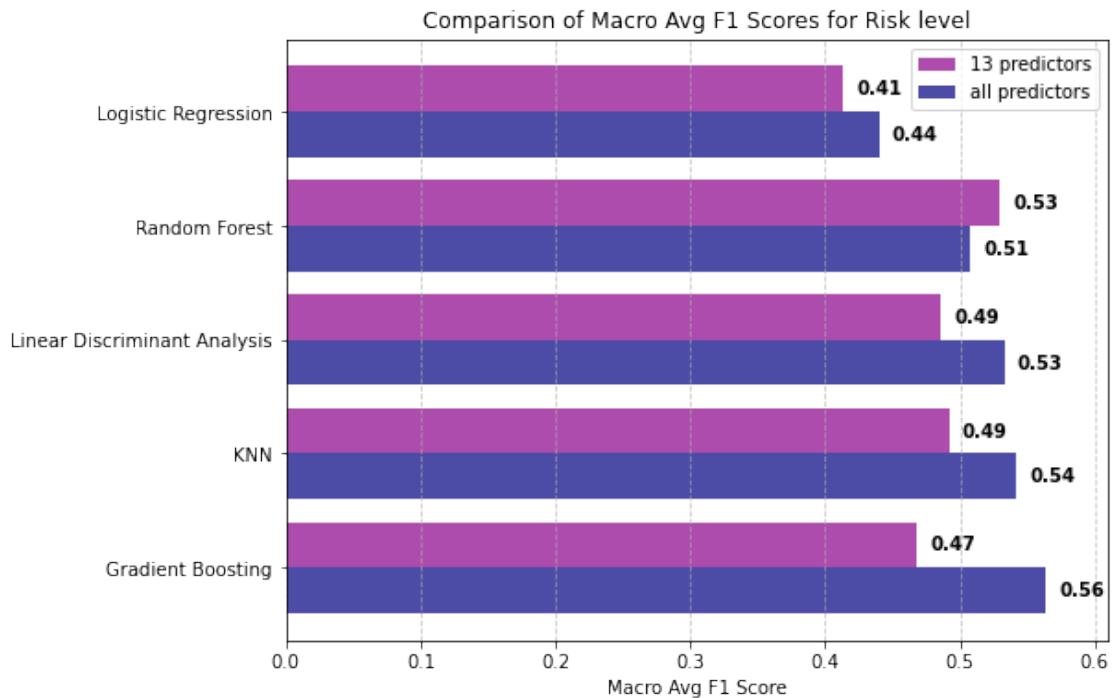
Logistic Regression Macro F1 Score: 0.4136

Best KNN Macro F1 Score (3 neighbors): 0.4917

```
/Users/kamila/opt/anaconda3/lib/python3.9/site-
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will
always use 'multinomial'. Leave it to its default value to avoid this warning.
warnings.warn(
```



```
[392]: compare_bar_charts(data1, data2, title='Comparison of Macro Avg F1 Scores',
    ↪ y_col = 'Risk level',
    label1 = 'all predictors', label2 = '13 predictors', xlim=0.
    ↪ 61,
    color1='navy', color2='darkmagenta')
```



```
[319]: class_models_macro_f1(data_inp=credit_rating_outl_scaled, X_cols=X_cols ,
    y_col = 'Rating_encoded', test_size=0.2, num_class = 6)
```

Gradient Boosting Macro F1 Score: 0.4683

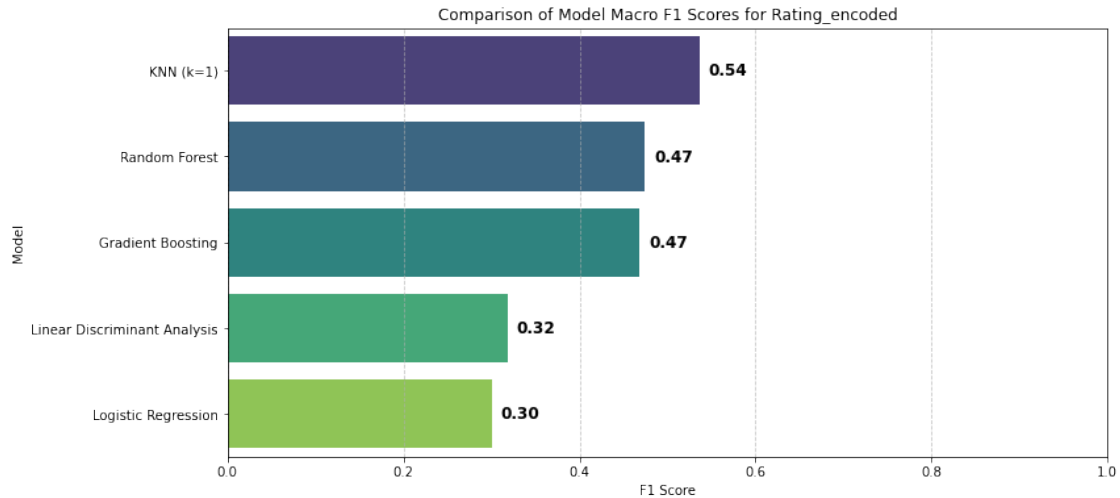
Random Forest Macro F1 Score: 0.4746

Linear Discriminant Analysis Macro F1 Score: 0.3188

Logistic Regression Macro F1 Score: 0.3006

/Users/kamila/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.
warnings.warn(

Best KNN Macro F1 Score (1 neighbors): 0.5368



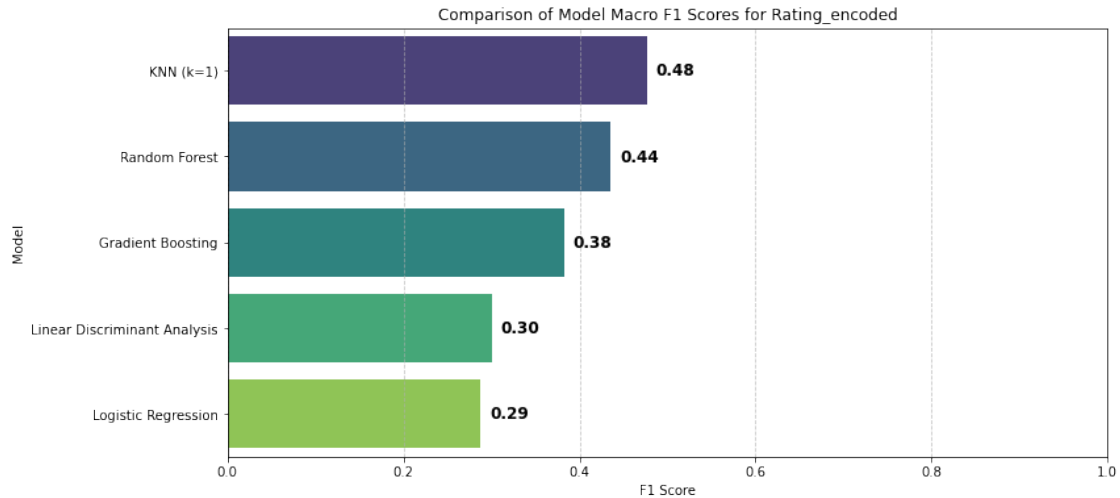
```
[319]:
```

	Model	F1 Score
4	KNN (k=1)	0.536838
1	Random Forest	0.474598
0	Gradient Boosting	0.468263
2	Linear Discriminant Analysis	0.318815
3	Logistic Regression	0.300554

```
[320]: class_models_macro_f1(data_inp = credit_rating_outl_scaled, X_cols = _
↳pred_anova_common2 ,
      y_col = 'Rating_encoded', test_size=0.2, num_class = 6)
```

```
Gradient Boosting Macro F1 Score: 0.3828
Random Forest Macro F1 Score: 0.4357
Linear Discriminant Analysis Macro F1 Score: 0.3003
Logistic Regression Macro F1 Score: 0.2878
Best KNN Macro F1 Score (1 neighbors): 0.4768
```

```
/Users/kamila/opt/anaconda3/lib/python3.9/site-
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will
always use 'multinomial'. Leave it to its default value to avoid this warning.
  warnings.warn(
```



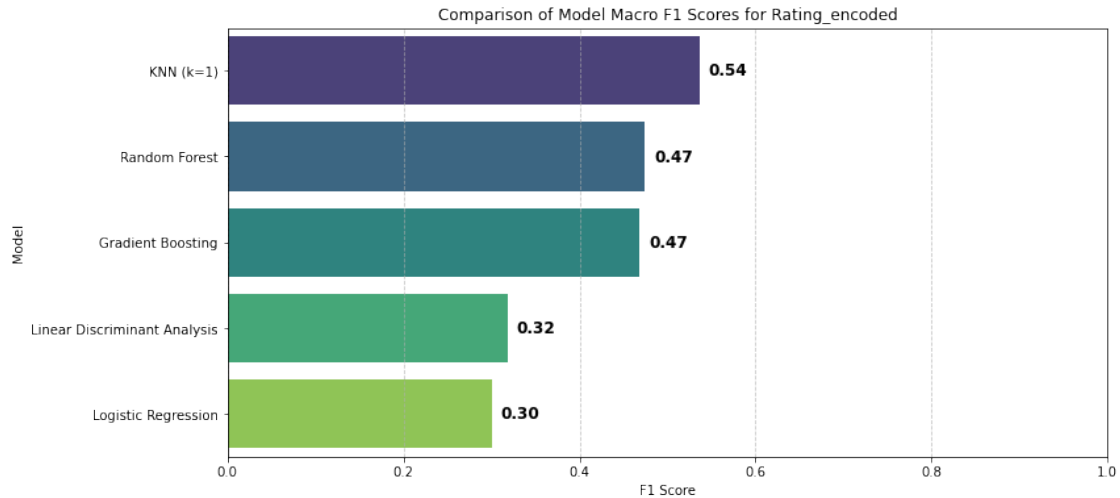
```
[320]:
```

	Model	F1 Score
4	KNN (k=1)	0.476832
1	Random Forest	0.435717
0	Gradient Boosting	0.382802
2	Linear Discriminant Analysis	0.300265
3	Logistic Regression	0.287833

```
[385]: data1 = class_models_macro_f1(data_inp = credit_rating_outl_scaled, X_cols = X_cols,
                                     y_col = 'Rating_encoded', test_size=0.2, num_class = 6)
data2 = class_models_macro_f1(data_inp = credit_rating_outl_scaled, X_cols = pred_anova_common2,
                               y_col = 'Rating_encoded', test_size=0.2, num_class = 6)
```

```
Gradient Boosting Macro F1 Score: 0.4683
Random Forest Macro F1 Score: 0.4746
Linear Discriminant Analysis Macro F1 Score: 0.3188
Logistic Regression Macro F1 Score: 0.3006
Best KNN Macro F1 Score (1 neighbors): 0.5368
```

```
/Users/kamila/opt/anaconda3/lib/python3.9/site-
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will
always use 'multinomial'. Leave it to its default value to avoid this warning.
warnings.warn(
```



Gradient Boosting Macro F1 Score: 0.3828

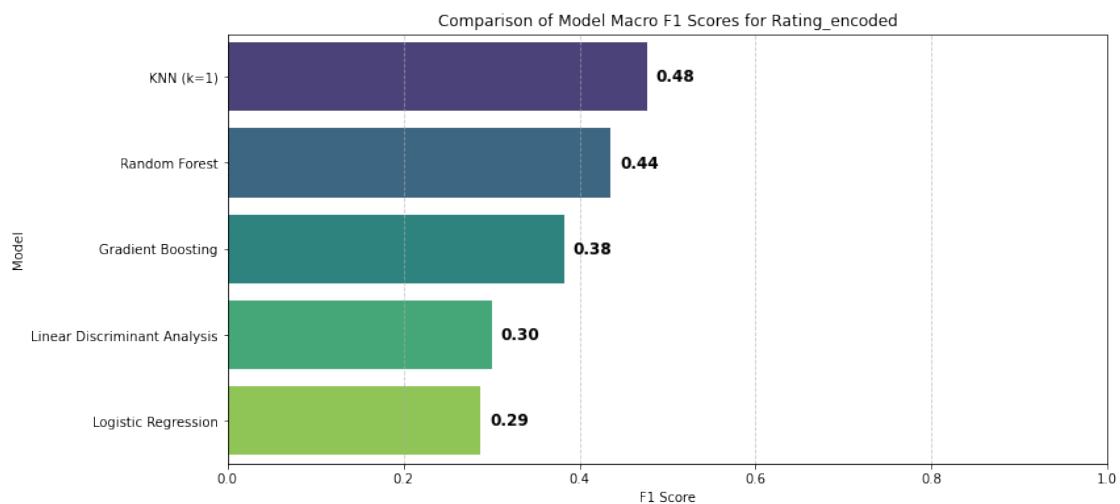
Random Forest Macro F1 Score: 0.4357

Linear Discriminant Analysis Macro F1 Score: 0.3003

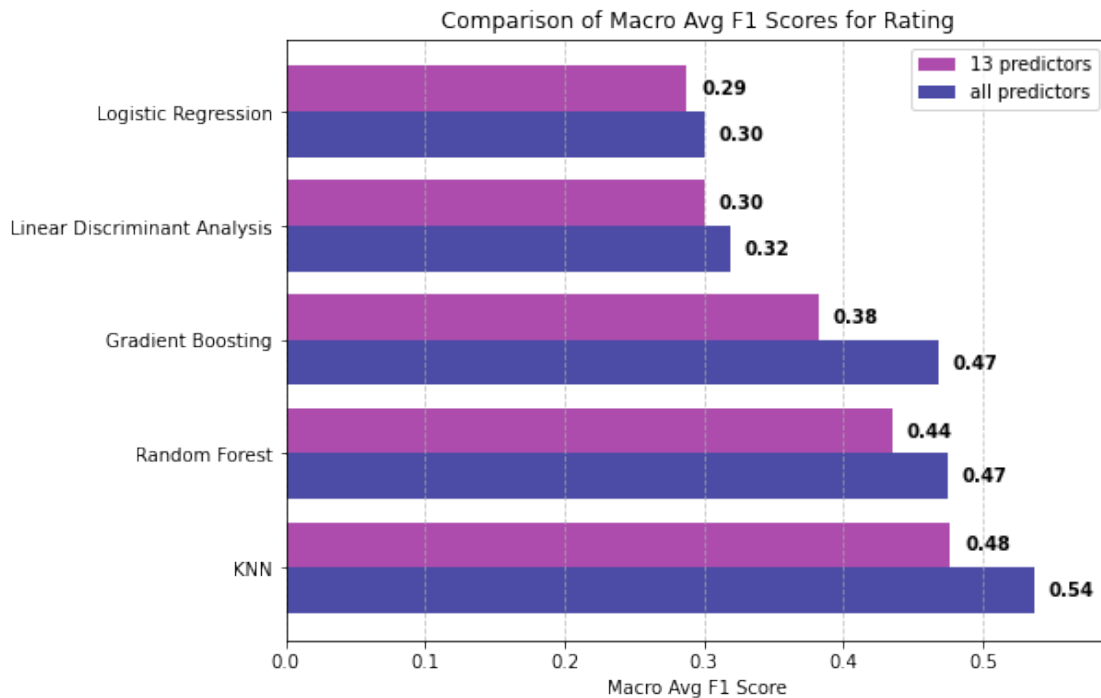
Logistic Regression Macro F1 Score: 0.2878

Best KNN Macro F1 Score (1 neighbors): 0.4768

/Users/kamila/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.
warnings.warn(



```
[389]: compare_bar_charts(data1, data2, title='Comparison of Macro Avg F1 Scores',
    ↪y_col = 'Rating',
    label1 = 'all predictors', label2 = '13 predictors', xlim=0.
    ↪59,
    color1='navy', color2='darkmagenta')
```



```
[ ]:
```

```
[660]: pred_anova_risk_rating = ['returnOnCapitalEmployed', 'returnOnAssets',
    ↪'pretaxProfitMargin',
    'ebitPerRevenue', 'netProfitMargin', 'returnOnEquity', 'operatingProfitMargin',
    'operatingCashFlowPerShare', 'debtRatio', 'freeCashFlowPerShare',
    ↪'effectiveTaxRate',
    'quickRatio', 'operatingCashFlowSalesRatio', 'currentRatio', 'cashRatio',
    ↪'grossProfitMargin',
    'freeCashFlowOperatingCashFlowRatio', 'companyEquityMultiplier',
    ↪'debtEquityRatio', 'cashPerShare',
    'Sector', 'fixedAssetTurnover']
```

```
[ ]: pred_anova_risk_rating = ['returnOnCapitalEmployed', 'returnOnAssets',
    ↪'pretaxProfitMargin',
    'ebitPerRevenue', 'netProfitMargin', 'returnOnEquity', 'operatingProfitMargin',
    'operatingCashFlowPerShare', 'debtRatio', 'freeCashFlowPerShare',
    ↪'effectiveTaxRate',
```

```
'cashRatio', 'currentRatio', 'quickRatio',  
  ↳'operatingCashFlowSalesRatio', 'currentRatio', 'cashRatio',  
  ↳'grossProfitMargin',  
'freeCashFlowOperatingCashFlowRatio', 'companyEquityMultiplier',  
  ↳'debtEquityRatio', 'cashPerShare',  
'Sector', 'fixedAssetTurnover']
```

```
[ ]:
```

```
[ ]:
```

```
[ ]:
```

```
[ ]: #Common  
'Sector' xx  
'currentRatio' xx  
'grossProfitMargin' xx  
'returnOnCapitalEmployed' xx  
'debtRatio' xx  
'operatingCashFlowPerShare' xx  
'operatingCashFlowSalesRatio' xx  
  
#different  
'assetTurnover' N N  
'fixedAssetTurnover' y N  
'debtEquityRatio' y  
'enterpriseValueMultiple' N N
```

```
[ ]: #Boxplot:  
  
# returnOnCapitalEmployed  
# returnOnAssets  
# pretaxProfitMargin  
# ebitPerRevenue  
# netProfitMargin  
# returnOnEquity  
# 'operatingCashFlowPerShare'  
# 'operatingProfitMargin'  
# 'freeCashFlowPerShare'  
# 'debtRatio'  
# 'effectiveTaxRate'  
# grossProfitMargin  
# 'operatingCashFlowSalesRatio'  
# 'payablesTurnover'  
# Sector
```



```

# 'debtEquityRatio' B
# 'companyEquityMultiplier', B

#ANOVA F-test:

# returnOnCapitalEmployed (1)(1)
# returnOnAssets (2)(2)
# pretaxProfitMargin (3)(3)
# ebitPerRevenue (4)(4)
# netProfitMargin (5)(5)
# returnOnEquity (6)(7)
# 'operatingCashFlowPerShare' (7)(6)
# 'operatingProfitMargin' (8)(8)
# 'freeCashFlowPerShare' (9)(10)
# 'debtRatio' (10)(9)
# 'effectiveTaxRate' (11)(11)
# cashRatio (12)(12)
# currentRatio (13)(14)
# quickRatio (14)(15)
# grossProfitMargin (15)(13)
# 'operatingCashFlowSalesRatio' (16)(16) x y
# cashPerShare (17)(17)
# freeCashFlowOperatingCashFlowRatio (18)
# 'payablesTurnover' (19) xx
# Sector
# 'companyEquityMultiplier'()(18)
# 'debtEquityRatio'()(19)
# fixedAssetTurnover () (20) yy

#difference

# cashRatio F
# currentRatio F
# quickRatio F
# cashPerShare F
# freeCashFlowOperatingCashFlowRatio F

#Based on boxplots: 'netProfitMargin', 'pretaxProfitMargin', (2)

```

```
# 'grossProfitMargin', 'operatingProfitMargin', 'returnOnAssets', (3)
# 'returnOnCapitalEmployed', 'returnOnEquity', 'debtEquityRatio', (3)
# 'debtRatio', 'effectiveTaxRate', 'freeCashFlowPerShare', (3)
# 'companyEquityMultiplier', 'ebitPerRevenue', 'operatingCashFlowPerShare', (3)
# 'operatingCashFlowSalesRatio', 'payablesTurnover' (2) + Sector
```

```
[616]: predictors_anova = ['returnOnCapitalEmployed', 'returnOnAssets',
    ↪ 'pretaxProfitMargin',
    ↪ 'ebitPerRevenue', 'netProfitMargin', 'returnOnEquity',
    ↪ 'operatingCashFlowPerShare',
    ↪ 'operatingProfitMargin', 'freeCashFlowPerShare',
    ↪ 'debtRatio', 'effectiveTaxRate',
    ↪ 'cashRatio', 'currentRatio', 'quickRatio',
    ↪ 'grossProfitMargin', 'operatingCashFlowSalesRatio']

predictors_anova_sector= ['returnOnCapitalEmployed', 'returnOnAssets',
    ↪ 'pretaxProfitMargin',
    ↪ 'ebitPerRevenue', 'netProfitMargin', 'returnOnEquity',
    ↪ 'operatingCashFlowPerShare',
    ↪ 'operatingProfitMargin', 'freeCashFlowPerShare',
    ↪ 'debtRatio', 'effectiveTaxRate',
    ↪ 'cashRatio', 'currentRatio', 'quickRatio',
    ↪ 'grossProfitMargin',
    ↪ 'operatingCashFlowSalesRatio', 'Sector']
```

```
[612]: cor(credit_rating_outl_scaled, predictors_anova)
```

	returnOnCapitalEmployed	returnOnAssets \
returnOnCapitalEmployed	1.000000	0.887018
returnOnAssets	0.887018	1.000000
pretaxProfitMargin	0.662520	0.701867
ebitPerRevenue	0.659554	0.681974
netProfitMargin	0.585488	0.755582
returnOnEquity	0.616245	0.680274
operatingCashFlowPerShare	0.260795	0.238936
operatingProfitMargin	0.543895	0.617132
freeCashFlowPerShare	0.364512	0.335259
debtRatio	-0.059022	-0.124251
effectiveTaxRate	0.184837	0.131880
cashRatio	-0.024113	0.043907
currentRatio	-0.033028	0.039200
quickRatio	-0.032124	0.038224
grossProfitMargin	-0.053749	-0.004122
operatingCashFlowSalesRatio	0.018203	0.085074

	pretaxProfitMargin	ebitPerRevenue \
returnOnCapitalEmployed	0.662520	0.659554

returnOnAssets	0.701867	0.681974
pretaxProfitMargin	1.000000	0.984009
ebitPerRevenue	0.984009	1.000000
netProfitMargin	0.898153	0.884276
returnOnEquity	0.481483	0.463969
operatingCashFlowPerShare	0.254659	0.246873
operatingProfitMargin	0.900086	0.902936
freeCashFlowPerShare	0.249365	0.239990
debtRatio	-0.142243	-0.150097
effectiveTaxRate	0.128690	0.133130
cashRatio	0.011482	0.007996
currentRatio	-0.036303	-0.049665
quickRatio	0.048278	0.041701
grossProfitMargin	0.226039	0.216260
operatingCashFlowSalesRatio	0.447026	0.457347

	netProfitMargin	returnOnEquity \
returnOnCapitalEmployed	0.585488	0.616245
returnOnAssets	0.755582	0.680274
pretaxProfitMargin	0.898153	0.481483
ebitPerRevenue	0.884276	0.463969
netProfitMargin	1.000000	0.525543
returnOnEquity	0.525543	1.000000
operatingCashFlowPerShare	0.207209	0.171138
operatingProfitMargin	0.868786	0.420494
freeCashFlowPerShare	0.212631	0.242287
debtRatio	-0.168280	0.063213
effectiveTaxRate	0.078650	0.059564
cashRatio	0.058272	-0.040515
currentRatio	-0.007933	-0.084788
quickRatio	0.071742	-0.041947
grossProfitMargin	0.225224	-0.020315
operatingCashFlowSalesRatio	0.432589	-0.021566

	operatingCashFlowPerShare	operatingProfitMargin \
returnOnCapitalEmployed	0.260795	0.543895
returnOnAssets	0.238936	0.617132
pretaxProfitMargin	0.254659	0.900086
ebitPerRevenue	0.246873	0.902936
netProfitMargin	0.207209	0.868786
returnOnEquity	0.171138	0.420494
operatingCashFlowPerShare	1.000000	0.188598
operatingProfitMargin	0.188598	1.000000
freeCashFlowPerShare	0.624753	0.190910
debtRatio	-0.082871	-0.088765
effectiveTaxRate	0.159807	0.155647
cashRatio	-0.141165	0.041486
currentRatio	-0.166358	-0.041878

quickRatio	-0.107110	0.071856
grossProfitMargin	0.128265	0.299462
operatingCashFlowSalesRatio	0.309117	0.494354

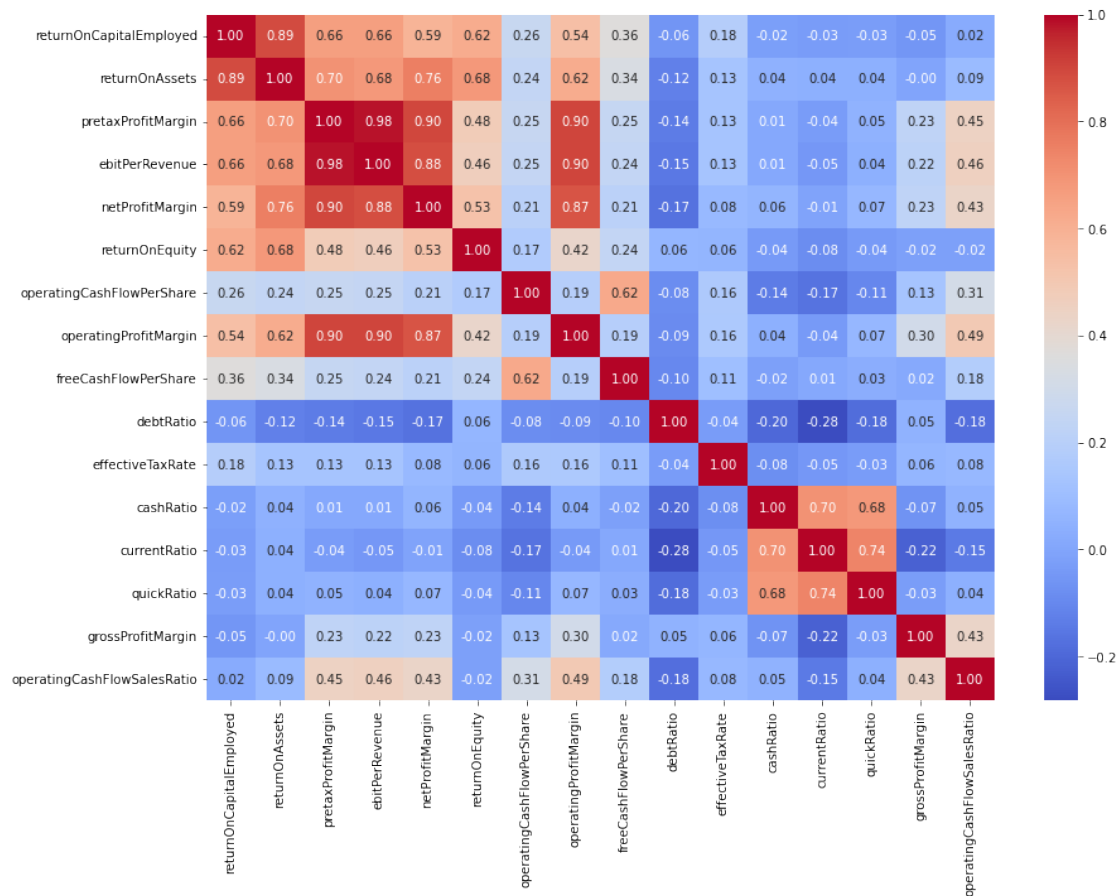
	freeCashFlowPerShare	debtRatio	\
returnOnCapitalEmployed	0.364512	-0.059022	
returnOnAssets	0.335259	-0.124251	
pretaxProfitMargin	0.249365	-0.142243	
ebitPerRevenue	0.239990	-0.150097	
netProfitMargin	0.212631	-0.168280	
returnOnEquity	0.242287	0.063213	
operatingCashFlowPerShare	0.624753	-0.082871	
operatingProfitMargin	0.190910	-0.088765	
freeCashFlowPerShare	1.000000	-0.099855	
debtRatio	-0.099855	1.000000	
effectiveTaxRate	0.106856	-0.036203	
cashRatio	-0.018395	-0.201487	
currentRatio	0.011959	-0.282656	
quickRatio	0.029721	-0.183789	
grossProfitMargin	0.017095	0.048218	
operatingCashFlowSalesRatio	0.180502	-0.179817	

	effectiveTaxRate	cashRatio	currentRatio	\
returnOnCapitalEmployed	0.184837	-0.024113	-0.033028	
returnOnAssets	0.131880	0.043907	0.039200	
pretaxProfitMargin	0.128690	0.011482	-0.036303	
ebitPerRevenue	0.133130	0.007996	-0.049665	
netProfitMargin	0.078650	0.058272	-0.007933	
returnOnEquity	0.059564	-0.040515	-0.084788	
operatingCashFlowPerShare	0.159807	-0.141165	-0.166358	
operatingProfitMargin	0.155647	0.041486	-0.041878	
freeCashFlowPerShare	0.106856	-0.018395	0.011959	
debtRatio	-0.036203	-0.201487	-0.282656	
effectiveTaxRate	1.000000	-0.080134	-0.052083	
cashRatio	-0.080134	1.000000	0.701051	
currentRatio	-0.052083	0.701051	1.000000	
quickRatio	-0.034838	0.683437	0.744222	
grossProfitMargin	0.063061	-0.066873	-0.219900	
operatingCashFlowSalesRatio	0.076522	0.048369	-0.147103	

	quickRatio	grossProfitMargin	\
returnOnCapitalEmployed	-0.032124	-0.053749	
returnOnAssets	0.038224	-0.004122	
pretaxProfitMargin	0.048278	0.226039	
ebitPerRevenue	0.041701	0.216260	
netProfitMargin	0.071742	0.225224	
returnOnEquity	-0.041947	-0.020315	
operatingCashFlowPerShare	-0.107110	0.128265	

operatingProfitMargin	0.071856	0.299462
freeCashFlowPerShare	0.029721	0.017095
debtRatio	-0.183789	0.048218
effectiveTaxRate	-0.034838	0.063061
cashRatio	0.683437	-0.066873
currentRatio	0.744222	-0.219900
quickRatio	1.000000	-0.029425
grossProfitMargin	-0.029425	1.000000
operatingCashFlowSalesRatio	0.038236	0.432852

	operatingCashFlowSalesRatio	
returnOnCapitalEmployed		0.018203
returnOnAssets		0.085074
pretaxProfitMargin		0.447026
ebitPerRevenue		0.457347
netProfitMargin		0.432589
returnOnEquity		-0.021566
operatingCashFlowPerShare		0.309117
operatingProfitMargin		0.494354
freeCashFlowPerShare		0.180502
debtRatio		-0.179817
effectiveTaxRate		0.076522
cashRatio		0.048369
currentRatio		-0.147103
quickRatio		0.038236
grossProfitMargin		0.432852
operatingCashFlowSalesRatio		1.000000



```
[613]: vif(credit_rating_outl_scaled, predictors_anova)
```

	feature	VIF
0	returnOnCapitalEmployed	8.875341
1	returnOnAssets	11.874778
2	pretaxProfitMargin	37.103342
3	ebitPerRevenue	37.144228
4	netProfitMargin	11.533029
5	returnOnEquity	2.032879
6	operatingCashFlowPerShare	1.889522
7	operatingProfitMargin	7.169855
8	freeCashFlowPerShare	1.845945
9	debtRatio	1.254978
10	effectiveTaxRate	1.097453
11	cashRatio	2.353014
12	currentRatio	3.369113
13	quickRatio	2.678637
14	grossProfitMargin	1.381631
15	operatingCashFlowSalesRatio	2.153748

```
[614]: class_models_macro_f1(data_inp=credit_rating_outl_scaled, X_cols=X_cols ,
                             y_col = 'Risk_level_encoded', test_size=0.2, num_class =4)
↪4)
```

Gradient Boosting Macro F1 Score: 0.5637

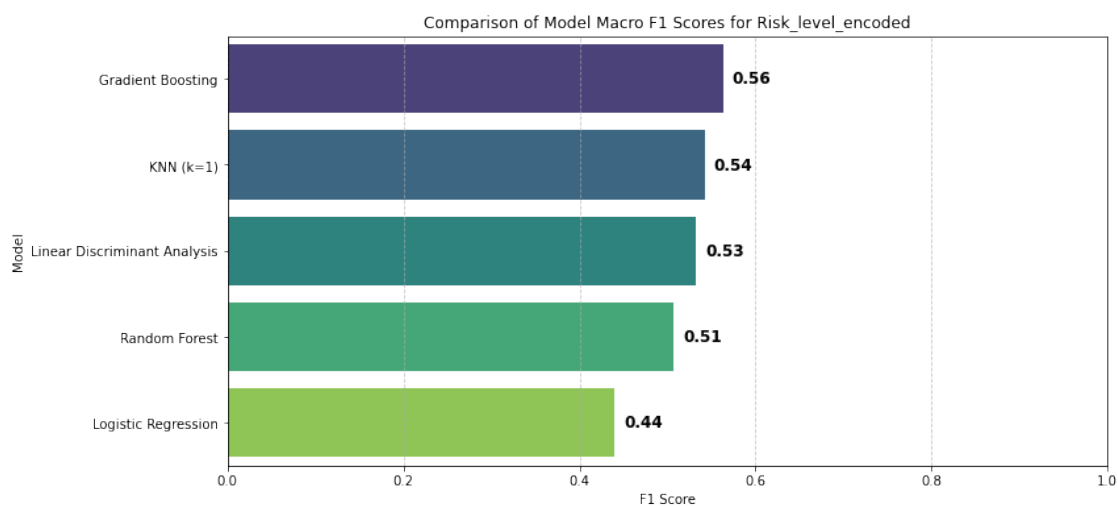
Random Forest Macro F1 Score: 0.5074

Linear Discriminant Analysis Macro F1 Score: 0.5328

Logistic Regression Macro F1 Score: 0.4400

/Users/kamila/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.
warnings.warn(

Best KNN Macro F1 Score (1 neighbors): 0.5421



```
[614]:
Model    F1 Score
0      Gradient Boosting 0.563669
4              KNN (k=1) 0.542134
2  Linear Discriminant Analysis 0.532848
1              Random Forest 0.507412
3      Logistic Regression 0.439982
```

```
[617]: class_models_macro_f1(data_inp=credit_rating_outl_scaled,
↪X_cols=predictors_anova_sector,
                             y_col = 'Risk_level_encoded', test_size=0.2, num_class =4)
↪4)
```

Gradient Boosting Macro F1 Score: 0.4745

Random Forest Macro F1 Score: 0.4918

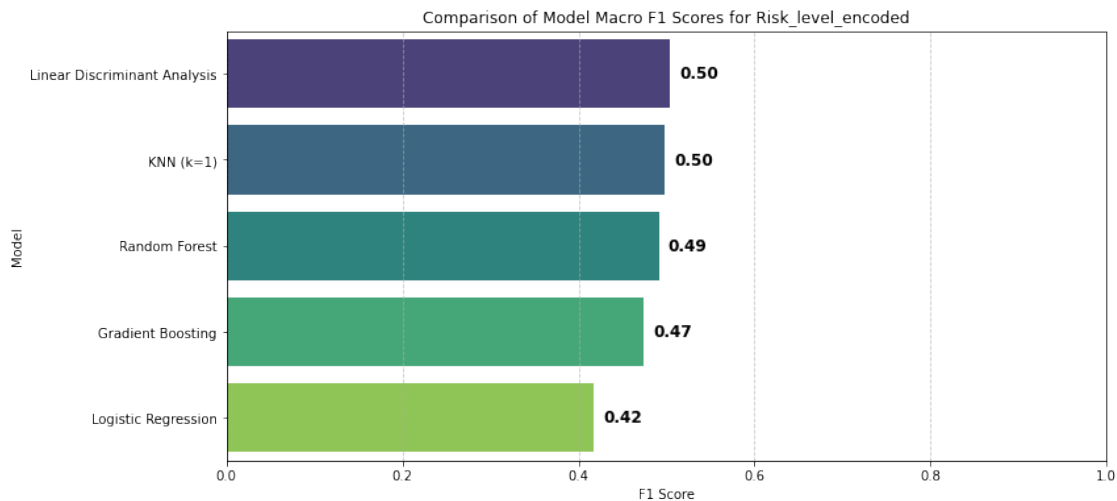
Linear Discriminant Analysis Macro F1 Score: 0.5043

Logistic Regression Macro F1 Score: 0.4174

```
/Users/kamila/opt/anaconda3/lib/python3.9/site-  
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'  
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will  
always use 'multinomial'. Leave it to its default value to avoid this warning.  
warnings.warn(  

```

Best KNN Macro F1 Score (1 neighbors): 0.4982



```
[617]:  
      Model  F1 Score  
2  Linear Discriminant Analysis  0.504263  
4      KNN (k=1)  0.498187  
1      Random Forest  0.491835  
0      Gradient Boosting  0.474497  
3      Logistic Regression  0.417429
```

```
[618]: class_models_macro_avg(data_inp=credit_rating_outl_scaled, X_cols=X_cols,  
                               y_col = 'Risk_level_encoded', test_size=0.2, num_class = 4)  
↪4)
```

Gradient Boosting Macro-Average Accuracy: 0.5620

Random Forest Macro-Average Accuracy: 0.5103

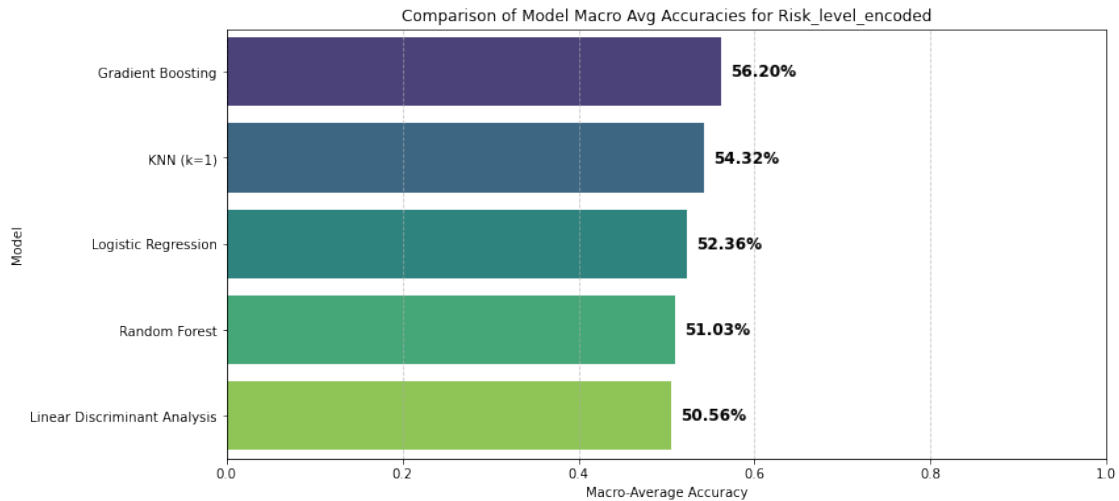
Linear Discriminant Analysis Macro-Average Accuracy: 0.5056

Logistic Regression Macro-Average Accuracy: 0.5236

```
/Users/kamila/opt/anaconda3/lib/python3.9/site-  
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'  
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will  
always use 'multinomial'. Leave it to its default value to avoid this warning.  
warnings.warn(  

```


Best KNN Macro-Average Accuracy (1 neighbors): 0.5432



```
[618]:
```

	Model	Macro Avg Accuracy
0	Gradient Boosting	0.562041
4	KNN (k=1)	0.543175
3	Logistic Regression	0.523632
1	Random Forest	0.510342
2	Linear Discriminant Analysis	0.505647

```
[619]: class_models_macro_avg(data_inp=credit_rating_outl_scaled,
    ↪X_cols=predictors_anova_sector,
    y_col = 'Risk_level_encoded', test_size=0.2, num_class =
    ↪4)
```

Gradient Boosting Macro-Average Accuracy: 0.4783

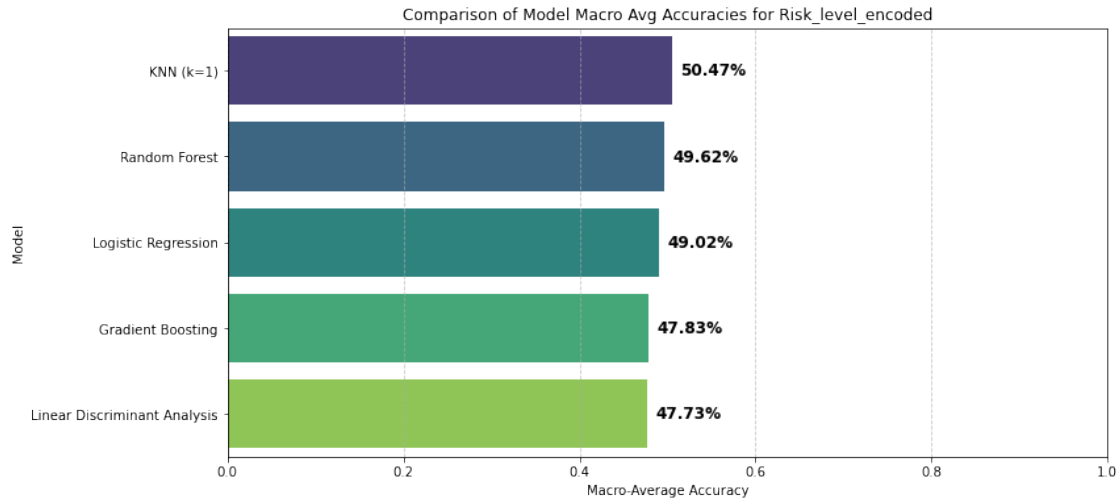
Random Forest Macro-Average Accuracy: 0.4962

Linear Discriminant Analysis Macro-Average Accuracy: 0.4773

Logistic Regression Macro-Average Accuracy: 0.4902

Best KNN Macro-Average Accuracy (1 neighbors): 0.5047

```
/Users/kamila/opt/anaconda3/lib/python3.9/site-
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will
always use 'multinomial'. Leave it to its default value to avoid this warning.
warnings.warn(
```



```
[619]:
```

	Model	Macro Avg Accuracy
4	KNN (k=1)	0.504700
1	Random Forest	0.496158
3	Logistic Regression	0.490202
0	Gradient Boosting	0.478346
2	Linear Discriminant Analysis	0.477289

6 Backward elimination for Subset Selection

```
[676]: from mlxtend.feature_selection import SequentialFeatureSelector
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import classification_report, make_scorer
from sklearn.utils.class_weight import compute_sample_weight
from sklearn.model_selection import train_test_split
from sklearn import preprocessing
import numpy as np

def backward_elim_in_gb(data_in, X_cols, y_col, test_size, num_class):

    data = data_in.copy()

    le = preprocessing.LabelEncoder()
    le.fit(data.Sector)
    data.Sector = le.transform(data.Sector) # Encode sector

    # Split dataset into training and testing while maintaining category
    ↳ proportions
    X = data[X_cols] # Features
    y = data[y_col] # Target
```

```

X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    test_size=test_size, stratify=y, random_state=123)

# Custom function to calculate macro-averaged F1-score
def macro_avg_f1(y_true, y_pred):
    report = classification_report(y_true, y_pred, output_dict=True)
    return report['macro avg']['f1-score']

# Define custom scoring metric
macro_scorer = make_scorer(macro_avg_f1)

# Define a wrapper model that applies sample weights correctly
class WeightedGBClassifier(GradientBoostingClassifier):
    def fit(self, X, y, **kwargs):
        sample_weights = compute_sample_weight(class_weight="balanced",
→y=y) # Compute per fold
        return super().fit(X, y, sample_weight=sample_weights, **kwargs)

# Initialize Gradient Boosting Model with sample weights
gb_model = WeightedGBClassifier(random_state=123)

# Backward Feature Selection using Gradient Boosting with sample weights
backward_feature_selector = SequentialFeatureSelector(
    estimator=gb_model, # Use wrapped model
    k_features=(1, X_train.shape[1]), # Select between 1 and all features
    forward=False, # Backward elimination
    floating=False, # Standard backward elimination
    verbose=2,
    scoring=macro_scorer, # Optimize for macro-averaged F1-score
    cv=5 # Cross-validation for stability
)

# Fit feature selector
backward_feature_selector.fit(X_train, y_train)

# Get selected feature names
selected_features = list(backward_feature_selector.k_feature_names_)
print("Optimal feature set:", selected_features)

return selected_features

def forward_select_gb(data_inp, X_cols, y_col, test_size, num_class):

```

```

data = data_inp.copy()

le = preprocessing.LabelEncoder()
le.fit(data.Sector)
data.Sector = le.transform(data.Sector)  # Encode sector

# Split dataset into training and testing while maintaining category
→ proportions
X = data[X_cols]  # Features
y = data[y_col]  # Target

X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    test_size=test_size, stratify=y, random_state=123)

# Custom function to calculate macro-averaged F1-score
def macro_avg_f1(y_true, y_pred):
    report = classification_report(y_true, y_pred, output_dict=True)
    return report['macro avg']['f1-score']

# Define custom scoring metric
macro_scorer = make_scorer(macro_avg_f1)

# Define a wrapper model that applies sample weights correctly
class WeightedGBClassifier(GradientBoostingClassifier):
    def fit(self, X, y, **kwargs):
        sample_weights = compute_sample_weight(class_weight="balanced",
→ y=y)  # Compute per fold
        return super().fit(X, y, sample_weight=sample_weights, **kwargs)

# Initialize Gradient Boosting Model with sample weights
gb_model = WeightedGBClassifier(random_state=123)

# Forward Feature Selection using Gradient Boosting with sample weights
forward_feature_selector = SequentialFeatureSelector(
    estimator=gb_model,  # Use wrapped model
    k_features=(1, X_train.shape[1]),  # Select between 1 and all features
    forward=True,  # Forward selection
    floating=False,  # Standard forward selection
    verbose=2,
    scoring=macro_scorer,  # Optimize for macro-averaged F1-score
    cv=5  # Cross-validation for stability
)

# Fit feature selector
forward_feature_selector.fit(X_train, y_train)

# Get selected feature names

```

```

selected_features = list(forward_feature_selector.k_feature_names_)
print("Optimal feature set:", selected_features)

return selected_features

```

```

[631]: backward_elimination(data_in=credit_rating_outl_scaled,
    ↪X_cols=predictors_anova_sector,
    y_col = 'Risk_level_encoded', test_size=0.2, num_class =
    ↪4)

```

```

[2025-02-05 16:28:55] Features: 16/1 -- score: 0.5068130853998115
[2025-02-05 16:33:41] Features: 15/1 -- score: 0.5052419862031748
[2025-02-05 16:37:53] Features: 14/1 -- score: 0.49547115690208676
[2025-02-05 16:41:34] Features: 13/1 -- score: 0.49764452202959897
[2025-02-05 16:44:50] Features: 12/1 -- score: 0.4856024747791441
[2025-02-05 16:47:35] Features: 11/1 -- score: 0.5007994522231897
[2025-02-05 16:49:50] Features: 10/1 -- score: 0.4973956682094055
[2025-02-05 16:51:40] Features: 9/1 -- score: 0.48899719342004816
[2025-02-05 16:53:21] Features: 8/1 -- score: 0.4807186891832296
[2025-02-05 16:54:37] Features: 7/1 -- score: 0.4853470165976269
[2025-02-05 16:55:33] Features: 6/1 -- score: 0.4850429348175762
[2025-02-05 16:56:16] Features: 5/1 -- score: 0.4770961242365651
[2025-02-05 16:56:47] Features: 4/1 -- score: 0.453149848680461
[2025-02-05 16:57:09] Features: 3/1 -- score: 0.41208544960827087
[2025-02-05 16:57:20] Features: 2/1 -- score: 0.3672594833913064

```

```

Optimal feature set: ['returnOnCapitalEmployed', 'returnOnAssets',
'pretaxProfitMargin', 'ebitPerRevenue', 'netProfitMargin', 'returnOnEquity',
'operatingCashFlowPerShare', 'operatingProfitMargin', 'freeCashFlowPerShare',
'debtRatio', 'effectiveTaxRate', 'currentRatio', 'quickRatio',
'grossProfitMargin', 'operatingCashFlowSalesRatio', 'Sector']

```

```

[2025-02-05 16:57:26] Features: 1/1 -- score: 0.30514380164450905

```

```

[631]: ['returnOnCapitalEmployed',
'returnOnAssets',
'pretaxProfitMargin',
'ebitPerRevenue',
'netProfitMargin',
'returnOnEquity',
'operatingCashFlowPerShare',
'operatingProfitMargin',
'freeCashFlowPerShare',
'debtRatio',
'effectiveTaxRate',
'currentRatio',

```

```
'quickRatio',
'grossProfitMargin',
'operatingCashFlowSalesRatio',
'Sector']
```

```
[644]: backward_elimination(data_in=credit_rating_outl_scaled, X_cols=X_cols,
                             y_col = 'Risk_level_encoded', test_size=0.2, num_class = 4)
```

```
[2025-02-05 17:19:48] Features: 25/1 -- score: 0.5292775606856699
[2025-02-05 17:35:42] Features: 24/1 -- score: 0.5302790136835925
[2025-02-05 17:46:19] Features: 23/1 -- score: 0.5352933968854107
[2025-02-05 17:55:59] Features: 22/1 -- score: 0.5323517159361164
[2025-02-05 18:04:58] Features: 21/1 -- score: 0.5358881468638662
[2025-02-05 18:13:03] Features: 20/1 -- score: 0.5382590835093491
[2025-02-05 18:20:25] Features: 19/1 -- score: 0.5305417286574994
[2025-02-05 18:27:04] Features: 18/1 -- score: 0.5345439333770117
[2025-02-05 18:33:08] Features: 17/1 -- score: 0.53483263048247
[2025-02-05 18:58:34] Features: 16/1 -- score: 0.5337792328895443
[2025-02-05 19:03:31] Features: 15/1 -- score: 0.5282051770652064
[2025-02-05 19:07:42] Features: 14/1 -- score: 0.5422598292618472
[2025-02-05 19:11:30] Features: 13/1 -- score: 0.5423118926305535
[2025-02-05 19:14:36] Features: 12/1 -- score: 0.550808443714381
[2025-02-05 19:17:22] Features: 11/1 -- score: 0.5534021602892036
[2025-02-05 19:19:38] Features: 10/1 -- score: 0.5497097802677354
[2025-02-05 19:21:37] Features: 9/1 -- score: 0.541285178144493
[2025-02-05 19:23:10] Features: 8/1 -- score: 0.5408370990665573
[2025-02-05 19:24:24] Features: 7/1 -- score: 0.52671270770047
[2025-02-05 19:25:20] Features: 6/1 -- score: 0.516146736378859
[2025-02-05 19:26:08] Features: 5/1 -- score: 0.49159971077804177
[2025-02-05 19:26:38] Features: 4/1 -- score: 0.4694738961150504
[2025-02-05 19:26:59] Features: 3/1 -- score: 0.44961960478056484
[2025-02-05 19:27:10] Features: 2/1 -- score: 0.4139143938126037
```

```
Optimal feature set: ['Sector', 'currentRatio', 'grossProfitMargin',
'returnOnCapitalEmployed', 'assetTurnover', 'fixedAssetTurnover',
'debtEquityRatio', 'debtRatio', 'enterpriseValueMultiple',
'operatingCashFlowPerShare', 'operatingCashFlowSalesRatio']
```

```
[2025-02-05 19:27:15] Features: 1/1 -- score: 0.38231589104456315
```

```
[644]: ['Sector',
'currentRatio',
'grossProfitMargin',
'returnOnCapitalEmployed',
'assetTurnover',
```

```
'fixedAssetTurnover',
'debtEquityRatio',
'debtRatio',
'enterpriseValueMultiple',
'operatingCashFlowPerShare',
'operatingCashFlowSalesRatio']
```

[639]: *#'cashRatio' was removed*

```
pred_back = ['returnOnCapitalEmployed', 'returnOnAssets', 'pretaxProfitMargin',
↳ 'ebitPerRevenue', 'netProfitMargin',
↳ 'returnOnEquity', 'operatingCashFlowPerShare', 'operatingProfitMargin',
↳ 'freeCashFlowPerShare', 'debtRatio',
↳ 'effectiveTaxRate', 'currentRatio', 'quickRatio',
↳ 'grossProfitMargin', 'operatingCashFlowSalesRatio',
'Sector']

pred_back_num = ['returnOnCapitalEmployed',
↳ 'returnOnAssets', 'pretaxProfitMargin', 'ebitPerRevenue', 'netProfitMargin',
↳ 'returnOnEquity', 'operatingCashFlowPerShare', 'operatingProfitMargin',
↳ 'freeCashFlowPerShare', 'debtRatio',
↳ 'effectiveTaxRate', 'currentRatio', 'quickRatio',
↳ 'grossProfitMargin', 'operatingCashFlowSalesRatio']
```

[640]: cor(credit_rating_outl_scaled, pred_back_num)

	returnOnCapitalEmployed	returnOnAssets	\
returnOnCapitalEmployed	1.000000	0.887018	
returnOnAssets	0.887018	1.000000	
pretaxProfitMargin	0.662520	0.701867	
ebitPerRevenue	0.659554	0.681974	
netProfitMargin	0.585488	0.755582	
returnOnEquity	0.616245	0.680274	
operatingCashFlowPerShare	0.260795	0.238936	
operatingProfitMargin	0.543895	0.617132	
freeCashFlowPerShare	0.364512	0.335259	
debtRatio	-0.059022	-0.124251	
effectiveTaxRate	0.184837	0.131880	
currentRatio	-0.033028	0.039200	
quickRatio	-0.032124	0.038224	
grossProfitMargin	-0.053749	-0.004122	
operatingCashFlowSalesRatio	0.018203	0.085074	

	pretaxProfitMargin	ebitPerRevenue	\
returnOnCapitalEmployed	0.662520	0.659554	
returnOnAssets	0.701867	0.681974	
pretaxProfitMargin	1.000000	0.984009	

ebitPerRevenue	0.984009	1.000000
netProfitMargin	0.898153	0.884276
returnOnEquity	0.481483	0.463969
operatingCashFlowPerShare	0.254659	0.246873
operatingProfitMargin	0.900086	0.902936
freeCashFlowPerShare	0.249365	0.239990
debtRatio	-0.142243	-0.150097
effectiveTaxRate	0.128690	0.133130
currentRatio	-0.036303	-0.049665
quickRatio	0.048278	0.041701
grossProfitMargin	0.226039	0.216260
operatingCashFlowSalesRatio	0.447026	0.457347

	netProfitMargin	returnOnEquity \
returnOnCapitalEmployed	0.585488	0.616245
returnOnAssets	0.755582	0.680274
pretaxProfitMargin	0.898153	0.481483
ebitPerRevenue	0.884276	0.463969
netProfitMargin	1.000000	0.525543
returnOnEquity	0.525543	1.000000
operatingCashFlowPerShare	0.207209	0.171138
operatingProfitMargin	0.868786	0.420494
freeCashFlowPerShare	0.212631	0.242287
debtRatio	-0.168280	0.063213
effectiveTaxRate	0.078650	0.059564
currentRatio	-0.007933	-0.084788
quickRatio	0.071742	-0.041947
grossProfitMargin	0.225224	-0.020315
operatingCashFlowSalesRatio	0.432589	-0.021566

	operatingCashFlowPerShare	operatingProfitMargin \
returnOnCapitalEmployed	0.260795	0.543895
returnOnAssets	0.238936	0.617132
pretaxProfitMargin	0.254659	0.900086
ebitPerRevenue	0.246873	0.902936
netProfitMargin	0.207209	0.868786
returnOnEquity	0.171138	0.420494
operatingCashFlowPerShare	1.000000	0.188598
operatingProfitMargin	0.188598	1.000000
freeCashFlowPerShare	0.624753	0.190910
debtRatio	-0.082871	-0.088765
effectiveTaxRate	0.159807	0.155647
currentRatio	-0.166358	-0.041878
quickRatio	-0.107110	0.071856
grossProfitMargin	0.128265	0.299462
operatingCashFlowSalesRatio	0.309117	0.494354

	freeCashFlowPerShare	debtRatio \
--	----------------------	-------------

returnOnCapitalEmployed	0.364512	-0.059022
returnOnAssets	0.335259	-0.124251
pretaxProfitMargin	0.249365	-0.142243
ebitPerRevenue	0.239990	-0.150097
netProfitMargin	0.212631	-0.168280
returnOnEquity	0.242287	0.063213
operatingCashFlowPerShare	0.624753	-0.082871
operatingProfitMargin	0.190910	-0.088765
freeCashFlowPerShare	1.000000	-0.099855
debtRatio	-0.099855	1.000000
effectiveTaxRate	0.106856	-0.036203
currentRatio	0.011959	-0.282656
quickRatio	0.029721	-0.183789
grossProfitMargin	0.017095	0.048218
operatingCashFlowSalesRatio	0.180502	-0.179817

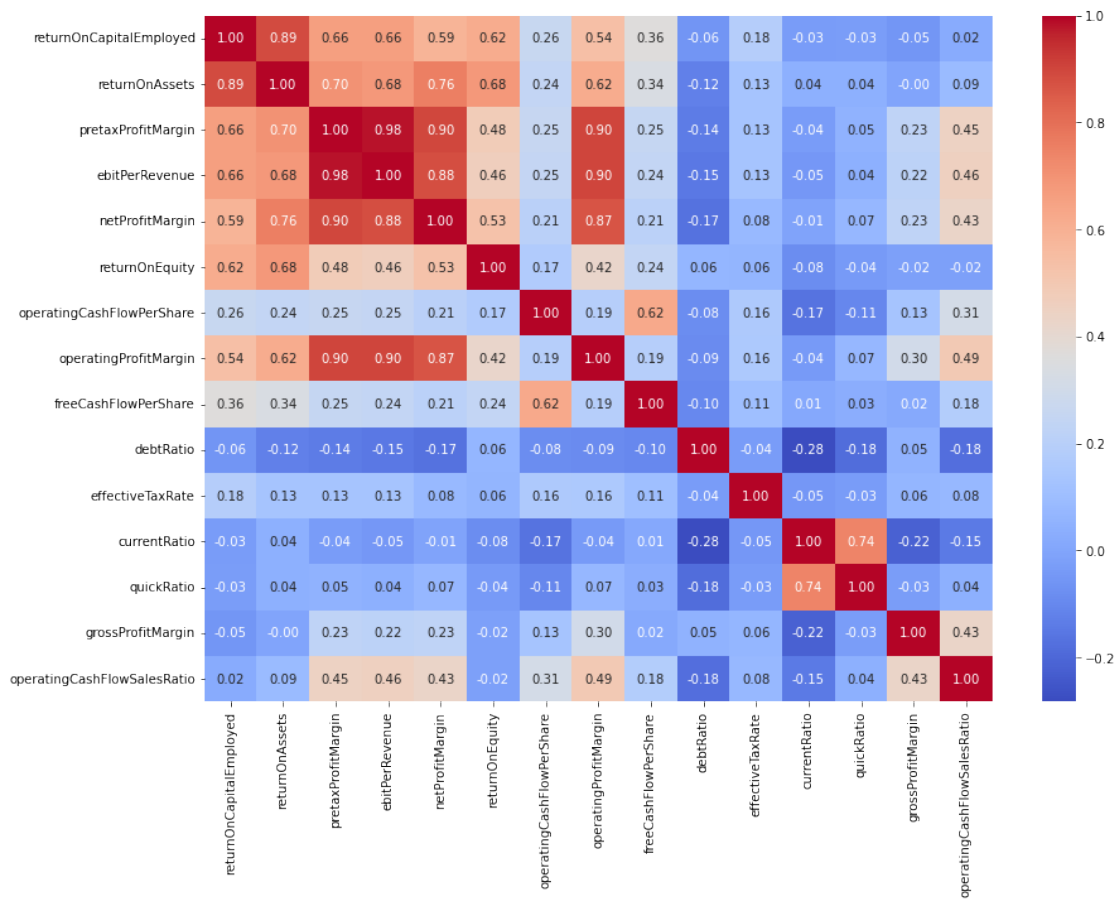
	effectiveTaxRate	currentRatio	quickRatio \
returnOnCapitalEmployed	0.184837	-0.033028	-0.032124
returnOnAssets	0.131880	0.039200	0.038224
pretaxProfitMargin	0.128690	-0.036303	0.048278
ebitPerRevenue	0.133130	-0.049665	0.041701
netProfitMargin	0.078650	-0.007933	0.071742
returnOnEquity	0.059564	-0.084788	-0.041947
operatingCashFlowPerShare	0.159807	-0.166358	-0.107110
operatingProfitMargin	0.155647	-0.041878	0.071856
freeCashFlowPerShare	0.106856	0.011959	0.029721
debtRatio	-0.036203	-0.282656	-0.183789
effectiveTaxRate	1.000000	-0.052083	-0.034838
currentRatio	-0.052083	1.000000	0.744222
quickRatio	-0.034838	0.744222	1.000000
grossProfitMargin	0.063061	-0.219900	-0.029425
operatingCashFlowSalesRatio	0.076522	-0.147103	0.038236

	grossProfitMargin	operatingCashFlowSalesRatio
returnOnCapitalEmployed	-0.053749	0.018203
returnOnAssets	-0.004122	0.085074
pretaxProfitMargin	0.226039	0.447026
ebitPerRevenue	0.216260	0.457347
netProfitMargin	0.225224	0.432589
returnOnEquity	-0.020315	-0.021566
operatingCashFlowPerShare	0.128265	0.309117
operatingProfitMargin	0.299462	0.494354
freeCashFlowPerShare	0.017095	0.180502
debtRatio	0.048218	-0.179817
effectiveTaxRate	0.063061	0.076522
currentRatio	-0.219900	-0.147103
quickRatio	-0.029425	0.038236
grossProfitMargin	1.000000	0.432852

operatingCashFlowSalesRatio

0.432852

1.000000



```
[641]: vif(credit_rating_outl_scaled, pred_back_num)
```

	feature	VIF
0	returnOnCapitalEmployed	8.839647
1	returnOnAssets	11.871796
2	pretaxProfitMargin	36.954863
3	ebitPerRevenue	37.143582
4	netProfitMargin	11.512274
5	returnOnEquity	2.030683
6	operatingCashFlowPerShare	1.885306
7	operatingProfitMargin	7.164503
8	freeCashFlowPerShare	1.841529
9	debtRatio	1.254965
10	effectiveTaxRate	1.091452
11	currentRatio	2.818958
12	quickRatio	2.436735
13	grossProfitMargin	1.381431
14	operatingCashFlowSalesRatio	2.085438

```
[642]: class_models_macro_f1(data_inp=credit_rating_outl_scaled, X_cols=X_cols,
                                y_col = 'Risk_level_encoded', test_size=0.2, num_class = 4)
↪4)
```

Gradient Boosting Macro F1 Score: 0.5637

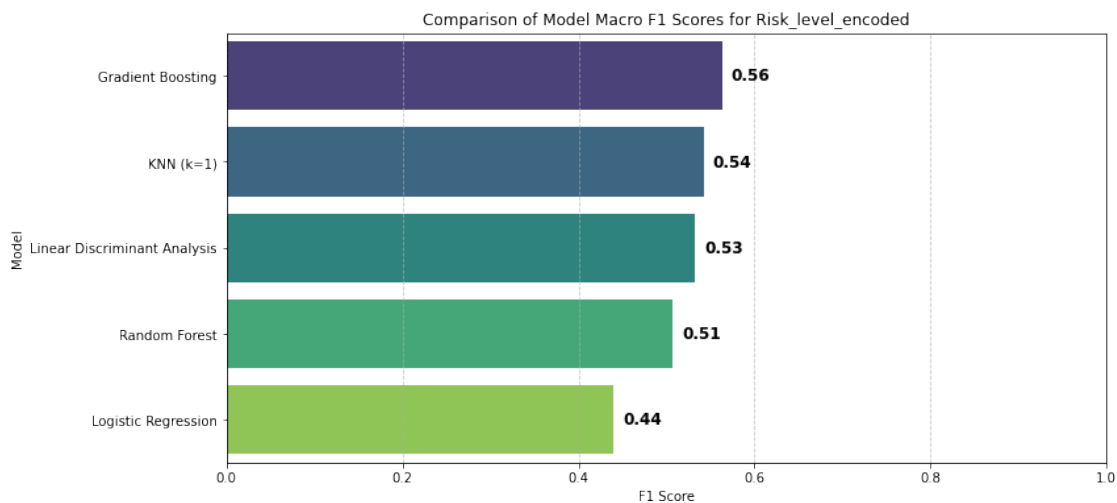
Random Forest Macro F1 Score: 0.5074

Linear Discriminant Analysis Macro F1 Score: 0.5328

Logistic Regression Macro F1 Score: 0.4400

/Users/kamila/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.
warnings.warn(

Best KNN Macro F1 Score (1 neighbors): 0.5421



```
[642]:
```

	Model	F1 Score
0	Gradient Boosting	0.563669
4	KNN (k=1)	0.542134
2	Linear Discriminant Analysis	0.532848
1	Random Forest	0.507412
3	Logistic Regression	0.439982

```
[643]: class_models_macro_f1(data_inp=credit_rating_outl_scaled, X_cols=pred_back,
                                y_col = 'Risk_level_encoded', test_size=0.2, num_class = 4)
↪4)
```

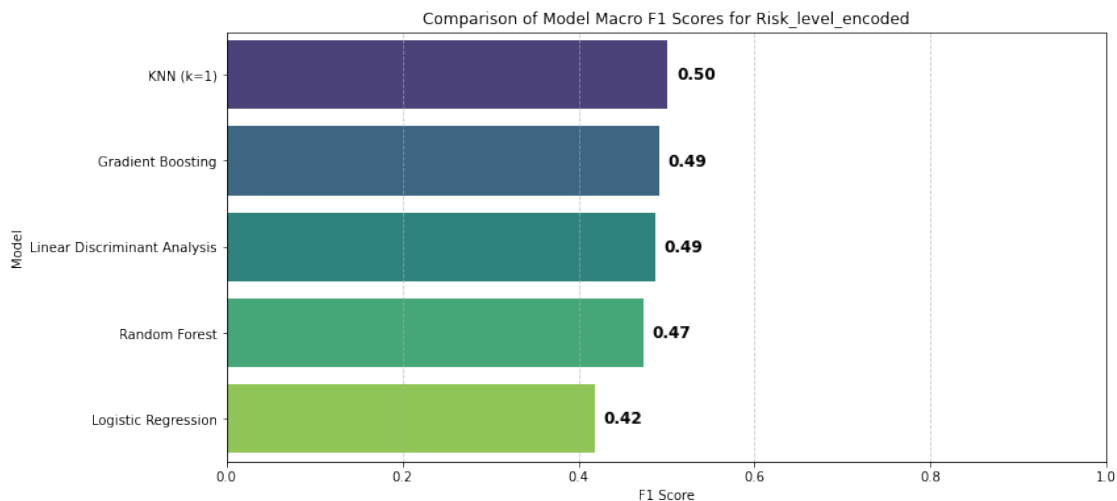
Gradient Boosting Macro F1 Score: 0.4913

Random Forest Macro F1 Score: 0.4741

Linear Discriminant Analysis Macro F1 Score: 0.4868

Logistic Regression Macro F1 Score: 0.4183
Best KNN Macro F1 Score (1 neighbors): 0.5011

/Users/kamila/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.
warnings.warn(



```
[643]:
```

	Model	F1 Score
4	KNN (k=1)	0.501126
0	Gradient Boosting	0.491319
2	Linear Discriminant Analysis	0.486798
1	Random Forest	0.474061
3	Logistic Regression	0.418274

```
[645]: pred_back2 = ['Sector', 'currentRatio', 'grossProfitMargin',  
↳ 'returnOnCapitalEmployed',  
↳ 'assetTurnover', 'fixedAssetTurnover', 'debtEquityRatio',  
↳ 'debtRatio',  
↳ 'enterpriseValueMultiple', 'operatingCashFlowPerShare',  
↳ 'operatingCashFlowSalesRatio']  
  
pred_back_num2 = [ 'currentRatio', 'grossProfitMargin',  
↳ 'returnOnCapitalEmployed',  
↳ 'assetTurnover', 'fixedAssetTurnover', 'debtEquityRatio',  
↳ 'debtRatio',  
↳ 'enterpriseValueMultiple', 'operatingCashFlowPerShare',  
↳ 'operatingCashFlowSalesRatio']
```

```

#Common
'Sector'
'currentRatio'
'grossProfitMargin'
'returnOnCapitalEmployed'
'debtRatio'
'operatingCashFlowPerShare'
'operatingCashFlowSalesRatio'

#different
'assetTurnover'
'fixedAssetTurnover'
'debtEquityRatio'
'enterpriseValueMultiple'

predictors_anova_sector= [, 'returnOnAssets', 'pretaxProfitMargin',
                           'ebitPerRevenue', 'netProfitMargin', 'returnOnEquity',
                           'operatingProfitMargin', 'freeCashFlowPerShare',
↪ 'effectiveTaxRate',
                           'cashRatio', 'quickRatio', ]

predictors_anova_sector= ['returnOnCapitalEmployed', 'returnOnAssets',
↪ 'pretaxProfitMargin',
                           'ebitPerRevenue', 'netProfitMargin', 'returnOnEquity',
↪ 'operatingCashFlowPerShare',
                           'operatingProfitMargin', 'freeCashFlowPerShare',
↪ 'debtRatio', 'effectiveTaxRate',
                           'cashRatio', 'currentRatio', 'quickRatio',
↪ 'grossProfitMargin',
                           'operatingCashFlowSalesRatio', 'Sector']

```

```
[646]: cor(credit_rating_outl_scaled, pred_back_num2)
```

	currentRatio	grossProfitMargin \
currentRatio	1.000000	-0.219900
grossProfitMargin	-0.219900	1.000000
returnOnCapitalEmployed	-0.033028	-0.053749
assetTurnover	0.021907	-0.451027
fixedAssetTurnover	0.062071	-0.142053
debtEquityRatio	-0.214413	0.003641
debtRatio	-0.282656	0.048218
enterpriseValueMultiple	0.003440	0.074222

operatingCashFlowPerShare	-0.166358	0.128265
operatingCashFlowSalesRatio	-0.147103	0.432852

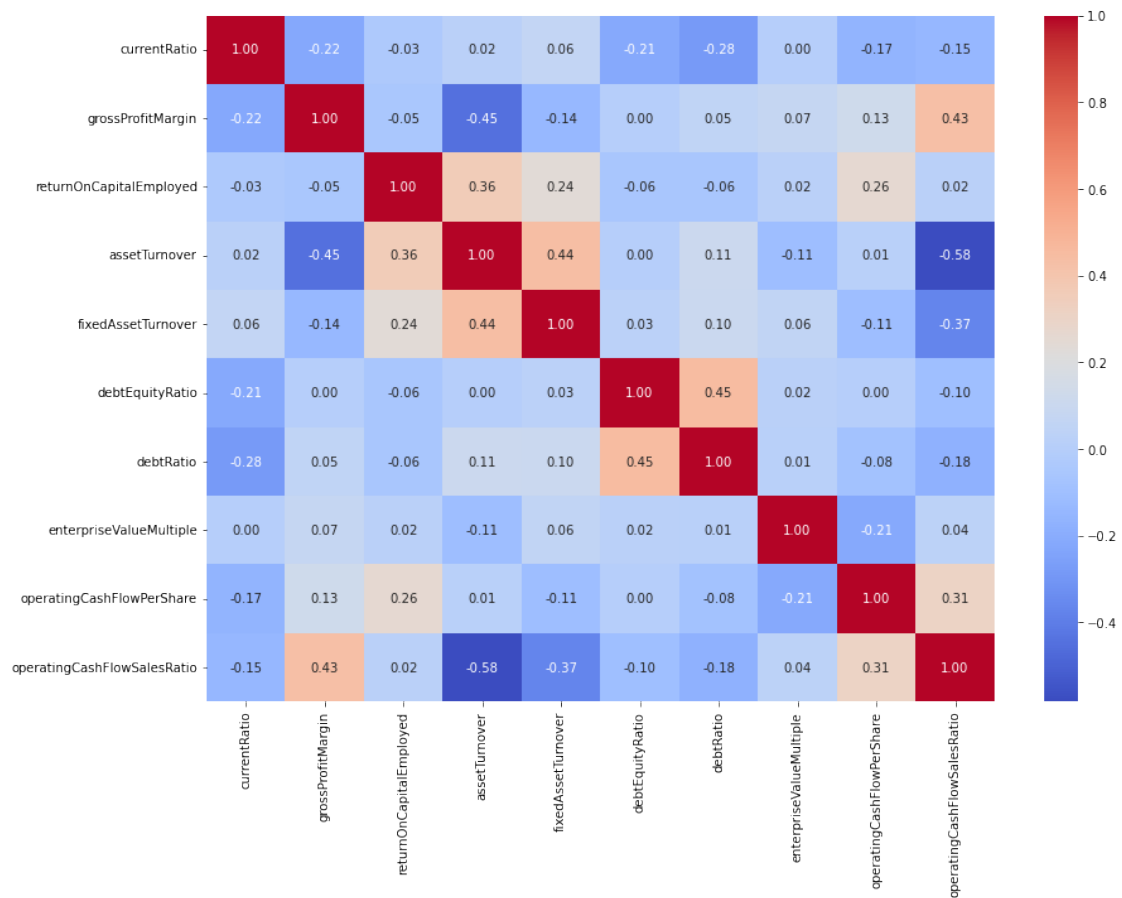
	returnOnCapitalEmployed	assetTurnover	\
currentRatio	-0.033028	0.021907	
grossProfitMargin	-0.053749	-0.451027	
returnOnCapitalEmployed	1.000000	0.359703	
assetTurnover	0.359703	1.000000	
fixedAssetTurnover	0.235372	0.442086	
debtEquityRatio	-0.058467	0.002146	
debtRatio	-0.059022	0.105760	
enterpriseValueMultiple	0.018585	-0.109994	
operatingCashFlowPerShare	0.260795	0.013965	
operatingCashFlowSalesRatio	0.018203	-0.582522	

	fixedAssetTurnover	debtEquityRatio	debtRatio	\
currentRatio	0.062071	-0.214413	-0.282656	
grossProfitMargin	-0.142053	0.003641	0.048218	
returnOnCapitalEmployed	0.235372	-0.058467	-0.059022	
assetTurnover	0.442086	0.002146	0.105760	
fixedAssetTurnover	1.000000	0.027987	0.101206	
debtEquityRatio	0.027987	1.000000	0.453393	
debtRatio	0.101206	0.453393	1.000000	
enterpriseValueMultiple	0.055767	0.019386	0.014406	
operatingCashFlowPerShare	-0.111880	0.002887	-0.082871	
operatingCashFlowSalesRatio	-0.368431	-0.103118	-0.179817	

	enterpriseValueMultiple	\
currentRatio	0.003440	
grossProfitMargin	0.074222	
returnOnCapitalEmployed	0.018585	
assetTurnover	-0.109994	
fixedAssetTurnover	0.055767	
debtEquityRatio	0.019386	
debtRatio	0.014406	
enterpriseValueMultiple	1.000000	
operatingCashFlowPerShare	-0.214936	
operatingCashFlowSalesRatio	0.035597	

	operatingCashFlowPerShare	\
currentRatio	-0.166358	
grossProfitMargin	0.128265	
returnOnCapitalEmployed	0.260795	
assetTurnover	0.013965	
fixedAssetTurnover	-0.111880	
debtEquityRatio	0.002887	
debtRatio	-0.082871	
enterpriseValueMultiple	-0.214936	

operatingCashFlowPerShare	1.000000
operatingCashFlowSalesRatio	0.309117
operatingCashFlowSalesRatio	
currentRatio	-0.147103
grossProfitMargin	0.432852
returnOnCapitalEmployed	0.018203
assetTurnover	-0.582522
fixedAssetTurnover	-0.368431
debtEquityRatio	-0.103118
debtRatio	-0.179817
enterpriseValueMultiple	0.035597
operatingCashFlowPerShare	0.309117
operatingCashFlowSalesRatio	1.000000



```
[647]: vif(credit_rating_outl_scaled, pred_back_num2)
```

	feature	VIF
0	currentRatio	1.215972
1	grossProfitMargin	1.432052

```

2     returnOnCapitalEmployed  1.364029
3         assetTurnover        2.333146
4         fixedAssetTurnover    1.345054
5         debtEquityRatio       1.295913
6         debtRatio            1.404154
7     enterpriseValueMultiple   1.089997
8     operatingCashFlowPerShare 1.333443
9     operatingCashFlowSalesRatio 2.065899

```

```

[649]: class_models_macro_f1(data_inp=credit_rating_outl_scaled, X_cols=X_cols,
        y_col = 'Risk_level_encoded', test_size=0.2, num_class = 4)

```

Gradient Boosting Macro F1 Score: 0.5637

Random Forest Macro F1 Score: 0.5074

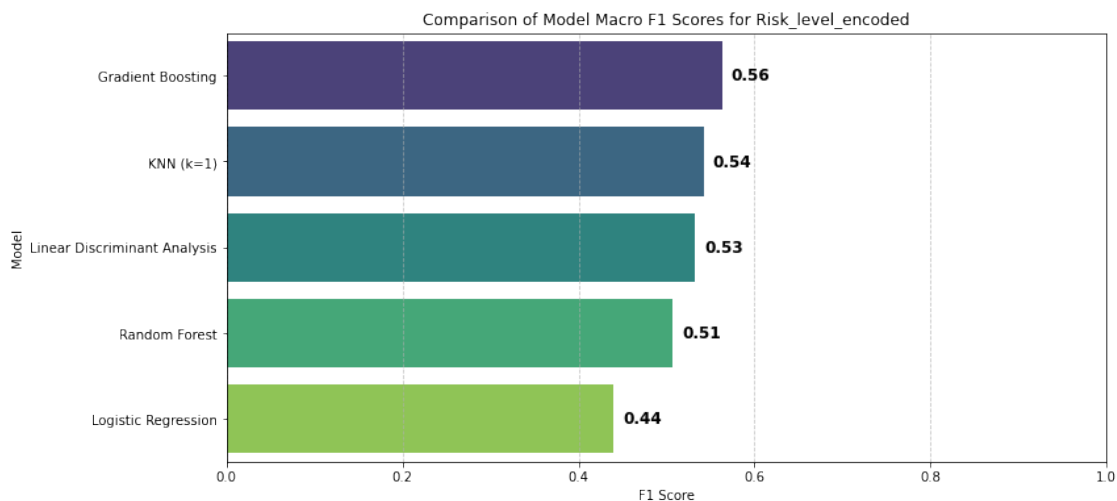
Linear Discriminant Analysis Macro F1 Score: 0.5328

Logistic Regression Macro F1 Score: 0.4400

/Users/kamila/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.

warnings.warn(

Best KNN Macro F1 Score (1 neighbors): 0.5421



```

[649]:
Model  F1 Score
0      Gradient Boosting  0.563669
4      KNN (k=1)         0.542134
2      Linear Discriminant Analysis  0.532848
1      Random Forest     0.507412

```


3 Logistic Regression 0.439982

```
[652]: class_models_macro_f1(data_inp=credit_rating_outl_scaled, X_cols=pred_back2,
                             y_col = 'Risk_level_encoded', test_size=0.2, num_class = 4)
      ↪ 4)
```

Gradient Boosting Macro F1 Score: 0.5133

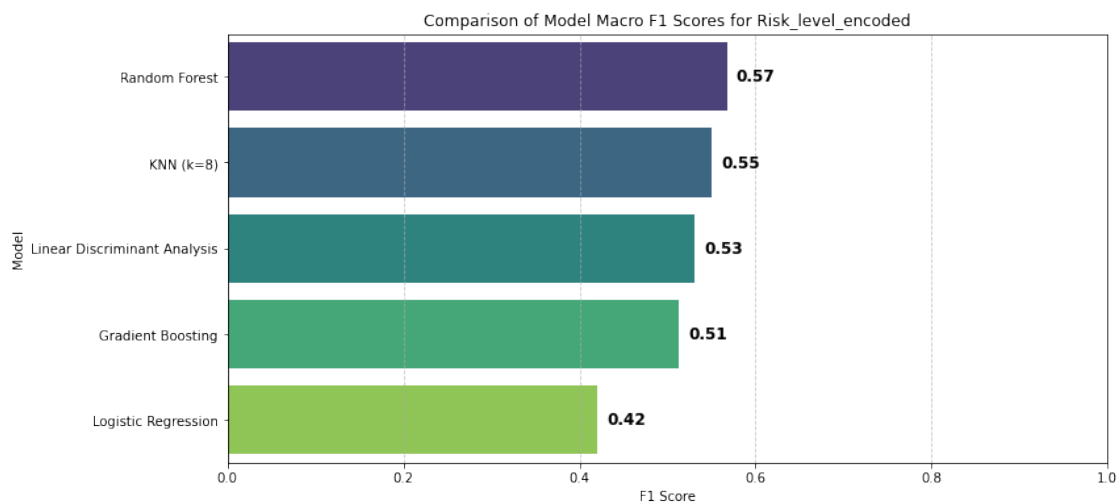
Random Forest Macro F1 Score: 0.5680

Linear Discriminant Analysis Macro F1 Score: 0.5311

Logistic Regression Macro F1 Score: 0.4208

```
/Users/kamila/opt/anaconda3/lib/python3.9/site-
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will
always use 'multinomial'. Leave it to its default value to avoid this warning.
  warnings.warn(
```

Best KNN Macro F1 Score (8 neighbors): 0.5505



```
[652]:
```

	Model	F1 Score
1	Random Forest	0.568035
4	KNN (k=8)	0.550536
2	Linear Discriminant Analysis	0.531054
0	Gradient Boosting	0.513295
3	Logistic Regression	0.420819

```
[653]: class_models_macro_f1(data_inp=credit_rating_outl_scaled, X_cols=X_cols,
                             y_col = 'Rating_encoded', test_size=0.2, num_class = 6)
```

Gradient Boosting Macro F1 Score: 0.4683

Random Forest Macro F1 Score: 0.4746

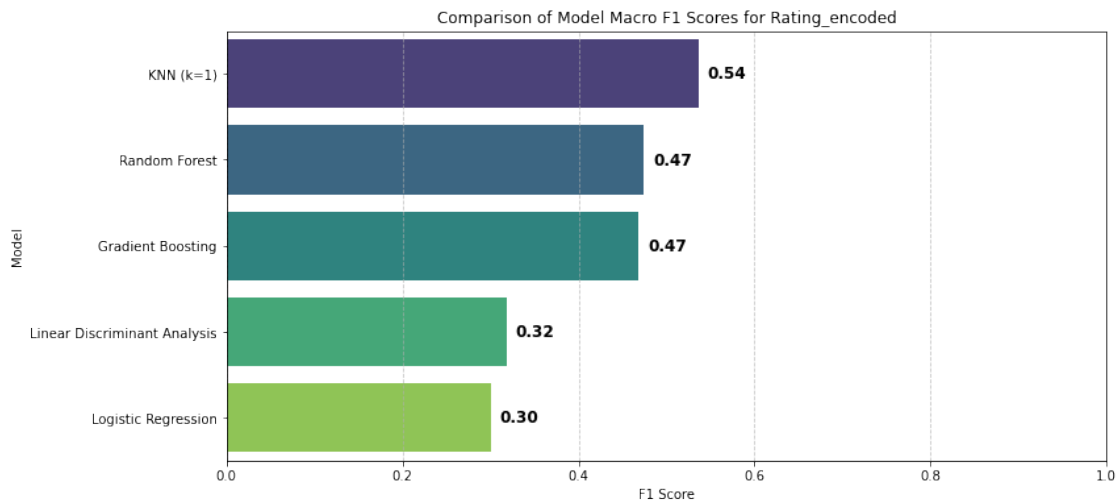
Linear Discriminant Analysis Macro F1 Score: 0.3188

Logistic Regression Macro F1 Score: 0.3006

```
/Users/kamila/opt/anaconda3/lib/python3.9/site-  
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'  
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will  
always use 'multinomial'. Leave it to its default value to avoid this warning.  
warnings.warn(  

```

Best KNN Macro F1 Score (1 neighbors): 0.5368



```
[653]:  
      Model  F1 Score  
4      KNN (k=1)  0.536838  
1      Random Forest  0.474598  
0      Gradient Boosting  0.468263  
2  Linear Discriminant Analysis  0.318815  
3      Logistic Regression  0.300554
```

```
[654]: class_models_macro_f1(data_inp=credit_rating_out1_scaled, X_cols=pred_back2,  
                             y_col = 'Rating_encoded', test_size=0.2, num_class = 6)
```

Gradient Boosting Macro F1 Score: 0.4162

Random Forest Macro F1 Score: 0.4868

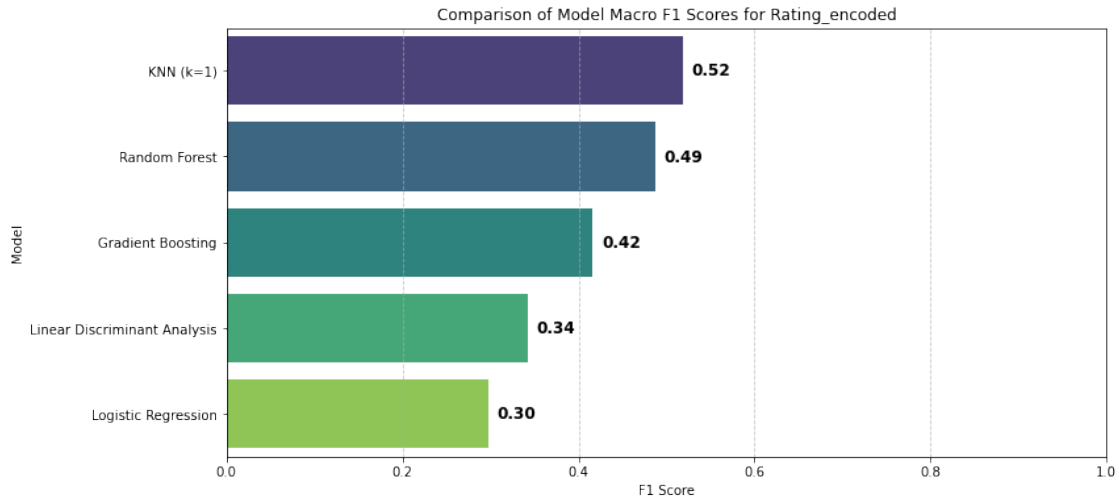
Linear Discriminant Analysis Macro F1 Score: 0.3424

Logistic Regression Macro F1 Score: 0.2978

```
/Users/kamila/opt/anaconda3/lib/python3.9/site-  
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'  
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will  
always use 'multinomial'. Leave it to its default value to avoid this warning.  
warnings.warn(  

```

Best KNN Macro F1 Score (1 neighbors): 0.5181



```
[654]:
```

	Model	F1 Score
4	KNN (k=1)	0.518143
1	Random Forest	0.486822
0	Gradient Boosting	0.416186
2	Linear Discriminant Analysis	0.342437
3	Logistic Regression	0.297799

```
[655]: pred_back2
```

```
[655]: ['Sector',
        'currentRatio',
        'grossProfitMargin',
        'returnOnCapitalEmployed',
        'assetTurnover',
        'fixedAssetTurnover',
        'debtEquityRatio',
        'debtRatio',
        'enterpriseValueMultiple',
        'operatingCashFlowPerShare',
        'operatingCashFlowSalesRatio']
```

```
[669]: backward_elimination(data_inp=credit_rating_outl_scaled,
    ↪ X_cols=pred_anova_risk_rating,
    ↪ y_col = 'Risk_level_encoded', test_size=0.2, num_class =
    ↪ 4)
```

```
[2025-02-05 21:47:34] Features: 21/1 -- score: 0.504843907032419
[2025-02-05 21:55:46] Features: 20/1 -- score: 0.5165433738103384
[2025-02-05 22:03:05] Features: 19/1 -- score: 0.5236961844670863
```

```
[2025-02-05 22:09:49] Features: 18/1 -- score: 0.5189568828438722
[2025-02-05 22:15:47] Features: 17/1 -- score: 0.5116284796577679
[2025-02-05 22:21:07] Features: 16/1 -- score: 0.5326567793011563
[2025-02-05 22:25:53] Features: 15/1 -- score: 0.5252484773768186
[2025-02-05 22:30:02] Features: 14/1 -- score: 0.532464800221612
[2025-02-05 22:33:41] Features: 13/1 -- score: 0.5418759013024349
[2025-02-05 22:36:51] Features: 12/1 -- score: 0.5322712662810556
[2025-02-05 22:38:33] Features: 11/1 -- score: 0.5308222067909979
[2025-02-05 22:40:03] Features: 10/1 -- score: 0.5387024346960121
[2025-02-05 22:41:19] Features: 9/1 -- score: 0.5339566824700815
[2025-02-05 22:42:16] Features: 8/1 -- score: 0.5290861991910063
[2025-02-05 22:43:03] Features: 7/1 -- score: 0.5257300549611742
[2025-02-05 22:43:39] Features: 6/1 -- score: 0.5383359887286892
[2025-02-05 22:44:05] Features: 5/1 -- score: 0.5313702695559822
[2025-02-05 22:44:25] Features: 4/1 -- score: 0.48951512041020795
[2025-02-05 22:44:39] Features: 3/1 -- score: 0.47510334286780537
[2025-02-05 22:44:46] Features: 2/1 -- score: 0.424601288484787
```

Optimal feature set: ['returnOnCapitalEmployed', 'returnOnAssets', 'operatingProfitMargin', 'operatingCashFlowPerShare', 'debtRatio', 'quickRatio', 'operatingCashFlowSalesRatio', 'currentRatio', 'cashRatio', 'grossProfitMargin', 'companyEquityMultiplier', 'Sector', 'fixedAssetTurnover']

```
[2025-02-05 22:44:51] Features: 1/1 -- score: 0.38231589104456315
```

```
[669]: ['returnOnCapitalEmployed',
        'returnOnAssets',
        'operatingProfitMargin',
        'operatingCashFlowPerShare',
        'debtRatio',
        'quickRatio',
        'operatingCashFlowSalesRatio',
        'currentRatio',
        'cashRatio',
        'grossProfitMargin',
        'companyEquityMultiplier',
        'Sector',
        'fixedAssetTurnover']
```

```
[671]: pred_back3 = ['returnOnCapitalEmployed',
                    'returnOnAssets',
                    'operatingProfitMargin',
                    'operatingCashFlowPerShare',
                    'debtRatio',
                    'quickRatio',
                    'operatingCashFlowSalesRatio',
                    'currentRatio',
```

```

'cashRatio',
'grossProfitMargin',
'companyEquityMultiplier',
'Sector',
'fixedAssetTurnover']

pred_back_num3 = ['returnOnCapitalEmployed',
'returnOnAssets',
'operatingProfitMargin',
'operatingCashFlowPerShare',
'debtRatio',
'quickRatio',
'operatingCashFlowSalesRatio',
'currentRatio',
'cashRatio',
'grossProfitMargin',
'companyEquityMultiplier',
'fixedAssetTurnover']

```

```
[672]: cor(credit_rating_outl_scaled, pred_back_num3)
```

	returnOnCapitalEmployed	returnOnAssets \
returnOnCapitalEmployed	1.000000	0.887018
returnOnAssets	0.887018	1.000000
operatingProfitMargin	0.543895	0.617132
operatingCashFlowPerShare	0.260795	0.238936
debtRatio	-0.059022	-0.124251
quickRatio	-0.032124	0.038224
operatingCashFlowSalesRatio	0.018203	0.085074
currentRatio	-0.033028	0.039200
cashRatio	-0.024113	0.043907
grossProfitMargin	-0.053749	-0.004122
companyEquityMultiplier	-0.063422	-0.102530
fixedAssetTurnover	0.235372	0.170188

	operatingProfitMargin	operatingCashFlowPerShare \
returnOnCapitalEmployed	0.543895	0.260795
returnOnAssets	0.617132	0.238936
operatingProfitMargin	1.000000	0.188598
operatingCashFlowPerShare	0.188598	1.000000
debtRatio	-0.088765	-0.082871
quickRatio	0.071856	-0.107110
operatingCashFlowSalesRatio	0.494354	0.309117
currentRatio	-0.041878	-0.166358
cashRatio	0.041486	-0.141165
grossProfitMargin	0.299462	0.128265
companyEquityMultiplier	-0.031203	-0.000698

fixedAssetTurnover	-0.079582	-0.111880
--------------------	-----------	-----------

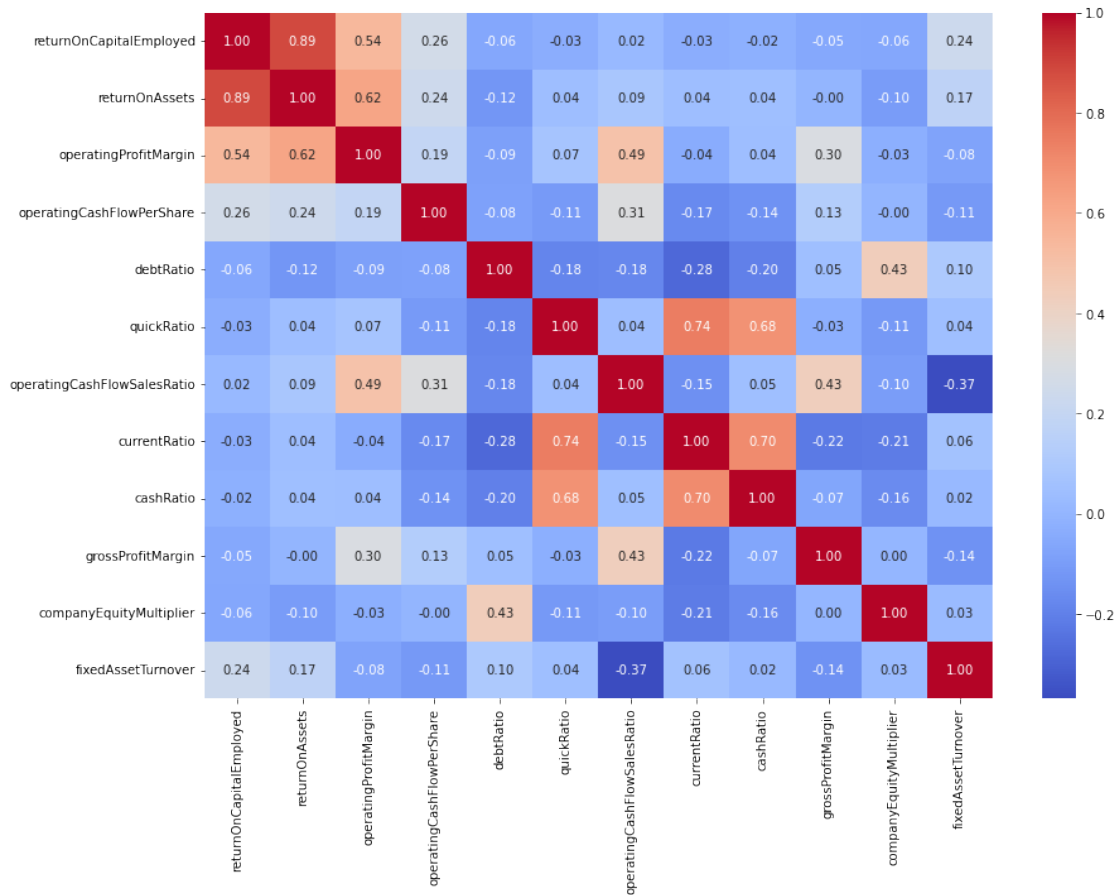
	debtRatio	quickRatio \
returnOnCapitalEmployed	-0.059022	-0.032124
returnOnAssets	-0.124251	0.038224
operatingProfitMargin	-0.088765	0.071856
operatingCashFlowPerShare	-0.082871	-0.107110
debtRatio	1.000000	-0.183789
quickRatio	-0.183789	1.000000
operatingCashFlowSalesRatio	-0.179817	0.038236
currentRatio	-0.282656	0.744222
cashRatio	-0.201487	0.683437
grossProfitMargin	0.048218	-0.029425
companyEquityMultiplier	0.433926	-0.113647
fixedAssetTurnover	0.101206	0.044912

	operatingCashFlowSalesRatio	currentRatio \
returnOnCapitalEmployed	0.018203	-0.033028
returnOnAssets	0.085074	0.039200
operatingProfitMargin	0.494354	-0.041878
operatingCashFlowPerShare	0.309117	-0.166358
debtRatio	-0.179817	-0.282656
quickRatio	0.038236	0.744222
operatingCashFlowSalesRatio	1.000000	-0.147103
currentRatio	-0.147103	1.000000
cashRatio	0.048369	0.701051
grossProfitMargin	0.432852	-0.219900
companyEquityMultiplier	-0.099772	-0.213520
fixedAssetTurnover	-0.368431	0.062071

	cashRatio	grossProfitMargin \
returnOnCapitalEmployed	-0.024113	-0.053749
returnOnAssets	0.043907	-0.004122
operatingProfitMargin	0.041486	0.299462
operatingCashFlowPerShare	-0.141165	0.128265
debtRatio	-0.201487	0.048218
quickRatio	0.683437	-0.029425
operatingCashFlowSalesRatio	0.048369	0.432852
currentRatio	0.701051	-0.219900
cashRatio	1.000000	-0.066873
grossProfitMargin	-0.066873	1.000000
companyEquityMultiplier	-0.162281	0.003464
fixedAssetTurnover	0.017586	-0.142053

	companyEquityMultiplier	fixedAssetTurnover
returnOnCapitalEmployed	-0.063422	0.235372
returnOnAssets	-0.102530	0.170188
operatingProfitMargin	-0.031203	-0.079582

operatingCashFlowPerShare	-0.000698	-0.111880
debtRatio	0.433926	0.101206
quickRatio	-0.113647	0.044912
operatingCashFlowSalesRatio	-0.099772	-0.368431
currentRatio	-0.213520	0.062071
cashRatio	-0.162281	0.017586
grossProfitMargin	0.003464	-0.142053
companyEquityMultiplier	1.000000	0.027088
fixedAssetTurnover	0.027088	1.000000



```
[673]: vif(credit_rating_outl_scaled, pred_back_num3)
```

	feature	VIF
0	returnOnCapitalEmployed	5.267462
1	returnOnAssets	5.595442
2	operatingProfitMargin	2.604743
3	operatingCashFlowPerShare	1.271488
4	debtRatio	1.383643
5	quickRatio	2.700040
6	operatingCashFlowSalesRatio	2.163152

```

7         currentRatio    3.298910
8         cashRatio       2.307002
9         grossProfitMargin 1.365574
10        companyEquityMultiplier 1.275672
11        fixedAssetTurnover 1.270430

```

```

[674]: class_models_macro_f1(data_inp=credit_rating_outl_scaled, X_cols=X_cols,
                                y_col = 'Risk_level_encoded', test_size=0.2, num_class = 4)

```

Gradient Boosting Macro F1 Score: 0.5637

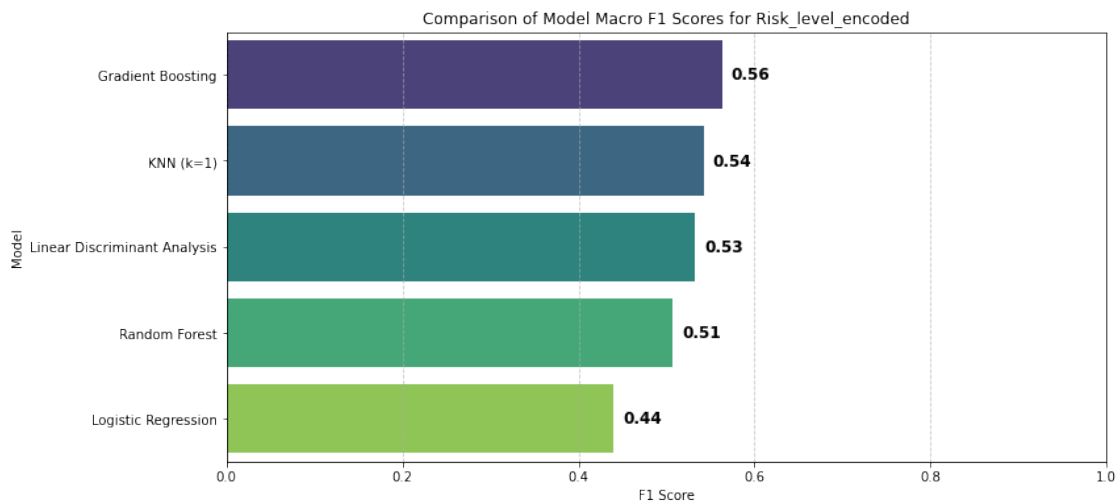
Random Forest Macro F1 Score: 0.5074

Linear Discriminant Analysis Macro F1 Score: 0.5328

Logistic Regression Macro F1 Score: 0.4400

/Users/kamila/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.
warnings.warn(

Best KNN Macro F1 Score (1 neighbors): 0.5421



```

[674]:
0         Gradient Boosting    0.563669
4         KNN (k=1)           0.542134
2  Linear Discriminant Analysis 0.532848
1         Random Forest       0.507412
3         Logistic Regression  0.439982

```



```
[675]: class_models_macro_f1(data_inp=credit_rating_outl_scaled, X_cols=pred_back3,
                             y_col = 'Risk_level_encoded', test_size=0.2, num_class = 4)
↳4)
```

Gradient Boosting Macro F1 Score: 0.5208

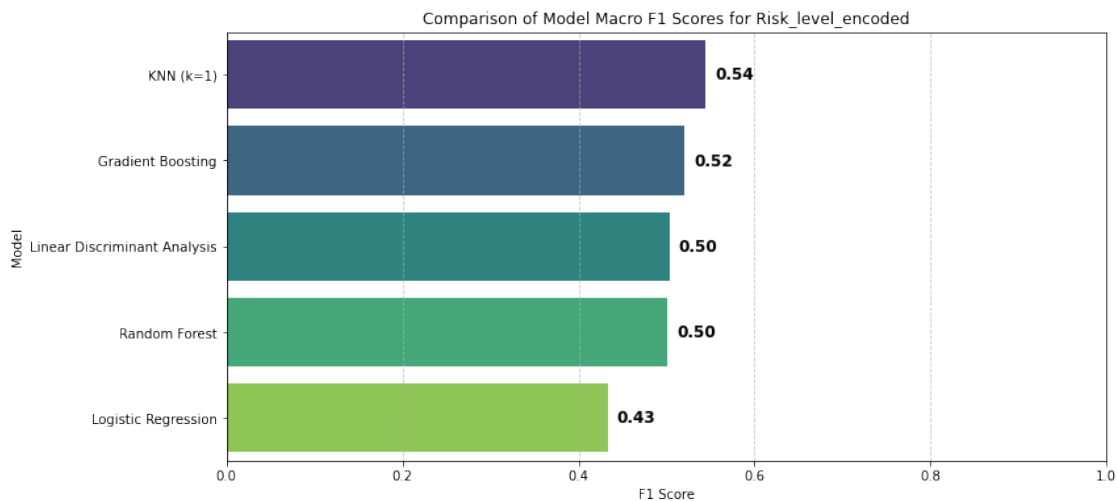
Random Forest Macro F1 Score: 0.5015

Linear Discriminant Analysis Macro F1 Score: 0.5038

Logistic Regression Macro F1 Score: 0.4338

Best KNN Macro F1 Score (1 neighbors): 0.5444

/Users/kamila/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.
warnings.warn(



```
[675]:
Model  F1 Score
4      KNN (k=1)  0.544425
0      Gradient Boosting  0.520794
2  Linear Discriminant Analysis  0.503817
1      Random Forest  0.501481
3      Logistic Regression  0.433781
```

```
[738]: class_models_macro_f1(data_inp=credit_rating_outl_scaled, X_cols=X_cols,
                             y_col = 'Rating_encoded', test_size=0.2, num_class = 6)
```

Gradient Boosting Macro F1 Score: 0.4683

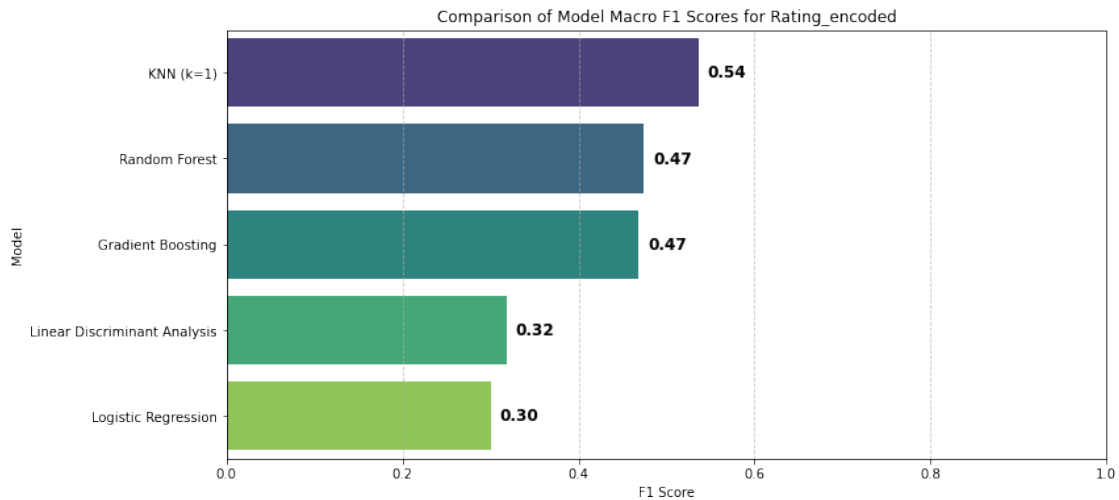
Random Forest Macro F1 Score: 0.4746

Linear Discriminant Analysis Macro F1 Score: 0.3188

Logistic Regression Macro F1 Score: 0.3006

```
/Users/kamila/opt/anaconda3/lib/python3.9/site-
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will
always use 'multinomial'. Leave it to its default value to avoid this warning.
warnings.warn(
```

Best KNN Macro F1 Score (1 neighbors): 0.5368



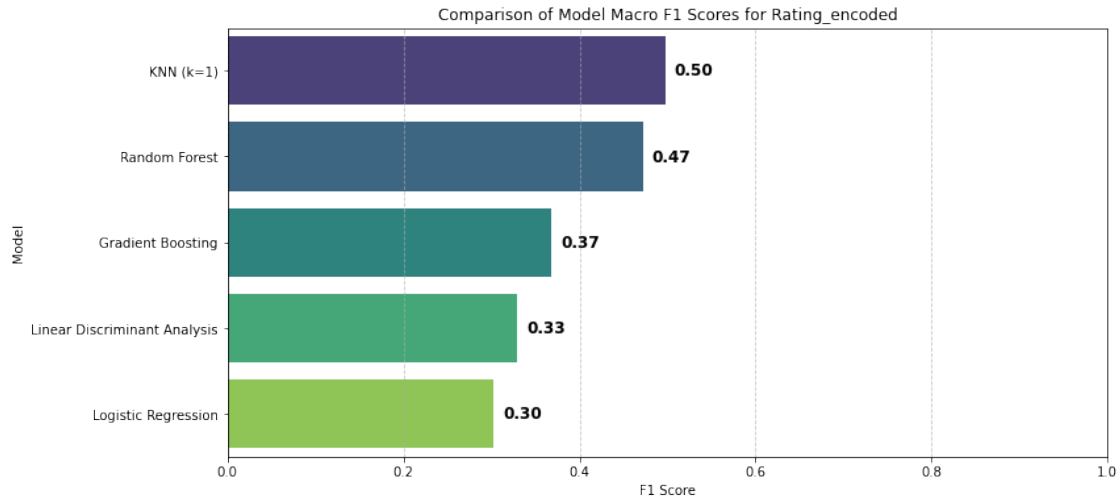
```
[738]:
```

	Model	F1 Score
4	KNN (k=1)	0.536838
1	Random Forest	0.474598
0	Gradient Boosting	0.468263
2	Linear Discriminant Analysis	0.318815
3	Logistic Regression	0.300554

```
[739]: class_models_macro_f1(data_inp=credit_rating_out1_scaled, X_cols=pred_back3,
                             y_col = 'Rating_encoded', test_size=0.2, num_class = 6)
```

Gradient Boosting Macro F1 Score: 0.3685
Random Forest Macro F1 Score: 0.4722
Linear Discriminant Analysis Macro F1 Score: 0.3295
Logistic Regression Macro F1 Score: 0.3028
Best KNN Macro F1 Score (1 neighbors): 0.4980

```
/Users/kamila/opt/anaconda3/lib/python3.9/site-
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will
always use 'multinomial'. Leave it to its default value to avoid this warning.
warnings.warn(
```



```
[739]:
Model  F1 Score
4      KNN (k=1)  0.497951
1      Random Forest  0.472195
0      Gradient Boosting  0.368499
2  Linear Discriminant Analysis  0.329468
3      Logistic Regression  0.302753
```

```
[677]: forward_select_gb(data_inp=credit_rating_outl_scaled,
    ↪X_cols=pred_anova_risk_rating,
    y_col = 'Risk_level_encoded', test_size=0.2, num_class =
    ↪4)
```

```
[2025-02-05 23:00:28] Features: 1/22 -- score: 0.38231589104456315
[2025-02-05 23:01:31] Features: 2/22 -- score: 0.424601288484787
[2025-02-05 23:02:40] Features: 3/22 -- score: 0.47510334286780537
[2025-02-05 23:03:49] Features: 4/22 -- score: 0.5011246901935245
[2025-02-05 23:05:13] Features: 5/22 -- score: 0.5283223636471753
[2025-02-05 23:06:42] Features: 6/22 -- score: 0.5397530751728429
[2025-02-05 23:08:12] Features: 7/22 -- score: 0.5341003384214797
[2025-02-05 23:09:49] Features: 8/22 -- score: 0.5411955962944145
[2025-02-05 23:11:28] Features: 9/22 -- score: 0.5322457623306309
[2025-02-05 23:13:09] Features: 10/22 -- score: 0.5464660245368421
[2025-02-05 23:14:57] Features: 11/22 -- score: 0.5475699830481376
[2025-02-05 23:16:38] Features: 12/22 -- score: 0.5348512529246349
[2025-02-05 23:18:14] Features: 13/22 -- score: 0.5343265075083485
[2025-02-05 23:20:00] Features: 14/22 -- score: 0.5321793037202189
[2025-02-05 23:21:45] Features: 15/22 -- score: 0.5235875979496043
[2025-02-05 23:23:12] Features: 16/22 -- score: 0.5331075647995666
[2025-02-05 23:24:30] Features: 17/22 -- score: 0.5297532707970842
```

```
[2025-02-05 23:25:37] Features: 18/22 -- score: 0.5244288310332426
[2025-02-05 23:26:32] Features: 19/22 -- score: 0.5142587762609325
[2025-02-05 23:27:13] Features: 20/22 -- score: 0.5126061060239728
[2025-02-05 23:27:43] Features: 21/22 -- score: 0.5024745674861918
```

```
Optimal feature set: ['returnOnCapitalEmployed', 'operatingCashFlowPerShare',
'debtRatio', 'freeCashFlowPerShare', 'effectiveTaxRate',
'operatingCashFlowSalesRatio', 'currentRatio', 'grossProfitMargin',
'debtEquityRatio', 'Sector', 'fixedAssetTurnover']
```

```
[2025-02-05 23:27:58] Features: 22/22 -- score: 0.49973069566094086
```

```
[677]: ['returnOnCapitalEmployed',
        'operatingCashFlowPerShare',
        'debtRatio',
        'freeCashFlowPerShare',
        'effectiveTaxRate',
        'operatingCashFlowSalesRatio',
        'currentRatio',
        'grossProfitMargin',
        'debtEquityRatio',
        'Sector',
        'fixedAssetTurnover']
```

```
[680]: pred_back4 = ['returnOnCapitalEmployed',
                    'operatingCashFlowPerShare',
                    'debtRatio',
                    'freeCashFlowPerShare',
                    'effectiveTaxRate',
                    'operatingCashFlowSalesRatio',
                    'currentRatio',
                    'grossProfitMargin',
                    'debtEquityRatio',
                    'Sector',
                    'fixedAssetTurnover']

pred_back_num4 = ['returnOnCapitalEmployed',
                  'operatingCashFlowPerShare',
                  'debtRatio',
                  'freeCashFlowPerShare',
                  'effectiveTaxRate',
                  'operatingCashFlowSalesRatio',
                  'currentRatio',
                  'grossProfitMargin',
                  'debtEquityRatio',
                  'fixedAssetTurnover']
```

```
[681]: cor(credit_rating_outl_scaled, pred_back_num4)
```

	returnOnCapitalEmployed \
returnOnCapitalEmployed	1.000000
operatingCashFlowPerShare	0.260795
debtRatio	-0.059022
freeCashFlowPerShare	0.364512
effectiveTaxRate	0.184837
operatingCashFlowSalesRatio	0.018203
currentRatio	-0.033028
grossProfitMargin	-0.053749
debtEquityRatio	-0.058467
fixedAssetTurnover	0.235372

	operatingCashFlowPerShare	debtRatio \
returnOnCapitalEmployed	0.260795	-0.059022
operatingCashFlowPerShare	1.000000	-0.082871
debtRatio	-0.082871	1.000000
freeCashFlowPerShare	0.624753	-0.099855
effectiveTaxRate	0.159807	-0.036203
operatingCashFlowSalesRatio	0.309117	-0.179817
currentRatio	-0.166358	-0.282656
grossProfitMargin	0.128265	0.048218
debtEquityRatio	0.002887	0.453393
fixedAssetTurnover	-0.111880	0.101206

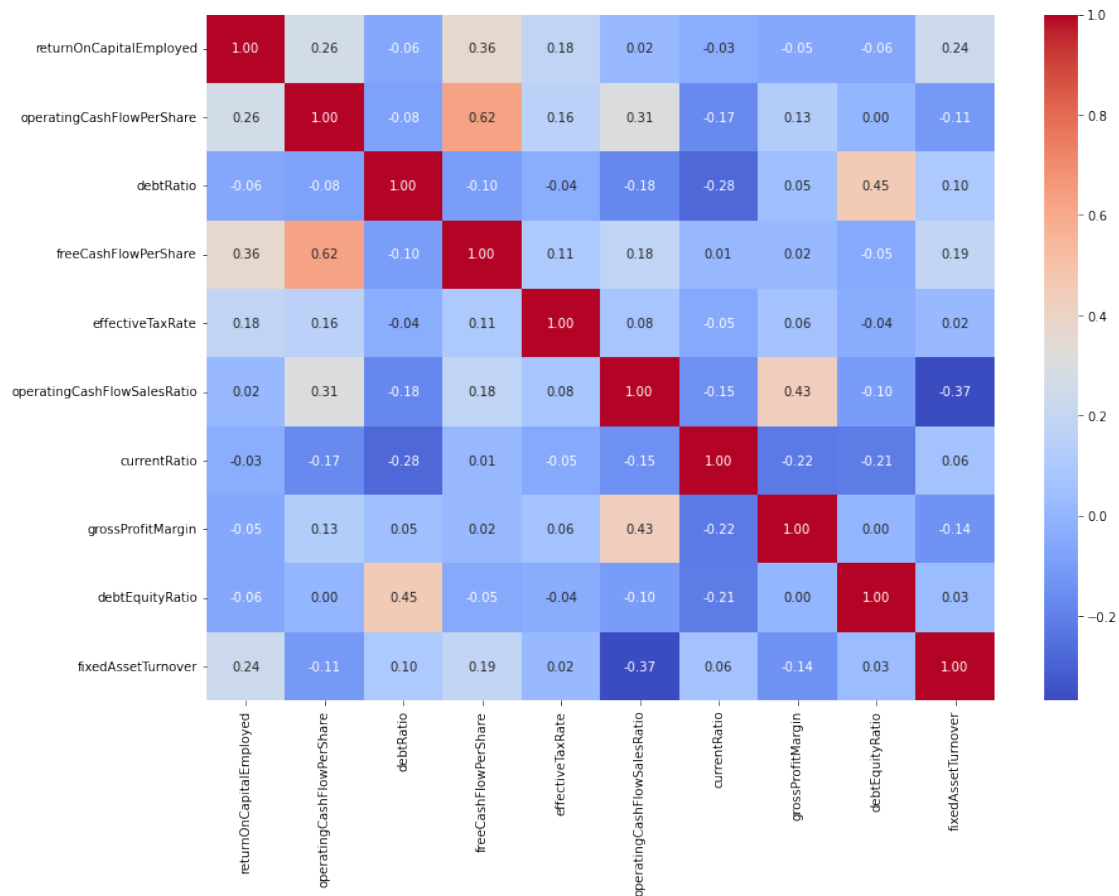
	freeCashFlowPerShare	effectiveTaxRate \
returnOnCapitalEmployed	0.364512	0.184837
operatingCashFlowPerShare	0.624753	0.159807
debtRatio	-0.099855	-0.036203
freeCashFlowPerShare	1.000000	0.106856
effectiveTaxRate	0.106856	1.000000
operatingCashFlowSalesRatio	0.180502	0.076522
currentRatio	0.011959	-0.052083
grossProfitMargin	0.017095	0.063061
debtEquityRatio	-0.051063	-0.040646
fixedAssetTurnover	0.189949	0.019901

	operatingCashFlowSalesRatio	currentRatio \
returnOnCapitalEmployed	0.018203	-0.033028
operatingCashFlowPerShare	0.309117	-0.166358
debtRatio	-0.179817	-0.282656
freeCashFlowPerShare	0.180502	0.011959
effectiveTaxRate	0.076522	-0.052083
operatingCashFlowSalesRatio	1.000000	-0.147103
currentRatio	-0.147103	1.000000
grossProfitMargin	0.432852	-0.219900

debtEquityRatio	-0.103118	-0.214413
fixedAssetTurnover	-0.368431	0.062071

	grossProfitMargin	debtEquityRatio \
returnOnCapitalEmployed	-0.053749	-0.058467
operatingCashFlowPerShare	0.128265	0.002887
debtRatio	0.048218	0.453393
freeCashFlowPerShare	0.017095	-0.051063
effectiveTaxRate	0.063061	-0.040646
operatingCashFlowSalesRatio	0.432852	-0.103118
currentRatio	-0.219900	-0.214413
grossProfitMargin	1.000000	0.003641
debtEquityRatio	0.003641	1.000000
fixedAssetTurnover	-0.142053	0.027987

	fixedAssetTurnover
returnOnCapitalEmployed	0.235372
operatingCashFlowPerShare	-0.111880
debtRatio	0.101206
freeCashFlowPerShare	0.189949
effectiveTaxRate	0.019901
operatingCashFlowSalesRatio	-0.368431
currentRatio	0.062071
grossProfitMargin	-0.142053
debtEquityRatio	0.027987
fixedAssetTurnover	1.000000



```
[682]: vif(credit_rating_outl_scaled, pred_back_num4)
```

	feature	VIF
0	returnOnCapitalEmployed	1.246317
1	operatingCashFlowPerShare	1.964979
2	debtRatio	1.397848
3	freeCashFlowPerShare	2.002913
4	effectiveTaxRate	1.056791
5	operatingCashFlowSalesRatio	1.600154
6	currentRatio	1.221725
7	grossProfitMargin	1.295520
8	debtEquityRatio	1.283023
9	fixedAssetTurnover	1.384938

```
[683]: class_models_macro_f1(data_inp=credit_rating_outl_scaled, X_cols=X_cols,
                             y_col = 'Risk_level_encoded', test_size=0.2, num_class = 4)
→4)
```

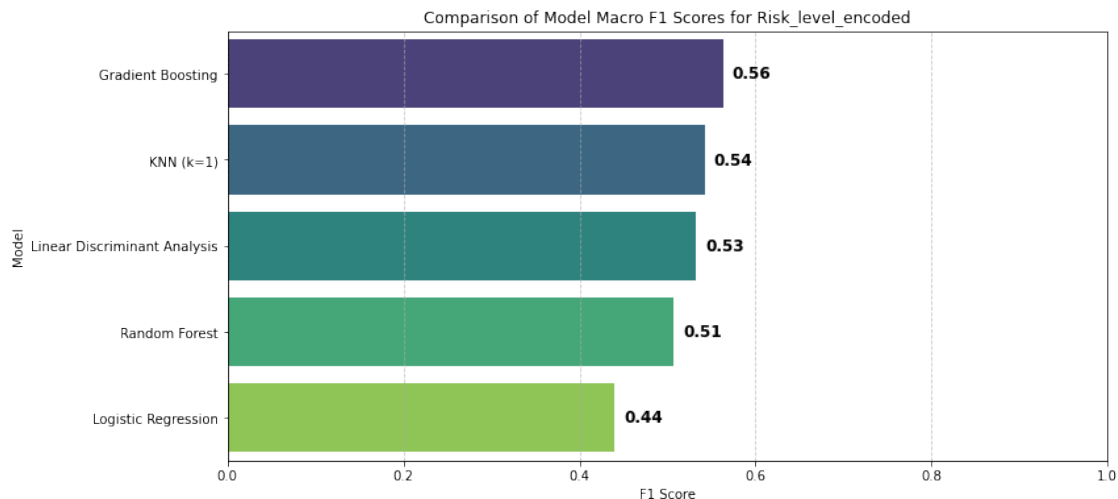
Gradient Boosting Macro F1 Score: 0.5637
Random Forest Macro F1 Score: 0.5074

Linear Discriminant Analysis Macro F1 Score: 0.5328

Logistic Regression Macro F1 Score: 0.4400

```
/Users/kamila/opt/anaconda3/lib/python3.9/site-  
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'  
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will  
always use 'multinomial'. Leave it to its default value to avoid this warning.  
warnings.warn(
```

Best KNN Macro F1 Score (1 neighbors): 0.5421



```
[683]:
```

	Model	F1 Score
0	Gradient Boosting	0.563669
4	KNN (k=1)	0.542134
2	Linear Discriminant Analysis	0.532848
1	Random Forest	0.507412
3	Logistic Regression	0.439982

```
[684]: class_models_macro_f1(data_inp=credit_rating_out1_scaled, X_cols= pred_back4,  
                             y_col = 'Risk_level_encoded', test_size=0.2, num_class = 4)  
↪4)
```

Gradient Boosting Macro F1 Score: 0.4963

Random Forest Macro F1 Score: 0.5473

Linear Discriminant Analysis Macro F1 Score: 0.4791

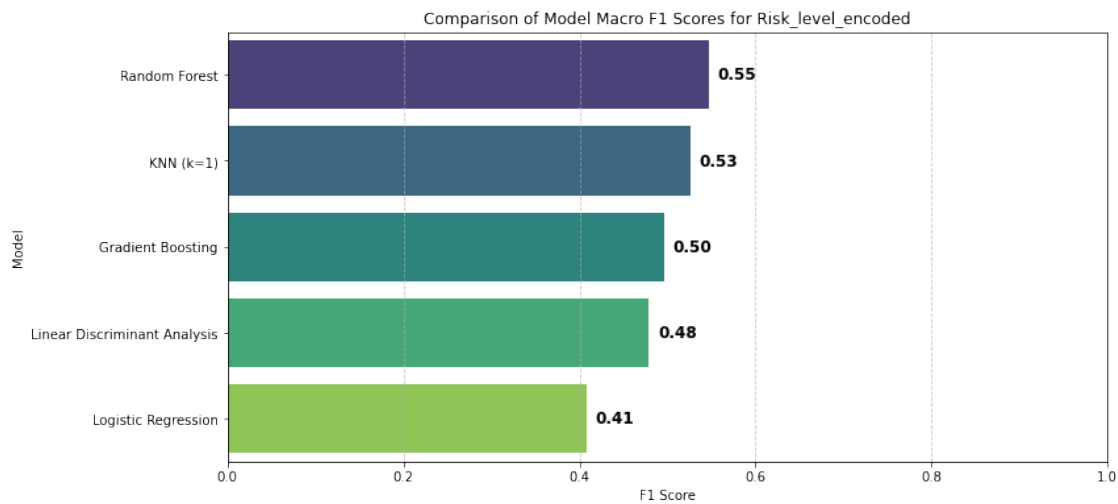
Logistic Regression Macro F1 Score: 0.4081

Best KNN Macro F1 Score (1 neighbors): 0.5264

```
/Users/kamila/opt/anaconda3/lib/python3.9/site-  
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'  
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will  
always use 'multinomial'. Leave it to its default value to avoid this warning.
```



```
warnings.warn(
```



```
[684]:
```

	Model	F1 Score
1	Random Forest	0.547258
4	KNN (k=1)	0.526372
0	Gradient Boosting	0.496282
2	Linear Discriminant Analysis	0.479070
3	Logistic Regression	0.408087

```
[736]: class_models_macro_f1(data_inp=credit_rating_outl_scaled, X_cols= X_cols,
                             y_col = 'Rating_encoded', test_size=0.2, num_class = 6)
```

Gradient Boosting Macro F1 Score: 0.4683

Random Forest Macro F1 Score: 0.4746

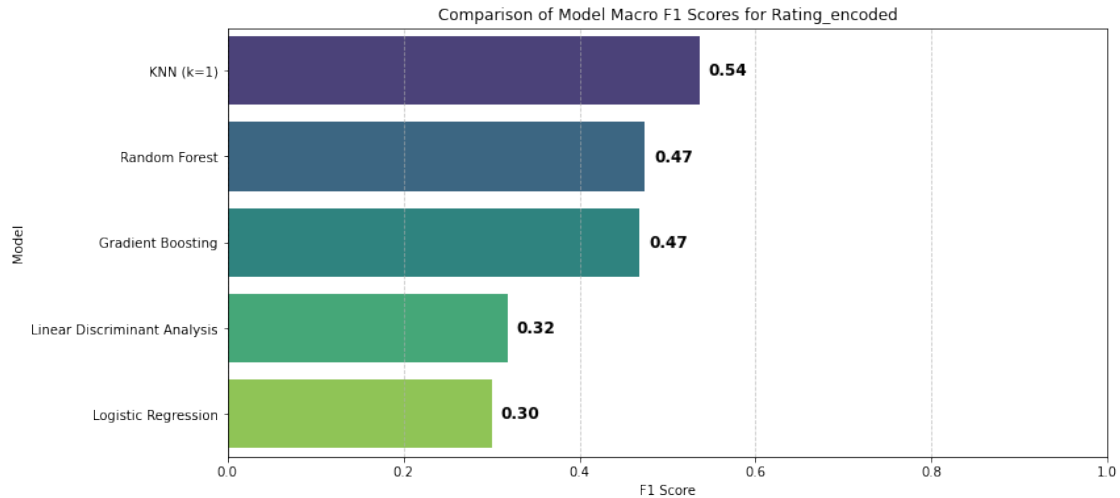
Linear Discriminant Analysis Macro F1 Score: 0.3188

Logistic Regression Macro F1 Score: 0.3006

/Users/kamila/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.

```
warnings.warn(
```

Best KNN Macro F1 Score (1 neighbors): 0.5368



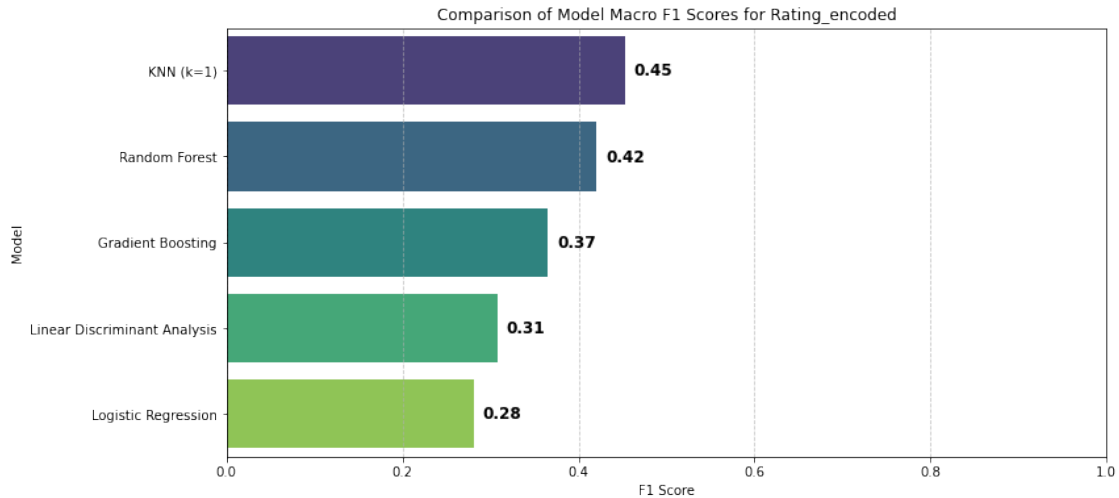
```
[736]:
```

	Model	F1 Score
4	KNN (k=1)	0.536838
1	Random Forest	0.474598
0	Gradient Boosting	0.468263
2	Linear Discriminant Analysis	0.318815
3	Logistic Regression	0.300554

```
[737]: class_models_macro_f1(data_inp=credit_rating_outl_scaled, X_cols= pred_back4,
                             y_col = 'Rating_encoded', test_size=0.2, num_class = 6)
```

```
Gradient Boosting Macro F1 Score: 0.3654
Random Forest Macro F1 Score: 0.4205
Linear Discriminant Analysis Macro F1 Score: 0.3077
Logistic Regression Macro F1 Score: 0.2808
Best KNN Macro F1 Score (1 neighbors): 0.4524
```

```
/Users/kamila/opt/anaconda3/lib/python3.9/site-
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will
always use 'multinomial'. Leave it to its default value to avoid this warning.
warnings.warn(
```



```
[737]:
```

	Model	F1 Score
4	KNN (k=1)	0.452416
1	Random Forest	0.420508
0	Gradient Boosting	0.365446
2	Linear Discriminant Analysis	0.307692
3	Logistic Regression	0.280800

```
[701]: pred_anova_risk_rating2 = X_cols.copy()
pred_anova_risk_rating2.remove('enterpriseValueMultiple')
pred_anova_risk_rating2.remove('daysOfSalesOutstanding')
```

```
[703]: len(pred_anova_risk_rating2)
```

```
[703]: 24
```

```
[704]: forward_select_gb(data_inp=credit_rating_outl_scaled,
↳X_cols=pred_anova_risk_rating2,
y_col = 'Risk_level_encoded', test_size=0.2, num_class =
↳4)
```

```
[2025-02-06 00:40:52] Features: 1/24 -- score: 0.38231589104456315
[2025-02-06 00:42:35] Features: 2/24 -- score: 0.424601288484787
[2025-02-06 00:44:42] Features: 3/24 -- score: 0.4744505308472924
[2025-02-06 00:46:46] Features: 4/24 -- score: 0.5013185236605358
[2025-02-06 00:49:05] Features: 5/24 -- score: 0.5283221377658398
[2025-02-06 00:51:45] Features: 6/24 -- score: 0.5396889073066697
[2025-02-06 00:54:50] Features: 7/24 -- score: 0.539588464813812
[2025-02-06 00:57:43] Features: 8/24 -- score: 0.5399173761844869
[2025-02-06 01:00:56] Features: 9/24 -- score: 0.5356684278301669
```

```
[2025-02-06 01:04:01] Features: 10/24 -- score: 0.5527970168843555
[2025-02-06 01:07:15] Features: 11/24 -- score: 0.5476102500387451
[2025-02-06 01:10:29] Features: 12/24 -- score: 0.5386846542590754
[2025-02-06 01:13:33] Features: 13/24 -- score: 0.5380758626622367
[2025-02-06 01:16:40] Features: 14/24 -- score: 0.5356473460710255
[2025-02-06 01:19:45] Features: 15/24 -- score: 0.5321620861022918
[2025-02-06 01:22:31] Features: 16/24 -- score: 0.5289501459808034
[2025-02-06 11:02:59] Features: 17/24 -- score: 0.5222633908372106
[2025-02-06 11:07:15] Features: 18/24 -- score: 0.5261365022778302
[2025-02-06 11:09:37] Features: 19/24 -- score: 0.5318988807007349
[2025-02-06 11:11:32] Features: 20/24 -- score: 0.5209714206312572
[2025-02-06 11:13:11] Features: 21/24 -- score: 0.5088670870217998
[2025-02-06 11:14:25] Features: 22/24 -- score: 0.5206829495364365
[2025-02-06 11:15:17] Features: 23/24 -- score: 0.5102988034655627
```

Optimal feature set: ['Sector', 'currentRatio', 'grossProfitMargin', 'returnOnCapitalEmployed', 'fixedAssetTurnover', 'debtEquityRatio', 'debtRatio', 'effectiveTaxRate', 'operatingCashFlowPerShare', 'operatingCashFlowSalesRatio']

```
[2025-02-06 11:15:44] Features: 24/24 -- score: 0.5121096220749821
```

```
[704]: ['Sector',
        'currentRatio',
        'grossProfitMargin',
        'returnOnCapitalEmployed',
        'fixedAssetTurnover',
        'debtEquityRatio',
        'debtRatio',
        'effectiveTaxRate',
        'operatingCashFlowPerShare',
        'operatingCashFlowSalesRatio']
```

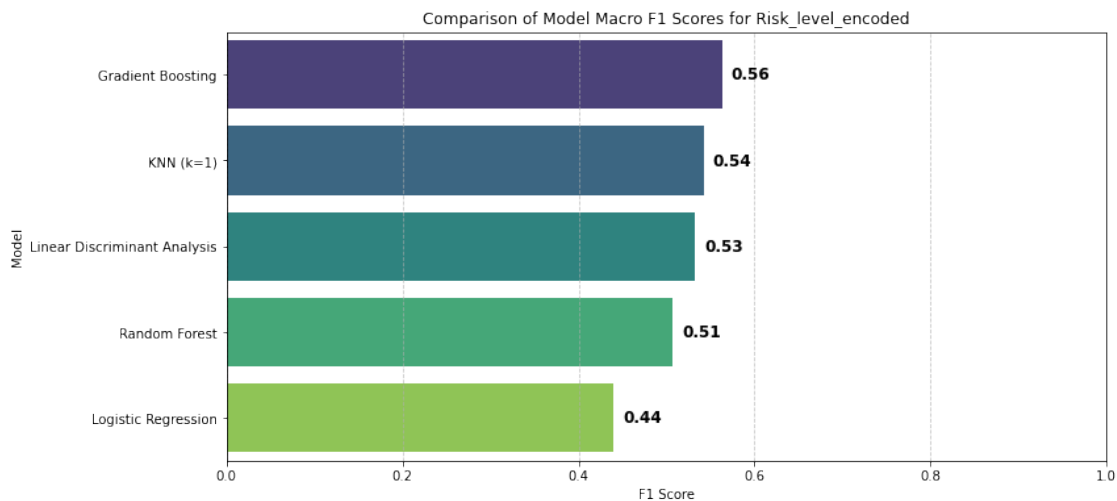
```
[709]: back5 = ['Sector',
                'currentRatio',
                'grossProfitMargin',
                'returnOnCapitalEmployed',
                'fixedAssetTurnover',
                'debtEquityRatio',
                'debtRatio',
                'effectiveTaxRate',
                'operatingCashFlowPerShare',
                'operatingCashFlowSalesRatio']

class_models_macro_f1(data_inp=credit_rating_outl_scaled, X_cols= X_cols,
                       y_col = 'Risk_level_encoded', test_size=0.2, num_class = 4)
→4)
```

Gradient Boosting Macro F1 Score: 0.5637
 Random Forest Macro F1 Score: 0.5074
 Linear Discriminant Analysis Macro F1 Score: 0.5328
 Logistic Regression Macro F1 Score: 0.4400

/Users/kamila/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.
 warnings.warn(

Best KNN Macro F1 Score (1 neighbors): 0.5421



```
[709]:
```

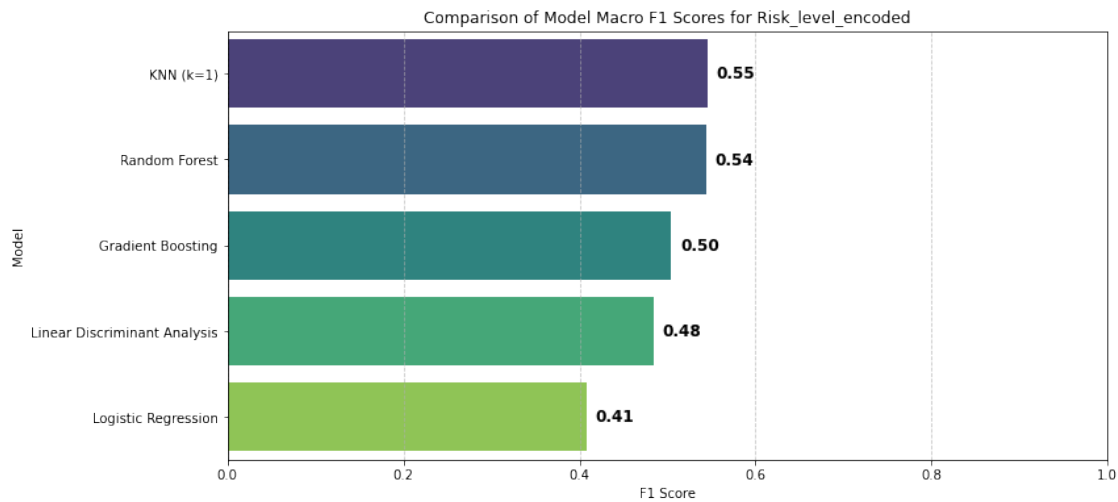
	Model	F1 Score
0	Gradient Boosting	0.563669
4	KNN (k=1)	0.542134
2	Linear Discriminant Analysis	0.532848
1	Random Forest	0.507412
3	Logistic Regression	0.439982

```
[710]: class_models_macro_f1(data_inp=credit_rating_outl_scaled, X_cols= back5,
                             y_col = 'Risk_level_encoded', test_size=0.2, num_class = 4)
↪4)
```

Gradient Boosting Macro F1 Score: 0.5043
 Random Forest Macro F1 Score: 0.5442
 Linear Discriminant Analysis Macro F1 Score: 0.4845
 Logistic Regression Macro F1 Score: 0.4080
 Best KNN Macro F1 Score (1 neighbors): 0.5452

/Users/kamila/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'

was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.
 warnings.warn(



```
[710]:
```

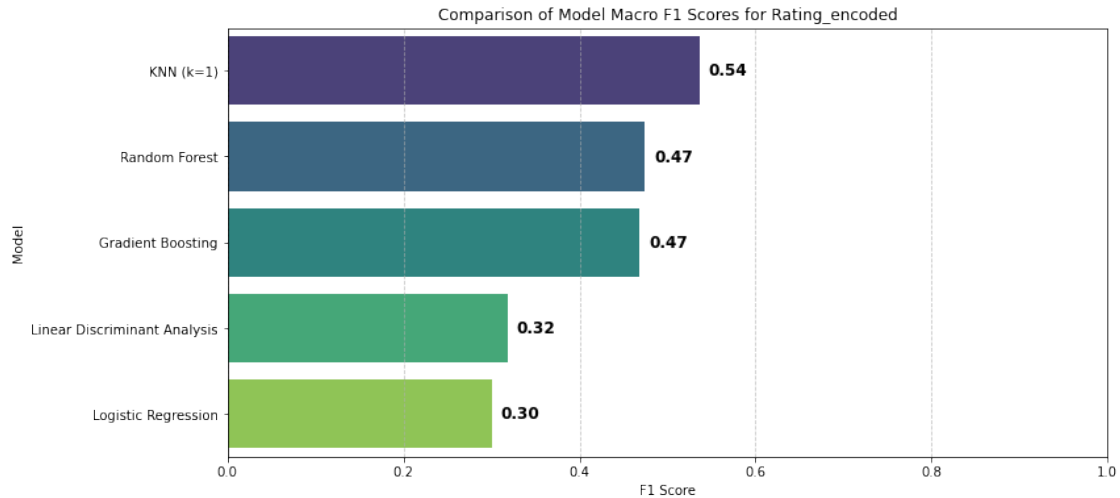
	Model	F1 Score
4	KNN (k=1)	0.545180
1	Random Forest	0.544221
0	Gradient Boosting	0.504324
2	Linear Discriminant Analysis	0.484500
3	Logistic Regression	0.408047

```
[734]: class_models_macro_f1(data_inp=credit_rating_outl_scaled, X_cols= X_cols,
                             y_col = 'Rating_encoded', test_size=0.2, num_class = 6)
```

Gradient Boosting Macro F1 Score: 0.4683
 Random Forest Macro F1 Score: 0.4746
 Linear Discriminant Analysis Macro F1 Score: 0.3188
 Logistic Regression Macro F1 Score: 0.3006

/Users/kamila/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.
 warnings.warn(

Best KNN Macro F1 Score (1 neighbors): 0.5368



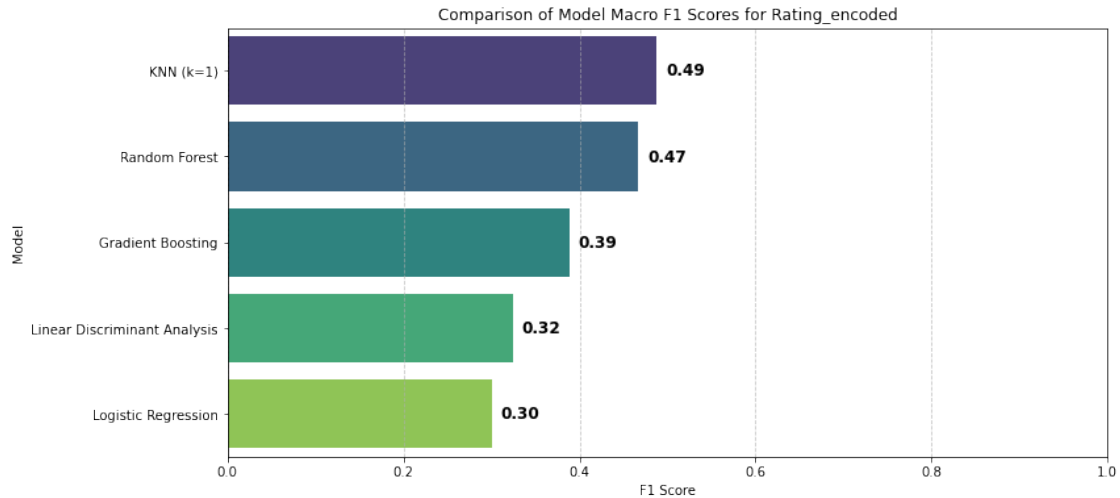
```
[734]:
```

	Model	F1 Score
4	KNN (k=1)	0.536838
1	Random Forest	0.474598
0	Gradient Boosting	0.468263
2	Linear Discriminant Analysis	0.318815
3	Logistic Regression	0.300554

```
[735]: class_models_macro_f1(data_inp=credit_rating_out1_scaled, X_cols= back5,
                             y_col = 'Rating_encoded', test_size=0.2, num_class = 6)
```

```
Gradient Boosting Macro F1 Score: 0.3890
Random Forest Macro F1 Score: 0.4670
Linear Discriminant Analysis Macro F1 Score: 0.3238
Logistic Regression Macro F1 Score: 0.3004
Best KNN Macro F1 Score (1 neighbors): 0.4880
```

```
/Users/kamila/opt/anaconda3/lib/python3.9/site-
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will
always use 'multinomial'. Leave it to its default value to avoid this warning.
warnings.warn(
```



```
[735]:
```

	Model	F1 Score
4	KNN (k=1)	0.487975
1	Random Forest	0.466958
0	Gradient Boosting	0.388968
2	Linear Discriminant Analysis	0.323788
3	Logistic Regression	0.300378

```
[733]: vif(credit_rating_outl_scaled, [
    'currentRatio',
    'grossProfitMargin',
    'returnOnCapitalEmployed',
    'fixedAssetTurnover',
    'debtEquityRatio',
    'debtRatio',
    'effectiveTaxRate',
    'operatingCashFlowPerShare',
    'operatingCashFlowSalesRatio'])
```

	feature	VIF
0	currentRatio	1.200639
1	grossProfitMargin	1.291940
2	returnOnCapitalEmployed	1.201784
3	fixedAssetTurnover	1.254658
4	debtEquityRatio	1.282646
5	debtRatio	1.397672
6	effectiveTaxRate	1.055625
7	operatingCashFlowPerShare	1.234857
8	operatingCashFlowSalesRatio	1.572614


```
[706]: backward_elimination(data_inp=credit_rating_outl_scaled,
    ↪X_cols=pred_anova_risk_rating2,
    y_col = 'Risk_level_encoded', test_size=0.2, num_class =
    ↪4)
```

```
[2025-02-06 11:36:45] Features: 23/1 -- score: 0.5172143158865978
[2025-02-06 11:46:26] Features: 22/1 -- score: 0.5259122249771842
[2025-02-06 11:55:32] Features: 21/1 -- score: 0.5193444129873781
[2025-02-06 12:03:44] Features: 20/1 -- score: 0.5337588770160611
[2025-02-06 12:11:05] Features: 19/1 -- score: 0.5256476488512699
[2025-02-06 12:17:38] Features: 18/1 -- score: 0.5283828753795337
[2025-02-06 12:23:44] Features: 17/1 -- score: 0.5183979899497014
[2025-02-06 12:29:02] Features: 16/1 -- score: 0.5150015444829956
[2025-02-06 12:34:01] Features: 15/1 -- score: 0.5245803898001183
[2025-02-06 12:38:11] Features: 14/1 -- score: 0.5229620688836334
[2025-02-06 12:41:50] Features: 13/1 -- score: 0.5301214012941082
[2025-02-06 12:44:57] Features: 12/1 -- score: 0.5247181178262184
[2025-02-06 12:47:50] Features: 11/1 -- score: 0.5232750027698141
[2025-02-06 12:50:13] Features: 10/1 -- score: 0.524703525007854
[2025-02-06 12:52:08] Features: 9/1 -- score: 0.5256323489794317
[2025-02-06 12:53:43] Features: 8/1 -- score: 0.5283014973980953
[2025-02-06 12:54:55] Features: 7/1 -- score: 0.5234241042385624
[2025-02-06 12:55:55] Features: 6/1 -- score: 0.5219248572378772
[2025-02-06 12:56:42] Features: 5/1 -- score: 0.5283221377658398
[2025-02-06 12:57:11] Features: 4/1 -- score: 0.5013185236605358
[2025-02-06 12:57:31] Features: 3/1 -- score: 0.4744505308472924
[2025-02-06 12:57:45] Features: 2/1 -- score: 0.424601288484787
```

```
Optimal feature set: ['Sector', 'currentRatio', 'quickRatio', 'cashRatio',
'netProfitMargin', 'pretaxProfitMargin', 'grossProfitMargin',
'operatingProfitMargin', 'returnOnAssets', 'returnOnCapitalEmployed',
'returnOnEquity', 'fixedAssetTurnover', 'debtEquityRatio', 'debtRatio',
'freeCashFlowPerShare', 'cashPerShare', 'companyEquityMultiplier',
'operatingCashFlowPerShare', 'operatingCashFlowSalesRatio', 'payablesTurnover']
```

```
[2025-02-06 12:57:51] Features: 1/1 -- score: 0.38231589104456315
```

```
[706]: ['Sector',
'currentRatio',
'quickRatio',
'cashRatio',
'netProfitMargin',
'pretaxProfitMargin',
'grossProfitMargin',
'operatingProfitMargin',
'returnOnAssets',
```

```

'returnOnCapitalEmployed',
'returnOnEquity',
'fixedAssetTurnover',
'debtEquityRatio',
'debtRatio',
'freeCashFlowPerShare',
'cashPerShare',
'companyEquityMultiplier',
'operatingCashFlowPerShare',
'operatingCashFlowSalesRatio',
'payablesTurnover']

```

```

[730]: back7 = ['Sector',
'currentRatio',
'quickRatio',
'cashRatio',
'netProfitMargin',
'pretaxProfitMargin',
'grossProfitMargin',
'operatingProfitMargin',
'returnOnAssets',
'returnOnCapitalEmployed',
'returnOnEquity',
'fixedAssetTurnover',
'debtEquityRatio',
'debtRatio',
'freeCashFlowPerShare',
'cashPerShare',
'companyEquityMultiplier',
'operatingCashFlowPerShare',
'operatingCashFlowSalesRatio',
'payablesTurnover']

back_num7 = [ 'currentRatio',
'quickRatio',
'cashRatio',
'netProfitMargin',
'pretaxProfitMargin',
'grossProfitMargin',
'operatingProfitMargin',
'returnOnAssets',
'returnOnCapitalEmployed',
'returnOnEquity',
'fixedAssetTurnover',
'debtEquityRatio',
'debtRatio',
'freeCashFlowPerShare',

```

```

'cashPerShare',
'companyEquityMultiplier',
'operatingCashFlowPerShare',
'operatingCashFlowSalesRatio',
'payablesTurnover']

class_models_macro_f1(data_inp=credit_rating_outl_scaled, X_cols= X_cols,
                        y_col = 'Risk_level_encoded', test_size=0.2, num_class = 4)

```

Gradient Boosting Macro F1 Score: 0.5637

Random Forest Macro F1 Score: 0.5074

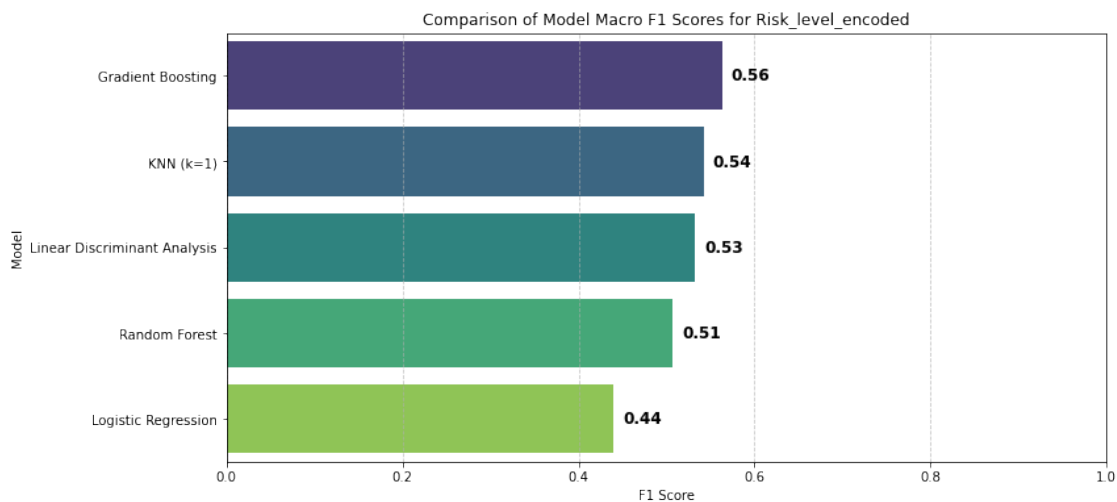
Linear Discriminant Analysis Macro F1 Score: 0.5328

Logistic Regression Macro F1 Score: 0.4400

/Users/kamila/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.

warnings.warn(

Best KNN Macro F1 Score (1 neighbors): 0.5421



[730]:

	Model	F1 Score
0	Gradient Boosting	0.563669
4	KNN (k=1)	0.542134
2	Linear Discriminant Analysis	0.532848
1	Random Forest	0.507412
3	Logistic Regression	0.439982

```
[714]: class_models_macro_f1(data_inp=credit_rating_outl_scaled, X_cols= back7 ,
                             y_col = 'Risk_level_encoded', test_size=0.2, num_class = 4)
↳4)
```

Gradient Boosting Macro F1 Score: 0.5167

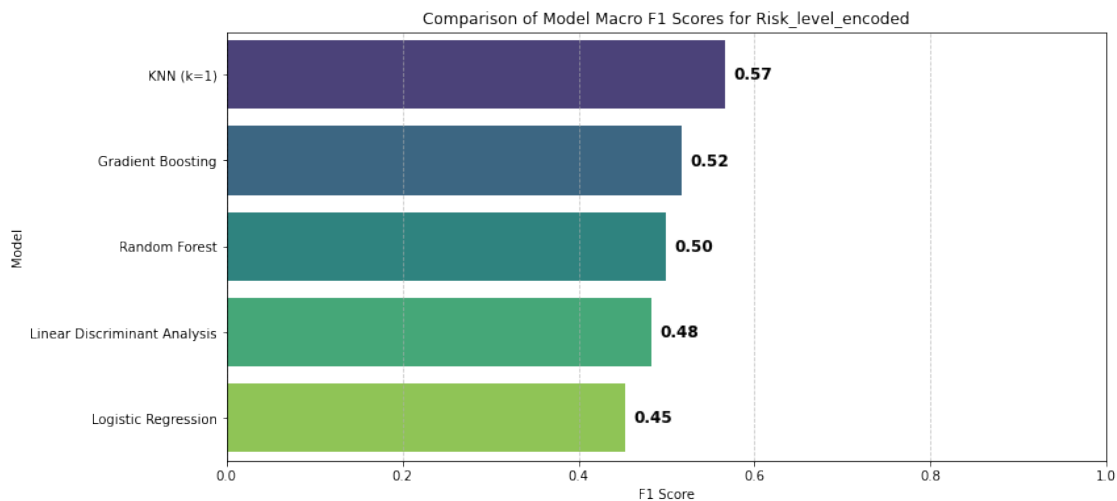
Random Forest Macro F1 Score: 0.4994

Linear Discriminant Analysis Macro F1 Score: 0.4826

Logistic Regression Macro F1 Score: 0.4530

Best KNN Macro F1 Score (1 neighbors): 0.5664

/Users/kamila/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.
warnings.warn(



```
[714]:
```

	Model	F1 Score
4	KNN (k=1)	0.566381
0	Gradient Boosting	0.516680
1	Random Forest	0.499377
2	Linear Discriminant Analysis	0.482585
3	Logistic Regression	0.453022

```
[731]: vif(credit_rating_outl_scaled, back_num7)
```

	feature	VIF
0	currentRatio	3.436166
1	quickRatio	3.193950
2	cashRatio	2.397265
3	netProfitMargin	11.785796
4	pretaxProfitMargin	10.663396

5	grossProfitMargin	1.715231
6	operatingProfitMargin	6.882993
7	returnOnAssets	11.851380
8	returnOnCapitalEmployed	8.473413
9	returnOnEquity	2.361034
10	fixedAssetTurnover	1.475451
11	debtEquityRatio	100.656987
12	debtRatio	1.511098
13	freeCashFlowPerShare	2.033463
14	cashPerShare	1.997794
15	companyEquityMultiplier	98.775376
16	operatingCashFlowPerShare	2.683638
17	operatingCashFlowSalesRatio	2.563451
18	payablesTurnover	1.637766

```
[707]: forward_select_gb(data_inp=credit_rating_outl_scaled,
↪X_cols=pred_anova_risk_rating2,
        y_col = 'Rating_encoded', test_size=0.2, num_class = 6)
```

```
[2025-02-06 12:59:45] Features: 1/24 -- score: 0.24784956044010248
[2025-02-06 13:02:20] Features: 2/24 -- score: 0.2923984495883305
[2025-02-06 13:05:27] Features: 3/24 -- score: 0.3285456131531432
[2025-02-06 13:08:41] Features: 4/24 -- score: 0.35716098312961064
[2025-02-06 13:12:13] Features: 5/24 -- score: 0.38802312875929446
[2025-02-06 13:14:38] Features: 6/24 -- score: 0.3996178592033549
[2025-02-06 13:17:13] Features: 7/24 -- score: 0.41648085774713584
[2025-02-06 13:19:50] Features: 8/24 -- score: 0.42082052558759775
[2025-02-06 13:22:37] Features: 9/24 -- score: 0.42137744016237055
[2025-02-06 13:25:26] Features: 10/24 -- score: 0.4205108275108865
[2025-02-06 13:28:21] Features: 11/24 -- score: 0.4314759963334402
[2025-02-06 13:31:12] Features: 12/24 -- score: 0.43538665505059654
[2025-02-06 13:34:05] Features: 13/24 -- score: 0.42764416026266777
[2025-02-06 13:36:53] Features: 14/24 -- score: 0.4347821026599602
[2025-02-06 13:39:35] Features: 15/24 -- score: 0.4282859281513963
[2025-02-06 13:42:14] Features: 16/24 -- score: 0.42628317774093716
[2025-02-06 13:44:42] Features: 17/24 -- score: 0.43639351990579645
[2025-02-06 13:47:00] Features: 18/24 -- score: 0.4325802531381891
[2025-02-06 13:49:00] Features: 19/24 -- score: 0.4412866893457202
[2025-02-06 13:50:46] Features: 20/24 -- score: 0.438454018532915
[2025-02-06 13:52:12] Features: 21/24 -- score: 0.433239890247391
[2025-02-06 13:53:22] Features: 22/24 -- score: 0.42544448251655476
[2025-02-06 13:54:11] Features: 23/24 -- score: 0.4240036556341013
```

```
Optimal feature set: ['Sector', 'quickRatio', 'cashRatio', 'netProfitMargin',
'grossProfitMargin', 'operatingProfitMargin', 'returnOnCapitalEmployed',
'assetTurnover', 'fixedAssetTurnover', 'debtEquityRatio', 'debtRatio',
'effectiveTaxRate', 'freeCashFlowOperatingCashFlowRatio', 'cashPerShare',
```

```
'companyEquityMultiplier', 'ebitPerRevenue', 'operatingCashFlowPerShare',  
'operatingCashFlowSalesRatio', 'payablesTurnover']
```

[2025-02-06 13:54:35] Features: 24/24 -- score: 0.4149975309121138

```
[707]: ['Sector',  
        'quickRatio',  
        'cashRatio',  
        'netProfitMargin',  
        'grossProfitMargin',  
        'operatingProfitMargin',  
        'returnOnCapitalEmployed',  
        'assetTurnover',  
        'fixedAssetTurnover',  
        'debtEquityRatio',  
        'debtRatio',  
        'effectiveTaxRate',  
        'freeCashFlowOperatingCashFlowRatio',  
        'cashPerShare',  
        'companyEquityMultiplier',  
        'ebitPerRevenue',  
        'operatingCashFlowPerShare',  
        'operatingCashFlowSalesRatio',  
        'payablesTurnover']
```

```
[728]: back8 = ['Sector',  
                'quickRatio',  
                'cashRatio',  
                'netProfitMargin',  
                'grossProfitMargin',  
                'operatingProfitMargin',  
                'returnOnCapitalEmployed',  
                'assetTurnover',  
                'fixedAssetTurnover',  
                'debtEquityRatio',  
                'debtRatio',  
                'effectiveTaxRate',  
                'freeCashFlowOperatingCashFlowRatio',  
                'cashPerShare',  
                'companyEquityMultiplier',  
                'ebitPerRevenue',  
                'operatingCashFlowPerShare',  
                'operatingCashFlowSalesRatio',  
                'payablesTurnover']  
  
back_num8 = [  
    'quickRatio',
```

```

'cashRatio',
'netProfitMargin',
'grossProfitMargin',
'operatingProfitMargin',
'returnOnCapitalEmployed',
'assetTurnover',
'fixedAssetTurnover',
'debtEquityRatio',
'debtRatio',
'effectiveTaxRate',
'freeCashFlowOperatingCashFlowRatio',
'cashPerShare',
'companyEquityMultiplier',
'ebitPerRevenue',
'operatingCashFlowPerShare',
'operatingCashFlowSalesRatio',
'payablesTurnover']

class_models_macro_f1(data_inp=credit_rating_outl_scaled, X_cols= X_cols ,
                        y_col = 'Risk_level_encoded', test_size=0.2, num_class = 4)
→4)

```

Gradient Boosting Macro F1 Score: 0.5637

Random Forest Macro F1 Score: 0.5074

Linear Discriminant Analysis Macro F1 Score: 0.5328

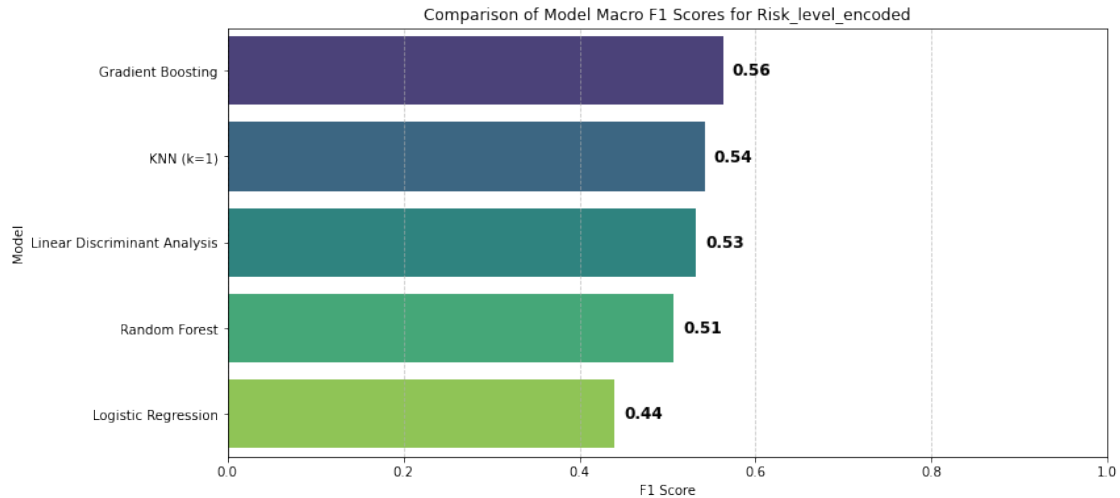
Logistic Regression Macro F1 Score: 0.4400

```

/Users/kamila/opt/anaconda3/lib/python3.9/site-
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will
always use 'multinomial'. Leave it to its default value to avoid this warning.
  warnings.warn(

```

Best KNN Macro F1 Score (1 neighbors): 0.5421



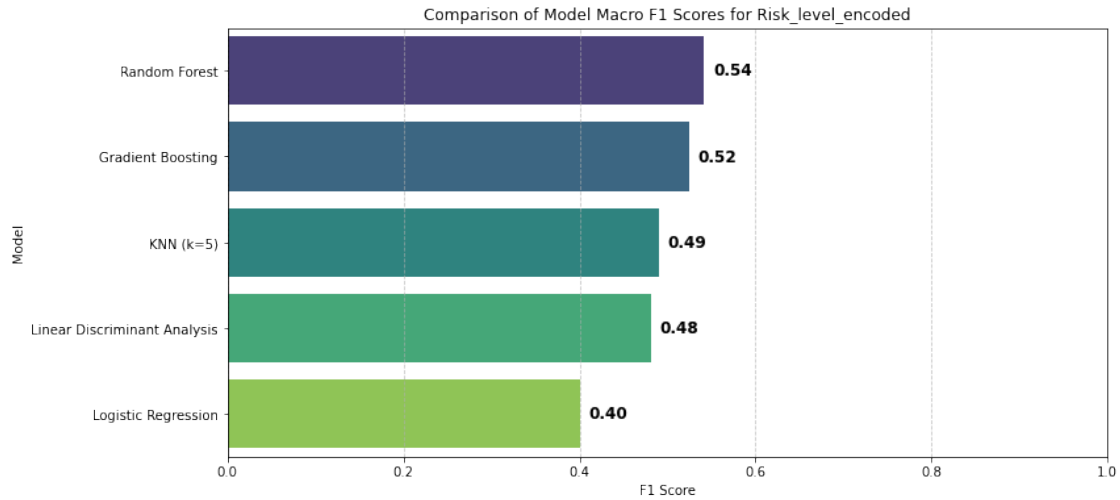
```
[728]:
```

	Model	F1 Score
0	Gradient Boosting	0.563669
4	KNN (k=1)	0.542134
2	Linear Discriminant Analysis	0.532848
1	Random Forest	0.507412
3	Logistic Regression	0.439982

```
[716]: class_models_macro_f1(data_inp=credit_rating_outl_scaled, X_cols= back8 ,
                             y_col = 'Risk_level_encoded', test_size=0.2, num_class = 4)
→4)
```

```
Gradient Boosting Macro F1 Score: 0.5240
Random Forest Macro F1 Score: 0.5417
Linear Discriminant Analysis Macro F1 Score: 0.4814
Logistic Regression Macro F1 Score: 0.4001
Best KNN Macro F1 Score (5 neighbors): 0.4902
```

```
/Users/kamila/opt/anaconda3/lib/python3.9/site-
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will
always use 'multinomial'. Leave it to its default value to avoid this warning.
warnings.warn(
```

```
[716]:
```

	Model	F1 Score
1	Random Forest	0.541718
0	Gradient Boosting	0.523998
4	KNN (k=5)	0.490188
2	Linear Discriminant Analysis	0.481360
3	Logistic Regression	0.400132

```
[729]: vif(credit_rating_outl_scaled, back_num8)
```

	feature	VIF
0	quickRatio	2.280023
1	cashRatio	2.019310
2	netProfitMargin	5.608703
3	grossProfitMargin	1.766429
4	operatingProfitMargin	7.356146
5	returnOnCapitalEmployed	3.677977
6	assetTurnover	2.906128
7	fixedAssetTurnover	1.577340
8	debtEquityRatio	100.614012
9	debtRatio	1.476470
10	effectiveTaxRate	1.104306
11	freeCashFlowOperatingCashFlowRatio	1.246309
12	cashPerShare	1.967759
13	companyEquityMultiplier	98.673439
14	ebitPerRevenue	9.439737
15	operatingCashFlowPerShare	1.990730
16	operatingCashFlowSalesRatio	2.492840
17	payablesTurnover	1.656140

```
[ ]:
```

```
[708]: backward_elimination(data_inp=credit_rating_outl_scaled,
    ↪X_cols=pred_anova_risk_rating2,
    y_col = 'Rating_encoded', test_size=0.2, num_class = 6)
```

```
[2025-02-06 14:04:40] Features: 23/1 -- score: 0.4286328820602834
[2025-02-06 14:13:35] Features: 22/1 -- score: 0.42856616231060285
[2025-02-06 14:21:42] Features: 21/1 -- score: 0.42689579593287774
[2025-02-06 14:29:06] Features: 20/1 -- score: 0.43211112607679975
[2025-02-06 14:35:53] Features: 19/1 -- score: 0.42571326127336456
[2025-02-06 14:41:56] Features: 18/1 -- score: 0.42696325478011393
[2025-02-06 14:47:25] Features: 17/1 -- score: 0.42455973233517275
[2025-02-06 14:52:19] Features: 16/1 -- score: 0.42648956371244784
[2025-02-06 14:56:40] Features: 15/1 -- score: 0.43025988607952187
[2025-02-06 15:00:28] Features: 14/1 -- score: 0.42875296845836103
[2025-02-06 15:04:37] Features: 13/1 -- score: 0.4268085675594303
[2025-02-06 15:09:22] Features: 12/1 -- score: 0.42332718816065096
[2025-02-06 15:13:26] Features: 11/1 -- score: 0.4157541703399888
[2025-02-06 15:16:52] Features: 10/1 -- score: 0.41520290113440933
[2025-02-06 15:19:40] Features: 9/1 -- score: 0.4077087550358872
[2025-02-06 15:22:01] Features: 8/1 -- score: 0.42436868093253965
[2025-02-06 15:23:51] Features: 7/1 -- score: 0.4205161625966777
[2025-02-06 15:25:13] Features: 6/1 -- score: 0.4061013549709081
[2025-02-06 15:26:19] Features: 5/1 -- score: 0.3773929884925731
[2025-02-06 15:27:03] Features: 4/1 -- score: 0.3688307456568834
[2025-02-06 15:27:39] Features: 3/1 -- score: 0.3294457931863281
[2025-02-06 15:27:58] Features: 2/1 -- score: 0.26822871028026474
```

```
Optimal feature set: ['Sector', 'quickRatio', 'cashRatio', 'netProfitMargin',
'grossProfitMargin', 'operatingProfitMargin', 'returnOnAssets',
'returnOnCapitalEmployed', 'returnOnEquity', 'assetTurnover',
'fixedAssetTurnover', 'debtEquityRatio', 'effectiveTaxRate',
'freeCashFlowOperatingCashFlowRatio', 'cashPerShare', 'companyEquityMultiplier',
'ebitPerRevenue', 'operatingCashFlowPerShare', 'operatingCashFlowSalesRatio',
'payablesTurnover']
```

```
[2025-02-06 15:28:06] Features: 1/1 -- score: 0.1960866050880436
```

```
[708]: ['Sector',
'quickRatio',
'cashRatio',
'netProfitMargin',
'grossProfitMargin',
'operatingProfitMargin',
'returnOnAssets',
'returnOnCapitalEmployed',
'returnOnEquity',
```

```

'assetTurnover',
'fixedAssetTurnover',
'debtEquityRatio',
'effectiveTaxRate',
'freeCashFlowOperatingCashFlowRatio',
'cashPerShare',
'companyEquityMultiplier',
'ebitPerRevenue',
'operatingCashFlowPerShare',
'operatingCashFlowSalesRatio',
'payablesTurnover']

```

```

[718]: back9= ['Sector',
'quickRatio',
'cashRatio',
'netProfitMargin',
'grossProfitMargin',
'operatingProfitMargin',
'returnOnAssets',
'returnOnCapitalEmployed',
'returnOnEquity',
'assetTurnover',
'fixedAssetTurnover',
'debtEquityRatio',
'effectiveTaxRate',
'freeCashFlowOperatingCashFlowRatio',
'cashPerShare',
'companyEquityMultiplier',
'ebitPerRevenue',
'operatingCashFlowPerShare',
'operatingCashFlowSalesRatio',
'payablesTurnover']

```

```

[719]: class_models_macro_f1(data_inp=credit_rating_outl_scaled, X_cols= X_cols ,
y_col = 'Risk_level_encoded', test_size=0.2, num_class = 4)

```

Gradient Boosting Macro F1 Score: 0.5637

Random Forest Macro F1 Score: 0.5074

Linear Discriminant Analysis Macro F1 Score: 0.5328

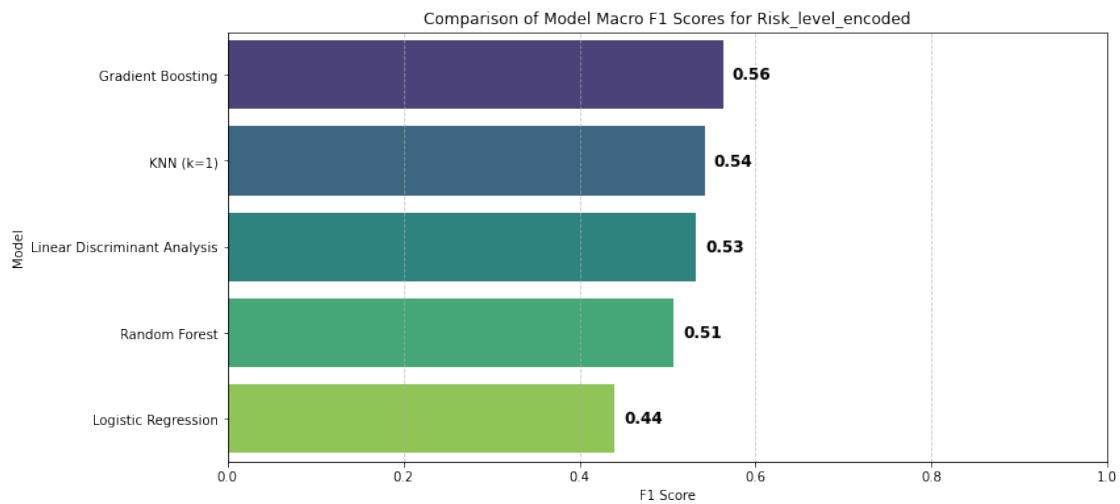
Logistic Regression Macro F1 Score: 0.4400

```

/Users/kamila/opt/anaconda3/lib/python3.9/site-
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will
always use 'multinomial'. Leave it to its default value to avoid this warning.
warnings.warn(

```

Best KNN Macro F1 Score (1 neighbors): 0.5421



```
[719]:
```

	Model	F1 Score
0	Gradient Boosting	0.563669
4	KNN (k=1)	0.542134
2	Linear Discriminant Analysis	0.532848
1	Random Forest	0.507412
3	Logistic Regression	0.439982

```
[720]: class_models_macro_f1(data_inp=credit_rating_out1_scaled, X_cols= back9 ,
                             y_col = 'Risk_level_encoded', test_size=0.2, num_class = 4)
↪4)
```

Gradient Boosting Macro F1 Score: 0.5087

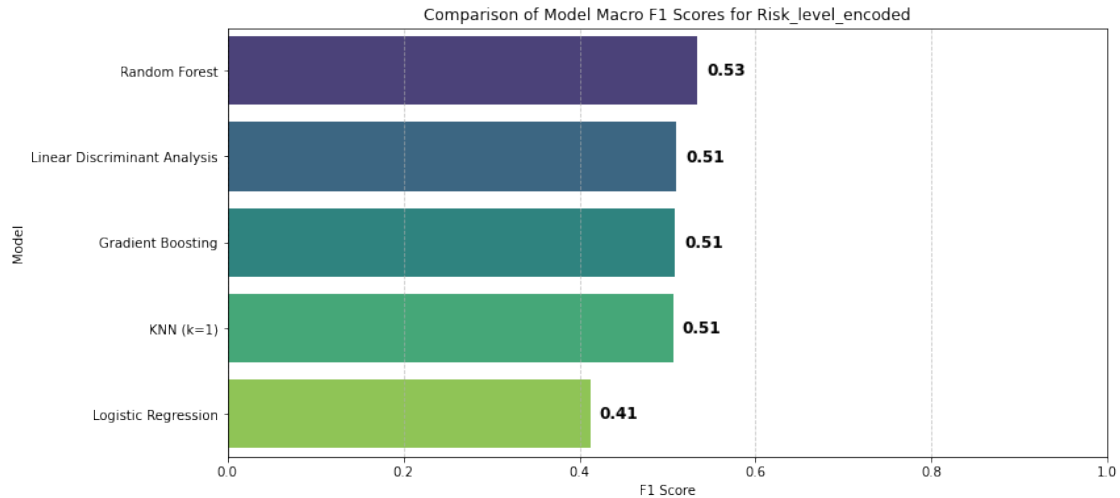
Random Forest Macro F1 Score: 0.5342

Linear Discriminant Analysis Macro F1 Score: 0.5102

Logistic Regression Macro F1 Score: 0.4123

```
/Users/kamila/opt/anaconda3/lib/python3.9/site-
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will
always use 'multinomial'. Leave it to its default value to avoid this warning.
  warnings.warn(
```

Best KNN Macro F1 Score (1 neighbors): 0.5062



```
[720]:
```

	Model	F1 Score
1	Random Forest	0.534244
2	Linear Discriminant Analysis	0.510154
0	Gradient Boosting	0.508691
4	KNN (k=1)	0.506185
3	Logistic Regression	0.412278

7 Best predictors

```
pred_back2 = ['Sector', 'currentRatio', 'grossProfitMargin', 'returnOnCapitalEmployed', 'asset-
Turnover', 'fixedAssetTurnover', 'debtEquityRatio', 'debtRatio', 'enterpriseValueMultiple', 'oper-
atingCashFlowPerShare', 'operatingCashFlowSalesRatio']
```

```
[396]: best_pred = ['Sector',
'currentRatio',
'grossProfitMargin',
'returnOnCapitalEmployed',
'assetTurnover',
'fixedAssetTurnover',
'debtEquityRatio',
'debtRatio',
'enterpriseValueMultiple',
'operatingCashFlowPerShare',
'operatingCashFlowSalesRatio']

best_pred_num = [
'currentRatio',
'grossProfitMargin',
'returnOnCapitalEmployed',
```

```
'assetTurnover',
'fixedAssetTurnover',
'debtEquityRatio',
'debtRatio',
'enterpriseValueMultiple',
'operatingCashFlowPerShare',
'operatingCashFlowSalesRatio']
```

```
[722]: class_models_macro_f1(data_inp=credit_rating_outl_scaled, X_cols= X_cols ,
                             y_col = 'Risk_level_encoded', test_size=0.2, num_class = 4)
↪4)
```

Gradient Boosting Macro F1 Score: 0.5637

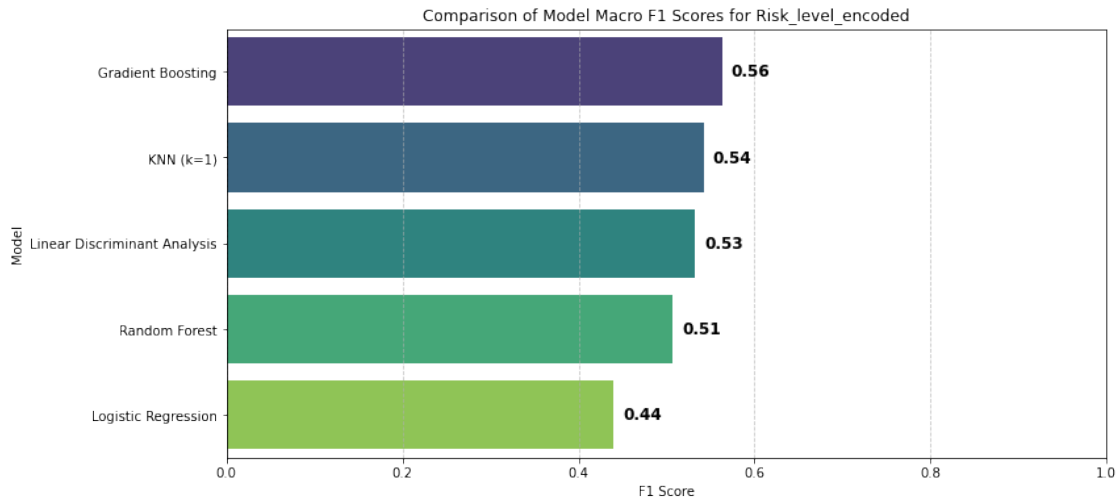
Random Forest Macro F1 Score: 0.5074

Linear Discriminant Analysis Macro F1 Score: 0.5328

Logistic Regression Macro F1 Score: 0.4400

/Users/kamila/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.
warnings.warn(

Best KNN Macro F1 Score (1 neighbors): 0.5421



```
[722]:
Model  F1 Score
0      Gradient Boosting 0.563669
4      KNN (k=1)         0.542134
2      Linear Discriminant Analysis 0.532848
1      Random Forest     0.507412
3      Logistic Regression 0.439982
```

```
[723]: class_models_macro_f1(data_inp=credit_rating_outl_scaled, X_cols= best_pred,
                                y_col = 'Risk_level_encoded', test_size=0.2, num_class =4)
    ↪4)
```

Gradient Boosting Macro F1 Score: 0.5133

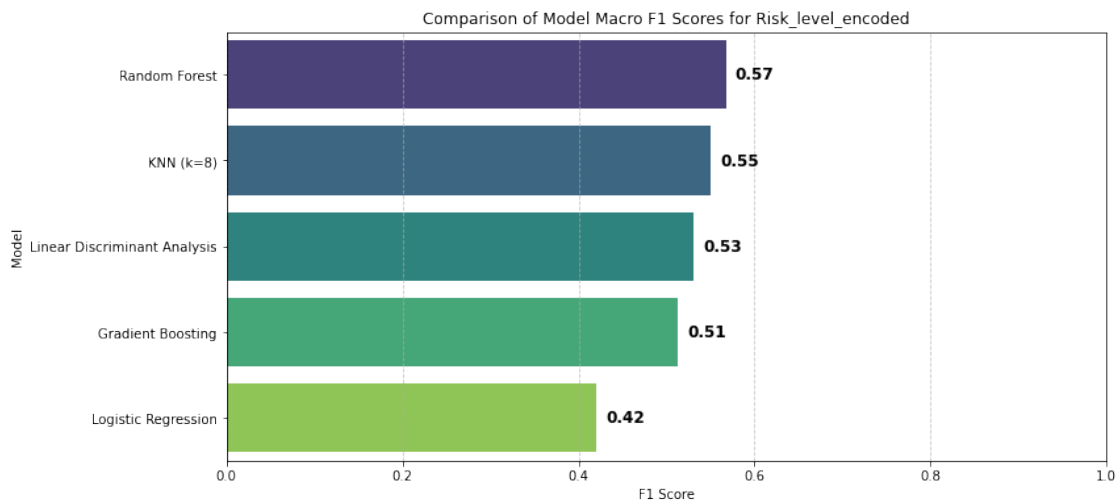
Random Forest Macro F1 Score: 0.5680

Linear Discriminant Analysis Macro F1 Score: 0.5311

Logistic Regression Macro F1 Score: 0.4208

Best KNN Macro F1 Score (8 neighbors): 0.5505

/Users/kamila/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.
warnings.warn(



```
[723]:
Model    F1 Score
1      Random Forest  0.568035
4      KNN (k=8)      0.550536
2  Linear Discriminant Analysis  0.531054
0      Gradient Boosting  0.513295
3      Logistic Regression  0.420819
```

```
[424]: data1 = class_models_macro_f1(data_inp=credit_rating_outl_scaled, X_cols =
    ↪X_cols,
                                y_col = 'Risk_level_encoded', test_size=0.2, num_class =4)
    ↪4)

data2 = class_models_macro_f1(data_inp=credit_rating_outl_scaled, X_cols=
    ↪best_pred,
```

```

y_col = 'Risk_level_encoded', test_size=0.2, num_class = 4)

compare_bar_charts(data1, data2, title = 'Comparison of Macro Avg F1 Scores',
y_col = 'Risk level',
label1 = 'all predictors', label2 = '11 predictors (BE)',
xlim=0.67,
color1='navy', color2='darkmagenta')

```

Gradient Boosting Macro F1 Score: 0.5637

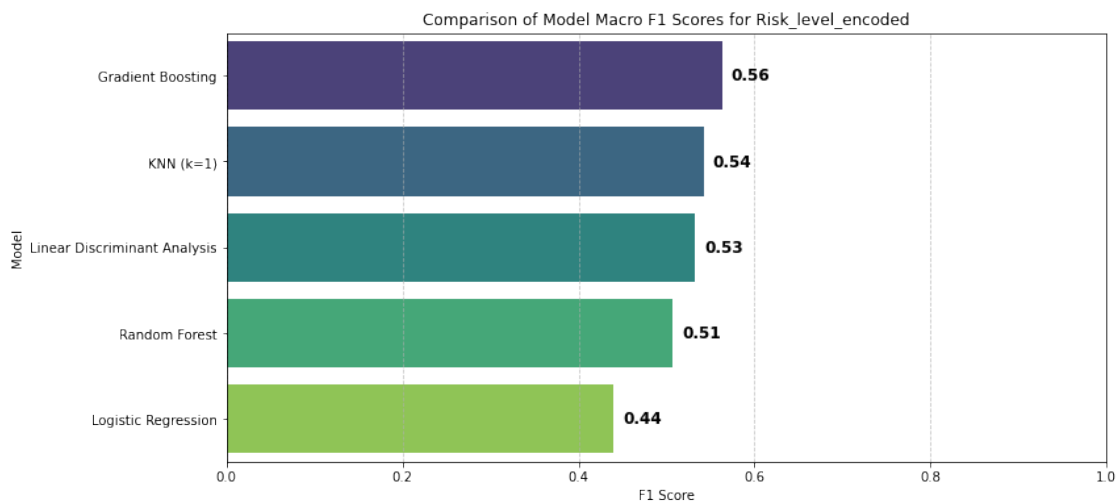
Random Forest Macro F1 Score: 0.5074

Linear Discriminant Analysis Macro F1 Score: 0.5328

Logistic Regression Macro F1 Score: 0.4400

Best KNN Macro F1 Score (1 neighbors): 0.5421

/Users/kamila/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.
warnings.warn(



Gradient Boosting Macro F1 Score: 0.5133

Random Forest Macro F1 Score: 0.5680

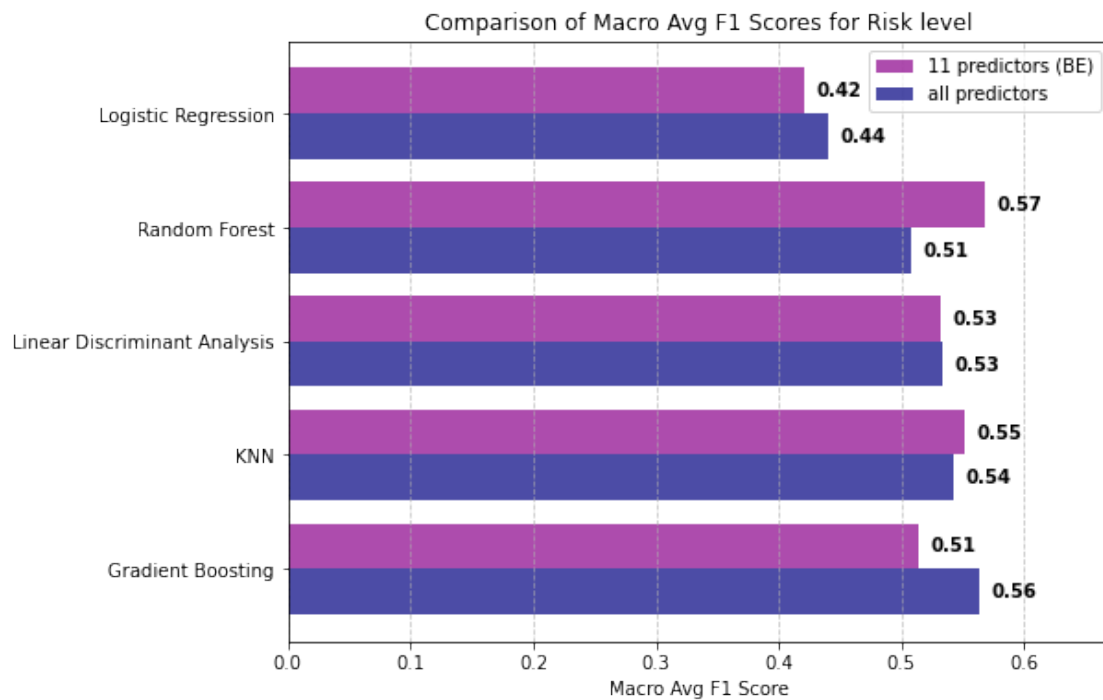
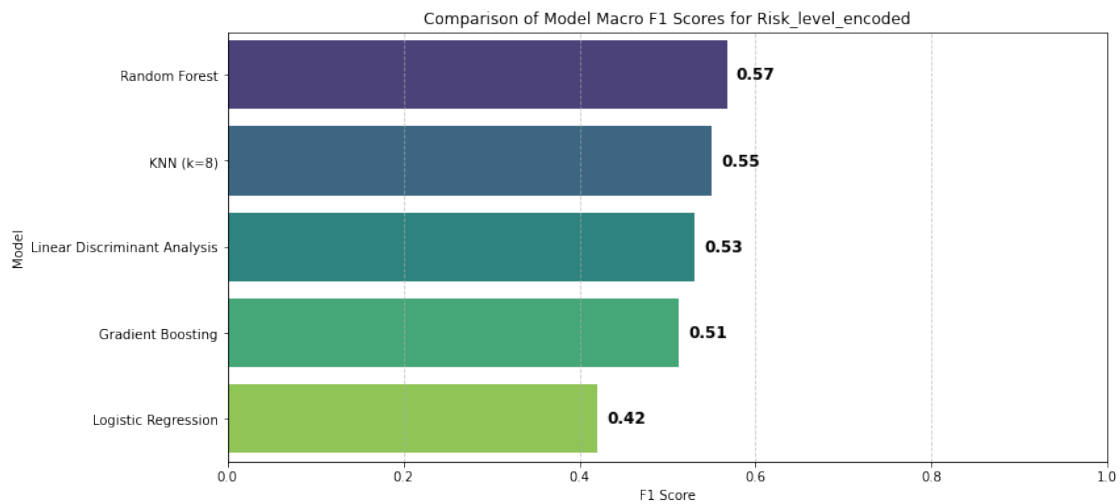
Linear Discriminant Analysis Macro F1 Score: 0.5311

Logistic Regression Macro F1 Score: 0.4208

Best KNN Macro F1 Score (8 neighbors): 0.5505

/Users/kamila/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.


```
warnings.warn(
```



```
[724]: class_models_macro_f1(data_inp=credit_rating_out1_scaled, X_cols= X_cols ,
                             y_col = 'Rating_encoded', test_size=0.2, num_class = 6)
```

Gradient Boosting Macro F1 Score: 0.4683

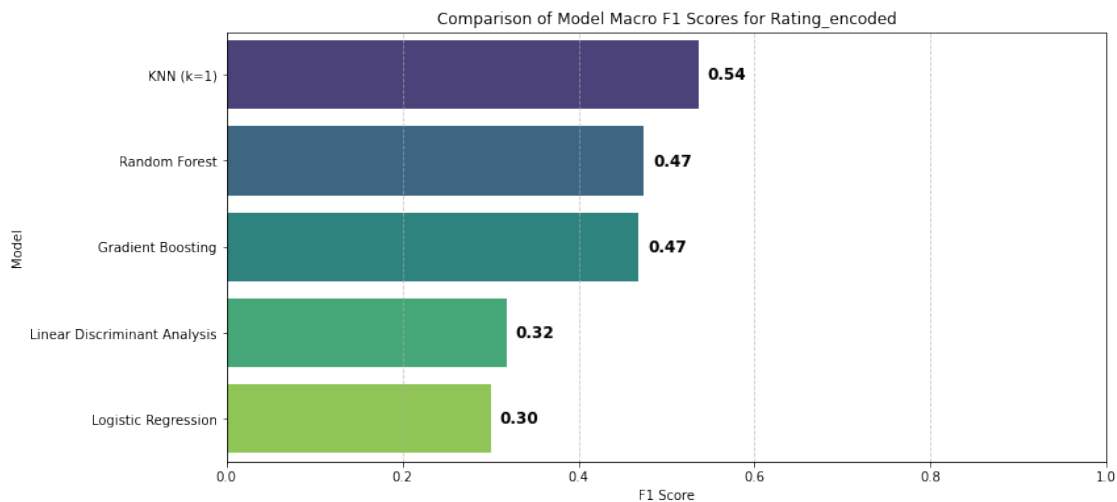
Random Forest Macro F1 Score: 0.4746

Linear Discriminant Analysis Macro F1 Score: 0.3188

Logistic Regression Macro F1 Score: 0.3006

```
/Users/kamila/opt/anaconda3/lib/python3.9/site-  
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'  
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will  
always use 'multinomial'. Leave it to its default value to avoid this warning.  
warnings.warn(
```

Best KNN Macro F1 Score (1 neighbors): 0.5368



```
[724]:
```

	Model	F1 Score
4	KNN (k=1)	0.536838
1	Random Forest	0.474598
0	Gradient Boosting	0.468263
2	Linear Discriminant Analysis	0.318815
3	Logistic Regression	0.300554

```
[725]: class_models_macro_f1(data_inp=credit_rating_outl_scaled, X_cols= best_pred ,  
                             y_col = 'Rating_encoded', test_size=0.2, num_class = 6)
```

Gradient Boosting Macro F1 Score: 0.4162

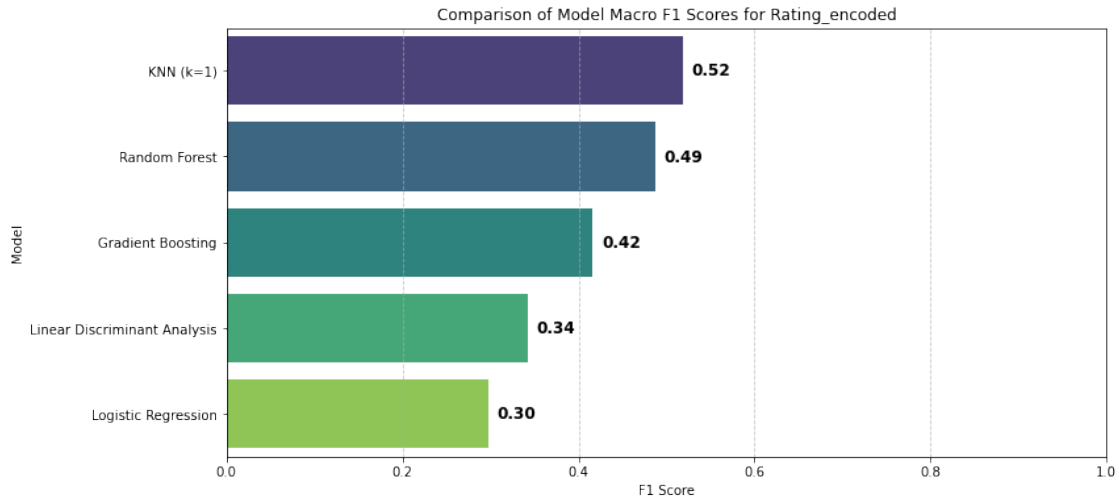
Random Forest Macro F1 Score: 0.4868

Linear Discriminant Analysis Macro F1 Score: 0.3424

Logistic Regression Macro F1 Score: 0.2978

Best KNN Macro F1 Score (1 neighbors): 0.5181

```
/Users/kamila/opt/anaconda3/lib/python3.9/site-  
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'  
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will  
always use 'multinomial'. Leave it to its default value to avoid this warning.  
warnings.warn(
```



```
[725]:
          Model  F1 Score
4          KNN (k=1)  0.518143
1      Random Forest  0.486822
0      Gradient Boosting  0.416186
2  Linear Discriminant Analysis  0.342437
3      Logistic Regression  0.297799
```

```
[428]: data1 = class_models_macro_f1(data_inp=credit_rating_out1_scaled, X_cols =
      ↪X_cols,
          y_col = 'Rating_encoded', test_size=0.2, num_class = 6)

data2 = class_models_macro_f1(data_inp=credit_rating_out1_scaled, X_cols=
      ↪best_pred,
          y_col = 'Rating_encoded', test_size=0.2, num_class = 6)

compare_bar_charts(data1, data2, title = 'Comparison of Macro Avg F1 Scores',
      ↪y_col = 'Rating',
          label1 = 'all predictors', label2 = '11 predictors (BE)',
      ↪xlim=0.58,
          color1='navy', color2='darkmagenta')
```

Gradient Boosting Macro F1 Score: 0.4683

Random Forest Macro F1 Score: 0.4746

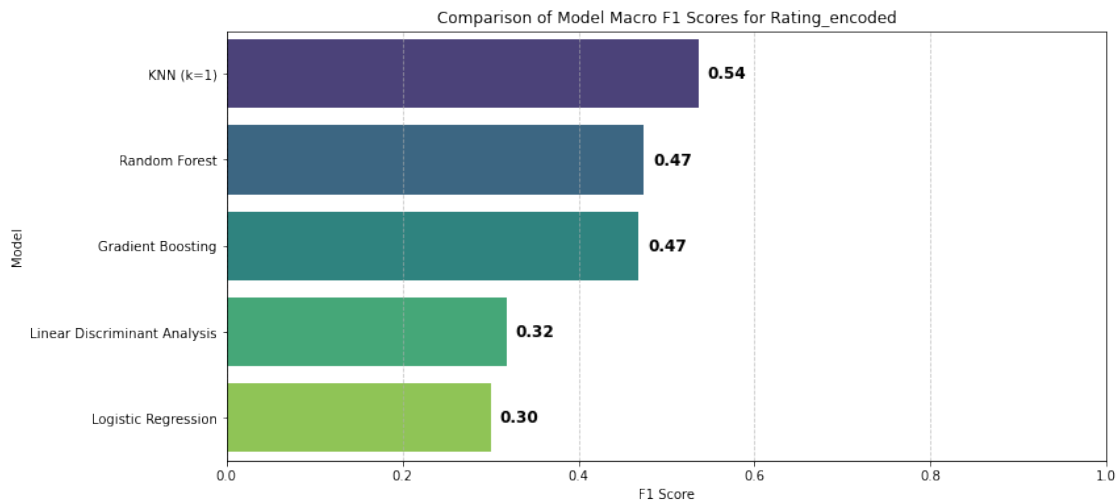
Linear Discriminant Analysis Macro F1 Score: 0.3188

Logistic Regression Macro F1 Score: 0.3006

/Users/kamila/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.

```
warnings.warn(
```

Best KNN Macro F1 Score (1 neighbors): 0.5368



Gradient Boosting Macro F1 Score: 0.4162

Random Forest Macro F1 Score: 0.4868

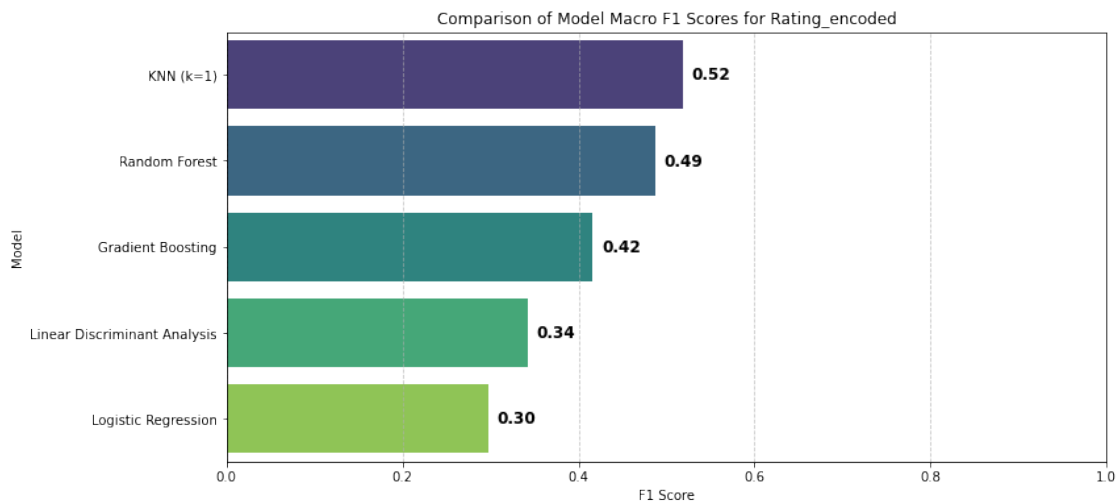
Linear Discriminant Analysis Macro F1 Score: 0.3424

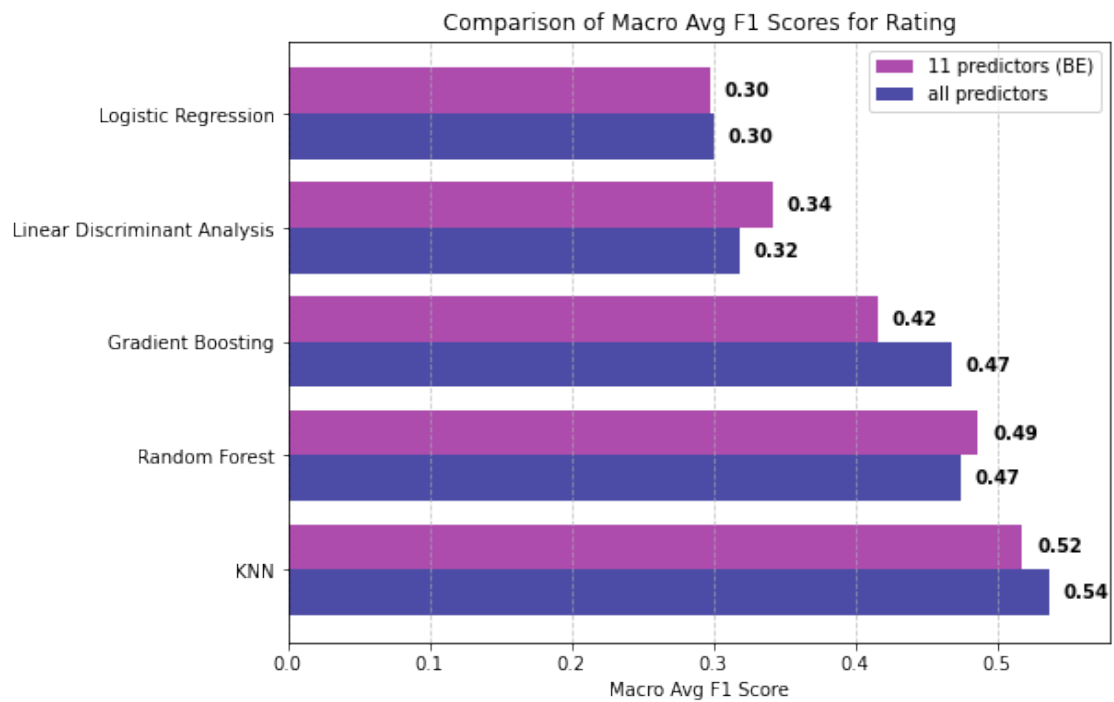
Logistic Regression Macro F1 Score: 0.2978

Best KNN Macro F1 Score (1 neighbors): 0.5181

/Users/kamila/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.

```
warnings.warn(
```





```
[421]: vif(credit_rating_outl_scaled, best_pred_num,
        '/Users/kamila/Desktop/tables pngs/vif_best_pred.png',font = 11)
```

```
[421]:
```

	feature	VIF
0	enterpriseValueMultiple	1.09
1	currentRatio	1.22
2	debtEquityRatio	1.30
3	operatingCashFlowPerShare	1.33
4	fixedAssetTurnover	1.35
5	returnOnCapitalEmployed	1.36
6	debtRatio	1.40
7	grossProfitMargin	1.43
8	operatingCashFlowSalesRatio	2.07
9	assetTurnover	2.33

feature	VIF
enterpriseValueMultiple	1.09
currentRatio	1.22
debtEquityRatio	1.3
operatingCashFlowPerShare	1.33
fixedAssetTurnover	1.35
returnOnCapitalEmployed	1.36
debtRatio	1.4
grossProfitMargin	1.43
operatingCashFlowSalesRatio	2.07
assetTurnover	2.33

```
[742]: cor(credit_rating_outl_scaled, best_pred_num)
```

```

currentRatio grossProfitMargin \
currentRatio      1.000000      -0.219900
grossProfitMargin -0.219900      1.000000
returnOnCapitalEmployed -0.033028      -0.053749
assetTurnover      0.021907      -0.451027
fixedAssetTurnover  0.062071      -0.142053
debtEquityRatio     -0.214413      0.003641
debtRatio           -0.282656      0.048218
enterpriseValueMultiple  0.003440      0.074222
operatingCashFlowPerShare -0.166358      0.128265
operatingCashFlowSalesRatio -0.147103      0.432852

returnOnCapitalEmployed assetTurnover \
currentRatio      -0.033028      0.021907
grossProfitMargin -0.053749      -0.451027
returnOnCapitalEmployed  1.000000      0.359703
assetTurnover      0.359703      1.000000
fixedAssetTurnover  0.235372      0.442086
debtEquityRatio     -0.058467      0.002146
debtRatio           -0.059022      0.105760
enterpriseValueMultiple  0.018585      -0.109994
operatingCashFlowPerShare  0.260795      0.013965
operatingCashFlowSalesRatio  0.018203      -0.582522

fixedAssetTurnover debtEquityRatio debtRatio \

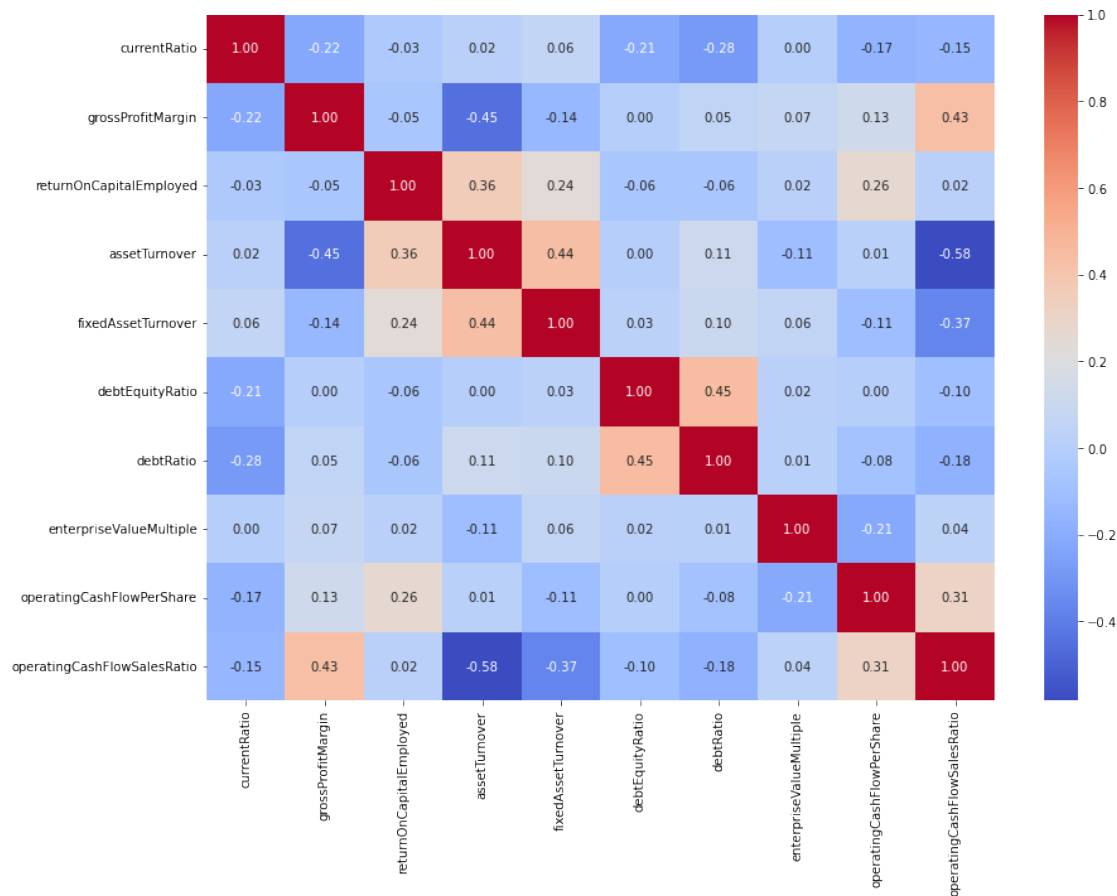
```

currentRatio	0.062071	-0.214413	-0.282656
grossProfitMargin	-0.142053	0.003641	0.048218
returnOnCapitalEmployed	0.235372	-0.058467	-0.059022
assetTurnover	0.442086	0.002146	0.105760
fixedAssetTurnover	1.000000	0.027987	0.101206
debtEquityRatio	0.027987	1.000000	0.453393
debtRatio	0.101206	0.453393	1.000000
enterpriseValueMultiple	0.055767	0.019386	0.014406
operatingCashFlowPerShare	-0.111880	0.002887	-0.082871
operatingCashFlowSalesRatio	-0.368431	-0.103118	-0.179817

	enterpriseValueMultiple \
currentRatio	0.003440
grossProfitMargin	0.074222
returnOnCapitalEmployed	0.018585
assetTurnover	-0.109994
fixedAssetTurnover	0.055767
debtEquityRatio	0.019386
debtRatio	0.014406
enterpriseValueMultiple	1.000000
operatingCashFlowPerShare	-0.214936
operatingCashFlowSalesRatio	0.035597

	operatingCashFlowPerShare \
currentRatio	-0.166358
grossProfitMargin	0.128265
returnOnCapitalEmployed	0.260795
assetTurnover	0.013965
fixedAssetTurnover	-0.111880
debtEquityRatio	0.002887
debtRatio	-0.082871
enterpriseValueMultiple	-0.214936
operatingCashFlowPerShare	1.000000
operatingCashFlowSalesRatio	0.309117

	operatingCashFlowSalesRatio
currentRatio	-0.147103
grossProfitMargin	0.432852
returnOnCapitalEmployed	0.018203
assetTurnover	-0.582522
fixedAssetTurnover	-0.368431
debtEquityRatio	-0.103118
debtRatio	-0.179817
enterpriseValueMultiple	0.035597
operatingCashFlowPerShare	0.309117
operatingCashFlowSalesRatio	1.000000



```
[ ]:
```

```
[ ]:
```

```
[ ]:
```

```
[ ]:
```

```
[ ]:
```

7.1 Parameter tuning ML models

```
[429]: from sklearn.model_selection import GridSearchCV
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import GradientBoostingClassifier, RandomForestClassifier
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import f1_score, make_scorer
```



```

from sklearn.utils.class_weight import compute_sample_weight
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split

def class_models_macro_f1_param_tunning(data_inp, X_cols, y_col, test_size):

    data = data_inp.copy()

    # Encode sector
    le = preprocessing.LabelEncoder()
    le.fit(data.Sector)
    data.Sector = le.transform(data.Sector)

    # Split dataset into training and testing while maintaining category
    ↪ proportions
    X = data[X_cols] # Features
    y = data[y_col] # Target

    X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                         test_size=test_size, stratify=y, random_state=123)

    #print('y_test classes: ', np.unique(y_test))
    #print('y_train classes: ', np.unique(y_train))

    # Compute sample weights
    sample_weights = compute_sample_weight(class_weight="balanced", y=y_train)

    # Define models and parameter grids
    param_grids = {
        "Gradient Boosting": {
            "model": GradientBoostingClassifier(random_state=123),
            "params": {"n_estimators": [100, 200, 300, 400, 500],
                       "learning_rate": [0.01, 0.05, 0.1, 0.2, 0.3],
            ↪ "max_depth": [3, 4, 5, 6, 7, 8, 9]}
        },
        "Random Forest": {
            "model": RandomForestClassifier(class_weight='balanced',
            ↪ random_state=123),
            "params": {"n_estimators": [100, 200, 300], "max_depth": [None, 10,
            ↪ 20], "min_samples_split": [2, 5, 10]}
        },
        "Linear Discriminant Analysis": {
            "model": LinearDiscriminantAnalysis(),
            "params": {"solver": ["svd", "lsqr"]}
        }
    }

```

```

    },
    "Logistic Regression": {
        "model": LogisticRegression(class_weight='balanced',
        ↪random_state=123, multi_class='multinomial', solver='newton-cg'),
        "params": {"C": [0.1, 1, 10]}
    },
    "KNN": {
        "model": KNeighborsClassifier(weights='distance'),
        "params": {"n_neighbors": list(range(1, 20))}
    }
}

f1_scorer = make_scorer(f1_score, average='macro', pos_label=None)
best_params_list = []

# Perform GridSearchCV for each model
for name, config in param_grids.items():
    grid_search = GridSearchCV(config["model"], config["params"],
    ↪scoring=f1_scorer, cv=5, n_jobs=-1, verbose=1)

    if name == "Gradient Boosting":
        #print("Unique classes in y_train:", np.unique(y_train))
        #print("Unique classes in y_test:", np.unique(y_test))
        grid_search.fit(X_train, y_train, sample_weight=sample_weights)
    else:
        #print("Unique classes in y_train:", np.unique(y_train))
        #print("Unique classes in y_test:", np.unique(y_test))
        grid_search.fit(X_train, y_train)

    best_params_list.append({"Model": name, "Best Parameters": grid_search.
    ↪best_params_, "Best F1 Score": grid_search.best_score_})

df_best_params = pd.DataFrame(best_params_list)
df_best_params = df_best_params.sort_values(by='Best F1 Score',
    ↪ascending=False)

# Plot the F1 scores
plt.figure(figsize=(12, 6))
ax = sns.barplot(x='Best F1 Score', y='Model', data=df_best_params,
    ↪palette='viridis')

for index, value in enumerate(df_best_params["Best F1 Score"]):
    ax.text(value + 0.01, index, f"{value:.2f}", va='center', fontsize=12,
    ↪fontweight='bold', color='black')

plt.xlabel("F1 Score")

```

```

plt.ylabel("Model")
plt.title("Comparison of Best Model Macro F1 Scores for " + y_col)
plt.xlim(0, 1)
plt.grid(axis='x', linestyle='--', alpha=0.7)
plt.show()

return df_best_params


def class_models_macro_f1_params(data_inp, X_cols, y_col, test_size,
    ↪model_params):
    data = data_inp.copy()

    # Encode sector
    le = preprocessing.LabelEncoder()
    le.fit(data.Sector)
    data.Sector = le.transform(data.Sector)

    # Split dataset into training and testing while maintaining category
    ↪proportions
    X = data[X_cols] # Features
    y = data[y_col] # Target

    X_train, X_test, y_train, y_test = train_test_split(X, y,
        test_size=test_size, stratify=y, random_state=123)

    # Compute sample weights
    sample_weights = compute_sample_weight(class_weight="balanced", y=y_train)

    f1_scores = {}

    # Train and evaluate each model using provided parameters
    for name, params in model_params.items():
        model = params["model"](**params["params"])

        if name == "Gradient Boosting":
            model.fit(X_train, y_train, sample_weight=sample_weights)
        else:
            model.fit(X_train, y_train)

        y_pred = model.predict(X_test)
        f1 = f1_score(y_test, y_pred, average='macro')
        f1_scores[name] = f1

```

```

        print(f"{name} Macro F1 Score: {f1:.4f}")

    # Convert to DataFrame and sort by F1 score
    df_f1 = pd.DataFrame(list(f1_scores.items()), columns=['Model', 'F1 Score'])
    df_f1 = df_f1.sort_values(by='F1 Score', ascending=False)

    # Plot the F1 scores
    plt.figure(figsize=(12, 6))
    ax = sns.barplot(x='F1 Score', y='Model', data=df_f1, palette='viridis')

    for index, value in enumerate(df_f1["F1 Score"]):
        ax.text(value + 0.01, index, f"{value:.2f}", va='center', fontsize=12,
        ↪fontweight='bold', color='black')

    plt.xlabel("F1 Score")
    plt.ylabel("Model")
    plt.title("Comparison of Model Macro F1 Scores for " + y_col)
    plt.xlim(0, 1)
    plt.grid(axis='x', linestyle='--', alpha=0.7)
    plt.show()

    return df_f1

```

```

[328]: prams = class_models_macro_f1_param_tunning(data_inp=credit_rating_outl_scaled,
        ↪X_cols= best_pred,
                                                y_col = 'Risk_level_encoded', test_size=0.
        ↪2)

```

Fitting 5 folds for each of 175 candidates, totalling 875 fits
 Fitting 5 folds for each of 27 candidates, totalling 135 fits
 Fitting 5 folds for each of 2 candidates, totalling 10 fits
 Fitting 5 folds for each of 3 candidates, totalling 15 fits
 Fitting 5 folds for each of 19 candidates, totalling 95 fits

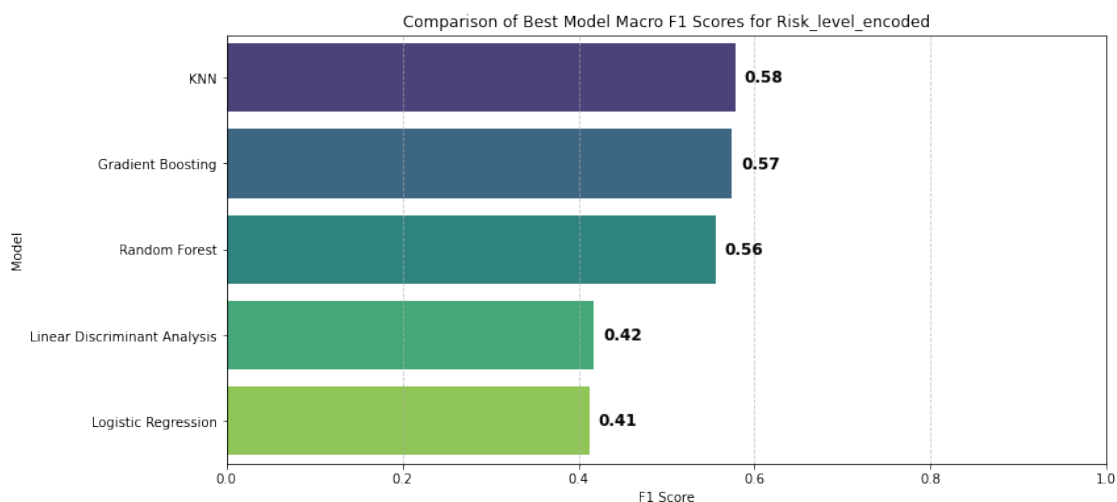
```

/Users/kamila/opt/anaconda3/lib/python3.9/site-
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will
always use 'multinomial'. Leave it to its default value to avoid this warning.
    warnings.warn(
/Users/kamila/opt/anaconda3/lib/python3.9/site-
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will
always use 'multinomial'. Leave it to its default value to avoid this warning.
    warnings.warn(
/Users/kamila/opt/anaconda3/lib/python3.9/site-
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will
always use 'multinomial'. Leave it to its default value to avoid this warning.

```


was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.

```
warnings.warn(
/Users/kamila/opt/anaconda3/lib/python3.9/site-
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will
always use 'multinomial'. Leave it to its default value to avoid this warning.
warnings.warn(
/Users/kamila/opt/anaconda3/lib/python3.9/site-
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will
always use 'multinomial'. Leave it to its default value to avoid this warning.
warnings.warn(
/Users/kamila/opt/anaconda3/lib/python3.9/site-
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will
always use 'multinomial'. Leave it to its default value to avoid this warning.
warnings.warn(
```



```
[335]: import pandas as pd

pd.set_option('display.max_columns', None) # Show all columns
#pd.set_option('display.width', 1000) # Avoid line breaks
prams['Best Parameters'][2]

#GB, RF, LDA, LR, KNN

#previous
# "Gradient Boosting": {
# "model": GradientBoostingClassifier,
```

```
#         "params": {"n_estimators": 200, 'learning_rate': 0.2, "max_depth": 9,
↳ "random_state": 123}
#     }
```

```
[335]: {'solver': 'svd'}
```

```
[441]: model_params = {
    "Gradient Boosting": {
        "model": GradientBoostingClassifier,
        "params": {"n_estimators": 300, 'learning_rate': 0.1, "max_depth": 7,
↳ "random_state": 123}
    },
    "Random Forest": {
        "model": RandomForestClassifier,
        "params": { "max_depth": 20, "n_estimators": 100, 'min_samples_split': 5,
↳ "class_weight": "balanced", "random_state": 123}
    },
    "Linear Discriminant Analysis": {
        "model": LinearDiscriminantAnalysis,
        "params": {"solver": "svd"}
    },
    "Logistic Regression": {
        "model": LogisticRegression,
        "params": {"class_weight": "balanced", "random_state": 123, 'C': 10,
↳ "multi_class": "multinomial", "solver": "newton-cg"}
    },
    "KNN": {
        "model": KNeighborsClassifier,
        "params": {"n_neighbors": 1, "weights": "distance"}
    }
}

class_models_macro_f1_params(data_inp=credit_rating_outl_scaled, X_cols=
↳ best_pred,
                                y_col = 'Risk_level_encoded', test_size=0.
↳ 2, model_params = model_params)
```

Gradient Boosting Macro F1 Score: 0.5337

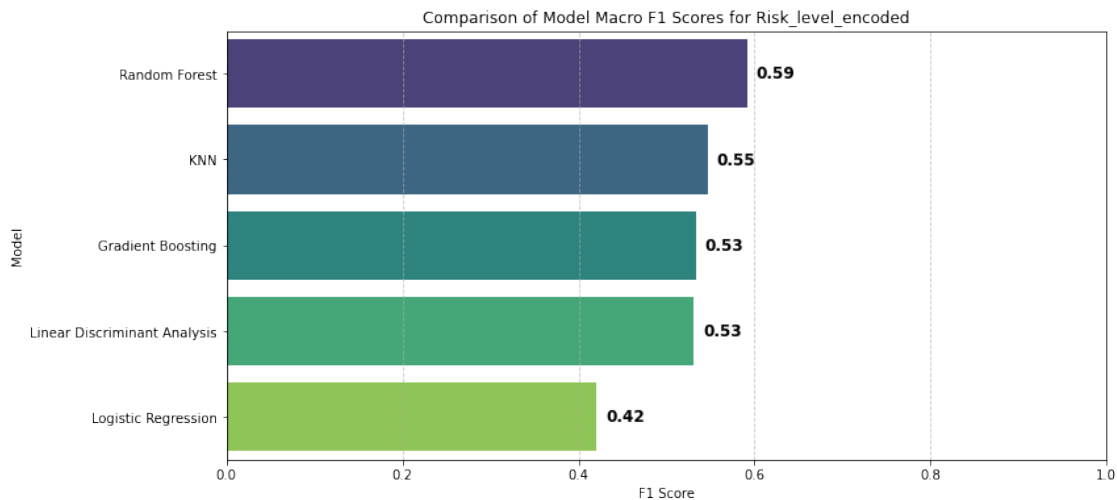
Random Forest Macro F1 Score: 0.5913

Linear Discriminant Analysis Macro F1 Score: 0.5311

Logistic Regression Macro F1 Score: 0.4208

KNN Macro F1 Score: 0.5466

```
/Users/kamila/opt/anaconda3/lib/python3.9/site-
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will
always use 'multinomial'. Leave it to its default value to avoid this warning.
warnings.warn(
```



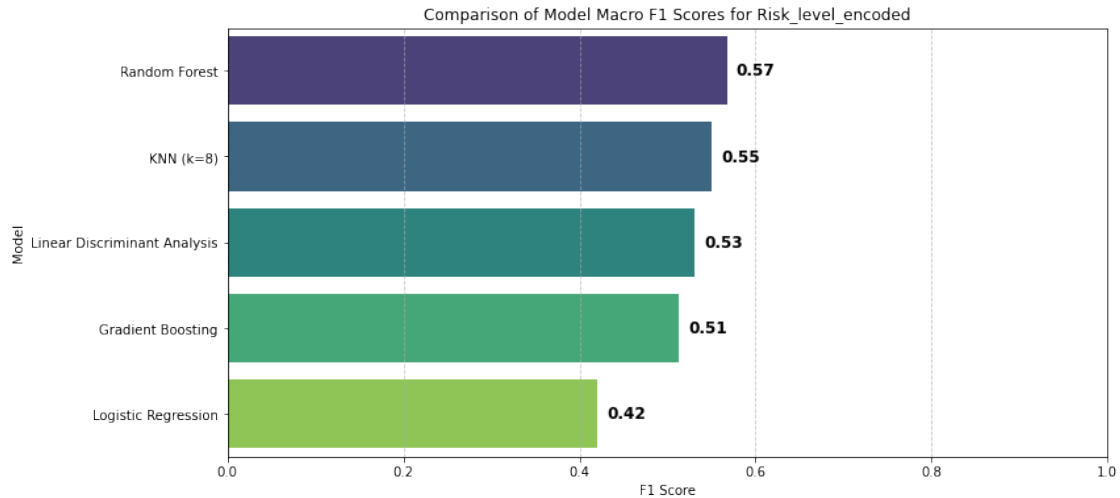
```
[441]:
      Model  F1 Score
1      Random Forest  0.591328
4              KNN  0.546634
0      Gradient Boosting  0.533653
2  Linear Discriminant Analysis  0.531054
3      Logistic Regression  0.420819
```

```
[431]: class_models_macro_f1(data_inp=credit_rating_out1_scaled, X_cols= best_pred,
      y_col = 'Risk_level_encoded', test_size=0.2, num_class = 4)
      ↪4)
```

```
Gradient Boosting Macro F1 Score: 0.5133
Random Forest Macro F1 Score: 0.5680
Linear Discriminant Analysis Macro F1 Score: 0.5311
Logistic Regression Macro F1 Score: 0.4208
```

```
/Users/kamila/opt/anaconda3/lib/python3.9/site-
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will
always use 'multinomial'. Leave it to its default value to avoid this warning.
warnings.warn(
```

```
Best KNN Macro F1 Score (8 neighbors): 0.5505
```

```
[431]:
          Model  F1 Score
1      Random Forest  0.568035
4          KNN (k=8)  0.550536
2  Linear Discriminant Analysis  0.531054
0      Gradient Boosting  0.513295
3      Logistic Regression  0.420819
```

```
[444]: data1 = class_models_macro_f1(data_inp=credit_rating_outl_scaled, X_cols=
      ↳ best_pred,
          y_col = 'Risk_level_encoded', test_size=0.2, num_class =
      ↳ 4)

data2 = class_models_macro_f1_params(data_inp=credit_rating_outl_scaled,
      ↳ X_cols= best_pred,
          y_col = 'Risk_level_encoded', test_size=0.
      ↳ 2, model_params = model_params)

compare_bar_charts(data1, data2, title = 'Comparison of Macro Avg F1 Scores',
      ↳ y_col = 'Risk level',
          label1 = 'Before param. tun.', label2 = 'After param. tun.',
      ↳ xlim=0.67,
          color1='navy', color2='darkmagenta')
```

Gradient Boosting Macro F1 Score: 0.5133

Random Forest Macro F1 Score: 0.5680

Linear Discriminant Analysis Macro F1 Score: 0.5311

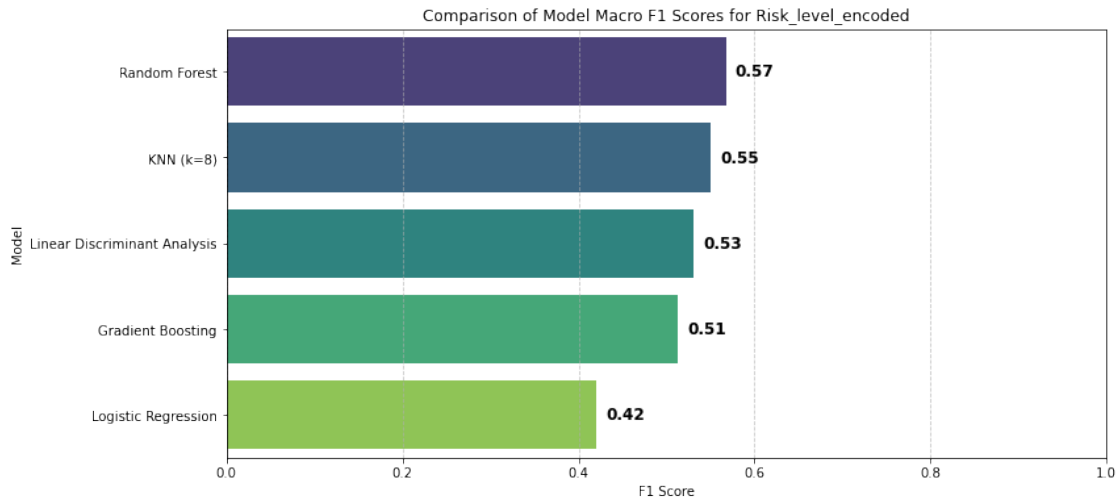
Logistic Regression Macro F1 Score: 0.4208

/Users/kamila/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'

was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.

```
warnings.warn(
```

Best KNN Macro F1 Score (8 neighbors): 0.5505



Gradient Boosting Macro F1 Score: 0.5337

Random Forest Macro F1 Score: 0.5913

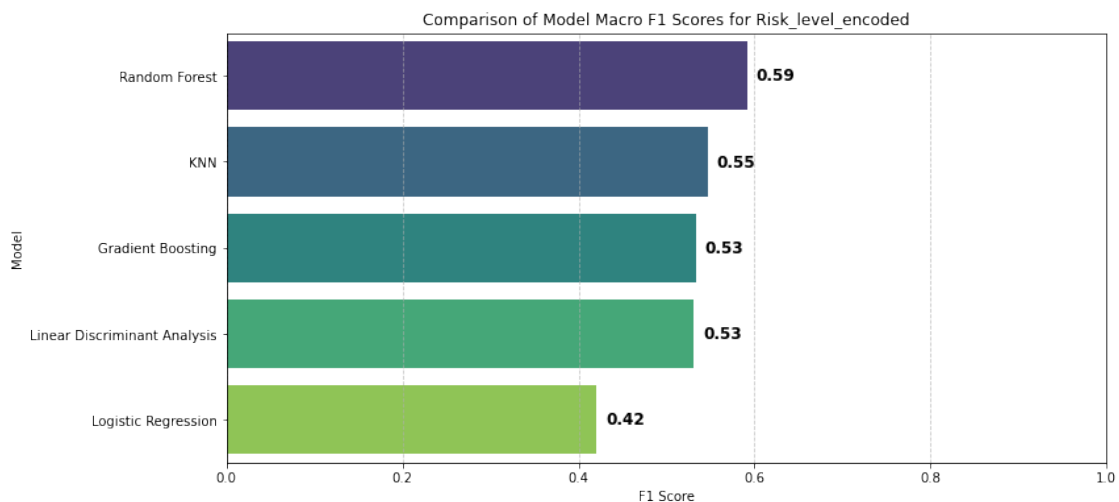
Linear Discriminant Analysis Macro F1 Score: 0.5311

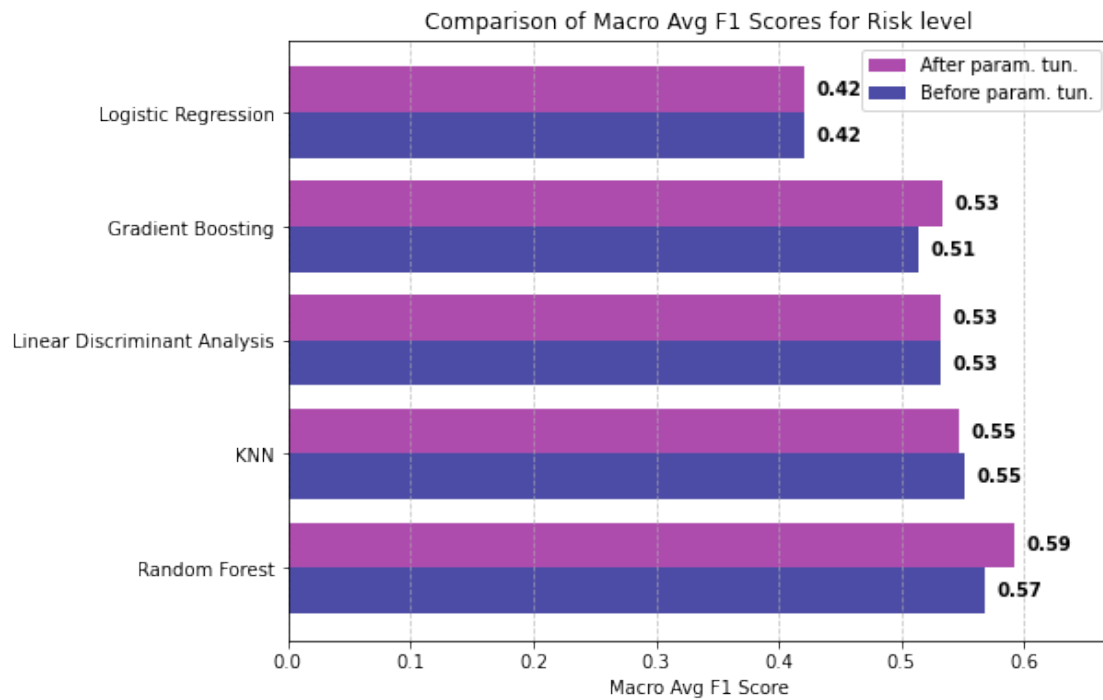
Logistic Regression Macro F1 Score: 0.4208

KNN Macro F1 Score: 0.5466

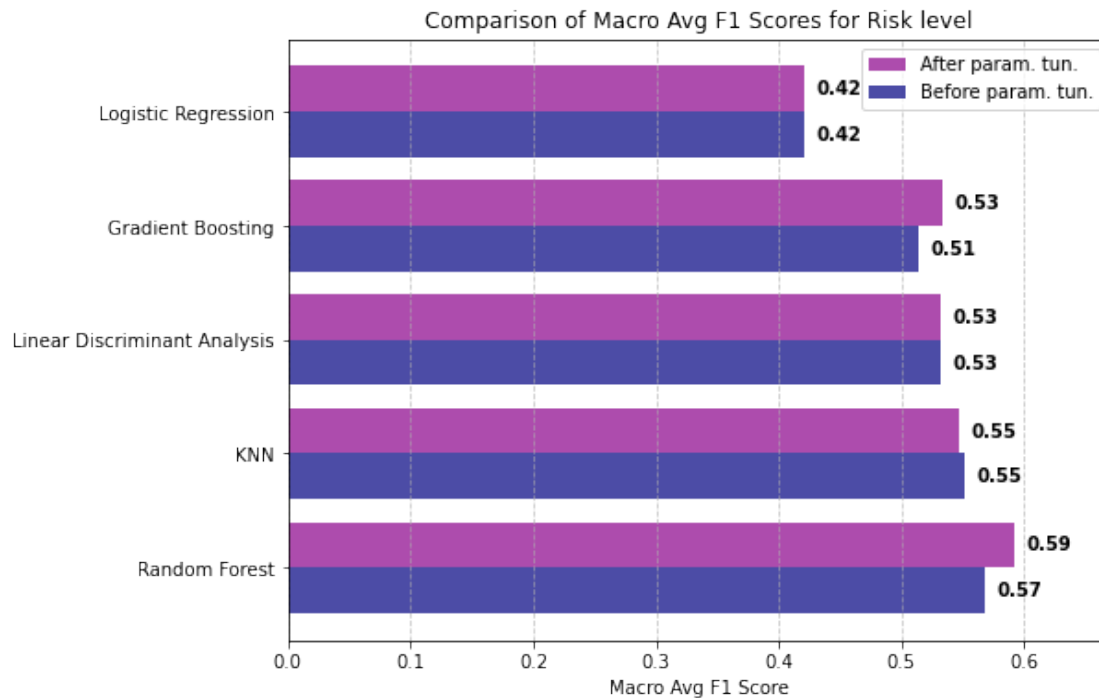
/Users/kamila/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.

```
warnings.warn(
```





```
[445]: compare_bar_charts(data1, data2, title = 'Comparison of Macro Avg F1 Scores',
    ↳ y_col = 'Risk level',
        label1 = 'Before param. tun.', label2 = 'After param. tun.',
    ↳ xlim=0.67,
        color1='navy', color2='darkmagenta')
```



```
[102]: prams = class_models_macro_f1_param_tunning(data_inp=credit_rating_outl_scaled,
↪X_cols= best_pred,
                                             y_col = 'Rating_encoded', test_size=0.2)
```

Fitting 5 folds for each of 175 candidates, totalling 875 fits

Fitting 5 folds for each of 27 candidates, totalling 135 fits

Fitting 5 folds for each of 2 candidates, totalling 10 fits

Fitting 5 folds for each of 3 candidates, totalling 15 fits

/Users/kamila/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.

warnings.warn(

/Users/kamila/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.

warnings.warn(

/Users/kamila/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.

warnings.warn(

/Users/kamila/opt/anaconda3/lib/python3.9/site-

```

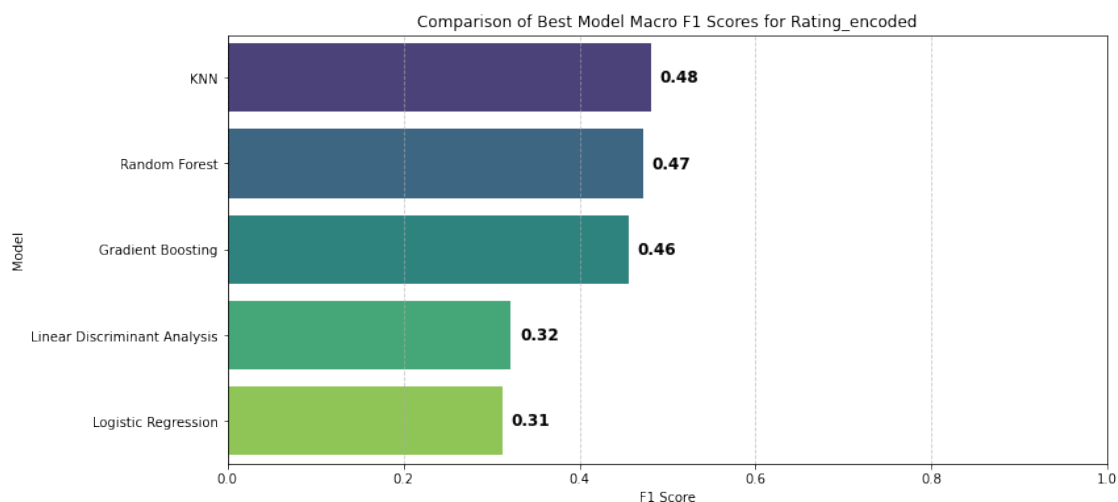
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will
always use 'multinomial'. Leave it to its default value to avoid this warning.
    warnings.warn(
/Users/kamila/opt/anaconda3/lib/python3.9/site-
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will
always use 'multinomial'. Leave it to its default value to avoid this warning.
    warnings.warn(
/Users/kamila/opt/anaconda3/lib/python3.9/site-
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will
always use 'multinomial'. Leave it to its default value to avoid this warning.
    warnings.warn(
/Users/kamila/opt/anaconda3/lib/python3.9/site-
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will
always use 'multinomial'. Leave it to its default value to avoid this warning.
    warnings.warn(
/Users/kamila/opt/anaconda3/lib/python3.9/site-
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will
always use 'multinomial'. Leave it to its default value to avoid this warning.
    warnings.warn(
/Users/kamila/opt/anaconda3/lib/python3.9/site-
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will
always use 'multinomial'. Leave it to its default value to avoid this warning.
    warnings.warn(
/Users/kamila/opt/anaconda3/lib/python3.9/site-
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will
always use 'multinomial'. Leave it to its default value to avoid this warning.
    warnings.warn(
/Users/kamila/opt/anaconda3/lib/python3.9/site-
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will
always use 'multinomial'. Leave it to its default value to avoid this warning.
    warnings.warn(
/Users/kamila/opt/anaconda3/lib/python3.9/site-
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will
always use 'multinomial'. Leave it to its default value to avoid this warning.
    warnings.warn(
/Users/kamila/opt/anaconda3/lib/python3.9/site-
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will
always use 'multinomial'. Leave it to its default value to avoid this warning.
    warnings.warn(

```

```
warnings.warn(
/Users/kamila/opt/anaconda3/lib/python3.9/site-
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will
always use 'multinomial'. Leave it to its default value to avoid this warning.
warni/Users/kamila/opt/anaconda3/lib/python3.9/site-
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will
always use 'multinomial'. Leave it to its default value to avoid this warning.
warnings.warn(
```

Fitting 5 folds for each of 19 candidates, totalling 95 fits

```
ngs.warn(
/Users/kamila/opt/anaconda3/lib/python3.9/site-
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will
always use 'multinomial'. Leave it to its default value to avoid this warning.
warnings.warn(
```



```
[108]: prams
        #GB, RF, LDA, LR, KNN
        prams['Best Parameters'][4]
```

```
[108]: {'n_neighbors': 1}
```

```
[446]: model_params2 = {
        "Gradient Boosting": {
            "model": GradientBoostingClassifier,
```

```

        "params": {"n_estimators": 100, 'learning_rate': 0.1, "max_depth": 7,
↪ "random_state": 123}
    },
    "Random Forest": {
        "model": RandomForestClassifier,
        "params": { "max_depth": None, "n_estimators": 200, 'min_samples_split':
↪2 , "class_weight": "balanced", "random_state": 123}
    },
    "Linear Discriminant Analysis": {
        "model": LinearDiscriminantAnalysis,
        "params": {"solver": "lsqr"}
    },
    "Logistic Regression": {
        "model": LogisticRegression,
        "params": {"class_weight": "balanced", "random_state": 123, 'C': 10,
↪ "multi_class": "multinomial", "solver": "newton-cg"}
    },
    "KNN": {
        "model": KNeighborsClassifier,
        "params": {"n_neighbors": 1, "weights": "distance"}
    }
}

```

```

[443]: class_models_macro_f1_params(data_inp=credit_rating_outl_scaled, X_cols=
↪ best_pred,
                                             y_col = 'Rating_encoded', test_size=0.2,
↪ model_params = model_params2)

```

Gradient Boosting Macro F1 Score: 0.4645

Random Forest Macro F1 Score: 0.4868

Linear Discriminant Analysis Macro F1 Score: 0.3443

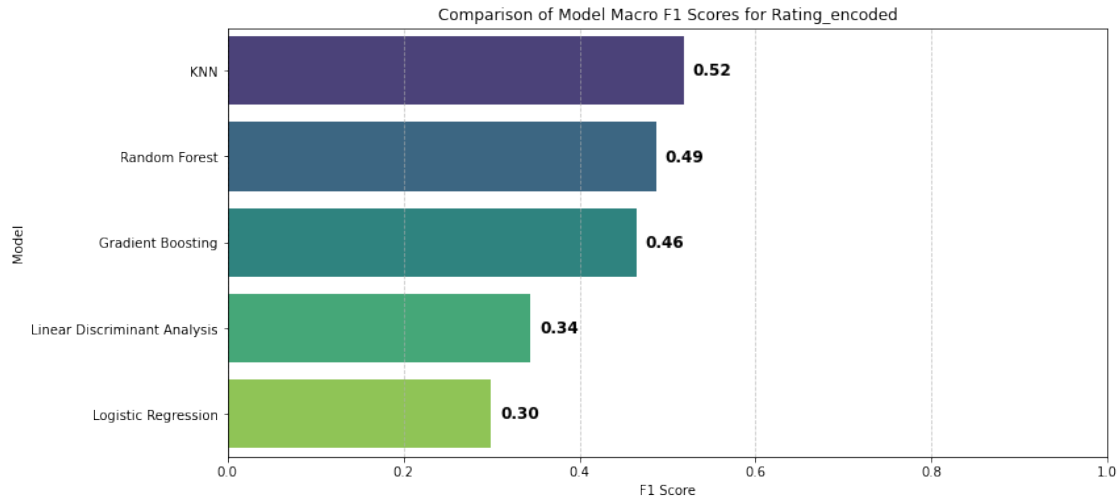
Logistic Regression Macro F1 Score: 0.2998

KNN Macro F1 Score: 0.5181

```

/Users/kamila/opt/anaconda3/lib/python3.9/site-
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will
always use 'multinomial'. Leave it to its default value to avoid this warning.
warnings.warn(

```



```
[443]:
```

	Model	F1 Score
4	KNN	0.518143
1	Random Forest	0.486822
0	Gradient Boosting	0.464482
2	Linear Discriminant Analysis	0.344346
3	Logistic Regression	0.299781

```
[111]: class_models_macro_f1(data_inp=credit_rating_outl_scaled, X_cols= best_pred,
                             y_col = 'Rating_encoded', test_size=0.2, num_class = 6)
```

Gradient Boosting Macro F1 Score: 0.4162

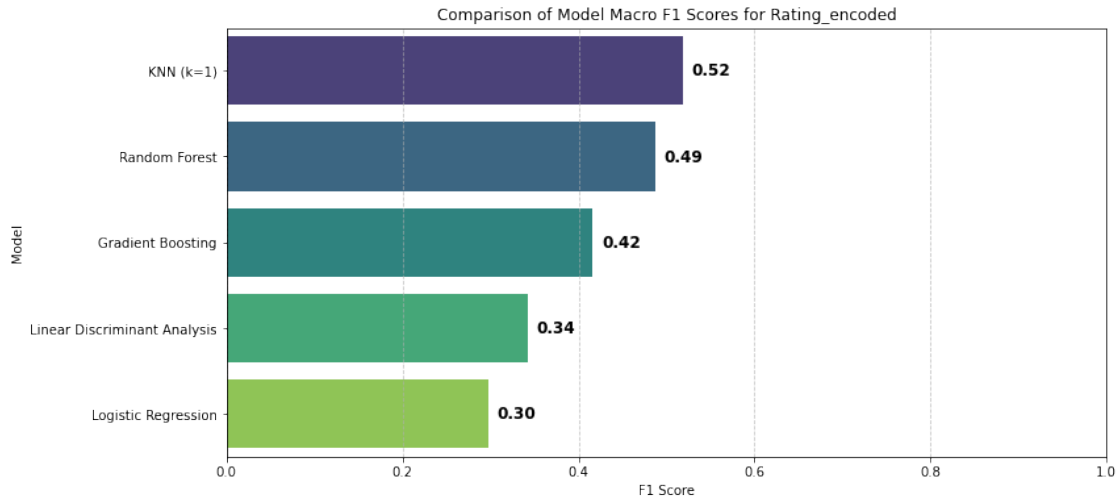
Random Forest Macro F1 Score: 0.4868

Linear Discriminant Analysis Macro F1 Score: 0.3424

Logistic Regression Macro F1 Score: 0.2978

```
/Users/kamila/opt/anaconda3/lib/python3.9/site-
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will
always use 'multinomial'. Leave it to its default value to avoid this warning.
  warnings.warn(
```

Best KNN Macro F1 Score (1 neighbors): 0.5181



```
[111]:
```

	Model	F1 Score
4	KNN (k=1)	0.518143
1	Random Forest	0.486822
0	Gradient Boosting	0.416186
2	Linear Discriminant Analysis	0.342437
3	Logistic Regression	0.297799

```
[447]: data1 = class_models_macro_f1(data_inp=credit_rating_outl_scaled, X_cols=
↳best_pred,
                                y_col = 'Rating_encoded', test_size=0.2, num_class = 6)

data2 = class_models_macro_f1_params(data_inp=credit_rating_outl_scaled,
↳X_cols= best_pred,
                                y_col = 'Rating_encoded', test_size=0.2,
↳model_params = model_params2)

compare_bar_charts(data1, data2, title = 'Comparison of Macro Avg F1 Scores',
↳y_col = 'Rating',
                                label1 = 'Before param. tun.', label2 = 'After param. tun.',
↳xlim=0.67,
                                color1='navy', color2='darkmagenta')
```

Gradient Boosting Macro F1 Score: 0.4162

Random Forest Macro F1 Score: 0.4868

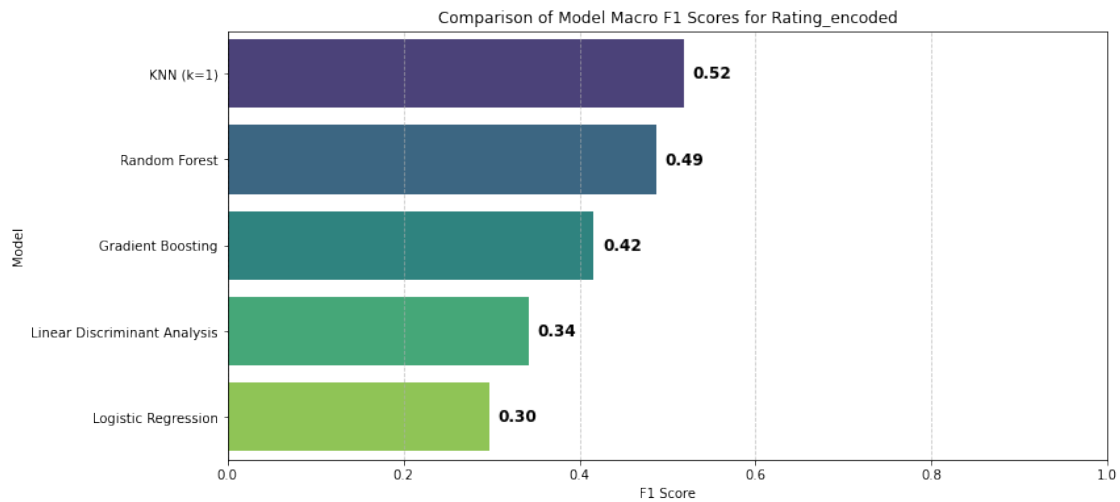
Linear Discriminant Analysis Macro F1 Score: 0.3424

Logistic Regression Macro F1 Score: 0.2978

Best KNN Macro F1 Score (1 neighbors): 0.5181

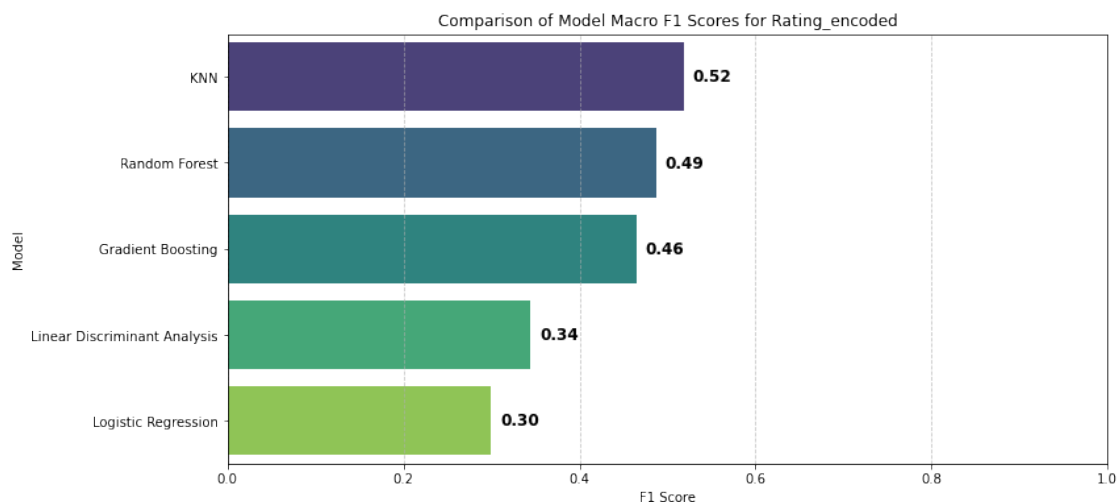
/Users/kamila/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'

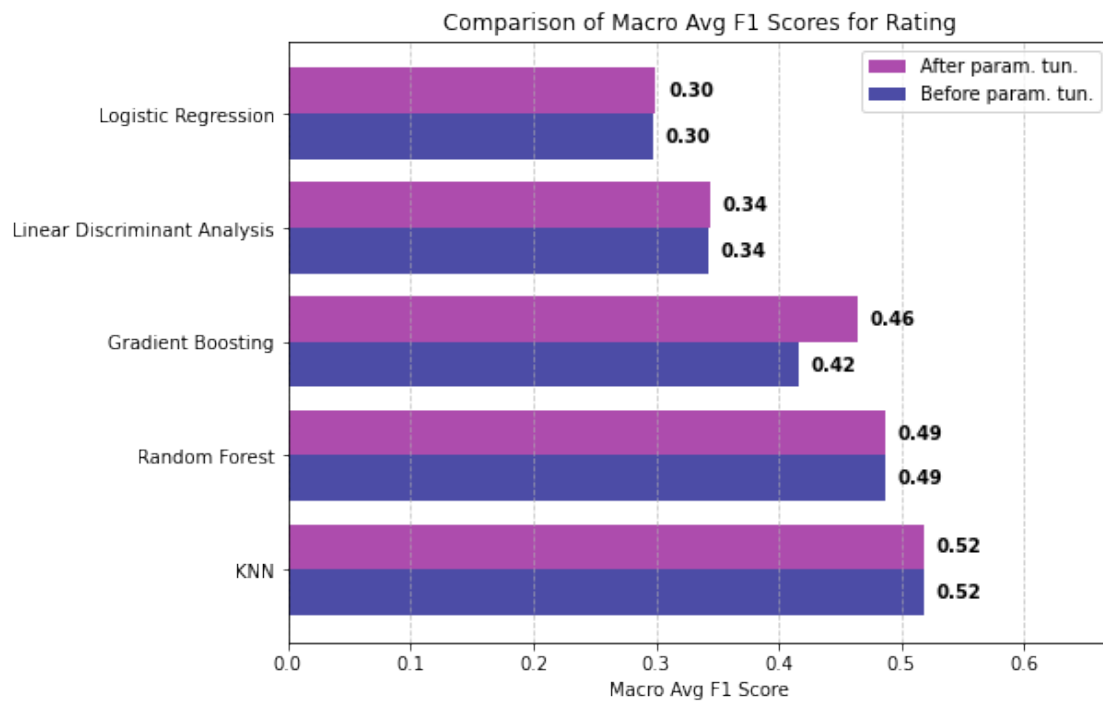
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.
warnings.warn(



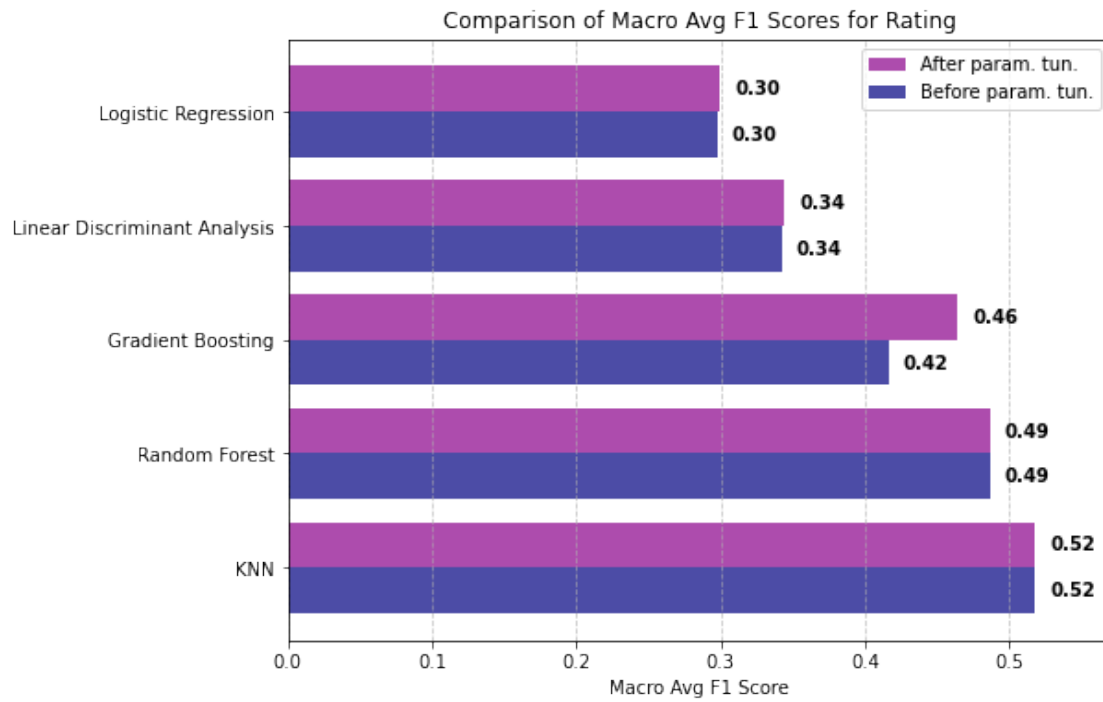
Gradient Boosting Macro F1 Score: 0.4645
Random Forest Macro F1 Score: 0.4868
Linear Discriminant Analysis Macro F1 Score: 0.3443
Logistic Regression Macro F1 Score: 0.2998
KNN Macro F1 Score: 0.5181

/Users/kamila/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.
warnings.warn(



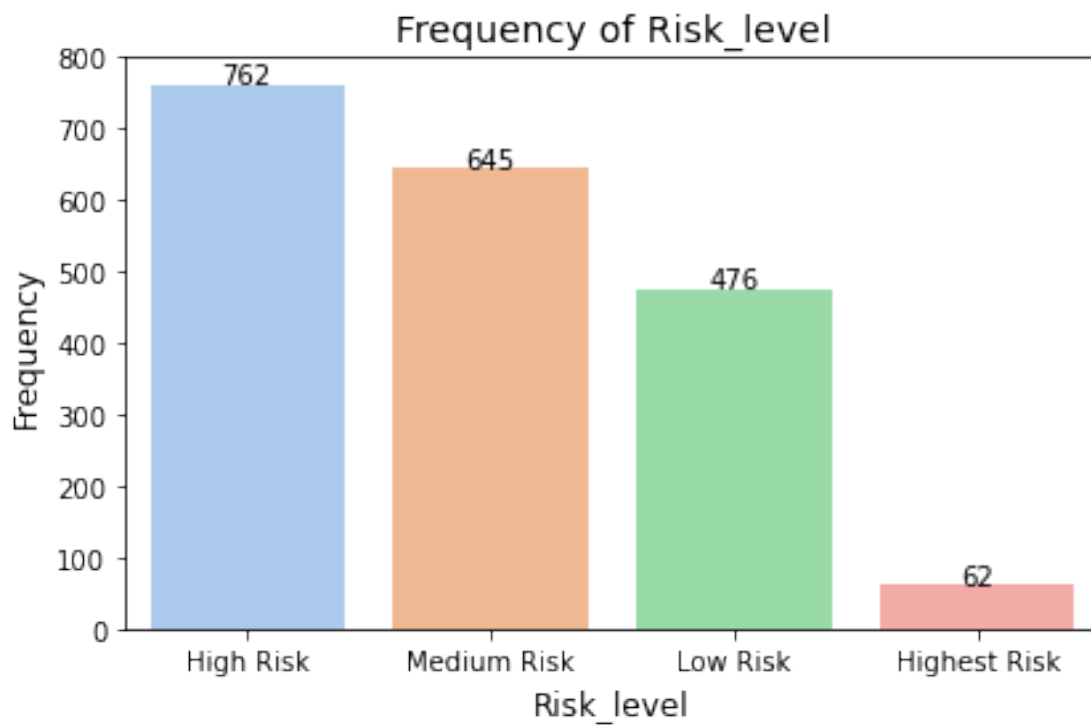


```
[450]: compare_bar_charts(data1, data2, title = 'Comparison of Macro Avg F1 Scores',
    ↳ y_col = 'Rating',
    label1 = 'Before param. tun.', label2 = 'After param. tun.',
    ↳ xlim=0.57,
    color1='navy', color2='darkmagenta')
```

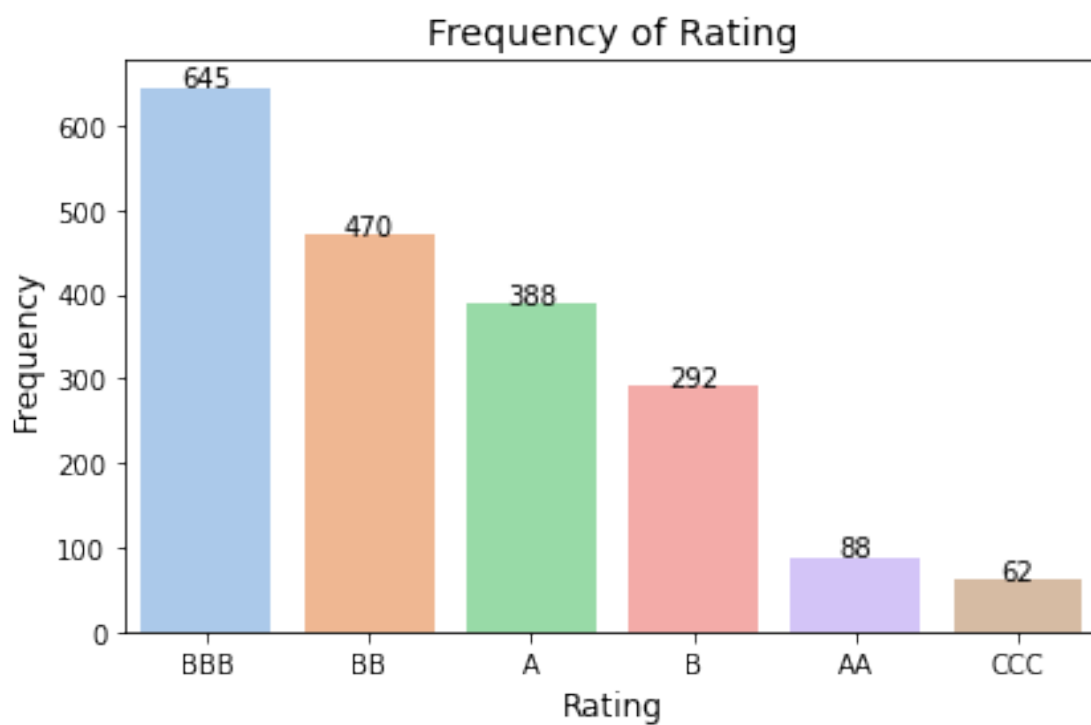


8 ML for Balanced dataset

```
[133]: freq_graph(credit_rating_out1_scaled.copy(), 'Risk_level', 0)
```



```
[134]: freq_graph(credit_rating_outl_scaled.copy(), 'Rating', 0)
```



```

[451]: def class_models_macro_f1_params_balanced(data_inp, X_cols, y_col, test_size,
↳model_params):
    data = data_inp.copy()

    # Encode sector
    le = preprocessing.LabelEncoder()
    le.fit(data.Sector)
    data.Sector = le.transform(data.Sector)

    # Split dataset into training and testing while maintaining category
↳proportions
    X = data[X_cols] # Features
    y = data[y_col] # Target

    X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                         test_size=test_size, random_state=123)

    f1_scores = {}

    # Train and evaluate each model using provided parameters
    for name, params in model_params.items():
        model = params["model"](**params["params"])

        model.fit(X_train, y_train)

        y_pred = model.predict(X_test)
        f1 = f1_score(y_test, y_pred, average='macro')
        f1_scores[name] = f1
        print(f"{name} Macro F1 Score: {f1:.4f}")

    # Convert to DataFrame and sort by F1 score
    df_f1 = pd.DataFrame(list(f1_scores.items()), columns=['Model', 'F1 Score'])
    df_f1 = df_f1.sort_values(by='F1 Score', ascending=False)

    # Plot the F1 scores
    plt.figure(figsize=(12, 6))
    ax = sns.barplot(x='F1 Score', y='Model', data=df_f1, palette='viridis')

    for index, value in enumerate(df_f1["F1 Score"]):
        ax.text(value + 0.01, index, f"{value:.2f}", va='center', fontsize=12,
↳fontweight='bold', color='black')

    plt.xlabel("F1 Score")
    plt.ylabel("Model")

```

```

plt.title("Comparison of Model Macro F1 Scores for " + y_col)
plt.xlim(0, 1)
plt.grid(axis='x', linestyle='--', alpha=0.7)
plt.show()

return df_f1

def class_models_macro_f1_balanced(data_inp, X_cols, y_col, test_size):

    data = data_inp.copy()

    le = preprocessing.LabelEncoder()
    le.fit(data.Sector)
    data.Sector = le.transform(data.Sector) # Encode sector

    # Split dataset into training and testing while maintaining category
    ↳ proportions
    X = data[X_cols] # Features
    y = data[y_col] # Target

    X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                         test_size=test_size, random_state=123)

    # Initialize models
    models = {
        "Gradient Boosting": GradientBoostingClassifier(random_state=123),
        "Random Forest": RandomForestClassifier(n_estimators=200,
    ↳ random_state=123),
        "Linear Discriminant Analysis": LinearDiscriminantAnalysis(),
        "Logistic Regression": LogisticRegression(random_state=123,
    ↳ multi_class='multinomial', solver='newton-cg')
    }

    f1_scores = {}

    # Train and evaluate each model using MACRO F1
    for name, model in models.items():

        model.fit(X_train, y_train)

```

```

y_pred = model.predict(X_test)

#Compute macro avg F1
f1 = f1_score(y_test, y_pred, average='macro')
f1_scores[name] = f1
print(f"{name} Macro F1 Score: {f1:.4f}")

# Train KNN models with k from 1 to 10 and find the best k
best_knn_k = 1
best_knn_f1 = 0

for k in range(1, 11):
    knn_model = KNeighborsClassifier(n_neighbors=k)
    knn_model.fit(X_train, y_train)
    y_pred_knn = knn_model.predict(X_test)

    f1_knn = f1_score(y_test, y_pred_knn, average='macro')

    if f1_knn > best_knn_f1:
        best_knn_f1 = f1_knn
        best_knn_k = k

# Store the best KNN Macro F1 score
f1_scores[f"KNN (k={best_knn_k})"] = best_knn_f1
print(f"Best KNN Macro F1 Score ({best_knn_k} neighbors): {best_knn_f1:.
→4f}")

# Convert to DataFrame and sort by F1 score
df_f1 = pd.DataFrame(list(f1_scores.items()), columns=['Model', 'F1 Score'])
df_f1 = df_f1.sort_values(by='F1 Score', ascending=False)

# Plot the F1 scores
plt.figure(figsize=(12, 6))
ax = sns.barplot(x='F1 Score', y='Model', data=df_f1, palette='viridis')

# Add percentage labels on bars
for index, value in enumerate(df_f1["F1 Score"]):
    ax.text(value + 0.01, index, f"{value:.2f}", va='center', fontsize=12,
→fontweight='bold', color='black')

plt.xlabel("F1 Score")
plt.ylabel("Model")
plt.title("Comparison of Model Macro F1 Scores for " + y_col)
plt.xlim(0, 1) # F1 score ranges from 0 to 1
plt.grid(axis='x', linestyle='--', alpha=0.7)

```



```
plt.show()

return df_f1
```

```
[452]: #Undersampling
#!pip install imbalanced-learn
#from imblearn.under_sampling import RandomUnderSampler
from imblearn.over_sampling import SMOTE

X_cols=['Sector','currentRatio', 'quickRatio', 'cashRatio',
↳ 'daysOfSalesOutstanding',
    'netProfitMargin', 'pretaxProfitMargin', 'grossProfitMargin',
    'operatingProfitMargin', 'returnOnAssets', 'returnOnCapitalEmployed',
    'returnOnEquity', 'assetTurnover', 'fixedAssetTurnover',
    'debtEquityRatio', 'debtRatio', 'effectiveTaxRate',
    'freeCashFlowOperatingCashFlowRatio', 'freeCashFlowPerShare',
    'cashPerShare', 'companyEquityMultiplier', 'ebitPerRevenue',
    'enterpriseValueMultiple', 'operatingCashFlowPerShare',
    'operatingCashFlowSalesRatio', 'payablesTurnover']

y_col = 'Risk_level_encoded'

X = credit_rating_outl_scaled[X_cols]

# Encode sector
le = preprocessing.LabelEncoder()
le.fit(X.Sector)
X.Sector = le.transform(X.Sector)

y = credit_rating_outl_scaled[y_col]

# rus = RandomUnderSampler(random_state=123)
# X_resampled, y_resampled = rus.fit_resample(X, y)

#print(X_resampled.iloc[80:90, :])
#print(y_resampled[80:90])

smote = SMOTE(random_state=123)
X_resampled, y_resampled = smote.fit_resample(X, y)

credit_rating_balanced = pd.DataFrame(X_resampled, columns=X_cols)

credit_rating_balanced[y_col] = y_resampled
```

/Users/kamila/opt/anaconda3/lib/python3.9/site-

```
packages/pandas/core/generic.py:5516: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
self[name] = value
/Users/kamila/opt/anaconda3/lib/python3.9/site-packages/sklearn/base.py:474:
FutureWarning: `BaseEstimator._validate_data` is deprecated in 1.6 and will be
removed in 1.7. Use `sklearn.utils.validation.validate_data` instead. This
function becomes public and is part of the scikit-learn developer API.
warnings.warn(
```

```
[185]: credit_rating_balanced.iloc[80:90, :]
```

```
[185]:
```

	Sector	currentRatio	quickRatio	cashRatio	daysOfSalesOutstanding \
80	0	-0.700246	-0.648947	-0.577016	0.378388
81	0	0.377204	0.269728	0.230619	-0.373109
82	8	-0.097811	0.694817	2.402654	-1.487025
83	1	0.438288	-0.999606	-0.047760	-1.487025
84	4	0.887473	0.450604	0.509958	1.567977
85	9	-1.185851	-1.109728	-1.083515	-0.445099
86	1	-0.573868	-1.001954	-0.472039	-0.420762
87	0	0.280445	-0.303494	-0.783609	0.421378
88	0	-0.582340	-0.727144	-0.201139	-0.920586
89	5	-0.838354	-1.485087	-0.943146	-1.487025

	netProfitMargin	pretaxProfitMargin	grossProfitMargin \
80	-0.745764	0.346021	-0.786480
81	0.328281	0.419765	-0.818440
82	1.650795	1.304126	-0.209377
83	-0.704557	-0.677070	-1.056717
84	0.437321	0.808744	0.398540
85	0.220460	0.416065	-0.258959
86	0.346686	-0.085110	3.455656
87	-0.142703	-0.082578	-0.872938
88	1.950041	2.097991	-0.325514
89	2.073574	2.097991	1.455586

	operatingProfitMargin	returnOnAssets	returnOnCapitalEmployed \
80	0.101050	-0.841097	-0.231197
81	0.238509	1.267741	1.167268
82	1.177618	1.569891	0.790484
83	-0.869547	-0.743846	-0.686731
84	1.371400	0.232046	0.359982
85	0.434124	-0.135630	-0.154033
86	-0.941612	0.739983	0.624944

87	-0.247598	0.223648	0.211671
88	1.975715	1.244871	0.971426
89	1.313918	0.776089	0.672786

	returnOnEquity	assetTurnover	fixedAssetTurnover	debtEquityRatio \
80	-0.745976	-0.780002	-0.883690	2.045609
81	1.621105	0.595420	-0.601316	0.211260
82	1.957768	-0.384135	2.433652	2.045609
83	-0.654835	0.175237	0.341977	0.469107
84	-1.606090	-0.492800	2.207414	-2.215450
85	0.063987	-0.676548	-0.909084	0.321566
86	1.957768	0.063733	0.272245	2.045609
87	-0.024709	0.360554	-0.007829	-0.589093
88	1.018206	-0.627972	-0.888129	-0.311214
89	0.205795	-0.882484	-0.935628	-0.784379

	debtRatio	effectiveTaxRate	freeCashFlowOperatingCashFlowRatio \
80	1.224816	0.143961	-0.778032
81	0.250566	0.270222	-0.538593
82	1.409769	-0.835262	0.759761
83	0.432686	1.590325	-2.362806
84	2.390122	0.657509	0.903841
85	0.333499	0.213422	-2.241723
86	1.971092	-2.046952	0.117938
87	-0.748916	0.104542	0.138325
88	-0.292391	-0.352092	0.206110
89	-1.198134	0.345038	1.033307

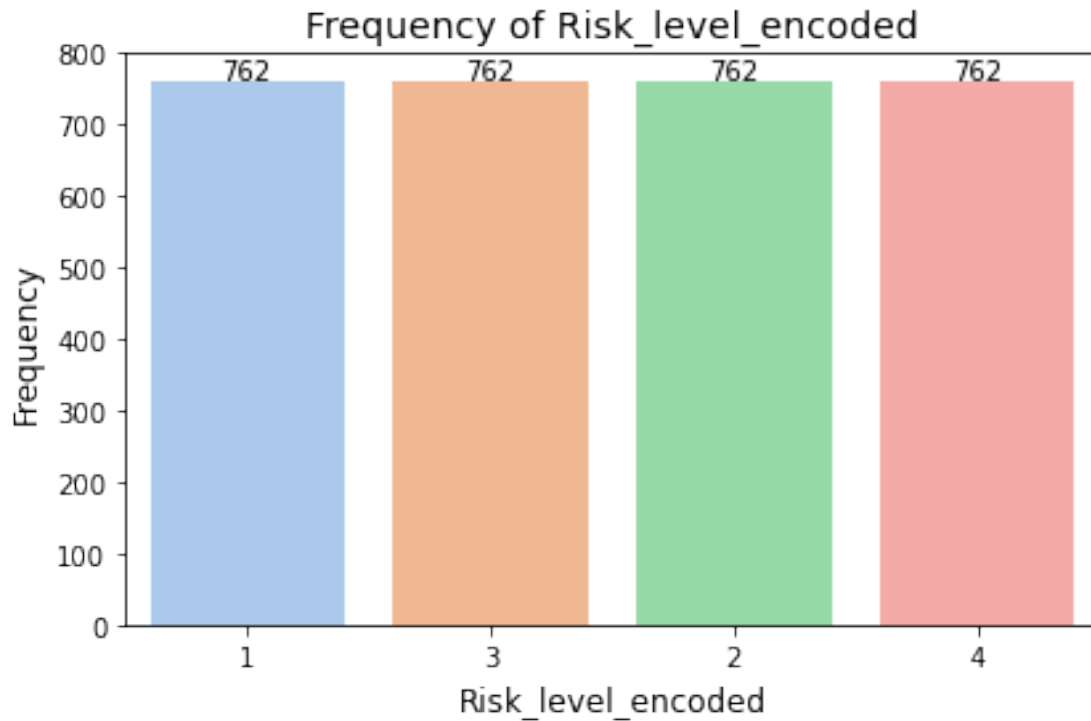
	freeCashFlowPerShare	cashPerShare	companyEquityMultiplier \
80	-0.563760	-0.308583	2.059807
81	-0.386253	0.080901	0.205722
82	-0.264815	-0.630702	2.059807
83	-0.888217	0.266761	0.461399
84	0.261657	-0.459094	-2.217843
85	-2.009611	-1.011944	0.315099
86	0.159103	1.690532	2.059807
87	-0.210380	-0.805438	-0.587893
88	2.148197	-0.242211	-0.312354
89	2.152900	-0.886975	-0.781535

	ebitPerRevenue	enterpriseValueMultiple	operatingCashFlowPerShare \
80	0.309255	-1.219622	-0.372595
81	0.382340	-0.601657	-0.187581
82	1.258805	-0.515133	-0.817389
83	-0.704702	1.314893	-1.224773
84	0.767846	-1.020514	-0.378149
85	0.378673	-0.660270	0.787988

86	-0.118027	2.081574	0.037412
87	-0.115517	0.011675	-0.516330
88	2.099395	-0.833124	2.279574
89	2.099395	-0.773485	2.279574

	operatingCashFlowSalesRatio	payablesTurnover	Risk_level_encoded
80	-0.007206	-0.049138	3
81	-0.593303	-0.176918	2
82	0.337599	-0.170826	1
83	-1.210587	0.079346	4
84	0.344472	0.021352	3
85	0.330260	-0.158302	2
86	-0.802714	-2.124318	1
87	-0.797087	0.512575	2
88	1.239424	0.689612	2
89	2.289387	-1.169047	1

```
[186]: freq_graph(credit_rating_balanced.copy(), 'Risk_level_encoded', 0)
```



```
[457]: model_params_bal = {
        "Gradient Boosting": {
            "model": GradientBoostingClassifier,
```

```

        "params": {"n_estimators": 300, 'learning_rate': 0.1, "max_depth": 7,
↪ "random_state": 123}
    },
    "Random Forest": {
        "model": RandomForestClassifier,
        "params": { "max_depth": 20, "n_estimators": 100, 'min_samples_split':
↪ 5, "random_state": 123}
    },
    "Linear Discriminant Analysis": {
        "model": LinearDiscriminantAnalysis,
        "params": {"solver": "svd"}
    },
    "Logistic Regression": {
        "model": LogisticRegression,
        "params": { "random_state": 123, 'C': 10, "multi_class":
↪ "multinomial", "solver": "newton-cg"}
    },
    "KNN": {
        "model": KNeighborsClassifier,
        "params": {"n_neighbors": 1}
    }
}

```

```

[458]: class_models_macro_f1_params_balanced(data_inp=credit_rating_balanced, X_cols=
↪ best_pred,
                                              y_col = 'Risk_level_encoded', test_size=0.
↪ 2, model_params = model_params_bal)

```

Gradient Boosting Macro F1 Score: 0.7477

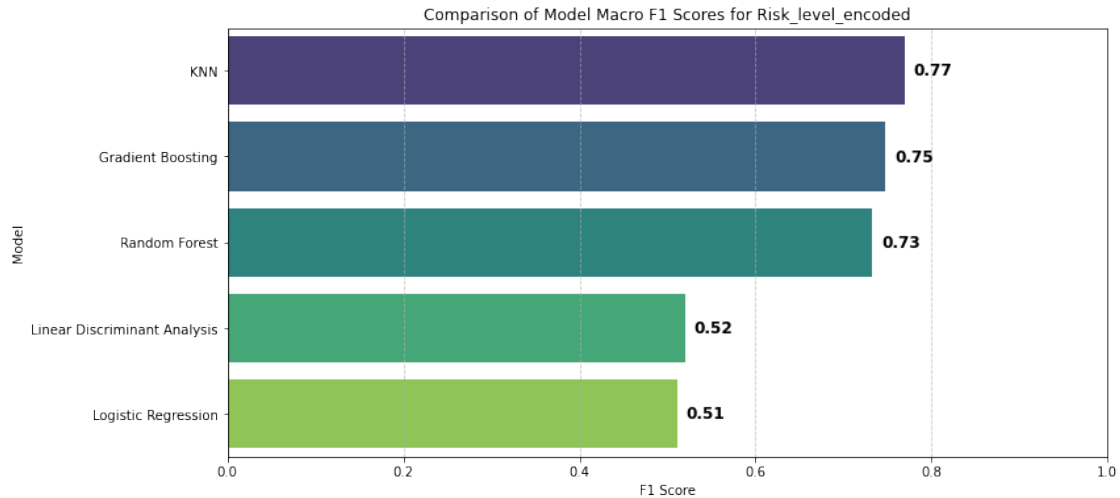
Random Forest Macro F1 Score: 0.7326

Linear Discriminant Analysis Macro F1 Score: 0.5199

Logistic Regression Macro F1 Score: 0.5115

KNN Macro F1 Score: 0.7694

/Users/kamila/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.
warnings.warn(



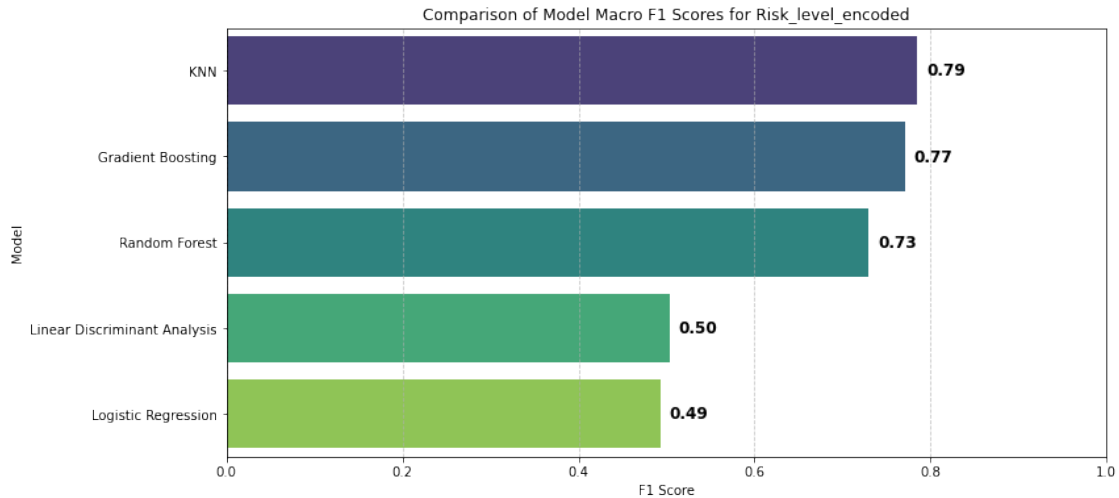
```
[458]:
```

	Model	F1 Score
4	KNN	0.769381
0	Gradient Boosting	0.747717
1	Random Forest	0.732621
2	Linear Discriminant Analysis	0.519921
3	Logistic Regression	0.511455

```
[459]: class_models_macro_f1_params(data_inp=credit_rating_balanced, X_cols= best_pred,
                                     y_col = 'Risk_level_encoded', test_size=0.
                                     ↪2, model_params = model_params)
```

```
Gradient Boosting Macro F1 Score: 0.7712
Random Forest Macro F1 Score: 0.7298
Linear Discriminant Analysis Macro F1 Score: 0.5032
Logistic Regression Macro F1 Score: 0.4927
KNN Macro F1 Score: 0.7850
```

```
/Users/kamila/opt/anaconda3/lib/python3.9/site-
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.7. From then on, it will
always use 'multinomial'. Leave it to its default value to avoid this warning.
warnings.warn(
```



```
[459]:
          Model  F1 Score
4              KNN  0.785019
0      Gradient Boosting  0.771184
1          Random Forest  0.729794
2  Linear Discriminant Analysis  0.503240
3      Logistic Regression  0.492737
```

```
[460]: data1 = class_models_macro_f1_params(data_inp=credit_rating_outl_scaled,
      ↪X_cols= best_pred,
      ↪y_col = 'Risk_level_encoded', test_size=0.
      ↪2, model_params = model_params)

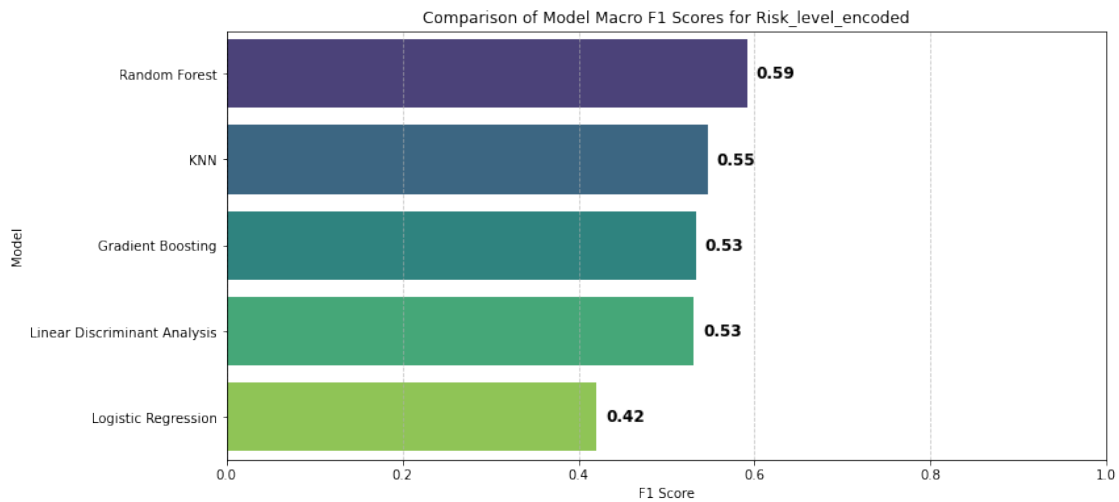
data2 = class_models_macro_f1_params_balanced(data_inp=credit_rating_balanced,
      ↪X_cols= best_pred,
      ↪y_col = 'Risk_level_encoded', test_size=0.
      ↪2, model_params = model_params_bal)

compare_bar_charts(data1, data2, title = 'Comparison of Macro Avg F1 Scores',
      ↪y_col = 'Risk level',
      label1 = 'Before SMOTE', label2 = 'After SMOTE', xlim=1,
      color1='navy', color2='darkmagenta')
```

```
Gradient Boosting Macro F1 Score: 0.5337
Random Forest Macro F1 Score: 0.5913
Linear Discriminant Analysis Macro F1 Score: 0.5311
Logistic Regression Macro F1 Score: 0.4208
KNN Macro F1 Score: 0.5466
```

```
/Users/kamila/opt/anaconda3/lib/python3.9/site-
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'
```

was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.
warnings.warn(



Gradient Boosting Macro F1 Score: 0.7477

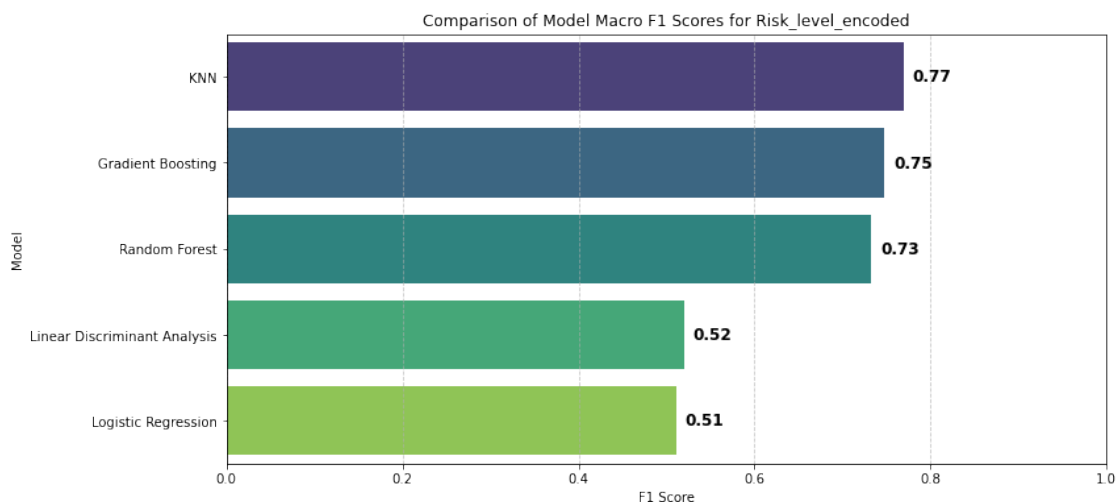
Random Forest Macro F1 Score: 0.7326

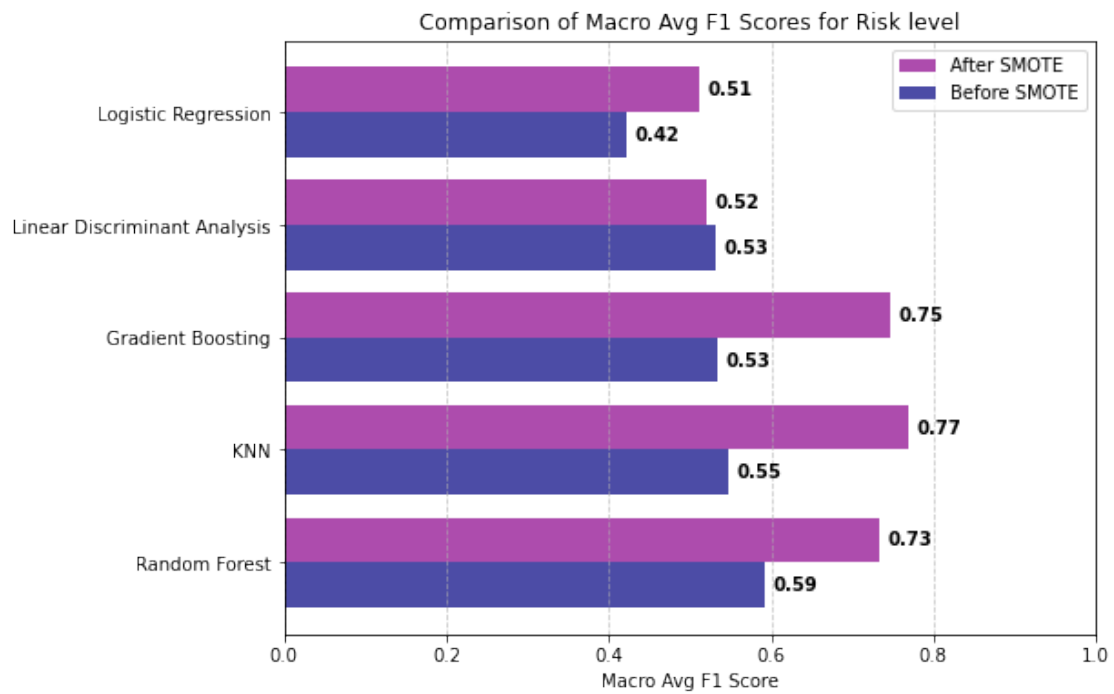
Linear Discriminant Analysis Macro F1 Score: 0.5199

Logistic Regression Macro F1 Score: 0.5115

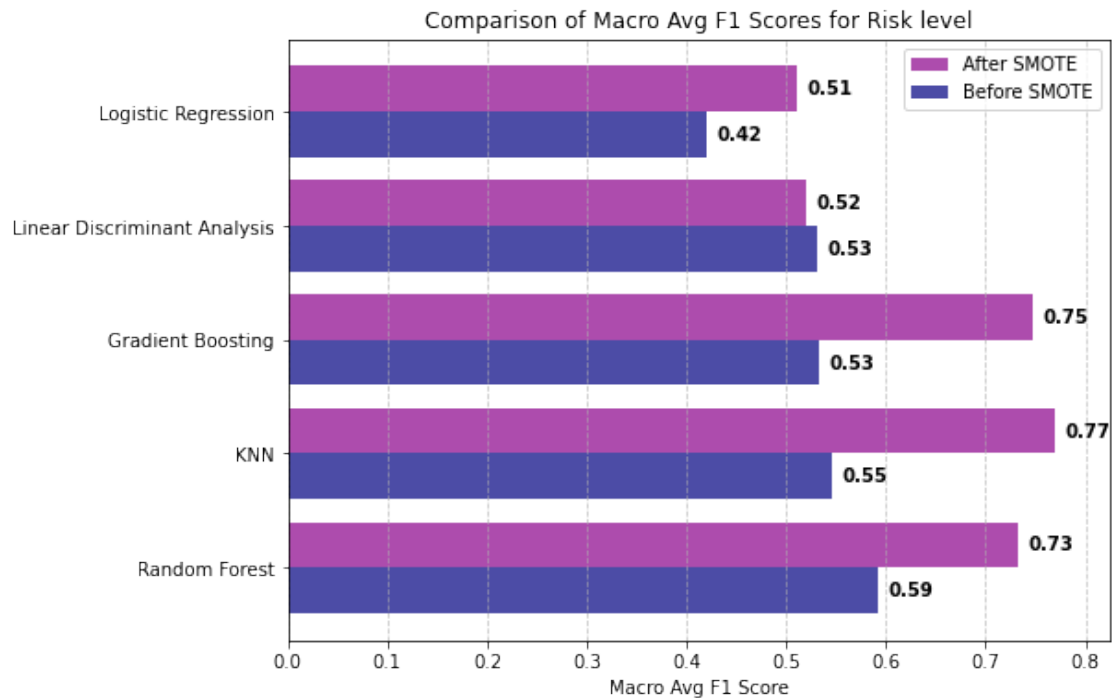
KNN Macro F1 Score: 0.7694

/Users/kamila/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.
warnings.warn(





```
[462]: compare_bar_charts(data1, data2, title = 'Comparison of Macro Avg F1 Scores',
    ↪ y_col = 'Risk level',
    label1 = 'Before SMOTE', label2 = 'After SMOTE', xlim=0.825,
    color1='navy', color2='darkmagenta')
```



```
[465]: #from imblearn.under_sampling import RandomUnderSampler
from imblearn.over_sampling import SMOTE

X_cols=['Sector','currentRatio', 'quickRatio', 'cashRatio', '
↳ 'daysOfSalesOutstanding',
        'netProfitMargin', 'pretaxProfitMargin', 'grossProfitMargin',
        'operatingProfitMargin', 'returnOnAssets', 'returnOnCapitalEmployed',
        'returnOnEquity', 'assetTurnover', 'fixedAssetTurnover',
        'debtEquityRatio', 'debtRatio', 'effectiveTaxRate',
        'freeCashFlowOperatingCashFlowRatio', 'freeCashFlowPerShare',
        'cashPerShare', 'companyEquityMultiplier', 'ebitPerRevenue',
        'enterpriseValueMultiple', 'operatingCashFlowPerShare',
        'operatingCashFlowSalesRatio', 'payablesTurnover']

y_col = 'Rating_encoded'

X = credit_rating_outl_scaled[X_cols]

y = credit_rating_outl_scaled[y_col]

# Encode sector
le = preprocessing.LabelEncoder()
le.fit(X.Sector)
```

```

X.Sector = le.transform(X.Sector)

# rus = RandomUnderSampler(random_state=42)
# X_resampled, y_resampled = rus.fit_resample(X, y)

smote = SMOTE(random_state=123)
X_resampled, y_resampled = smote.fit_resample(X, y)

credit_rating_balanced2 = pd.DataFrame(X_resampled, columns=X_cols)

credit_rating_balanced2[y_col] = y_resampled

```

```

/Users/kamila/opt/anaconda3/lib/python3.9/site-
packages/pandas/core/generic.py:5516: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

```

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```

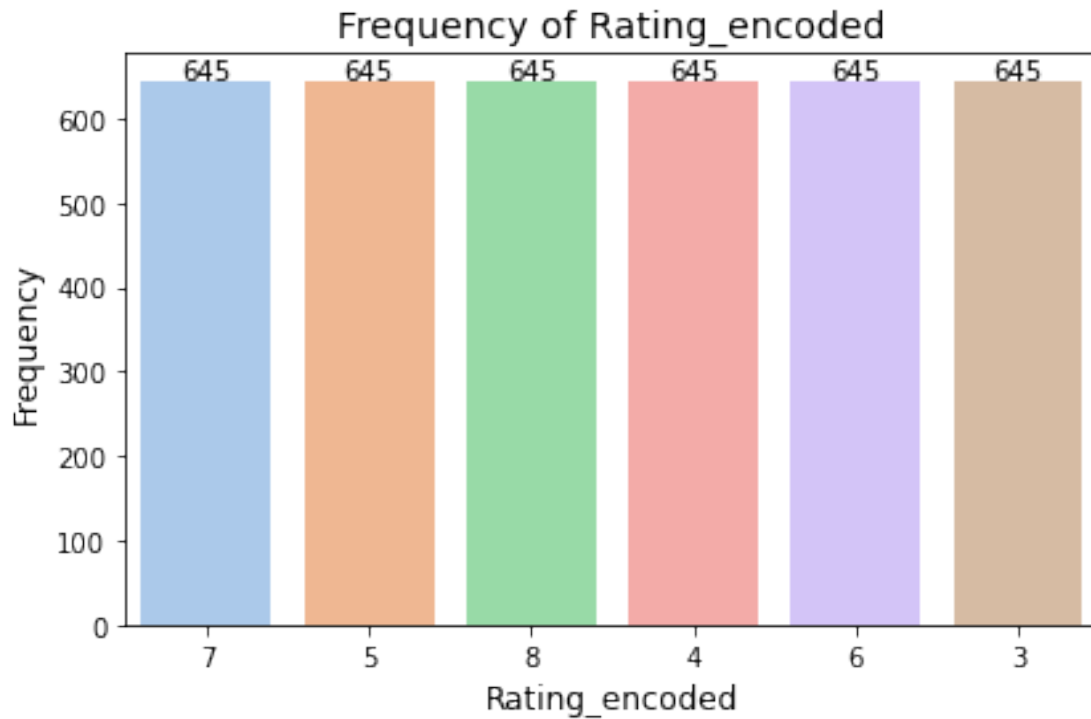
self[name] = value
/Users/kamila/opt/anaconda3/lib/python3.9/site-packages/sklearn/base.py:474:
FutureWarning: `BaseEstimator._validate_data` is deprecated in 1.6 and will be
removed in 1.7. Use `sklearn.utils.validation.validate_data` instead. This
function becomes public and is part of the scikit-learn developer API.
warnings.warn(

```

```

[193]: freq_graph(credit_rating_balanced2.copy(), 'Rating_encoded', 0)

```



```
[471]: model_params_bal2 = {
    "Gradient Boosting": {
        "model": GradientBoostingClassifier,
        "params": {"n_estimators": 100, 'learning_rate': 0.1, "max_depth": 7,
↪ "random_state": 123}
    },
    "Random Forest": {
        "model": RandomForestClassifier,
        "params": { "max_depth": None, "n_estimators": 200, 'min_samples_split': 2
↪ , "random_state": 123}
    },
    "Linear Discriminant Analysis": {
        "model": LinearDiscriminantAnalysis,
        "params": {"solver": "lsqr"}
    },
    "Logistic Regression": {
        "model": LogisticRegression,
        "params": {"random_state": 123, 'C': 10, "multi_class": "multinomial",
↪ "solver": "newton-cg"}
    },
    "KNN": {
        "model": KNeighborsClassifier,
        "params": {"n_neighbors": 1}
    }
}
```

```
}
}
```

```
[195]: class_models_macro_f1_params(data_inp=credit_rating_balanced2, X_cols=
↳best_pred,
y_col = 'Rating_encoded', test_size=0.2,
↳model_params = model_params2)
```

Gradient Boosting Macro F1 Score: 0.7494

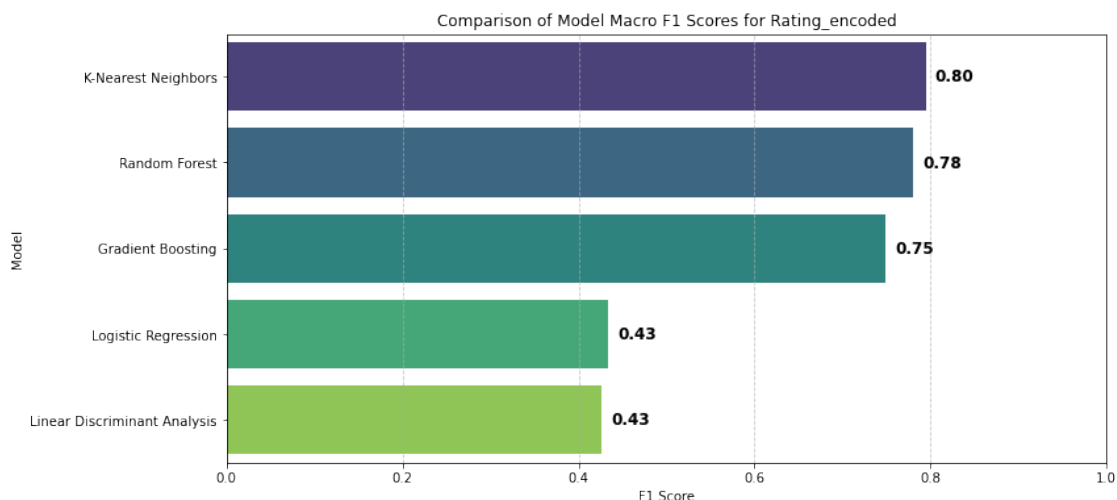
Random Forest Macro F1 Score: 0.7803

Linear Discriminant Analysis Macro F1 Score: 0.4266

Logistic Regression Macro F1 Score: 0.4341

K-Nearest Neighbors Macro F1 Score: 0.7952

/Users/kamila/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.
warnings.warn(



```
[195]:
Model  F1 Score
4      K-Nearest Neighbors  0.795185
1      Random Forest      0.780312
0      Gradient Boosting   0.749397
3      Logistic Regression  0.434149
2      Linear Discriminant Analysis  0.426589
```

```
[196]: class_models_macro_f1_params_balanced(data_inp=credit_rating_balanced2, X_cols=
↳best_pred,
```

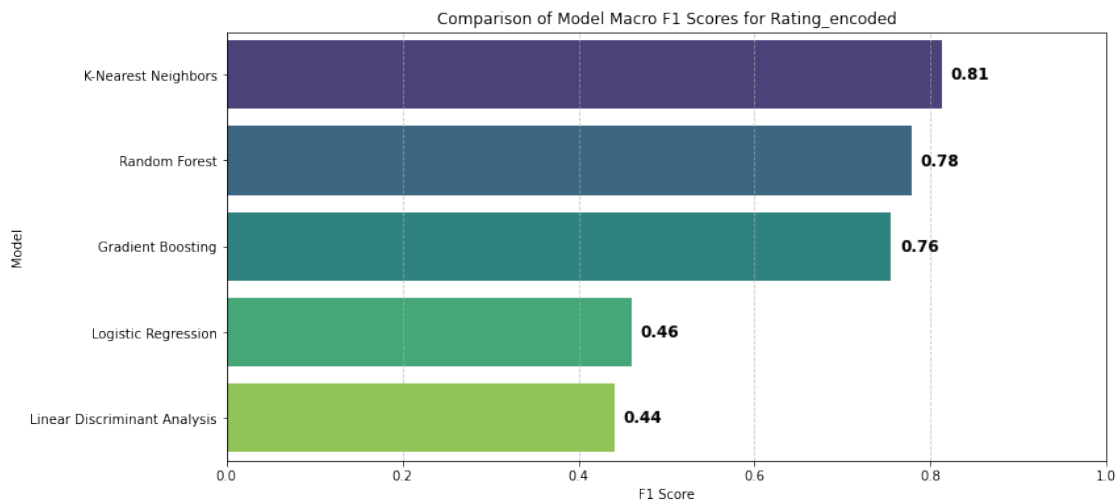
```

y_col = 'Rating_encoded', test_size=0.2,
↪model_params = model_params_bal2)

```

Gradient Boosting Macro F1 Score: 0.7552
 Random Forest Macro F1 Score: 0.7781
 Linear Discriminant Analysis Macro F1 Score: 0.4404
 Logistic Regression Macro F1 Score: 0.4599
 K-Nearest Neighbors Macro F1 Score: 0.8131

/Users/kamila/opt/anaconda3/lib/python3.9/site-
 packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class'
 was deprecated in version 1.5 and will be removed in 1.7. From then on, it will
 always use 'multinomial'. Leave it to its default value to avoid this warning.
 warnings.warn(



```

[196]:
4      K-Nearest Neighbors  0.813115
1      Random Forest       0.778143
0      Gradient Boosting    0.755229
3      Logistic Regression   0.459914
2  Linear Discriminant Analysis  0.440386

```

```

[472]: data1 = class_models_macro_f1_params(data_inp=credit_rating_outl_scaled,
↪X_cols= best_pred,
y_col = 'Rating_encoded', test_size=0.2,
↪model_params = model_params2)

data2 = class_models_macro_f1_params_balanced(data_inp=credit_rating_balanced2,
↪X_cols= best_pred,

```

```

y_col = 'Rating_encoded', test_size=0.2,
↪model_params = model_params_bal2)

compare_bar_charts(data1, data2, title = 'Comparison of Macro Avg F1 Scores',
↪y_col = 'Rating',
                    label1 = 'Before SMOTE', label2 = 'After SMOTE', xlim=1,
                    color1='navy', color2='darkmagenta')

```

Gradient Boosting Macro F1 Score: 0.4645

Random Forest Macro F1 Score: 0.4868

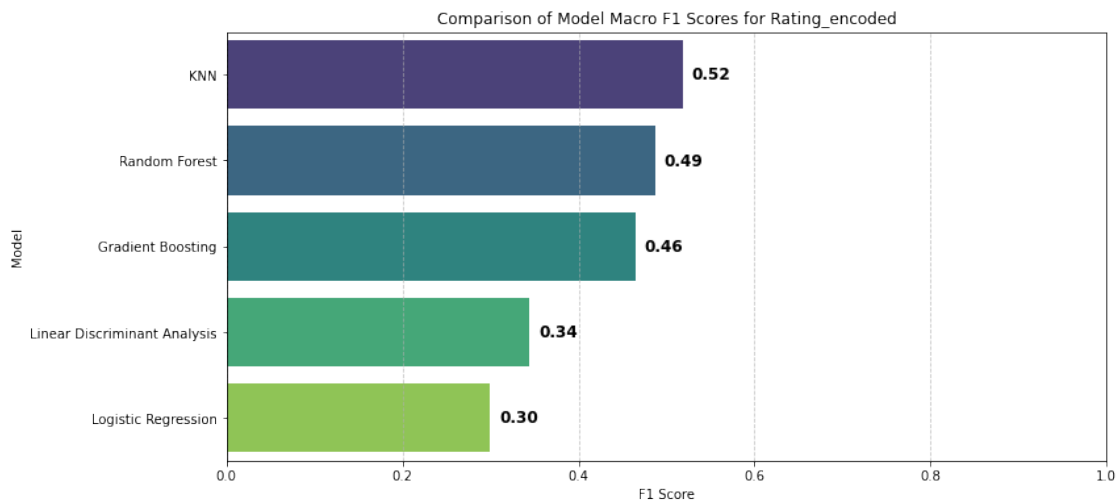
Linear Discriminant Analysis Macro F1 Score: 0.3443

Logistic Regression Macro F1 Score: 0.2998

KNN Macro F1 Score: 0.5181

/Users/kamila/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.

warnings.warn(



Gradient Boosting Macro F1 Score: 0.7552

Random Forest Macro F1 Score: 0.7781

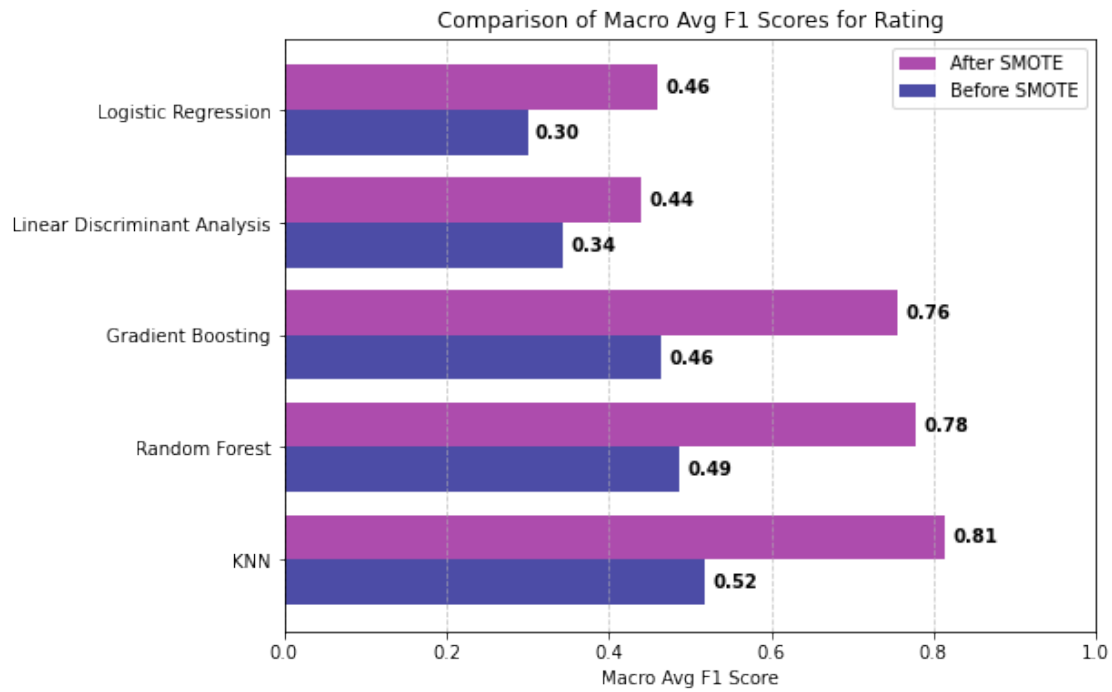
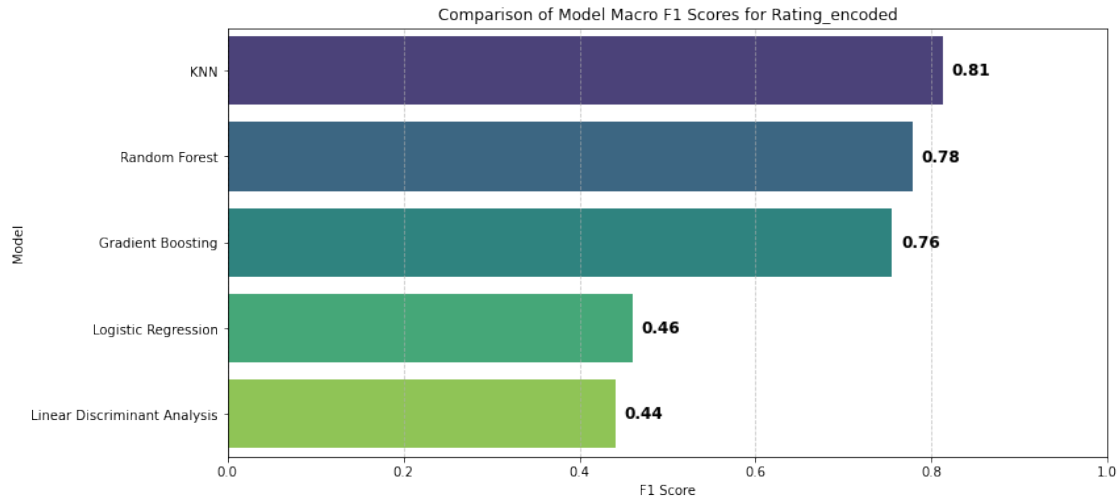
Linear Discriminant Analysis Macro F1 Score: 0.4404

Logistic Regression Macro F1 Score: 0.4599

KNN Macro F1 Score: 0.8131

/Users/kamila/opt/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.

warnings.warn(



```
[475]: compare_bar_charts(data1, data2, title = 'Comparison of Macro Avg F1 Scores',
    ↳ y_col = 'Rating',
    label1 = 'Before SMOTE', label2 = 'After SMOTE', xlim=0.875,
    color1='navy', color2='darkmagenta')
```